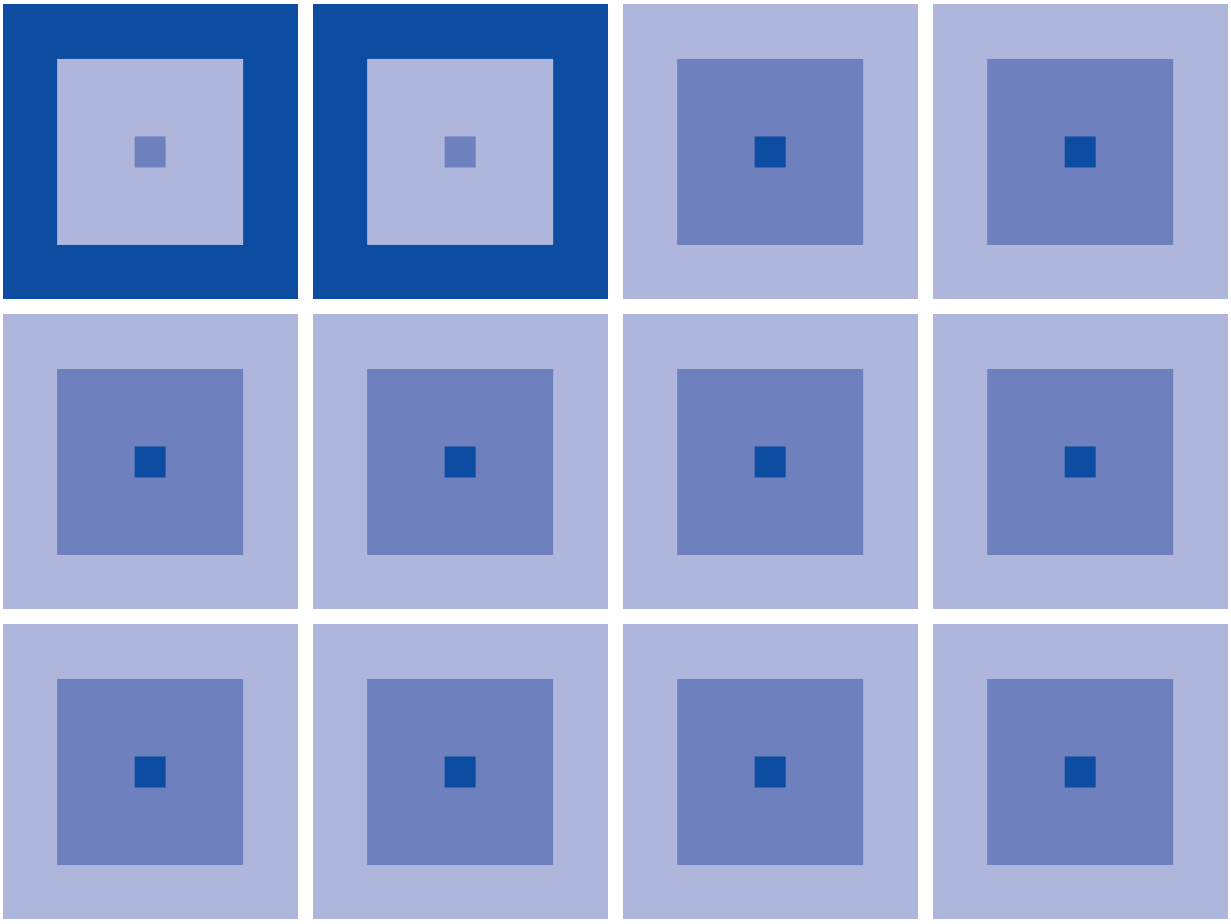
MF1098-05

EPSON

CMOS 32-BIT SINGLE CHIP MICROCOMPUTER

S5U1C33000C Manual

(S1C33 Family Cコンパイラパッケージ)
(Ver. 4)



ENERGY SAVING EPSON
夢の「省」技術。

セイコーエプソン株式会社

本資料のご使用につきましては、次の点にご留意願います。

1. 本資料の内容については、予告なく変更することがあります。
2. 本資料の一部、または全部を弊社に無断で転載、または、複製など他の目的に使用することは堅くお断りします。
3. 本資料に掲載される応用回路、プログラム、使用方法等はあくまでも参考情報であり、これらに起因する第三者の権利(工業所有権を含む)侵害あるいは損害の発生に対し、弊社は如何なる保証を行うものではありません。また、本資料によって第三者または弊社の工業所有権の実施権の許諾を行うものではありません。
4. 特性表の数値の大小は、数直線上の大小関係で表しています。
5. 本資料に掲載されている製品のうち、「外国為替および外国貿易法」に定める戦略物資に該当するものについては、輸出する場合、同法に基づく輸出許可が必要です。
6. 本資料に掲載されている製品は、一般民生用です。生命維持装置その他、きわめて高い信頼性が要求される用途を前提としていません。よって、弊社は本(当該)製品をこれらの用途に用いた場合の如何なる責任についても負いかねます。

Windows95、Windows98およびWindows NTは米国マイクロソフト社の登録商標です。
PC/ATおよびIBMは、米国International Business Machines社の登録商標です。
その他のブランド名または製品名は、それらの所有者の商標もしくは登録商標です。

© **SEIKO EPSON CORPORATION** 2001

製品型番変更のご案内

2001年4月1日より、弊社半導体製品の製品型番が以下のとおり変更となっておりますので、ご発注につきましては変更後の製品型番にてお願い申し上げます。
なお、製品型番の詳細仕様につきましては、弊社営業担当にお問い合わせください。

製品型番体系

●デバイス

S1 C 33104 F 0A01 00

- 00 — 梱包仕様
- 0A01 — 仕様
- F — 形状 (D: ベアチップ、F: QFP)
- 33104 — 機種番号
- C — 機種名称 (C: マイコン、デジタル製品)
- S1 — 製品分類 (S1: 半導体)

●開発ツール

S5U1 C 33L01 D1 1 00

- 00 — 梱包仕様
- 1 — バージョン (1: Version 1 *2)
- D1 — ツール種類 (D1: デモツール *1)
- 33L01 — 対応機種番号 (33L01: S1C33L01用)
- C — ツール分類 (C: マイコン用)
- S5U1 — 製品分類 (S5U1: 半導体用開発ツール)

*1: ツールの種類は、新旧型番対応表を参照してください。(マニュアル類には一桁で記載されているものもあります。)
*2: マニュアル類には、実際のバージョンは記載されておりません。

新旧型番対応表

●S1C33ファミリ

旧型番	新型番
E0C33A104	S1C33104
E0C33202	S1C33202
E0C33204	S1C33204
E0C33208	S1C33208
E0C33209	S1C33209
E0C332T01	S1C33T01
E0C332L01	S1C33L01
E0C332L02	S1C33L02
E0C332S08	S1C33S01
E0C332129	S1C33221
E0C33264	S1C33222
E0C332F128	S1C33240

旧型番	新型番
CC33	S5U1C33000C
CF33	S5U1C330C1S
COSIM33	S5U1C330C2S
GRAPHIC33	S5U1C330G1S
HMM33	S5U1C330H1S
JPEG33	S5U1C330J1S
MON33	S5U1C330M2S
MELODY33	S5U1C330M1S
PEN33	S5U1C330P1S
ROS33	S5U1C330R1S
SOUND33	S5U1C330S1S
SMT33	S5U1C330S2S
TS33	S5U1C330T1S
USB33	S5U1C330U1S
VOX33	S5U1C330V1S
VRE33	S5U1C330V2S

開発ツール新旧型番対応表

●S1C33ファミリ関係の開発ツール

旧型番	新型番
ICE33	S5U1C33104H
EM33-4M	S5U1C33104E
PRC33001	S5U1C33104P1
POD33001	S5U1C33104P2
ICD33	S5U1C33000H
DMT33004	S5U1C33104D1
DMT33004PD	S5U1C33104D2
DMT33005	S5U1C33208D1
DMT33005PD	S5U1C33208D2
DMT33006LV	S5U1C33L01D1
DMT33006PDLV	S5U1C33L01D2
DMT33007	S5U1C33208D3
DMT33007PD	S5U1C33208D4
DMT33008LV	S5U1C33T01D1
DMT33008PDLV	S5U1C33T01D2
DMT332S08LV	S5U1C33S01D1
DMT332S08PDLV	S5U1C33S01D2
DMT33209LV	S5U1C33209D1
DMT33209PDLV	S5U1C33209D2
DMT332F128LV	S5U1C33240D1
DMT33MON	S5U1C330M1D1
DMT33MONLV	S5U1C330M2D1
DMT33AMP	S5U1C330A1D1
DMT33AMP2	S5U1C330A2D1
DMT33AMP3	S5U1C330A3D1
DMT33AMP4	S5U1C330A4D1
DMT33CF	S5U1C330C1D1
DMT33CPLD400KLV	S5U1C330C2D1

旧型番	新型番
DMT33LIF	S5U1C330L1D1
DMT33SMT	S5U1C330S1D1
DMT33LCD26	S5U1C330L2D1
DMT33LCD37	S5U1C330L3D1
EPOD33001	S5U1C33208E1
EPOD33001LV	S5U1C33208E2
EPOD33208	S5U1C33208E3
EPOD33208LV	S5U1C33208E4
EPOD332L01LV	S5U1C33L01E1
EPOD332T01	S5U1C33T01E1
EPOD332T01LV	S5U1C33T01E2
EPOD33209	S5U1C33209E1
EPOD33209LV	S5U1C33209E2
EPOD332128	S5U1C33220E1
EPOD332128LV	S5U1C33220E2
EPOD332S08LV	S5U1C33S01E1
MEM33201	S5U1C33001M1
MEM33201LV	S5U1C33001M2
MEM33202	S5U1C33002M1
MEM33202LV	S5U1C33002M2
MEM33203	S5U1C33003M1
MEM33203LV	S5U1C33003M2
MEM33DIP42	S5U1C330D1M1
MEM33TSOP48	S5U1C330T1M1
EPOD176CABLE	S5U1C33T00E31
EPOD100CABLE	S5U1C33S00E31
EPOD33SRAM5V	S5U1C33000S
EPOD33SRAM3V	S5U1C33001S

はじめに

本書はS1C33 Familyの全機種に共通するソフトウェア開発ツール「S1C33 Family Cコンパイラパッケージ」によるCソースプログラムのコンパイルからデバッグ、最終的に提出していただくマスクデータの作成までの手順および各ツールの使用方法を説明します。

マニュアルの読み方

本書はプログラム開発に従事されている方を対象に編集されています。したがって、以下の内容が予備知識として必要です。

・C言語(ANSI C準拠)に関する知識およびCソースプログラムの作成方法
・アセンブリ言語に関する一般的な知識
・Cコンパイラおよびアセンブラを使用したプログラム開発全般の基礎知識
・Windows®95、 Windows®98またはWindows NT®4.0の基本的な操作方法

これらの内容につきましては、ANSI Cを解説した一般の書籍、Windows®のマニュアル等を参照してください。

■インストールの前に

1章をお読みください。パッケージの構成品や各ツールの概要が記載されています。

■インストール

2章に記載されたインストール手順に従って、ツールのインストールを行ってください。

■プログラム開発全体の流れと、その操作手順を理解するには

3章のチュートリアルをお読みください。コンパイルからデバッグ、最終的なマスクデータの作成までの概要を把握していただけます。

■ソースプログラム作成時

4章の必要個所をお読みください。ソースファイル作成上の注意事項や、アセンブリソースファイルの内容等が記載されています。
ソースファイルを作成する際には、以下のマニュアルも併せて参照してください。

S1C33xxxテクニカルマニュアル
　デバイスの仕様、周辺回路の動作と制御方法について
S1C33000コアCPUマニュアル
　Core CPUの機能と動作、命令セットの詳細について

■プログラムのデバッグについて

16章でデバッガの詳細を解説しています。16.1項～16.9項でデバッグ機能の概要を把握していただけます。各デバッグコマンドの詳細については16.10項を参照してください。
なお、デバッグ前に、使用するデバッグ用ツールのマニュアルも参照してください。

S1C33 Family In-Circuit Emulator Manual
　インサーキットエミュレータS5U1C33104Hの取り扱い方法について
S1C33 Family Peripheral Circuit Board Manual
　S5U1C33104H用周辺回路ボードの機能と取り扱い方法について
S1C33 Family In-Circuit Debugger Manual
　インサーキットデバッガS5U1C33000Hの取り扱い方法について
S1C33 Family Debug Monitor Operation Manual
　デバッグモニタの機能と実装方法について

■各ツールの詳細について

5～17の各章で、それぞれのツールの詳細を解説しています。必要に応じて参照してください。

■一通りの操作に慣れたら

Appendixに掲載されているインストラクションセットやコマンドの一覧を利用されると便利でしょう。

マニュアルの表記

このマニュアルは、次の表記規則に従って記述されています。

■サンプル画面

サンプルの画面はすべてWindows®95(英語版)での表示例です。システムや搭載されているフォントにより表示が異なる場合があります。

■各部の名称

ウィンドウ名、メニューおよびメニューコマンド名、ボタン名、ダイアログボックス名、キーの名称は、[ ]で囲んで記述されています。たとえば、[Command]ウィンドウ、[File]メニュー/[Exit]コマンド([File]メニュー内の[Exit]コマンド)、[Escape break]ボタン、[q]キーなどのように記述します。

■命令などの名称

CPUの命令やコマンド名など、大文字、小文字の両方が許されるものについては小文字を使用しています。ただし、ユーザーが指定するシンボル例などは除きます。

■数値の表記

数値は次のように記述しています。

10進数: 数値の前後に何も付けない。(例: 123、1000)
16進数: 数値の前に"0x"を付ける。(例: 0x0110、0xffff)
2進数: 数値の前に"0b"を付ける。(例: 0b0001、0b10)

ただし、表示例などには、16進数、2進数に何も付かない場合があります。

■マウス操作

クリック: 目的の位置にカーソル(ポインタ)を合わせ、マウスの左ボタンを一度押す操作をすべて「クリック」で表現します。右ボタンのクリック操作は「右クリック」という言葉を使います。

ダブルクリック: 目的の位置にカーソル(ポインタ)を合わせ、マウスの左ボタンを二度連続的に押す操作をすべて「ダブルクリック」で表現します。

ドラッグ: クリックにより選択したファイル(アイコン)などを、左ボタンを押したまま別の場所に移動させる操作を「ドラッグ」で表現します。

選択: メニューコマンドなどをクリックして選択する操作を、「選択」のみで表現します。

■キー操作

特定のキーを押す操作は「キーを入力」、または「キーを押す」と表現しています。
[Ctrl]+[C]キーのように+を使ったキーの組み合わせは、[Ctrl]キーを押しながら[C]キーを押す操作を表します。
キーボードからの入力例については特に[ ]で囲んでいません。また、入力例中の[Enter]キーを押す操作は↵で表しています。
なお、このマニュアルでは、マウスで可能な操作は、すべてマウス操作方法のみを記述しています。キーボードによる操作についてはWindowsのマニュアルやヘルプ等を参考にしてください。

■コマンドや起動時オプション、メッセージの一般形

[ ]で囲まれた項目はユーザー選択項目で、入力しなくても動作することを表しています。
<>で囲まれた内容は、特定の名称が用いられることを表しています。たとえば、<file name>や<ファイル名>は実際のファイル名に置き換えることを示しています。
{ }で囲まれ、:で区切られた項目はその中から1つを選択することを示します。たとえば、{A:B}はAまたはBのどちらか1つを選択することを示します。

目　次

1　概要

1.1　特長.....1
1.2　ツール構成.....1
1.2.1　パッケージ構成.....1
1.2.2　ソフトウェアツールの概要.....1

2　インストール

2.1　動作環境.....3
2.2　インストール方法.....4

3　ソフトウェア開発手順

3.1　ソフトウェア開発フロー.....6
3.2　チュートリアル(ワークベンチによる操作の流れ).....9
3.2.1　ワークベンチwb33の起動.....9
3.2.2　ディレクトリの選択とファイルの表示.....11
3.2.3　makeファイルの作成.....12
3.2.4　コンパイルからリンクまでの自動実行.....13
3.2.5　ツールを個別に実行するには.....13
3.2.6　デバッガ用パラメータファイルの作成.....14
3.2.7　デバッグ.....15
3.2.8　ディスアセンブリファイルの作成.....22
3.2.9　ROMデータの作成.....23
3.2.10　最適化.....24
3.2.11　終わりに.....25
3.3　デバッグ環境.....26
3.3.1　インサーキットエミュレータS5U1C33104H.....26
3.3.2　デバッグモニタS5U1C330M2S.....27
3.3.3　インサーキットデバッガS5U1C33000H.....30
3.3.4　メモリボードS5U1C33001M1.....33
3.4　プログラムの構造と各種メモリとの対応.....35

4　ソースファイル

4.1　ファイル形式とファイル名.....40
4.2　Cソースの文法.....41
4.2.1　データ型.....41
4.2.2　ライブラリ関数とヘッダファイル.....41
4.2.3　インラインアセンブル.....42
4.3　アセンブリソースの文法.....43
4.3.1　ステートメント.....43
4.3.2　オペランドの表記.....47
4.3.3　拡張命令.....50
4.3.4　プリプロセッサによる付加機能.....51
4.4　ソース作成上の注意事項.....52

5　ワークベンチ

5.1　機能.....53
5.2　操作方法.....54
5.2.1　起動および終了方法.....54
5.2.2　ウィンドウ.....55
5.2.3　ファイルの選択とソース表示.....57
5.2.4　個別ツールの実行.....58

5.2.5 実行条件の選択 63
5.2.6 make 64
5.2.7 パラメータファイルジェネレータ 72
5.2.8 汎用エディタの指定 75
5.2.9 コマンドライン入力 76
5.2.10 オプションの保存と復帰 76
5.3 エラーメッセージ 77
5.4 注意事項 77

6 Cコンパイラ

6.1 機能 78
6.2 入出力ファイル 78
6.2.1 入力ファイル 78
6.2.2 出力ファイル 78
6.3 起動方法 79
6.3.1 起動フォーマット 79
6.3.2 起動オプション 79
6.4 メッセージ 81
6.5 コンパイラの出力 82
6.5.1 出力内容 82
6.5.2 データ表現 83
6.5.3 レジスタ使用法 84
6.5.4 関数呼び出し 85
6.5.5 スタックフレーム 86
6.6 デバッグ情報 87
6.6.1 ソース情報 87
6.6.2 シンボル情報 87
6.7 gccの機能と注意事項について 92

7 エミュレーションライブラリ

7.1 概要 93
7.2 浮動小数点演算ライブラリ(fp.lib) 93
7.2.1 関数一覧 93
7.2.1 浮動小数点フォーマット 94
7.3 整数剰余演算ライブラリ(idiv.lib) 95
7.4 浮動小数点ライブラリ(fpp.lib) 95

8 ANSIライブラリ

8.1 概要 96
8.2 ANSIライブラリ関数一覧 97
8.2.1 入出力関数(io.lib) 97
8.2.2 ユーティリティ関数(lib.lib) 99
8.2.3 日付と時刻関数(lib.lib) 100
8.2.4 数学関数(math.lib) 101
8.2.5 文字関数(string.lib) 102
8.2.6 文字種判定/変換関数(ctype.lib) 103
8.2.7 可変引数マクロ(stdarg.h) 103
8.3 グローバル変数の宣言と初期化 104
8.4 下位レベル関数 105
8.4.1 read関数 105
8.4.2 write関数 106
8.4.3 _exit関数 106

9 プリプロセッサ

9.1 機能.....107
9.2 入出力ファイル.....107
9.2.1 入力ファイル.....107
9.2.2 出力ファイル.....107
9.3 起動方法.....108
9.3.1 起動フォーマット.....108
9.3.2 起動オプション.....108
9.4 メッセージ.....109
9.5 プリプロセッサ擬似命令.....110
9.5.1 インクルード命令(#include).....110
9.5.2 デファイン命令(#define).....111
9.5.3 マクロ命令(#macro～#endm).....113
9.5.4 条件アセンブル命令(#ifdef～#else～#endif, #ifndef～#else～#endif).....115
9.6 数値演算子.....116
9.7 デバッグ情報.....118
9.8 コメント付加機能.....119
9.9 その他の機能.....119
9.9.1 ASCII文字の16進数への変換.....119
9.9.2 コメント行.....119
9.10 処理の流れ.....120
9.11 実行例.....120
9.12 エラー/ワーニングメッセージ.....122
9.12.1 エラー.....122
9.12.2 ワーニング.....123
9.13 注意事項.....123

10 命令エクステンダ

10.1 機能.....124
10.2 入出力ファイル.....124
10.2.1 入力ファイル.....124
10.2.2 出力ファイル.....125
10.3 起動方法.....125
10.3.1 起動フォーマット.....125
10.3.2 起動オプション.....125
10.4 コマンドファイル.....127
10.5 メッセージ.....128
10.6 拡張命令.....129
10.6.1 算術演算命令.....129
10.6.2 比較命令.....130
10.6.3 論理演算命令.....131
10.6.4 シフト&ローテート命令.....132
10.6.5 スタック～レジスタ間データ転送命令.....133
10.6.6 メモリ～レジスタ間データ転送命令.....134
10.6.7 即値ロード命令.....139
10.6.8 ビット操作命令.....140
10.6.9 分岐命令.....143
10.7 最適化機能.....145
10.7.1 相対分岐命令の最適化.....145
10.7.2 グローバルポインタによる最適化.....146
10.7.3 シンボル情報による最適化.....146

10.8　その他の機能 147
10.8.1　コメント付加機能 147
10.8.2　ローカルシンボルの分類 147
10.8.3　文法チェック 147
10.9　実行例 148
10.10　エラー/ワーニングメッセージ 155
10.10.1　エラー 155
10.10.2　ワーニング 156
10.11　注意事項 156

11　アセンブラ

11.1　機能 157
11.2　入出力ファイル 157
11.2.1　入力ファイル 157
11.2.2　出力ファイル 157
11.3　起動方法 158
11.3.1　起動フォーマット 158
11.3.2　起動時オプション 158
11.4　メッセージ 159
11.5　リロケータブルアセンブルとアブソリュートアセンブル 160
11.5.1　リロケータブルアセンブル 160
11.5.2　アブソリュートアセンブル 160
11.6　スコープ 161
11.7　セクション定義 162
11.8　アセンブラ擬似命令 165
11.8.1　アブソリュートアセンブル擬似命令(.abs) 165
11.8.2　セクション定義擬似命令(.code, .data) 166
11.8.3　領域確保擬似命令(.comm, .lcomm) 167
11.8.4　ロケーションカウンタ制御擬似命令(.org) 169
11.8.5　シンボル定義擬似命令(.set) 170
11.8.6　データ定義擬似命令(.word, .half, .byte, .ascii, .space) 171
11.8.7　アライメント擬似命令(.align) 174
11.8.8　グローバル宣言擬似命令(.global) 174
11.8.9　リスト制御擬似命令(.list, .nolist) 175
11.8.10　デバッグ擬似命令(.file, .endfile, .loc, .def) 176
11.9　アセンブリリストファイル 177
11.10　エラー/ワーニングメッセージ 178
11.10.1　エラー 178
11.10.2　ワーニング 179
11.11　注意事項 179

12　リンカ

12.1　機能 180
12.2　入出力ファイル 180
12.2.1　入力ファイル 181
12.2.2　出力ファイル 181
12.3　起動方法 182
12.3.1　起動フォーマット 182
12.3.2　起動時オプション 183
12.4　メッセージ 184
12.5　lk33 Ver. 2.1以前のリンク処理 185
12.5.1　リンカコマンドファイル 185
12.5.2　リンカコマンド一覧(Ver. 2.1) 187
12.5.3　セクションの配置 193

12.5.4　仮想・共用(U)セクション.....196
12.5.5　セクションシンボル.....199
12.5.6　ライブラリのリンク.....201
12.6　lk33 Ver. 3.0のリンク処理.....203
12.6.1　リンカコマンドファイル.....203
12.6.2　リンカコマンド一覧(Ver. 3.0).....205
12.6.3　ブロックの配置.....215
12.6.4　仮想ブロックと共用ブロック.....218
12.6.5　Ver. 3.0モードで生成されるシンボル.....222
12.6.6　ライブラリのリンク.....224
12.7　シンボルの解明.....225
12.8　リンクマップファイル.....226
12.9　シンボルファイル.....227
12.10　クロスリファレンスファイル.....228
12.11　エラー/ワーニングメッセージ.....229
12.11.1　エラー.....229
12.11.2　ワーニング.....231
12.12　注意事項.....232

13　ディスアセンブラ

13.1　機能.....233
13.2　入出力ファイル.....233
13.2.1　入力ファイル.....233
13.2.2　出力ファイル.....233
13.3　起動方法.....234
13.3.1　起動フォーマット.....234
13.3.2　起動オプション.....234
13.4　メッセージ.....235
13.5　ディスアセンブル出力.....236
13.5.1　ミックス出力.....236
13.5.2　コード出力.....238
13.5.3　データ出力.....239
13.6　エラー/ワーニングメッセージ.....240
13.6.1　エラー.....240
13.6.2　ワーニング.....240
13.7　注意事項.....241

14　バイナリ/HEXコンバータ

14.1　機能.....242
14.2　入出力ファイル.....242
14.2.1　入力ファイル.....242
14.2.2　出力ファイル.....242
14.3　起動方法.....243
14.3.1　起動フォーマット.....243
14.3.2　起動オプション.....243
14.4　メッセージ.....244
14.5　HEXファイルの内容.....245
14.5.1　モトローラS3形式.....245
14.5.2　絶対アドレス出力.....245
14.5.3　オフセットアドレス出力.....245
14.6　エラー/ワーニングメッセージ.....246
14.6.1　エラー.....246
14.6.2　ワーニング.....246
14.7　注意事項.....247

15 ライブラリアン

15.1 機能 248
15.2 入出力ファイル 248
15.2.1 入力ファイル 248
15.2.2 出力ファイル 248
15.3 起動方法 249
15.3.1 起動フォーマット 249
15.3.2 起動オプション 249
15.4 メッセージ 250
15.5 ライブラリの編集機能 251
15.5.1 新規作成 251
15.5.2 モジュールの追加 251
15.5.3 登録モジュールの一覧表示 252
15.5.4 モジュールの削除 252
15.5.5 オブジェクトファイルの復元 252
15.6 エラー/ワーニングメッセージ 253
15.6.1 エラー 253
15.6.2 ワーニング 253
15.7 注意事項 253

16 デバッガ

16.1 特長 254
16.2 入出力ファイル 254
16.2.1 入力ファイル 254
16.2.2 出力ファイル 255
16.3 起動方法 256
16.3.1 起動フォーマット 256
16.3.2 起動時オプション 256
16.3.3 起動メッセージ 258
16.3.4 終了方法 261
16.4 ウィンドウ 262
16.4.1 ウィンドウの基本構成 262
16.4.2 [Command]ウィンドウ 264
16.4.3 [Source]ウィンドウ 265
16.4.4 [Memory]ウィンドウ 268
16.4.5 [Register]ウィンドウ 270
16.4.6 [Trace]ウィンドウ 271
16.4.7 [Symbol]ウィンドウ 273
16.4.8 [Simulated I/O]ウィンドウ 274
16.5 ツールバー 275
16.5.1 ツールバーの構成 275
16.5.2 [Key break]ボタン 275
16.5.3 [Load file]ボタン 275
16.5.4 [Source], [Mix], [Unassemble]ボタン 275
16.5.5 [Reset cold], [Reset hot]ボタン 275
16.5.6 [Go], [Go to], [Step], [Next]ボタン 275
16.5.7 [Soft PC break], [Hard PC break]ボタン 276
16.5.8 [Symbol watch], [Symbol add], [Symbol delete]ボタン 276
16.5.9 [Display trace], [Resume trace]ボタン 277
16.5.10 [Select source]コンボボックス 277
16.6 メニュー 278
16.6.1 メニューの構成 278
16.6.2 [File]メニュー 278

16.6.3 [Edit]メニュー....278
16.6.4 [Run]メニュー....278
16.6.5 [Break]メニュー....279
16.6.6 [Symbol]メニュー....279
16.6.7 [Window]メニュー....280
16.6.8 [Help]メニュー....280
16.7 ファンクションキー....281
16.8 コマンド実行方法....282
16.8.1 コマンドのキーボード入力....282
16.8.2 パラメータの入力形式....283
16.8.3 メニュー/ツールバー/ファンクションキーによる実行....286
16.8.4 コマンドファイルによる実行....287
16.8.5 ログファイル....288
16.9 デバッグ機能....289
16.9.1 デバッグモード....289
16.9.2 ファイルの読み込み....294
16.9.3 ソース表示およびシンボリックデバッグ機能....295
16.9.4 メモリデータ、レジスタの表示と変更....298
16.9.5 プログラムの実行....301
16.9.6 ブレーク機能....305
16.9.7 トレース機能....314
16.9.8 シミュレーテッドI/O....323
16.9.9 フラッシュメモリの操作....325
16.9.10 その他の機能....328
16.9.11 ビッグエンディアンへの対応について....329
16.10 コマンドリファレンス....330
16.10.1 コマンド一覧....330
16.10.2 メモリ操作コマンド....331
fb (fill byte) [ICD / ICE / SIM / MON]....331
fh (fill half) [ICD / ICE / SIM / MON]....332
fw (fill word) [ICD / ICE / SIM / MON]....333
db (dump byte) [ICD / ICE / SIM / MON]....334
dh (dump half) [ICD / ICE / SIM / MON]....336
dw (dump word) [ICD / ICE / SIM / MON]....338
df (dump file) [ICD / ICE / SIM / MON]....340
eb (enter byte) [ICD / ICE / SIM / MON]....341
eh (enter half) [ICD / ICE / SIM / MON]....342
ew (enter word) [ICD / ICE / SIM / MON]....343
mv (move) [ICD / ICE / SIM / MON]....344
mvh (move half) [ICD / (ICE) / SIM / MON]....345
mvw (move word) [ICD / (ICE) / SIM / MON]....346
w (watch) [ICD / ICE / SIM / MON]....347
rm (read memory) [ICD]....348
mc (memory compare) [ICD / ICE / SIM / MON]....349
16.10.3 レジスタ操作コマンド....350
rd (register display) [ICD / ICE / SIM / MON]....350
rs (register set) [ICD / ICE / SIM / MON]....351
16.10.4 プログラム実行コマンド....352
g (go) [ICD / ICE / SIM / MON]....352
s (step) [ICD / ICE / SIM / MON]....354
n (next) [ICD / ICE / SIM / MON]....356
o (step out) [ICD / ICE / SIM / MON]....357
call, callh (call) [ICD / ICE / SIM / MON]....358
callmd (call mode) [ICD / ICE / SIM / MON]....360

16.10.5 CPUリセットコマンド 361
rstc (cold reset CPU) [ICD / ICE / SIM / MON] 361
rsth (hot reset CPU) [ICD / ICE / SIM / MON] 362
16.10.6 割り込みコマンド 363
int (interrupt) [SIM] 363
16.10.7 ブレーク設定コマンド 364
bp (break point set) [ICD / ICE / SIM / MON] 364
bs (break software) [ICD / ICE / SIM / MON] 368
bc (break clear) [ICD / ICE / SIM / MON] 369
bh (break hardware) [ICD / ICE / SIM / MON] 370
bhc (break hardware clear) [ICD / ICE / SIM / MON] 372
bh2 (break hardware2) [ICD / (ICE) / SIM / MON] 373
bhc2 (break hardware2 clear) [ICD / (ICE) / SIM / MON] 375
bph (break point set hardware) [ICD / ICE / SIM / MON] 376
bd (data break) [ICD / ICE / SIM / MON] 380
bsq (break sequential) [ICE] 382
ba (break area) [ICD+MEM33] 385
bb (break bus) [ICD+MEM33] 387
bl (break list) [ICD / ICE / SIM / MON] 389
bac (break all clear) [ICD / ICE / SIM / MON] 390
16.10.8 ソース表示コマンド 391
u (unassemble) [ICD / ICE / SIM / MON] 391
sc (source code) [ICD / ICE / SIM / MON] 393
m (mix) [ICD / ICE / SIM / MON] 395
ss (search strings) [ICD / ICE / SIM / MON] 397
16.10.9 シンボル情報表示コマンド 398
sy (symbol list) [ICD / ICE / SIM / MON] 398
sa (symbol add) [ICD / ICE / SIM / MON] 403
sd (symbol delete) [ICD / ICE / SIM / MON] 406
sw (symbol watch) [ICD / ICE / SIM / MON] 407
16.10.10 ファイル読み込みコマンド 410
lf (load file) [ICD / ICE / SIM / MON] 410
lh (load hex) [ICD / ICE / SIM / MON] 412
ld (load debug) [ICD / ICE / SIM / MON] 413
16.10.11 フラッシュメモリ操作コマンド 414
fls (flash memory set) [ICD / (ICE) / MON] 414
fle (flash memory erase) [ICD / (ICE) / MON] 415
lfl (load from flash memory) [ICE] 416
sfl (save to flash memory) [ICE] 417
efl (erase flash memory) [ICE] 418
maf (map flash memory) [ICE] 419
16.10.12 トレースコマンド 420
tm (trace mode) [ICD / ICE / SIM] 420
tm2 (trace mode 2) [SIM] 426
td (trace dump) [ICD / ICE] 427
ts (trace search) [ICD / ICE] 431
tf (trace file) [ICD / ICE] 432
16.10.13 シミュレーテッドI/O 433
stdin (standard input) [ICD / ICE / SIM / MON] 433
stdout (standard output) [ICD / ICE / SIM / MON] 434

16.10.14 その他のコマンド 435
com (execute command file) [ICD / ICE / SIM / MON] 435
cmw (execute command file with wait) [ICD / ICE / SIM / MON] 436
log (logging) [ICD / ICE / SIM / MON] 437
od (option data dump) [ICE].....438
ct (change type) [ICD / ICE / SIM / MON] 439
ext (extended instruction) [ICD / ICE / SIM / MON] 441
ma (map information) [ICD / ICE / SIM / MON] 443
md (mode) [ICD / ICE / SIM / MON] 444
alias (alias) [ICD / ICE / SIM / MON] 446
exit (exit) [ICD / ICE / SIM / MON] 448
? (help) [ICD / ICE / SIM / MON] 449
ice (ice) [ICE].....450
16.11 パラメータファイルについて 451
16.12 INIファイル 457
16.13 ステータス/エラー/ワーニングメッセージ 459
16.13.1 ステータスメッセージ 459
16.13.2 エラーメッセージ 459
16.13.3 ワーニングメッセージ 462

17 その他のツール

17.1 make.....463
17.1.1 起動方法.....463
17.1.2 メッセージ.....464
17.1.3 makeファイル 465
17.1.4 2パスmake 470
17.1.5 clean 470
17.1.6 エラー/ワーニングメッセージ 471
17.1.7 注意事項.....471
17.2 cwait.....472
17.2.1 機能.....472
17.2.2 使用方法.....472
17.3 ccap 473
17.3.1 機能.....473
17.3.2 使用方法.....473

Appendix srf33ファイルの構造

A-1 srf33オブジェクトファイルの構造.....475
A-2 ライブラリファイルの構造.....480

クイックリファレンス

1 概要

1.1 特長

S1C33 Family Cコンパイラパッケージには、Cソースプログラムのコンパイル、アセンブリソースプログラムのアセンブルからデバッグ、さらにROM書き込みデータや提出用マスクデータの作成までを行う一連のソフトウェアツールやユーティリティが含まれています。すべてのツール/ユーティリティはS1C33 Familyの全機種で共通に使用することができます。
主な特長を以下に示します。

・強力なオプティマイズ機能

CコンパイラはS1C33アーキテクチャに対して最適化されており、非常にコンパクトなコードを生成します。また、高いオプティマイズ能力にもかかわらず、デバッグ情報が失われることが少なく、Cソースレベルのデバッグを可能とします。
さらに、リンク後の情報をもとに、よりコンパクトなコードを生成可能なツール構成となっています。

・使いやすい拡張命令セットによりS1C33000命令セットをサポート

S1C33000命令セットの即値拡張機能(ext命令)や複数の命令を1命令で記述可能な拡張命令を提供します。これにより、データサイズを特に意識せずにアセンブリソースを作成することができます。

・シミュレータ機能を持つ、Cソースおよびアセンブリソースレベルデバッガ

デバッガはCソースレベルデバッグおよびアセンブリソースレベルデバッグに対応しています。S5U1C33000H等を使用することにより、ターゲットボードを動作させた状態でデバッグが行えます。また、パソコンのみでもデバッグが行えるシミュレータモードを持っています。

・ワークベンチによる統合化作業環境

WindowsのGUIに対応したワークベンチによって、一連のツールが一つのウィンドウ上で実行できます。基本的な操作はすべてマウスのみで行え効率よい作業環境を提供します。

1.2 ツール構成

1.2.1 パッケージ構成

S1C33 Family Cコンパイラパッケージの内容を以下に示します。開梱時にすべて揃っていることを確認してください。

(1)ツールディスク(CD-ROM)	1枚
(2)S5U1C33000C Manual (S1C33 Family Cコンパイラパッケージ) (本マニュアル)	和英 各1冊
(3)保証書	和英 各1部

1.2.2 ソフトウェアツールの概要

本パッケージに含まれる主要なソフトウェアツールの概要を以下に示します。

(1) Cコンパイラ(gcc33.exe)

Free Software Foundation, Inc.で開発されたGNU Cコンパイラがベースとなっています。
ANSI Cに準拠したCコンパイラで、CソースファイルをS1C33 Family用のアセンブリソースファイルにコンパイルします。強力なオプティマイズ能力を持ち、非常にコンパクトなコードを生成します。Cコンパイラはgcc33.exe、cpp.exe、cc1.exeの3つのファイルで構成されています。

(2) プリプロセッサ(pp33.exe)

プリプロセッサpp33はアセンブリ言語によるプログラム開発時にソースファイルを最初に処理するツールです。複数のステートメント群を一つの命令のように使用することができるマクロ文や、他のファイルを挿入するインクルード文など、アセンブリソース作成のための付加機能を処理し、命令エクステンダext33に入力するアセンブリソースファイルを生成します。

(3) **命令エクステンダ**(ext33.exe)

命令エクステンダext33はS1C33000命令セットが持つ即値拡張命令(ext)を最適化するツールです。アセンブリソース作成時にext命令を意識せずにプログラムを記述可能な拡張命令を、このツールが提供します。また、リンク後のシンボルマップ情報を元にコードをより最適化する2パスmakeにも対応しています。

(4) **アセンブラ**(as33.exe)

アセンブラas33は命令エクステンダが出力するアセンブリソースファイルをアセンブルし、ソースファイルのニーモニックをS1C33000のオブジェクト(機械語)コードに変換します。結果はリンクおよびライブラリ化が可能なオブジェクトファイルとして出力されます。

(5) **リンカ**(lk33.exe)

アセンブラas33によって生成したオブジェクトコードのメモリロケーションを決定し、実行可能なオブジェクトコードを生成します。複数のオブジェクトファイルやライブラリも、このツールによって1つにまとめられます。

(6) **ディスアセンブラ**(dis33.exe)

リンカが出力するsrf33形式のオブジェクトファイルをディスアセンブルし、ニーモニックコードとソースが参照できるファイルを生成します。リンク後にソースファイルと絶対アドレスの対応を見たい場合などに有効です。

(7) **バイナリ**/HEX**コンバータ**(hex33.exe)

リンカが出力するsrf33形式のオブジェクトファイルを、ROM書き込み用のモトローラS3フォーマットのHEXファイルに変換します。このツールにより生成された外部ROM用HEXデータはROMライタによってROMに書き込むことができます。内蔵ROM用HEXデータはマスク化のためのデータとなります。

(8) **デバッガ**(db33.exe)

ハードウェアツール(S5U1C33104H、S5U1C33000H)やデバッグモニタS5U1C330M2Sを制御してデバッグを行うソフトウェアです。特別なツールを使用せずにパソコン上でデバッグを行うシミュレーション機能も持っています。ブレークやステップ実行など、頻繁に使用するコマンドはツールバーに登録されており、キーボード操作の量を抑えています。また、Cソースレベルおよびアセンブリソースレベルデバッグに対応しており、各種のデータもマルチウィンドウとして表示できるため、デバッグ作業が効率良く行えます。

(9) **ライブラリアン**(lib33.exe)

ライブラリの編集ツールです。アセンブラが生成したオブジェクトファイルをライブラリに登録することができます。ライブラリからオブジェクトを削除したり、オブジェクトファイルに復元することもできます。

(10) Make(make.exe)

コンパイルからアセンブルまでを自動実行します。実行内容を記述するmakeファイルはワークベンチwb33によって生成可能です。

(11) **ワークベンチ**(wb33.exe)

上記のツールを1つのウィンドウ上から起動できるようにしたソフトウェアです。ファイルや主な起動時オプションの選択、各ツールの起動はマウス操作のみで行え、効率の良い作業環境を提供します。

本パッケージには、サンプルプログラムおよびいくつかのユーティリティプログラムが含まれています。それらの内容につきましては、ディスク上の"readmeja.txt"(日本語)または"readme.txt"(英語)をご覧ください。

2 インストール

この章ではS1C33 Family Cコンパイラパッケージに収められたツールの動作環境とインストール方法について説明します。

2.1 動作環境

S1C33 Family Cコンパイラパッケージを使用するには、次の動作環境が必要です。

●パーソナルコンピュータ

IBM PC/ATまたは完全互換機が必要です。Pentium 90MHz以上のCPUと32MB以上のRAMを搭載した機種を推奨します。
別売のインサーキットエミュレータS5U1C33104Hやインサーキットデバッガ S5U1C33000Hを使用するには、シリアルポート(D-sub 9pin)とパラレルポート(D-sub 25pin)が必要です。デバッグモニタS5U1C330M2SとS5U1C330M1D1を使用する場合はシリアルポート(D-sub 9pin)のみ必要となります。

●ディスプレイ

800×600ドット以上のディスプレイが必要です。

●ハードディスク

S1C33 Family Cコンパイラパッケージをインストールするには、ハードディスクに約10MB以上の空き領域が必要です。

●CD-ROMドライブ

CD-ROMからインストールするため、CD-ROMドライブが1台必要です。

●マウス

ツールの操作にマウスが必要です。

●デバッグツール

ターゲットシステムも含めデバッグを行うには、本パッケージとは別売のインサーキットエミュレータ(S5U1C33104H)、インサーキットデバッガ(S5U1C33000H)、あるいはデバッグモニタ(S5U1C330M2SとS5U1C330M1D1)等が必要です。

●システムソフトウェアについて

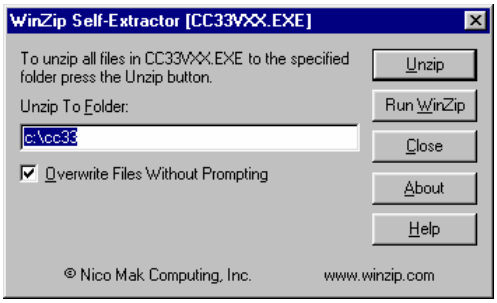
S1C33 Family CコンパイラパッケージはMicrosoft® Windows®95、Windows NT®4.0またはそれ以降のバージョン(日本語版と英語版)に対応しています。

●その他

ディスク上の"readmeVxx.txt"(英語/日本語、xxはバージョン番号を表します。)に注意点や制限事項が載っていますので、必ずお読みください。

2.2 インストール方法

S1C33 Family Cコンパイラパッケージの全ツールはCD-ROM 1枚で提供されます。CD-ROM上の自己解凍形式のファイル"cc33vXX.exe"を実行し、インストールを行ってください(ファイル名のXXはバージョン番号を表します。たとえばver. 2.0の場合は"cc33v20.exe")。
"cc33vXX.exe"をダブルクリックして起動すると次のダイアログボックスが表示されます。



テキストボックスにインストール先のパス/フォルダ名を入力し、[Unzip]ボタンをクリックしてください。指定のフォルダが作成され、すべてのファイルがその中にコピーされます。
指定のパスに指定のフォルダが存在している場合、[Overwrite Files Without Prompting]がチェックされていると確認なしに上書きされます。

コピー後のディレクトリとファイルの構成を以下に示します。

ルートDIR- (C:¥CC33¥)	readmeVxx.txt	ツールの内容説明(英語と日本語) xxはバージョン番号を表しています。
	GNU_COPYRIGHT	GNUコピーライト
	wb33.exe, ccap.exe	ワークベンチと付随ツール
	make.exe, cwait.exe	makeと付随ツール
	gcc33.exe, cpp.exe, cc1.exe	Cコンパイラ
	pp33.exe	プリプロセッサ
	ext33.exe	命令エクステンダ
	as33.exe	アセンブラ
	lk33.exe	リンカ
	lib33.exe	ライブラリアン
	db33.exe	デバッガ
	dis33.exe	ディスアセンブラ
	hex33.exe	バイナリ/HEXコンバータ
	vb40032.dll, olepro32.dll, msvcrt40.dll	ワークベンチ用dll
	lib¥ -	io.lib , lib.lib, math.lib, ctype.lib, string.lib, idiv.lib, fp.lib
	include¥ -	stdio.h, stdlib.h, time.h, math.h, errno.h, float.h, limits.h, ctype.h, string.h, stdarg.h
	sample¥	
	utility¥	

sampleおよびutilityディレクトリの内容については、"readmeja.txt"(日本語)、"readme.txt"(英語)あるいは、"*_man.txt"(英語)を参照してください。

●OS設定上の注意

・画面表示は、コントロールパネルの"画面"で"小さいフォント"の設定にしてください。

・ネットワーク上のドライブをツールまたは作業用として使用する場合は、必ずドライブ名を割り当ててください。ネットワーク名を使用することはできません。

・デバッグツール(S5U1C33104H、S5U1C33000H、S5U1C330M2S)との通信に使用するCOMxポートおよびLPTxポートは、他のドライバやアプリケーションで使用しないでください。また、ノートPCではCOMポートとLPTポートがディセーブル状態になっている場合がありますので、デバッグツールを使用する前に使用できる状態になっていることを確認してください。

・デバッガdb33などにおいて、正常な表示ができないなど、GUI上の問題がおきる場合はグラフィックカードのアクセラレート機能のレベルを下げるか、Windowsに標準添付されている機能の低いドライバを使用してください。

●ツールを削除するには

ファイルはすべて指定ディレクトリ(デフォルトは"C:¥CC33¥")にコピーされます。したがって、すべてのツールを削除するには、そのディレクトリ(フォルダ)をごみ箱に入れて消去してください。

●GNUコピーライトについて

本パッケージのCコンパイラgcc33はFree Software Foundation, Inc.のGNU Cコンパイラがベースとなっています。ライセンスに関してはテキストファイル"GNU_COPYRIGHT"に記述されていますので、ご使用前にご覧ください。

3 ソフトウェア開発手順

この章ではワークベンチwb33の基本操作方法をとおして、プログラムのコンパイルからデバッグ、マスクデータの生成までの流れを説明します。この章で使用するサンプルプログラムがsample¥tst¥およびsample¥dmt33005¥ディレクトリに収められていますので、マニュアルを読みながら操作していただけます。

3.1 ソフトウェア開発フロー

ソフトウェア開発の流れを図3.1.1に示します。

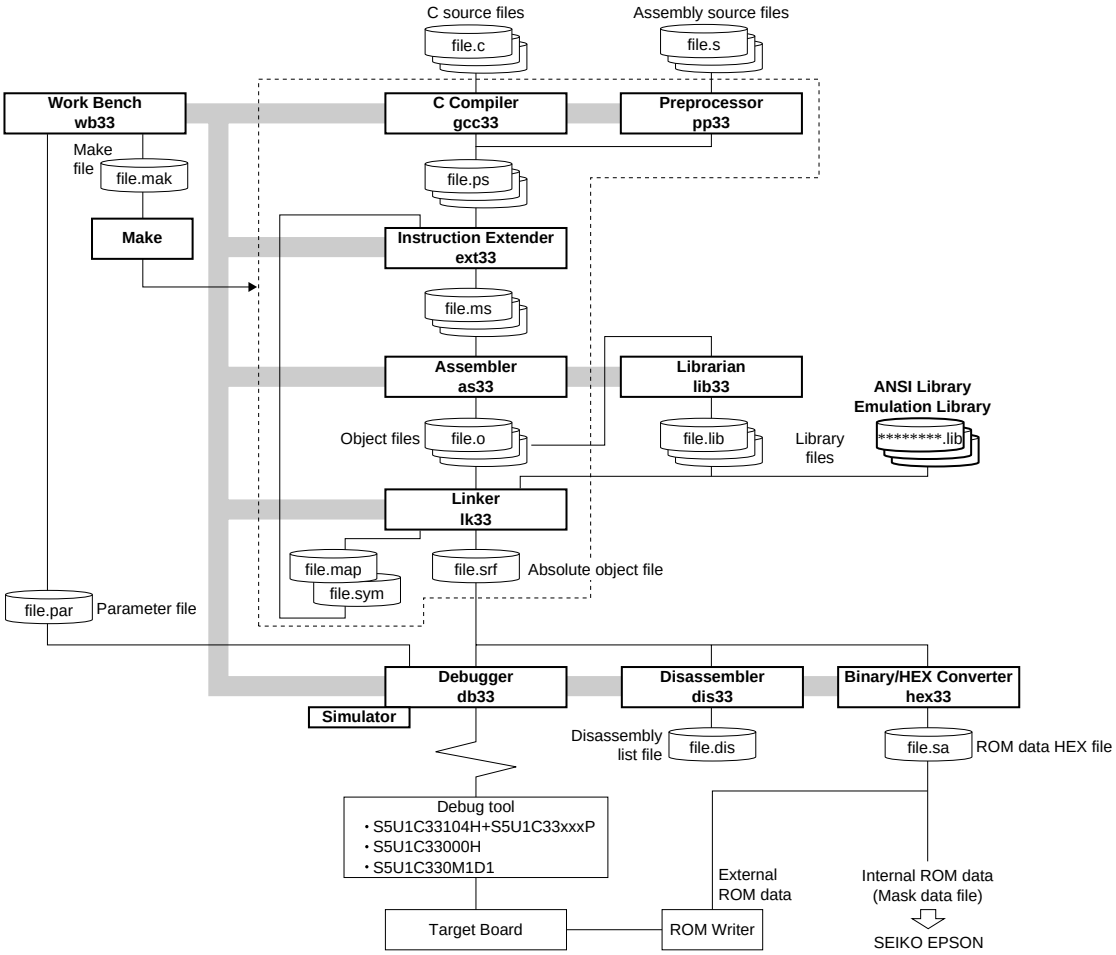


図3.1.1 **ソフトウェア開発フロー**

図に示したとおり、本パッケージのツールはソースプログラム作成後のすべてのソフトウェア処理を受け持っています。

開発の流れは次のようになります。

(1) ソースプログラムの作成

ソースファイルを汎用のエディタ等を使用して作成します。プログラムは複数のモジュール(ソースファイル)に分けて作成することができます。

(2) makeファイルの作成

コンパイルやアセンブルの前処理からリンクまでの処理を自動化するmakeファイルを作成します。基本的なmakeファイルはワークベンチ上で簡単に作成することができます。

(3) makeの実行

作成したmakeファイルを使用してmakeを実行し、デバッグ可能なsrf33形式のオブジェクトファイルを生成します。
makeでは下記の中で必要な処理が順次自動的に実行されます。

・コンパイル(Cソースの場合)

C言語で作成したソースファイルをCコンパイラgcc33でコンパイルし、命令エクステンダext33に入力するアセンブリソースファイル(.ps)を生成します。

・プリプロセッサによる前処理(アセンブリソースの場合)

アセンブリ言語で作成したソースファイルは、最初にプリプロセッサpp33で処理します。ここで、プリプロセッサが提供する機能を記述した部分をアセンブラが処理可能なニーモニックに展開し、命令エクステンダext33に入力するアセンブリソースファイル(.ps)を生成します。

・命令エクステンダによる最適化

命令エクステンダext33はソースファイル(.ps)中に記述された拡張命令をアセンブラが処理可能なニーモニックに展開し、アセンブラas33に入力するアセンブリソースファイル(.ms)を生成します。
また、即値拡張(ext)命令の使用数など、アセンブリソースの最適化が行われます。
複数モジュールをリンクする場合はリンクが終了するまでシンボルの絶対アドレスが決定しません。このため、リンク後のシンボル/マップファイルを入力して再度ext命令の最適化を行う2パスmakeが行えるようになっています。2パスmakeをmakeファイル中で指定すると、一度リンクまでの処理を行った後に、再度命令エクステンダ以降の処理を実行します。

・アセンブル

命令エクステンダext33の出力ファイルをアセンブラas33によってアセンブルし、リンクやライブラリ化が可能なオブジェクトファイルを生成します。

マルチモジュール(複数のソースファイル)として開発する場合は、ここまでの処理をすべてのソースファイルについて行います。

・リンク

アセンブルにより、1個または複数のオブジェクトファイルが揃います。これらのファイルを、複数のものは1つにまとめ、ROM上に配置して実行可能なオブジェクトファイルとして生成するツールがリンカlk33です。
リンカlk33は、デバッグに必要な情報などを含んだsrf33形式のオブジェクトファイルを出力します。

(4) デバッグ

リンカが生成したsrf33形式のオブジェクトファイルを使用して、動作の確認とデバッグをデバッガdb33で行います。S5U1C33104H、S5U1C33000H、デバッグモニタ等を使用することにより、ハードウェアの動作まで含めたデバッグが行えますが、パソコン上でS1C33000コアCPUとメモリモデルとしての動作をシミュレートするシミュレータモードも備わっています。

(5) ディスアセンブル

ディスアセンブラdis33は、リンク後のオブジェクトファイルをディスアセンブルしてソースコードと絶対アドレスの対応を確認したり、データ領域の内容をダンプ出力するためのソフトウェアです。プログラム開発に必須のツールではありませんが、1つのユーティリティとして使用してください。

(6) ROMデータ/マスクデータの作成

バイナリ/HEXコンバータhex33を使用して、リンカが生成したsrf33形式のオブジェクトファイルから、外部ROM書き込み用のHEXファイルおよび内部ROM用のマスクデータファイルを作成します。最終的に、このマスクデータファイルをセイコーエプソンに提出していただきます。

上記のツールの実行はすべてワークベンチ上で行えます。makeを使用せずにツールを個別に実行することも可能です。
また、これらのツール以外にライブラリアンlib33が用意されています。汎用的な処理を行うモジュール(アセンブラが出力したオブジェクトファイル)をライブラリとしてまとめておくことができますので、今後のS1C33 Familyを使用したアプリケーションの開発に有効です。

3.2　チュートリアル(ワークベンチによる操作の流れ)

前項に示した一連のツールは、本パッケージに含まれるワークベンチwb33によって簡単に起動することができます。本項ではチュートリアルとしてワークベンチwb33による操作の流れを見ていきます。各ツールの詳細については、それぞれのツールの章を参照してください。

使用するファイル

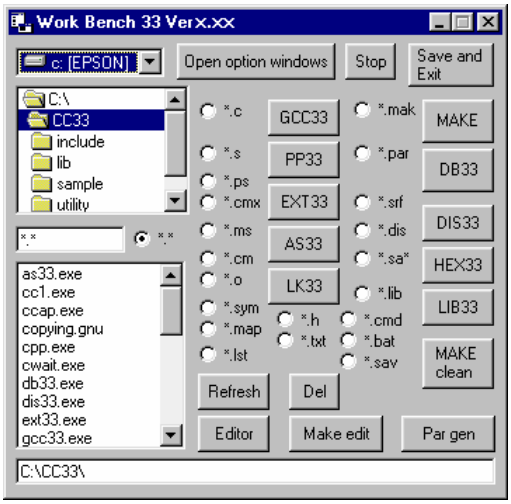
説明はsample¥tst¥ディレクトリに以下のファイルがコピーされているものとして行います。

main.c　　...Cソースファイル
boot.s　　...アセンブリソースファイル

以下、" main.c"と" boot.s"の2つのソースファイルのコンパイル/プリプロセスからリンクまでをmakeによって処理し、デバッガで動作を確認するまでの操作方法を説明します。その後、マスク化に必要な処理を簡単に説明します。

3.2.1　ワークベンチwb33の起動

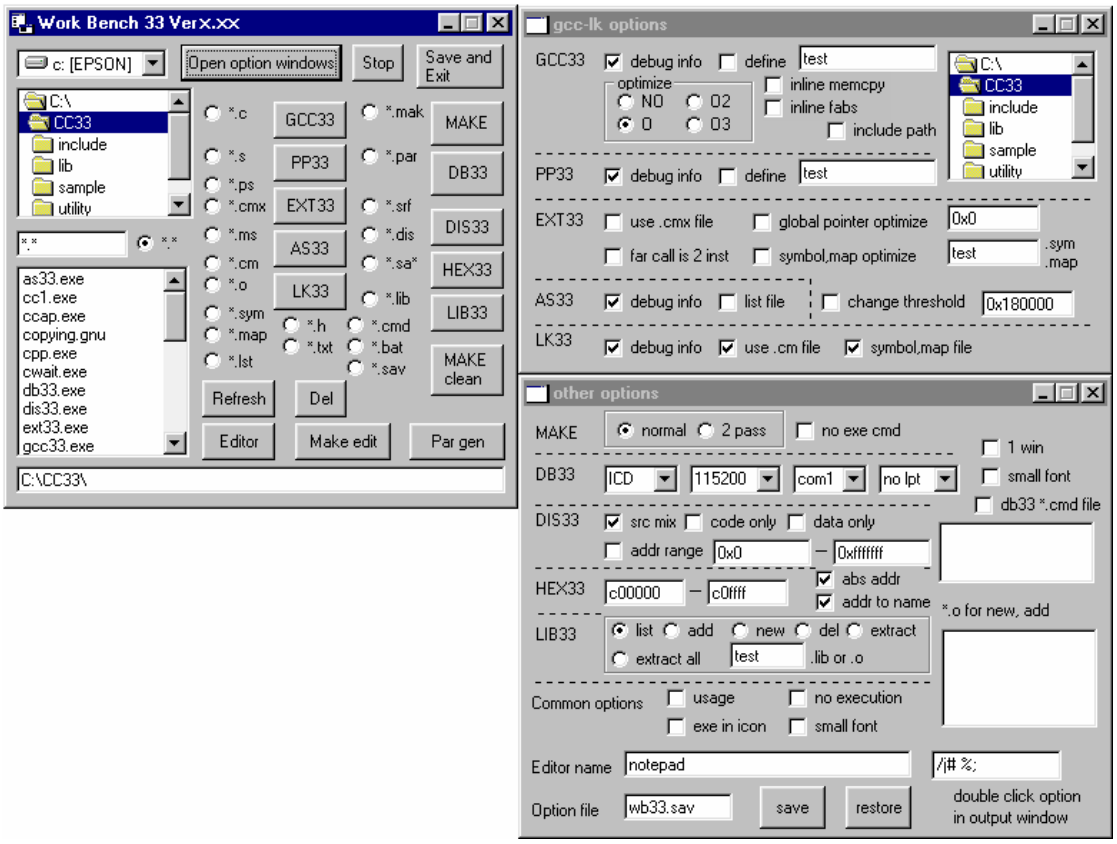
"cc33"フォルダ内の"wb33.exe"のアイコンをダブルクリックし、ワークベンチwb33を起動させてください。実行ウィンドウが開きます。



実行ウィンドウには、ファイルを選択するリストボックスや各ツールを起動させるボタンが配置されています。

操作1)　[Open option window]ボタンをクリックしてください。2つのオプションウィンドウが開きます。

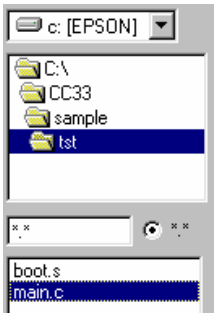
[Open option window]**ボタン**



[gcc-lk options]ウィンドウでは、Cコンパイラからリンカまでのツールの起動オプションが選択できます。[other options]ウィンドウでは、他のツールの起動オプションと各ツールに共通なオプションが選択できます。

オプションを指定するチェックボックスは、ワークベンチwb33の起動時に初期設定され、通常指定するものにチェックが付きます。以下の説明は特に指定する場合を除き、初期設定のまま行います。詳細については"5 ワークベンチ"、および各ツールの章を参照してください。

3.2.2 ディレクトリの選択とファイルの表示



最初に、すべてのツールの起動に共通するファイル選択部を見ておきましょう。
ワークベンチwb33起動時は、ツールをインストールしたドライブとディレクトリの内容が表示されます。
では、チュートリアルに使用するファイルを表示させてみます。

操作2) ディレクトリリストボックスでsample¥tst¥ディレクトリを選択してください。

初期設定では[*.*]ラジオボタンがチェックされ、tst¥ディレクトリ内のすべてのファイル名が表示されます。
各ツールの起動ボタンの前には、そのツールが入力するファイル形式に対応したラジオボタンがありますので、指定形式のファイルのみを表示させることも可能です。

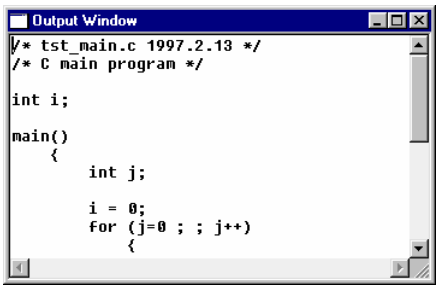
操作3) [*.c]ボタンをクリックしてください。

ファイルリストボックスには"main.c"のみが表示されます。

●テキストファイルの内容を表示させるには

ワークベンチwb33にはテキストファイルの表示機能があります。

操作4) ファイルリストボックス内のファイル名(main.c)をダブルクリックします。



出力ウィンドウが開き、"main.c"の内容を表示します。

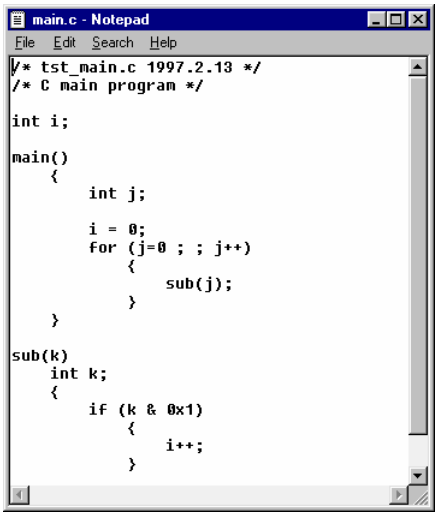
注:
- **このウィンドウに表示できるファイルはテキストファイルのみで、サイズは32KBまでです。また、ASCIIキャラクタ以外のコードが含まれていると文字化けを起こすことがあります。**
- **ウィンドウ内で文字列をコピーしたり修正することはできますが、保存することはできません。単なる表示機能のみとしてお使いください。**

●エディタを開くには

エディタを開いてテキストファイルを表示させることもできます。

[Editor]ボタン

操作5) "main.c"を選択し、実行ウィンドウの[Editor]ボタンをクリックしてください。



Windowsの「メモ帳」が開き、"main.c"の内容を表示します。この機能によって、ソースファイルの修正等が即時に行えます。

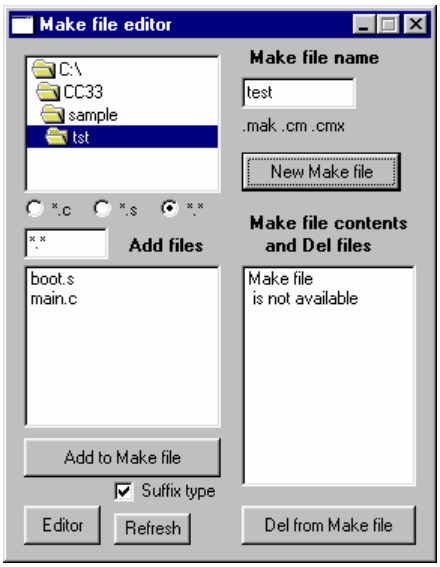
エディタは初期設定で「メモ帳(notepad)」に設定されていますが、通常使用しているエディタを開くこともできます。[other options]ウィンドウの[Editor name]テキストボックスにエディタの起動コマンドをフルパスで入力してください。

3.2.3 makeファイルの作成

makeファイルはコンパイルからリンクまでの処理手順を記述したファイルで、makeユーティリティが読み込んで記述された一連の処理を自動実行します。ファイルの更新をサポートしており、前回のmake以降に変更されたファイルがある場合にのみ処理を実行させることができます。
では、サンプルのソースファイル("main.c"と"boot.s")を処理するmakeファイルを作成します。

●makeファイルを作成するには

[Make edit]ボタン



操作6) [Make edit]ボタンをクリックしてください。

[Make file editor]ウィンドウが表示されます。

操作7) ファイルリストボックスの"main.c"と"boot.s"を選択し、[New Make file]ボタンをクリックします。(2つのファイルを選択するには、最初に"boot.s"をクリックし、次に[Ctrl]キーを押しながら"main.c"をクリックしてください。)

終了メッセージが表示され、次の3つのファイルが生成されます。

test.mak　makeファイル
test.cm　リンカ用のコマンドファイル
test.cmx　命令エクステンダ用のコマンドファイル

これらのファイルはテキスト形式です。出力ウィンドウやエディタで表示可能です。

ファイル名には[Make file name]テキストボックスに設定されている名称が用いられます。テキストボックスの内容を変更することによって他のファイル名も指定可能です。この名称は、リンク後のオブジェクトファイルやその他のファイルにも適用されます。

ここで作成されたmakeファイルは基本的な内容のみを記述したテンプレートとして使用できますが、場合によってはカスタマイズが必要となります。makeファイルの詳細については"17.1 make"を参照してください。

[Make file editor]は[閉じる]ボタンで終了させてください。

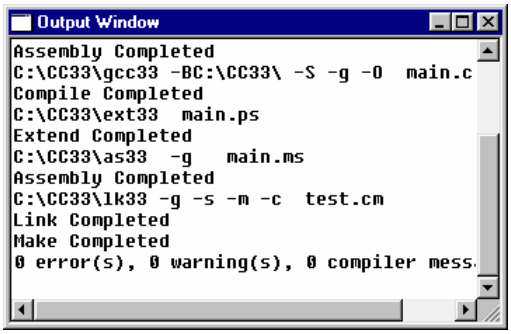
3.2.4　コンパイルからリンクまでの自動実行

前項で作成した"test.mak"を使用してmakeを実行します。

●makeを実行するには

操作8)　実行ウィンドウのファイルリストボックスから"test.mak"を選択し、[MAKE]ボタンをクリックします。

[MAKE]ボタン



"boot.s"のプリプロセスからアセンブル、"main.c"のコンパイルからアセンブル、それぞれのオブジェクトファイルのリンク処理が連続して実行され、sample¥tst¥ディレクトリには各ツールの出力ファイルが生成されます。

boot.ps
　　プリプロセッサpp33の出力ファイル

main.ps
　　Cコンパイラgcc33の出力ファイル

boot.ms, main.ms
　　命令エクステンダext33の出力ファイル

boot.o, main.o
　　アセンブラas33の出力ファイル

test.srf, test.sym
　　リンカlk33の出力ファイル

3.2.5　ツールを個別に実行するには

makeを使用せずにツールを個別に実行させることもできます。
たとえば、コンパイルのみを行うには

操作9)　[*.c]ラジオボタンをチェックしてCソースファイル名を表示させます。(必要な場合のみ)

操作10)　ファイルリストボックスの"main.c"を選択し、[GCC33]ボタンをクリックします。

ソースファイルの構文エラー修正時などは、この方法でコンパイルのみが行えます。
他のツールも同様の操作で個別に実行可能です。

3.2.6 デバッガ用パラメータファイルの作成

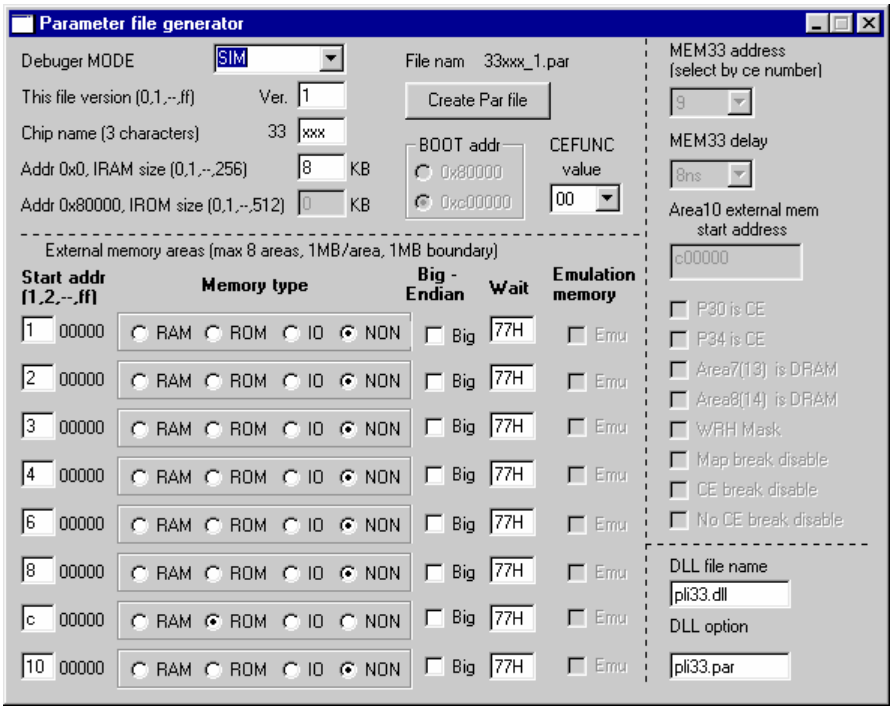
デバッグを行う前にデバッガ用のパラメータファイルを作成する必要があります。
デバッガdb33はS1C33 Familyの全機種に対応しています。ただし、機種によってメモリ構成や対応ボード等が異なるため、各機種ごとに使用可能なメモリ範囲やボードの情報が必要です。パラメータファイルはそれらの情報をデバッガに設定するためのファイルです。

● パラメータファイルを作成するには

[Par gen]**ボタン**

操作11) [Par gen]ボタンをクリックしてください。

[Parameter file generator]ウィンドウが表示されます。



操作12) [Debugger MODE]リストボックスから[SIM](シミュレータモード)を選択します。

これは、後でデバッガをシミュレータモードで起動するための選択です。実際のアプリケーション開発時は、デバッグツールに合わせたモード選択が必要です。
このほかにも、メモリ構成等を指定する必要がありますが、ここではデフォルト設定のままパラメータファイルを作成します。

操作13) [Create Par file]ボタンをクリックしてください。

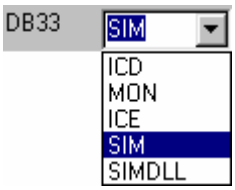
"33xxx_1.par"というパラメータファイルが作成されました。

パラメータファイルの内容および各パラメータの指定方法については、"16.11 パラメータファイルについて"を参照してください。

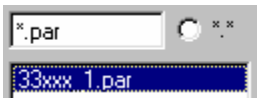
3.2.7　デバッグ

makeによって実行形式(srf33)のオブジェクトファイルができ上がりましたので、プログラムのデバッグが行えます。
S5U1C33104HやS5U1C33000Hを使用することにより、より高度なデバッグが行えますが、ここでは、まず基本的な操作を見るために、パソコンのみでデバッグ可能なシミュレータモードでデバッガdb33を起動します。

●デバッガdb33を起動するには



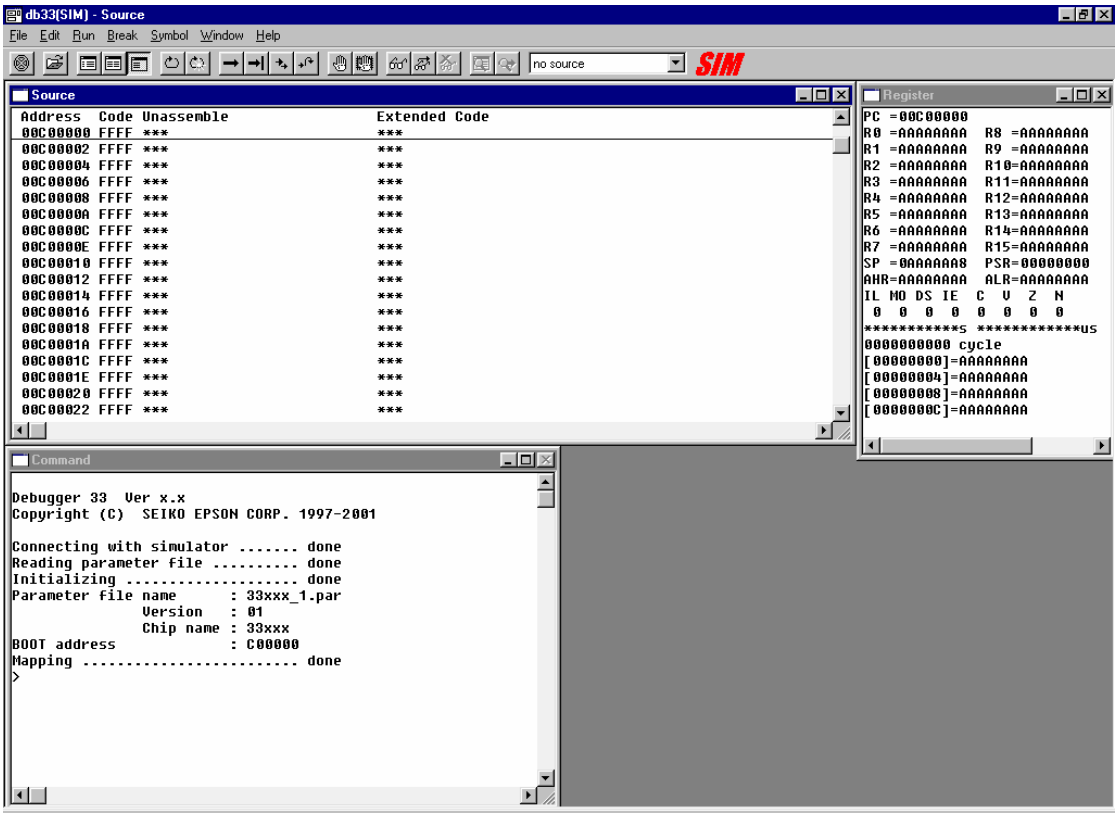
操作14)　[other option]ウィンドウのDB33用オプション選択部で、[SIM]を選択してください。(シミュレータモードの指定)



操作15)　実行ウィンドウのファイルリストボックスでパラメータファイル(33xxx_1.par)を選択し、 [DB33]ボタンをクリックしてください。[*.par]ラジオボタンをチェックしておくと、ファイル選択が容易になります。

[DB33]**ボタン**

デバッガdb33が起動すると、次のウィンドウが開きます。



はじめに、デバッグするファイルを読み込みます。

●ファイルを読み込むには

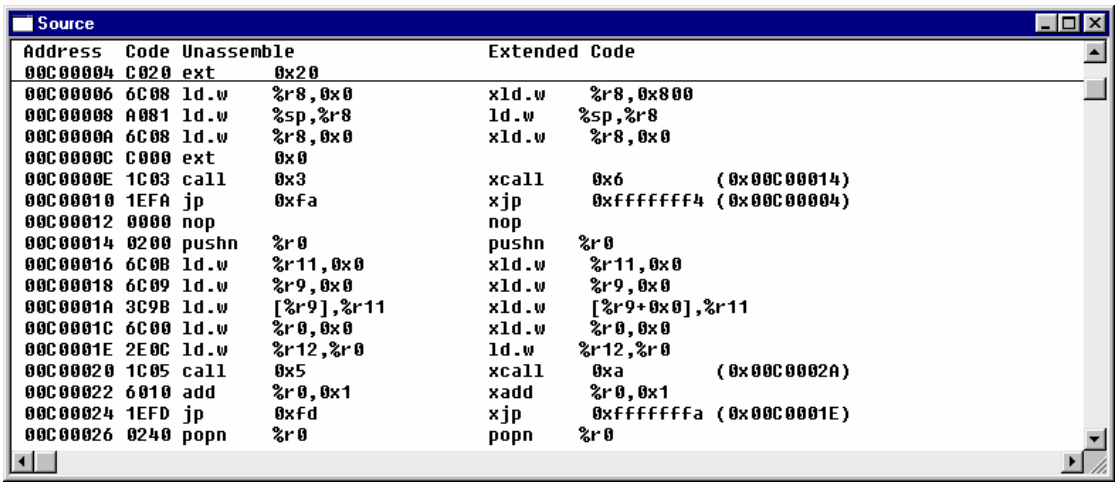
[Load file]**ボタン**

操作16) ツールバーの[Load file]ボタンをクリックしてください。ファイル選択用のダイアログボックスが開きます。

操作17) ダイアログボックスのファイルリストボックスから"test.srf"を選択し[Open]ボタンをクリックしてください。
オブジェクトファイル"test.srf"が読み込まれます。

[Reset cold]**ボタン**

操作18) ツールバーの[Reset cold]ボタンをクリックしてください。(PCがプログラムの開始アドレスに設定されます。)



[Source]ウィンドウ内には、オブジェクトコードを逆アセンブルした内容が表示されています。これをソースの表示やミックス表示(逆アセンブル内容とソースの両方を表示)に切り換えることができます。

●ソースを表示させるには

[Source]ボタン

操作19)　ツールバーの[Source]ボタンをクリックしてください。[Source]ウィンドウの表示が切り換わります。

```
Source  - boot.s -
Address    Line  SourceCode
           00001 ; boot.s  1997.2.13
           00002 ; boot program
           00003
           00004 #defnum SP_INI  0x0800  ; sp is in end of 2KB internal RAM
           00005 #defnum GP_INI  0x0000  ; global pointer %r8 is 0x0
           00006
           00007         .code
           00008         .word BOOT              ; BOOT VECTOR
           00009 BOOT:
00C00004   00010         xld.w   %r8,SP_INI
00C00008   00011         ld.w    %sp,%r8         ; set SP
00C0000A   00012         ld.w    %r8,GP_INI      ; set global pointer
00C0000C   00013         xcall   main            ; goto main
00C00012   00014         xjp     BOOT            ; infinity loop
```

[Source]ウィンドウ内には、現在のPC位置のコードを含むソースファイル(boot.s)の内容が表示されます。オブジェクトファイルが他のソースも参照していれば(サンプルではmain.c)、ツールバーのコンボボックスで選択することができます。

●ミックス表示させるには

[Mix]ボタン

操作20)　ツールバーの[Mix]ボタンをクリックしてください。[Source]ウィンドウの表示が切り換わります。

```
Source
Address   Code Unassemble                Line    SourceCode
00C00004  C020 ext      0x20             00010           xld.w   %r8,SP_INI
00C00006  6C08 ld.w     %r8,0x0
00C00008  A081 ld.w     %sp,%r8          00011           ld.w    %sp,%r8         ; set SP
00C0000A  6C08 ld.w     %r8,0x0          00012           ld.w    %r8,GP_INI      ; set global pc
00C0000C  C000 ext      0x0              00013           xcall   main            ; goto main
00C0000E  C000 ext      0x0
00C00010  1C02 call     0x2
00C00012  1EF9 jp       0xf9             00014           xjp     BOOT            ; infinity loop
                                              --- main.c ---
                                         00001  /* tst_main.c 1997.2.13 */
                                         00002  /* C main program */
                                         00003
                                         00004  int i;
                                         00005
                                         00006  main()
00C00014  0200 pushn    %r0              00007      {
                                         00008           int j;
                                         00009
```

[Source]ウィンドウ内には、逆アセンブル結果とソースファイルの内容が表示されます。これにより、ソースとニーモニックの対応もよく分かります。
次に実行される命令(アドレス)は黒の下線で示されます。

では、とりあえず読み込んだプログラムを実行させてみましょう。

●プログラムを実行させるには

[Go]ボタン

操作21) ツールバーの[Go]ボタンをクリックしてください。

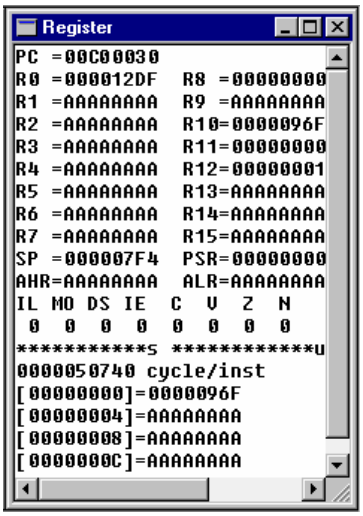
このプログラムはRAM領域の変数i(0x0000000～0x0000003番地)をカウンタとしてインクリメントを永遠に繰り返します。ICEモードの場合は、オンザフライ機能により[Register]ウィンドウの内容がリアルタイムで更新されているのが分かります。シミュレータモードでは、プログラムをブレークさせるまで[Register]ウィンドウの内容は表示されません。また、マウスのカーソルは砂時計表示になります。

このような永久ループは、強制ブレークによって停止させます。

●強制的にブレークさせるには

[Key break]ボタン

操作22) ツールバーの[Key break]ボタンをクリックしてください。(ボタン上でもカーソルは砂時計表示のままですが、クリック可能です。)



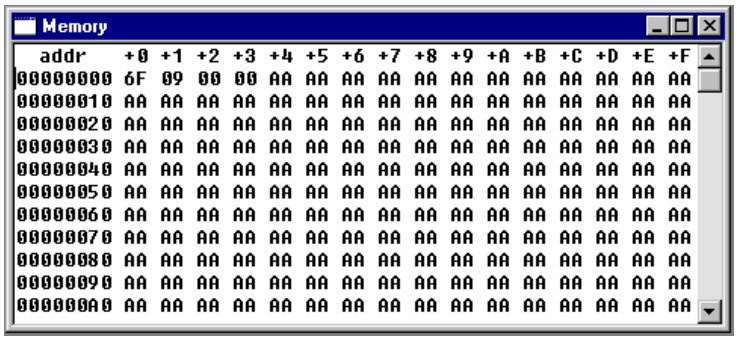
この図はプログラムが0x00c00030番地(PC)でブレークし、それまでに50740サイクルを実行したことを示しています。0番地から設定したカウンタは0x96Fになっています。([0000001]から[000000C]はコマンドで設定可能なデータメモリの監視アドレスで、ここでは初期設定の0x0、0x4、0x8、0xC番地が表示されています。"[0000000] ="の右に表示されているメモリの内容は右端が0番地、左端が3番地の内容です。)

監視アドレス以外のデータメモリの内容は[Memory]ウィンドウで確認することもできます。

●[Memory]ウィンドウを開くには

[Window]メニュー

操作23) [Window]メニューから[Memory]コマンドを選択してください。



[Memory]ウィンドウが開き、メモリの内容が表示されます。垂直スクロールバーで先頭部分を表示させてください。(0～3番地以外の"AA"はRAM領域の初期設定内容です。)
垂直スクロールさせることにより、データメモリ全域のデータが確認できます。

ここまでは変数iの内容をアドレスで確認しましたが、Cソースレベルのデバッグでは実際的ではありません。そこで、変数の内容を直接表示させてみます。[Command]ウィンドウに表示させることもできますが、ここでは[Symbol]ウィンドウを使用します。

●[Symbol]ウィンドウを開くには

[Window]メニュー

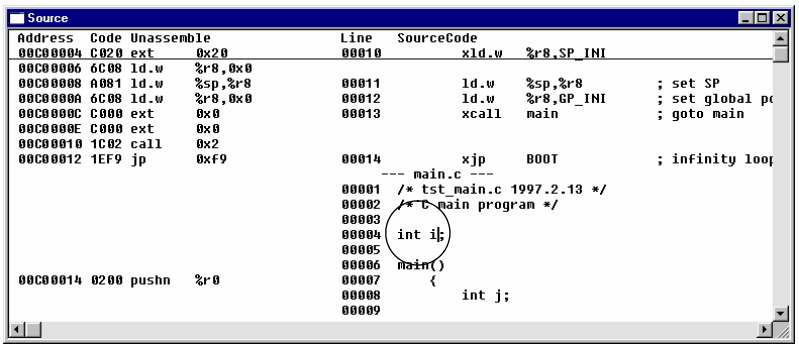
操作24)　[Window]メニューから[Symbol]コマンドを選択してください。

[Symbol]ウィンドウに変数の内容を表示させるためには、そのシンボルを登録する必要があります。

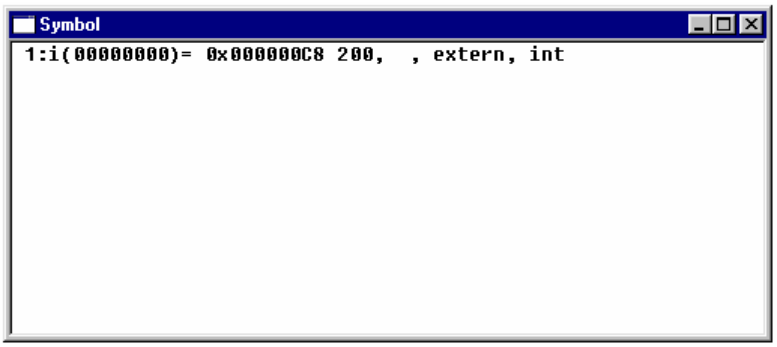
●[Symbol]ウィンドウに監視するシンボルを追加するには

[Symbol add]ボタン

操作25)　[Source]ウィンドウの変数iが記述されている個所にカーソルを置き、ツールバーの[Symbol add]ボタンをクリックしてください。



[Symbol]ウィンドウに変数iの情報が表示されます。



再度プログラムを実行させてみてください(操作21と22)。強制ブレーク後、変数iの内容が変更されているのが分かります。

ここまでの実行では動きがよく分かりませんので、適当な場所でブレークするように設定してみます。

●ブレークポイントを設定するには

[Soft PC break]
ボタン

操作26) [Source]ウィンドウ内のアドレス0x00c00032(i++)の行をクリックしてその行にカーソルを表示させ、ツールバーの[Soft PC break]ボタンをクリックしてください。

アドレス0x00c00032の行に赤の下線が引かれるとともに行の先頭に"!"マークが付いて、ブレークポイントが設定されたことを示します。(ここでもう一度[Soft PC break]ボタンをクリックするとブレーク設定は解除されます。)
ブレーク設定後、もう一度プログラムを実行させてください。

操作27) ツールバーの[Go]ボタンをクリックしてください。

[Source]ウィンドウのアドレス0x00c00032の行に黒の下線が付き、ブレークしていることが分かります。
この後、操作27を繰り返すと、変数iがインクリメントされることが分かります。

この方法で、目的の動作を行っているか確認することができます。
動きに問題がある場合、もう少し詳しく動作確認する必要があります。
最後にステップ動作、NEXT動作を見てみましょう。
その前に、操作26をもう一度行い、アドレス0x00c00032のブレークポイントを解除してください(NEXT動作確認のために必要)。

●ステップ実行するには

[Step]**ボタン**

操作28) ツールバーの[Step]ボタンをクリックしてください。

[Source]ウィンドウ内で黒の下線が引かれた命令を実行し、下線が次に実行する命令に移ります。
操作28を繰り返すことで、各ステップを順次実行します。プログラムに問題がなければ、レジスタの表示などがステップごとに正しく変わっていることが分かります。
ステップ動作ではすべての命令を1ステップずつ実行します。

NEXT動作は基本的にはステップ動作と同じですが、関数やサブルーチンをスキップ(1ステップとして実行)します。デバッグの終わったサブルーチンなどは1ステップずつ実行させる必要もないため、NEXT実行が便利です。

●NEXT実行するには

[Next]**ボタン**

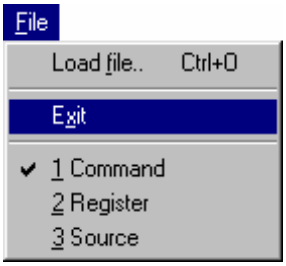
操作29) ツールバーの[Next]ボタンをクリックしてください。

操作29を繰り返して、ステップ実行との違いを[Source]ウィンドウで確認してください。
関数sub( )をスキップしましたが、変数iの内容が更新され、関数内は連続実行されたことが分かります。

以上、デバッガdb33の基本的な操作を見てきましたが、[Command]ウィンドウにキーボードからコマンドを入力して、さらに高度なデバッグを行うことができます。その詳細については"16 デバッガ"を参照してください。

なお、デバッガdb33は次の方法で終了します。

●デバッガを終了するには



[File]**メニュー**(db33)

操作30)　[File]メニューから[Exit]コマンドを選択してください。

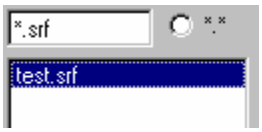
ウィンドウが閉じて、ワークベンチのウィンドウに戻ります。

本項で行ったシミュレータモード以外にも、インサーキットエミュレータS5U1C33104H、デバッグモニタを実装したターゲットボードとS5U1C330M1D1、あるいはターゲットボードとS5U1C33000Hを使用してデバッグを行うこともできます。それぞれのモードでのデバッグ方法については3.3項を参照してください。

3.2.8 ディスアセンブリファイルの作成

ディスアセンブラdis33はリンカが出力したsrf33形式のオブジェクトファイルを逆アセンブルするとともに、Cソースやアセンブリソースと対応させたリストファイルを生成します。このリストによって、ソースとオブジェクトコードの対応を見ることができます。

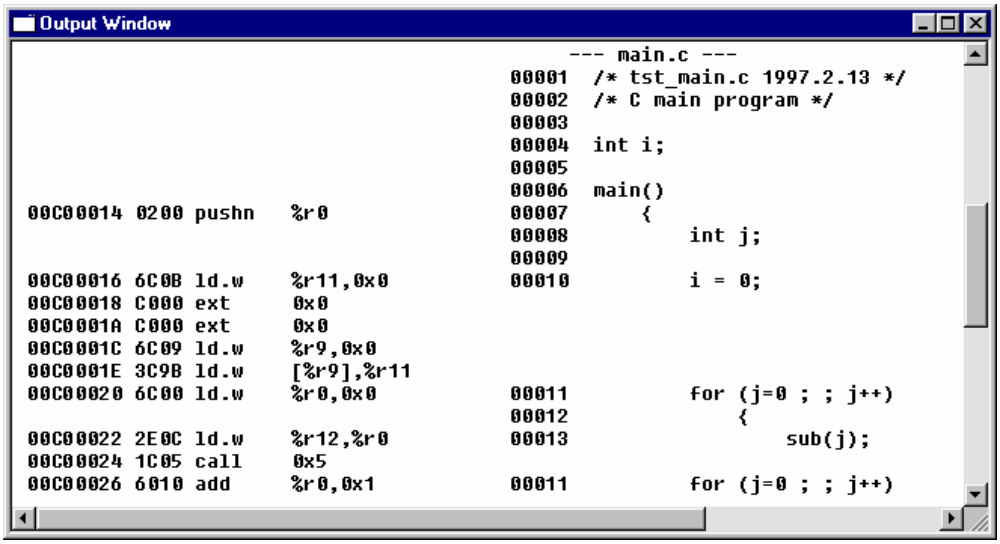
●ディスアセンブリファイルを作成するには



[DIS33]**ボタン**

操作31) ワークベンチのファイルリストボックスで"test.srf"を選択し、[DIS33]ボタンをクリックします。[*.srf]ラジオボタンをチェックしておくと、ファイル選択が容易になります。

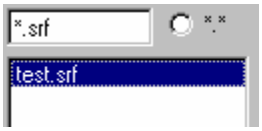
"test.dis"というリストファイルが生成されますので、そのファイル名をダブルクリックして内容を表示させてみてください。Cソースがどのようにニーモニックに変換されているかがよく分かります。



3.2.9　ROMデータの作成

ターゲットボードに搭載する外部ROM用のデータや内蔵ROMのマスクデータを作成するにはHEXファイルが必要です。バイナリ/HEXコンバータhex33はリンカが出力したsrf33形式のオブジェクトファイルの指定アドレス領域をモトローラS3形式のHEXファイルに変換します。

●HEXファイルを作成するには



[HEX33]ボタン

操作32)　ファイルリストボックスで"test.srf"を選択し、[HEX33]ボタンをクリックします。
[*.srf]ラジオボタンをチェックしておくと、ファイル選択が容易になります。

"test.sa_c00000_c0ffff"という外部ROM用のHEXファイルが生成されます。

ここではhex33のオプション指定をワークベンチのデフォルト設定のまま変換しましたので、0xc00000番地～0xc0ffff番地までの64KBのHEXデータが生成されました。実際の開発時には、機種のメモリ構成に合わせて、[other options]ウィンドウのHEX33用オプション選択部でアドレスを指定する必要があります。

●提出用マスクデータの作成

ROMを内蔵する機種のプログラムが完成後は、内蔵ROM用のマスクデータをセイコーエプソンに提出していただくことになります。このマスクデータも上記のHEXファイル作成手順と同様に行います。
ただし、内蔵ROMのアドレスを正しく設定するよう注意してください。また、[abs addr]を必ずチェックし、絶対アドレスのデータを作成してください。
次の例は、0x80000番地～0x80fff番地までの4KBのマスクデータを生成する場合の設定です。

生成したマスクデータファイルは、セイコーエプソンが指定する名称にリネームして提出してください。
例: c32640a0.sa0 (S1C33264用マスクデータファイル)

<u>マスクデータ作成時の注意事項</u>

ファイルのコピーミス、万一のツールのバグ等を避けるため、最終動作検収はsrf33ファイルではなく、モトローラS3フォーマットのHEXファイル(.sa)をデバッガのlhコマンドで読み込んで行うようお願いします。

3.2.10 最適化

開発手順を一通り見てきましたが、最後に本パッケージの特長であるコードの最適化について触れておきます。
[gcc-lk options]ウィンドウのgcc33用オプション選択部には[optimize]フィールドが用意されており、その中のラジオボタンで最適化レベルを指定することができます。サンプルプログラム程度では効果が確認できませんので、他の方法を紹介しておきます。

ひとつはグローバルポインタを使用する方法です。
グローバルポインタはグローバル変数領域の先頭アドレスのことで、汎用レジスタR8をその領域のアクセス専用に使用します。これにより、グローバル変数のアクセスに必要な命令数を削減することができます。サンプルのアセンブリソースでR8レジスタを初期化しているのは、このグローバルポインタを設定するための処理です。
この機能は命令エクステンダのオプションで、ワークベンチのデフォルトでは選択されません。したがって、チュートリアルのmakeではこの最適化は行われていません。
ext33のオプション[global pointer optimize]をチェックしてmakeを行うと出力コードの違いが確認できます。サンプルプログラムでグローバル変数i(アドレスは0にマップされる)をアクセスしている部分の違いを以下に示します。

グローバルポインタを使用しない場合

```
00C00032 C000 ext     0x0              00022                    i++;
00C00034 C000 ext     0x0
00C00036 6C09 ld.w    %r9,0x0
00C00038 309A ld.w    %r10,[%r9]
00C0003A 601A add     %r10,0x1
00C0003C C000 ext     0x0
00C0003E C000 ext     0x0
00C00040 6C09 ld.w    %r9,0x0
00C00042 3C9A ld.w    [%r9],%r10
```

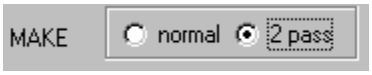
グローバルポインタを使用した場合

```
00C00030 C000 ext     0x0              00022                    i++;
00C00032 C000 ext     0x0
00C00034 308A ld.w    %r10,[%r8]
00C00036 601A add     %r10,0x1
00C00038 C000 ext     0x0
00C0003A C000 ext     0x0
00C0003C 3C8A ld.w    [%r8],%r10
```

グローバルポインタを使用することにより、2命令削減されたことが分かります。

もうひとつの最適化方法として2パスmakeがあります。2パスmakeでは一度リンクまでの処理を行った後に、決定したシンボルの絶対アドレス情報をもとに再度命令エクステンダ以降の処理を行います。これにより、外部モジュールにある分岐先ラベルやシンボル参照に使用している不要なext命令を削除することができます。

●2パスmakeを行うには



操作33) [other options]ウィンドウのmake用オプション選択部で[2 pass]ラジオボタンをチェックしてから[MAKE]ボタンでmakeを実行してください。

サンプルファイルを2パスmakeして最適化された部分を以下に示します。

1パスmakeの場合

```
00C00030 C000 ext     0x0              00022                    i++;
00C00032 C000 ext     0x0
00C00034 308A ld.w    %r10,[%r8]
00C00036 601A add     %r10,0x1
00C00038 C000 ext     0x0
00C0003A C000 ext     0x0
00C0003C 3C8A ld.w    [%r8],%r10
```

2パスmakeの場合

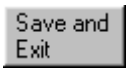
```
00C0002C 308A ld.w    %r10,[%r8]       00022                    i++;
00C0002E 601A add     %r10,0x1
00C00030 3C8A ld.w    [%r8],%r10
```

上記例は、それぞれのmake実行により生成されたオブジェクトファイル"test.srf"をディスアセンブラdis33で読み込み作成したディスアセンブリリストの一部です。デバッガ上でソースをミックス表示させることによっても確認できます。
例からは、変数iはアドレスが0と判明したため、アクセスに使用しているext命令が2パスmakeによって削除されたことが分かります。具体的には、ext33がこの処理を行います。

3.2.11 終わりに

このチュートリアルでは開発手順に添って基本的な操作を見てきました。各ツールの詳細はそれぞれの章を参照してください。

●ワークベンチを終了するには



[Save and Exit]**ボタン**

操作34) [Save and Exit]ボタンをクリックしてください。

これにより、wb33は"wb33.sav"というファイルにオプション設定情報を保存して終了します。
次回からは、作成された"wb33.sav"を"wb33.exe"アイコンにドラッグ&ドロップすることによって、今回と同じオプション設定でwb33を起動することができます。この操作を容易にするため、"wb33.exe"のショートカットをデスクトップ上に作成しておくことを推奨します。

注: make**も含め、各ツールは**DOS**プロンプト上でコマンド入力またはバッチファイルを使用して起動することもできます。**

S5U1C33104H、S5U1C33000H、S5U1C330M1D1などのデバッグターゲットをお持ちの方は、引き続き"3.3 デバッグ環境"に記載の、各ツールを使用する場合の操作手順をお読みください。

3.3 デバッグ環境

前項のデバッグ例はパーソナルコンピュータのみで可能なシミュレータモードによるものでしたが、デバッグモニタS5U1C330M2S、インサーキットエミュレータS5U1C33104H、あるいはインサーキットデバッガS5U1C33000Hを使用することによりターゲットシステムを実際に動作させてデバッグを行うことができます。
以下に、それぞれのシステムの概要とデバッグの開始手順を説明します。詳細についてはそれぞれのマニュアルを参照してください。またオンボードのフラッシュ書き込みについては"cc33¥utility¥fls33"も参照してください。

注: 各システムの接続と切り離しは、必ずすべての機器の電源をOFFにして行ってください。

3.3.1 インサーキットエミュレータS5U1C33104H

S5U1C33104HはS1C33 Familyのmodel 1で使用可能なインサーキットエミュレータで、最も高度なデバッグ環境を提供します。内蔵周辺機能はS5U1C33104Pボードで提供されます。このシステムではデバッガdb33のほとんどの機能が使用可能で、オプションのメモリカードにより最大8MBの外部メモリのエミュレーションも行えます。

●システム構成

図3.3.1.1にS5U1C33104Hを使用するデバッグシステムの構成を示します。

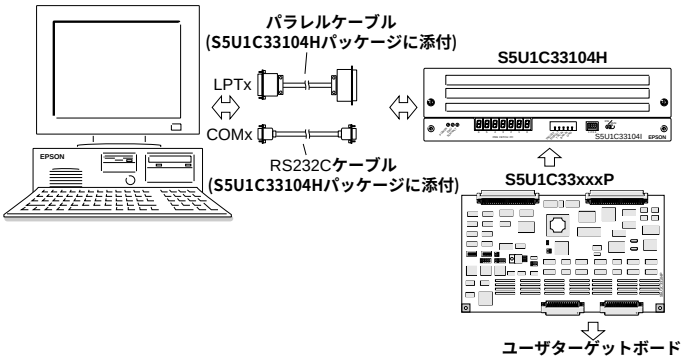
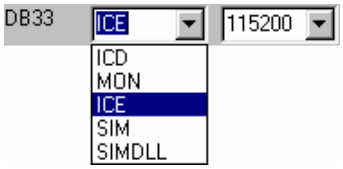


図3.3.1.1 S5U1C33104H**を使用するデバッグシステム**

●起動と動作確認

デバッガをICEモード(wb33上で[ICE]を選択)で起動してください。



起動する前に、以下の点を確認してください。

1) RS232Cケーブルは指定のものが使われているか？
2) パソコン側のCOM1ポートに接続されているか？(COM1以外の場合、wb33で変更)
3) S5U1C33104HのICE/RUNスイッチはICE側になっているか？
4) S5U1C33104HのDIPスイッチは1、3、4がOPEN、2がON(115200bps、自己診断OFF)になっているか？
5) S5U1C33104Hに正しいS5U1C33xxxPボードが正しく装着されているか？
6) S5U1C33104Hの電源がON(POWER LEDが点灯)になっているか？

上記の設定が正しくないと"time out"エラー等が発生し、デバッガdb33は正常には起動しません。

ICEモードで起動後、前項で行った操作をシミュレータモードと同様に行ってください("3.2.7デバッグ"参照)。[Go]ボタンによるプログラムの実行時は、[Register]ウィンドウのPC、フラグ、監視データの内容がオンザフライ機能によりリアルタイムに更新されます。

●注意事項

(1) S5U1C33104Hのエミュレーションメモリの構成はデバッガ用パラメータファイルの内容に従って設定されます。パラメータファイルはターゲットシステムに合わせ、正しく作成してください。

(2) S5U1C33104HをRS232CケーブルでCOMポートに接続するのみで、S5U1C33104Hの全機能を使用することができます。LPTポートへは、ファイルの高速ダウンロードが必要な場合にパラレルケーブルで接続してください。それぞれのポートによる標準的なダウンロード速度は以下のとおりです(使用するPCや条件により変わります)。

シリアル転送時:	RAMへのダウンロード	9KB/S
	フラッシュメモリへのダウンロード	8KB/S
パラレル転送時:	RAMへのダウンロード	50KB/S
	フラッシュメモリへのダウンロード	30KB/S

(3) S5U1C33104Hの出荷時は、Ver. 1のファームウェアが実装されています。本パッケージのデバッガdb33で問題なく使用できますが、ターゲットボード上のフラッシュメモリへの書き込み(fls、fle)、ハードウェアPCブレーク2(bh2、bhc2)、ハーフワード/ワード単位のメモリコピー(mvh、mvw)には対応していません。これらの機能を使用する場合は、"cc33¥utility¥ice33v20¥"ディレクトリ内のプログラムでS5U1C33104Hのファームウェアをアップデートしてください。

(4) その他S5U1C33104Hの制限事項については、"S1C33 Family In-Circuit Emulator Manual"を参照してください。

3.3.2 デバッグモニタS5U1C330M2S

デバッグモニタS5U1C330M2SはS1C33 Family用のミドルウェアで、S5U1C33xxxDボードやユーザターゲットボード上でのプログラムデバッグ機能を提供します。S5U1C330M2Sを実装したボードをS5U1C330M1D1ボードを介してパーソナルコンピュータに接続することにより、デバッガdb33を使用したプログラムのデバッグが行えます。ここでは、デバッグモニタが実装されているS5U1C33104D1/S5U1C33209D1ボードを開発ツールとして使用してプログラムのデバッグを行う方法を説明します。

●システム構成と接続

図3.3.2.1にS5U1C33104D1/S5U1C33209D1ボードを使用したデバッグシステムの構成を示します。

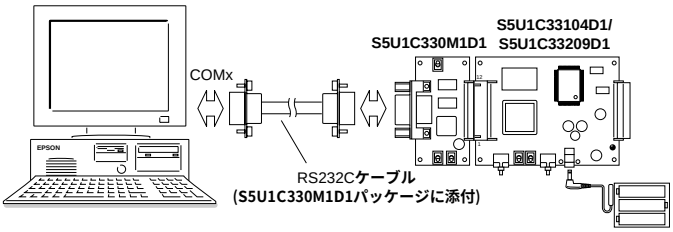


図3.3.2.1 S5U1C33104D1/S5U1C33209D1**ボードを使用するデバッグシステム**

●起動と動作確認

以下のサンプルプログラムで動作確認が行えます。

S5U1C33104D1ボード用: "¥cc33¥sample¥dmt33004¥led.srf"
"¥cc33¥sample¥dmt33004¥led2.srf"
S5U1C33209D1ボード用: "¥cc33¥sample¥dmt33005¥led.srf"
"¥cc33¥sample¥dmt33005¥led2.srf"

すべてS5U1C33xxxDボード上のLEDを点滅させるサンプルプログラムです。"led.srf"はRAM(0x600000~)上で、"led2.srf"はFlashメモリ(0x200000~)上でデバッグ可能なように作成されています。

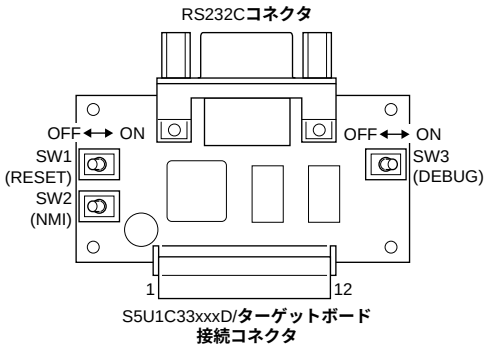
プログラムの内容については、各ディレクトリ内のソースファイル(led.s)を参照してください。

サンプルプログラムは、実行形式のオブジェクトファイルが用意されていますので、ソースの修正が不要な場合はmakeを実行する必要はありません。

ソースを修正して使用する場合は、それぞれのディレクトリに用意されているmakeファイルを使用してmakeを実行してください。

(1) デバッグモニタの起動

S5U1C33104D1/S5U1C33209D1の0xC00000からマップされているブートルーチンは、K63入力ポートが"0"(S5U1C330M1D1の[DEBUG]スイッチがON)に設定されている場合にデバッグモニタを起動するようになっています。
ターゲットシステムとパーソナルコンピュータを接続後、次の手順でデバッグモニタを起動してください。



1) S5U1C330M1D1のSW3[DEBUG]をON
2) S5U1C33104D1/S5U1C33209D1の電源をON
3) S5U1C33104D1/S5U1C33209D1をリセット
(S5U1C330M1D1のSW1[RESET]をON→OFF)
4) パーソナルコンピュータの電源をONし、Windowsを起動
5) デバッガdb33を起動(起動方法は後述)

図3.3.2.2 S5U1C330M1D1**ボードレイアウト**

注: S5U1C330M1D1**の**SW3[DEBUG]**を**OFF**した状態で**S5U1C33104D1/S5U1C33209D1**の電源を**ON**すると、デバッグモニタは起動せずに**Flash**メモリ**(0x200000~)**の先頭に**TTBR**を設定し、そのブートアドレスに分岐します。この場合は**SW3[DEBUG]**を**ON**し、**SW1[RESET]**でリセットしてデバッグモニタを起動してください。**

(2) RAM上でのデバッグ

S5U1C33104D1/S5U1C33209D1のRAM(0x600000~)上でデバッグを行う場合のサンプルプログラムは"led.srf"です。トラップテーブルアドレスをRAMの先頭に設定し"led.srf"をRAMにロードするデバッグコマンドファイル"led.cmd"が用意されていますので、デバッガ起動時に-cオプションで指定してください。DOSプロンプトからデバッガを起動させる操作手順は次のとおりです。

1) 前述のとおりデバッグモニタを起動
2) "¥cc33¥sample¥dmt33004¥"(または"¥cc33¥sample¥dmt33005¥")をカレントディレクトリに設定
3) db33.exeへのPATHを設定
4) DOSプロンプト上から次のコマンドでデバッガを起動

```
C:¥cc33¥sample¥dmt33004¥>db33 -mon -b 115200 -p 33104_m.par -c led.cmd
```

デバッガがデバッグモニタモードで起動し、"led.srf"のデバッグが行える状態になります。たとえば、gコマンドを実行するとS5U1C33104D1/S5U1C33209D1ボード上のLEDが点滅します。

デバッグモニタではキーブレークなどの強制ブレーク機能は使用できません。
このため、"led.cmd"では"led.srf"のNMIルーチン内のラベル位置にブレークポイントを設定しています。S5U1C330M1D1のSW2をONするとNMIが発生し、プログラムの実行を疑似的に強制ブレークすることができます。

(3) Flashメモリ上でのデバッグ

S5U1C33104D1/S5U1C33209D1のFlashメモリ(0x200000~)上でデバッグを行う場合のサンプルプログラムは"led2.srf"です。
Flashメモリにサンプルプログラムを書き込むには、まずFlash消去/書き込みルーチン"am29f800.srf"をロードします。さらに、fls、fleコマンドを実行してFlash関係の初期化とイレースを行い、lfコマンドでサンプルプログラムをロードします。コマンドの実行手順については、サンプルのデバッグコマンドファイル"led2.cmd"を参照してください。

デバッグコマンドファイル"led2.cmd"にはFlashイレース/書き込みルーチンのロード、トラップテーブルアドレスの設定、"led2.srf"のFlashメモリへの書き込みを行うデバッグコマンドが記述されていますので、デバッガ起動時に-cオプションで指定してください。

DOSプロンプトからデバッガを起動させる操作手順は次のとおりです。

1) 前述のとおりデバッグモニタを起動
2) "¥cc33¥sample¥dmt33004¥"(または"¥cc33¥sample¥dmt33005¥")をカレントディレクトリに設定
3) db33.exeへのPATHを設定
4) DOSプロンプト上から次のコマンドでデバッガを起動

```
C:¥cc33¥sample¥dmt33004¥>db33 -mon -b 115200 -p 33104_m.par -c led2.cmd
```

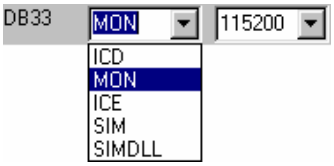
デバッガがデバッグモニタモードで起動し、"led2.srf"のデバッグが行える状態になります。たとえば、gコマンドを実行するとS5U1C33104D1/S5U1C33209D1ボード上のLEDが点滅します。

デバッグモニタではキーブレークなどの強制ブレーク機能は使用できません。
S5U1C330M1D1のSW2をONするとNMIが発生し、プログラムの実行を強制ブレークすることができます。

フラッシュメモリに書き込んだプログラムをS5U1C33104D1/S5U1C33209D1単体で実行させることもできます。
デバッガを終了後、システムの電源をOFFにしてRS232Cケーブルを切り離し、S5U1C330M1D1のSW3[DEBUG]をOFFにしてS5U1C33104D1/S5U1C33209D1の電源をONしてください。"led2.srf"がフラッシュメモリ上で実行され、LEDが点滅します。

(4) wb33からの実行

1) 前述のとおりデバッグモニタを起動
2) wb33を起動し、実行ウィンドウでパラメータファイル"¥cc33¥sample¥dmt33004¥33104_m.par"(または"¥cc33¥sample¥dmt33005¥33104_m.par")を選択
3) [Other options]ウィンドウのDB33オプションで、MON、115200bps、コマンドファイル"led.cmd"または"led2.cmd"を選択



4) [DB33]ボタンでデバッガを起動

以上の操作で上記の(2)、(3)と同様にデバッグを開始できます。

●注意事項

ユーザプログラムをS5U1C33104D1/S5U1C33209D1を使用してデバッグする場合は、以下の点に注意してください。

(1) S5U1C33104D1のデバッグモニタは、"mon33ch1.lib"をリンクして実装しています。したがって、ユーザプログラムから内蔵シリアルインタフェースCh.1を使用することはできません。
S5U1C33209D1のデバッグモニタは、"mon33ch0.lib"をリンクして実装しています。したがって、ユーザプログラムから内蔵シリアルインタフェースCh.0を使用することはできません。

(2) デバッグモニタではキーブレークなどの強制ブレーク機能は使用できません。
デバッグするプログラムのNMIやキー入力割り込み処理ルーチン内にラベルを設け、そこにハードウェアPCブレークを設定して対応してください。
また、その他のデバッグ機能も制限されます。デバッグモニタが対応している機能、コマンドについては、"16 デバッガ"を参照してください。

(3) RAMへのダウンロード速度は約8KB/S、フラッシュメモリへのダウンロード速度は約7KB/Sです。ただし、使用するPCや条件により変わります。

(4) デバッグするプログラムはS5U1C33104D1/S5U1C33209D1のRAMまたはFlashメモリの空き領域にロードして実行できるように作成してください。デバッガからはプログラムのロードアドレスを指定できませんので、リンク時にアドレスを決定しておく必要があります。
特に、内蔵RAMの0～0x2Fと外部SRAMの0x6FF640～0x6FFFFFはS5U1C330M2Sが使用しており、ここを書き換えると動作しなくなりますので注意してください。

また、メモリをメモリ操作コマンドで書き換える場合も、この領域が変更されないように注意してください。

CPU: S1C33A104(S5U1C33104D1)/S1C33209(S5U1C33209D1)

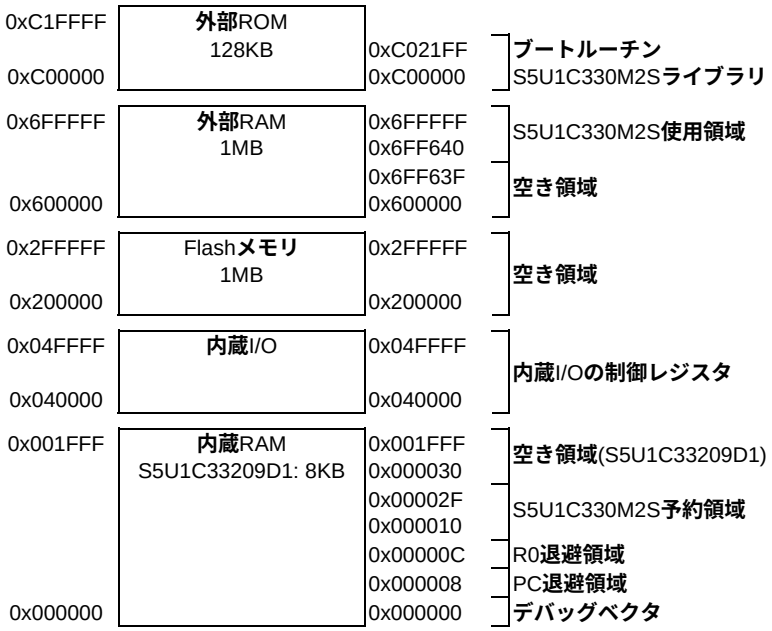


図3.3.2.3　S5U1C33104D1/S5U1C33209D1**メモリマップ**

(5) その他デバッグモニタの制限事項については、"S1C33 Family Debug Monitor Operation Manual"を参照してください。

3.3.3　インサーキットデバッガS5U1C33000H

インサーキットデバッガS5U1C33000Hはデバッガdb33のコマンドに従ってS1C33チップ上のデバッグ機能を制御するツールです。デバッグモニタと同等のデバッグ機能に加え、内蔵のトレースメモリによりトレース機能も提供します。ここでは、S5U1C33000HとS5U1C33209D1ボードを開発ツールとして使用してプログラムのデバッグを行う方法を説明します。

●システム構成と接続

図3.3.3.1にS5U1C33000HとS5U1C33209D1ボードを使用したデバッグシステムの構成を示します。

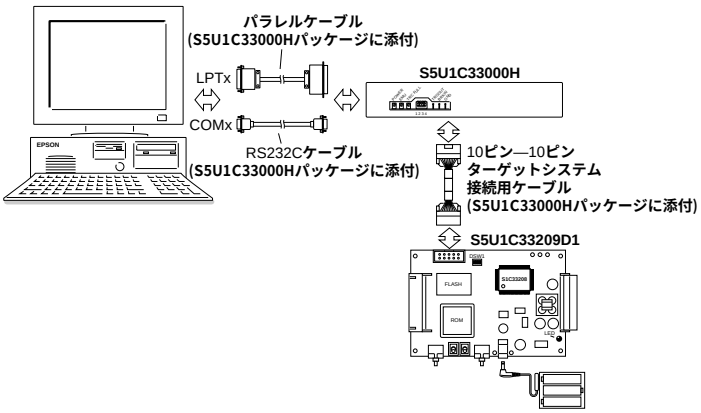


図3.3.3.1　S5U1C33000H**と**S5U1C33209D1**ボードによるデバッグシステム**

注:　S5U1C33000H**とデバッグモニタの併用はできません。**S5U1C33209D1**ボードに**S5U1C330M1D1**は接続しないでください。取り付けて使用する場合は、**S5U1C330M1D1**の**[DEBUG]**スイッチ**(SW3)**は必ず**OFF**にしてください。**

●起動と動作確認

以下のサンプルプログラムで動作確認が行えます。
"¥cc33¥sample¥dmt33005¥led.srf"
"¥cc33¥sample¥dmt33005¥led2.srf"
どちらもS5U1C33209D1ボード上のLEDを点滅させるサンプルプログラムです。"led.srf"はRAM(0x600000~)上で、"led2.srf"はFlashメモリ(0x200000~)上でデバッグ可能なように作成されています。
プログラムの内容については、ソースファイル(led.s)を参照してください。
サンプルプログラムは、実行形式のオブジェクトファイルが用意されていますので、ソースの修正が不要な場合はmakeを実行する必要はありません。
ソースを修正して使用する場合は、同じディレクトリに用意されているmakeファイルを使用してmakeを実行してください。

(1) システムの起動

S5U1C33000H、S5U1C33209D1、パーソナルコンピュータを接続後、次の手順でシステムを起動してください。

1) S5U1C33000Hのディップスイッチをすべて OPEN(上)
2) S5U1C33209D1の電源をON
3) S5U1C33000Hの電源をON
4) パーソナルコンピュータの電源をONし、Windowsを起動
5) デバッガdb33を起動(起動方法は後述)

(2) RAM上でのデバッグ

S5U1C33209D1のRAM(0x600000~)上でデバッグを行う場合のサンプルプログラムは"led.srf"です。トラップテーブルアドレスをRAMの先頭に設定し"led.srf"をRAMにロードするデバッグコマンドファイル"led.cmd"が用意されていますので、デバッガ起動時に-cオプションで指定してください。
DOSプロンプトからデバッガを起動させる操作手順は次のとおりです。

1) 前述のとおりシステムを起動
2) "¥cc33¥sample¥dmt33005¥"をカレントディレクトリに設定
3) db33.exeへのPATHを設定
4) DOSプロンプト上から次のコマンドでデバッガを起動

```
C:¥cc33¥sample¥dmt33005¥>db33 -icd -b 115200 -p 33104_m.par -c led.cmd
```

デバッガがICDモードで起動し、"led.srf"のデバッグが行える状態になります。たとえば、gコマンドを実行するとS5U1C33209D1ボード上のLEDが点滅します。

S5U1C33000Hではキーブレーク機能も使用可能です。デバッガの[key break]ボタンでプログラムの実行を中断することができます。また、トレース機能も使用可能です。トレース方法については"16デバッガ"を参照してください。

(3) Flashメモリ上でのデバッグ

S5U1C33209D1のFlashメモリ(0x200000~)上でデバッグを行う場合のサンプルプログラムは"led2.srf"です。
Flashメモリにサンプルプログラムを書き込むには、まずFlash消去/書き込みルーチン"am29f800.srf"をロードします。さらに、fls、fleコマンドを実行してFlash関係の初期化とイレースを行い、lfコマンドでサンプルプログラムをロードします。コマンドの実行手順については、サンプルのデバッグコマンドファイル"led2.cmd"を参照してください。

デバッグコマンドファイル"led2.cmd"にはFlashイレース/書き込みルーチンのロード、トラップテーブルアドレスの設定、"led2.srf"のFlashメモリへの書き込みを行うデバッグコマンドが記述されていますので、デバッガ起動時に-cオプションで指定してください。

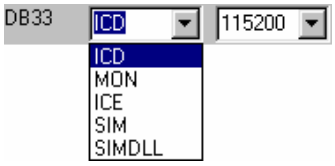
DOSプロンプトからデバッガを起動させる操作手順は次のとおりです。

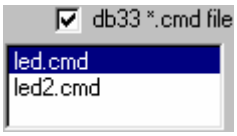
1) 前述のとおりシステムを起動
2) "¥cc33¥sample¥dmt33005¥"をカレントディレクトリに設定
3) db33.exeへのPATHを設定
4) DOSプロンプト上から次のコマンドでデバッガを起動

```
C:¥cc33¥sample¥dmt33005¥>db33 -icd -b 115200 -p 33104_m.par -c led2.cmd
```

(4) wb33からの実行

1) 前述のとおりシステムを起動
2) wb33を起動し、実行ウインドウでパラメータファイル"¥cc33¥sample¥dmt33005¥33104_m.par"を選択
3) [Other options]ウィンドウのDB33オプションで、ICD、115200bps、コマンドファイル"led.cmd"または"led2.cmd"を選択





4) [DB33]ボタンでデバッガを起動

以上の操作で上記の(2)、(3)と同様にデバッグを開始できます。

●注意事項

ユーザプログラムをS5U1C33000HとS5U1C33209D1を使用してデバッグする場合は、以下の点に注意してください。

(1) デバッグするプログラムはS5U1C33209D1のRAMまたはFlashメモリの空き領域にロードして実行できるように作成してください。デバッガからはプログラムのロードアドレスを指定できませんので、リンク時にアドレスを決定しておく必要があります。S5U1C33209D1のメモリマップについては図3.3.2.3を参照してください。

(2) S5U1C33104Hに比べ、デバッグ機能は制限されます。S5U1C33000Hが対応している機能、コマンドについては、"16 デバッガ"を参照してください。

(3) S5U1C33000HをRS232CケーブルでCOMポートに接続するのみで、S5U1C33000Hの全機能を使用することができます。LPTポートへは、ファイルの高速ダウンロードが必要な場合にパラレルケーブルで接続してください。それぞれのポートによる標準的なダウンロード速度は以下のとおりです(使用するPCや条件により変わります)。

シリアル転送時:	RAMへのダウンロード	8KB/S
	フラッシュメモリへのダウンロード	7KB/S
パラレル転送時:	RAMへのダウンロード	30KB/S
	フラッシュメモリへのダウンロード	20KB/S

(4) その他S5U1C33000Hの制限事項については、"S1C33 Family In-Circuit Debugger Manual"を参照してください。

3.3.4 メモリボードS5U1C33001M1

メモリボードS5U1C33001M1には、S5U1C33000Hのデバッグ機能を強化する拡張機能やエミュレーション機能が実装されており、より充実した開発環境を構成することが可能となります。ここでは、S5U1C33000H、S5U1C33208E3およびS5U1C33001M1を開発ツールとして使用する場合のデバッグ方法について説明します。

●システム構成と接続

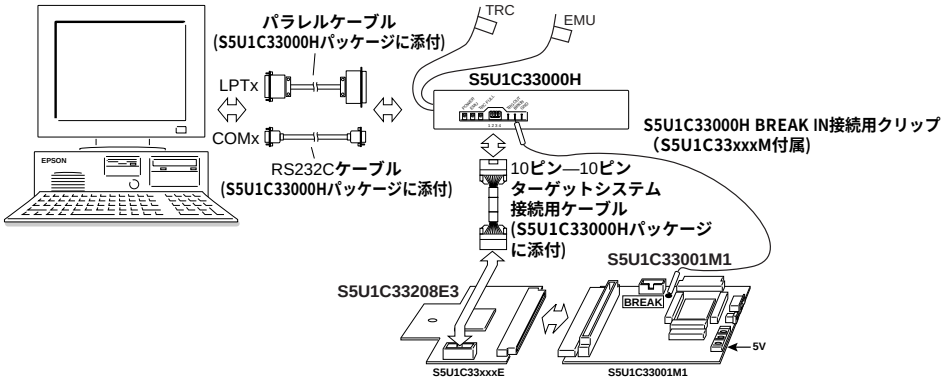


図3.3.4.1 S5U1C33000H、S5U1C33208E3、S5U1C33001M1**によるデバッグシステム**

デフォルトの設定に対して以下の内容を変更してください。

S5U1C33208E3

JP3の1-2をショート #RESET信号をS5U1C33001M1から供給する

S5U1C33001M1

JP1の1-2をショート #RESET信号をS5U1C33xxxEに供給する

JP7の1-2をショート V_{DDE}をS5U1C33xxxEに供給する

JP8の1-2をショート V_{DD}(3.3V)をS5U1C33xxxEに供給する

S5U1C33000HのディップSWはすべて上側(デフォルト)にセットしてください。

S5U1C33000Hは必ずVer.2以降をお使いください。(TRCとEMUの信号線が黄色と赤色の線で引き出されているもの)

●起動と動作確認

次のサンプルファイルで動作確認が行えます。

"¥cc33¥sample¥mem33201¥demo.bat"

プログラムは、内蔵RAMで実行され外部メモリをアクセスします。そのアクセスをS5U1C33001M1が検出し、ブレークします。プログラムの詳細は、"sample.c"を参照してください。

ソースを修正して使用する場合は、同じディレクトリにあるmakeファイルを使用してmakeを実行してください。

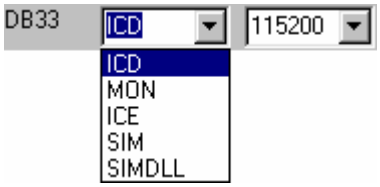
(1) システムの起動

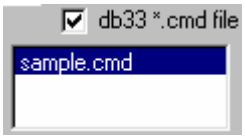
S5U1C33000H、S5U1C33208E3、S5U1C33001M1、パーソナルコンピュータを接続後、次の手順でシステムを起動してください。

1) S5U1C33001M1の電源をON
2) S5U1C33001M1のSW1(リセットSW)を押す
3) S5U1C33000Hの電源をON(S5U1C33000HのEMUランプ(赤)が点灯します。)
4) S5U1C33001M1のSW1(リセットSW)を放す(S5U1C33000HのEMUランプ(赤)が消えれば正常です。)
5) パーソナルコンピュータの電源をONし、Windowsを起動
6) MS-DOSプロンプトウィンドウを開く
7) ディレクトリを"¥cc33¥sample¥mem33201"に移動する
8) "demo.bat"を実行する

(2) wb33を用いたシステムの起動

1) 前述のとおりシステムを起動
2) wb33を起動し、実行ウィンドウでパラメータファイル"¥cc33¥sample¥mem33201¥33208_1.par"を選択
3) [Other options]ウィンドウのDB33オプションで、ICD、115200bps、コマンドファイル"sample.cmd"を選択





4) [DB33]ボタンでデバッガを起動

(3) 動作確認

1) db33は、"sample.cmd"ファイルを実行し、エリアブレーク、バスブレーク等を設定します。db33の右上にICD+MEM33と黄色く表示されたら、正しく動作しています。
2) ここでプロンプトからgコマンドを入力してください。バスブレークによりターゲットが停止します。
3) 再度gコマンドを入力してください。エリアブレークでターゲットが停止します。
4) 再度gコマンドを入力してください。マップブレークでターゲットが停止します。
5) 再度gコマンドを入力してください。マップブレークとCEブレークでターゲットが停止します。
6) 以下に、この操作によるコマンドウィンドウの表示内容を示します。

```
>g
Break by external break.
Break by MEM33 bus break. No.1
>g
Break by external break.
Break by MEM33 area break. Area1
>g
Break by external break.
Break by MEM33 map break.
>g
Break by external break.
Break by MEM33 map break.
Break by MEM33 ce break. CE5(15,15+16)
>
```

S5U1C33001M1は、S5U1C33000Hの外部ブレーク機能を使用していますので、外部ブレークも同時に表示されます。

7) exitコマンドでデバッガを終了してください。

●注意事項

(1) S5U1C33001M1を使用する場合、チップセレクト信号を1本占有します。占有しているエリアは、ブレーク機能が動作しないので注意してください。

(2) S5U1C33001M1に対応したコマンドの詳細は、"16 デバッガ"を参照してください。

(3) db33はパラメータファイルの内容に従ってS5U1C33001M1の設定を行います。ターゲットシステムに合わせ正しく記述してください。

(4) S5U1C33208E3、S5U1C33001M1の制限事項は、それぞれのマニュアルを参照してください。

3.4 プログラムの構造と各種メモリとの対応

ここでは、ソースファイルの作成およびリンクの際に必要となるセクション管理の概念を簡単に説明しておきます。Cソース内では特にセクションを意識することはありませんが、アセンブリソースではセクションを明示的に定義しておく必要がありますので覚えておいてください。

ソースファイルには、CPUや周辺回路を制御するプログラムの他に、フォントデータなどの初期化が不要な常時固定のデータ、RAMに置かれる変数のシンボルやI/Oメモリの制御レジスタなども記述されます。最終的にはこれらの異なる属性を持つデータやシンボルを、プログラムはプログラムROM領域に、固定データはデータROM領域にといった具合に、リンカによってそれぞれに対応した物理メモリに再配置する必要があります。このために、オブジェクトコードを属性ごとにセクションとして分類できるようにしています。
セクションの種類としては次の3つがあります。

1. CODE**セクション**　プログラムおよび初期値を持つ固定データのブロック
2. DATA**セクション**　初期値を持ち、R/W可能なデータのブロック
3. BSS**セクション**　RAMなどに割り当てるブロック

●アセンブリソースの場合

アセンブリソースの場合、以下のアセンブラ擬似命令を使用してセクションを指定します。

.code擬似命令	CODEセクションの開始
.data擬似命令	DATAセクションの開始
.comm/.lcomm擬似命令	BSSセクションへのシンボル定義と領域確保

指定方法は次のとおりです。(詳細は11章参照)

- プログラムおよびROMに書き込む固定データを記述する前に.code擬似命令でCODEセクションの開始を宣言します。
  それ以降のソースコードはCODEセクションのオブジェクトとしてアセンブルされます。
  アセンブラはセクションの定義が行われない場合、ファイルの先頭からCODEセクションとみなします。
- 初期値を持つRAM上のデータを設定する前に.data擬似命令でDATAセクションの開始を宣言します。それ以降のソースコードはDATAセクションのオブジェクトとしてアセンブルされます。ただし、DATAセクションの実データはROM上に生成されますので、プログラムによってRAM上にコピーすることが必要です。
- RAM上に変数やワーク領域を確保して、そのアドレスをシンボルで参照する場合、.commまたは.lcomm擬似命令で領域確保とシンボル定義を行います。
  アセンブラはBSSセクション内に指定領域を確保します。この領域はRAMやI/Oメモリが対象で、オブジェクトコードは作成されません。複数のモジュールからその領域を参照可能とするシンボル情報がBSSセクションとして作成されます。

リロケータブルアセンブリソース(Cソースをコンパイルしたものも含む)の場合、同じ属性のセクションは連続した1つのセクションとしてまとめられます。したがって、アセンブルされたモジュールはCODEセクション、DATAセクション、BSSセクションを1つずつ持つオブジェクトとなります(宣言が行われていないセクションも実データなしとして作成されます)。
絶対アドレスを指定したアセンブリソースでは、同一セクションを1つにまとめることができません。したがって、ソース内で分割して指定した数のセクションが作成されます。

●Cソースの場合

Cソースでは、これらの宣言をCコンパイラが行いますので、ソース中で指定する必要はありません。コンパイル後の命令はすべてCODEセクションに配置されます。データはその属性により次のように各セクションに対応します。

初期値を持たない変数(例: int i;):	BSSセクション
初期値を持つ変数(例: int i=1;):	DATAセクション
定数(例: const int i=1;):	CODEセクション

簡単な例として、チュートリアルで使用したサンプルプログラムのセクション定義を見てみましょう。
アセンブリソースファイルboot.sはプログラムコードのみのため、.code擬似命令のみが使用されています。

<boot.s>

```
; boot.s  1997.2.13
; boot program

#define SP_INI 0x0800; sp is in end of 2KB internal RAM
#define GP_INI 0x0000; global pointer %r8 is 0x0

          .code
          .word BOOT          ; BOOT VECTOR
BOOT:
          xld.w    %r8,SP_INI
          ld.w     %sp,%r8     ; set SP
          ld.w     %r8,GP_INI  ; set global pointer
          xcall    main        ; goto main
          xjp      BOOT        ; infinity loop
```

(プログラム説明)

スタックポインタとグローバルポインタを初期化し、main関数をコールするブート処理を行っています。実際のアプリケーションではトラップ処理ベクタの設定等が必要になりますので、このプログラムをアプリケーションには使用しないでください。

<main.c>

```
/* main.c 1997.2.13 */
/* C main program */

int i;

main()
    {
          int j;

          i = 0;
          for (j=0 ; ; j++)
              {
                    sub(j);
              }
    }

sub(k)
    int k;
    {
          if (k & 0x1)
              {
                    i++;
              }
    }
```

(プログラム説明)

main()関数はグローバル変数iをカウンタとして用いるために0にクリアします。その後はローカル変数jにより永久ループを作成し、jを引数としてsub()関数のコールを繰り返します。
sub()関数は2回の呼び出しごとにグローバル変数iをインクリメントします。

Cコンパイラの出力"main.ps"を見ると、コンパイラがCODEセクションとBSSセクションを定義しているのが分かります。

<main.ps>

```
        .file     "main.c"

;  GNU C 2.7.2 [AL 1.1, MM 40] RISC NEWS-OS compiled by CC

;  Cc1 defaults:
;  -mmemcpy

;  Cc1 arguments (-G value = 0, Cpu = 3000, ISA = 1):
;  -quiet -dumpbase -g -O -o

gcc2_compiled.:
__gnu_compiled_c:
        .code
        .align    1
        .def      main,     val       main,     scl       2,        type      0x24,     endef
        .global   main
                  :
                  :
        .comm     i 4

        .endfile
```

BSSセクションには".comm"擬似命令によりグローバル変数iのシンボル定義と4バイトの領域が確保されます。変数jやkはローカル変数のため、汎用レジスタやスタックに割り当てられ、BSSセクションは使用されません。

チュートリアルのmakeによりこれらのモジュールをリンクすると、それぞれのCODEセクションが一つにまとめられます。

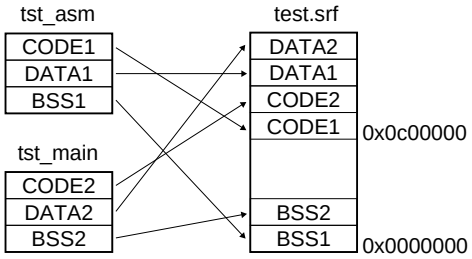


図3.4.1　**リンク後の配置**

リンク時は、各ファイルおよび各セクションを実際のメモリ構成に対応させるようにアドレスの指定が行えます。詳細は"12 リンカ"を参照してください。

lk33が生成するリンクマップファイルや、リンク後の"test.srf"からdis33で作成したディスアセンブリリストを見ると、リンク後の配置がよく分かります。

リンクマップファイル<test.map>

```
Code Section mapping
Address     Vaddress    Size        File                        ID  Attr
00C00000    --------    00000014    boot.o                      0   REL
00C00014    --------    00000034    main.o                      0   REL

Data Section mapping
Address     Vaddress    Size        File                        ID  Attr
00C00048    --------    00000000    boot.o                      1   REL
00C00048    --------    00000000    main.o                      1   REL

Bss Section mapping
Address     Vaddress    Size        File                        ID  Attr
00000000    --------    00000000    boot.o                      2   REL
00000000    --------    00000004    main.o                      2   REL
```

各ファイル(File)のセクションと配置アドレス(Address)の対応を示しています。

ディスアセンブリリストファイル<test.dis>

```
 **** Disassemble code and source code ****
   Addr   Code  Unassemble                Line        Source
 00C00000 0004 ***
 00C00002 0008 ***
                                              --- boot.s ---
                                        00001  ; boot.s  1997.2.13
                                        00002  ; boot program
                                        00003
                                        00004  #define SP_INI 0x0800          ; sp is in end of 2KB internal RAM
                                        00005  #define GP_INI 0x0000          ; global pointer %r8 is 0x0
                                        00006
                                        00007          .code
                                        00008          .word BOOT             ; BOOT VECTOR
                                        00009  BOOT:
 00C00004 C020 ext     0x20             00010          xld.w      %r8,SP_INI
 00C00006 6C08 ld.w    %r8,0x0
 00C00008 A081 ld.w    %sp,%r8          00011          ld.w       %sp,%r8    ; set SP
 00C0000A 6C08 ld.w    %r8,0x0          00012          ld.w       %r8,GP_INI ; set global pointer
 00C0000C C000 ext     0x0              00013          xcall      main       ; goto main
 00C0000E C000 ext     0x0
 00C00010 1C02 call    0x2
 00C00012 1EF9 jp      0xf9             00014          xjp        BOOT       ; infinity loop
                                              --- main.c ---
                                        00001  /* tst_main.c 1997.2.13 */
                                        00002  /* C main program */
                                        00003
                                        00004  int i;
                                        00005
                                        00006  main()
 00C00014 0200 pushn   %r0              00007      {
                                        00008          int j;
                                        00009
 00C00016 6C0B ld.w    %r11,0x0         00010          i = 0;
 00C00018 C000 ext     0x0
 00C0001A C000 ext     0x0
 00C0001C 6C09 ld.w    %r9,0x0
 00C0001E 3C9B ld.w    [%r9],%r11
 00C00020 6C00 ld.w    %r0,0x0          00011          for (j=0 ; ; j++)
                                        00012              {
 00C00022 2E0C ld.w    %r12,%r0         00013                     sub(j);
 00C00024 1C05 call    0x5
 00C00026 6010 add     %r0,0x1          00011          for (j=0 ; ; j++)
 00C00028 1EFD jp      0xfd
                                        00014              }
 00C0002A 0240 popn    %r0              00015      }
 00C0002C 0640 ret
                                        00016
                                        00017  sub(k)
                                        00018      int k;
                                        00019      {
 00C0002E 701C and     %r12,0x1         00020          if (k & 0x1)
 00C00030 180A jreq    0xa
                                        00021              {
```

```
00C00032 C000 ext     0x0                00022                    i++;
00C00034 C000 ext     0x0
00C00036 6C09 ld.w    %r9,0x0
00C00038 309A ld.w    %r10,[%r9]
00C0003A 601A add     %r10,0x1
00C0003C C000 ext     0x0
00C0003E C000 ext     0x0
00C00040 6C09 ld.w    %r9,0x0
00C00042 3C9A ld.w    [%r9],%r10
                                         00023              }
00C00044 0640 ret                        00024      }
00C00046 0000 nop
```

ここでは、セクションの概要のみを説明しました。詳細についてはアセンブラおよびリンカの章を参照してください。

4 ソースファイル

この章ではソースファイルを作成する際の規則、文法について解説します。

4.1 ファイル形式とファイル名

ソースファイルは汎用のエディタ、あるいはワープロソフトで作成してください。

※ファイル形式　標準のテキストファイルとしてセーブしてください。

※ファイル名　Cソースファイル　＜ファイル名＞.c
アセンブリソースファイル　＜ファイル名＞.s
＜ファイル名＞は、32文字以内で、以下の英数記号で指定します。
a～z、A～Z、0～9、_(アンダースコア)
なお、S1C33ツールのファイル名には、すべてこの規定が適用されます。
Cソースファイルの拡張子は".c"(小文字のみ)としてください。他の拡張子ではCコンパイラgcc33に入力することができません。

※ディレクトリ名　ディレクトリ名もファイル名と同じ英数記号のみが使用可能です。スペースや漢字等は使用しないでください。ディレクトリ名を含むファイル名は64文字以下としてください。

※行数　1つのCソースファイルおよびアセンブリソースファイルに許可されている行数は最大30,000行です。

※タブ設定　タブストップはできるだけ8文字ごとに設定してください。8文字以外のタブ設定をしたソースをディスアセンブラdis33でミックス処理、あるいはデバッガdb33でソース表示/ミックス表示すると、ソース部分がずれて出力される場合があります。

※EOF　各ステートメントは必ず改行し、EOFは改行後に付くように(ファイルの最後に独立するように)してください。

4.2 Cソースの文法

本パッケージのCコンパイラgcc33はANSI C準拠のGNU Cコンパイラ(ver. 2.7.2)です。
asm関数を除き標準の仕様となっていますので、CソースはANSI Cの規定に従って作成してください。
文法について不明な点がある場合は、ANSI Cを解説している一般の書籍の参照をお願いいたします。

4.2.1 データ型

gcc33はANSI Cのすべてのデータ型に対応しています。各型のサイズ(バイト数)、表現できる数値の有効範囲を表4.2.1.1に示します。

表4.2.1.1 型の種類とサイズ

型	サイズ	数値の有効範囲
char	1	-128〜127
unsigned char	1	0〜255
short	2	-32768〜32767
unsigned short	2	0〜65535
int	4	-2147483648〜2147483647
unsigned int	4	0〜4294967295
long	4	-2147483648〜2147483647
unsigned long	4	0〜4294967295
pointer	4	0〜4294967295
float	4	1.175e-38〜3.403e+38 (正規化数)
double	8	2.225e-308〜1.798e+308 (正規化数)

float型、double型はIEEE標準規格のフォーマットに準拠しています。

4.2.2 ライブラリ関数とヘッダファイル

本パッケージには、ANSI標準ライブラリと浮動小数点/整数剰余演算用のエミュレーションライブラリが用意されています。

また、"include"ディレクトリ内のヘッダファイルには、ライブラリの関数宣言、マクロなどが定義されています。ライブラリ関数を使用する場合は、その宣言を含んだヘッダファイルを"#include"命令でインクルードしてください。

ライブラリファイルの種類およびヘッダファイルとの対応は次のとおりです。

表4.2.2.1 ライブラリファイル/関数一覧

ANSI標準ライブラリ

ファイル名	関数/マクロ	対応ヘッダファイル
io.lib	tmpfile*, tmpnam*, remove*, fopen*, freopen, fclose*, setbuf*, setvbuf*, fflush*, clearerr*, feof*, ferror*, perror, fseek*, fgetpos*, fsetpos*, ftell*, rewind*, getchar, fgetc, getc, gets, fgets, fscanf, scanf, sscanf, fread, putchar, fputc, putc, puts, fputs, ungetc,fprintf, printf, sprintf, vfprintf, vprintf, vsprintf, fwrite	stdio.h
	abort, exit, atexit*,getenv*, system*, malloc, calloc, realloc, free, atoi, atol, atof, strtol, strtoul, strtod, abs, labs, div, ldiv, rand, srand, bsearch, qsort	stdlib.h
	time, difftime*, clock*, mktime, localtime*, gmtime, asctime*, ctime*	time.h
math.lib	acos, asin, atan, atan2, ceil, cos, cosh, exp, fabs, floor, fmod, frexp, ldexp, log, log10, modf, pow, sin, sinh, sqrt, tan, tanh	math.h, errno.h, float.h, limits.h
string.lib	memchr, memmove, strchr, strcspn, strncat, strpbrk, strstr, memcmp, memset, strcmp, strerror, strncmp, strrchr, strtok, memcpy, strcat, strcpy, strlen, strncpy, strspn	string.h
ctype.lib	isalnum, iscntrl, isgraph, isprint, isspace, isxdigit, toupper, isalpha, isdigit, islower, ispunct, isupper, tolower	ctype.h
–	va_start, va_arg, va_end	stdarg.h

*の付いた関数はダミー関数です。

エミュレーションライブラリ

ファイル名	関数
fp.lib	adddf3, subdf3, muldf3, divds3, negdf2, addds3, subds3, mulds3, divds3, negds2, fixunsdfsi, fixdfsi, floatsidf, fixunssfsi, fixsfsi, floatsisf, truncdfsf2, extendsfdf2, fcmpd, fcmps
idiv.lib	divsi3, udivsi3, modsi3, umodsi3

ライブラリに含まれる関数の機能については"7 エミュレーションライブラリ"および"8 ANSIライブラリ"を参照してください。
また、ライブラリ関数を使用した場合は、リンク時にその関数を含むライブラリファイルをリンカコマンドで指定してください。リンカは指定されたライブラリファイルの中から必要なオブジェクトモジュールのみを取り出してリンクします。

4.2.3 インラインアセンブル

gcc33はインラインアセンブルに対応しており、asm文が使用可能です。これに伴い、"asm"は予約語となりますので注意してください。

形式: **asm("文字列");**

例1:
```
/* HALT mode */
        asm("halt");
```

例2:
```
/* Trap Table*/
        asm(".word   BOOT
             .space  8
             .word   ZERO_DIV
             .space  4
             .word   ADDR_ERR
             .word   NMI
             .space  32
             .word   INT1
             .word   INT2");
```

例3:
```
BOOT(){
        asm("xld.w   %r8,0x0800");
        asm("ld.w    %sp,%r8");     /* set SP */
        asm("ld.w    %r8,0x0000");  /* set global pointer */
             :
    }
```

アセンブリソースの記述方法の詳細については"4.3 アセンブリソースの文法"を参照してください。
なお、アセンブリソースには命令エクステンダext33で処理可能な拡張命令、アセンブラ擬似命令(アブソリュートアセンブル用の命令を除く)は使用できますが、プリプロセッサpp33が提供する機能は使用できませんので注意してください。

4.3 アセンブリソースの文法

4.3.1 ステートメント

アセンブリソースの個々の命令や定義をステートメントと呼びます。基本的なステートメントの構成は次のとおりです。

構文パターン

1	**ニーモニック**	**(オペランド)**	**(;コメント)**
2	**アセンブラ擬似命令**	**(パラメータ)**	**(;コメント)**
3	**ラベル:**		**(;コメント)**
4	**;コメント**		
5	**拡張命令**	**オペランド**	**(;コメント)**
6	**プリプロセッサ擬似命令**	**(パラメータ)**	**(;コメント)**

例: ステートメント 構文パターン

```
; boot.s  1997.2.13                                          ..4
; boot program                                               ..4

#define SP_INI 0x0800          ; sp is in end of 2KB internal RAM  ..6
#define GP_INI 0x0000          ; global pointer %r8 is 0x0         ..6

        .code                                                  ..2
        .word BOOT             ; BOOT VECTOR                       ..2
BOOT:                                                          ..3
        xld.w   %r8,SP_INI                                     ..5
        ld.w    %sp,%r8        ; set SP                            ..1
        ld.w    %r8,GP_INI     ; set global pointer                ..1
        xcall   main           ; goto main                         ..5
        xjp     BOOT           ; infinity loop                     ..5
  :       :        :              :
```

上記の例が一般的なソース記述方法です。視認性を高めるため、各ステートメントを構成する要素はタブやスペースで位置を揃えています。

※ 5は命令エクステンダext33、6はプリプロセッサpp33の機能で、アセンブラas33で処理可能なステートメントではありません。

●制限事項

- 各行には1つのステートメントのみ記述できます。1行に2つ以上の命令を記述、あるいはラベルと命令が混在する記述はエラーとなります。ただし、コメントは命令やラベルの行にも記述することができます。

  例: ;OK
  ```
  BOOT:
          ld.w    %r0,%r1
  ;Error
  BOOT:   ld.w    %r0,%r1
  ```

- 1つのステートメントを複数行に分けて記述することはできません。1行で完結していないステートメントはエラーとなります。

  例: ;OK
  ```
          .byte   0x0,0x1,0x2,0x3
          .byte   0xa,0xb,0xc,0xd
  ;Error
          .byte   0x0,0x1,0x2,0x3,
                  0xa,0xb,0xc,0xd
  ```

- 1行に記述可能な文字数はスペースも含めて最大100文字(ASCIIキャラクタ)です。
- コメントを除き、使用可能な文字はASCIIキャラクタ(英数記号)に限られます。また、使用できる記号にも制限があります(詳細は後述)。コメントにはASCIIキャラクタ以外の漢字なども使用できます。

(1) 命令(ニーモニック&オペランド)

CPUに対する命令は「ニーモニック」＋「オペランド」の構成となります。命令によっては、オペランドを持たないものもあります。

●命令表記の一般形

一般形: **<ニーモニック>**
<ニーモニック> タブまたはスペース <オペランド>
<ニーモニック> タブまたはスペース <オペランド1>，<オペランド2>

例: nop
```
call    SUB1
ld.w    %r0,0x4
```

行の中で、ニーモニックの記述開始位置に制限はありません。ニーモニックの前にあるタブやスペースは無視されます。一般的にはタブによってニーモニックの頭揃えを行います。

オペランドを持つ命令の場合、ニーモニックとオペランド間は1個以上のタブまたはスペースで区切る必要があります。また、オペランドが複数の場合は、オペランド間を1個のカンマ(,)で区切ります。オペランド間のスペースは無視されます。

オペランドの要素については後述します。

●ニーモニックの種類

S1C33 Familyで使用できるニーモニックは次の61種類です。

adc add and bclr bnot brk bset btst call cmp div0s div0u div1 div2s div3s ext halt int jp jreq/jreq.d jrge/jrge.d jrgt/jrgt.d jrle/jrle.d jrlt/jrlt.d jrne/jrne.d jruge/jruge.d jrugt/jrugt.d jrule/jrule.d jrult/jrult.d ld.b ld.h ld.ub ld.uh ld.w mac mirror mlt.h mlt.w mltu.h mltu.w nop not or popn pushn ret/ret.d retd reti rl rr sbc scan0 scan1 sla sll slp sra srl sub swap xor

各命令の詳細については"S1C33000コアCPUマニュアル"を参照してください。

●文字の制限

ニーモニックは大文字(A～Z)、小文字(a～z)のどちらで記述しても受け付けられます。
たとえば、"ld.w"、"LD.W"、"Ld.w"はすべて"ld.w"命令として受け付けられます。
シンボル等と区別するため、本マニュアルでは小文字を使用します。
オペランドについては後述します。

(2) アセンブラ擬似命令

アセンブラas33には、次に示す20種類の擬似命令が用意されています。

擬似命令		機能
.abs		アブソリュートアセンブル指定
.org	<address>	コード内の絶対アドレスを設定(アブソリュート専用)
.code		CODEセクションを宣言
.data		DATAセクションを宣言
.word	<data 1>[,<data 2>..,<data n>]	CODE/DATAセクション内にワードデータを定義
.half	<data 1>[,<data 2>..,<data n>]	CODE/DATAセクション内にハーフワードデータを定義
.byte	<data 1>[,<data 2>..,<data n>]	CODE/DATAセクション内にバイトデータを定義
.ascii	<string>	CODE/DATAセクション内にASCII文字列を定義
.space	<length>	CODE/DATAセクション内に空白領域(0x0)を定義
.align	<value>	指定境界アドレスにアライメント
.comm	<symbol>,<length>	BSSセクション内にグローバル領域を確保
.lcom	<symbol>,<length>	BSSセクション内にローカル領域を確保
.global	<symbol>	シンボルのグローバル宣言
.set	<symbol>,<address>	シンボルに絶対アドレスを定義(アブソリュート専用)
.list		アセンブリリスト出力を制御
.nolist		アセンブリリスト出力を制御
.file	<strings>	デバッグ情報
.endfile		デバッグ情報
.loc	<value>	デバッグ情報
.def	<symbol>, ... ,endef	デバッグ情報

いずれの命令もピリオド(.)で始まります。

例: .data
```
    .align    2
    .word     1,2,3,4
```

各擬似命令表記方法、機能の詳細は"11.8 アセンブラ擬似命令"を参照してください。

(3) ラベル

ラベルはプログラム中の任意のアドレスを参照するための識別子です。プログラムの分岐先アドレスやCODE/DATAセクション内の任意のアドレスをラベルとして定義したシンボルを使って参照することができます。

●ラベルの定義

次の形式で記述したシンボルがラベルとみなされます。

<シンボル>：

先頭のスペースやタブは無視されます。一般的には行の先頭から記述します。
定義されたシンボルは記述された位置のアドレスを示します。
実際のアドレス値はリンク時に決定します。

●制限事項

・ラベルはソースプログラムの1行を占有します。命令等を同じ行に記述するとエラーとなります。ただし、コメントは記述可能です。

・ラベルの文字数は最大128文字です(コロンは文字数に含みません)。

・使用できる文字は以下のものに限られます。
A～Z　a～z　_　0～9
ただし、数字で始まることはできません。大文字と小文字は区別されます。

```
例: ;OK                ;Error
    FOO:               1label:
    _Abcd:             0_ABC:
    L1:
```

(4) コメント

コメントは一連のルーチン、あるいは各ステートメントの意味を記述しておくもので、コード化の対象とはなりません。

●コメントの定義

セミコロン(;)で始まり、改行で終わる文字列がコメントとみなされます。
コメントにはASCIIキャラクタのほかに漢字などの非ASCIIキャラクタも使用可能です。
また、ラベルや命令と同じ行に記述することも可能です。

```
例: ; This line is a comment line.
    LABEL:                          ;This is the comment for LABEL.
                ld        %a,%b     ;This is the comment for the instruction on the left.
```

●制限事項

・コメントの長さはセミコロン(;)、コメント前後あるいは内部のスペース、復帰/改行コードを含めて100文字までに制限されます。

・コメントが複数行に渡る場合は、各行ごとにセミコロンで始める必要があります。

```
例: ;These are
        comment lines.    ...2行目はコメントとはみなされません。エラーとなります。

    ;These are
    ;   comment lines.    ...2行ともコメントとみなされます。
```

(5) 空白行

本アセンブラでは、復帰/改行コードのみの空白行も許されます。一連のルーチンの区切りなど、セミコロンでコメント行にする必要はありません。

4.3.2 オペランドの表記

ここでは命令のオペランドに使用するレジスタ名、シンボル、定数の表記法を説明します。

(1) レジスタ名

コアCPU S1C33000の内部レジスタ名には、すべてパーセント(%)を付けます。レジスタ名は大文字でも、小文字でも受け付けます(区別されません)。

汎用レジスタ(%rd, %rs, %rb)	**表記**
汎用レジスタR0～R15	%r0～%r15または%R0～%R15

特殊レジスタ(%sd, %ss)	**表記**
プロセッサステータスレジスタPSR	%psrまたは%PSR
スタックポインタSP	%spまたは%SP
算術演算ローレジスタALR	%alrまたは%ALR
算術演算ハイレジスタAHR	%ahrまたは%AHR

[ ]を伴うレジスタ間接アドレッシングの指定にも、レジスタ名に%が必要です。
例: [%r8]　[%r1]+　[%sp+imm6]

注: **レジスタ名の先頭に%がない場合、シンボルとみなされます。**
逆に%で始まるものはすべてレジスタとみなされ、存在しないレジスタ名を使用するとエラーとなります。

(2) 数値表記

本アセンブラは10進形式、16進形式、2進形式の3種類の数値表記に対応しています。

●10進形式の数値表記

0～9の数値のみで表記されたものは10進数とみなされます。負の数を指定する場合は、マイナス(-)符号を数値の前に付けてください。
例: 1　255　-3

0～9、および符号(-)以外の文字は使用できません。

●16進形式の数値表記

16進数を指定する場合は数値の前に"0x"を付けてください。
例: 0x1a　0xff00

"0x"の後ろには0～9、a～f、A～F以外の文字は使用できません。

注: "x"**は小文字のみ使用可能です。**"0X"**はエラーとなります。**

●2進形式の数値表記

2進数を指定する場合は数値の前に"0b"を付けてください。
例: 0b1001　0b01001100

"0b"の後ろには0と1以外の文字は使用できません。

注: "b"**は小文字のみ使用可能です。**"0B"**はエラーとなります。**

●数値の指定範囲

即値データのサイズ(指定範囲)は各命令により異なります。
各即値データの指定可能範囲は次のとおりです。

表4.3.2.1 即値データの種類と指定可能範囲

記号	種類	10 進数	16 進数	2 進数
imm2	**2 ビット 即値**	0~3	0x0~0x3	0b0~0b11
imm3	**3 ビット 即値**	0~7	0x0~0x7	0b0~0b111
imm4	**4 ビット 即値**	0~15	0x0~0xf	0b0~0b1111
imm6	**6 ビット 即値**	0~63	0x0~0x3f	0b0~0b111111
sign6	**符号付き 6 ビット 即値**	-32~31	0x0~0x3f	0b0~0b111111
imm8	**8 ビット 即値**	0~255	0x0~0xff	0b0~0b11111111
sign8	**符号付き 8 ビット 即値**	-128~127	0x0~0xff	0b0~0b11111111
imm10	**10 ビット 即値**	0~1023	0x0~0x3ff	0b0~0b1111111111
imm13	**13 ビット 即値**	0~8191	0x0~0x1fff	0b0~0b1111111111111
imm32	**32 ビット 即値**	0~4294967295	0x0~0xffffffff	0b0~ 0b11111111111111111111111111111111
sign32	**符号付き 32 ビット 即値**	-2147483648~ 2147483647	0x0~0xffffffff	0b0~ 0b11111111111111111111111111111111

(3) シンボル

即値データによるアドレス指定には、他の場所で定義されているシンボルを使用することができます。

注: ここで説明するシンボルはアセンブラで処理可能なアドレスを示すものです。デファイン名など、文字列を示すシンボルについてはプリプロセッサpp33の章で説明します。

●シンボル定義

使用可能なシンボルは次の3つのいずれかの方法によって32ビットの値として定義されます。

1. ラベルとして記述(CODEおよびDATAセクション内)
   例: LABEL1: ... LABEL1はCODEまたはDATAセクション内の記述された位置(アドレス)を示すシンボル
2. .commまたは.lcomm擬似命令で定義(BSSセクション内)
   例: .comm BUF1 4 ... BUF1はBSSセクション内の記述された位置(アドレス)を示すシンボル
3. .set擬似命令で定義(アブソリュートアセンブル専用のシンボル定義)
   例: .set ADDR1 0xff00 ... ADDR1は絶対アドレス0x0000ff00を示すシンボル

●文字の制限

・シンボルの文字数は最大128文字です。これを越えた場合、エラーとなります。

・使用できる文字は以下のものに限られます。
A～Z a～z _ 0～9
ただし、数字で始まることはできません。大文字と小文字は区別されます。

●ローカルシンボルとグローバルシンボル

通常、定義したシンボルはローカルシンボルとなり、そのファイル内でのみ参照可能です。このため、複数のファイルで同じ名称のシンボルを定義することもできます。他のファイルで定義されているシンボルを参照するには、シンボルを定義したファイル内で、.global擬似命令によるグローバル宣言が必要です。

※.comm擬似命令で定義したシンボルは、グローバル宣言されたものとして扱われます。.global擬似命令によるグローバル宣言は不要です。

●シンボルの拡張表記

通常、シンボルでアドレスを参照する場合、アドレスを指定するオペランドにそのシンボル名を記述します。

```
例: call    LABEL          ...LABEL = sign8
    ld.w    %rd, LABEL     ...LABEL = sign6
```

アセンブラでは、次のようなディスプレースメント付きの参照も許可します。

LABEL + imm32　LABEL + sign32

```
例: call    LABEL+0x10
```

●シンボルマスク

S1C33000命令セットの基本命令は、個々の命令のオペランドで指定可能な即値サイズが限られています。このため、そのサイズを越える値を持つシンボルを使用するとアセンブラでエラーとなってしまいます。基本命令を使用する場合はext命令に上位ビットを振り分けて記述する必要があり、これにはシンボルマスクを使用します。

シンボルマスクは、シンボルの値からext命令と基本命令に振り分ける値を取得するためのもので、シンボルの直後に記述します。

拡張命令を使用する場合は、命令エクステンダext33が必要なシンボルマスクを付けて展開しますので、特にext命令やシンボルマスクを意識することはありません。

<u>シンボルマスクの種類</u>

次に示す8種類のシンボルマスクが使用可能です。

シンボルマスク	機能
@rhまたは@RH	相対アドレスの上位10ビットを取得
@rmまたは@RM	相対アドレスの中間13ビットを取得
@rlまたは@RL	相対アドレスの下位8ビットを取得
@hまたは@H	絶対アドレスの上位13ビットを取得
@mまたは@M	絶対アドレスの中間13ビットを取得
@lまたは@L	絶対アドレスの下位6ビットを取得
@ahまたは@AH	相対アドレスの上位13ビットを取得
@alまたは@AL	相対アドレスの下位13ビットを取得

```
例:
ext     LABEL@rh
ext     LABEL@rm
call    LABEL@rl        ..."call    LABEL"として働きます。

ext     LABEL@h
ext     LABEL@m
ld.w    %rd, LABEL@l    ..."ld.w    %rd, LABEL"として働きます。

ext     LABEL@ah
ext     LABEL@al
ld.w    %rd, [%rb]      ..."ld.w    %rd, [%rb+LABEL]"として働きます。
```

注:
- **シンボルマスクは定義されたシンボルにのみ有効です。数値にシンボルマスクを使用した場合はエラーとなります。**
- **シンボルマスクを省略すると、その命令に有効な下位側のビットが使用されます。ただし、指定可能な範囲を超えるとエラーまたはワーニングとなります。**

4.3.3 拡張命令

命令エクステンダext33はアセンブリソースファイルの作成に拡張命令を提供します。
拡張命令は、通常ext命令を含む複数の命令で記述する内容を1つの命令として記述できるようにしたもので、命令エクステンダにより必要最小限の基本命令に展開されます。

●拡張命令の種類

xadd xsub xcmp xand xoor xxor xnot xsll xsrl xsla xsra xrl xrr
xld.b xld.ub xld.h xld.uh xld.w xbset xbclr xbtst xbnot
xjp xjreq xjrne xjrgt xjrge xjrlt xjrle xjrugt xjruge xjrult xjrule xcall
xjp.d xjreq.d xjrne.d xjrgt.d xjrge.d xjrlt.d xjrle.d xjrugt.d xjruge.d xjrult.d xjrule.d xcall.d

基本命令の先頭に"x"を付けると拡張命令となります(or命令のみはxoor)。

●拡張命令の使用方法

オペランドには即値拡張後のサイズの数値、シンボルを直接記述可能です。

```
例: xcall    LABEL              ; ext     LABEL@rh
                                ; ext     LABEL@rm
                                ; call    LABEL@rl

    xld.w    %r1, sign32        ; ext     sign32@h
                                ; ext     sign32@m
                                ; ld.w    %r1, sign32@l
```

また、基本命令の即値拡張機能以外にも、以下のようなオペランドの指定も命令により可能です。

```
例: xadd     %r0, %r1, 0x1      ; R0 ← R1 + 1
    xsub     %sp, %sp, %r1      ; SP ← SP + R1
    xld.w    %r0, [symbol + 0x10]  ; R0 ← [symbol + 0x10]
    xjp      LABEL + 5          ; LABEL+5のアドレスにジャンプ
    xrl      %r0, 15            ; R0の内容を15ビット左にローテート
```

オペランドを含む拡張命令の詳細については"10.6 拡張命令"を参照してください。

注: 拡張命令は命令エクステンダext33で処理する必要があります。直接、アセンブラas33に入力することはできません(エラーとなります)。

4.3.4 プリプロセッサによる付加機能

プリプロセッサpp33はアセンブリソースファイルの作成に付加機能を提供します。ここでは、それらの表記に関して説明します。各機能の詳細については、"9 プリプロセッサ"を参照してください。プリプロセッサpp33はそれらの表記部分をアセンブル可能なニーモニックのステートメントに展開し、アセンブリソースファイルを出力します。

注: 以下のステートメントはプリプロセッサpp33で処理する必要があります。直接、アセンブラas33に入力することはできません(エラーとなります)。

●プリプロセッサ擬似命令

プリプロセッサpp33には、次に示す5種類の擬似命令が用意されています。

#include	ファイルの挿入
#define	文字列の定義
#macro～#endm	マクロ定義
#ifdef(ifndef)～#else～#endif	条件アセンブル

いずれの命令もシャープ(#)で始まります。

```
例: #include   "define.h"
    #define    NULL      0
    #macro     ADDM      $1, $2
               xld.w     %r0, [$1]
               xld.w     %r1, [$2]
               add      %r0, %r1
               xld.w     [$1], %r0
    #endm
    #ifdef     TYPE1
               ld.w      %r0, 0
    #else
               ld.w      %r0, -1
    #endif
```

各擬似命令表記方法、機能の詳細は"9.5 プリプロセッサ擬似命令"を参照してください。

●数値演算子

ソース中の数値の指定には、以下の演算子を用いた式が使用できます。

		例
+	加算、単項プラス	+0xff, 1+2
-	減算、単項マイナス	-1+2, 0xfff-0b111
*	乗算	0xf*5
/	除算	0x123/0x56
%%	剰余	0x123%%0x56
>>	右シフト	1>>2
<<	左シフト	0x113<<3
&	論理積	0b1101&0b111
|	論理和	0x123|0xff
^	排他的論理和	12^35
~	論理否定	~0x1234
^H	ビットフィールド(31:19)の取得	0x1234^H
^M	ビットフィールド(18:6)の取得	0x1234^M
^L	ビットフィールド(5:0)の取得	0x1234^L
^AH	ビットフィールド(25:13)の取得	0x1234^AH
^AL	ビットフィールド(12:0)の取得	0x1234^L
( )	カッコ	1+(1+2*5)

数値部分には、プリプロセッサ擬似命令#defineで数値を定義したシンボルを使用することもできます。

4.4 ソース作成上の注意事項

(1) タブストップはできるだけ8文字ごとに設定してください。8文字以外のタブ設定をしたソースをディスアセンブラdis33でミックス処理、あるいはデバッガdb33でソース表示/ミックス表示すると、ソース部分がずれて出力される場合があります。

(2) コンパイラ、プリプロセッサでCソース、アセンブリソースをデバッグ情報付きで処理する場合には、他のソースファイルをインクルード(#include)しないでください。デバッガdb33が正しく動作しなくなります。ソースを含まない通常のヘッダファイルは問題ありません。

(3) アセンブリソースをアブソリュート形式で記述する場合、CODE、DATA、BSSセクションは、それぞれ1つ以下としてください。
実際には同一形式のセクションを複数記述することは可能ですが、アドレスを昇順に記述しないと誤動作する場合があります。その他の問題も含め、同一セクションを複数記述したアブソリュートアセンブリソースは動作保証されません。

(4) Cモジュールとアセンブラモジュールを混在して使用する場合、引数や戻り値のサイズ、受け渡し方法には十分注意してください。詳細については、6章のコンパイラの内容、特に"6.5.3 レジスタ使用法"と"6.5.4 関数呼び出し"を参照してください。

5 ワークベンチ

この章では、ワークベンチwb33の機能と操作方法を説明します。

5.1 機能

ワークベンチwb33(以下wb33と記述)は、Cコンパイラおよびプリプロセッサからデバッガまでの統合操作環境を実現するソフトウェアです。機能と特長を次にまとめます。

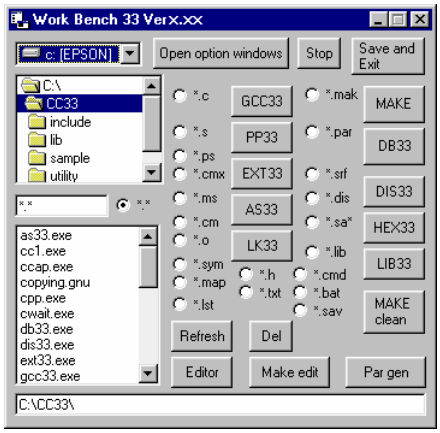
- S1C33 Familyのプログラム開発に必要なソフトウェアツールを1つのウィンドウ上から同一感覚の操作で起動できます。
- 基本的なmakeファイルやデバッガ用のパラメータファイルが、エディタ等を使用せずに簡単に作成できます。
- ほとんどの操作はすべてマウスのみで行え、各ツールの標準的な起動時オプションもチェックボックスで選択可能です。
- ソースファイルの選択時に、その内容を表示可能です(32KBまで)。さらに、選択したソースファイルを指定のエディタで開くことができ、ソースの修正等も効率よく行えます。
- DOSコマンドの実行を含め、キーボードからのコマンドライン入力もwb33上で行えます。

5.2 操作方法

5.2.1 起動および終了方法

●wb33を起動するには

"wb33.exe"のアイコンをダブルクリックし、ワークベンチwb33を起動させてください。
オプションファイル(.sav)を"wb33.exe"アイコン上にドラッグ&ドロップすることによってもwb33は起動します。
wb33が起動すると次の実行ウィンドウが表示されます。



●wb33の起動コマンド

DOSプロンプトからwb33を起動する場合のコマンドは次のとおりです。

<u>起動コマンド</u>
>wb33 [<オプションファイル名>]↵

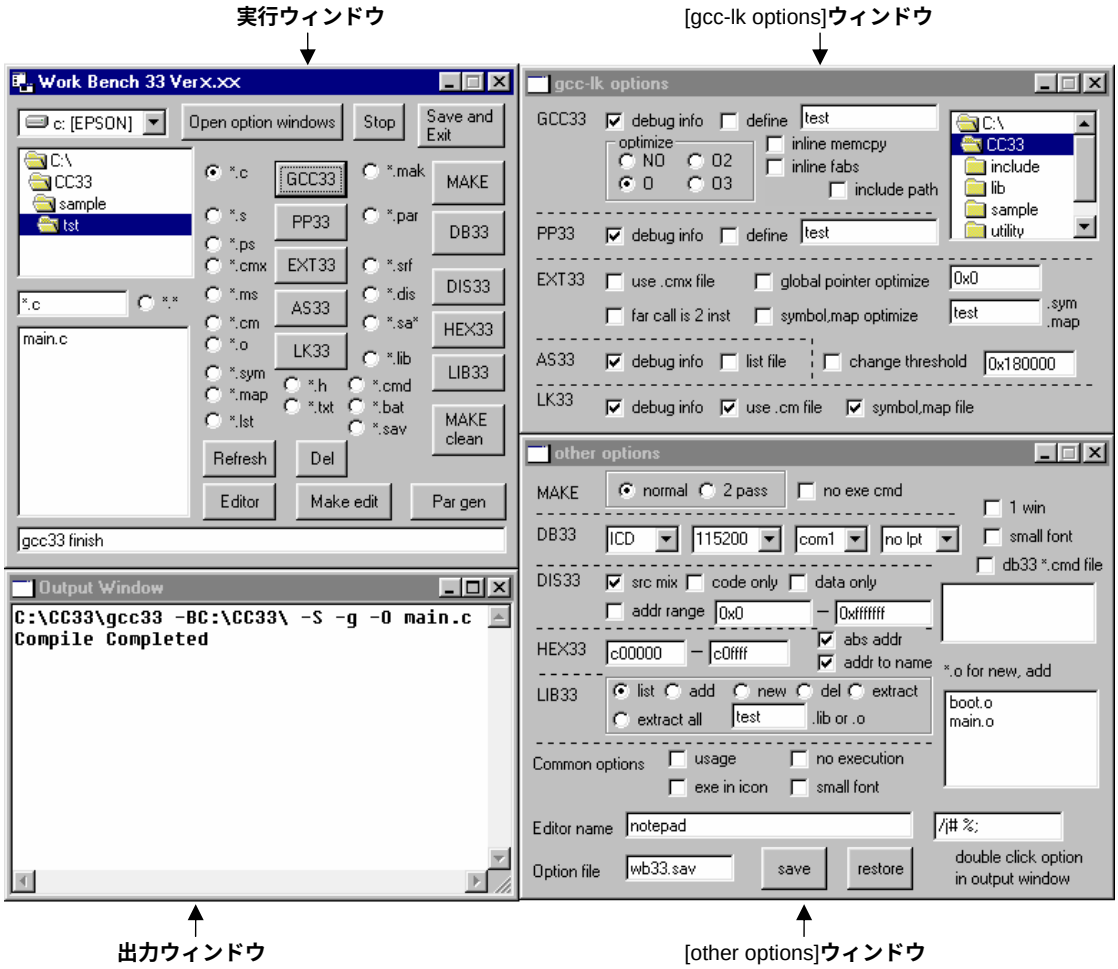
<オプションファイル名>: オプションウィンドウの設定状態が記録されているオプションファイルをフルパスで指定します。
例: C:CC33>wb33 c:¥cc33¥sample¥tst¥wb33.sav↵
省略した場合、オプションウィンドウはデフォルトの状態に設定されます。

●wb33を終了するには

実行ウィンドウ上の[Save and Exit]ボタンをクリックしてください。

5.2.2 ウィンドウ

wb33のウィンドウ構成は次のとおりです。



●実行ウィンドウ

wb33の起動時は実行ウィンドウのみが開きます。
実行ウィンドウでは、以下の操作が行えます。

- 各ツールに入力するファイルの選択(5.2.3節参照)
- 各ツールの実行(5.2.4節参照)
- オプションウィンドウのオープン(下記参照)
- makeファイルの作成(5.2.6節参照)
- パラメータファイルの作成(5.2.7節参照)
- ソースファイルの表示およびエディタによるソースファイルのオープン(5.2.3、5.2.8節参照)
- DOSコマンド等、コマンドライン入力(5.2.9節参照)

●オプションウィンドウ

実行ウィンドウの[Open option window]ボタンをクリックすることによって2つのオプションウィンドウが同時に開きます。

[gcc-lk options]ウィンドウ

このウィンドウでは、以下のツールの起動オプションが選択/指定できます。(5.2.4節参照)

- Cコンパイラgcc33
- プリプロセッサpp33
- 命令エクステンダext33
- アセンブラas33
- リンカlk33

makeにより実行されるツールのオプションがまとめられています。

[other options]ウィンドウ

このウィンドウでは、以下のツールの起動オプションが選択/指定できます。(5.2.4節参照)

- make
- デバッガdb33
- ディスアセンブラdis33
- バイナリ/HEXコンバータhex33
- ライブラリアンlib33

このほかに、wb33のオプションおよび全ツールに共通なオプションの選択(5.2.5節参照)、選択したオプション設定の保存と復帰(5.2.10節参照)が行えます。

●出力ウィンドウ

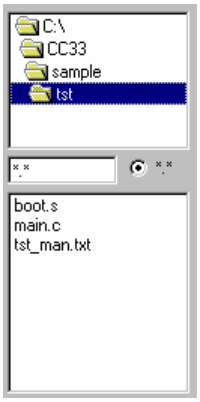
出力ウィンドウはソースの表示や、各ツールの実行結果を表示するためのウィンドウです。
次の2つの場合に開きます。

- 各ツールを実行させた場合
- 実行ウィンドウのファイルリストボックスで、ソースファイル(テキストファイル)をダブルクリックした場合(5.2.3節参照)

●ウィンドウ操作上の注意

- 実行ウィンドウおよび2つのオプションウィンドウの最大化ボタンは無効です。また、ウィンドウサイズを縮小した場合、スクロールバーは表示されませんので、できるだけデフォルトのサイズで使用してください。出力ウィンドウは最大化および元のサイズに戻すことができます。
- タスクバーボタンへの最小化はすべてのウィンドウが対応しています。実行ウィンドウ以外のウィンドウは個別に最小化および元のサイズに戻すことができます。
  実行ウィンドウを最小化すると、他のウィンドウも同時に最小化されます。元に戻す場合も同様です。
- 実行ウィンドウの[閉じる]ボタンでwb33を終了させることができます。ただし、オプションファイルへの保存は行われません。
  オプションウィンドウの[閉じる]ボタンはそれぞれのウィンドウのみを閉じますが、再び[Open option window]ボタンで開いた際に、閉じる前に選択していたオプションがすべて初期状態に戻ってしまいますので注意してください。
- 出力ウィンドウを除く各ウィンドウのリストボックスやテキストボックスはASCIIキャラクタの表示/入力にのみ対応しています。このため、漢字などは文字化けを起こします。
  出力ウィンドウは漢字も表示できますが、ソースファイルに制御文字等が含まれているとやはり文字化けを起こすことがあります。

5.2.3 ファイルの選択とソース表示



実行ウィンドウにはドライブ、ディレクトリおよびファイルを選択するリストボックスがあります。
各ツールの入力ファイルをここで選択します。

●ディレクトリの選択

wb33の起動時は、wb33および他のツールをインストールしたドライブとディレクトリが選択されます。
プログラム開発用に作業ディレクトリを作成し、そのディレクトリを選択してください。

注: wb33上での一連の処理は、すべて同一ディレクトリで行ってください。また、wb33が開いている間は、CDコマンドによってカレントディレクトリを変更しないでください。

●ファイルの種類

wb33の起動時は[*.*]ラジオボタンが選択され、ファイルリストボックスには選択したディレクトリ内のすべてのファイル名が表示されます。

各ツールの実行ボタンの前には、そのツールの入力ファイルの種類を選択するラジオボタンが用意されていますので、それを選択することでファイルの選択が容易になります。

注: すべての種類を表示させた状態でファイルを選択してツールを実行可能ですが、ツールに許可されていない種類のファイルを選択してもツールが起動しますので注意が必要です。また、命令エクステンダ、リンカはオプションの選択により、入力ファイルの種類が変わりますので注意してください。

●ファイルリストボックスの更新

[Refresh]**ボタン**

ファイルリストボックスはディレクトリの選択やツールの実行によって更新されますが、ファイルのコピーや他のアプリケーションでファイルを作成しても更新されません。
実行ウィンドウ上の[Refresh]ボタンをクリックすると、ファイルリストボックスが更新されますので、必要に応じて使用してください。ただし、ディレクトリリストボックスは更新されません。

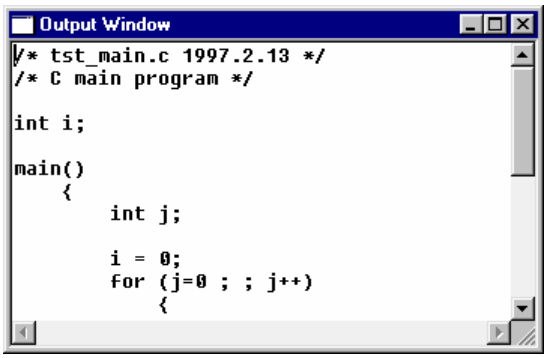
●ファイルの削除

[Del]**ボタン**

ファイルリストボックスで選択したファイルを削除します。

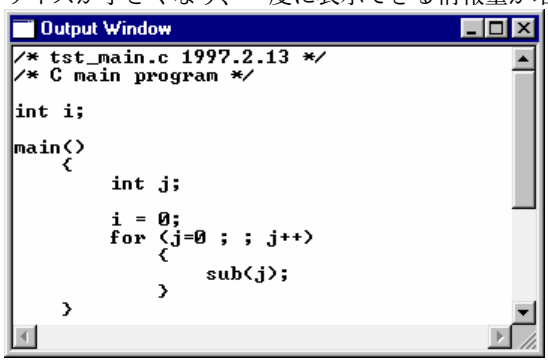
●ソースファイルの表示

ファイルリストボックス内のソースファイル名(テキストファイル)をダブルクリックすることで、そのファイルの内容を出力ウィンドウに表示させることができます。出力ウィンドウが閉じている状態でも、ファイル名のダブルクリックで開きます。



出力ウィンドウに表示可能なファイルはテキストファイルのみで、表示サイズも先頭からの32KBまでに限られます。
また、出力ウィンドウは表示専用です。文字の入力やコピー&ペーストは行えますが保存はできません。
32KB以上のテキストを表示させる場合や内容を編集するには、エディタを使用してください(5.2.8節参照)。

※ [other options]ウィンドウの[small font]チェックボックスを選択すると、出力ウィンドウのフォントサイズが小さくなり、一度に表示できる情報量が増えます。



縮小表示にはTerminalフォントの10ポイントを使用しています。このフォントがインストールされていない場合、効果は保証できません。デフォルトフォントはFixedSys 14ポイントです。

5.2.4 個別ツールの実行

各ツールは、実行ウィンドウ上のボタンで実行することができます。

●ツールを実行するには

1 オプションウィンドウで実行するツールの起動オプションを選択してください。
2 各ツールに入力するファイルをファイルリストボックスで選択し、ツールの実行ボタンをクリックしてください。
複数の入力ファイルを選択することができます。その場合は、選択したファイル数分ツールを繰り返し実行します。

●[Stop]ボタンについて

[Stop]ボタン

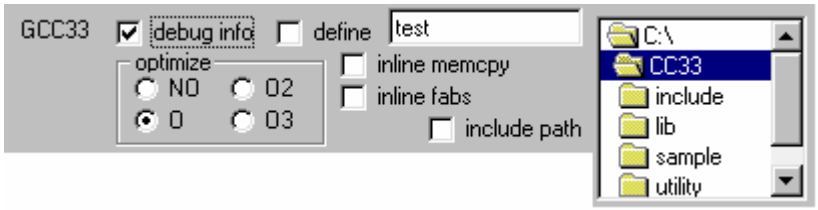
実行ボタンでツールを実行すると、そのツール(makeも含む)が処理を終了するまでwb33で処理を中止することはできません。
ただし、複数のファイルを選択してツールを実行した場合は、[Stop]ボタンによって中止することができます。複数のファイルを選択した場合もツールはファイルを1つずつ処理しますので、[Stop]ボタンがクリックされた時点のファイルの処理が終了後実行を中止します。

以下、各ツールの入出力ファイルとwb33で選択可能な起動オプションの概要を示します(makeについては5.2.6節参照)。各オプションの説明は選択した場合の機能です。
なお、詳細については各ツールの章を参照してください。

●Cコンパイラgcc33

実行ボタン:

入力ファイル: *.c(Cソースファイル、".c"は小文字のみ使用可能、大文字は不可)
出力ファイル: *.ps(入力と同じ名称のアセンブリソースファイル)
オプション:



□**debug info**: -gオプションの選択(デフォルト: ON)
Cソースレベルデバッグに必要な情報を出力ファイル内に生成します。通常は選択してください。

□**define**: -Dオプションの選択(デフォルト: OFF)
条件コンパイルなどに使用するマクロ名を定義します。定義内容は、このオプションの右にあるテキストボックスに入力してください。複数のマクロ名を定義する場合はカンマ(,)で区切って入力してください。
○**NO,** ○**O,** ○**O2,** ○**O3**: -O, -O2, -O3オプションの選択(デフォルト: O)
最適化レベルを選択します。
□**inline memcopy**: -mno-memcpyオプションの選択(デフォルト: OFF)
strcpy、memcpy関数コールをインライン展開します。
□**include path**: -Iオプションの選択(デフォルト: OFF)
右側のディレクトリリストボックスで選択したディレクトリが、インクルードファイルを検索するディレクトリのひとつに設定されます。

●プリプロセッサpp33

実行ボタン: PP33

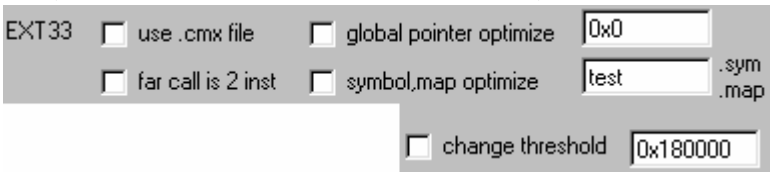
入力ファイル: *.s(アセンブリソースファイル)
出力ファイル: *.ps(入力と同じ名称のアセンブリソースファイル)
オプション:

□**debug info**: -gオプションの選択(デフォルト: ON)
アセンブリソースレベルデバッグに必要な情報を出力ファイル内に生成します。通常は選択してください。
□**define**: -dオプションの選択(デフォルト: OFF)
条件アセンブルに使用するデファイン名を定義します。定義内容は、このオプションの右にあるテキストボックスに入力してください。複数のデファイン名を定義する場合はカンマ(,)で区切って入力してください。

●命令エクステンダext33

実行ボタン: EXT33

入力ファイル: *.ps(アセンブリソースファイル)
*.cmx(コマンドファイル 要オプション指定)
出力ファイル: *.ms(入力と同じ名称のアセンブリソースファイル)
オプション:



□**use .cmx file**: -cオプションの選択(デフォルト: OFF)
コマンドファイル(.cmx)を入力して実行します。
このオプションを選択した場合は、実行ウィンドウのファイルリストボックスで必ず.cmxのファイルを選択してください。
□**global pointer optimize**: -gpオプションの選択(デフォルト: OFF)
グローバルポインタによる最適化を行います。選択する場合は右側のテキストボックスにグローバルポインタのアドレスを入力してください。
□**far call is 2 inst**: -nearオプションの選択(デフォルト: OFF)
処理中のファイルにないラベルへの分岐を2命令(ext 1個+分岐命令)で生成します。選択しない場合は3命令(ext 2個+分岐命令)で生成されます。
□**symbol,map optimize**: -lkオプションの選択(デフォルト: OFF)
リンカが出力したシンボルファイルとリンクマップファイルを利用して最適化を行います。1度リンクまで行ったソースファイルが最適化できます。選択する場合は右側のテキストボックスにシンボル/マップファイルの共通名称を入力してください。

□**change threshold**: -jオプションの選択(デフォルト: OFF)
相対距離の大きな分岐命令の最適化に使用する敷居値を指定します。指定する場合は右側のテキストボックスに敷居値を入力してください。指定のない場合、敷居値は0x180000として処理されます。

●アセンブラas33

実行ボタン: AS33

入力ファイル: *.ms(アセンブリソースファイル)
出力ファイル: *.o(入力と同じ名称のオブジェクトファイル)
オプション: AS33 ☑ debug info ☐ list file

□**debug info**: -gオプションの選択(デフォルト: ON)
デバッグに必要な情報を出力ファイル内に生成します。通常は選択してください。
□**list file**: -lオプションの選択(デフォルト: OFF)
アセンブリリストファイルを生成します。

●リンカlk33

実行ボタン: LK33

入力ファイル: *.cm(コマンドファイル 要オプション指定)
*.o(オブジェクトファイル)
出力ファイル: *.srf(srf33形式オブジェクトファイル)
オプション: LK33 ☑ debug info ☑ use .cm file ☑ symbol,map file

□**debug info**: -gオプションの選択(デフォルト: ON)
デバッグに必要な情報を出力ファイル内に生成します。通常は選択してください。
□**use .cm file**: -cオプションの選択(デフォルト: ON)
コマンドファイルに記述されたコマンドに従ってリンクを行います。
このオプションを選択した場合は、実行ウィンドウのファイルリストボックスで必ず.cmのファイルを選択してください。通常はこのオプションを指定します。
□**symbol,map file**: -sおよび-mオプションの選択(デフォルト: ON)
シンボルファイルおよびリンクマップファイルを生成します。これらのファイルは命令エクステンダによる最適化に使用します。

●デバッガdb33

実行ボタン: DB33

入力ファイル: *.par(パラメータファイル)
オプション: DB33 ICD 115200 com1 no lpt ☐ 1 win ☐ small font ☐ db33 *.cmd file

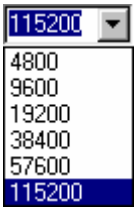
ICD
ICD
MON
ICE
SIM
SIMDLL

デバッガモード: -sim、-icd、-monオプションの選択
(デフォルト: ICDモード)
デバッガの動作モードを選択します。

SIM DLLモードについては、"S5U1C330C2S Manual"を参照してください。

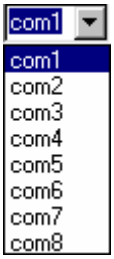
□**1 win**: -wオプションの選択(デフォルト: OFF)
デバッガの起動時に[Command]ウィンドウのみを開きます。
選択しない場合は、[Command]、[Source]、[Register]ウィンドウが開きます。

□**small font**: -sfオプションの選択(デフォルト: OFF)
デバッガのウィンドウに表示する文字をTerminalフォントの10ポイントとします。デフォルトフォントはFixedSys 14ポイントです。



通信速度: - bオプションの選択(デフォルト: 115200bps)
S5U1C33104H、 S5U1C33000H、 S5U1C330M2S(S5U1C330M1D1)との通信速度を選択します。

S5U1C33104H、S5U1C33000Hを使用する場合は、それぞれのディップスイッチの設定も確認してください。



シリアルポート: -comXオプションの選択(デフォルト: com1)
パソコンのシリアルポートを選択します。

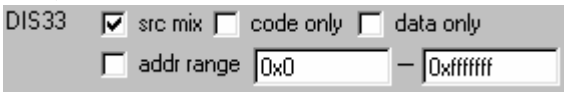
□**db33 *.cmd file**: -cオプションの選択(デフォルト: OFF)
デバッガの起動時に指定のデバッグコマンドファイルを実行させます。選択する場合は、オプション選択ボタンの下にあるファイルリストボックスでデバッグコマンドファイルを選択してください。このリストボックスには、実行ウィンドウで選択されているディレクトリ内のデバッグコマンドファイル名が表示されます。

パラレルポート: -lptXオプションの選択(デフォルト: なし)
パソコンのパラレルポートを選択します。

●ディスアセンブラdis33

実行ボタン: DIS33

入力ファイル: *.srf(srf33形式オブジェクトファイル)
出力ファイル: *.dis(入力と同じ名称のディスアセンブリリストファイル)
オプション:



□**src mix**: -mオプションの選択(デフォルト: ON)
ディスアセンブリリストにソースを対応させて出力します。データ部があれば、そのダンプ出力も行います。
□**code only**: -cオプションの選択(デフォルト: OFF)
コード部のディスアセンブリリストのみを出力します。
□**data only**: -dオプションの選択(デフォルト: OFF)
データ部のダンプ出力のみを行います。

注: この3つのオプションは同時に選択可能ですが、いずれか1つを選択してください。複数のオプションを同時に選択した場合は、組み合わせにより1つまたはすべての指定が無効になります。(13.3.2節参照)

□**addr range**: -aオプションの選択(デフォルト: OFF)
ディスアセンブルするアドレス範囲を指定します。この機能を使用する場合は、テキストボックスに開始アドレスおよび終了アドレスを入力してください。

●バイナリ/HEXコンバータhex33

実行ボタン: HEX33

入力ファイル: *.srf(srf33形式オブジェクトファイル)
出力ファイル: *.sa*(入力と同じ名称のモトローラS3形式ファイル)
オプション:

アドレス範囲:アドレス範囲の選択(デフォルト: 0xc00000～0xc0ffff)
入力ファイルのデータを変換するアドレス範囲を指定します。開始アドレス、終了アドレスには32バイトの境界アドレスを指定する必要があります。

□**abs addr**: -zオプションの選択(デフォルト: ON)
絶対アドレス情報を持つ出力ファイルが生成されます。このオプションを選択しない場合、出力されるアドレスは変換開始アドレスからのオフセットアドレスとなります。

□**addr to name**: -xオプションの選択(デフォルト: ON)
出力ファイルの拡張子".sa"の後ろに指定したアドレス範囲の情報を付加します。
例: test.sa_c00000_c0ffff
選択しない場合、拡張子は".sa"のみとなります。

●ライブラリアンlib33

実行ボタン: LIB33

入力ファイル: *.lib(ライブラリファイル)　実行ウィンドウで選択
*.o(リロケータブルオブジェクトファイル)　オプションウィンドウで選択
出力ファイル: *.lib(ライブラリファイル)
*.o(リロケータブルオブジェクトファイル)
オプション:

○**list**: -lオプションの選択(デフォルト: ON)
ライブラリファイルに登録されているオブジェクトファイルの一覧を、登録順に出力ウィンドウ(デフォルト)に表示します。

○**add**: -aオプションの選択(デフォルト: OFF)
[*.o for new, add]リストボックスで選択したオブジェクトファイル(複数選択可)を指定のライブラリファイルに追加します。

○**new**: オプションなし(デフォルト: OFF)
新しいライブラリファイルを作成します。ライブラリファイル名はオプションボタンの下のテキストボックスに入力してください。拡張子は不要です。登録するオブジェクトファイルは[*.o for new, add]リストボックスから選択します(複数選択可)。カレントディレクトリに存在するライブラリ名を指定した場合は、[add]オプション選択時と同様に、既存ファイルに選択したオブジェクトファイルが追加されます。

○**del**: -dオプションの選択(デフォルト: OFF)
ライブラリファイルから指定のオブジェクトを削除します。削除するオブジェクト名はオプションボタンの下のテキストボックスに入力してください。拡張子は不要です。

○**extract**: -xオプションの選択(デフォルト: OFF)
ライブラリファイルの指定オブジェクトをファイルとしてカレントディレクトリに復元します。復元するオブジェクト名はオプションボタンの下のテキストボックスに入力してください。拡張子は不要です。

○**extract all**: -xオプションの選択(デフォルト: OFF)
ライブラリファイルのすべてのオブジェクトをファイルとしてカレントディレクトリに復元します。

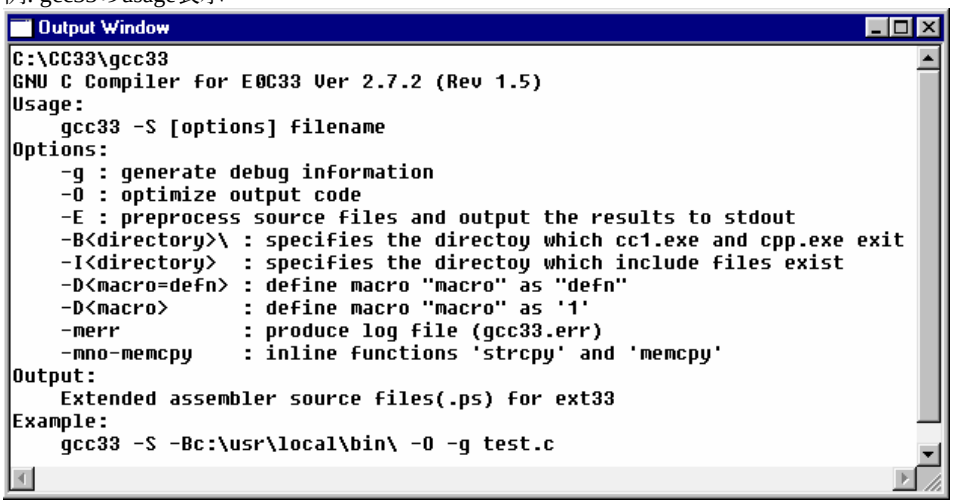
5.2.5 実行条件の選択

ここではツールの実行にかかわるオプションについて説明します。
以下のオプションはwb33の機能で、各ツールの起動オプションではありません。選択は[other options]ウィンドウで行います。

●[usage]**チェックボックス(デフォルト**: OFF)

このチェックボックスを選択してツールの実行ボタンをクリックすると、そのツールの起動コマンドと起動オプションの一覧を出力ウィンドウ(デフォルト)に表示します。入力ファイルやオプションが選択されていてもツールは実行されません。

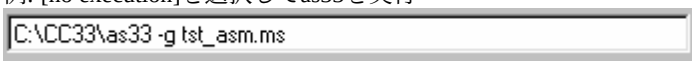
例: gcc33のusage表示



●[no execution]**チェックボックス(デフォルト**: OFF)

このチェックボックスを選択してツールの実行ボタンをクリックすると、ウィンドウ上で選択されている入力ファイルやオプションの指定を含むツールの起動コマンドを実行ウィンドウのテキストボックスに表示します。ツールは実行されません。
オプションウィンドウで指定できないオプションを、テキストボックス内のコマンドラインに追加してツールを実行させる場合などに有効です(5.2.9節参照)。

例: [no execution]を選択してas33を実行



[usage]チェックボックスが同時に選択されていると、usage表示を行うためのコマンドラインがテキストボックスに表示され、usage表示は行われません。

●[exe in icon]**チェックボックス(デフォルト**: OFF)

通常、各ツールの実行中はMS-DOSウィンドウが開き、ツールの出力メッセージが表示されます。
このチェックボックスを選択すると、各ツールはタスクバーに最小化された状態で実行されます。

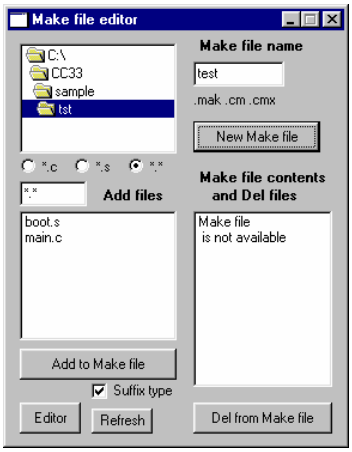
5.2.6 make

wb33にはmakeファイルを作成・編集する機能があります。

●makeファイルを作成するには

1 [gcc-lk options]ウィンドウでCコンパイラからリンカまでのオプションを設定してください。

2 [Make edit]ボタンをクリックしてください。[Make file editor]ウィンドウが表示されます。



3 作成するmakeファイル名を[Make file name]テキストボックスに入力してください。拡張子は不要です。

4 ファイルリストボックスから、使用するソースファイルを選択してください。
なお、".c"、".s"以外の拡張子を持つファイルは選択しても無視されます。

5 サフィックス定義を使用するmakeファイルを作成する場合は、[suffix]オプションの選択をしてください。
下記のmakeファイルはサフィックス定義を使用しない場合の例です。

6 [New Make file]ボタンをクリックしてください。

以上の操作でmakeファイル(.mak)がカレントディレクトリに作成されます。同時に、命令エクステンダ用およびリンカ用の2つのコマンドファイル(.cmx、.cm)も作成されます。
サフィックス定義を使用しない場合、異なるディレクトリにあるソースファイルをmakeファイルに追加することが可能です。
その場合は、ディレクトリリストボックスでディレクトリを変更し、ファイルリストボックスからソースファイルを選択した後、[Add to Make file]ボタンをクリックしてください。

makeファイル(.mak)

指定したオプションとソースファイルによるCコンパイラからリンカまでの実行手順が記述されています。makeファイルの詳細については"17.1 make"を参照してください。
例: チュートリアルで使用したmakeファイル(test.mak)

```
# make file made by wb33

# macro definitions for tools & dir

TOOL_DIR = C:\cc33
GCC33 = $(TOOL_DIR)\gcc33
PP33  = $(TOOL_DIR)\pp33
EXT33 = $(TOOL_DIR)\ext33
AS33  = $(TOOL_DIR)\as33
LK33  = $(TOOL_DIR)\lk33
LIB33  = $(TOOL_DIR)\lib33
MAKE  = $(TOOL_DIR)\make
SRC_DIR =

# macro definitions for tool flags

GCC33_FLAG = -B$(TOOL_DIR)\ -S -g -O -fno-builtin
PP33_FLAG  = -g
EXT33_FLAG =
AS33_FLAG  = -g
LK33_FLAG  = -g -s -m -c
EXT33_CMX_FLAG = -lk test -c
```

サフィックス定義を選んだ場合、このマクロを使いソースファイルの依存リストが記述されます。

```
# suffix & rule definitions

.SUFFIXES : .c .s .ps .ms .o .srf

.c.ms :
   $(GCC33) $(GCC33_FLAG) $(SRC_DIR)$*.c
   $(EXT33) $(EXT33_FLAG) $*.ps

.s.ms :
   $(PP33)  $(PP33_FLAG)  $(SRC_DIR)$*.s
   $(EXT33) $(EXT33_FLAG) $*.ps

.ms.o :
   $(AS33)  $(AS33_FLAG)  $*.ms

# dependency list start

### src definition start
```

サフィックス定義を選ばなかった場合、### src definition startから### src definition end間にSRC1_DIR=相対パスでマクロが記述され、さらにそのマクロを使いソースファイルの依存リストが記述されます。

```
### src definition end

test.srf : test.cm \
   boot.o \
   main.o \

$(LK33) $(LK33_FLAG) test.cm

## boot.s
boot.ms : $(SRC1_DIR)boot.s
$(PP33) $(PP33_FLAG) $(SRC1_DIR)boot.s
$(EXT33) $(EXT33_FLAG) boot.ps
boot.o : boot.ms
$(AS33) $(AS33_FLAG) boot.ms

## main.c
main.ms : $(SRC1_DIR)main.c
       $(GCC33) $(GCC33_FLAG) $(SRC1_DIR)main.c
       $(EXT33) $(EXT33_FLAG) main.ps
main.o : main.ms
       $(AS33) $(AS33_FLAG) main.ms

# dependency list end

# optimaization by 2 pass make

opt:
   $(MAKE) -f test.mak
   $(TOOL_DIR)\cwait 2
   $(EXT33) $(EXT33_CMX_FLAG) test.cmx
   $(MAKE) -f test.mak
```

```
# clean files except source

clean:
   del *.srf
   del *.o
   del *.ms
   del *.ps
   del *.map
   del *.sym
```

命令エクステンダ用コマンドファイル(.cmx)

命令エクステンダに入力するファイル名(選択したソースファイルの拡張子を.psに変更したもの)がリストされています。2パスmake実行時は命令エクステンダがこのファイル内に記述されたファイルを2パス目に入力して最適化の処理をします。

例: チュートリアルで使用したコマンドファイル(test.cmx)

```
;Files start
boot.ps
main.ps
;Files end
```

リンカ用コマンドファイル(.cm)

リンクを制御するリンカコマンドが記述されています。make実行時はリンカがこのファイル内に記述されたコマンドに従ってリンクを行います。

例: チュートリアルで使用したコマンドファイル(test.cm)

```
;Map set
-code 0x0c00000            ; set relative code section start address
;-data 0x0c01000           ; set relative data section start address
;-bss  0x0000000           ; set relative bss  section start address

;-code 0x0c00100 {test2.o test3.o} ; set code sections to absolute address
;-data 0x0c01100 {test2.o test3.o} ; set data sections to absolute address
;-bss  0x0000200 {test2.o test3.o} ; set bss  sections to absolute address

;Library path
-l C:\CC33\lib

;Executable file
-o test.srf

;Object files start
boot.o
main.o
;Object files end

;Library files
;io.lib
;lib.lib
math.lib
string.lib
ctype.lib
fp.lib
idiv.lib
```

配置アドレスを指定するコマンド等はコメントとしてリンカコマンドファイルが作成されます。開発するシステムのメモリ構成に合わせ、カスタマイズして使用してください。

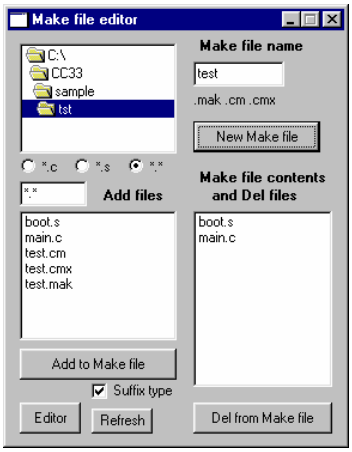
リンカコマンドの詳細は"12 リンカ"を参照してください。

●makeファイルを編集するには

作成済みのmakeファイルに対しソースファイルの追加/削除を行う場合は、次の手順で[Make file editor]ウィンドウを表示させてください。

1 実行ウィンドウのファイルリストボックスから編集するmakeファイルを選択します。

2 [Make edit]ボタンをクリックしてください。[Make file editor]ウィンドウが開き、指定のmakeファイルに登録されているソースファイルの一覧が[Make file contents and Del files]リストボックスに表示されます。



makeファイルに新たなソースファイルを追加する場合は、[Add files]リストボックスからソースを選択し(複数選択可能)、[Add to Make file]ボタンをクリックしてください。
追加するソースファイルは、すでに登録されているソースファイルと同じディレクトリに作成しておく必要があります。

makeファイルからソースファイルを削除する場合は、[Make file contents and Del files]リストボックスから削除するソースを選択し(複数選択可能)、[Del from Make file]ボタンをクリックしてください。

[Add to Make file]ボタン、[Del from Make file]ボタンによって追加/削除を行うと、".mak"ファイル以外に、同名の".cm"、".cmx"ファイルも自動的に修正されます。

[Make file editor]ウィンドウ上の[Editor]ボタンは、実行ウィンドウ上の[Editor]ボタンと同機能です。[Add files]リストボックス内のテキストファイルをエディタで開くことができます。
[Refresh]ボタンは、[Add files]リストボックス内の表示を最新の状態に更新します。

●makeファイル編集上の注意事項

・サフィックス定義を選択し、[Make file editor]ウィンドウ上でmakeファイルを作成した場合、ソースファイルのディレクトリを示すマクロSRC_DIRは次のように設定されます。
  1 実行ウィンドウで選択されているディレクトリ(カレントディレクトリ)からソースファイルを選択した場合、SRC_DIRは空白となります。
  2 他のディレクトリからソースファイルを選択した場合は、その絶対パスが定義されます。

  したがって、makeファイル作成後にソースファイルを他のディレクトリに移動した場合は、再度makeファイルを作成するか、SRC_DIRを修正する必要があります。また、ソースが複数のディレクトリにある場合もマニュアルで修正してください。
  サフィックス定義を選択しない場合、相対パスでmakeファイルが記述されます。ディレクトリ構成に変更のない移動であればmakeファイルの修正は必要ありません。

・[Make file editor]ウィンドウ上でのファイルの追加/削除は、以下に示すとおりファイル内のコメントや文字パターンに従って行われます。それらをマニュアルで修正した場合、その後のファイルの追加や削除で誤動作することがありますので注意してください。

makeファイルへの追加

元のmakeファイル

```
# dependency list start

### src definition start
SRC1_DIR=

### src definition end

test.srf : test.cm \
    boot.o \
    main.o \
                                    ....... *1
   $(LK33) $(LK33_FLAG) test.cm

## boot.s
boot.ms : $(SRC1_DIR)boot.s
$(PP33) $(PP33_FLAG) $(SRC1_DIR)boot.s
$(EXT33) $(EXT33_FLAG) boot.ps
boot.o : boot.ms
   $(AS33) $(AS33_FLAG) boot.ms

## main.c
main.ms : $(SRC1_DIR)main.c
   $(GCC33)              $(GCC33_FLAG)
   $(SRC1_DIR)main.c
   $(EXT33) $(EXT33_FLAG) main.ps
main.o : main.ms
   $(AS33) $(AS33_FLAG) main.ms

# dependency list end
```

例: "sys.c"追加後のmakeファイル

```
# dependency list start

### src definition start
SRC1_DIR=

### src definition end

test.srf : test.cm \
    boot.o \
    main.o \
    sys.o \
                                                    ....... *1
   $(LK33) $(LK33_FLAG) test.cm

## boot.s
boot.ms : $(SRC1_DIR)boot.s
   $(PP33) $(PP33_FLAG) $(SRC1_DIR)boot.s
   $(EXT33) $(EXT33_FLAG) boot.ps
boot.o : boot.ms
   $(AS33) $(AS33_FLAG) boot.ms

## main.c
main.ms : $(SRC1_DIR)main.c
   $(GCC33) $(GCC33_FLAG) $(SRC1_DIR)main.c
   $(EXT33) $(EXT33_FLAG) main.ps
main.o : main.ms
   $(AS33) $(AS33_FLAG) main.ms

## sys.c
sys.ms : $(SRC1_DIR)sys.c
   $(GCC33) $(GCC33_FLAG) $(SRC1_DIR)sys.c
   $(EXT33) $(EXT33_FLAG) sys.ps
main.o : sys.ms
   $(AS33) $(AS33_FLAG) sys.ms

# dependency list end
```

ファイル名や依存リストは"# dependency list start"と"# dependency list end"の間に追加されます。
したがって、この間の修正には十分注意してください。

オブジェクトファイル名は*1で示した行の直前に挿入されます。
*1の行にはスペースが1文字記述されていますので、この行を削除しないでください。
挿入されるオブジェクトファイル名の形式は次のとおりです。
∧∧∧∧<file>.o∧¥　　(∧はスペース)

依存リストは"# dependency list end"の直前に挿入されます。
"# dependency list end"の行を削除しないでください。
依存リストの形式は次のとおりです。
##∧<file>.s/c
<file>.ms∧:∧$(SRC_DIR)<file>.s/c
<file>.o∧:∧<file>.ms
(空白行)

"### src definition start"と"### src definition end"の間は、ソースファイルの相対パスを示すマクロが記述されます。したがって、ディレクトリの異なるソースファイルを追加すると、マクロ記述が増えます。

相対パス記述の形式は以下のようになっています。

### src definition start↵	相対パス記述の開始を表します。この行を削除しないでください。
SRC1_DIR=..¥sample¥¥↵	makeファイルからの相対パスをSRC1_DIRに定義しています。
↵	¥で終了するマクロ記述のためここに改行のみの行が必要です。
SRC2_DIR=..¥demo¥¥↵	makeファイルからの相対パスをSRC2_DIRに定義しています。
↵	
### src definition end↵	相対パス記述の終了を表します。この行を削除しないでください。

スペースの挿入等をしないでください。
ディレクトリの異なるソースファイルをmakeファイルに追加することにより、マクロ定義が追加になる場合、SRCxxx_DIR=相対パスの形式で追加されます。この場合のxxxにはすでに定義されているマクロの最大値＋１が入ります。xxxが2147483647を超えるmakeファイルは作成できません。

makeファイルからの削除

元のmakeファイル

```
# dependency list start

### src definition start
SRC1_DIR=

### src definition end

test.srf : test.cm \
    boot.o \
    main.o \
                                    ..... *1
   $(LK33) $(LK33_FLAG) test.cm

## boot.s
boot.ms : $(SRC1_DIR)boot.s
$(PP33) $(PP33_FLAG) $(SRC1_DIR)boot.s
   $(EXT33) $(EXT33_FLAG) boot.ps
boot.o : boot.ms
   $(AS33) $(AS33_FLAG) boot.ms

## main.c
main.ms : $(SRC1_DIR)main.c
   $(GCC33)              $(GCC33_FLAG)
   $(SRC1_DIR)main.c
   $(EXT33) $(EXT33_FLAG) main.ps
main.o : main.ms
   $(AS33) $(AS33_FLAG) main.ms

 # dependency list end
```

例: "boot.s"削除後のmakeファイル

```
# dependency list start

### src definition start
SRC1_DIR=

### src definition end

test.srf : test.cm \
    main.o \

   $(LK33) $(LK33_FLAG) test.cm

## main.c
main.ms : $(SRC1_DIR)main.c
   $(GCC33) $(GCC33_FLAG) $(SRC1_DIR)main.c
   $(EXT33) $(EXT33_FLAG) main.ps
main.o : main.ms
   $(AS33) $(AS33_FLAG) main.ms

# dependency list end
```

オブジェクトファイル名は指定名称で"∧∧∧∧<file>.o∧¥"(∧はスペース)の形式で記述されている行が削除されます。スペースの数も含め、この形式を変更しないでください。

依存リストは##で始まる指定ソースファイル名の行から最後の空白行まで削除されます。##で始まる行や空白行の変更や削除は行わないでください。

相対パスを記述したマクロ定義は、wb33から削除できません。エディタで削除する場合、必要なマクロを削除してしまうとwb33では元に戻せないので十分注意してください。

リンカコマンドファイルに対する追加と削除

```
;Object files start
boot.o
main.o
;Object files end
```

ソースファイルの追加/削除に伴い、リンカコマンドファイル内のオブジェクトファイル名も変更されます。追加/削除は、";Object files start"と";Object files end"の間を対象に行われます。したがって、これらのコメントの変更や削除は行わないでください。
追加の場合は、";Object files end"の行の直前に新たなオブジェクトファイル名が挿入されます。
削除の場合は、指定ファイル名の行が削除されます。

命令エクステンダ用コマンドファイルに対する追加と削除

```
;Files start
boot.ps
main.ps
;Files end
```

ソースファイルの追加/削除に伴い、命令エクステンダ用コマンドファイル内のファイル名も変更されます。追加/削除は、";Files start"と";Files end"の間を対象に行われます。したがって、これらのコメントの変更や削除は行わないでください。
追加の場合は、";Files end"の行の直前に新たなファイル名が挿入されます。
削除の場合は、指定ファイル名の行が削除されます。

●makeを実行するには

1 makeを実行する前に、作成したmakeファイルとリンカコマンドファイルのカスタマイズを必要に応じて行ってください。

2 makeのオプションを[other options]ウィンドウで選択してください。

3 実行ウィンドウのファイルリストボックスでmakeファイルを選択し、[MAKE]ボタンをクリックしてください。

以上の操作のみで、リンク後のsrf33形式オブジェクトファイルが得られます。makeの機能および動作については"17.1 make"を参照してください。

makeのオプション

○**normal**: 1パスmake(デフォルト: ON)
ターゲット名の引数を指定せずにmakeを実行します。
makeファイル内の最初のターゲットのmakeとなりますので、.srfファイルが作成または更新されるまでの1パス処理となります。

○**2 pass**: 2パスmake(デフォルト: OFF)
ターゲット名"opt"を引数としてmakeを実行します。
makeファイル内の"opt:"から記述されたコマンドを実行します。これにより、リンクまでの1パス目の処理が終了後、リンク後のシンボル情報を元にした命令エクステンダによる最適化が行われます。最適化より生成されたファイルは再度アセンブル、リンクされます。

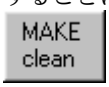
□**no exe cmd**: -nオプションの選択(デフォルト: OFF)
makeの-nオプションを指定してmakeを実行します。
makeにより実行されるコマンドを表示するのみで、実際には実行されません。前回のmake以降に変更されたファイルがあるか確認する場合などに利用できます。

●MAKE clean

wb33が作成するmakeファイルには、ソース以外の中間ファイルおよびオブジェクトファイルを削除するコマンドがターゲット名"clean"として記載されています。記載されたコマンドは次のとおりです。

```
# clean files except source

clean:
        del *.srf
        del *.o
        del *.ms
        del *.ps
        del *.map
        del *.sym
```

このコマンドはwb33の実行ウィンドウでmakeファイルを選択し、[MAKE clean]ボタンをクリックすることによって実行することができます。



[MAKE clean]**ボタン**

カレントディレクトリ内の".srf"、".o"、".ms"、".ps"、".map"、".sym"の拡張子を持つファイルがすべて削除されます。

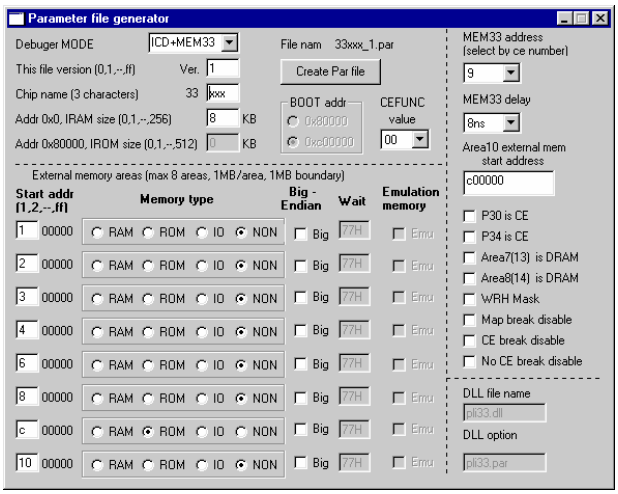
5.2.7 パラメータファイルジェネレータ

デバッグに必要なパラメータファイルをwb33上で簡単に作成することができます。
パラメータファイルは、開発する機種のメモリ構成をデバッガdb33に設定するためのものです。

●[Parameter file generator]ウィンドウ

実行ウィンドウの[Par gen]ボタンをクリックすると、[Parameter file generator]ウィンドウが開きます。

[Par gen]ボタン



このウィンドウで以下に説明する内容を指定し、[Create Par file]ボタンをクリックするとパラメータファイルがカレントディレクトリに作成されます。

デバッガモード

[Debugger MODE]のコンボボックスより対応するデバッガのモードを選択します。
デバッガの各モードに必要な情報だけが入力可能となります。

パラメータファイル名

[This file version (0,1,--,ff)]のテキストボックスにファイルのバージョン(v)を、[Chip name (3 characters)]のテキストボックスに機種名の下位3桁(xxx)を入力してください。
この指定により、パラメータファイルは"33xxx_v.par"という名称で作成されます。

内部メモリ容量の指定

[IRAM size (0,1,--,256)]と[IROM size (0,1,--,512)]のテキストボックスに、それぞれ内蔵RAM容量と内蔵ROM容量をKB単位で入力します。ICEモード以外では、[IROM size (0,1,--,512)]を設定することはできません。

ブートアドレスの選択

システムが外部ROMからブートする場合は[0xc00000]、内蔵ROMからブートする場合は[0x80000]のラジオボタンを選択してください。ICEモード以外では、[0xc00000]に固定されます。

外部メモリ領域の選択

外部メモリを使用する場合は、使用する領域の開始アドレス(1MB単位)をテキストボックスに入力し、メモリタイプ(RAM、ROM、I/O、MON)をラジオボタンで選択してください。外部メモリは最大8領域まで指定可能です。
外部メモリ領域は、デフォルトではリトルエンディアン形式でアクセスされます。これをビッグエンディアン形式に変更する場合は、[Big]を選択してください。[Big]を選択した場合、シミュレータモードはスタート時よりCPUのバスがビッグエンディアン方式であるものとしてデータを処理します。その他のモードでは、プログラムやコマンドで実ICのBCUレジスタを書き換えてビッグエンディアンに設定する必要がありますので注意してください。デバッガ内での動作については、16.9.11節内の"デバッグコマンドの動作"を参照してください。

インサーキットエミュレータS5U1C33104Hには、最大8MBの外部メモリ用のエミュレーションメモリを搭載可能で、ターゲットボード上にメモリを搭載しなくてもデバッグが行えるようになっています。指定した外部メモリ領域をこのエミュレーションメモリ内に確保する場合は[Emu]ボックスをチェックします。ターゲットボード上のデバイスを使用する場合は、[Emu]ボックスをチェックしないでください。[Emu]ボックスはICEモード時のみ有効です。

シミュレータモードでは領域ごとにウェイト数の設定が可能です。この値はシミュレータモードでのCPUクロック数の表示に使用されます。詳しくは、"16.11 パラメータファイルについて"を参照してください。[Wait]ボックスはシミュレータモード時のみ有効です。

S5U1C33001M1の設定

以下の項目はデバッグモードに"ICD+MEM33"を選択した場合にのみ選択/入力可能になります。

CEFUNC value:
DRAMタイミングを設定レジスタ(S1C33208では0x48130)のD9–DAビットに設定する値を選択してください。

注: この設定は、外部メモリ領域を設定するテキストボックス(開始アドレス)およびメモリタイプを選択するラジオボタンと連動します。したがって、最初にこの設定をしてください。

MEM33 address:
S5U1C33001M1がマップされるエリア(S5U1C33001M1のDSW6で選択したCE信号の数値)に設定してください。

MEM33 delay:
通常は8nsで使用してください。

Area10 external mem start address:
外部メモリエリア10の先頭アドレスを入力してください。

P30 is CE:
P30をCE信号として使用する場合に選択してください。

P34 is CE:
P34をCE信号として使用する場合に選択してください。

Area7(13) is DRAM:
エリア7(13)にDRAMを使用する場合、選択してください。

Area8(14) is DRAM:
エリア8(14)にDRAMを使用する場合、選択してください。

WRH Mask:
WRH信号を使用しない場合(BSL、BSH方式で×16のSRAMを使用する場合)に選択してください。

Map break disable:
マップブレークのみ使用不可にしたい場合、選択してください。

CE break disable:
CEブレークのみ使用不可にしたい場合、選択してください。

No CE break disable:
No CEブレークのみ使用不可にしたい場合、選択してください。

SIM DLLモードの設定

シミュレータモードでは[DLL file name]と[DLL option]が入力可能になります。
通常のシミュレータモードでは入力/変更する必要はありません。
SIM DLLモードの詳細については、"S5U1C330C2S Manual"を参照してください。

●パラメータファイル

作成されたパラメータファイルの例を示します。
例: デフォルト設定(ICDモード)で作成

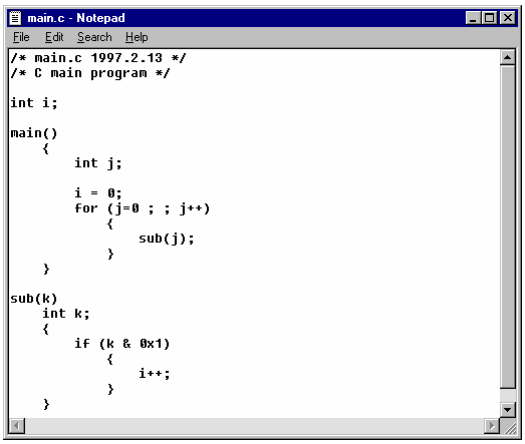
```
CHIP            33xxx               ; chip name (33XXX)
VER             1                   ; this file version

; Map allocation (max 31 areas, 256bytes boundary)

RAM     0       1FFF                ; internal RAM area 8KB
IO      40000   4ffff               ; internal IO area 64KB
ROM     c00000  cfffff              ; external ROM 1MB

END
```

テキストファイルとして作成されますので、エディタ等でカスタマイズができます。内容の詳細については、"16.11 パラメータファイルについて"を参照してください。

5.2.8 汎用エディタの指定

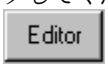
wb33では、ソースファイルの表示やツールの実行結果の表示を指定のエディタで行うことができます。特に、wb33で選択したソース(テキスト)ファイルを直接エディタで開く機能は、ソースの修正などに有効です。

●ソースファイルをエディタで開くには



1 実行ウィンドウのファイルリストボックスで開きたいテキストファイルを選択してください。ソースファイル以外のmakeファイルやコマンドファイルも含み、複数のテキストファイルが選択可能です。

2 実行ウィンドウの[Editor]ボタンをクリックしてください。



[Editor]**ボタン**

エディタ(デフォルトは「メモ帳(note-pad)」)が起動し、選択したファイルを開きます。

[Make file editor]ウィンドウの[Editor]ボタンも上記と同機能です。

●「メモ帳」以外のエディタを使用するには

wb33が開くエディタはデフォルトで「メモ帳(notepad)」が設定されています。
[other options]ウィンドウにある、[Editor name]テキストボックスを変更することにより、エディタを変更することができます。

ここにエディタの起動コマンドをフルパスで入力してください。

この設定は、[other options]ウィンドウを閉じるとデフォルトの設定に戻るため、[save options]ボタンでオプションファイルにセーブしてください。(5.2.10節参照)

●タグジャンプ

[Editor name]の右側のテキストボックスには、出力ウィンドウに表示されるエラーメッセージからソースへタグジャンプを行うためにエディタに送るコマンドを設定できます。
デフォルトの「メモ帳(notepad)」ではこの機能は使用できません。起動時にファイル名と行番号がコマンドで指定可能なエディタを使用する場合にのみ有効です。

デフォルト設定の"/j# %;"はエディタ"秀丸"用のコマンドです。
出力ウィンドウ上に表示されたファイル名と行番号の部分をダブるクリックすると、コマンドの#が行番号、%がファイル名に置き換えられてエディタに送られます。

例: boot.ms(6): Error: Invalid instruction. - define near boot.s(4)
boot.ms(6)をダブルクリックすると、#が6、%がboot.msに置き換えられてエディタに送られます。エディタが対応していれば、boot.msを開き6行目にジャンプします。
上記のメッセージ例ではboot.s(6)もタグジャンプの対象になります。

main.c:4: warning: data definition has no type or storage class
このメッセージではmain.c:4がタグジャンプの対象になります。

5.2.9 コマンドライン入力

実行ウィンドウには各ツールの起動コマンドを表示あるいは編集するためのテキストボックスが設けられています。オプションウィンドウのチェックボックスやボタンで対応していない起動オプションをこのテキストボックスで追加して実行させることができます。
また、COPYなどのDOSコマンドをこのテキストボックスで実行することもできます。

●コマンドラインを編集して実行するには

1 実行するツールのオプションと入力ファイルを選択してください。

2 [other options]ウィンドウの[no execution]チェックボックスを選択してツールの実行ボタンをクリックしてください。
テキストボックスに選択した内容を含む起動コマンドが表示されます。

```
C:\CC33\as33 -g tst_asm.ms
```

3 必要なオプションを追加入力し[Enter]キーを押してください。

編集したコマンドラインが実行されます。

注: 各ツールの実行ボタンでは起動させないでください。実行ボタンをクリックすると、編集前のコマンドラインでツールが起動します。
また、コマンドラインで起動した場合、実行結果を出力ウィンドウやエディタに表示することはできません。-eオプションが指定されていても、実行終了時のメッセージ(エラーファイルの内容)は表示されません。また、[exe in icon]チェックボックスの指定も無視されます。さらにファイルリストボックスも更新されません。

●DOSコマンドを実行するには

DOSコマンドの先頭に">"を付けてテキストボックスに入力し、[Enter]キーを押すとそのDOSコマンドを実行します。

```
>copy test.sa_c00000_c00fff 33000tst.sa
```

注: DIRなどを実行しても、出力ウィンドウやエディタに結果を表示させることはできません。
CDコマンド等、wb33の動作に影響するコマンドもありますので、ファイルのコピーやリネームなどにのみ使用してください。

5.2.10 オプションの保存と復帰

オプションウィンドウで選択した内容は、[other options]ウィンドウの[save]ボタンでファイルに保存し、[restore]ボタンで復帰させることができます。オプションウィンドウは開く際に設定内容をデフォルトに戻しますので、頻繁に選択するオプションやエディタの設定などは、この方法で保存してください。

Option file [wb33.sav] [save] [restore]

保存するファイル名はデフォルトで"wb33.sav"となっています。複数の設定を保存しておく場合には、それぞれ名前を変えてください。上書きの確認メッセージは出力されません。
ファイルはテキスト形式で作成されます。

[Save and Exit]ボタンでワークベンチを終了する際にもオプションファイルへの保存が行われます。
また、作成されたオプションファイルを"wb33.exe"アイコンにドラッグ&ドロップしてワークベンチを起動すると、ファイルに保存されたオプション設定を起動時に復帰することができます。これは、コマンドラインでオプションファイルを指定して起動する場合も同様です。

5.3 エラーメッセージ

wb33でエラーが発生すると、メッセージ表示用のダイアログボックスが開きます。メッセージ確認後、[OK]ボタンをクリックしてダイアログボックスを閉じてください。

例:

エラーメッセージの一覧を以下に示します。個別ツール実行中のエラーメッセージについては、各ツールの章を参照してください。

表5.3.1 wb33エラーメッセージ一覧

メッセージ	内容
Can not execute command xxxxxx	各ツールボタンやコマンドライン(xxxxxx)が実行できません。 ・コマンドラインが不正です。 ・ツールや必要なファイルが見つかりません。
Can not open file xxxxxx	ファイル(xxxxxx)がオープンできません。 ・[Make edit]実行時 ・ソース表示 ・オプションの保存/復帰時
Write error xxxxxx	ファイル(xxxxxx)への書き込みが行えません。 ・[Make edit]実行時 ・オプションの保存時
Read error xxxxxx	ファイル(xxxxxx)が読み込めません。 ・ソース表示 ・オプションの復帰時
R/W error xxxxxx or xxxxxx	ファイルの読み込み、または書き込みが行えません。 ・[Make file editor]実行時
Can not delete file xxxxxx	ファイルを削除できません。 ・[Del]実行時
File size over 32KB, so cut down to 32KB xxxxxx	32KB を越えるファイル(xxxxxx)が選択されました。 ・ソース表示 ファイルの先頭から 32KB 分のみが有効となります。

5.4 注意事項

wb33上での一連の処理は、すべて同一ディレクトリで行ってください。
また、wb33が開いている間は、CDコマンドによってカレントディレクトリを変更しないでください。

6 Cコンパイラ

この章ではCコンパイラgcc33の使用方法とアセンブリソースとのインタフェースに関する内容を説明します。Cコンパイラの標準機能やCソースの文法については、ANSI Cを解説している一般の書籍をご覧ください。

6.1 機能

Cコンパイラgcc33(以下gcc33と記述)はCソースファイルをコンパイルし、S1C33000命令セットのニーモニック、命令エクステンダの拡張命令、アセンブラの擬似命令を含むアセンブリソースファイルを生成します。本パッケージのCコンパイラgcc33はANSI C準拠のGNU Cコンパイラです。特殊な文法はサポートしていませんので、他機種用のプログラムの移植なども容易に行えます。
また、強力なオプティマイズ能力を持ち、非常にコンパクトなコードを生成しますので、組み込み用アプリケーションの開発に最適です。
Cコンパイラはgcc33.exe、cpp.exe、cc1.exeの3つのファイルで構成されています。

gcc33はFree Software Foundation, Inc.のCコンパイラをベースとしています。ライセンスに関してはテキストファイル"GNU_COPYRIGHT"に記述されていますので、ご使用前にご覧ください。

6.2 入出力ファイル

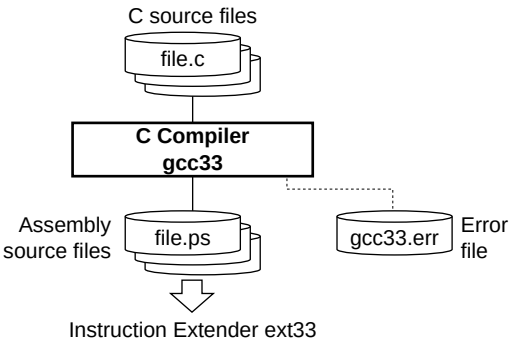


図6.2.1 **フローチャート**

6.2.1 入力ファイル

●Cソースファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.c
内容: Cソースプログラムを記述したファイルです。

6.2.2 出力ファイル

●アセンブリソースファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.ps (<ファイル名>は入力ファイルと同じです。)
出力先: カレントディレクトリ
内容: 命令エクステンダext33に入力するアセンブリソースファイルです。
拡張命令を含むため、アセンブラas33に直接入力することはできません。

●エラーファイル

ファイル形式: テキストファイル
ファイル名: gcc33.err
出力先: カレントディレクトリ
内容: -merrオプションが指定された場合に出力されるファイルで、エラーメッセージなど、gcc33が標準出力(stdout)に対して出力する内容を記録します。
他のツールと異なり、-merr指定時は画面上にメッセージが表示されません。

6.3 起動方法

6.3.1 起動フォーマット

●コマンドラインの一般形

gcc33 ^ [<起動オプション>] ^ [<ファイル名>]

^はスペースを示します。
[ ]は省略可能なことを示します。

<ファイル名>: Cソースファイルを拡張子(.c)も含めて指定します。

●ワークベンチからの起動

起動オプションとソースファイルを選択し、[GCC33]ボタンをクリックします。

コマンドラインで複数のソースファイルが指定可能で、すべてを一度に処理できます。
wb33でも複数のファイルが選択可能ですが、wb33ではファイル数分gcc33を実行します。

6.3.2 起動オプション

gcc33には11種類の起動時オプションが用意されています。

-S

機能: アセンブリコードの出力
wb33上の指定: なし(常に指定)
説明:
- アセンブリソースファイルを出力するためのスイッチです。
- このオプションは必ず指定しなければなりません。指定せずに起動するとUsageを表示するのみでコンパイルは行いません。

-B<パス名>

機能: コンパイラのパス指定
wb33上の指定: なし(不要)
説明:
- コンパイラ本体cc1.exeとCプリプロセッサcpp.exeが存在するディレクトリを指定します。
- <パス名>は相対パスまたは絶対パスで-Bに続けて入力し、最後に¥を入力します。
- 環境変数GCC_EXEC_PREFIXまたはPATHにコンパイラ本体とCプリプロセッサが存在するディレクトリが登録されていれば-Bスイッチは不要です。優先順位は-Bスイッチ、GCC_EXEC_PREFIX、PATHの順です。
  GCC_EXEC_PREFIXは-Bスイッチと同様に相対パスまたは絶対パス+¥の形式で登録する必要があります。
- -BスイッチおよびGCC_EXEC_PREFIXがない場合、PATHで指定されたディレクトリかカレントディレクトリが指定されたものとして処理します。

-E

機能: Cプリプロセッサのみの実行
wb33上の指定: なし
説明:
- 指定のCソースファイルでCプリプロセッサのみを実行し、標準出力に結果を出力します。

-I<パス名>

機能: インクルードファイルのあるディレクトリを指定
wb33上の指定: [Include path]をチェック、ディレクトリをリストボックスで選択
説明:
- Cソース中にインクルードしたファイルが存在するディレクトリを指定します。
- <パス名>は-Iに続けて入力してください。
- 複数のディレクトリを指定することができます。その場合は、-I<パス名>を必要数入力してください。インクルードファイルの検索は、コマンドライン上の出現順に行われます。
- 環境変数C_INCLUDE_PATHにディレクトリが登録されていれば-Iスイッチは不要です。
- ファイル検索の優先順位は、カレントディレクトリ、-Iスイッチ、C_INCLUDE_PATHの順です。

-D<マクロ名>[=<置き換え文字>]

機能: マクロ名の定義

wb33上の指定: [define]をチェック、テキストボックスに入力

説明:
- マクロ名を定義します。#defineと同じ働きをします。=<置き換え文字>が指定されている場合はマクロにその値を定義します。指定されていない場合、マクロの値は1に設定されます。
- <マクロ名>[=<置き換え文字>]は-Dに続けて入力してください。
- 複数のマクロ名を指定することができます。その場合は、-D<マクロ名>[=<置き換え文字>]を必要数入力してください。wb33の場合は<マクロ名>[=<置き換え文字>]をカンマ(,)で区切って入力してください。

-O, -O2, -O3

機能: 最適化の指定

wb33上の指定: [NO]、[O]、[O2]、[O3]の1つをチェック

説明:
- いずれか1つのスイッチを指定し、最適化処理を行います。コード効率と速度(主にコード効率)を重視して最適化されたコードが出力されます。
- スイッチが指定されない場合、またはwb33で[NO]を選択した場合は最適化は行われません。
- -Oの数値が上がるほどコード効率が高まりますが、デバッグ情報が一部出力されないなどの問題が発生することがありますので注意してください。
  正常に実行できないときは、最適化の数値を下げてください。通常は、-Oでコンパイルすることを推奨します。
- 最適化を指定すると、S5U1C33000Cはメモリからレジスタにロードした値を再利用し、メモリのリード/ライトを減らすようにします。そのため、メモリへのアクセスの発生を保証できなくなります。これを避けるには、以下の方法で対応してください。
  - ・変数のvolatile宣言　例)volatile char　IO_port1;
  - ・最適化を行わない
  - ・-fvolatile指定　ポインタがvolatileとしてアクセスされます。
    -fvolatile-global指定　外部変数がvolatileとしてアクセスされます。

-g

機能: デバッグ情報の付加

wb33上の指定: [debug info]をチェック

説明:
- デバッグ情報を含む出力ファイルが生成されます。
- Cソースレベルデバッグを行う場合は、必ず指定してください。
- デバッグ情報については6.6節を参照してください。

-mno-memcopy

機能: strcpy、memcpy関数コールのインライン展開

wb33上の指定: [inline memcpy]をチェック

説明:
- strcpyおよびmemcpy関数コールがインライン展開されます。

-fno-builtin

機能: fabs関数コールのインライン展開

wb33上の指定: [inline fabs]をチェック

説明:
- fabs関数がインライン展開されません。fabsはcallする方が高速です。

-merr

機能: エラーファイルの出力

wb33上の指定: なし

説明:
- エラーメッセージなど、gcc33が標準出力(stdout)に対して出力する内容をファイル(gcc33.err)に出力します。
- このオプションが指定された場合、画面にはメッセージが出力されません。

コマンドラインにオプションを入力する場合、オプションの前後には1個以上のスペースが必要です。

例: c:¥cc33¥gcc33　-S　-Bc:¥user¥local¥bin¥　-O　-g　test.c

6.4 メッセージ

gcc33はメッセージをすべて標準出力(stdout)に対して出力します。
wb33の[GCC33]ボタンでgcc33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

●終了メッセージ

正常に終了した場合は次の終了メッセージのみを出力します。

```
Compile Completed
```

●Usage出力

ファイル名が指定されなかったり、オプション指定が正しくない場合、次の使用方法のメッセージを出力して終了します。

```
C:\CC33\gcc33
GNU C Compiler for EOC33 Ver 2.7.2 (Rev x.xx)
Usage:
    gcc33 -S [options] filename
Options:
    -g : generate debug information
    -O : optimize output code
    -E : preprocess source files and output the results to stdout
    -B<directory>\ : specifies the directoy which cc1.exe and cpp.exe exit
    -I<directory>  : specifies the directoy which include files exist
    -D<macro=defn> : define macro "macro" as "defn"
    -D<macro>      : define macro "macro" as '1'
    -merr          : produce log file (gcc33.err)
    -mno-memcpy    : produce string variable initializer statement inline
Output:
    Extended assembler source files(.ps) for ext33
Example:
    gcc33 -S -Bc:\usr\local\bin\ -O -g test.c
```

●エラー/ワーニング発生時

エラー/ワーニングが発生した場合は、終了メッセージの前にエラー/ワーニングメッセージが表示されます。
エラーの場合、gcc33は出力ファイルを作成せずに終了します。ワーニングの場合、gcc33は出力ファイルを作成して終了しますが、正常な動作は保証されません。

6.5 コンパイラの出力

ここではgcc33が出力するアセンブリソースや使用するレジスタについて説明します。

6.5.1 出力内容

gcc33はCソースをコンパイルし、以下の内容を出力します。
・ S1C33000命令セットのニーモニック
・ 拡張命令のニーモニック
・ アセンブラ擬似命令

基本命令以外は拡張命令で出力されます。したがって、出力されたアセンブリソースファイルは命令エクステンダext33で処理してください。アセンブラas33で直接アセンブルすることはできません。
また、出力されたアセンブリソースファイルをプリプロセッサpp33には通さないでください。

システム制御命令やMAC命令はCソースでは表現できませんので、asmによるインラインアセンブルまたはアセンブリソースファイルで対応してください。
例: asm ("mac　%r12");

セクションやデータ定義にはアセンブラ擬似命令を出力します。アセンブラ擬似命令については"11.8 アセンブラ擬似命令"を参照してください。
命令やデータが設定されるセクションは以下のとおりです。

命令

すべてCODEセクションに配置されます。

初期値を持たないグローバル変数とスタティック変数

BSSセクションに配置されます。

```
例: int　i;                .comm i 4
```

初期値を持つグローバル変数とスタティック変数

DATAセクションに配置されます。

```
例: int i=1;               .global i
                           .data
                           .align 2
                   i2:
                           .word 1
```

定数

CODEセクションに配置されます。

```
例: const int i=1          .global i
                           .code
                           .align 2
                   i2:
                           .word 1
```

関数名やラベル等、すべてのシンボルにはアセンブラ擬似命令.defによるシンボル情報が挿入されます(-gオプション指定時)。シンボル情報については"6.6 デバッグ情報"を参照してください。

ラベルは次の形式で出力されます。
__Limm31　分岐先ラベル
__LCimm31　文字列定数ラベル
__Lbimm31　ブロック開始位置ラベル
__Leimm31　ブロック終了位置ラベル　　(imm31は0～2147483647の10進数)

6.5.2　データ表現

gcc33はANSI Cのすべてのデータ型に対応しています。各型のサイズ(バイト数)、表現できる数値の有効範囲を表6.5.2.1に示します。

表6.5.2.1　型の種類とサイズ

型	サイズ	数値の有効範囲
char	1	-128〜127
unsigned char	1	0〜255
short	2	-32768〜32767
unsigned short	2	0〜65535
int	4	-2147483648〜2147483647
unsigned int	4	0〜4294967295
long	4	-2147483648〜2147483647
unsigned long	4	0〜4294967295
pointer	4	0〜4294967295
float	4	1.175e-38〜3.403e+38 (正規化数)
double	8	2.225e-308〜1.798e+308 (正規化数)

float型、double型はIEEE標準規格のフォーマットに準拠しています。

●メモリ上の格納位置

データのメモリ上の格納位置はデータの型に依存します。データは、グローバルかローカルかにかかわらずメモリアドレスがそのデータ型のサイズで割り切れるような場所からサイズ分配置されます。

double型は4バイトの境界アドレスにアライメントされ、データの下位4バイト(仮数部(31:0))、がメモリの下位側の4バイトに、上位4バイトデータ(符号、指数部、仮数部(51:32))がメモリの上位側4バイトに格納されます。

●構造体データ

構造体データはdouble型と同様4バイト境界(4で割り切れるアドレス)から配置されます。メンバーは定義の出現順に各データ型のサイズに従って配置されます。

構造体の定義と配置の例を次に示します。

例:
```
struct Sample {
        char        cData;
        short       hData;
        char        cArray[3];
        int         iData;
        double      dData;
};
```

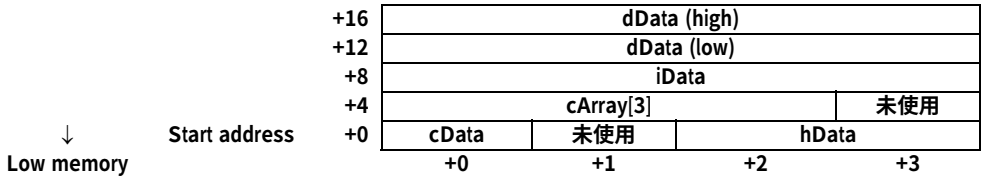


図6.5.2.1　構造体データのメモリ配置例

図のように、メンバーのデータ型により未使用領域ができることがあります。

6.5.3 レジスタ使用法

gcc33は汎用レジスタを次のように使用します。

表6.5.3.1 汎用レジスタの使用法

レジスタ	使用法
R0	関数呼び出し時に値を保存する必要のあるレジスタ
R1	
R2	
R3	
R4	スクラッチレジスタ
R5	
R6	
R7	
R8	グローバルポインタ(未使用、ext33 で使用)
R9	拡張命令展開用スクラッチレジスタ(未使用、ext33 で使用)
R10	戻り値格納用レジスタ(8/16/32 ビットデータ、double 型データの下位 32 ビット)
R11	戻り値格納用レジスタ(double 型データの上位 32 ビット)
R12	引数渡し用レジスタ(第 1 ワード)
R13	引数渡し用レジスタ(第 2 ワード)
R14	引数渡し用レジスタ(第 3 ワード)
R15	引数渡し用レジスタ(第 4 ワード)

●関数呼び出し時に値を保存するレジスタ(R0〜R3)

式の計算結果やローカル変数が格納されます。これらのレジスタは、関数からリターンした際に呼び出し時の値を保持していなければなりません。したがって、呼び出された関数内でこれらのレジスタを変更する場合、その関数はsave/restoreによりレジスタの内容を退避/復帰する必要があります。

●スクラッチレジスタ(R4〜R7)

式の計算結果やローカル変数が一時的に格納されます。これらのレジスタは関数呼び出し時に保存する必要はありません。

●グローバルポインタ(R8)

グローバルポインタを格納するために予約されています。gcc33ではR8レジスタを使用しません。

●拡張命令展開用スクラッチレジスタ(R9)

アセンブラ用のスクラッチレジスタで、命令エクステンダext33が拡張命令を展開する際に使用します。gcc33ではR9レジスタを使用しません。

●戻り値格納用レジスタ(R10, R11)

関数の戻り値を格納します。戻り値を格納する前はスクラッチレジスタとして使用されます。

●引数渡し用レジスタ(R12〜R15)

関数呼び出し時に引数を格納します。4ワードを越えた分はスタック内に格納して渡します。引数を格納する前はスクラッチレジスタとして使用されます。

注: **Cルーチンから呼び出されるアセンブラルーチンの中ではレジスタの使い方に注意してください。**

- R4〜R7**は自由に使用できます。内容の退避と復帰は不要です。**
- R10**と**R11**はリターン前に戻り値をセットするまでは自由に使用できます。**
- R12〜R15**は引数を使い終わった後は自由に使用できます。リターン前に復帰させる必要はありません。**
- R8**と**R9**はできるだけ使用しないでください。**
- R0〜R3**を使用する場合は、使用する前に必ず**pushn**命令で内容を退避させ、リターン前に**popn**命令で復帰させてください。**

6.5.4 関数呼び出し

●引数の渡し方

関数呼び出しの際、引数は4ワードまでが引数渡し用レジスタ(R12～R15)に、それを越える分は呼び出し側関数のスタックフレーム(次節で説明)に格納されて呼び出される関数に渡されます。

例: func(w1, w2, w3, w4, w5, w6); ...w1→R12, w2→R13, w3→R14, w4→R15, w5&w6→スタック
(wN: ワードサイズ以下の引数)

基本的には、引数の指定順にR12～R15に格納されます。

●引数のデータサイズ

4バイト以下のデータサイズの引数は、データ型にかかわらずワード(4バイト)単位で扱われます。char型、short型のデータは符号拡張、unsigned char型、unsigned short型はゼロ拡張されます。double型のみ8バイト単位で扱われます。double型引数を渡す場合、R12～R15に2つのレジスタの空きがなければスタック渡しとなります。

例: func(w1, d2, d3, w4); ...w1→R12, d2(L)→R13, d2(H)→R14, w4→R15, d3→スタック
(wN: ワードサイズ以下の引数, dN: double型の引数)

●構造体引数の扱い (注)

引数が構造体データの場合、構造体のメンバーの値はスタックを介して渡されます。

例:
```
struct _foo {
        int     a;
        short   b;
        char    c;
};

callee(struct _foo foo, int d);
```

上記例ではdのみが引数渡し用レジスタ(R12)に格納され、fooはすべてスタックに格納されます。

●構造体を返す関数への引数渡し (注)

構造体データを返す関数を呼び出す場合、結果を格納する構造体のアドレスが第1引数としてR12レジスタに格納され、関数に渡されます。したがって、ソース上で記述されている引数は1つずつ繰り下げられます。

戻り値となる構造体が使用されない場合、コンパイラはダミーの構造体データを呼び出し側関数のローカル変数領域に割り当て、そのアドレスを渡します。

呼び出された関数は第1引数で渡されたポインタを戻り値として返します。

●レジスタの退避

呼び出された関数内でR0～R3レジスタを変更する場合、その関数はsave/restoreにより呼び出された時点のレジスタの内容を退避/復帰する必要があります。

R4～R7およびR10～R15レジスタは、自由に使用することができます。

R8とR9レジスタは、命令エクステンダext33の処理と競合しない範囲で使用可能です。

●戻り値

ワードサイズ以下の戻り値はR10レジスタに格納されます。

ダブルワード(double)サイズの戻り値はR10(下位ワード)とR11(上位ワード)に格納されます。

注: **構造体をサブルーチンに渡したり、サブルーチンから構造体を返す場合、その都度構造体全体をmemcpy関数でコピーするという特殊な処理を行う実コードが生成されます。これはコードサイズや実行速度、あるいはコンパイラのバグの誘発という面からも望ましくありません。構造体はできるだけポインタで渡して処理することを推奨します。**

6.5.5 スタックフレーム

gcc33は関数呼び出し時に図6.5.5.1に示すスタックフレームを生成します。
スタックフレームの開始アドレスは常にワード(32ビット)境界アドレスとなります。

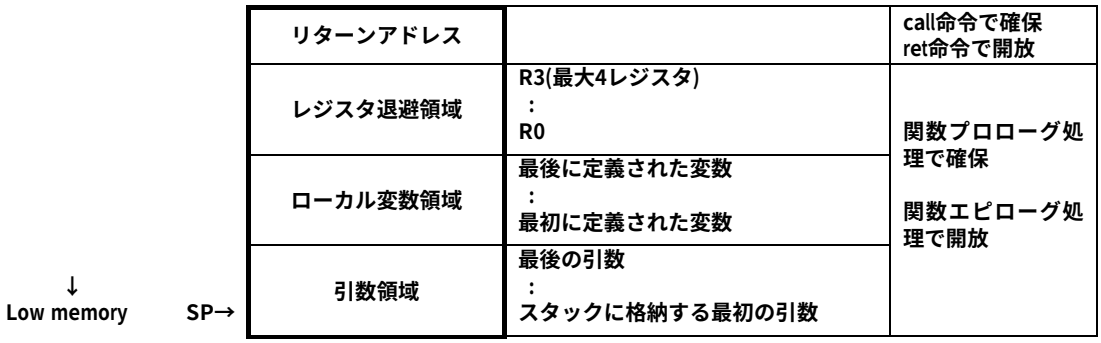


図6.5.5.1 スタックフレーム

●リターンアドレス

呼び出し側の関数へのリターンアドレス(1ワード)です。

●レジスタ退避領域

R0～R3レジスタの中で、呼び出し側の関数が使用しているレジスタがあれば、番号の大きい順に格納されます。
呼び出し側の関数がR0～R3レジスタを使用していない場合、この領域は確保されません。

●ローカル変数領域

呼び出された関数内で定義されているローカル変数で、レジスタに格納できないものについては、スタックフレーム内に領域を確保します。その後、最後に宣言された変数から順に、データ型に従った境界アドレス(double型は4バイト境界)に格納されます。

例: {

```
        char    c;
        short   s;
        int     i;
            :
}
```

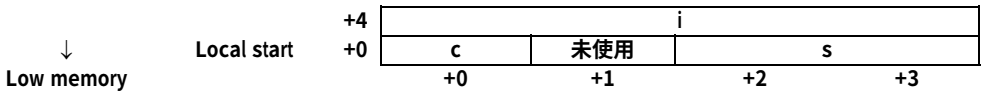


図6.5.5.2 ローカル変数の配置例

スタックに割り当てるローカル変数がない場合、この領域は確保されません。

●引数領域

呼び出された関数内にある別の関数呼び出しの引数の中で、引数渡し用レジスタに格納できないものについてスタックフレーム内に領域を確保します(前節参照)。引数はすべて4バイト境界に配置されます。double型の下位32ビットは下位側アドレスに、上位32ビットは上位側のアドレスに格納されます。
関数呼び出しがない場合、この領域は確保されません。

●領域の確保と開放

リターンアドレス用の領域確保と格納はcall命令が行い、取り出しと領域の開放はret命令が行います。
それ以外の領域は、関数のプロローグ処理で必要に応じて確保され、エピローグ処理で開放されます。

6.6　デバッグ情報

起動時オプションとして-g(wb33の[debug info]をチェック)が指定されると、gcc33はCソースレベルデバッグ、シンボリックデバッグに対応するためのアセンブラ擬似命令をデバッグ情報として出力ファイルに挿入します。

注:
- **デバッガdb33でCソースレベルデバッグ、シンボリックデバッグを行う場合にはこのデバッグ情報が必要です。**
- **デバッグ情報は-gオプションを指定する以外の方法で作成しないでください。また出力されたデバッグ情報を修正しないでください。as33、lk33、db33、dis33等の誤動作につながります。**
- **gcc33(pp33も同様)で-gオプションを指定しても、lk33で-gオプションを指定しないとデバッグ情報はリンクの際にすべてカットされます。**
- **gcc33で-gオプションを指定しても、as33で-gオプションを指定しないとシンボル情報がすべてカットされます。ソース情報はカットされません。**
- **gcc33で-gオプションを指定せずにas33で-gオプションを指定した場合、シンボル名とアドレスのみのシンボル情報がアセンブル時に付加されます。**

6.6.1　ソース情報

次の3種類のデバッグ擬似命令がデバッガでCソースを表示させるために出力されます。

1) .file　"<path_name>"

ファイルの開始を示します。ファイルの開始位置に挿入されます。<path_name>はファイルのパス名です。

2) .endfile

ファイルの終わりを示します。ファイルの最後に挿入されます。

3) .loc　<line_no>

ソースファイルの行情報を示します。Cソース行に対応するアセンブリコードの先頭に挿入されます。<line_no>はソースの行番号です。

6.6.2　シンボル情報

すべての関数や変数の情報が.def擬似命令として出力されます。.def擬似命令の形式は次のとおりです。

一般形) **.def　<symbol>, <parameter>, [<parameter>, ... ,]　endef**

以下、出力される.def擬似命令の内容をシンボルの種類別に示します。

●自動変数，構造体・共用体・enum型メンバー，引数

.def　<sym>, val <expr1>, scl <expr2>, type <expr3>, endef

<sym>	Cソース中のシンボル名(変数名/メンバー名)
<expr1>	自動変数/引数(スタック)：スタックポインタからのオフセット(10進数) 自動変数/引数(レジスタ)：レジスタ番号(10進数) 構造体・共用体メンバー：構造体・共用体の先頭からのオフセット(10進数) enum型メンバー：メンバーを示す値(10進数)
<expr2>	<sym>のストレージクラス(10進数)
<expr3>	<sym>のデータタイプ(16進数)

<sym>が自動変数または構造体・共用体・enum型のメンバーであることを示します。

●静的変数，グローバル変数，関数

.def　<sym1>, val <sym2>, scl <expr1>, type <expr2>, endef

<sym1>	Cソース中のシンボル名(変数名/メンバー名)
<sym2>	<sym1>に対応するリロケータブルシンボル名
<expr1>	<sym2>のストレージクラス(10進数)
<expr2>	<sym2>のデータタイプ(16進数)

<sym2>がCソースの変数名<sym1>に対応した関数、静的変数またはグローバル変数であることを示します。

●構造体・共用体・enum型のタグ宣言(開始)

.def <sym>, scl <expr1>, type <expr2>, size <expr3>, endef

<sym> Cソース中の構造体・共用体・enum型タグ名
<expr1> <sym>のストレージクラス(10進数)
<expr2> <sym>のデータタイプ(16進数)
<expr3> <sym>のデータサイズ(10進数)

タグ名が<sym>の構造体・共用体・enum型が宣言されており、メンバーの情報が次の.def擬似命令以降にあることを示します。

●ビットフィールドメンバー(構造体・共用体のメンバー)

.def <sym>, val <expr1>, scl <expr2>, type <expr3>, size <expr4>, endef

<sym> ビットフィールドメンバー名
<expr1> 構造体・共用体の先頭からのビットオフセット(10進数)
<expr2> <sym>のストレージクラス(10進数)
<expr3> <sym>のデータタイプ(16進数)
<expr4> <sym>のビットサイズ(10進数)

<sym>がビットフィールドメンバーであることを示します。

●構造体・共用体・enum型のタグ宣言(終了)

.def <sym1>, val <expr1>, scl <expr2>, tag <sym2>, size <expr3>, endef

<sym1,2> Cソース中の構造体・共用体・enum型タグ名
<expr1> <sym1>のデータサイズ(10進数)
<expr2> 102(固定)
<expr3> <sym1>のデータサイズ(10進数)

タグ名が<sym1>の構造体・共用体・enum型が宣言されており、メンバーの情報が直前の.def擬似命令までであることを示します。

●構造体・共用体・enum型変数(自動変数，引数，構造体・共用体メンバー)

.def <sym1>, val <expr1>, scl <expr2>, tag <sym2>, size <expr3>, type <expr4>, endef

<sym1> Cソース中のシンボル名(変数名/メンバー名)
<sym2> <sym1>が示す構造体・共用体・enum型変数のタグ名
<expr1> 自動変数/引数：スタックポインタからのオフセット(10進数)
構造体・共用体メンバー：構造体・共用体の先頭からのオフセット(10進数)
<expr2> <sym1>のストレージクラス(10進数)
<expr3> <sym2>の構造体・共用体・enum型のデータサイズ(10進数)
<expr4> <sym1>のデータタイプ(16進数)

<sym1>がタグ名<sym2>の構造体・共用体自動変数または構造体・共用体メンバーの構造体・共用体・enum型データであることを示します。

●構造体・共用体・enum型変数(静的変数，グローバル変数)

.def <sym1>, val <sym2>, scl <expr1>, tag <sym3>, size <expr2>, type <expr3>, endef

<sym1> Cソース中のシンボル名(変数名/メンバー名)
<sym2> <sym1>に対応するリロケータブルシンボル名
<sym3> <sym2>が示す構造体・共用体・enum型変数のタグ名
<expr1> <sym2>のストレージクラス(10進数)
<expr2> <sym3>の構造体・共用体・enum型のデータサイズ(10進数)
<expr3> <sym2>のデータタイプ(16進数)

<sym2>がCソースの変数名<sym1>に対応し、構造体・共用体・enum型の静的変数または構造体・共用体のグローバル変数であることを示します。

●配列(自動変数，引数，構造体・共用体メンバー)

.def <sym>, val <expr1>, scl <expr2>, dim <expr_list>, size <expr3>, type <expr4>, endef

<sym> Cソース中のシンボル名(変数名/メンバー名)
<expr1> 自動変数/引数：スタックポインタからのオフセット(10進数)
構造体・共用体メンバー：構造体・共用体の先頭からのオフセット(10進数)
引数(レジスタ渡し)：配列の先頭要素を格納するレジスタ番号(10進数)
<expr2> <sym>のストレージクラス(10進数)
<expr3> 配列のデータサイズ(10進数)
<expr4> 配列要素のデータタイプ(16進数)
<expr_list> 配列の次元を示す値のリスト(10進数、最大4次元)
例: int array[2][3] → dim 2 3

<sym>が自動配列変数または構造体・共用体メンバーの配列データであることを示します。

●配列(静的変数，グローバル変数)

.def <sym1>, val <sym2>, scl <expr1>, dim <expr_list>, size <expr2>, type <expr3>, endef

<sym1> Cソース中のシンボル名(変数名/メンバー名)
<sym2> <sym1>に対応するリロケータブルシンボル名
<expr1> <sym2>のストレージクラス(10進数)
<expr2> 配列のデータサイズ(10進数)
<expr3> 配列要素のデータタイプ(16進数)
<expr_list> 配列の次元を示す値のリスト(10進数、最大4次元)

<sym2>がCソースの変数名<sym1>に対応した静的配列変数またはグローバル配列変数であることを示します。

●構造体・共用体・enum型配列(自動変数，構造体・共用体メンバー)

.def <sym1>, val <expr1>, scl <expr2>, tag <sym2>, dim <expr_list>, size <expr3>, type <expr4>, endef

<sym1> Cソース中のシンボル名(変数名/メンバー名)
<sym2> <sym1>が示す構造体・共用体・enum型変数のタグ名
<expr1> 自動変数/引数：スタックポインタからのオフセット(10進数)
構造体・共用体メンバー：構造体・共用体の先頭からのオフセット(10進数)
<expr2> <sym1>のストレージクラス(10進数)
<expr3> <sym2>の構造体・共用体・enum型配列のデータサイズ(10進数)
<expr4> <sym1>の配列要素のデータタイプ(16進数)
<expr_list> 配列の次元を示す値のリスト(10進数、最大4次元)

<sym1>がタグ名<sym2>の構造体・共用体・enum型配列データであることを示します。

●構造体・共用体・enum型配列(静的変数，グローバル変数)

.def <sym1>, val <sym2>, scl <expr1>, tag <sym3>, dim <expr_list>, size <expr2>, type <expr3>, endef

<sym1> Cソース中のシンボル名(変数名/メンバー名)
<sym2> <sym1>に対応するリロケータブルシンボル名
<sym3> <sym2>が示す構造体・共用体・enum型変数のタグ名
<expr1> <sym2>のストレージクラス(10進数)
<expr2> <sym3>の構造体・共用体・enum型配列のデータサイズ(10進数)
<expr3> <sym2>の配列要素のデータタイプ(16進数)
<expr_list> 配列の次元を示す値のリスト(10進数、最大4次元)

<sym2>がCソースの変数名<sym1>に対応した静的配列変数またはグローバル配列変数であることを示します。

●typedef(標準の型を用いた場合)

.def <sym>, scl <expr1>, type <expr2>, endef

<sym> typedefで新たに定義されたデータタイプの名称
<expr1> 定義されたデータタイプのストレージクラス(10進数)
<expr2> 元にしたデータタイプ(16進数)

<sym>がtypedefにより新たなデータ型として定義されたことを示します。

●typedef(標準の型以外を用いた場合)

.def <sym1>, scl <expr1>, tag <sym2>, size <expr2>, type <expr3>, endef

<sym1> typedefで新たに定義されたデータタイプの名称
<sym2> 元にした構造体・共用体・enum型のタグ名
<expr1> 定義されたデータタイプのストレージクラス(10進数)
<expr2> <sym2>の構造体・共用体・enum型のデータサイズ(10進数)
<expr3> <sym2>の構造体・共用体・enum型のデータタイプ(16進数)

<sym1>がtypedefにより新たなデータ型として定義されたことを示します。

●ユーザー定義ラベル

.def <sym1>, val <sym2>, scl <expr1>, type <expr2>, endef

<sym1> ユーザー定義ラベル名
<sym2> ユーザー定義ラベルに対応するローカルラベル名
<expr1> ユーザー定義ラベルのストレージクラス(10進数)
<expr2> ユーザー定義ラベルのデータタイプ(16進数)

<sym1>がユーザー定義ラベルで、gcc33が生成したローカルラベル<sym2>に対応することを示します。

●関数・ブロックの開始と終了

.def <sym>, scl <expr>, type 0x0, endef

<sym> "ent"： 関数の開始
"end"： 関数の終了
"begin"： ブロックの開始
"bend"： ブロックの終了
<expr> 101： 関数の開始
111： 関数の終了
100： ブロックの開始
110： ブロックの終了

現在位置が関数・ブロックの開始または終了位置であることを示します。

●ストレージクラスとデータタイプの値

ストレージクラスとデータタイプの値は以下のように定義されています。

ストレージクラス(scl)の値

1 自動変数
2 グローバルシンボル(関数/変数)
3 ローカルシンボル(関数/変数)
4 レジスタ変数
6 ユーザー定義ラベル
8 構造体のメンバー
9 引数(スタック渡し)
10 構造体のタグ
11 共用体のメンバー
12 共用体のタグ
13 typedefで定義された型
15 enum型のタグ
16 enum型のメンバー
17 引数(レジスタ渡し)
18 ビットフィールド
100 ブロックの開始位置(begin)
101 関数の開始位置(ent)
102 構造体・共用体・enum型定義の終端
110 ブロックの終了位置(bend)
111 関数の終了位置(end)

データタイプ(type)の値

A		B	
0x0	ユーザー定義ラベル	0x0	ポインタ・関数・配列以外
0x1	void	0x1	ポインタ
0x2	char	0x2	関数
0x3	short	0x3	配列
0x4	int		
0x5	long		
0x6	float		
0x7	double		
0x8	struct		
0x9	union		
0xA	enum		
0xB	enumのメンバー		
0xC	unsigned char		
0xD	unsigned short		
0xE	unsigned int		
0xF	unsigned long		

データタイプの値は次のように計算します。
A + (B(1) << 4) + (B(2) << 6) .... + (B(N) << (2 + 2 * N))

たとえば、構造体へのポインタを返す関数は次の値になります。
0x8 + (0x2 << 4) + (0x1 << 6) = 0x68

6.7 gccの機能と注意事項について

・ calloc関数は使用できません。

・ その他、gccの詳細についてはgccに添付されているドキュメントを参照してください。ドキュメントは、世界各地にあるGNUのミラーサイトからインターネット等を利用して入手可能です。

7 エミュレーションライブラリ

この章では浮動少数点演算関数および整数剰余演算関数を含むエミュレーションライブラリについて説明します。

7.1 概要

本パッケージには、IEEE形式に準拠した単精度(32ビット)および倍精度(64ビット)浮動小数点数の四則演算、比較、型変換をサポートする浮動小数点演算ライブラリ(fp.lib)と、整数の剰余演算をサポートする整数剰余演算ライブラリ(idiv.lib)が含まれています。
Cコンパイラgcc33は、浮動小数点演算、整数剰余演算が行われると、これらのライブラリ内の関数を呼び出します。
ライブラリ関数は、既定の汎用レジスタを介してデータのやり取りを行いますので、アセンブリソースから呼び出すこともできます。
エミュレーションライブラリ関数を使用する場合は、リンク時にfp.lib、idiv.libをライブラリとして指定してください。指定は、fp.lib、idiv.libの順に行ってください。

エミュレーションライブラリ関数は、すべてアセンブリソースにより作成、チューニングされています。

7.2 浮動小数点演算ライブラリ(fp.lib)

7.2.1 関数一覧

表7.2.1.1に浮動小数点演算ライブラリ(fp.lib)関数一覧を示します。

表7.2.1.1 浮動小数点演算ライブラリ(fp.lib)関数一覧

分類	関数名	機能	
倍精度浮動小数点演算	__adddf3	加算	(%r11, %r10) ← (%r13, %r12) + (%r15, %r14)
	__subdf3	減算	(%r11, %r10) ← (%r13, %r12) - (%r15, %r14)
	__muldf3	乗算	(%r11, %r10) ← (%r13, %r12) * (%r15, %r14)
	__divdf3	除算	(%r11, %r10) ← (%r13, %r12) / (%r15, %r14)
	__negdf2	符号反転	(%r11, %r10) ← -(%r13, %r12)
単精度浮動小数点演算	__addsf3	加算	%r10 ← %r12 + %r13
	__subsf3	減算	%r10 ← %r12 - %r13
	__mulsf3	乗算	%r10 ← %r12 * %r13
	__divsf3	除算	%r10 ← %r12 / %r13
	__negsf2	符号反転	%r10 ← -%r12
型変換	__fixunsdfsi	double → unsigned int	%r10 ← (unsigned int) (%r13, %r12)
	__fixdfsi	double → int	%r10 ← (int) (%r13, %r12)
	__floatsidf	int → double	(%r11, %r10) ← (double) %r12
	__fixunssfsi	float → unsigned int	%r10 ← (unsigned int) %r12
	__fixsfsi	float → int	%r10 ← (int) %r12
	__floatsisf	int → float	%r10 ← (float) %r12
	__truncdfsf2	double → float	%r10 ← (float) (%r13, %r12)
	__extendsfdf2	float → double	(%r11, %r10) ← (double) %r12
浮動小数点比較	__fcmpd	double 型比較	%psr 変更 ← (%r13, %r12) - (%r15, %r14)
	__fcmps	float 型比較	%psr 変更 ← %r12 - %13

- 演算結果がオーバーフローまたはアンダーフローした場合は、無限大または-無限大(次節参照)を返します。
- 比較関数はop1 - op2の結果により、PSRのC、V、Z、Nを次のように変更します。他のフラグは変更されません。

比較結果	C	V	Z	N
op1 > op2	0	0	0	0
op1 = op2	0	0	1	0
op1 < op2	1	0	0	1

- 型変換時は0方向への丸めが行われます。

7.2.1 浮動小数点フォーマット

Cコンパイラgcc33はIEEE標準規格に準拠したfloat型(単精度浮動小数点、32ビット)とdouble型(倍精度浮動小数点、64ビット)をサポートしています。
浮動小数点数の内部形式を以下に示します。

●倍精度浮動小数点数のフォーマット

double型の実数は次のように64ビットで構成されます。

63	62　　52	51　　0
S	指数部	仮数部

ビット63:　　符号ビット(1ビット)
ビット62～52:　指数部(11ビット)
ビット51～0:　仮数部(52ビット)

レジスタに格納する場合は2つのレジスタを占有します。たとえば、浮動小数点演算の結果は次のようにR11およびR10レジスタに格納されます。
　R11レジスタ: 符号ビット、指数部および仮数部の上位20ビット(51:32)
　R10レジスタ: 仮数部の下位32ビット(31:0)

double型の有効範囲、浮動小数点表記と内部データの対応は次のとおりです。

+0:	**0.0e+0**	**0x00000000 00000000**
-0:	**-0.0e+0**	**0x80000000 00000000**
最大正規化数:	**1.79769e+308**	**0x7feffff ffffffff**
最小正規化数:	**2.22507e-308**	**0x00100000 00000000**
最大非正規化数:	**2.22507e-308**	**0x000fffff ffffffff**
最小非正規化数:	**4.94065e-324**	**0x00000000 00000001**
無限大:		**0x7ff00000 00000000**
-無限大:		**0xfff00000 00000000**

0x7ff00000 00000001～0x7fffffff ffffffffおよび0xfff00000 00000001～0xffffffff ffffffffは数値としては認められません。

●単精度浮動小数点数のフォーマット

float型の実数は次のように32ビットで構成されます。

31	30　　23	22　　0
S	指数部	仮数部

ビット31:　　符号ビット(1ビット)
ビット30～23:　指数部(8ビット)
ビット22～0:　仮数部(23ビット)

float型データは1つのレジスタに格納できます。
float型の有効範囲、浮動小数点表記と内部データの対応は次のとおりです。

+0:	**0.0e+0f**	**0x00000000**
-0:	**-0.0e+0f**	**0x80000000**
最大正規化数:	**3.40282e+38f**	**0x7f7fffff**
最小正規化数:	**1.17549e-38f**	**0x00800000**
最大非正規化数:	**1.17549e-38f**	**0x007fffff**
最小非正規化数:	**1.40129e-45f**	**0x00000001**
無限大:		**0x7f800000**
-無限大:		**0xff800000**

0x7f800001～0x7fffffffおよび0xff800001～0xffffffffは数値としては認められません。

●注意

無限大の扱い等を含め、IEEE準拠のFPUとは精度および機能的な差異があります。

7.3 整数剰余演算ライブラリ(idiv.lib)

表7.3.1に整数剰余演算ライブラリ(idiv.lib)関数一覧を示します。

表7.3.1 整数剰余演算ライブラリ(idiv.lib)関数一覧

分類	関数名	機能	
整数除算	__divsi3	符号付き 整数除算	%r10 ← %r12 / %r13
	__udivsi3	符号なし 整数除算	%r10 ← %r12 / %r13
剰余	__modsi3	符号付き モジュロ演算	%r10 ← %r12 % %r13
	__umodsi3	符号なし モジュロ演算	%r10 ← %r12 % %r13

・ これらの関数は入力レジスタの値をチェックしません。そのため、R13レジスタに0が設定されている場合、関数内のdiv0sまたはdiv0u命令でゼロ除算例外が発生します。

7.4 浮動小数点ライブラリ(fpp.lib)

本パッケージに含まれているfpp.libは、浮動小数点ライブラリfp.libと同一の関数で構成されています。ただし、以下の関数の演算精度がfp.libのものより上がっています。

__adddf3, __subdf3, __muldf3, __divdf3

fp.libでは仮数部の有効範囲以下の桁は切り捨てとなりますが、fpp.libの上記4つの関数では、有効範囲以下の桁も演算対象となり、四捨五入されて仮数部の最下位ビットに反映されます。
sin、cos、tan等の数学関数で、高い演算精度が必要となる場合に使用してください。

8 ANSIライブラリ

この章では本パッケージに含まれるANSIライブラリについて説明します。

8.1 概要

本パッケージには、ANSIライブラリが含まれています。
各関数はANSI標準の機能を持っています。ただし、組み込み型マイコンの仕様上、ファイル関連の関数の中にはダミーとなっているものもあります。
ANSIライブラリファイルの種類は次の5種類で、libディレクトリにインストールされます。
io.lib　lib.lib　math.lib　ctype.lib　string.lib

また、各関数に関する定義が記録されている以下のヘッダファイルがincludeディレクトリにインストールされます。
stdio.h　stdlib.h　time.h　math.h　errno.h　float.h　limits.h　ctype.h　string.h　stdarg.h

ライブラリの使用方法については、"cc33¥sample¥ansilib¥"ディレクトリ内のサンプルファイルを参照してください。
また、すべてのソースコードは"cc33¥utility¥lib_src"ディレクトリ内にあります。詳細はソースを参照し、必要に応じて修正してください。

注:　リンク時のライブラリ指定は、エミュレーションライブラリ(fp.lib、idiv.lib)も含め、次の順番で行ってください。
io.lib　lib.lib　math.lib　ctype.lib　string.lib　fp.lib　idiv.lib

io.libの中にはlib.libまたはmath.libの関数を呼び出す関数があります。またlib.libもmath.libの関数を呼び出します。ライブラリ間の参照は後から指定されるライブラリ内に対してのみ有効となりますので、上記と異なる順番で並べた場合はリンク時にワーニングとなる場合があります。

8.2 ANSIライブラリ関数一覧

以下にリエントラント欄の記号について説明します。

○ リエントラントな関数
× ノンリエントラントな関数
– ダミー関数(お客様さまのシステムに対応させた修正が必要な関数です。)
△ ノンリエントラントな関数(グローバル変数を参照しているもの、ダミー関数をコールしているものです。グローバル変数の変更がなく、お客さまが記述するread(),write()がリエントラントな関数である場合、リエントラントな関数として使えます。)

8.2.1 入出力関数(io.lib)

以下、io.libに含まれる入出力関数の一覧を示します。

表8.2.1.1 入出力関数一覧

ヘッダファイル: stdio.h

関数	機能	ﾘｴﾝﾄﾗﾝﾄ	備考
FILE *fopen(char *filename, char *mode);	ダミー	–	
FILE *freopen(char *filename, char *mode, FILE *stream);	ダミー	–	
int fclose(FILE *stream);	ダミー	–	
int fflush(FILE *stream);	ダミー	–	
int fseek(FILE *stream, long int offset, int orign);	ダミー	–	
long int ftell(FILE *stream);	ダミー	–	
void rewind(FILE *stream);	ダミー	–	
int fgetpos(FILE *stream, fpos_t *ptr);	ダミー	–	
int fsetpos(FILE *stream, fpos_t *ptr);	ダミー	–	
size_t fread(void *ptr, size_t size, size_t count, FILE *stream);	stdinから配列要素を入力	△	グローバル変数stdin, _iobを参照、read関数をコール
size_t fwrite(void *ptr, size_t size, size_t count, FILE *stream);	stdoutへ配列要素を出力	△	グローバル変数stdout, stderr, _iobを参照、write関数をコール
int fgetc(FILE *stream);	stdinから1文字入力	△	グローバル変数stdin, _iobを参照、read関数をコール
int getc(FILE *stream);	stdinから1文字入力	△	グローバル変数stdin, _iobを参照、read関数をコール
int getchar( );	stdinから1文字入力	△	グローバル変数stdin, _iobを参照、read関数をコール
int ungetc(int c, FILE *stream);	入力バッファに1文字押し戻す	×	グローバル変数stdin, stdout, stderr, _iobを参照、戻した文字が上書きされる
char *fgets(char *s, int n, FILE *stream);	stdinから文字列を入力	△	グローバル変数stdin, _iobを参照、read関数をコール
char *gets(char *s);	stdinから文字列を入力	△	グローバル変数stdin, _iobを参照、read関数をコール
int fputc(int c, FILE *stream);	stdoutに1文字出力	△	グローバル変数stdout, stderr, _iobを参照、write関数をコール
int putc(int c, FILE *stream);	stdoutに1文字出力	△	グローバル変数stdout, stderr, _iobを参照、write関数をコール
int putchar(int c);	stdoutに1文字出力	△	グローバル変数stdout, stderr, _iobを参照、write関数をコール
int fputs(char *s, FILE *stream);	stdoutへ文字列を出力	△	グローバル変数stdout, stderr, _iobを参照、write関数をコール
int puts(char *s);	stdoutへ文字列を出力	△	グローバル変数stdout, stderr, _iobを参照、write関数をコール
int remove(char *filename);	ダミー	–	
int rename(char *oldname, char *newname);	ダミー	–	
void setbuf(FILE *stream, char *buf);	ダミー	–	
int setvbuf(FILE *stream, char *buf, int type, size_t size);	ダミー	–	
FILE *tmpfile( );	ダミー	–	
char *tmpnam(char *buf);	ダミー	–	
int feof(FILE *stream);	ダミー	–	

関数	機能	ﾘｴﾝﾄﾗﾝﾄ	備考
int ferror(FILE *stream);	ダミー	ー	
void clearerr(FILE *stream);	ダミー	ー	
void perror(char *s);	stdoutへのエラー情報出力	×	グローバル変数stdout, stderr, _iobを参照、errnoを変更、write関数をコール
int fscanf(FILE *stream, char *format, ...);	stdinからの書式指定付き入力	×	グローバル変数stdout, _iobを参照、errnoを変更、read関数をコール
int scanf(char *format, ...);	stdinからの書式指定付き入力	×	グローバル変数stdout, _iobを参照、errnoを変更、read関数をコール
int sscanf(char *s, char *format, ...);	文字列からの書式指定付き入力	×	グローバル変数errnoを変更
int fprintf(FILE *stream, char *format, ...);	stdoutへの書式指定付き出力	△	グローバル変数stdout, stderr, _iobを参照、write関数をコール
int printf(char *format, ...);	stdoutへの書式指定付き出力	△	グローバル変数stdout, stderr, _iobを参照、write関数をコール
int sprintf(char *s, char *format, ...);	配列への書式指定付き出力	○	
int vfprintf(FILE *stream, char *format, va_list arg);	stdoutへの変換出力	△	グローバル変数stdout, stderr, _iobを参照、write関数をコール
int vprintf(FILE *stream, char *format, va_list arg);	stdoutへの変換出力	△	グローバル変数stdout, stderr, _iobを参照、write関数をコール
int vsprintf(char *s, char *format, va_list arg);	配列への変換出力	○	

注:　ファイルシステムは扱えません。stdin、stdoutのみです。stdinはread、stdoutはwrite関数が必要です。詳細は8.4節を参照してください。

8.2.2 ユーティリティ関数(lib.lib)

以下、lib.libに含まれるユーティリティ関数の一覧を示します。

表8.2.2.1 ユーティリティ関数一覧

ヘッダファイル: stdlib.h

関数	機能	ﾘｴﾝﾄﾗﾝﾄ	備考
void *malloc(size_t size);	領域の確保	×	グローバル変数errno, ansi_ucStartAlloc, ansi_ucEndAlloc, ansi_ucNxtAlcP, ansi_ucTblPtr, ansi_ulRowを変更
void *calloc(size_t elt_count, size_t elt_size);	配列領域の確保	×	メモリアロケート中から呼び出しのため不正
void free(void *ptr);	領域の解放	×	メモリアロケート中から呼び出しのため不正
void *realloc(void *ptr, size_t size);	領域サイズの変更	×	メモリアロケート中から呼び出しのため不正
int system(char *command);	ダミー	−	
void exit(int status);	プログラムの正常終了	○	exitを参照 後から呼び出された側で終了
void abort( );	プログラムの異常終了	○	exitを参照 後から呼び出された側で終了
int atexit(void (*func)(void));	ダミー	−	
char *getenv(char *str);	ダミー	−	
void *bsearch(void *key, void *base, size_t count, size_t size, int (*compare)(void *, void *));	バイナリサーチ	○	
void qsort(void *base, size_t count, size_t size, int (*compare)(void *, void *));	クイックソート	○	
int abs(int x);	絶対値を返す(int型)	○	
long int labs(long int x);	絶対値を返す(long型)	○	
div_t div(int n, int d);	int型除算	×	グローバル変数errnoを変更
ldiv_t ldiv(int n, int d);	long型除算	×	グローバル変数errnoを変更
int rand( );	擬似乱数を返す	×	グローバル変数seedを変更
void srand(unsigned int seed);	擬似乱数種の設定	×	グローバル変数seedを変更
long int atol(char *str);	文字列をlong型に変換	×	グローバル変数errnoを変更
int atoi(char *str);	文字列をint型に変換	×	グローバル変数errnoを変更
double atof(char *str);	文字列をdouble型に変換	×	グローバル変数errnoを変更
double strtod(char *str, char **ptr);	文字列をdouble型に変換	×	グローバル変数errnoを変更
long int strtol(char *str, char **ptr, int base)	文字列をlong型に変換	×	グローバル変数errnoを変更
unsigned long int strtoul(char *str, char **ptr, int base);	文字列をunsigned long型に変換	×	グローバル変数errnoを変更
int setjmp(jmp_buf *ptr);	非局所分岐	○	
void longjmp(jmp_buf *ptr, int status);	非局所分岐	○	

8.2.3 日付と時刻関数(lib.lib)

以下、lib.libに含まれる日付と時刻関数の一覧を示します。

表8.2.3.1 日付と時刻関数一覧

ヘッダファイル: time.h

関数	機能	ﾘｴﾝﾄﾗﾝﾄ	備考
clock_t clock( );	ダミー	−	
char *asctime(struct tm *ts);	ダミー	−	
char *ctime(time_t *timeptr);	ダミー	−	
double difftime(time_t ti, time_t t2);	ダミー	−	
struct tm *gmtime(time_t *t);	カレンダー時間を標準時間に変換	×	スタティック変数を変更
struct tm *localtime(time_t *t);	ダミー	−	
time_t mktime(struct tm *tmptr);	標準時間をカレンダー時間に変換	×	スタティック変数を変更
time_t time(time_t *tptr);	現在のカレンダー時間を返す	△	グローバル変数gm_secを参照

8.2.4 数学関数(math.lib)

以下、math.libに含まれる数学関数の一覧を示します。

表8.2.4.1 数学関数一覧

ヘッダファイル: math.h、errno.h、float.h、limits.h

関数	機能	ﾘｴﾝﾄﾗﾝﾄ	備考
double fabs(double x);	絶対値を返す(double型)	○	
double ceil(double x);	double型小数部の切り上げ	×	グローバル変数errnoを変更、ALR, AHRを破壊
double floor(double x);	double型小数部の切り捨て	×	グローバル変数errnoを変更、ALR, AHRを破壊
double fmod(double x, double y);	double型の剰余計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double exp(double x);	指数計算(e^x)	×	グローバル変数errnoを変更、ALR, AHRを破壊
double log(double x);	自然対数の計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double log10(double x);	常用対数の計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double frexp(double x, int *nptr);	浮動小数点数の仮数と指数を返す	×	グローバル変数errnoを変更、ALR, AHRを破壊
double ldexp(double x, int n);	仮数と指数から浮動小数点数を返す	×	グローバル変数errnoを変更、ALR, AHRを破壊
double modf(double x, double *nptr);	浮動小数点数の整数部と少数部を返す	×	グローバル変数errnoを変更、ALR, AHRを破壊
double pow(double x, double y);	x^Yを計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double sqrt(double x);	平方根を計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double sin(double x);	正弦(サイン)計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double cos(double x);	余弦(コサイン)計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double tan(double x);	正接(タンジェント)計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double asin(double x);	逆正弦(アークサイン)計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double acos(double x);	逆余弦(アークコサイン)計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double atan(double x);	逆正接(アークタンジェント)計算	×	ALR, AHRを破壊
double atan2(double y, double x);	y/xの逆正接(アークタンジェント)計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double sinh(double x);	双曲線正弦(ハイパボリックサイン)計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double cosh(double x);	双曲線余弦(ハイパボリックコサイン)計算	×	グローバル変数errnoを変更、ALR, AHRを破壊
double tanh(double x);	双曲線正接(ハイパボリックタンジェント)計算	×	ALR, AHRを破壊

注: Ver. 3以降の関数はVer. 1、Ver. 2に比べ高速化されています。

8.2.5 文字関数(string.lib)

以下、string.libに含まれる文字関数の一覧を示します。

表8.2.5.1 文字関数一覧

ヘッダファイル: string.h

関数	機能	ﾘｴﾝﾄﾗﾝﾄ	備考
char *memchr(char *s, int c, int n);	記憶域内の指定文字の位置を返す	○	
int memcmp(char *s1, char *s2, int n);	記憶域の比較	○	
char *memcpy(char *s1, char *s2, int n);	記憶域のコピー	○	
char *memmove(char *s1, char *s2, int n);	記憶域のコピー(オーバーラップ可能)	○	
char *memset(char *s, int c, int n);	記憶域に文字を設定	○	
char *strcat(char *s1, char *s2);	文字列の連結	○	
char *strchr(char *s, int c);	文字列内で最初に現れた指定文字の位置を返す	○	
int strcmp(char *s1, char *s2);	文字列の比較	○	
char *strcpy(char *s1, char *s2);	文字列のコピー	○	
size_t *strcspn(char *s1, char *s2);	文字列の先頭から指定文字(複数候補)が現れるまでの文字数を返す	○	
char *strerror(int code);	エラーメッセージの文字列を返す	○	
size_t strlen(char *s);	文字列の長さを返す	○	
size_t strncat(char *s1, char *s2, int n);	文字列の連結(文字数指定付き)	○	
int strncmp(char *s1, char *s2, int n);	文字列の比較(文字数指定付き)	○	
char *strncpy(char *s1, char *s2, int n);	文字列のコピー(文字数指定付き)	○	
char *strpbrk(char *s1, char *s2);	文字列内で最初に現れた指定文字(複数候補)の位置を返す	○	
char *strrchr(char *s, int c);	文字列内で最後に現れた指定文字の位置を返す	○	
size_t strspn(char *s1, char *s2);	文字列の先頭から指定文字(複数候補)以外が現れるまでの文字数を返す	○	
char *strstr(char *s1, char *s2);	指定文字列が最初に現れた位置を返す	○	
char *strtok(char *s1, char *s2);	文字列をトークンに分割	×	スタティック変数を変更

※ strerrorを除き、アセンブリソースで作成、チューニングされています。

8.2.6 文字種判定/変換関数(ctype.lib)

以下、ctype.libに含まれる文字関数の一覧を示します。

表8.2.6.1 文字種判定/変換関数一覧

ヘッダファイル: ctype.h

関数	機能	ﾘｴﾝﾄﾗﾝﾄ
int isalnum(char c);	文字種判定(10進数またはアルファベット)	○
int isalpha(char c);	文字種判定(アルファベット)	○
int iscntrl(char c);	文字種判定(制御文字)	○
int isdigit(char c);	文字種判定(10進数)	○
int isgraph(char c);	文字種判定(図形文字)	○
int islower(char c);	文字種判定(アルファベット小文字)	○
int isprint(char c);	文字種判定(印刷可能文字)	○
int ispunct(char c);	文字種判定(区切り文字)	○
int isspace(char c);	文字種判定(空白文字)	○
int isupper(char c);	文字種判定(アルファベット大文字)	○
int isxdigit(char c);	文字種判定(16進数)	○
int tolower(char c);	文字種変換(アルファベット大文字→小文字)	○
int toupper(char c);	文字種変換(アルファベット小文字→大文字)	○

8.2.7 可変引数マクロ(stdarg.h)

以下、stdarg.hに定義された可変引数マクロの一覧を示します。

8.2.7.1 可変引数マクロ一覧

ヘッダファイル: stdarg.h

マクロ	機能
void va_start(va_list ap, type lastarg);	可変個引数集の初期化
type va_arg(va_list ap, type);	実引数の値を返す
void va_end(va_list ap);	可変個引数関数からの正常復帰

8.3 グローバル変数の宣言と初期化

ANSIライブラリ関数は表8.3.1に示すグローバル変数を参照します。これらの変数はライブラリ内には定義されていませんので、Cソースプログラム内でライブラリ関数を呼び出す前に宣言し初期化してください。

表8.3.1 宣言が必要なグローバル変数

グローバル変数	初期設定	関連するヘッダファイル/関数

FILE _iob[FOPEN_MAX +1];
FOPEN_MAX=3,
stdio.h**にて定義**
標準入出力ストリーム用
ファイル構造体データ
	_iob[N]._flg = _UGETN;
	_iob[N]._buf = 0;
	_iob[N]._fd = N;
	(N=0~2)
	_iob[0]: stdin**用入力データ**
	_iob[1]: stdout**用出力データ**
	_iob[2]: stderr**用出力データ**
		stdio.h, smcvals.h
		fgets, fread, fscanf, getc, getchar, gets, scanf, ungetc, perror, fprintf, fputs, fwrite, printf, putc, putchar, puts, vfprintf, vprintf

FILE *stdin; **標準入出力ファイル構造体データ**iob[0]**へのポインタ**	stdin = &_iob[0];	stdio.h fgets, fread, fscanf, getc, getchar, gets, scanf, ungetc
FILE *stdout; **標準入出力ファイル構造体データ**iob[1]**へのポインタ**	stdout = &_iob[1];	stdio.h fprintf, fputs, fwrite, printf, putc, putch, puts, vfprintf, vprin
FILE *stderr; **標準入出力ファイル構造体データ**iob[2]**へのポインタ**	stderr = &_iob[2];	stdio.h fprintf, fputs, fwrite, printf, perror, putc, putchar, puts, vfprintf, vprintf
int errno; **エラー番号を格納する変数**	errno = 0;	errno.h fopen, freopen, fseek, fsetpos, perror, remove, rename, tmpfile, tmpnam, fprintf, printf, sprintf, vprintf, vfprintf, fscanf, scanf, sscanf atof, atoi, calloc, div, ldiv, malloc, realloc, strtod, strtol, strtoul acos, asin, atan2, ceil, cos, cosh, exp, fabs, floor, fmod, frexp, ldexp, log, log10, modf, pow, sin, sinh, sqrt, tan
unsigned int seed; **乱数の種を格納する変数**	seed = 1;	stdlib.h rand, srand
time_t gm_sec; **タイマー関数の**70**年**1**月**1**日** 0**時**0**分**0**秒からの秒数**	gm_sec = -1;	time.h time

※ これらのグローバル変数の宣言と初期化を行うソースファイル例については、sample¥ansilib¥ディレクトリ内のlib.cを参照してください。

ANSIライブラリ関数より参照されるグローバル変数のうち、malloc、calloc、realloc、freeの各関数で使用するグローバル変数は次の初期化関数をコールすることにより初期化されます。この関数はstdlib.hで宣言されています。

int ansi_InitMalloc(unsigned long START_ADDRESS, unsigned long END_ADDRESS);

START_ADDRESSには使用するメモリの先頭アドレスを、END_ADDRESSには終了アドレスを設定します。これらのアドレスは関数内で4バイト境界に調整されます。
実際の使用にあたっては、sample¥ansilib¥内のlib.cを参照してください。

この関数により、以下のグローバル変数が初期化されます。これらの変数はstdlib.hで宣言されています。

unsigned char *ansi_ucStartAlloc; ヒープ領域の先頭アドレスを指すポインタ
unsigned char *ansi_ucEndAlloc; ヒープ領域の終端アドレスを指すポインタ
unsigned char *ansi_ucNxtAlcP; 次に割り当てられる新規領域の先頭を指すアドレスポインタ
unsigned char *ansi_ucTblPtr; 次に割り当てられる管理領域の先頭を指すアドレスポインタ
unsigned long ansi_ulRow; 次に割り当てられる管理領域を指す行ポインタ

8.4 下位レベル関数

以下に示す3つの関数(read、write、_exit)はライブラリ関数が呼び出す下位レベル関数です。これらの関数はハードウェアに依存するため、ライブラリの中には用意されていません。ユーザープログラム上で定義してください。
なお、これらの関数を定義するCソースファイル例については、sample¥ansilib¥ディレクトリ内のsys.cを参照してください。

8.4.1 read関数

●read関数の内容

形式: **int read(int fd, char *buf, int nbytes);**

引数:
- int fd; 入力を示すファイル記述子
  ライブラリ関数から呼ばれる場合、0(stdin)が渡されます。
- char *buf; 入力データを格納するバッファへのポインタ
- int nbytes; 転送バイト数

機能: ユーザーが定義する入力用バッファからデータを最大nbytes分読み込み、bufで示されるバッファに格納します。

戻り値: 実際に入力用バッファから読み込んだデータのバイト数
入力用バッファが空(EOF)またはnbytesが0のときは、0を返します。
エラー発生時は、-1を返します。

read関数を呼び出すライブラリ関数:

直接コール:	fread, getc, _doscan	(_doscanはscanf系内部関数)
間接コール:	fgetc, fgets, getchar, gets	(getcをコール)
	scanf, fscanf, sscanf	(_doscanをコール)

●入力用バッファの定義

形式: **unsigned char READ_BUF[65];** (変数名は任意、サイズは65バイト固定)
unsigned char READ_EOF;

バッファの内容:
バッファの先頭(READ_BUF[0])には、入力されたデータのサイズ(1～Max. 64)が格納されます。0はEOFを示します。
入力データはREAD_BUF[1]以降に格納されます。
READ_EOFはすでにEOFになったかどうかを示すステータスを格納します。

●シミュレーテッドI/Oを利用する場合の注意

デバッガのシミュレーテッドI/O機能(詳細はデバッガの章を参照)を利用する場合、read関数内にデバッガが入力用バッファを更新するためのグローバルなラベル"READ_FLASH"を定義し、その位置で入力用バッファに新たなデータが読み込まれるように関数を作成してください。

8.4.2 write関数

●write関数の内容

形式: **int write(int fd, char *buf, int nbytes);**

引数: int fd; 出力を示すファイル記述子
ライブラリ関数から呼ばれる場合、1(stdout)または2(stderr)が渡されます。
char *buf; 出力データを格納するバッファへのポインタ
int nbytes; 転送バイト数

機能: bufで示されるバッファに格納されたデータを、ユーザーが定義する出力用バッファにnbytes分書き込みます。

戻り値: 実際に出力用バッファに書き込んだデータのバイト数
書き込みが正常に行われたときはnbytesを返します。
書き込みエラー発生時は、nbytes以外の数値を返します。

write関数を呼び出すライブラリ関数:

直接コール:	fwrite, putc, _doprint	(_doprintはprintf系内部関数)
間接コール:	fputc, fputs, putchar, puts	(putccをコール)
	printf, fprintf, sprintf, vprintf, vfprintf	(_doprintをコール)
	perror	(fprintfをコール)

●出力用バッファの定義

形式: **unsigned char WRITE_BUF[65];** (変数名は任意、サイズは65バイト固定)

バッファの内容:
バッファの先頭(WRITE_BUF[0])には、出力するデータのサイズ(1〜Max. 64)を格納します。0はEOFを示します。
出力データはWRITE_BUF[1]以降に格納します。

●シミュレーテッドI/Oを利用する場合の注意

デバッガのシミュレーテッドI/O機能(詳細はデバッガの章を参照)を利用する場合、write関数内にデバッガが出力用バッファを更新するためのグローバルなラベル"WRITE_FLASH"を定義し、その位置で出力用バッファ内のデータが出力されるように関数を作成してください。

8.4.3 _exit関数

●_exit関数の内容

形式: **void _exit(void);**

機能: プログラムの終了処理を行います。

引数/戻り値: なし

_exit関数を呼び出すライブラリ関数:
直接コール: abort, exit

9 プリプロセッサ

この章ではプリプロセッサpp33の機能を説明します。

9.1 機能

プリプロセッサpp33(以下pp33と記述)はアセンブリソースファイルを最初に処理するツールで、アセンブラas33に付加機能を提供するソフトウェアです。アセンブリソースファイルに記述された付加機能部分をアセンブル可能なニーモニックコードに展開し出力します。pp33が提供する機能は以下のとおりです。

・ マクロ定義とマクロ呼び出し
・ デファイン名の定義
・ 数値演算子
・ 他のファイルのインクルード(挿入)
・ 条件アセンブル
・ アセンブリソース表示用デバッグ情報の付加

9.2 入出力ファイル

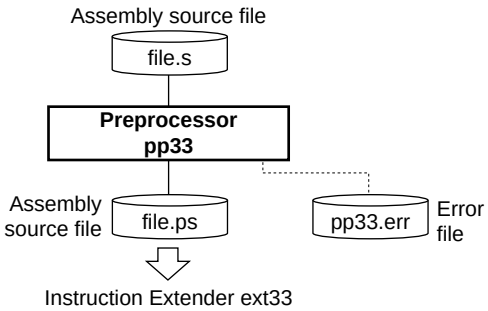


図9.2.1 **フローチャート**

9.2.1 入力ファイル

●ソースファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.s (".s"以外の拡張子も使用可能)
内容: アセンブリソースプログラムを記述したファイルです。プリプロセッサ用の命令および拡張命令が記述できます。

9.2.2 出力ファイル

●アセンブリソースファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.ps (<ファイル名>は入力ファイルと同じです。)
出力先: カレントディレクトリ
内容: プリプロセッサ用の命令がアセンブル可能な形式に展開されたファイルです。

●エラーファイル

ファイル形式: テキストファイル
ファイル名: pp33.err
出力先: カレントディレクトリ
内容: -eオプションが指定された場合に出力されるファイルで、エラーメッセージなど、pp33が標準出力(stdout)に対して出力する内容を記録します。

9.3 起動方法

9.3.1 起動フォーマット

●コマンドラインの一般形

pp33 ^ [<起動オプション>] ^ [<ファイル名>]

^はスペースを示します。
[ ]は省略可能なことを示します。

<ファイル名>: アセンブリソースファイルを拡張子も含めて指定します。

●ワークベンチからの起動

起動オプションとソースファイルを選択し、[PP33]ボタンをクリックします。

コマンドラインで一度に指定可能なソースファイルは1つのみです。
wb33では複数のファイルを選択可能で、ファイル数分pp33を実行します。

9.3.2 起動オプション

pp33には3種類の起動時オプションが用意されています。

-d <デファイン名>

機能: デファイン名の定義
wb33上の指定: [define]テキストボックスに入力
説明:
- ソースの先頭に"#define <デファイン名>"を記述したのと同じ働きをします。条件アセンブルを起動時に制御するためのオプションです。ただし、#define定義と異なり、ソース中の置き換えは行われません。
- -dと<デファイン名>の間には1個以上のスペースが必要です。
- 定義できるデファイン名の形式・制限については9.5.2節を参照してください。
- 2つ以上のデファイン名を定義する場合は、"-d <デファイン名>"の指定を繰り返してください。wb33の場合は<デファイン名>をカンマ(,)で区切って入力してください。

-g

機能: デバッグ情報の付加
wb33上の指定: [debug info]をチェック
説明:
- デバッグ情報を含む出力ファイルが生成されます。
- アセンブリソースレベルデバッグを行う場合は、必ず指定してください。
- デバッグ情報については9.7節を参照してください。

-e

機能: エラーファイルの出力
wb33上の指定: なし
説明:
- エラーメッセージなど、pp33が標準出力(stdout)に対して出力する内容をファイル(pp33.err)にも出力します。

コマンドラインにオプションを入力する場合、オプションの前後には1個以上のスペースが必要です。
例: c:¥cc33¥pp33 -g -e -d TEST1 -d TEST2 test.s

9.4 メッセージ

pp33はメッセージをすべて標準出力(stdout)に対して出力します。
wb33の[PP33]ボタンでpp33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

●終了メッセージ

正常に終了した場合は次の終了メッセージのみを出力します。

```
Pre Processor Completed
```

●Usage出力

ファイル名が指定されなかったり、オプション指定が正しくない場合、次の使用方法のメッセージを出力して終了します。

```
Pre Processor 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    pp33 [options] filename
Options:
    -e : produce log file (pp33.err)
    -g : generate debug information in output file
    -d string : define string
Output:
    Assembler source file for ext33 (.ps)
Example:
    pp33 -e -g -d TYPE1 test.s
```

●エラー・ワーニング発生時

エラーが発生した場合は、終了メッセージの前にエラーメッセージが表示されます。
例: **test.s(431): Error: Invalid Syntax.**
Pre Processor Completed

エラーの場合、pp33は出力ファイルを作成せずに終了します。

ワーニングが発生した場合は、終了メッセージの前にワーニングメッセージが表示されます。
例: **test.s(211): Warning: Multi define symbol.**
Pre Processor Completed

ワーニングの場合、pp33は出力ファイルを作成して終了します。

エラーとワーニングの詳細は"9.12 エラー/ワーニングメッセージ"を参照してください。

9.5 プリプロセッサ擬似命令

pp33の主要な目的は、以下に説明するプリプロセッサ擬似命令をアセンブラas33が処理可能なニーモニックに展開することです。
アセンブラ擬似命令と区別するため、プリプロセッサ擬似命令はすべてシャープ(#)で始まります。命令は行の先頭から記述してください。
擬似命令自体はすべて小文字のみです。パラメータは大文字と小文字の両方が使用でき、それぞれ区別されます。なお、プリプロセッサ擬似命令の行も、ステートメントの表記規則(4章参照)に従います。

9.5.1 インクルード命令(#include)

インクルード命令は、ソースファイルの任意の場所に他のファイルの内容を挿入します。同じソースを複数のソースファイルで共通に使用する場合などに有効です。

●命令の形式

#include "<ファイル名>"

・ <ファイル名>にはドライブ名、パス名も指定できます。
・ 命令と"<ファイル名>"の間には1個以上のスペースまたはタブが必要です。

記述例: #include "sample.def"

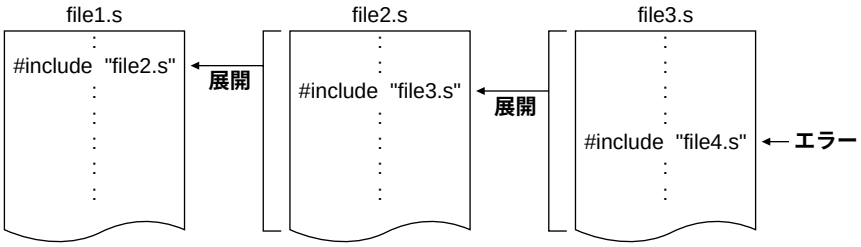
●展開規則

指定したファイルは#includeが記述された位置に挿入されます。
展開例は"9.11 実行例"を参照してください。

●注意事項

・挿入可能なファイル形式はテキストファイルのみです。

・ネスティングは2レベルまでです。これを越えるとエラーとなります。



"file2.s"**内に**"file3.s"**はインクルードできますが、**"file3.s"**に**"file4.s"**はインクルードできません。**

図9.5.1.1 **インクルードのネスティングレベル**

・ファイル名に相対パスを付加する場合、ファイルを挿入するソースファイルが存在するディレクトリからの相対パスを指定してください。

9.5.2 デファイン命令(#define)

デファイン命令(#define)によって任意の置換文字列をデファイン名として定義しておくことができ、その定義内容をプログラムの各所からデファイン名で参照することができます。

●命令の形式

#define <デファイン名> [<置換文字列>]

<デファイン名>:
- 1文字目はa～z、A～Z、_に限られます。
- 2文字目以降はa～z、A～Z、0～9、_が使用可能です。
- 大文字と小文字は区別されます。
- 命令とデファイン名の間には1個以上のスペースまたはタブが必要です。

<置換文字列>:
- 使用可能な文字はa～z、A～Z、0～9、_、%、.(ピリオド)に限られます。スペースやカンマ(,)を含むことはできません。
  数値、数値演算子、ニーモニック、ラベル、レジスタ名も指定可能です。
- 大文字と小文字は区別されます。
- デファイン名と置換文字列の間には1個以上のスペースまたはタブが必要です。
- 置換文字列は省略可能です。その場合は置換文字列としてNULLが定義されます。条件アセンブル命令に利用できます。

定義例:

```
#define    TYPE1
#define    L1        LABEL_01
#define    li        ld.w
#define    r1        %r1

#define    Mr1       [%r1]              ...エラー　[ ]や[ ]+は使用できません。
```

●展開規則

定義されたデファイン名がソース内に現れると、pp33はそのデファイン名を定義された文字列に置き換えます。

展開例:

```
#define    gp        %r8
  :
           ld.w      [gp], %r1          ..."ld.w　[%r8], %r1"に展開されます。
  :
```

置き換え文字列に数値を定義した場合、以下の規則が適用されます。
- 内部的には、すべて符号付き32ビットに変換されます。
  16進数として展開されます。
- 置き換え文字列には数値演算子を使用した式を記述することができます。式の項には定義済みのデファイン名も指定可能です。

展開例:

```
#define    A         0x12
#define    B         A*2

           ld.w      %r2, A+B           ..."ld.w　%r2, 0x36"に展開されます。
```

●注意事項

・pp33はデファイン名の後方参照のみを許可しています。定義は参照する前に行われている必要があります。

・一度定義したデファイン名を取り消すことはできません。ただし、定義されたデファイン名を使って再定義することができます。

```
例: #define    XH      %ahr
    #define    XHigh   XH
               ld.w    XHigh, %r1          ..."ld.w   %ahr, %r1"に展開されます。
```

・デファイン名を重複して定義した場合は、ワーニングメッセージが表示され、再定義された置き換え文字列が有効となります。

・[ ]、[ ]+で囲む場合を除き、ソース中のデファイン名の前後にはデリミタ(スペース、タブ、改行やカンマ)を除く文字を付加することはできません。ただし、後ろに記述するシンボルマスク(@..)は有効です。

```
例: #define    H       %ah
               ld.w    Hr, %r1     ;Hr = %ahr? ...このような指定は行えません。

    #define    L       LABEL
               ext     L@h                 ..."ext   LABEL@h"に置き換えられます。
```

・pp33は文字列を置き換えたことによるステートメントの妥当性はチェックしません。

・数値を定義した場合、内部的にはすべて32ビットとして扱われます。展開結果(指定値またはその計算結果)が負の場合はマイナス符号付きの10進数で、正の場合は16進数で出力されます。使用する際には、命令の即値サイズに注意してください。特に負(マイナス)の数値を扱う際には注意してください。

```
例: #define    NUM -1              ...内部的には-1は0xffffffffとなります。
               ld.w   %r1, NUM     ..."ld.w   %r1, -1"となります。
               ld.w   %r1, NUM^L   ..."ld.w   %r1, 0x3f"となります。
```

9.5.3 マクロ命令(#macro〜#endm)

マクロ命令(#macro)によって任意のステートメント列をマクロとして定義しておくことができ、その定義内容をプログラムの各所からマクロ名で呼び出すことができます。サブルーチンとは異なり、マクロを呼び出している箇所はpp33によって定義内容と置き換えられます。

●命令の形式

```
#macro      <マクロ名>  [$1] [,$2]・・・[,$32]
            <ステートメント列>
#endm
```

<マクロ名>:

- 1文字目はa〜z、A〜Z、_に限られます。
- 2文字目以降はa〜z、A〜Z、0〜9、_が使用可能です。
- 大文字と小文字は区別されます。
- 命令とマクロ名の間には1個以上のスペースまたはタブが必要です。

$1〜$32:

- マクロ定義用の仮パラメータシンボルで、定義するマクロにパラメータが必要な場合に記述します。最大32個まで指定可能です。
- $1〜$32以外のシンボルは使用できません。また、必ず$1から始め、昇順($1→$32)に並べてください。
- マクロ名と$1の間には1個以上のスペースまたはタブが必要です。また、複数のパラメータを記述する場合、それぞれの間にカンマ(,)が必要です。

<ステートメント列>:

- 以下のステートメントが記述できます。
  - -基本命令(ニーモニック&オペランド)
  - -拡張命令(命令エクステンダ参照)
  - -条件アセンブル命令
  - -分岐用内部ラベル*
  - -コメント
- 以下のステートメントは記述できません。
  - -プリプロセッサ擬似命令(条件アセンブル命令を除く)
  - -アセンブラ擬似命令
  - -分岐用内部ラベル以外のラベル定義
  - -マクロ呼び出し

＊ 分岐用内部ラベル

マクロはソース内の複数箇所に展開されます。このため、マクロ内にラベルを記述するとラベルの二重定義エラーとなります。そこで、マクロ内でのみ有効な分岐用内部ラベルを使用します。

- 1つのマクロに最大64個の分岐用内部ラベルが記述できます。
- ラベルは記述する順に$$1〜$$64とします。(マクロごとに$$1から始められます。)

定義例:

```
#define   Areg     %r1
#macro    ADD      $1, $2
          ld.w     Areg, $1
          add      Areg, $2
#ifdef    DEBUG
          cmp      Areg, 0x1
#else
          cmp      Areg, 0x2
#endif
          xjrne    $$1
          ld.w     [%r2], 0b11
$$1:
          ld.w     %r3, [%r2]+
          jr       LABEL1
#endm
```

●展開規則

定義されたマクロ名がソース内に現れると、pp33はその箇所に定義されたステートメント列を挿入します。
このとき実パラメータが記述されていれば、その順序に従って仮パラメータシンボル($1～$32)が実パラメータに置き換えられます。
分岐用内部ラベルは、ソースの先頭から現れる順序に従って、それぞれ__L0001～__L9999に置き換えられます。

展開例: 前記のマクロADDが定義されている場合

マクロ呼び出し

```
#define     DEBUG
              :
            ADD       1, 2
              :
```

展開後

```
              :
            ld.w      Areg, $1    ;     ADD   1, 2
            add       Areg, $2
;#ifdef     DEBUG
            cmp       Areg, 0x1
;#else
;           cmp       Areg, 0x2
;#endif
            xjrne     __L0001
            ld.w      [%r2], 0b11
__L0001:
            ld.w      %r3, [%r2]+
            jr        LABEL1
```

("__L0001"はソース内で初めて分岐用内部ラベルが展開された場合です。)

●注意事項

- pp33のマクロ呼び出しは後方参照のみを許可しています。定義は呼び出す前に行われている必要があります。
- 一度定義したマクロ名を取り消すことはできません。マクロ名を重複して定義した場合は、ワーニングメッセージが表示され、再定義されたマクロが有効となります。
- [ ]、[ ]+で囲む場合を除き、定義するステートメント中の仮パラメータの前後にはデリミタ(スペース、タブ、改行やカンマ)を除く文字を付加することはできません。ただし、後ろに記述するシンボルマスク(@..)は有効です。
- マクロ名にデファイン名と同一の文字列を使用することはできません。
- 仮パラメータ数と実パラメータ数が異なる場合、エラーとなります。
- 1つのマクロで指定可能なパラメータ数は最大32個、分岐用内部ラベルは最大64個です。これを越えるとエラーとなります。
- 分岐用内部ラベルに使用される"__L####"は他のラベルやシンボルとしては使用しないでください。
- 1つのソースファイル内で展開可能な分岐用内部ラベルは最大9999個です。これを越えるとエラーとなります。

9.5.4 条件アセンブル命令(#ifdef〜#else〜#endif, #ifndef〜#else〜#endif)

条件アセンブル命令は、指定する名前(デファイン名)の定義の有無によって指定範囲のアセンブルを行うかどうかを決定します。

●命令の形式

形式1)
```
#ifdef      <名前>
            <ステートメント列1>
[#else
            <ステートメント列2>]
#endif
```

<名前>が定義されていれば<ステートメント列1>をアセンブルの対象とします。
<名前>が定義されておらず、#else〜<ステートメント列2>が記述されている場合は<ステートメント列2>をアセンブルの対象とします。#else〜<ステートメント列2>は省略可能です。

形式2)
```
#ifndef     <名前>
            <ステートメント列1>
[#else
            <ステートメント列2>]
#endif
```

<名前>が定義されていなければ<ステートメント列1>をアセンブルの対象とします。
<名前>が定義されており、#else〜<ステートメント列2>が記述されている場合は<ステートメント列2>をアセンブルの対象とします。#else〜<ステートメント列2>は省略可能です。

<名前>:
- デファイン名の制限に従います。(#defineを参照)

<ステートメント列>:
- 条件アセンブル命令を除くすべてのステートメントが記述できます。

記述例:
```
#ifdef      TYPE1
            ld.w    %r1, 0x12
#else
            ld.w    %r1, 0x13
#endif
```

●名前の定義

名前の定義は、条件アセンブル命令が実行される前に次の2つの方法のいずれかで行っておく必要があります。

1) pp33の起動時オプション(-d)を使用して定義
   例: pp33 -d TYPE1 sample.s
2) デファイン名定義命令(#define)を使用してソースファイル内に定義
   例: #define TYPE1

#defineステートメントは、そのデファイン名を使用する条件アセンブル命令の前であればインクルードされるファイル内でも有効です。条件アセンブル命令より後に定義された名前は、未定義とみなされます。

条件アセンブルにのみ使用する場合は、置換文字列を指定する必要はありません。

●展開規則

アセンブル対象となったステートメント列は、他のプリプロセッサ命令の展開規則に従って展開されます。(プリプロセッサ命令を含んでいない場合は、そのままの状態でファイルに出力されます。)
アセンブル対象とならないステートメント列はすべてコメントとして出力されます。

●注意事項

- 条件に指定する名前は大文字と小文字を区別して評価されます。同一のデファイン名が定義されている場合のみ条件成立とみなされます。
- 条件アセンブル命令内のステートメント列に、#ifdef(#ifndef)命令は使用できません。#define、#macro、#include命令は使用できます。

9.6 数値演算子

即値の指定には、数値と数値演算子を組み合わせた式を使用することができます。
式は内部的に、すべて符号付き32ビットとして処理されます。
数値、演算子間にスペース等は許されていません。連続して記述してください。

●演算子の種類

		例
+	加算、単項プラス	+0xff, 1+2
-	減算、単項マイナス	-1+2, 0xfff-0b111
*	乗算	0xf*5
/	除算	0x123/0x56
%%	剰余	0x123%%0x56
>>	右シフト	1>>2
<<	左シフト	0x113<<3
&	論理積	0b1101&0b111
|	論理和	0x123|0xff
^	排他的論理和	12^35
~	論理否定	~0x1234
^H	ビットフィールド(31:19)の取得	0x1234^H
^M	ビットフィールド(18:6)の取得	0x1234^M
^L	ビットフィールド(5:0)の取得	0x1234^L
^AH	ビットフィールド(25:13)の取得	0x1234^AH
^AL	ビットフィールド(12:0)の取得	0x1234^L
( )	カッコ	1+(1+2*5)

●優先順位

演算子は、以下のような優先順位となっています。同じ優先順位の場合は、式の左より計算します。

1. ( )　　優先順位が高い
2. +(単項プラス)、-(単項マイナス)、~　　↑
   ^H、^M、^L、^AH、^AL
3. *、/、%%
4. +、-
5. <<、>>
6. &
7. ^　　↓
8. |　　優先順位が低い

●演算式の項

演算式の項には以下の内容が記述可能です。

- 32ビットの有効範囲内の2進数、10進数、16進数
  符号なし整数: 0～4294967295(0x0～0xffffffff)
  符号付き整数: 0～2147483647(0x0～0x7fffffff)、-1～-2147483648(0xffffffff～0x80000000)
- 数値を定義したデファイン名(#defineで定義済みの名前)
- シンボル
  シンボルがデファイン名以外の場合、演算式は次の形式に制限されます。
  SYMBOL [+SYMBOL...] + 数値式　または　SYMBOL [+SYMBOL...] - 数値式

●使用例

```
#define    BAR     0x1
           ld.w    %r0, BAR+2          ...ld.w   %r0, 0x3
           xcall   LABEL+BAR*2         ...xcall  LABEL+0x2
           xld.w   %r1, [FOO+BAR+1]    ...xld.w  %r1, [FOO+0x2]
           xld.w   %r1, [BAR+FOO+1]    ...FOOがデファイン名以外の場合はエラー
```

●注意事項

・内部的な演算は符号付き32ビットとして行われますので、演算の種類によっては注意が必要です。
16進数を使った>>、/、%%の計算結果には注意してください。

```
例: #define NUM1 0xfffffffe/2        ...-2/2 = -1 (0xffffffff) となります。
                                         /と%%は符号付き32ビットの範囲でしか使用できません。
    #define NUM2 0xfffffffe>>1       ...-2>>1 = -1 (0xffffffff) となります。
                                         (0xfffffffe>>1)&0x7fffffffのようにマスクしてください。
```

・演算結果が負の場合はマイナス符号付きの10進数で、正の場合は16進数で出力されます。

```
例: add   %r0, -2+1                 ... "add   %r0, -1"となります。
    add   %r0,(-2+1)&0x3f           ... "add   %r0, 0x3f"となります。
```

・演算子、数値、デファイン名の間には、スペース、タブ等を入れないでください。

```
例: ld.w   %r0, 1+1                 ...OK
    ld.w   %r0, 1 + 1               ...NG
    ld.w   %r0, (1+NUM1)*2          ...OK
    ld.w   %r0, (1 + NUM1)*2        ...NG
```

9.7 デバッグ情報

起動時オプションとして-g(wb33の[debug info]をチェック)が指定されると、pp33はアセンブリソース表示デバッグに対応するためのアセンブラ擬似命令をデバッグ情報として出力ファイルに挿入します。

注:
- **デバッガdb33でアセンブリソースを表示させたデバッグを行う場合にはこのデバッグ情報が必要です。**
- **デバッグ情報は-gオプションを指定する以外の方法で作成しないでください。また出力されたデバッグ情報を修正しないでください。as33、lk33、db33、dis33等の誤動作につながります。**
- **pp33(gcc33も同様)で-gオプションを指定しても、lk33で-gオプションを指定しないとデバッグ情報はリンクの際にすべてカットされます。**
- **pp33で-gオプションを指定して作成したソース情報は、as33で-gオプションを指定しない場合でもカットされません。**
- **アセンブラレベルのシンボル情報(シンボル名とアドレスのみ)はas33で-gオプションを指定した場合に生成されます。**

●出力されるアセンブラ擬似命令

次の3種類のデバッグ擬似命令が出力されます。
アンダーライン部以外の文字は固定です。

1) .file "*PATH_NAME*"
ファイルの開始を示します。現在のファイルの先頭、インクルードしたファイルの開始位置に挿入されます。*PATH_NAME*はファイルのパス名です。

2) .endfile
ファイルの終わりを示します。現在のファイルの最後に挿入されます。
インクルードファイルの終了位置には、元のファイルの再開を示す.file擬似命令が挿入され、.endfile擬似命令は挿入されません。

3) .loc *LINE_NO*
ソースファイルの行情報を示します。ニーモニックステートメント(オブジェクトコードにアセンブルされる行)にのみ付加されます。*LINE_NO*はソースの行番号です。

●出力例

起動コマンド: pp33 -g base_file.s

処理前:

・ソースファイル"base_file.s"

```
; file start
#include    "inc.def"
            ld.w        %r1, [%r7]
            ld.w        [%r3], %r1
```

・インクルードファイル"inc.def"

```
; This is an empty file.
```

処理後:

・アセンブリソースファイル"base_file.ms"

```
.file       "base_file.s"                   "base_file.s"の開始
; file start                                (コメントにデバッグ情報は付加されません。)
;#include   "inc.def"
.file       "inc.def"                       "inc.def"の開始
; This is an empty file.
.file       "base_file.s"                   "base_file.s"の再開
.loc        3                               行番号3(base_file.s)
            ld.w        %r1, [%r7]
.loc        4                               行番号4(base_file.s)
            ld.w        [%r3], %r1
.endfile                                    "base_file.s"の終了
```

9.8 コメント付加機能

プリプロセッサの命令は、すべてアセンブル可能なコードに展開され、ファイルに出力されます。その後でも最初に書かれていた命令がわかるように、それらの命令はセミコロン(;)で始まるコメントに置き換えられます。ただし、デファイン名の置き換え、数値の演算式については、コメントとして残りません。

コメントは展開後の最初の行の後ろに付加されます。元のステートメントにコメントが付いていた場合は、そのコメントも含めて付加します。
マクロ定義などは行の先頭にセミコロン(;)を挿入します。

例: 展開前

```
#define      R0       %r0

#macro       ADDM     $1
             ld.w     R0, $1
             add      R0, [%r4]
             ld.w     [%r5], R0
#endm

             ADDM     0x10        ; [%r5] = [%r4] + 0x10
```

展開後(デバッグ情報なし)

```
;#define     R0       %r0

;#macro      ADDM     $1
;            ld.w     R0, $1
;            add      R0, [%r4]
;            ld.w     [%r5], R0
;#endm

             ld.w     %r0, 0x10  ;        ADDM   0x10        ; [%r5] = [%r4] + 0x10
             add      %r0, [%r4]
             ld.w     [%r5], %r0
```

9.9 その他の機能

9.9.1 ASCII文字の16進数への変換

pp33は入力ソースファイル内の' 'で囲まれた1文字のASCII文字を16進数に変換する機能を持っています。出力されるアセンブリソースファイルの該当部分は、16進数のASCIIコードに置き換えられます。

例:

```
ld.w   %r1, '1'     →   ld.w   %r1, 0x31
ld.w   %r1, '1'+1   →   ld.w   %r1, 0x32
```

注: 記述可能なASCII文字は1文字のみです。
また、'\t'、'\n'も0x9、0xAとして使用できます。

9.9.2 コメント行

pp33はセミコロン(;)によるコメント行の指定に加え、//または/*で始まる行もコメントとみなします。
これらの文字の先頭の/は、;に変換されます。

例:

```
//comment.........   →   ;/comment.........
/*comment.........   →   ;*comment..........
```

9.10 処理の流れ

pp33の命令の処理の流れを以下に示します。

1. 条件アセンブル命令(#ifdef、#ifndef)の条件が不成立のステートメント列をスキップします。
2. コメント、.ascii擬似命令はそのまま出力します。
3. 各行をトークンに置き換え、デファイン名をすべて#defineによる定義内容に置き換えます。
   このとき、スペース、タブ、;、[、]、@、,、数値演算子がデリミタとみなされます。
4. 数値と数値演算子による式を計算し、式を計算結果で置き換えます。
5. #macro、#include等のプリプロセッサ擬似命令を処理します。

9.11 実行例

●入力ソースファイル(pp.s)

```
; pp.s     1997.2.20
; sample source for pp33

#include "pp.def"                       ; include file
#define SP_IRAM                         ; definition for #ifdef
#ifdef SP_IRAM                          ; condition assemble
          #define SP_INIT_ADDR 0x400    ; set number to defnum symbol
#else
          #define SP_INIT_ADDR 0x880000
#endif
#define BLK_ADDR  0x0+0x10              ; defnum symbol can use with arithmetic operators
                                        ; operators : +,-,*,/,%%,>>,<<,&,!,^,~,^H,^M,^L,^AH,^AL,(,)
#define gpr %r8                         ; define replace define symbol to string
#define GP_INIT_ADDR 0x0                ; define treat 0x0 as string, not number

#macro  FILL_AREA $1 $2 $3              ; macro argument is $1, $2, --- $32
          xld.w     %r1,$1              ; $1 is start address
          xld.w     %r2,$2              ; $2 is fill pattern (8bit)
          xld.w     %r3,$3              ; $3 is fill size (byte address)
$$1:                                    ; $$1 -- $$64 is local jump label
          cmp       %r3,0
          jreq      $$2
          ld.b      [%r1]+,%r2
          sub       %r3,1
          jp        $$1
$$2:
#endm

          .word BOOT
BOOT:
          ext       SP_INIT_ADDR^H
          ext       SP_INIT_ADDR^M
          ld.w      %r0,SP_INIT_ADDR^L
          ld.w      %sp,%r0
          ld.w      gpr,GP_INIT_ADDR
          FILL_AREA BLK_ADDR 0b01010101 10        ; fill 0x10-0x1f with 0x55
          FILL_AREA BLK_ADDR+0x10 0 10            ; fill 0x20-0x2f with 0x00
          jp        BOOT
```

●インクルードファイル(pp.def)

```
; pp.def
; This is empty file
```

●出力ファイル(pp.ms) "pp33 -g pp.s"を実行した場合

```
          .file     "pp.s"
; pp.s    1997.2.20
; sample source for pp33

;#include "pp.def"                        ; include file
          .file     "pp.def"
; pp.def
; This is empty file
          .file     "pp.s"
;#define SP_IRAM                          ; definition for #ifdef
;#ifdef SP_IRAM                           ; condition assemble
;         #define SP_INIT_ADDR 0x400       ; set number to defnum symbol
;#else
;         #define SP_INIT_ADDR 0x880000
;#endif
;#define BLK_ADDR  0x0+0x10               ; defnum symbol can use with arithmetic operators
                                          ; operators : +,-,*,/,%%,>>,<<,&,!,^,~,^H,^M,^L,^AH,^AL,(,)
;#define gpr %r8                          ; define replace define symbol to string
;#define GP_INIT_ADDR 0x0                 ; define treat 0x0 as string, not number

;#macro  FILL_AREA $1 $2 $3               ; macro argument is $1, $2, --- $32
;         xld.w     %r1,$1                 ; $1 is start address
;         xld.w     %r2,$2                 ; $2 is fill pattern (8bit)
;         xld.w     %r3,$3                 ; $3 is fill size (byte address)
;$$1:                                      ; $$1 -- $$64 is local jump label
;         cmp       %r3,0
;         jreq      $$2
;         ld.b      [%r1]+,%r2
;         sub       %r3,1
;         jp        $$1
;$$2:
;#endm

          .word BOOT
BOOT:
          .loc      31
          ext       0x0
          .loc      32
          ext       0x10
          .loc      33
          ld.w      %r0,0x0
          .loc      34
          ld.w      %sp,%r0
          .loc      35
          ld.w      %r8,0x0
          .loc      36
          xld.w     %r1,0x10               ; $1 is start address;         FILL_AREA  BLK_ADDR 0b01010101
10        ; fill 0x10-0x1f with 0x55
          xld.w     %r2,0b01010101         ; $2 is fill pattern (8bit)
          xld.w     %r3,10                 ; $3 is fill size (byte address)
__L0001:                                  ; $$1 -- $$64 is local jump label
          cmp       %r3,0
          jreq      __L0002
          ld.b      [%r1]+,%r2
          sub       %r3,1
          jp        __L0001
__L0002:
          .loc      37
          xld.w     %r1,0x20               ; $1 is start address;         FILL_AREA  BLK_ADDR+0x10  0  10
                    ; fill 0x20-0x2f with 0x00
          xld.w     %r2,0                  ; $2 is fill pattern (8bit)
          xld.w     %r3,10                 ; $3 is fill size (byte address)
__L0003:                                  ; $$1 -- $$64 is local jump label
          cmp       %r3,0
          jreq      __L0004
          ld.b      [%r1]+,%r2
          sub       %r3,1
          jp        __L0003
__L0004:
          .loc      38
          jp        BOOT
          .endfile
```

9.12 エラー/ワーニングメッセージ

エラーおよびワーニングメッセージは標準出力(stdout)に表示/出力されます。-eオプションを指定した場合は、ファイル"pp33.err"にも出力されます。
wb33の[PP33]ボタンでpp33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

9.12.1 エラー

pp33で発生するエラーは、システムエラーとプリプロセッサエラーの2つに分類されます。
システムエラーが発生した場合、pp33はエラーメッセージを表示して、直ちに処理を終了します。アセンブリソースファイルは出力されません。

表9.12.1.1 システムエラーメッセージ一覧

エラーメッセージ	内容
Error: Cannot open file.	ソースファイル、インクルードファイルがオープンできません。ファイルが指定のディレクトリに存在しません。
Error: Cannot allocate memory.	メモリが確保できません。
Error: Cannot open output file.	出力ファイルがオープンできません。
Error: Cannot open working file.	作業用の一時ファイルがオープンできません。
Error: Cannot write file.	ファイルへの書き込みができません。
Error: Cannot close file.	ファイルをクローズできません。
Error: Cannot read file. Line size is too long.	ステートメントが長すぎて読み込めません。 各行で読み込み可能な文字数は最大 255 文字です。
Error: Preprocessor limit: macro label number full.	マクロ展開時に、分岐用内部ラベル数が制限を越えました。 インクルードファイルも含め、一度に展開可能な分岐用内部ラベル数は 9999 個までです。

プリプロセッサエラーは、ソース中にpp33が処理できない構文や記述があった場合に発生します。このエラーが発生しても、システムエラーが発生しない限り入力ファイルの最終行まで処理は継続されます。ただし、アセンブリソースファイルは出力されません。

表9.12.1.2 プリプロセッサエラーメッセージ一覧

エラーメッセージ	内容
<file name>(line No.): Error: Invalid syntax.	構文エラーです。プリプロセッサ擬似命令が間違った形式で記述されています。
<file name>(line No.): Error: Nesting level too deep.	#include 擬似命令のネスティングが制限(2 レベル)を越えています。
<file name>(line No.): Error: Unknown preprocessor instruction.	不明なプリプロセッサ命令です。
<file name>(line No.): Error: Too many macro parameters.	33 個以上のマクロ仮パラメータが定義されています。
<file name>(line No.): Error: Invalid macro parameter.	仮パラメータの並びが異常です。または、実パラメータ数が仮パラメータ数と異なります。仮パラメータは$1〜$32 までで、$1 から順次並べてください。
<file name>(line No.): Error: Invalid macro label.	マクロ定義内の分岐用内部ラベルが異常です。 分岐用内部ラベルは$$1〜$$64(64 個)までで、$$1 から順次並べてください。
<file name>(line No.): Error: Invalid expression.	数値演算子の記述形式が不正です。
<file name>(line No.): Error: Multi symbol.	デファイン名に同じ名前が定義されています。#define 擬似命令によって同じ名前で多重定義されました。

9.12.2 ワーニング

ワーニングの場合、pp33は処理を継続します。他のエラーが発生しなければ、ワーニングメッセージを表示して処理を終了します。アセンブリソースファイルも出力されます。

表9.12.2.1 ワーニングメッセージ一覧

ワーニングメッセージ	内容
<file name>(line No.): Warning: Multi define symbol.	同じマクロ名またはデファイン名が複数定義されています。 再定義された内容が有効になります。 なお、デファイン定義とマクロ定義で同じ名称を使用するとデファイン名が優先され、ワーニングとはなりません。

9.13 注意事項

(1) pp33はプリプロセスに必要な文法チェックしか行いません。展開後の妥当性も含め、ニーモニック、オペランド、拡張命令、アセンブラ擬似命令のチェックは行いませんので注意してください。

(2) db33でアセンブリソースを表示させたデバッグを行う場合には必ず-gオプションを指定してas33を実行してください。
また、lk33で-gオプションを指定しないとデバッグ情報はリンクの際にすべてカットされます。

デバッグ情報は-gオプションを指定する以外の方法で作成しないでください。また出力されたデバッグ情報を修正しないでください。as33、lk33、db33、dis33等の誤動作につながります。

10　命令エクステンダ

この章では命令エクステンダext33の機能を説明します。

10.1　機能

命令エクステンダext33(以下ext33と記述)はCコンパイラgcc33およびプリプロセッサpp33が生成したアセンブリソースファイルを処理するツールで、アセンブリソースファイルに記述された拡張命令をアセンブル可能なニーモニックコードに展開し出力します。ext命令による即値拡張や複数の命令で記述する内容を拡張命令1つで記述でき、アセンブリソースを作成する場合に、即値サイズの制限などを意識せずにプログラミングが行えます。
また、ext33は次に示す2つの最適化機能を起動オプションで指定することができます。

・ 不要なext命令を削除する最適化
　　リンク後のシンボル情報を元にした最適化も行えます。
・ グローバルポインタを使用した最適化
　　グローバル変数を参照するための命令数が削減できます。

10.2　入出力ファイル

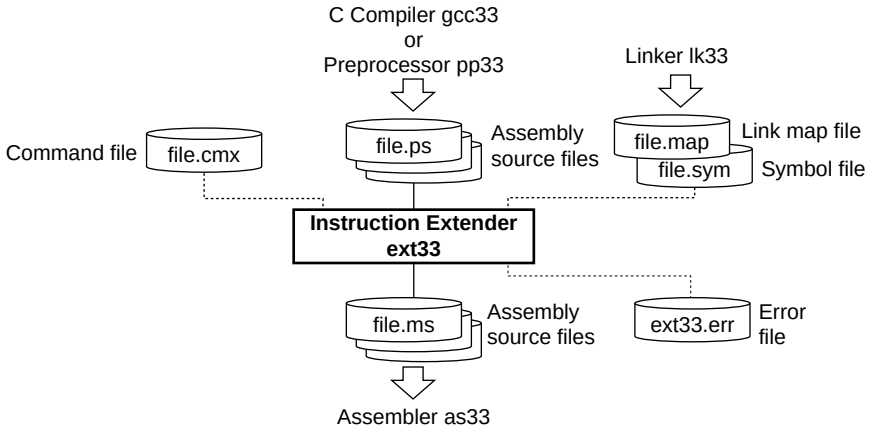


図10.2.1　**フローチャート**

10.2.1　入力ファイル

●アセンブリソースファイル(-cオプション未指定時)

ファイル形式:　テキストファイル
ファイル名:　<ファイル名>.ps　(".ps"以外の拡張子も使用可能、".ms"は不可)
内容:　gcc33またはpp33が生成したアセンブリソースファイルです。

●コマンドファイル(-cオプション指定時)

ファイル形式:　テキストファイル
ファイル名:　<ファイル名>.cmx
内容:　ext33の起動オプションおよび入力ファイルリストを記述します。(10.4節参照)

●リンクマップファイル

ファイル形式:　テキストファイル
ファイル名:　<ファイル名>.map
内容:　リンクによって各セクションのどのアドレスから配置されたかを示すマップ情報ファイルです。リンカlk33で生成されたファイルを入力して、コードの最適化を行います。

●シンボルファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.sym
内容: すべてのモジュール内で定義されているシンボルとそのアドレス情報を持つファイルです。リンカlk33で生成されたファイルを入力して、コードの最適化を行います。

10.2.2　出力ファイル

●アセンブリソースファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.ms (<ファイル名>は入力ファイルと同じです。)
出力先: カレントディレクトリ
内容: 拡張命令がアセンブル可能な形式に展開されたファイルです。

●エラーファイル

ファイル形式: テキストファイル
ファイル名: ext33.err
出力先: カレントディレクトリ
内容: 起動時オプション(-e)が指定された場合に出力されるファイルで、エラーメッセージなど、ext33が標準出力(stdout)に対して出力する内容を記録します。

10.3　起動方法

10.3.1　起動フォーマット

●コマンドラインの一般形

形式1) **ext33 ^ [<起動オプション>] ^ [<ソースファイル名>]**

形式2) **ext33 ^ [<起動オプション>] ^ -c ^ <コマンドファイル名>**

^はスペースを示します。
[ ]は省略可能なことを示します。

<ソースファイル名>: アセンブリソースファイルを拡張子も含めて指定します。
<コマンドファイル名>: コマンドファイルを拡張子も含めて指定します。

●ワークベンチからの起動

起動オプションとソースファイルを選択し、[EXT33]ボタンをクリックします。

コマンドラインで複数のソースファイルが指定可能で、すべてを一度に処理できます。
wb33でも複数のファイルが選択可能ですが、wb33ではファイル数分ext33を実行します。コマンドファイルから取得した場合は一度に処理します。

10.3.2　起動オプション

ext33には6種類の起動時オプションが用意されています。

-c <コマンドファイル名>

機能: コマンドファイルの実行
wb33上の指定: [use .cmx file]をチェック
説明: ・起動オプション、入力ファイル名を指定のコマンドファイルから取得します。
起動オプションはコマンドファイルに含めずにコマンドラインで指定することもできます。

-gp <アドレス>

機能: グローバルポインタを使用

wb33上の指定: [global pointer optimize]をチェック、<アドレス>をテキストボックスに入力

説明:
- グローバルポインタを使用した最適化を行います。
- 指定した<アドレス>がグローバルポインタのアドレスとなります。<アドレス>は小文字の16進数(0x####、#=0–9またはa–fのみ)で指定してください。
- グローバルポインタについては10.7.2節を参照してください。

-lk <ファイル名>

機能: シンボル情報による最適化

wb33上の指定: [symbol,map optimize]をチェック、<ファイル名>は実行ウィンドウ内のテキストボックスの内容が有効

説明:
- リンカが生成したシンボルファイルおよびリンクマップファイルを読み込み、確定したシンボル情報を元にext命令の最適化を行います。
- <ファイル名>(オブジェクトファイル名)でシンボルファイル(.sym)およびリンクマップファイル(.map)を指定します。拡張子は不要です。
- シンボル情報による最適化については10.7.3節を参照してください。

-near

機能: 分岐命令を2命令に展開

wb33上の指定: [far call is 2 inst]をチェック

説明:
- 処理中のファイルに存在しないラベルへの拡張分岐命令を2命令(ext命令1個+分岐命令、符号付き22ビットディスプレースメント)に展開します。
- -nearの指定がない場合は3命令(ext命令2個+分岐命令、符号付き32ビットディスプレースメント)に展開します。
- 分岐命令の最適化については10.7.1節を参照してください。

-j <敷居値>

機能: 最適化分岐の敷居値指定

wb33上の指定: [change threshold]をチェック、<敷居値>をテキストボックスに入力

説明:
- 拡張分岐命令の展開命令数を判定する敷居値を設定します。
- <敷居値>の有効範囲は0x100～0x1fffffです。小文字の16進数(0x####、#=0–9またはa–fのみ)で指定してください。
- -jオプションの指定がない場合、敷居値はデフォルトの0x180000に設定されます。
- 分岐命令の最適化については10.7.1節を参照してください。

-e

機能: エラーファイルの出力

wb33上の指定: なし

説明:
- エラーメッセージなど、ext33が標準出力(stdout)に対して出力する内容をファイル(ext33.err)にも出力します。

コマンドラインにオプションを入力する場合、オプションの前後には1個以上のスペースが必要です。

例: c:¥cc33¥ext33 -gp 0x0 -lk test -near -j 0x180000 -e test1.ps test2.ps
c:¥cc33¥ext33 -gp 0x0 -c test.cmx

10.4　コマンドファイル

ext33はコマンドラインの入力を-cオプションで指定したコマンドファイル(.cmx)から行うことができます。
コマンドファイルには、指定するオプション、入力するソースファイルを1行に1つずつ記述してください。行の先頭にセミコロン(;)を置くことで、コメントを記述することもできます。

例: sample.cmx

```
; This is a sample command file.          ←コメント行
-gp 0x80000
-near
-lk sample
-e
sample1.ps
sample2.ps
sample3.ps
```

コマンドラインでコマンドファイル内と重複するオプションを指定した場合は、先に出現したオプションが有効となります。

例: ext33 -gp 0x0 -c sample.cmx　(上記例のsample.cmxを指定)

この例では"-gp 0x0"オプションが有効となり、コマンドファイル内の"-gp 0x80000"は無効となります。

10.5 メッセージ

ext33はメッセージをすべて標準出力(stdout)に対して出力します。
wb33の[EXT33]ボタンでext33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

●実行中

入力ファイルが2つ以上指定された場合は、処理中のファイル名を表示します。

```
file1.ps
file2.ps
 :
```

1つの入力ファイルのみの場合は、表示されません。

●終了メッセージ

正常に終了した場合は次の終了メッセージのみを出力します。

```
Extend Completed
```

●Usage出力

ファイル名が指定されなかったり、オプション指定が正しくない場合、次の使用方法のメッセージを出力して終了します。

```
Extender 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    ext33 [options] filename
    ext33 [options] -c commandfile (.cmx)
Options:
    -e           : produce log file (ext33.err)
    -lk program  : optimize with program information (program.sym, program.map)
    -gp address  : optimize with global pointer (0x0 - 0xffffffff)
    -near        : specifies all xjmp and xcall extract 2 instruction
    -j threshold : specifies jump optimization threshold (0x100 - 0x1fffff)
Output:
    Assembler source file for as33(.ms)
Example:
    ext33 -e -lk test -gp 0x8000 test.ps
```

●エラー・ワーニング発生時

エラーが発生した場合は、終了メッセージの前にエラーメッセージが表示されます。

例: `test.ps(431): Error: Invalid Syntax.`
`Extend Completed`

エラーの場合、ext33は出力ファイルを作成せずに終了します。

ワーニングが発生した場合は、終了メッセージの前にワーニングメッセージが表示されます。

例: `Warning: Map file test.map does not exist.`
`Extend Completed`

ワーニングの場合、ext33は出力ファイルを作成して終了します。

エラーとワーニングの詳細は"10.10 エラー/ワーニングメッセージ"を参照してください。

10.6　拡張命令

ext33は以下に説明する拡張命令を、アセンブラas33でアセンブル可能なニーモニックに展開します。
拡張命令は、通常ext命令を含む複数の命令で記述する内容を1つの命令として記述できるようにしたもので、命令の機能およびオペランドの即値サイズに従って必要最小限の基本命令に展開されます。

説明に使用するシンボル

immX　　符号なしXビット即値
signX　　符号付きXビット即値
symbol　　メモリアドレスを示すシンボル
label　　分岐先ラベル
(X:Y)　　ビットXからビットYのビットフィールド

10.6.1　算術演算命令

●拡張命令の種類と機能

拡張命令	機能	展開形式
xadd　%rd, %rd, imm32	%rd ← %rd + imm32	(1)
xsub　%rd, %rd, imm32	%rd ← %rd - imm32	(1)
xadd　%sp, %sp, imm32	%sp ← %sp + imm32	(2)
xsub　%sp, %sp, imm32	%sp ← %sp - imm32	(2)
xadd　%rd, %rs, imm32	%rd ← %rs + imm32	(3)
xsub　%rd, %rs, imm32	%rd ← %rs - imm32	(3)
xadd　%rd, %sp, imm32	%rd ← %sp + imm32	(4)
xsub　%rd, %sp, imm32	%rd ← %sp - imm32	(4)
xadd　%rd, %rd, %sp	%rd ← %rd + %sp	(5)
xsub　%rd, %rd, %sp	%rd ← %rd - %sp	(5)
xadd　%sp, %sp, %rs	%sp ← %sp + %rs	(6)
xsub　%sp, %sp, %rs	%sp ← %sp - %rs	(6)

これらの拡張命令により、加減算で32ビット即値が直接指定可能となります。また、スタックポインタSPと汎用レジスタ間の加減算もサポートしています。

●展開後の基本命令

xadd　　add命令に展開
xsub　　sub命令に展開

●展開形式

(1) xOP　%rd, %rd, imm32　　(OP = add, sub)

例: xadd　%rd, %rd, imm32

imm32 ≤ 0x3f	0x3f < imm32 ≤ 0x7ffff	imm32 > 0x7ffff
add　%rd, imm32(5:0)	ext　imm32(18:6) add　%rd, imm32(5:0)	ext　imm32(31:19) ext　imm32(18:6) add　%rd, imm32(5:0)

(2) xOP　%sp, %sp, imm32　　(OP = add, sub)

例: xadd　%sp, %sp, imm32

imm32 ≤ 0xfff	0xfff < imm32 ≤ 0x7ffff	imm32 > 0x7ffff
add　%sp, imm32(11:2)	ld.w　%r9, %sp ext　imm32(18:6) add　%r9, imm32(5:0) ld.w　%sp, %r9	ld.w　%r9, %sp ext　imm32(31:19) ext　imm32(18:6) add　%r9, imm32(5:0) ld.w　%sp, %r9

(3) xOP %rd, %rs, imm32 (OP = add, sub)

例: xadd %rd, %rs, imm32

imm32 ≤ 0x1fff	0x1fff < imm32 ≤ 0x3ffffff	imm32 > 0x3ffffff
ext imm32(12:0) add %rd, %rs	ext imm32(25:13) ext imm32(12:0) add %rd, %rs	ld.w %rd, %rs ext imm32(31:19) ext imm32(18:6) add %rd, imm32(5:0)

(4) xOP %rd, %sp, imm32 (OP = add, sub)

例: xadd %rd, %sp, imm32

imm32 ≤ 0x3f	0x3f < imm32 ≤ 0x7ffff	imm32 > 0x7ffff
ld.w %rd, %sp add %rd, imm32(5:0)	ld.w %rd, %sp ext imm32(18:6) add %rd, imm32(5:0)	ld.w %rd, %sp ext imm32(31:19) ext imm32(18:6) add %rd, imm32(5:0)

(5) xOP %rd, %rd, %sp (OP = add, sub)

例: xadd %rd, %rd, %sp

```
ld.w    %r9, %sp
add     %rd, %r9
```

(6) xOP %sp, %sp, %rs (OP = add, sub)

例: xadd %sp, %sp, %rs

```
ld.w    %r9, %sp
add     %r9, %rs
ld.w    %sp, %r9
```

10.6.2 比較命令

●拡張命令の種類と機能

拡張命令	機能	展開形式
xcmp %rd, sign32	%rd - sign32 (**PSRのC, V, Z, Nフラグを変更**)	(1)
xcmp %rd, %sp	%rd - %sp (**PSRのC, V, Z, Nフラグを変更**)	(2)
xcmp %sp, %rs	%sp - %rs (**PSRのC, V, Z, Nフラグを変更**)	(3)

これらの拡張命令により、汎用レジスタと符号付き32ビット即値との比較、スタックポインタSPと汎用レジスタ間の比較が行えます。

●展開後の基本命令

xcmp cmp命令に展開

●展開形式

(1) xcmp %rd, sign32

-32 ≤ sign32 ≤ 31	-262144 ≤ sign32 < -32 **または** 31 < sign32 ≤ 262143	sign32 < -262144 **または** 262143 < sign32
cmp %rd, sign32(5:0)	ext sign32(18:6) cmp %rd, sign32(5:0)	ext sign32(31:19) ext sign32(18:6) cmp %rd, sign32(5:0)

(2) xcmp %rd, %sp

```
ld.w    %r9, %sp
cmp     %rd, %r9
```

(3) xcmp %sp, %rs

```
ld.w    %r9, %sp
cmp     %r9, %rs
```

10.6.3　論理演算命令

●拡張命令の種類と機能

拡張命令		機能	展開形式
xand	%rd, %rd, sign32	%rd ← %rd & sign32	(1)
xoor	%rd, %rd, sign32	%rd ← %rd | sign32	(1)
xxor	%rd, %rd, sign32	%rd ← %rd ^ sign32	(1)
xand	%rd, %rs, sign32	%rd ← %rs & sign32	(2)
xoor	%rd, %rs, sign32	%rd ← %rs | sign32	(2)
xxor	%rd, %rs, sign32	%rd ← %rs ^ sign32	(2)
xnot	%rd, sign32	%rd ← !sign32	(3)

これらの拡張命令により、論理演算で符号付き32ビット即値が直接指定可能となります。

●展開後の基本命令

xand	and命令に展開
xoor	or命令に展開
xxor	xor命令に展開
xnot	not命令に展開

●展開形式

(1) xOP　%rd, %rd, sign32　　(OP = and, oor, xor)

例: xand　%rd, %rd, sign32

-32 ≤ sign32 ≤ 31	-262144 ≤ sign32 < -32 **または** 31 < sign32 ≤ 262143	sign32 < -262144 **または** 262143 < sign32
and　%rd, sign32(5:0)	ext　sign32(18:6) and　%rd, sign32(5:0)	ext　sign32(31:19) ext　sign32(18:6) and　%rd, sign32(5:0)

(2) xOP　%rd, %rs, sign32　　(OP = and, oor, xor)

例: xand　%rd, %rs, sign32

0x0 ≤ sign32 ≤ 0x1fff (13**ビット 以内**)	0x1fff < sign32 ≤ 0x3ffffff (26**ビット 以内**)	0x3ffffff < sign32 < 0xfffc0000 (26**ビット** < sign32 < -262144)
ext　sign32(12:0) and　%rd, %rs	ext　sign32(25:13) ext　sign32(12:0) and　%rd, %rs	ld.w　%rd, %rs ext　sign32(31:19) ext　sign32(18:6) and　%rd, sign32(5:0)
0xfffc0000 ≤ sign32 < 0xffffffe0 (-262144 ≤ sign32 < -32)	0xffffffe0 ≤ sign32 ≤ 0xffffffff (-32 ≤ sign32 ≤ -1)	
ld.w　%rd, %rs ext　sign32(18:6) and　%rd, sign32(5:0)	ld.w　%rd, %rs and　%rd, sign32(5:0)	

(3) xnot　%rd, sign32

-32 ≤ sign32 ≤ 31	-262144 ≤ sign32 < -32 **または** 31 < sign32 ≤ 262143	sign32 < -262144 **または** 262143 < sign32
not　%rd, sign32(5:0)	ext　sign32(18:6) not　%rd, sign32(5:0)	ext　sign32(31:19) ext　sign32(18:6) not　%rd, sign32(5:0)

10.6.4 シフト&ローテート命令

●拡張命令の種類と機能

拡張命令		機能	展開形式
xsrl	%rd, %rs	論理右シフト	(1)
xsll	%rd, %rs	論理左シフト	(1)
xsra	%rd, %rs	算術右シフト	(1)
xsla	%rd, %rs	算術左シフト	(1)
xrr	%rd, %rs	右ローテート	(1)
xrl	%rd, %rs	左ローテート	(1)
xsrl	%rd, imm5	論理右シフト	(2)
xsll	%rd, imm5	論理左シフト	(2)
xsra	%rd, imm5	算術右シフト	(2)
xsla	%rd, imm5	算術左シフト	(2)
xrr	%rd, imm5	右ローテート	(2)
xrl	%rd, imm5	左ローテート	(2)

これらの拡張命令により、シフト・ローテートのシフト量を31ビットまで指定することができます。

●展開後の基本命令

xsrl	srl命令に展開
xsll	sll命令に展開
xsra	sra命令に展開
xsla	sla命令に展開
xrr	rr命令に展開
xrl	rl命令に展開

●展開形式

(1) xOP %rd, %rs (OP = srl, sll, sra, sla, rr, rl)

例: xsrl %rd, %rs

```
ld.w    %r9, %rs     ; シフト量の格納
and     %r9, 0x1f    ; シフト量の限定(シフト量 < 31)
cmp     %r9, 0x8     ; while (シフト量 > 0x8)
jrle    4            ; {
srl     %rd, 0x8     ;        %rd ← %rd shift 0x8
jp.d    -3           ;        シフト量 -= 0x8
sub     %r9, 0x8     ; }
rl      %rd, %r9     ; %rd ← %rd shift シフト量
```

(2) xOP %rd, imm5 (OP = srl, sll, sra, sla, rr, rl)

例: xsrl %rd, imm5

imm5 ≤ 8	8 < imm5 < 16	imm5 = 16
srl %rd, imm5(3:0)	srl %rd, 0x8 srl %rd, imm5(2:0)	srl %rd, 0x8 srl %rd, 0x8
16 < imm5 ≤ 24	**imm5 > 24**	
srl %rd, 0x8 srl %rd, 0x8 srl %rd, imm5(3:0)	srl %rd, 0x8 srl %rd, 0x8 srl %rd, 0x8 srl %rd, imm5(2:0)	

10.6.5　スタック～レジスタ間データ転送命令

●拡張命令の種類と機能

拡張命令		機能	展開形式
xld.b	%rd, [%sp+imm32]	%rd ← B[%sp+imm32] (**符号拡張**)	(1)
xld.ub	%rd, [%sp+imm32]	%rd ← B[%sp+imm32] (**ゼロ拡張**)	(1)
xld.h	%rd, [%sp+imm32]	%rd ← H[%sp+imm32] (**符号拡張**)	(2)
xld.uh	%rd, [%sp+imm32]	%rd ← H[%sp+imm32] (**ゼロ拡張**)	(2)
xld.w	%rd, [%sp+imm32]	%rd ← W[%sp+imm32]	(3)
xld.b	[%sp+imm32], %rs	B[%sp+imm32] ← %rs(7:0)	(1)
xld.h	[%sp+imm32], %rs	H[%sp+imm32] ← %rs(15:0)	(2)
xld.w	[%sp+imm32], %rs	W[%sp+imm32] ← %rs	(3)
xld.w	[%sp+imm32], %sp	W[%sp+imm32] ← %sp	(4)

これらの拡張命令により、32ビットまでのディスプレースメントが直接指定可能となります。
imm32は省略可能です。

●展開後の基本命令

xld.b	ld.b命令に展開
xld.ub	ld.ub命令に展開
xld.h	ld.h命令に展開
xld.uh	ld.uh命令に展開
xld.w	ld.w命令に展開

●展開形式

imm32を省略した場合、[%sp+0x0]を指定したものとして展開されます。

(1) バイトデータ転送(xld.b, xld.ub)

例: xld.b %rd, [%sp+imm32]

imm32 ≤ 0x3f	0x3f < imm32 ≤ 0x7ffff	imm32 > 0x7ffff
ld.b　%rd, [%sp+imm32(5:0)]	ext　imm32(18:6) ld.b　%rd, [%sp+imm32(5:0)]	ext　imm32(31:19) ext　imm32(18:6) ld.b　%rd, [%sp+imm32(5:0)]

(2) ハーフワードデータ転送(xld.h, xld.uh)

例: xld.h %rd, [%sp+imm32]

imm32 ≤ 0x7f	0x7f < imm32 ≤ 0x7ffff	imm32 > 0x7ffff
ld.h　%rd, [%sp+imm32(6:1)]	ext　imm32(18:6) ld.h　%rd, [%sp+imm32(5:0)]	ext　imm32(31:19) ext　imm32(18:6) ld.h　%rd, [%sp+imm32(5:0)]

(3) ワードデータ転送(xld.w)

例: xld.w %rd, [%sp+imm32]

imm32 ≤ 0xff	0xff < imm32 ≤ 0x7ffff	imm32 > 0x7ffff
ld.w　%rd, [%sp+imm32(7:2)]	ext　imm32(18:6) ld.w　%rd, [%sp+imm32(5:0)]	ext　imm32(31:19) ext　imm32(18:6) ld.w　%rd, [%sp+imm32(5:0)]

(4) スタックポインタをソースとしたワードデータ転送(xld.w [%sp+imm32], %sp)

imm32 ≤ 0xff	0xff < imm32 ≤ 0x7ffff	imm32 > 0x7ffff
ld.w　%r9, %sp ld.w　[%sp+imm32(7:2)], %r9	ld.w　%r9, %sp ext　imm32(18:6) ld.w　[%sp+imm32(5:0)], %r9	ld.w　%r9, %sp ext　imm32(31:19) ext　imm32(18:6) ld.w　[%sp+imm32(5:0)], %r9

10.6.6 メモリ～レジスタ間データ転送命令

●拡張命令の種類と機能

拡張命令		機能	展開形式
xld.b	%rd, [symbol±imm32]	%rd ← B[symbol±imm32] (**符号拡張**)	(1)
xld.ub	%rd, [symbol±imm32]	%rd ← B[symbol±imm32] (**ゼロ拡張**)	(1)
xld.h	%rd, [symbol±imm32]	%rd ← H[symbol±imm32] (**符号拡張**)	(1)
xld.uh	%rd, [symbol±imm32]	%rd ← H[symbol±imm32] (**ゼロ拡張**)	(1)
xld.w	%rd, [symbol±imm32]	%rd ← W[symbol±imm32]	(1)
xld.b	[symbol±imm32], %rs	B[symbol±imm32] ← %rs(7:0)	(1)
xld.h	[symbol±imm32], %rs	H[symbol±imm32] ← %rs(15:0)	(1)
xld.w	[symbol±imm32], %rs	W[symbol±imm32] ←%rs	(1)
xld.w	[symbol±imm32], %sp	W[symbol±imm32] ← %sp	(2)
xld.b	%rd, [imm32]	%rd ← B[imm32] (**符号拡張**)	(3)
xld.ub	%rd, [imm32]	%rd ← B[imm32] (**ゼロ拡張**)	(3)
xld.h	%rd, [imm32]	%rd ← H[imm32] (**符号拡張**)	(3)
xld.uh	%rd, [imm32]	%rd ← H[imm32] (**ゼロ拡張**)	(3)
xld.w	%rd, [imm32]	%rd ← W[imm32]	(3)
xld.b	[imm32], %rs	B[imm32] ← %rs(7:0)	(3)
xld.h	[imm32], %rs	H[imm32] ← %rs(15:0)	(3)
xld.w	[imm32], %rs	W[imm32] ← %rs	(3)
xld.w	[imm32], %sp	W[imm32] ← %sp	(4)
xld.b	%rd, [%rb+symbol±imm32]	%rd ← B[%rb+symbol±imm32] (**符号拡張**)	(5)
xld.ub	%rd, [%rb+symbol±imm32]	%rd ← B[%rb+symbol±imm32] (**ゼロ拡張**)	(5)
xld.h	%rd, [%rb+symbol±imm32]	%rd ← H[%rb+symbol±imm32] (**符号拡張**)	(5)
xld.uh	%rd, [%rb+symbol±imm32]	%rd ← H[%rb+symbol±imm32] (**ゼロ拡張**)	(5)
xld.w	%rd, [%rb+symbol±imm32]	%rd ← W[%rb+symbol±imm32]	(5)
xld.b	[%rb+symbol±imm32], %rs	B[%rb+symbol±imm32] ← %rs(7:0)	(5)
xld.h	[%rb+symbol±imm32], %rs	H[%rb+symbol±imm32] ← %rs(15:0)	(5)
xld.w	[%rb+symbol±imm32], %rs	W[%rb+symbol±imm32] ← %rs	(5)
xld.w	[%rb+symbol±imm32], %sp	W[%rb+symbol±imm32] ← %sp	(6)
xld.b	%rd, [%rb+imm32]	%rd ← B[%rb+imm32] (**符号拡張**)	(7)
xld.ub	%rd, [%rb+imm32]	%rd ← B[%rb+imm32] (**ゼロ拡張**)	(7)
xld.h	%rd, [%rb+imm32]	%rd ← H[%rb+imm32] (**符号拡張**)	(7)
xld.uh	%rd, [%rb+imm32]	%rd ← H[%rb+imm32] (**ゼロ拡張**)	(7)
xld.w	%rd, [%rb+imm32]	%rd ← W[%rb+imm32]	(7)
xld.b	[%rb+imm32], %rs	B[%rb+imm32] ← %rs(7:0)	(7)
xld.h	[%rb+imm32], %rs	H[%rb+imm32] ← %rs(15:0)	(7)
xld.w	[%rb+imm32], %rs	W[%rb+imm32] ← %rs	(7)
xld.w	[%rb+imm32], %sp	W[%rb+imm32] ← %sp	(8)

※ "symbol±imm32"は"symbol+imm32"と"symbol-imm32"の指定が可能

これらの拡張命令により、シンボルや32ビット即値でアドレスを指定するメモリアクセスが可能となります。ただし、ポストインクリメント機能([ ]+)は使用できません。

●展開後の基本命令

xld.b	ld.b命令に展開
xld.ub	ld.ub命令に展開
xld.h	ld.h命令に展開
xld.uh	ld.uh命令に展開
xld.w	ld.w命令に展開

●展開形式

(1) xld.* %rd, [symbol+imm32]、xld.* %rd, [symbol-imm32](* = b, ub, h, uh, w)
xld.* [symbol+imm32], %rs、xld.* [symbol-imm32], %rs(* = b, h, w)

- **[symbol+imm32]指定時**

例: xld.w %rd, [symbol+imm32]

グローバルポインタが指定されていない場合

symbol+imm32 ≤ 0x1f	0x1f < symbol+imm32 ≤ 0x3ffff	symbol+imm32 > 0x3ffff
ld.w %r9, symbol+imm32@l ld.w %rd, [%r9]	ext symbol+imm32@m ld.w %r9, symbol+imm32@l ld.w %rd, [%r9]	ext symbol+imm32@h ext symbol+imm32@m ld.w %r9, symbol+imm32@l ld.w %rd, [%r9]
symbol**不定時**		
lext symbol+imm32@h ext symbol+imm32@m ld.w %r9, symbol+imm32@l ld.w %rd, [%r9]		

グローバルポインタ(gp)が指定されている場合　(sign32 = -gp+imm32)

symbol+sign32 = 0x0	0x0 < symbol+sign32 ≤ 0x1fff	0x1fff < symbol+sign32 ≤ 0x3fffff
ld.w %rd, [%r8]	ext symbol+sign32@al ld.w %rd, [%r8]	ext symbol+sign32@ah ext symbol+sign32@al ld.w %rd, [%r8]
symbol+sign32 > 0x3fffff	symbol**不定時**	gp > symbol+imm32
ext symbol+imm32@h ext symbol+imm32@m ld.w %r9, symbol+imm32@l ld.w %rd, [%r9]	ext symbol+sign32@ah ext symbol+sign32@al ld.w %rd, [%r8]	symbol+imm32**の値に従って、** gp**の指定がない場合の展開形式** **となります。**

imm32は省略可能です。imm32を省略した場合、[symbol+0x0]を指定したものとして展開されます。

- **[symbol-imm32]指定時**

例: xld.w %rd, [symbol-imm32]
常に以下の形に展開されます。

```
ext     symbol-imm32@h
ext     symbol-imm32@m
ld.w    %r9, symbol-imm32@l
ld.w    %rd, [%r9]
```

(2) xld.w [symbol+imm32], %sp、xld.w [symbol-imm32], %sp

- **[symbol+imm32]指定時**

グローバルポインタが指定されていない場合

symbol+imm32 ≤ 0x1f	0x1f < symbol+imm32 ≤ 0x3ffff	symbol+imm32 > 0x3ffff
ld.w %r9, %sp pushn %r0 ld.w %r0, symbol+imm32@l ld.w [%r0], %r9 popn %r0	ld.w %r9, %sp pushn %r0 ext symbol+imm32@m ld.w %r0, symbol+imm32@l ld.w [%r0], %r9 popn %r0	ld.w %r9, %sp pushn %r0 ext symbol+imm32@h ext symbol+imm32@m ld.w %r0, symbol+imm32@l ld.w [%r0], %r9 popn %r0
symbol**不定時**		
ld.w %r9, %sp pushn %r0 ext symbol+imm32@h ext symbol+imm32@m ld.w %r0, symbol+imm32@l ld.w [%r0], %r9 popn %r0		

グローバルポインタ(gp)が指定されている場合 (sign32 = -gp+imm32)

symbol+sign32 = 0x0	0x0 < symbol+sign32 ≤ 0x1fff	0x1fff < symbol+sign32 ≤ 0x3ffffff
ld.w %r9, %sp ld.w [%r8], %r9	ld.w %r9, %sp ext symbol+sign32@al ld.w [%r8], %r9	ld.w %r9, %sp ext symbol+sign32@ah ext symbol+sign32@al ld.w [%r8], %r9
symbol+sign32 > 0x3ffffff	**symbol不定時**	**gp > symbol+imm32**
ld.w %r9, %sp pushn %r0 ext symbol+imm32@h ext symbol+imm32@m ld.w %r0, symbol+imm32@l ld.w [%r0], %r9 popn %r0	ld.w %r9, %sp pushn %r0 ext symbol+imm32@h ext symbol+imm32@m ld.w %r0, symbol+imm32@l ld.w [%r0], %r9 popn %r0	symbol+imm32の値に従って、 gpの指定がない場合の展開形式 となります。

imm32は省略可能です。imm32を省略した場合、[symbol+0x0]を指定したものとして展開されます。

- **[symbol-imm32]指定時**

常に以下の形に展開されます。

```
ld.w    %r9, %sp
pushn   %r0
ext     symbol-imm32@h
ext     symbol-imm32@m
ld.w    %r0, symbol-imm32@l
ld.w    [%r0], %r9
popn    %r0
```

(3) xld.* %rd, [imm32](*=b, ub, h, uh, w)、xld.* [imm32], %rs(*=b, h, w)

例: xld.w %rd, [imm32]

グローバルポインタが指定されていない場合

imm32 ≤ 0x1f	0x1f < imm32 ≤ 0x3ffff	imm32 > 0x3ffff
ld.w %r9, imm32(5:0) ld.w %rd, [%r9]	ext imm32(18:6) ld.w %r9, imm32(5:0) ld.w %rd, [%r9]	ext imm32(31:19) ext imm32(18:6) ld.w %r9, imm32(5:0) ld.w %rd, [%r9]

グローバルポインタ(gp)が指定されている場合 (sign32 = -gp+imm32)

sign32 = 0x0	0x0 < sign32 ≤ 0x1fff	0x1fff < sign32 ≤ 0x3ffffff
ld.w %rd, [%r8]	ext sign32(12:0) ld.w %rd, [%r8]	ext sign32(25:13) ext sign32(12:0) ld.w %rd, [%r8]
sign32 > 0x3ffffff	**gp > imm32**	
ext imm32(31:19) ext imm32(18:6) ld.w %r9, imm32(5:0) ld.w %rd, [%r9]	imm32の値に従って、 gpの指定がない場合の展開形式 となります。	

(4) xld.w [imm32], %sp

グローバルポインタが指定されていない場合

imm32 ≤ 0x1f	0x1f < imm32 ≤ 0x3ffff	imm32 > 0x3ffff
ld.w %r9, %sp pushn %r0 ld.w %r0, imm32(5:0) ld.w [%r0], %r9 popn %r0	ld.w %r9, %sp pushn %r0 ext imm32(18:6) ld.w %r0, imm32(5:0) ld.w [%r0], %r9 popn %r0	ld.w %r9, %sp pushn %r0 ext imm32(31:19) ext imm32(18:6) ld.w %r0, imm32(5:0) ld.w [%r0], %r9 popn %r0

グローバルポインタ(gp)が指定されている場合 (sign32 = -gp+imm32)

sign32 = 0x0	0x0 < sign32 ≤ 0x1fff	0x1fff < sign32 ≤ 0x3ffffff
ld.w %r9, %sp ld.w [%r8], %r9	ld.w %r9, %sp ext sign32(12:0) ld.w [%r8], %r9	ld.w %r9, %sp ext sign32(25:13) ext sign32(12:0) ld.w [%r8], %r9
sign32 > 0x3ffffff	gp > imm32	
ld.w %r9, %sp pushn %r0 ext imm32(31:19) ext imm32(18:6) ld.w %r0, imm32(5:0) ld.w [%r0], %r9 popn %r0	imm32**の値に従って、** gp**の指定がない場合の展開形式** **となります。**	

(5) xld.* %rd, [%rb+symbol+imm32]、xld.* %rd, [%rb+symbol-imm32](* = b, ub, h, uh, w)
xld.* [%rb+symbol+imm32], %rs、xld.* [%rb+symbol-imm32], %rs(* = b, h, w)

- **[%rb+symbol+imm32]指定時**

例: xld.w %rd, [%rb+symbol+imm32]

symbol+imm32 = 0x0	0x0 < symbol+imm32 ≤ 0x1fff	0x1fff < symbol+imm32 ≤ 0x3ffffff
ld.w %rd, [%rb]	ext symbol+imm32@al ld.w %rd, [%rb]	ext symbol+imm32@ah ext symbol+imm32@al ld.w %rd, [%rb]
symbol+imm32 > 0x3ffffff	symbol**不定時**	
ext symbol+imm32@h ext symbol+imm32@m ld.w %r9, symbol+imm32@l add %r9, %rb ld.w %rd, [%r9]	ext symbol+imm32@h ext symbol+imm32@m ld.w %r9, symbol+imm32@l add %r9, %rb ld.w %rd, [%r9]	

imm32は省略可能です。imm32を省略した場合、[%rb+symbol+0x0]を指定したものとして展開されます。

- **[%rb+symbol-imm32]指定時**

例: xld.w %rd, [%rb+symbol-imm32]
常に以下の形に展開されます。

```
ext     symbol-imm32@h
ext     symbol-imm32@m
ld.w    %r9, symbol-imm32@l
add     %r9, %rb
ld.w    %rd, [%r9]
```

(6) xld.w [%rb+symbol+imm32], %sp、xld.w [%rb+symbol-imm32], %sp

- **[%rb+symbol+imm32]指定時**

symbol+imm32 = 0x0	0x0 < symbol+imm32 ≤ 0x1fff	0x1fff < symbol+imm32 ≤ 0x3ffffff
ld.w %r9, %sp ld.w [%rb], %r9	ld.w %r9, %sp ext symbol+imm32@al ld.w [%rb], %r9	ld.w %r9, %sp ext symbol+imm32@ah ext symbol+imm32@al ld.w [%rb], %r9
symbol+imm32 > 0x3ffffff **および** %rb ≠ %r0	symbol+imm32 > 0x3ffffff **および** %rb = %r0	
ld.w %r9, %sp pushn %r0 ext symbol+imm32@h ext symbol+imm32@m ld.w %r0, symbol+imm32@l add %r0, %rb ld.w [%r0], %r9 popn %r0	ld.w %r9, %sp pushn %r1 ext symbol+imm32@h ext symbol+imm32@m ld.w %r1, symbol+imm32@l add %r1, %rb ld.w [%r1], %r9 popn %r1	

symbol**不定および** %rb ≠ %r0	symbol**不定および** %rb = %r0
ld.w %r9, %sp	ld.w %r9, %sp
pushn %r0	pushn %r1
ext symbol+imm32@h	ext symbol+imm32@h
ext symbol+imm32@m	ext symbol+imm32@m
ld.w %r0, symbol+imm32@l	ld.w %r1, symbol+imm32@l
add %r0, %rb	add %r1, %rb
ld.w [%r0], %r9	ld.w [%r1], %r9
popn %r0	popn %r1

imm32は省略可能です。imm32を省略した場合、[%rb+symbol+0x0]を指定したものとして展開されます。

- **[%rb+symbol-imm32]指定時**

常に以下の形に展開されます。

%rb ≠ %r0	%rb = %r0
ld.w %r9, %sp	ld.w %r9, %sp
pushn %r0	pushn %r1
ext symbol-imm32@h	ext symbol-imm32@h
ext symbol-imm32@m	ext symbol-imm32@m
ld.w %r0, symbol-imm32@l	ld.w %r1, symbol-imm32@l
add %r0, %rb	add %r1, %rb
ld.w [%r0], %r9	ld.w [%r1], %r9
popn %r0	popn %r1

(7) xld.* %rd, [%rb+imm32](*=b, ub, h, uh, w)、xld.* [%rb+imm32], %rs(* =b, h, w)

例: xld.w %rd, [%rb+imm32]

imm32 = 0x0	0x0 < imm32 ≤ 0x1fff	0x1fff < imm32 ≤ 0x3ffffff
ld.w %rd, [%rb]	ext imm32(12:0) ld.w %rd, [%rb]	ext imm32(25:13) ext imm32(12:0) ld.w %rd, [%rb]
imm32 > 0x3ffffff		
ext imm32(31:19) ext imm32(18:6) ld.w %r9, imm32(5:0) add %r9, %rb ld.w %rd, [%r9]		

imm32は省略可能です。imm32を省略した場合、[%rb+0x0]を指定したものとして展開されます。

(8) xld.w [%rb+imm32], %sp

imm32 = 0x0	0x0 < imm32 ≤ 0x1fff	0x1fff < imm32 ≤ 0x3ffffff
ld.w %r9, %sp ld.w [%rb], %r9	ld.w %r9, %sp ext imm32(12:0) ld.w [%rb], %r9	ld.w %r9, %sp ext imm32(25:13) ext imm32(12:0) ld.w [%rb], %r9
imm32 > 0x3ffffff **および** %rb ≠ %r0	imm32 > 0x3ffffff **および** %rb = %r0	
ld.w %r9, %sp pushn %r0 ext imm32(31:19) ext imm32(18:6) ld.w %r0, imm32(5:0) add %r0, %rb ld.w [%r0], %r9 popn %r0	ld.w %r9, %sp pushn %r1 ext imm32(31:19) ext imm32(18:6) ld.w %r1, imm32(5:0) add %r1, %rb ld.w [%r1], %r9 popn %r1	

imm32は省略可能です。imm32を省略した場合、[%rb+0x0]を指定したものとして展開されます。

10.6.7　即値ロード命令

●拡張命令の種類と機能

拡張命令	機能	展開形式
xld.w　%rd, symbol±imm32	%rd ← symbol±imm32	(1)
xld.w　%rd, sign32	%rd ← sign32	(2)

※ "symbol±imm32"は"symbol+imm32"と"symbol-imm32"の指定が可能

これらの拡張命令により、32ビットの即値を直接汎用レジスタにロードすることができます。即値指定にはシンボルも使用可能です。

●展開後の基本命令

xld.w　　ld.w命令に展開

●展開形式

(1) xld.w　%rd, symbol+imm32、xld.w　%rd, symbol-imm32

- **symbol+imm32指定時**

symbol+imm32 ≦ 0x1f	0x1f < symbol+imm32 ≦ 0x3ffff	symbol+imm32 > 0x3ffff
ld.w　%rd, symbol+imm32	ext　symbol+imm32@m ld.w　%rd, symbol+imm32@l	ext　symbol+imm32@h ext　symbol+imm32@m ld.w　%rd, symbol+imm32@l
symbol**不定時**		
ext　symbol+imm32@h ext　symbol+imm32@m ld.w　%rd, symbol+imm32@l		

imm32は省略可能です。imm32を省略した場合、symbol+0x0を指定したものとして展開されます。

- **symbol-imm32指定時**

常に以下の形に展開されます。

ext　symbol-imm32@h ext　symbol-imm32@m ld.w　%rd, symbol-imm32@l

(2) xld.w　%rd, sign32

-32 ≦ sign32 ≦ 31	-262144 ≦ sign32 < -32 **または** 31 < sign32 ≦ 262143	sign32 < -262144 **または** 262143 < sign32
ld.w　%rd, sign32(5:0)	ext　sign32(18:6) ld.w　%rd, sign32(5:0)	ext　sign32(31:19) ext　sign32(18:6) ld.w　%rd, sign32(5:0)

10.6.8 ビット操作命令

●拡張命令の種類と機能

拡張命令		機能	展開形式
xbtst	[symbol±imm32], imm3	B[symbol±imm32]のビット テスト	(1)
xbclr	[symbol±imm32], imm3	B[symbol±imm32]のビット クリア	(1)
xbset	[symbol±imm32], imm3	B[symbol±imm32]のビット セット	(1)
xbnot	[symbol±imm32], imm3	B[symbol±imm32]のビット 反転	(1)
xbtst	[imm32], imm3	B[imm32]のビット テスト	(2)
xbclr	[imm32], imm3	B[imm32]のビット クリア	(2)
xbset	[imm32], imm3	B[imm32]のビット セット	(2)
xbnot	[imm32], imm3	B[imm32]のビット 反転	(2)
xbtst	[%rb+symbol±imm32], imm3	B[%rb+symbol±imm32]のビット テスト	(3)
xbclr	[%rb+symbol±imm32], imm3	B[%rb+symbol±imm32]のビット クリア	(3)
xbset	[%rb+symbol±imm32], imm3	B[%rb+symbol±imm32]のビット セット	(3)
xbnot	[%rb+symbol±imm32], imm3	B[%rb+symbol±imm32]のビット 反転	(3)
xbtst	[%rb+imm32], imm3	B[%rb+imm32]のビット テスト	(4)
xbclr	[%rb+imm32], imm3	B[%rb+imm32]のビット クリア	(4)
xbset	[%rb+imm32], imm3	B[%rb+imm32]のビット セット	(4)
xbnot	[%rb+imm32], imm3	B[%rb+imm32]のビット 反転	(4)
xbtst	[%sp+imm32], imm3	B[%sp+imm32]のビット テスト	(5)
xbclr	[%sp+imm32], imm3	B[%sp+imm32]のビット クリア	(5)
xbset	[%sp+imm32], imm3	B[%sp+imm32]のビット セット	(5)
xbnot	[%sp+imm32], imm3	B[%sp+imm32]のビット 反転	(5)

※ "symbol±imm32"は"symbol+imm32"と"symbol-imm32"の指定が可能

これらの拡張命令により、シンボルや32ビット即値でビットを操作するメモリアドレスが指定可能となります。

注: ビット番号を指定する第2オペランド(imm3)は、符号なし32ビットの範囲内であればext33ではエラーとなりません。そのまま基本命令のオペランドとして出力されます。基本命令の有効範囲は符号なし3ビットですので注意してください。

●展開後の基本命令

xbtst	btst命令に展開
xbclr	bclr命令に展開
xbset	bset命令に展開
xbnot	bnot命令に展開

●展開形式

(1) xOP [symbol+imm32], imm3、xOP [symbol-imm32], imm3 (OP = btst, bclr, bset, bnot)

- **[symbol+imm32]指定時**

  例: xbtst [symbol+imm32], imm3

グローバルポインタが指定されていない場合

symbol+imm32 ≤ 0x1f	0x1f < symbol+imm32 ≤ 0x3ffff	symbol+imm32 > 0x3ffff
ld.w %r9, symbol+imm32@l btst [%r9], imm3	ext symbol+imm32@m ld.w %r9, symbol+imm32@l btst [%r9], imm3	ext symbol+imm32@h ext symbol+imm32@m ld.w %r9, symbol+imm32@l btst [%r9], imm3

symbol不定時
ext symbol+imm32@h ext symbol+imm32@m ld.w %r9, symbol+imm32@l btst [%r9], imm3

グローバルポインタ(gp)が指定されている場合 (sign32 = -gp+imm32)

symbol+sign32 = 0x0	0x0 < symbol+sign32 ≤ 0x1fff	0x1fff < symbol+sign32 ≤ 0x3ffffff
btst [%r8], imm3	ext symbol+sign32@al btst [%r8], imm3	ext symbol+sign32@ah ext symbol+sign32@al btst [%r8], imm3
symbol+sign32 > 0x3ffffff	**symbol不定時**	**gp > symbol+imm32**
ext symbol+imm32@h ext symbol+imm32@m ld.w %r9, symbol+imm32@l btst [%r9], imm3	ext symbol+sign32@ah ext symbol+sign32@al btst [%r8], imm3	symbol+imm32の値に従って、gpの指定がない場合の展開形式となります。

imm32は省略可能です。imm32を省略した場合、[symbol+0x0]を指定したものとして展開されます。

- **[symbol-imm32]指定時**

  常に以下の形に展開されます。

```
ext     symbol-imm32@h
ext     symbol-imm32@m
ld.w    %r9, symbol-imm32@l
btst    [%r9], imm3
```

(2) xOP [imm32], imm3 (OP = btst, bclr, bset, bnot)

例: xbtst [imm32], imm3

グローバルポインタが指定されていない場合

imm32 ≤ 0x1f	0x1f < imm32 ≤ 0x3ffff	imm32 > 0x3ffff
ld.w %r9, imm32(5:0) btst [%r9], imm3	ext imm32(18:6) ld.w %r9, imm32(5:0) btst [%r9], imm3	ext imm32(31:19) ext imm32(18:6) ld.w %r9, imm32(5:0) btst [%r9], imm3

グローバルポインタ(gp)が指定されている場合 (sign32 = -gp+imm32)

sign32 = 0x0	0x0 < sign32 ≤ 0x1fff	0x1fff < sign32 ≤ 0x3ffffff
btst [%r8], imm3	ext sign32(12:0) btst [%r8], imm3	ext sign32(25:13) ext sign32(12:0) btst [%r8], imm3
sign32 > 0x3ffffff	**gp > imm32**	
ext imm32(31:19) ext imm32(18:6) ld.w %r9, imm32(5:0) btst [%r9], imm3	imm32の値に従って、gpの指定がない場合の展開形式となります。	

(3) xOP [%rb+symbol+imm32], imm3、xOP [%rb+symbol-imm32], imm3 (OP = btst, bclr, bset, bnot)

- **[%rb+symbol+imm32]指定時**

  例: xbtst [%rb+symbol+imm32], imm3

symbol+imm32 = 0x0	0x0 < symbol+imm32 ≤ 0x1fff	0x1fff < symbol+imm32 ≤ 0x3ffffff
btst [%rb], imm3	ext symbol+imm32@al btst [%rb], imm3	ext symbol+imm32@ah ext symbol+imm32@al btst [%rb], imm3
symbol+imm32 > 0x3ffffff	**symbol不定時**	
ext symbol+imm32@h ext symbol+imm32@m ld.w %r9, symbol+imm32@l add %r9, %rb btst [%r9], imm3	ext symbol+imm32@h ext symbol+imm32@m ld.w %r9, symbol+imm32@l add %r9, %rb btst [%r9], imm3	

imm32は省略可能です。imm32を省略した場合、[%rb+symbol+0x0]を指定したものとして展開されます。

- **[%rb+symbol-imm32]指定時**

例: xbtst [%rb+symbol-imm32], imm3
常に以下の形に展開されます。

```
ext     symbol-imm32@h
ext     symbol-imm32@m
ld.w    %r9, symbol-imm32@l
add     %r9, %rb
btst    [%r9], imm3
```

(4) xOP [%rb+imm32], imm3 (OP = btst, bclr, bset, bnot)

例: xbtst [%rb+imm32], imm3

imm32 = 0x0	0x0 < imm32 ≤ 0x1fff	0x1fff < imm32 ≤ 0x3ffffff
btst [%rb], imm3	ext imm32(12:0) btst [%rb], imm3	ext imm32(25:13) ext imm32(12:0) btst [%rb], imm3
imm32 > 0x3ffffff		
ext imm32(31:19) ext imm32(18:6) ld.w %r9, imm32(5:0) add %r9, %rb btst [%r9], imm3		

imm32は省略可能です。imm32を省略した場合、[%rb+0x0]を指定したものとして展開されます。

(5) xOP [%sp+imm32], imm3 (OP = btst, bclr, bset, bnot)

例: xbtst [%sp+imm32], imm3

imm32 = 0x0	0x0 < imm32 ≤ 0x3f	0x3f < imm32 ≤ 0x7ffff
ld.w %r9, %sp btst [%r9], imm3	ld.w %r9, %sp add %r9, imm32(5:0) btst [%r9], imm3	ld.w %r9, %sp ext imm32(18:6) add %r9, imm32(5:0) btst [%r9], imm3
imm32 > 0x7ffff		
ld.w %r9, %sp ext imm32(31:19) ext imm32(18:6) add %r9, imm32(5:0) btst [%r9], imm3		

10.6.9　分岐命令

●拡張命令の種類と機能

拡張命令		機能	展開形式
xcall	label+imm32	サブルーチンコール	(1)
xcall.d	label+imm32	サブルーチンコール(ディレイド分岐)	(1)
xjp	label+imm32	無条件ジャンプ	(1)
xjp.d	label+imm32	無条件ジャンプ(ディレイド分岐)	(1)
xjreq	label+imm32	条件付きジャンプ	(1)
xjreq.d	label+imm32	条件付きジャンプ(ディレイド分岐)	(1)
xjrne	label+imm32	条件付きジャンプ	(1)
xjrne.d	label+imm32	条件付きジャンプ(ディレイド分岐)	(1)
xjrgt	label+imm32	条件付きジャンプ	(1)
xjrgt.d	label+imm32	条件付きジャンプ(ディレイド分岐)	(1)
xjrge	label+imm32	条件付きジャンプ	(1)
xjrge.d	label+imm32	条件付きジャンプ(ディレイド分岐)	(1)
xjrlt	label+imm32	条件付きジャンプ	(1)
xjrlt.d	label+imm32	条件付きジャンプ(ディレイド分岐)	(1)
xjrle	label+imm32	条件付きジャンプ	(1)
xjrle.d	label+imm32	条件付きジャンプ(ディレイド分岐)	(1)
xjrugt	label+imm32	条件付きジャンプ	(1)
xjrugt.d	label+imm32	条件付きジャンプ(ディレイド分岐)	(1)
xjruge	label+imm32	条件付きジャンプ	(1)
xjruge.d	label+imm32	条件付きジャンプ(ディレイド分岐)	(1)
xjrult	label+imm32	条件付きジャンプ	(1)
xjrult.d	label+imm32	条件付きジャンプ(ディレイド分岐)	(1)
xjrule	label+imm32	条件付きジャンプ	(1)
xjrule.d	label+imm32	条件付きジャンプ(ディレイド分岐)	(1)
xcall	sign32	サブルーチンコール	(2)
xcall.d	sign32	サブルーチンコール(ディレイド分岐)	(2)
xjp	sign32	無条件ジャンプ	(2)
xjp.d	sign32	無条件ジャンプ(ディレイド分岐)	(2)
xjreq	sign32	条件付きジャンプ	(2)
xjreq.d	sign32	条件付きジャンプ(ディレイド分岐)	(2)
xjrne	sign32	条件付きジャンプ	(2)
xjrne.d	sign32	条件付きジャンプ(ディレイド分岐)	(2)
xjrgt	sign32	条件付きジャンプ	(2)
xjrgt.d	sign32	条件付きジャンプ(ディレイド分岐)	(2)
xjrge	sign32	条件付きジャンプ	(2)
xjrge.d	sign32	条件付きジャンプ(ディレイド分岐)	(2)
xjrlt	sign32	条件付きジャンプ	(2)
xjrlt.d	sign32	条件付きジャンプ(ディレイド分岐)	(2)
xjrle	sign32	条件付きジャンプ	(2)
xjrle.d	sign32	条件付きジャンプ(ディレイド分岐)	(2)
xjrugt	sign32	条件付きジャンプ	(2)
xjrugt.d	sign32	条件付きジャンプ(ディレイド分岐)	(2)
xjruge	sign32	条件付きジャンプ	(2)
xjruge.d	sign32	条件付きジャンプ(ディレイド分岐)	(2)
xjrult	sign32	条件付きジャンプ	(2)
xjrult.d	sign32	条件付きジャンプ(ディレイド分岐)	(2)
xjrule	sign32	条件付きジャンプ	(2)
xjrule.d	sign32	条件付きジャンプ(ディレイド分岐)	(2)

これらの拡張命令により、ディスプレースメント付きラベル、符号付き32ビット即値で分岐先を指定することができます。条件付きジャンプ命令の分岐条件は基本命令と同じです。

●展開後の基本命令

xcall/xcall.d	call/call.d命令に展開
xjp/xjp.d	jp/jp.d命令に展開
xjreq/xjreq.d	jreq/jreq.d命令に展開
xjrne/xjrne.d	jrne/jrne.d命令に展開
xjrgt/xjrgt.d	jrgt/jrgt.d命令に展開
xjrge/xjrge.d	jrge/jrge.d命令に展開
xjrlt/xjrlt.d	jrlt/jrlt.d命令に展開
xjrle/xjrle.d	jrle/jrle.d命令に展開
xjrugt/xjrugt.d	jrugt/jrugt.d命令に展開
xjruge/xjruge.d	jruge/jruge.d命令に展開
xjrult/xjrult.d	jrult/jrult.d命令に展開
xjrule/xjrule.d	jrule/jrule.d命令に展開

●展開形式

(1) xOP label+imm32 (OP = call, call.d, jp, jp.d, jr*, jr*.d)

例: xcall label+imm32

1命令に展開する場合	2命令に展開する場合	3命令に展開する場合
call label+imm32	ext label+imm32@rm call label+imm32@rl	ext label+imm32@rh ext label+imm32@rm call label+imm32@rl

imm32は省略可能です。imm32を省略した場合、label+0x0を指定したものとして展開されます。
展開後の命令数の決定方法については"10.7.1 相対分岐命令の最適化"を参照してください。

(2) xOP sign32 (OP = call, call.d, jp, jp.d, jr*, jr*.d)

例: xcall sign32

-256 ≤ sign32 ≤ 254	-2097152 ≤ sign32 < -256 または 254 < sign32 ≤ 2097150	sign32 > 2097150 または sign32 < -2097152
call sign32(8:1)	ext sign32(21:9) call sign32(8:1)	ext sign32(31:22)<<0x3 ext sign32(21:9) call sign32(8:1)

10.7　最適化機能

10.7.1　相対分岐命令の最適化

S1C33000の基本相対分岐命令は、1命令で符号付き9ビット(最下位ビット=0)の相対アドレス範囲内の分岐が可能で、それを越えるアドレスへの分岐はext命令による拡張が必要です。
ext33では10.6.9節に示したようにext命令を意識せずに拡張分岐命令が記述でき、分岐先ラベルまでの相対距離に従って確実に分岐可能で最小の命令数となるように展開されます。

最適化の対象となるのは以下の拡張分岐命令です。
xcall　xjp　xjreq　xjrne　xjrgt　xjrge　xjrlt　xjrle　xjrugt　xjruge　xjrult　xjrule
xcall.d　xjp.d　xjreq.d　xjrne.d　xjrgt.d　xjrge.d　xjrlt.d　xjrle.d　xjrugt.d　xjruge.d　xjrult.d　xjrule.d

ext命令付きで記述された基本分岐命令の最適化は行いません。

展開後の命令数(1～3命令)を決定する条件は次のとおりです。
- 拡張分岐命令と分岐先ラベルの相対距離
- 拡張分岐命令と分岐先ラベルが同一ファイル内にある/ない
- -nearオプション指定の有無
- -lkオプションによるシンボルファイル/リンクマップファイル指定の有無
- 相対距離判定敷居値0x180000(デフォルト)または-jオプションによる指定値

これらの条件判定により、展開後の命令数は次のようになります。

表10.7.1.1　**展開命令数**

相対距離の絶対値（命令数）	命令とラベルの位置	-nearフラグ	展開命令数
0～126　*1	同一ファイル内	ー	1
	異なるファイル	ー	2
～敷居値　*2	ー	ー	2
～0x7fffffff　*3	ー	あり	2
	ー	なし	3
相対距離不明	ー	あり	2
	ー	なし	3

*1:　プラスアドレス方向には125
*2:　同一ファイル内のプラスアドレス方向の場合は敷居値-1まで
*3:　同一ファイル内のプラスアドレス方向の場合は0x7ffffffeまで
敷居値は-jオプションで指定した値の1/2です。-jオプションのデフォルト値0x180000の場合、敷居値は0xc0000となります。
低アドレス方向へ分岐する場合、3回ほどの繰り返しにより相対距離を決めているため、ソースによってはこの値よりも敷居値が低くなる場合があります。

展開後の基本形式は次のとおりです。
例: xcall　sign32

1命令		2命令		3命令	
call	sign32(8:1)	ext	sign32(21:9)	ext	sign32(31:22)<<0x3
		call	sign32(8:1)	ext	sign32(21:9)
				call	sign32(8:1)

ラベルの指定方法により、上記とは異なる展開結果にもなります。それらについては10.6.9節を参照してください。

-lkオプションの指定により、リンク後のシンボル情報を元にして異なるファイルへの分岐についても最適化が行えます。ただし、内蔵RAMへのキャッシングによる実行なども考えられるため、オブジェクトファイル上は1命令の範囲内への分岐であっても、異なるファイル内のラベルへの分岐は最低2命令として展開されます。

10.7.2 グローバルポインタによる最適化

シンボルを使用した拡張命令によるメモリアクセスは、R9レジスタをスクラッチレジスタとして使用し、次のように展開されます。

```
例: xld.w   [i],%r10    (リンク前の展開例)
            ext         i+0x0@h
            ext         i+0x0@m
            ld.w        %r9,i+0x0@l
            ld.w        [%r9],%r10
```

メモリに常駐するグローバル変数などをアクセスする場合、グローバル変数領域の先頭アドレスをグローバルポインタとして設定しておくことにより、展開後の命令数を削減することができます。
ext33はグローバルポインタとしてR8レジスタを使用しますので、初期化ルーチン内でR8レジスタにグローバルポインタのアドレスを設定しておく必要があります。また、すべてのアクセスはグローバルポインタのアドレスからプラス26ビットまでの空間内に限られます。
ext33に対する指定は-gp <アドレス>オプション(wb33上は[global pointer optimize])によって行います。-gpオプションが指定されると、ext33はR8レジスタが指定された<アドレス>のグローバルポインタとして設定されているものとして処理します。
前記例をグローバルポインタ指定付きで展開すると次のようになります。

```
例: xld.w   [i],%r10    (リンク前の展開例)
            ext         i+0x0@ah
            ext         i+0x0@al
            ld.w        [%r8],%r10
```

スクラッチレジスタを使用しない分、1回のアクセスに付き1命令削減できます。

10.7.3 シンボル情報による最適化

複数のリロケータブルモジュールをリンクして1つのプログラムを作成する場合、リンクを行うまではアセンブリソースの各命令の絶対アドレスが確定しません。
この段階での最適化は同一ファイル内で解決できるものに限られます。
このため、ext33は-lkオプションを指定することによって、リンカが出力するシンボルファイルとリンクマップファイルからシンボル情報を取得できるようになっています。それぞれのファイルには、リンク後に確定するシンボルの絶対アドレスなどの情報が含まれていますので、他のファイル内で定義されているシンボルの参照を解決することができます。
したがって、一度すべてのソースファイルをext33も含めリンク処理まで行い、-lkオプションを指定してext33を再度実行してください。ext33は未確定シンボルを使用してアクセスを行っていた拡張命令を最適化してアセンブリソースファイルを生成します。その後、再度アセンブルとリンクを行ってください。
シンボルファイルとリンクマップファイルを生成するには、リンカの起動時に-mおよび-sオプションの指定が必要です。
たとえば、前節で使用した例の変数iがグローバルポインタのアドレスから4バイト目に配置されたものであれば、次のように最初のext命令が削除されます。

```
例: xld.w   [i],%r10    (リンク後のシンボル情報を使用した展開例)
            ext         i+0x0@al
            ld.w        [%r8],%r10
```

また、逆にグローバルポインタが使用できないアドレスについては、グローバルポインタによる最適化を行わないという判断もします。
ext33は-lkオプションで指定されたファイル名を持つシンボルファイル、マップファイルが同一ディレクトリに作成されているかチェックします。どちらか1つまたは両方のファイルが見つからない場合、ext33はワーニングを出力し、-lkオプションによる最適化は行いません。したがって、これらのファイルは必ず同じ名称で同じディレクトリに置いてください。

10.8　その他の機能

10.8.1　コメント付加機能

ext33は拡張命令を展開してアセンブリソースファイルを作成する際、展開前の内容が分かるように、拡張命令をセミコロン(;)で始まるコメントに置き換えます。コメントは展開後の最初の命令の行の後ろに付加されます。元のステートメントにコメントが付いていた場合は、そのコメントも含めて付加します。

例:　展開前

```
xcall        main        ; goto main
```

展開後

```
    ext      main@rm  ;          xcall      main       ; goto main
    call     main@rl
```

10.8.2　ローカルシンボルの分類

ext33はプリプロセッサ、コンパイラが生成するファイル内でのみ有効となるローカルラベルを分類し、拡張相対分岐命令に使用している"__L????"を"__LX????"に変更します。
変更されたラベルは、そのファイルですでに参照が解決ずみです。
このため、アセンブラas33は"__LX"で始まるラベルの情報をローカルなものと解釈し、オブジェクトファイルに出力しません。

なお、拡張相対分岐命令以外が使用しているローカルシンボルは"__L????"であっても変更されません。

10.8.3　文法チェック

ext33は拡張命令と以下のアセンブラ擬似命令の文法チェックのみを行います。

ext33がチェックするアセンブラ擬似命令

.org, .space, .align, .comm, .lcomm, .set擬似命令
　　アドレスを指定するオペランドが32ビットの有効範囲内であるかチェックします。
.ascii擬似命令
　　文字列が" "で囲まれて指定されているかチェックします。

拡張命令または上記アセンブラ擬似命令が不正な文法で記述されている場合はエラーとなります。
オペランドの数値が不正な場合はワーニングを出力して処理を継続します。

基本命令の文法チェック、および展開後の命令の妥当性についてはチェックしません。

10.9 実行例

●入力ファイル例(ext.ps)

```
; ext.ps   1997.2.23
; sample source for ext33 extended instructions
; not for execution, just for ext33 extension only

          .word BOOT
BOOT:

; summary of major patterns

; xld.w
          xld.w     %r8,0                ; immediate load
          xld.w     %r1,DATA1            ; symbol immediate load
          xld.w     %r2,DATA1+4          ; symbol+offset

          xadd      %r1,%r2,0x12345678   ; 3 operand for xadd, xsub

          xand      %r14,%r15,0xff000000 ; 3 operand for xand, xoor, xxor
          xnot      %r8,0b1111100000

          xsrl      %r3,8                ; immediate shift
          xrr       %r7,%r8              ; register shift
                                         ; for xsrl, xsll, xsra, xsla, xrr, xrl

          xld.w     %r1,[0x1234568]      ; immediate address
          xld.ub    %r5,[DATA1]          ; symbol address
          xbtst     [COMM1+0x400],2      ; symbol address + offset
          xld.uh    %r10,[%sp+0x222]     ; sp relative
          xld.b     [%sp],%r7            ; sp relative
          xld.uh    %r1,[%r15+0x1234568] ; resister + immediate address
          xld.h     %r5,[%r11+DATA1]     ; register + symbol address
          xbset     [%r9+COMM1+0x400],2  ; register + symbol address + offset
                                         ; for xld.w, xld.uh, xld.h, xld.ub, xld.b,
                                         ;     xbset, xbclr, xbtst, xbnot

          xjp       -2                   ; immediate relative
          xjrgt.d   BOOT                 ; symbol relative
                                         ; for xjp, xjreq, xjrne, xjrgt, xjrge, xjrlt, xjrle,
                                         ;     xjrugt, xjruge, xjrult, xjrule, xcall

; more detail samples

; xld.w              load immediate to register operation

          xld.w     %r8,0                ; decimal
          xld.w     %r0,0x12345678       ; hex
          xld.w     %r0,0b10101          ; binary
          xld.w     %r1,DATA1            ; symbol
          xld.w     %r2,DATA1+4          ; symbol+offset(hex,dec,bin)
          xld.w     %r2,DATA1+0x5
          xld.w     %r2,DATA1+0b110

; xadd, xsub         add and sub, arithmetic operations

          xadd      %r1,%r2,0x12345678   ; 3 operand No.1
          xsub      %r2,%r1,0x12345      ; 3 operand No.2
          xadd      %r0,%r1,1            ; 3 operand No.3
          xsub      %r2,%r2,5            ; 3 operand No.4
          xadd      %r1,%r2,%sp          ; for C compiler
          xsub      %sp,%sp,%r1          ; for C compiler

; xand, xoor, xxor, xnot
```

```
;                   and, or, xor, and not, logical operations

          xand      %r14,%r15,0xff000000 ; 3 operand No.1
          xoor      %r12,%r11,0xfedc     ; 3 operand No.2
          xxor      %r9,%r9,-1           ; 3 operand No.3
          xnot      %r8,0b1111100000

; xsrl, xsll, xsra, xsla, xrr, xrl        shift operations

          xsrl      %r3,8                ; immediate shift No.1
          xsll      %r4,15               ; immediate shift No.2
          xsra      %r5,17               ; immediate shift No.3
          xsla      %r6,31               ; immediate shift No.4
          xrr       %r7,%r8              ; register shift

; xld.w, xld.uh, xld.h, xld.ub, xld.b, xbset, xbclr, xbtst, xbnot
;                   load, bit operation from/to absolute address

          xld.w     %r1,[0x1234568]      ; immediate address No.1
          xld.uh    %r2,[0xABC]          ; immediate address No.2
          xld.h     [10], %r3            ; immediate address No.3
          xld.ub    %r4,[0]              ; immediate address No.4
          xld.b     %r5,[DATA1]          ; symbol address No.1
          xbnot     [COMM1],1            ; symbol address No.2
          xbtst     [COMM1+0x400],2      ; symbol address + offset No.1
          xbset     [COMM1+0x10],3       ; symbol address + offset No.2
          xbclr     [COMM1+1],4          ; symbol address + offset No.3

; xld.w, xld.uh, xld.h, xld.ub, xld.b, xbset, xbclr, xbtst, xbnot
;                   load, bit operation from/to SP relative address

          xld.w     %r15,[%sp+0x4444444] ; sp relative No.1
          xld.uh    %r10,[%sp+0x222]     ; sp relative No.2
          xld.b     [%sp],%r7            ; sp relative No.3
          xbset     [%sp+0x14],5         ; sp relative No.4

; xld.w, xld.uh, xld.h, xld.ub, xld.b, xbset, xbclr, xbtst, xbnot
;                   load, bit operation from/to register relative address

          xld.w     %r1,[%r15+0x1234568] ; + immediate address No.1
          xld.uh    %r2,[%r14+0xABC]     ; + immediate address No.2
          xld.h     [%r13+10], %r3       ; + immediate address No.3
          xld.ub    %r4,[%r12]           ; + immediate address No.4
          xld.b     %r5,[%r11+DATA1]     ; + symbol address No.1
          xbnot     [%r10+COMM1],1       ; + symbol address No.2
          xbtst     [%r9+COMM1+0x400],2  ; + symbol address + offset No.1
          xbset     [%r8+COMM1+0x10],3   ; + symbol address + offset No.2
          xbclr     [%r7+COMM1+1],4      ; + symbol address + offset No.3

; xld.w             load word operation from sp register for C support

          xld.w     [%sp],%sp
          xld.w     [%sp+0x2468],%sp
          xld.w     [0x12340],%sp
          xld.w     [COMM1],%sp
          xld.w     [COMM1+4],%sp
          xld.w     [%r5],%sp
          xld.w     [%r6+0b1100],%sp
          xld.w     [%r7+DATA1],%sp
          xld.w     [%r7+DATA1+200],%sp

; xjp, xjreq, xjrne, xjrgt, xjrge, xjrlt, xjrle, xjrugt, xjruge, xjrult
; xjrule, xcall and with .d               relative branchs
```

```
NEAR:
        xjp.d     -2                  ; immediate relative No.1
        xjreq     800                 ; immediate relative No.2
        xjrne     0x1000000           ; immediate relative No.3
        xjrgt.d   BOOT                ; symbol relative No.1
        xjrge     COMM1               ; symbol relative No.2
        xjruge    NEAR                ; symbol relative No.3

        .data
DATA1:
        .word     0x12345678

        .comm COMM1 4
```

●出力ファイル(ext.ms)　　"ext33 -gp 0x0 text.ps"を実行した場合

(* コメントの出力位置は実際の結果とは異なります)

```
; ext.ps  1997.2.23
; sample source for ext33 extended instructions
; not for execution, just for ext33 extension only

        .word BOOT
BOOT:

; summary of major patterns

; xld.w
        ld.w      %r8,0x0             ; xld.w   %r8,0                ; immediate load
        ext       DATA1+0x0@h         ; xld.w   %r1,DATA1            ; symbol immediate load
        ext       DATA1+0x0@m
        ld.w      %r1,DATA1+0x0@l
        ext       DATA1+0x4@h         ; xld.w   %r2,DATA1+4          ; symbol+offset
        ext       DATA1+0x4@m
        ld.w      %r2,DATA1+0x4@l

        ld.w      %r1,%r2             ; xadd    %r1,%r2,0x12345678   ; 3 operand for xadd, xsub
        ext       0x246
        ext       0x1159
        add       %r1,0x38

        ld.w      %r14,%r15           ; xand    %r14,%r15,0xff000000 ; 3 operand for xand, xoor, xxor
        ext       0x1fe0
        ext       0x0
        and       %r14,0x0
        ext       0xf                 ; xnot    %r8,0b1111100000
        not       %r8,0x20

        srl       %r3,0x8             ; xsrl    %r3,8                ; immediate shift
        ld.w      %r9,%r8             ; xrr     %r7,%r8              ; register shift
        and       %r9,0x1f
        cmp       %r9,0x8
        jrle      4
        rr        %r7,0x8
        jp.d      -3
        sub       %r9,0x8
        rr        %r7,%r9
                                      ; for xsrl, xsll, xsra, xsla, xrr, xrl

        ext       0x91a               ; xld.w   %r1,[0x1234568]      ; immediate address
        ext       0x568
        ld.w      %r1,[%r8]
        ext       DATA1+0x0@ah        ; xld.ub  %r5,[DATA1]          ; symbol address
        ext       DATA1+0x0@al
        ld.ub     %r5,[%r8]
        ext       COMM1+0x400@ah      ; xbtst   [COMM1+0x400],2      ; symbol address + offset
```

```
        ext       COMM1+0x400@al
        btst      [%r8],0x2
        ext       0x8                 ; xld.uh   %r10,[%sp+0x222]     ; sp relative
        ld.uh     %r10,[%sp+0x22]
        ld.b      [%sp+0x0],%r7       ; xld.b    [%sp],%r7            ; sp relative
        ext       0x91a               ; xld.uh   %r1,[%r15+0x1234568] ; resister + immediate address
        ext       0x568
        ld.uh     %r1,[%r15]
        ext       DATA1+0x0@h         ; xld.h    %r5,[%r11+DATA1]     ; register + symbol address
        ext       DATA1+0x0@m
        ld.w      %r9,DATA1+0x0@l
        add       %r9,%r11
        ld.h      %r5,[%r9]
        ext       COMM1+0x400@h       ; xbset    [%r9+COMM1+0x400],2
                                      ; register + symbol address + offset
        ext       COMM1+0x400@m
        ld.w      %r9,COMM1+0x400@l
        add       %r9,%r9
        bset      [%r9],0x2
                                      ; for xld.w, xld.uh, xld.h, xld.ub, xld.b,
                                      ;     xbset, xbclr, xbtst, xbnot

        jp        0xff                ; xjp      -2                   ; immediate relative
        ext       BOOT@rh             ; xjrgt.d  BOOT                 ; symbol relative
        ext       BOOT@rm
        jrgt.d    BOOT@rl
                                      ; for xjp, xjreq, xjrne, xjrgt, xjrge, xjrlt, xjrle,
                                      ;     xjrugt, xjruge, xjrult, xjrule, xcall

; more detail samples

; xld.w           load immediate to register operation

        ld.w      %r8,0x0             ; xld.w    %r8,0                ; decimal
        ext       0x246               ; xld.w    %r0,0x12345678       ; hex
        ext       0x1159
        ld.w      %r0,0x38
        ld.w      %r0,0x15            ; xld.w    %r0,0b10101          ; binary
        ext       DATA1+0x0@h         ; xld.w    %r1,DATA1            ; symbol
        ext       DATA1+0x0@m
        ld.w      %r1,DATA1+0x0@l
        ext       DATA1+0x4@h         ; xld.w    %r2,DATA1+4          ; symbol+offset(hex,dec,bin)
        ext       DATA1+0x4@m
        ld.w      %r2,DATA1+0x4@l
        ext       DATA1+0x5@h         ; xld.w    %r2,DATA1+0x5
        ext       DATA1+0x5@m
        ld.w      %r2,DATA1+0x5@l
        ext       DATA1+0x6@h         ; xld.w    %r2,DATA1+0b110
        ext       DATA1+0x6@m
        ld.w      %r2,DATA1+0x6@l

; xadd, xsub      add and sub, arithmetic operations

        ld.w      %r1,%r2             ; xadd     %r1,%r2,0x12345678   ; 3 operand No.1
        ext       0x246
        ext       0x1159
        add       %r1,0x38
        ext       0x9                 ; xsub     %r2,%r1,0x12345      ; 3 operand No.2
        ext       0x345
        sub       %r2,%r1
        ext       0x1                 ; xadd     %r0,%r1,1            ; 3 operand No.3
        add       %r0,%r1
        sub       %r2,0x5             ; xsub     %r2,%r2,5            ; 3 operand No.4
        ld.w      %r9,%sp             ; xadd     %r1,%r2,%sp          ; for C compiler
```

```
        add     %r1,%r9
        ld.w    %r9,%sp             ; xsub    %sp,%sp,%r1          ; for C compiler
        sub     %r9,%r1
        ld.w    %sp,%r9

; xand, xoor, xxor, xnot
;                   and, or, xor, and not, logical operations

        ld.w    %r14,%r15           ; xand    %r14,%r15,0xff000000 ; 3 operand No.1
        ext     0x1fe0
        ext     0x0
        and     %r14,0x0
        ext     0x7                 ; xoor    %r12,%r11,0xfedc     ; 3 operand No.2
        ext     0x1edc
        or      %r12,%r11
        xor     %r9,0x3f            ; xxor    %r9,%r9,-1           ; 3 operand No.3
        ext     0xf                 ; xnot    %r8,0b1111100000
        not     %r8,0x20

; xsrl, xsll, xsra, xsla, xrr, xrl        shift operations

        srl     %r3,0x8             ; xsrl    %r3,8                ; immediate shift No.1
        sll     %r4,0x8             ; xsll    %r4,15               ; immediate shift No.2
        sll     %r4,0x7
        sra     %r5,0x8             ; xsra    %r5,17               ; immediate shift No.3
        sra     %r5,0x8
        sra     %r5,0x1
        sla     %r6,0x8             ; xsla    %r6,31               ; immediate shift No.4
        sla     %r6,0x8
        sla     %r6,0x8
        sla     %r6,0x7
        ld.w    %r9,%r8             ; xrr     %r7,%r8              ; register shift
        and     %r9,0x1f
        cmp     %r9,0x8
        jrle    4
        rr      %r7,0x8
        jp.d    -3
        sub     %r9,0x8
        rr      %r7,%r9

; xld.w, xld.uh, xld.h, xld.ub, xld.b, xbset, xbclr, xbtst, xbnot
;                   load, bit operation from/to absolute address

        ext     0x91a               ; xld.w   %r1,[0x1234568]      ; immediate address No.1
        ext     0x568
        ld.w    %r1,[%r8]
        ext     0xabc               ; xld.uh  %r2,[0xABC]          ; immediate address No.2
        ld.uh   %r2,[%r8]
        ext     0xa                 ; xld.h   [10], %r3            ; immediate address No.3
        ld.h    [%r8],%r3
        ld.ub   %r4,[%r8]           ; xld.ub  %r4,[0]              ; immediate address No.4
        ext     DATA1+0x0@ah        ; xld.b   %r5,[DATA1]          ; symbol address No.1
        ext     DATA1+0x0@al
        ld.b    %r5,[%r8]
        ext     COMM1+0x0@ah        ; xbnot   [COMM1],1            ; symbol address No.2
        ext     COMM1+0x0@al
        bnot    [%r8],0x1
        ext     COMM1+0x400@ah      ; xbtst   [COMM1+0x400],2      ; symbol address + offset No.1
        ext     COMM1+0x400@al
        btst    [%r8],0x2
        ext     COMM1+0x10@ah       ; xbset   [COMM1+0x10],3       ; symbol address + offset No.2
        ext     COMM1+0x10@al
        bset    [%r8],0x3
        ext     COMM1+0x1@ah        ; xbclr   [COMM1+1],4          ; symbol address + offset No.3
```

```
        ext     COMM1+0x1@al
        bclr    [%r8],0x4

; xld.w, xld.uh, xld.h, xld.ub, xld.b, xbset, xbclr, xbtst, xbnot
;               load, bit operation from/to SP relative address

        ext     0x88                ; xld.w   %r15,[%sp+0x4444444] ; sp relative No.1
        ext     0x1111
        ld.w    %r15,[%sp+0x4]
        ext     0x8                 ; xld.uh  %r10,[%sp+0x222]     ; sp relative No.2
        ld.uh   %r10,[%sp+0x22]
        ld.b    [%sp+0x0],%r7       ; xld.b   [%sp],%r7            ; sp relative No.3
        ld.w    %r9,%sp             ; xbset   [%sp+0x14],5         ; sp relative No.4
        add     %r9,0x14
        bset    [%r9],0x5

; xld.w, xld.uh, xld.h, xld.ub, xld.b, xbset, xbclr, xbtst, xbnot
;               load, bit operation from/to register relative address

        ext     0x91a               ; xld.w   %r1,[%r15+0x1234568] ; + immediate address No.1
        ext     0x568
        ld.w    %r1,[%r15]
        ext     0xabc               ; xld.uh  %r2,[%r14+0xABC]     ; + immediate address No.2
        ld.uh   %r2,[%r14]
        ext     0xa                 ; xld.h   [%r13+10], %r3       ; + immediate address No.3
        ld.h    [%r13],%r3
        ld.ub   %r4,[%r12]          ; xld.ub  %r4,[%r12]           ; + immediate address No.4
        ext     DATA1+0x0@h         ; xld.b   %r5,[%r11+DATA1]     ; + symbol address No.1
        ext     DATA1+0x0@m
        ld.w    %r9,DATA1+0x0@l
        add     %r9,%r11
        ld.b    %r5,[%r9]
        ext     COMM1+0x0@h         ; xbnot   [%r10+COMM1],1       ; + symbol address No.2
        ext     COMM1+0x0@m
        ld.w    %r9,COMM1+0x0@l
        add     %r9,%r10
        bnot    [%r9],0x1
        ext     COMM1+0x400@h       ; xbtst   [%r9+COMM1+0x400],2  ; + symbol address + offset No.1
        ext     COMM1+0x400@m
        ld.w    %r9,COMM1+0x400@l
        add     %r9,%r9
        btst    [%r9],0x2
        ext     COMM1+0x10@h        ; xbset   [%r8+COMM1+0x10],3   ; + symbol address + offset No.2
        ext     COMM1+0x10@m
        ld.w    %r9,COMM1+0x10@l
        add     %r9,%r8
        bset    [%r9],0x3
        ext     COMM1+0x1@h         ; xbclr   [%r7+COMM1+1],4      ; + symbol address + offset No.3
        ext     COMM1+0x1@m
        ld.w    %r9,COMM1+0x1@l
        add     %r9,%r7
        bclr    [%r9],0x4

; xld.w         load word operation from sp register for C support

        ld.w    %r9,%sp             ; xld.w   [%sp],%sp
        ld.w    [%sp+0x0],%r9
        ld.w    %r9,%sp             ; xld.w   [%sp+0x2468],%sp
        ext     0x91
        ld.w    [%sp+0x28],%r9
        ld.w    %r9,%sp             ; xld.w   [0x12340],%sp
        pushn   %r0
        ext     0x48d
        ld.w    %r0,0x0
```

```
        ld.w    [%r0],%r9
        popn    %r0
        ld.w    %r9,%sp             ; xld.w   [COMM1],%sp
        pushn   %r0
        ext     COMM1+0x0@h
        ext     COMM1+0x0@m
        ld.w    %r0,COMM1+0x0@l
        ld.w    [%r0],%r9
        popn    %r0
        ld.w    %r9,%sp             ; xld.w   [COMM1+4],%sp
        pushn   %r0
        ext     COMM1+0x4@h
        ext     COMM1+0x4@m
        ld.w    %r0,COMM1+0x4@l
        ld.w    [%r0],%r9
        popn    %r0
        ld.w    %r9,%sp             ; xld.w   [%r5],%sp
        ld.w    [%r5],%r9
        ld.w    %r9,%sp             ; xld.w   [%r6+0b1100],%sp
        ext     0xc
        ld.w    [%r6],%r9
        ld.w    %r9,%sp             ; xld.w   [%r7+DATA1],%sp
        pushn   %r0
        ext     DATA1+0x0@h
        ext     DATA1+0x0@m
        ld.w    %r0,DATA1+0x0@l
        add     %r0,%r7
        ld.w    [%r0],%r9
        popn    %r0
        ld.w    %r9,%sp             ; xld.w   [%r7+DATA1+200],%sp
        pushn   %r0
        ext     DATA1+0xc8@h
        ext     DATA1+0xc8@m
        ld.w    %r0,DATA1+0xc8@l
        add     %r0,%r7
        ld.w    [%r0],%r9
        popn    %r0

; xjp, xjreq, xjrne, xjrgt, xjrge, xjrlt, xjrle, xjrugt, xjruge, xjrult
; xjrule, xcall and with .d             relative branchs

NEAR:
        xjp.d   -2                  ; immediate relative No.1
        ext     0x1                 ; xjreq   800       ; immediate relative No.2
        jreq    0x90
        ext     0x20                ; xjrne   0x1000000 ; immediate relative No.3
        ext     0x0
        jrne    0x0
        xjrgt.d BOOT                ; symbol relative No.1
        ext     COMM1@rh            ; xjrge   COMM1     ; symbol relative No.2
        ext     COMM1@rm
        jrge    COMM1@rl
        jruge   NEAR                ; xjruge  NEAR      ; symbol relative No.3

        .data
DATA1:
        .word   0x12345678

        .comm COMM1 4
```

10.10　エラー/ワーニングメッセージ

エラーおよびワーニングメッセージは標準出力(stdout)に表示/出力されます。-eオプションを指定した場合は、ファイル"ext33.err"にも出力されます。
wb33の[EXT33]ボタンでext33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

10.10.1　エラー

ext33で発生するエラーは、システムエラー、エクステンダエラーの2つに分類されます。

システムエラーは処理が継続不可能なエラーのことです。このエラーが発生した場合、ext33はエラーメッセージを表示して、直ちに処理を終了します。アセンブリソースファイルは出力されません。

表10.10.1.1　システムエラーメッセージ一覧

エラーメッセージ	内容
Error: Cannot allocate memory.	メモリが確保できません。
Error: Cannot open input file "<file name>".	ソースファイルがオープンできません。ファイル<file name>が指定のディレクトリに存在しません。
Error: Invalid input filename "<file name>".	ソースファイルがオープンできません。入力ファイル名の指定が不正です。
Error: Cannot open output file "<file name>".	出力ファイルがオープンできません。
Error: Cannot open error file "ext33.err".	エラーファイルがオープンできません。
Error: Cannot open command file "<file name>".	コマンドファイルがオープンできません。ファイル<file name>が指定のディレクトリに存在しません。
Error: Cannot write error file.	エラーファイルへの書き込みができません。
Error: Cannot write output file "<file name>".	出力ファイルへの書き込みができません。

エクステンダエラーは、コマンドラインやコマンドファイル、またはソースファイル中にext33が処理できない構文や記述があった場合に発生します。このエラーが発生した場合、アセンブリソースファイルは出力されません。

表10.10.1.2　エクステンダエラーメッセージ一覧

エラーメッセージ	内容
Error: Too long filename "<file name>".	ファイル名が長すぎます。ファイル名はパスを含めて 255 文字以内としてください。
Error: Invalid command file description "<description>".	コマンドファイル内の<description>が不正です。オプション、入力ファイルを 1 行に 1 つずつ記述してください。コメントは";"で始め、他のオプションや入力ファイルと同じ行には記述しないでください。
Error: Invalid command file format.	コマンドファイル形式が不正です。テキストファイル以外の形式のファイルが指定されました。
Error: Invalid jump threshold "<parameter>".	-j オプションのパラメータが不正です。敷居値は 0x100～0x1fffff の範囲内で指定してください。
Error: Invalid GP address "<parameter>".	-gp オプションのパラメータが不正です。有効アドレスを 16 進数で指定してください。
Error: Too many input files.	入力ファイルが多すぎます。682 個以上の入力ファイルが指定されました。
Error: No input file is specified.	入力ファイルが指定されていません。
<file name>(line No.): Error: Invalid syntax.　*1	拡張命令の文法エラーです。

1　デバッグ情報を含むソースファイルを入力した場合、メッセージの後ろに"near <file name>(<line No.>)"が表示されます。内容はデバッグ情報で示されるオリジナルのソースファイル名(.c、*.s)と行番号です。

10.10.2 ワーニング

ワーニングの場合、ext33は処理を継続します。他のエラーが発生しなければ、ワーニングメッセージを表示して処理を終了します。アセンブリソースファイルも出力されます。

表10.10.2.1 ワーニングメッセージ一覧

ワーニングメッセージ	内容
Warning: Map file "<file name>" does not exist.	リンクマップファイルが見つかりません。-lk オプションの指定を無効として処理を継続します。
Warning: Symbol file "<file name>" does not exist.	シンボルファイルが見つかりません。-lk オプションの指定を無効として処理を継続します。
<file name>: Warning: No map information in map file.	入力ファイルに対応するマップ情報がリンクマップファイル内に存在しません。-lk オプションの指定を無効として処理を継続します。
<file name>: Warning: Invalid map file format.	不正なマップファイル形式です。-lk オプションの指定を無効として処理を継続します。
<file name>: Warning: Invalid symbol file format.	不正なシンボルファイル形式です。-lk オプションの指定を無効として処理を継続します。
<file name>: Warning: Cannot find the symbol "<symbol>" in symbol table. ∗1	<symbol>の情報がシンボルファイルに見当たりません。<symbol>を未定義として処理を継続します。
<file name>: Warning: Operand exceeds maximum address. ∗1	オペランドのアドレスが 32 ビットの有効範囲を越えています。
<file name>: Warning: Invalid address operand. ∗1	不正な境界アドレスが指定されています。ハーフワードデータを扱う拡張命令で、最下位ビットが 0 以外のアドレスが指定されています。ワードデータを扱う拡張命令で、下位 2 ビットが 0 以外のアドレスが指定されています。
<file name>: Warning: Invalid operand value. ∗1	拡張命令のオペランドに有効範囲外の数値が指定されています。

1 デバッグ情報を含むソースファイルを入力した場合、メッセージの後ろに"near <file name>(<line No.>)"が表示されます。内容はデバッグ情報で示されるオリジナルのソースファイル名(.c、*.s)と行番号です。

10.11 注意事項

(1) ext33では、汎用レジスタR8がグローバルポインタとして、R9が拡張命令展開用のスクラッチレジスタとして予約されています。アセンブリソースを作成する際には、この2つのレジスタを使用しないでください。

(2) ext33は拡張命令の展開および最適化に必要な文法チェックしか行いません。展開後の妥当性も含め、ニーモニック、オペランド、アセンブラ擬似命令(一部を除く)のチェックは行いませんので注意してください。

11 アセンブラ

この章ではアセンブラas33の機能を説明します。
アセンブリソースの文法については"4.3 アセンブリソースの文法"を参照してください。

11.1 機能

アセンブラas33(以下as33と記述)は命令エクステンダが出力したアセンブリソースファイルをアセンブル(翻訳)し、機械語のオブジェクトファイルを生成します。
as33の機能、特長を以下に示します。

・ アブソリュートアセンブルおよびリロケータブルアセンブルに対応
・ モジュール別のプログラム開発が可能
・ シンボリックデバッグ用のデバッグ情報を出力可能

11.2 入出力ファイル

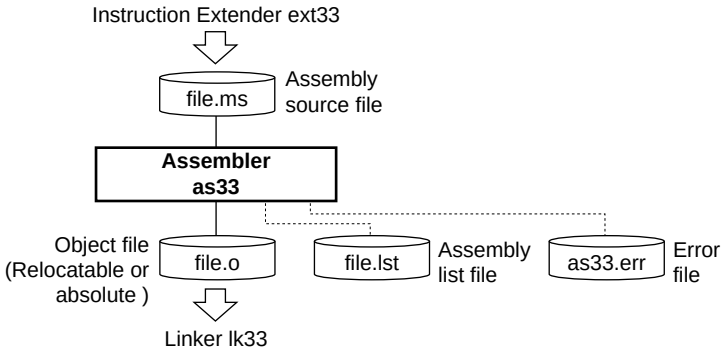


図11.2.1 フローチャート

11.2.1 入力ファイル

●アセンブリソースファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.ms (".ms"以外の拡張子が使用可能、パス指定も可能)
内容: ソースプログラムを記述したファイルです。通常、命令エクステンダext33が出力したファイルを入力します。
基本命令とアセンブラ擬似命令のみを記述したソースファイルを作成した場合は、直接as33に入力することも可能です。

11.2.2 出力ファイル

●オブジェクトファイル

ファイル形式: srf33形式のバイナリファイル
ファイル名: <ファイル名>.o (<ファイル名>は入力ファイルと同じです。)
出力先: カレントディレクトリ
内容: プログラムコード(機械語)にシンボル情報やデバッグ情報等を付加したファイルです。
srf33形式についてはAppendixの"srf33ファイルの構造"を参照してください。

●アセンブリリストファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.lst (<ファイル名>は入力ファイルと同じです。)
出力先: カレントディレクトリ
内容: アセンブリソースファイルの各行にアセンブル結果(アドレスやオブジェクトコード)を付加したファイルです。-lオプションが指定された場合に出力されます。
具体例については"11.9 アセンブリリストファイル"を参照してください。

●エラーファイル

ファイル形式: テキストファイル
ファイル名: as33.err
出力先: カレントディレクトリ
内容: -eオプションが指定された場合に出力されるファイルで、エラーメッセージなど、as33が標準出力(stdout)に対して出力する内容を記録します。

11.3 起動方法

11.3.1 起動フォーマット

●コマンドラインの一般形

as33 ^ [<起動時オプション>] ^ [<ファイル名>]

^はスペースを示します。
[ ]は省略可能なことを示します。

<ファイル名>: アセンブリソースファイルを拡張子も含めて指定します。

●ワークベンチからの起動

起動オプションとソースファイルを選択し、[AS33]ボタンをクリックします。

コマンドラインで一度に指定可能なソースファイルは1つのみです。
wb33では複数のファイルを選択可能で、ファイル数分as33を実行します。

11.3.2 起動時オプション

as33には3種類の起動時オプションが用意されています。

-g

機能: デバッグ情報の付加
wb33上の指定: [debug info]をチェック
説明:
- シンボリックデバッグ情報を含む出力ファイルが生成されます。
- シンボリックデバッグを行う場合は、必ず指定してください。
- このオプションを指定しない場合でも、Cコンパイラgcc33、プリプロセッサpp33で付加したソース表示用のデバッグ情報はカットされません。

-l

機能: アセンブリリストファイルの出力
wb33上の指定: [list file]をチェック
説明:
- アセンブリリストファイルを出力します。

-e

機能: エラーファイルの出力
wb33上の指定: なし
説明:
- エラーメッセージなど、as33が標準出力(stdout)に対して出力する内容をファイル(as33.err)にも出力します。

コマンドラインにオプションを入力する場合、オプションの前後には1個以上のスペースが必要です。
例: c:¥cc33¥as33 -g -e -l test.ms

11.4 メッセージ

as33はメッセージをすべて標準出力(stdout)に対して出力します。
wb33の[AS33]ボタンでas33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

●終了メッセージ

正常に終了した場合は次の終了メッセージのみを出力します。

```
Assembly Completed
```

●Usage出力

ファイル名が指定されなかったり、オプション指定が正しくない場合、次の使用方法のメッセージを出力して終了します。

```
Assembler 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    as33 [options] <filename>
Options:
    -e : produce log file (as33.err)
    -g : generate debug information in object file
    -l : produce list file
Output:
    object file (.o)
    list file   (.lst)
    log file    (as33.err)
Example:
    as33 -e -g -l sample.ms
```

●エラー・ワーニング発生時

エラーが発生した場合は、終了メッセージの前にエラーメッセージが表示されます。
例: `test.ms(7): Error: Invalid instruction syntax.`
`Assembly Completed`

エラーの場合、as33は出力ファイルを作成せずに終了します。

ワーニングが発生した場合は、終了メッセージの前にワーニングメッセージが表示されます。
例: `test.ms(12): Warning: Numeric range.`
`Assembly Completed`

ワーニングの場合、as33は出力ファイルを作成して終了します。

エラーとワーニングの詳細は"11.10 エラー/ワーニングメッセージ"を参照してください。

11.5 リロケータブルアセンブルとアブソリュートアセンブル

as33はリロケータブルアセンブルとアブソリュートアセンブルの両方に対応しています。
リロケータブルモジュールとアブソリュートモジュールが混在した開発も可能です。

11.5.1 リロケータブルアセンブル

リロケータブルアセンブルは、モジュールがメモリのどこに配置されても動作するように、アドレスを固定せずにアセンブルする方式です。したがって、ソースファイル内では絶対アドレスの指定は行いません。Cコンパイラが生成するアセンブリソースファイルもリロケータブル形式です。

リロケータブルアセンブルでは、各セクションの先頭からの相対アドレスを個々のコードに適用して処理が行われます。
これらの情報はコードとともにオブジェクトファイルに出力され、リンカlk33の処理によってすべてのアドレスが決定します。
この方式でアセンブルしたモジュールは、他のモジュールと自由に組み合わせることができます。これにより、リロケータブルアセンブルした汎用モジールを、他機種の開発の際のライブラリとして使用することができます。
lk33による再配置については"12 リンカ"を参照してください。また、出力オブジェクトのライブラリ化については"15 ライブラリアン"を参照してください。

リロケータブルソースファイルの中では、.abs擬似命令、.org擬似命令および.set擬似命令は使用できません。

11.5.2 アブソリュートアセンブル

アブソリュートアセンブルは、ソースの段階からコードが配置される絶対アドレスを指定しておくアセンブル方式です。
ただし、他のモジュールとリンクする際の汎用性が失われますので、テスト用のプログラムを簡単に作成する場合などに限定して使用してください。
アブソリュートアセンブルの指定は、ソースファイルの1行目に.abs擬似命令を書き込むことよって行い、アドレスの設定には.org擬似命令を使用します。絶対アドレスを定義する.org擬似命令と.set擬似命令はアブソリュートソース内でのみ使用可能です。

```
例:     .abs                    ...アブソリュートアセンブルを指定
        .code
        .org    0x80000         ...以下のコードをアドレスの0x80000番地から配置します。
        .word   0x80004
BOOT:
        ext     0x20
        ld.w    %r8,0x0
        ld.w    %sp,%r8         ; set SP
        ld.w    %r8,0x0         ; set global pointer
          :
        .data
        .org    0xC0000         ...以下のデータをアドレス0xC0000番地から配置します。
        .word   0x12345678
```

この方式では、そのファイル内の全コードが絶対アドレスを持つことになります。ファイル内の一部をリロケータブルにすることはできません。
ただし、プログラムを1つのアブソリュートソースファイルとして作成した場合でも、最終的なアドレスが確定した実行ファイルを作成するためにはlk33を通す必要があります。(1つのファイルの場合、ワークベンチwb33では、[use .cm file]のチェックを外してリンクを行ってください。)

1つのアブソリュートファイル内のCODEセクション、DATAセクション、BSSセクションはそれぞれ1つまでとしてください。
基本的にはリロケータブルアセンブリソースファイルを作成し、lk33によるマップ機能で再配置してください。

11.6 スコープ

各ソースファイル内で定義されたシンボルは、そのファイル内では自由に参照することができます。このようなシンボルの参照範囲をスコープといいます。
リロケータブルアセンブル/アブソリュートアセンブルのどちらにおいてもこのスコープは同じで、通常は定義されたファイル内でのみ参照可能です。そのファイル内に見つからないシンボルが参照されるとas33はそのシンボルを未定義シンボルとしてオブジェクトファイルを作成し、解決をlk33に任せます。
複数のソースファイルを使用する開発ではスコープを他のソースファイルにも広げる必要があり、as33にはそのための擬似命令(.global、.comm)が用意されています。この命令によってシンボルのグローバル宣言を行い、他のソースファイルでも参照可能にします。
定義したファイル内でのみ参照可能なシンボルをローカルシンボル、グローバル宣言されたシンボルをグローバルシンボルと呼びます。ローカルシンボルの場合、複数のファイルで同一のシンボル名を指定しても、それぞれ別のシンボルとして扱われます。グローバルシンボルは複数のファイルで多重定義するとlk33でワーニングとなります。

例: **グローバルシンボルを定義するファイルfile1**

```
        .global   SYMBOL          ...シンボルを定義するファイルでグローバル宣言
SYMBOL:
                    :
                    :
LABEL:                            ...ローカルシンボル
                    :              (このファイル内でのみ参照)
        .comm     VAR1     4
```

グローバルシンボルを参照するファイルfile2

```
        ext       SYMBOL@rh
        ext       SYMBOL@m
        call      SYMBOL@rl       ...シンボルを外部参照
                    :
        ext       VAR1@h
        ext       VAR1@m
        ld.w      %r1, VAR1@l     ...シンボルを外部参照
LABEL:                            ...ローカルシンボル
                    :             (file1のLABELとは別のシンボルとして扱われます。)
```

as33はfile2のシンボルSYMBOLおよびVAR1をアドレス未定としてアセンブルし、その情報を出力するオブジェクトファイルに含めます。それらのアドレスはリンクの処理によって最終的に決定します。

11.7 セクション定義

ソースファイルには、CPUや周辺回路を制御するプログラムの他に、キャラクタジェネレータなどの初期化が不要な常時固定のデータ、RAMに置かれる変数のシンボルやI/Oメモリの制御レジスタなども記述されます。最終的にはこれらの異なる属性を持つデータやシンボルを、プログラムはROMのプログラム領域に、固定データはROMのデータ領域にといった具合に、リンカによってそれぞれに対応した物理メモリに再配置する必要があります。このために、オブジェクトコードを属性ごとにセクションとして分類できるようにしています。
セクションの種類としては次の3つがあります。

1. CODEセクション　プログラムのブロック
2. DATAセクション　ROMに書き込むデータのブロック
3. BSSセクション　RAMなどに割り当てるブロック

アセンブリソースファイル内でこれらのセクションを指定するために、as33には擬似命令が用意されています。
Cソースではコンパイラが擬似命令を生成するため、セクションを意識したプログラミングは不要です。

●CODEセクション

.code擬似命令はCODEセクションを定義します。この命令から、他のセクション定義命令までのステートメントはプログラムコード/データとみなされ、CODEセクションのオブジェクトとなります。ソースファイルはデフォルトでCODEセクションとみなされます。したがって、ファイルの先頭から他のセクション定義までの間はCODEセクションとして処理されます。

●DATAセクション

.data擬似命令はDATAセクションを定義します。この命令から、他のセクション定義命令までのステートメントはデータとみなされ、DATAセクションのオブジェクトとなります。したがって、この領域にはアドレスを参照するためのシンボルと、データ定義擬似命令(.word、.half、.byte、.ascii、.space)、アライメント擬似命令(.align)、コメント以外は記述できません。
CODEセクションにもデータは記述できますが、リンク後にデータのブロックをプログラムとは別にまとめたい場合には、DATAセクションに記述しておく必要があります。

●BSSセクション

.comm擬似命令および.lcomm擬似命令は変数領域のシンボルとサイズを定義する命令で、どちらかの命令が記述されると、そのシンボルをBSSセクションに設定します。BSSセクションは基本的にRAM領域ですが、I/Oメモリなどのデータメモリ領域にも使用可能です。S1C33 Familyのような組み込み型マイコンでは、この領域へのコード定義は意味を持ちません。上記2命令以外の命令や宣言等が続いた場合、それらはBSSセクションの直前のものと同一のセクションに配置されるように処理されます。
オブジェクトとしては実データを持ちませんが、シンボルおよびマップ情報を生成するために必要です。

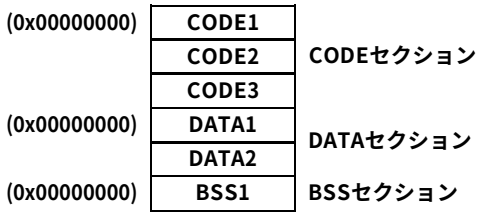
●リロケータブルソースのセクション管理

リロケータブルアセンブルでは、同一セクションが連続するようにまとめられ、CODE、DATA、BSSセクションが各1個の3セクションで構成されたオブジェクトファイルを生成します。データがまったく記述されていない、または定義を行わなかったセクションについても、オブジェクトファイルにはセクション情報が出力されます。

セクション定義例

```
        :
        CODE1(プログラム)
        :
.data
        :
        DATA1(データ定義)
        :
.comm           RAM0,1
        :
        BSS1(RAM領域定義)
        :
.code
        :
        CODE2(プログラム)
        :
.data
        :
        DATA2(データ定義)
        :
.code
        :
        CODE3(プログラム)
        :
```

上記のようなセクション定義を行うと、as33は次のように構成されたオブジェクトファイルを生成します。



●アブソリュートソースのセクション管理

アブソリュートアセンブルでは、同一セクションを前述のセクション定義擬似命令または.org擬似命令で分割した場合、すべてを独立したセクションとしてオブジェクトファイルを生成します。また、セクション定義擬似命令でセクションの種類を変更する場合は、.org擬似命令を使用して個々のセクションの開始アドレスを指定する必要があります。

セクション定義例

```
.abs
.code
.org            0x80100
        :
        CODE1(プログラム)        →セクション1
        :
.data
.org            0x80f00   ...この指定を行わないと、CODE1に続くアドレスからDATAセクション
        :                     を開始します。
        DATA1(データ定義)        →セクション2
        :
.org            0x0
.comm           RAM0,1           →セクション3(BSS領域定義)
```

上記のようなセクション定義を行うと、as33は3つのセクションを持つアブソリュートオブジェクトファイルを生成します。

●注意事項

あるセクション内に、他のセクション用のステートメントが現れるとエラーとなります。

例: .data

```
    ld.w        %r1, %r0   ...エラー
```

11.8 アセンブラ擬似命令

アセンブラ擬似命令は、実行コードに変換される命令ではなく、アセンブルを制御したりデータを設定する命令です。
他の命令と区別するため、アセンブラ擬似命令はすべてピリオド(.)で始まります。
命令はすべて小文字で記述してください。パラメータは大文字と小文字が区別されます。

11.8.1 アブソリュートアセンブル擬似命令(.abs)

.abs擬似命令

●命令の形式

.abs

●機能

アブソリュートアセンブルを指定します。この指定により、as33はそのファイルをアブソリュートファイルとしてアセンブルを行います。なお、デフォルトではファイルの先頭が0x0番地となります。

●注意事項

・.abs擬似命令は他の基本命令や擬似命令に先立って指定することが必要です。ファイルの1行目に記述してください。

・アブソリュートアセンブル専用の擬似命令(.org、.set)は.abs擬似命令が記述されていないと使用できません。使用した場合、エラーとなります。

11.8.2 セクション定義擬似命令(.code, .data)

.code擬似命令

●命令の形式

.code

●機能

CODEセクションの開始を宣言します。本命令以降のステートメントは、他のセクションが宣言されるまで、CODEセクションとしてアセンブルされます。
CODEセクションがas33のデフォルト設定です。したがって、ソースファイルの先頭では.code擬似命令を省略できます。他のセクションからCODEセクションに変更する場合は必ず記述してください。
セクションの詳細については"11.7 セクション定義"を参照してください。

●注意事項

・CODEセクションをソースファイル中の複数箇所に分割して定義することができます(それぞれの開始位置に.code擬似命令を記述)。
ただし、アブソリュートアセンブルでは別のセクションとして定義されますので注意してください。

・アブソリュートソースの場合は、.code擬似命令の前または次の行で必ず.org擬似命令によるアドレスの指定を行ってください。

.data擬似命令

●命令の形式

.data

●機能

DATAセクションの開始を宣言します。本命令以降のステートメントは、他のセクションが宣言されるまで、DATAセクションとしてアセンブルされます。
セクションの詳細については"11.7 セクション定義"を参照してください。

●注意事項

・DATAセクションにはデータ定義擬似命令(.word、.half、.byte、.ascii、.space)、アライメント擬似命令(.align)、ロケーションカウンタ制御擬似命令(.org)、シンボル、コメント以外記述できません。
記述した場合、エラーとなります。

・DATAセクションをソースファイル中の複数箇所に分割して定義することができます(それぞれの開始位置に.data擬似命令を記述)。
ただし、アブソリュートアセンブルでは別のセクションとして定義されますので注意してください。

・アブソリュートソースの場合は、.data擬似命令の前または次の行で必ず.org擬似命令によるアドレスの指定を行ってください。

11.8.3 領域確保擬似命令(.comm, .lcomm)

.comm擬似命令

●命令の形式

.comm <シンボル>[,] <サイズ>

シンボル: メモリアクセス(アドレス参照)用のシンボル
- 1文字目はa～z、A～Z、_に限られます。
- 2文字目以降はa～z、A～Z、0～9、_が使用可能です。
- シンボル名に使用可能な文字数は128文字までです。
- 大文字と小文字は区別されます。
- 命令とシンボルの間には1個以上のスペース、タブまたはカンマ(,)が必要です。

サイズ: 確保する領域のバイト数
- 10進数、2進数、16進数の数値のみ記述可能です。
- シンボルとサイズの間には1個以上のスペース、タブまたはカンマ(,)が必要です。

記述例: .comm FOO 4

●機能

BSSセクション内に指定のサイズの領域を設定し、その先頭アドレスを示すシンボルを指定の名称で生成します。このシンボルを使用してRAM等をアクセスする命令を記述することができます。
定義したシンボルは他のモジュールから外部参照が可能なグローバルシンボルとなります。
BSSセクションとして処理されるのは、.commおよび.lcomm擬似命令のみです。これらの命令以外のステートメントが引き続いて記述されると、それらは直前までのものと同種のセクションとして処理されます。
セクションの詳細については"11.7 セクション定義"を参照してください。

●注意事項

- .comm擬似命令によりBSSセクションをソースファイル中の複数箇所に分割して定義することができます(それぞれの開始位置に.comm擬似命令を記述)。
  ただし、アブソリュートアセンブルでは別のセクションとして定義されますので注意してください。1つのソースファイル内に定義可能なすべてのセクションの合計は最大256個までです。
- シンボルが示すアドレスは、サイズの値によってアライメントされます。
  サイズ: 1 ...バイト境界
  2 ...ハーフワード境界
  3以上 ...ワード境界

.lcomm擬似命令

●命令の形式

.lcomm　<シンボル>[,]　<サイズ>

シンボル: メモリアクセス(アドレス参照)用のシンボル
- 1文字目はa～z、A～Z、_に限られます。
- 2文字目以降はa～z、A～Z、0～9、_が使用可能です。
- シンボル名に使用可能な文字数は128文字までです。
- 大文字と小文字は区別されます。
- 命令とシンボルの間には1個以上のスペース、タブまたはカンマ(,)が必要です。

サイズ: 確保する領域のバイト数
- 10進数、2進数、16進数の数値のみ記述可能です。
- シンボルとサイズの間には1個以上のスペース、タブまたはカンマ(,)が必要です。

記述例: .lcomm　BAR　0x10

●機能

BSSセクション内に指定のサイズの領域を設定し、その先頭アドレスを示すシンボルを指定の名称で生成します。このシンボルを使用してRAM等をアクセスする命令を記述することができます。定義したシンボルはローカルシンボルとなり、他のモジュールからは参照することができません。
BSSセクションとして処理されるのは、.commおよび.lcomm擬似命令のみです。これらの命令以外のステートメントが引き続いて記述されると、それらは直前までのものと同種のセクションとして処理されます。
セクションの詳細については"11.7 セクション定義"を参照してください。

●注意事項

- .lcomm擬似命令によりBSSセクションをソースファイル中の複数箇所に分割して定義することができます(それぞれの開始位置に.lcomm擬似命令を記述)。
  ただし、アブソリュートアセンブルでは別のセクションとして定義されますので注意してください。1つのソースファイル内に定義可能なすべてのセクションの合計は最大256個までです。
- シンボルが示すアドレスは、サイズの値によってアライメントされます。
  サイズ: 1　...バイト境界
  2　...ハーフワード境界
  3以上　...ワード境界

11.8.4 ロケーションカウンタ制御擬似命令(.org)

.org擬似命令

●命令の形式

.org <アドレス>

アドレス: 絶対アドレス指定
- 10進数、2進数、16進数の数値のみ記述可能です。
- 指定可能なアドレスは0から0xffffffffまでです。
- 命令とアドレスの間には1個以上のスペース、タブまたはカンマ(,)が必要です。

記述例:
```
.org    0
.org    0x80000
```

●機能

アブソリュートアセンブリソースファイル内で絶対アドレスを指定します。as33は、この命令以降のステートメントが指定されたアドレスから始まるものとしてアセンブルを行います。
また、指定アドレスから新たなセクションを開始します。

●注意事項

- .org擬似命令は.abs擬似命令が記述されていないと(リロケータブルソース内では)使用できません。使用した場合、エラーとなります。
- .org擬似命令はセクションの開始アドレスを指定のアドレスに設定します。ただし、奇数番地等を指定した場合は、次に記述された命令等に従った境界アドレスにアライメントされることがありますので注意が必要です。

11.8.5　シンボル定義擬似命令(.set)

.set擬似命令

●命令の形式

.set　<シンボル>[,]　<アドレス>

シンボル: メモリアクセス(アドレス参照)用のシンボル
- 1文字目はa～z、A～Z、_に限られます。
- 2文字目以降はa～z、A～Z、0～9、_が使用可能です。
- 大文字と小文字は区別されます。
- 命令とシンボルの間には1個以上のスペース、タブまたはカンマ(,)が必要です。

アドレス: 絶対アドレス指定
- 10進数、2進数、16進数の数値のみ記述可能です。
- 指定可能なアドレスは0から0xffffffffまでです。
- シンボルとアドレスの間には1個以上のスペース、タブまたはカンマ(,)が必要です。

記述例:　.set　DATA1　0x80000

●機能

シンボルに絶対アドレス(32ビット)を定義します。

●注意事項

- .set擬似命令は.abs擬似命令が記述されていないと(リロケータブルソース内では)使用できません。使用した場合、エラーとなります。
- 設定したシンボルはローカルシンボルとなります。グローバルシンボルとして使用する場合は、.global擬似命令によるグローバル宣言を行ってください。

●参考

汎用データや文字列の定義はプリプロセッサの#define擬似命令を使用してください。(9.5.2項参照)

11.8.6 データ定義擬似命令(.word, .half, .byte, .ascii, .space)

.word擬似命令

●命令の形式

形式1) **.word <データ>**[[,] **<データ>・・・**[,] **<データ>**]
形式2) **.word <シンボル>**

データ: ワードデータ(32ビット)
- 10進数、2進数、16進数の数値のみ記述可能です。
- 指定可能なデータは0から0xffffffffまでです。
- 命令と最初のデータ間には1個以上のスペース、タブまたはカンマ(,)が必要です。
- データ間にはスペース、タブまたはカンマ(,)が必要です。

シンボル: シンボル名

記述例:
```
.word   0x10000000   0x20000000   0x30000000   0x4000000
.word   256
.word   FOO
```

●機能

形式1) ワードデータを定義します。CODEセクションおよびDATAセクションに定義可能です。

形式2) シンボルの値をワードデータとして定義します。CODEセクションおよびDATAセクションに定義可能です。

●注意事項

- .word擬似命令はCODEまたはDATAセクション内で使用可能です。
- 形式1では複数のデータが定義可能です。ただし、1行は空白文字も含め255文字に限られます。
- 定義したデータは直前に.align擬似命令がない限り、ワード境界アドレスから配置されます。現在位置がワード境界アドレス以外の場合、データを配置するワード境界アドレスまでの間は0x00が設定されます。

.half擬似命令

●命令の形式

.half <データ>[[,] **<データ>・・・**[,] **<データ>**]

データ: ハーフワードデータ(16ビット)
- 10進数、2進数、16進数の数値のみ記述可能です。
- 指定可能なデータは0から0xffffまでです。
- 命令と最初のデータ間には1個以上のスペース、タブまたはカンマ(,)が必要です。
- データ間にはスペース、タブまたはカンマ(,)が必要です。

記述例:
```
.half   0xfffc   0xfffd   0xfffe   0xffff
.half   256
```

●機能

ハーフワードデータを定義します。CODEセクションおよびDATAセクションに定義可能です。

●注意事項

- .half擬似命令はCODEまたはDATAセクション内で使用可能です。
- 複数のデータが定義可能です。ただし、1行は空白文字も含め255文字に限られます。
- 定義したデータは直前に.align擬似命令がない限り、ハーフワード境界アドレスから配置されます。現在位置が奇数アドレスの場合、現在位置には0x00が設定されます。

.byte擬似命令

●命令の形式

.byte　<データ>[[,]　<データ>・・・[,]　<データ>]

データ:　バイトデータ(8ビット)
- 10進数、2進数、16進数の数値のみ記述可能です。
- 指定可能なデータは0から0xffまでです。
- 命令と最初のデータ間には1個以上のスペース、タブまたはカンマ(,)が必要です。
- データ間にはスペース、タブまたはカンマ(,)が必要です。

記述例:
```
.byte    0xfc    0xfd    0xfe    0xff
.byte    255
```

●機能

バイトデータを定義します。CODEセクションおよびDATAセクションに定義可能です。

●注意事項

- .byte擬似命令はCODEまたはDATAセクション内で使用可能です。
- 複数のデータが定義可能です。ただし、1行は空白文字も含め255文字に限られます。
- 定義したデータは直前に.align擬似命令がない限り、現在のアドレスに配置されます。
  CODEセクションの偶数アドレスにバイトデータを定義し次に命令が記述されている場合、その命令がハーフワード境界から始まるように、データを定義したアドレスの次の奇数アドレスには0x00が設定されます。

.ascii擬似命令

●命令の形式

.ascii　"<文字列>"

文字列:　ASCII文字列
- 指定可能なサイズは0から0xffまでです。
- 文字列にはASCIIキャラクタおよび¥記号で始まるエスケープシーケンスが記述可能です。たとえば、文字列中に二重引用符を設定する場合は¥"、¥を設定する場合は¥¥と記述します。
- 命令と文字列間には1個以上のスペース、タブまたはカンマ(,)が必要です。

記述例:
```
.ascii    "abcd ¥"E¥" fg"    (=abcd "E" fg)
```

●機能

ASCII文字列を定義します。CODEセクションおよびDATAセクションに定義可能です。

●注意事項

- .ascii擬似命令はCODEまたはDATAセクション内で使用可能です。
- 定義したデータは直前に.align擬似命令がない限り、現在のアドレスから配置されます。

.space擬似命令

●命令の形式

.space <サイズ>

サイズ: 0x0を埋め込むバイト数
- 10進数のみ記述可能です。
- 指定可能なサイズは1から2147483647までです。
- 命令とサイズ間には1個以上のスペース、タブまたはカンマ(,)が必要です。

記述例: .space 16

●機能

指定サイズ分の領域を0x0に設定します。
CODEセクションおよびDATAセクションに定義可能です。

●注意事項

- .space擬似命令はCODEまたはDATAセクション内で使用可能です。
- 直前に.align擬似命令がない限り、現在のアドレスから指定サイズ分の領域が0x0に設定されます。

定義データのアライメントについて

データ定義擬似命令によって、データは直前に.align擬似命令がない限り、データサイズに従った境界アドレスから配置されます。
また、CODEセクションでは命令がハーフワード境界から配置されます。したがって、定義したデータの最終アドレスから次の命令あるいはデータまでの間に空白が生じることがあります。その空白となるアドレスには0x0が設定されます。

例:
```
jp      SYMBOL
.byte   0x41            ...0x41の後ろに0x0が設定されます。
ld.w    %r1, %r7

.word   0x00
.byte   0x01            ...0x01の後ろに3バイト分の0x0が設定されます。
.word   0x02
```

11.8.7 アライメント擬似命令(.align)

.align擬似命令

●命令の形式

.align <境界指定値>

境界指定値: 境界を指定する数値
・ 指定可能な数値は0から8までです。
2^0～2^8バイトの境界へのアライメントを指定します。
・ 命令と指定値間には1個以上のスペース、タブまたはカンマ(,)が必要です。

記述例: .align 2 (4バイト境界へのアライメント)

●機能

この擬似命令の直後に出現するデータを2^Nバイト境界にアライメントします(N=境界指定値)。

●注意事項

.align擬似命令は直後にあるデータ定義擬似命令に対してのみ有効です。したがって、アライメントが必要なデータを定義する場合には、データ定義擬似命令個々に.align擬似命令を使用する必要があります。

11.8.8 グローバル宣言擬似命令(.global)

.global擬似命令

●命令の形式

.global <シンボル>

シンボル: 現在のファイルで定義するグローバルシンボル
・ 命令とシンボル間には1個以上のスペース、タブまたはカンマ(,)が必要です。

記述例: .global SUB1

●機能

シンボルのグローバル宣言を行います。シンボルを定義したファイル内での宣言は、そのシンボルを他のモジュールから参照可能なグローバルシンボルに設定します。

●注意事項

・ 他のモジュールから参照されるシンボルはグローバル宣言が必要です。
.comm擬似命令で定義したシンボルはグローバルシンボルとなりますので、.global擬似命令による宣言は不要です。

・ 現在のファイル内に宣言されていないシンボルは、他のファイルで宣言されているグローバルシンボルとして処理されます。

11.8.9 リスト制御擬似命令(.list, .nolist)

.nolist擬似命令

●命令の形式

.nolist

●機能

アセンブリリストファイルへの出力を制御します。
.nolist擬似命令は、それ以降のアセンブリリストファイルへの出力を停止します。
デフォルト設定では(指定のない場合)、すべてのステートメントがアセンブリリストファイルに出力されます。

●注意事項

as33は-lオプションを指定して起動した場合にのみ、アセンブリリストファイルを出力します。
したがって、-lオプションを指定しない場合、この擬似命令は無効です。

.list擬似命令

●命令の形式

.list

●機能

アセンブリリストファイルへの出力を制御します。
.list擬似命令は.nolist擬似命令によって停止した出力を記述した位置から再開します。

●注意事項

as33は-lオプションを指定して起動した場合にのみ、アセンブリリストファイルを出力します。
したがって、-lオプションを指定しない場合、この擬似命令は無効です。

11.8.10 デバッグ擬似命令(.file, .endfile, .loc, .def)

.file, .endfile, .loc擬似命令

●命令の形式

(1) **.file** "**<ファイル名>**"
(2) **.endfile**
(3) **.loc** <行番号>

●機能

これらの擬似命令は、ソース情報をオブジェクトファイルに付加するためのもので、Cコンパイラgcc33およびプリプロセッサpp33が生成します。
as33は、これらの命令に従ったデバッグ情報を含めてsrf33形式のオブジェクトファイルを出力します。
これらのデバッグ情報はデバッガdb33によってソースレベルデバッグを行う場合に必要です。
as33の-gオプションが指定されていない場合でも、これらのデバッグ情報がカットされることはありません。
.file擬似命令はソースファイルの開始位置情報を出力します。この擬似命令に続くコードは<ファイル名>で指定されたファイルの内容であることを示します。ソースファイルの先頭、あるいはインクルードファイルなど、ファイルが変更される個所に挿入されます。
.endfile擬似命令はファイルの終了位置情報を出力します。インクルードファイルの終了位置には挿入されません。
.loc擬似命令はソースファイル内における命令の行番号情報を出力します。コメントなど命令以外の行には付加されません。

.def擬似命令

●命令の形式

.def <シンボル名> [, <パラメータ>,…<パラメータ>] , endef

●機能

この擬似命令は、Cソースのシンボル情報をオブジェクトファイルに付加するためのもので、gcc33がシンボルごとに生成します。pp33はこの情報を生成しません。
as33は、-gオプションが指定されると、この命令に従ったデバッグ情報を含めてsrf33形式のオブジェクトファイルを出力します。
これらのデバッグ情報はデバッガdb33によってCソースのシンボリックデバッグを行う場合に必要です。
as33の-gオプションが指定されていないと、これらのデバッグ情報はカットされます。

<パラメータ>はシンボルの型やストレージクラスなどの情報を示します。内容については"6.6 デバッグ情報"を参照してください。

Cコンパイラ、プリプロセッサによるデバッグ擬似命令の挿入

gcc33およびpp33の各起動オプションとして-g(ワークベンチの[debug info]をチェック)が指定されると、それぞれ上記デバッグ擬似命令を出力ファイル(アセンブリソースファイル)に挿入します。.def擬似命令はpp33では挿入されません。
したがって、ソースファイル作成時にこれらの擬似命令を記述する必要はありません。

11.9 アセンブリリストファイル

アセンブリリストファイルは、アセンブリソースファイルの各行の前半にアセンブル結果(アドレスやオブジェクトコード)を付加したファイルです。起動時オプション(-l)が指定された場合にのみ出力されます。.nolist擬似命令がソース中に記述されている場合は、それ以降、ファイルの終わりまたは.list擬似命令までの内容は出力されません。

ファイル形式はテキストファイルで、ファイル名は"<ファイル名>.lst"(<ファイル名>は入力ソースファイルと同じ名称)です。

アセンブリリストファイルの各行の形式は次のとおりです。

アドレス　コード　ソースステートメント

例:

```
                                .file     "boot.s"
                        ; boot.s  1997.2.13
                        ; boot program

                        ;#defnum SP_INI 0x0800        ; sp is in end of 2KB internal RAM
                        ;#defnum GP_INI 0x0000        ; global pointer %r8 is 0x0

                                .code
00000000  00000000              .word BOOT ; BOOT VECTOR
                        BOOT:
                                .loc      10
00000004  c020                  ext       0x20      ;         xld.w     %r8,0x800
00000006  6c08                  ld.w      %r8,0x0
                                .loc      11
00000008  a081                  ld.w      %sp,%r8   ; set SP
                                .loc      12
0000000a  6c08                  ld.w      %r8,0x0   ; set global pointer
                                .loc      13
0000000c  c000                  ext       main@rh   ;         xcall     main      ; goto main
0000000e  c000                  ext       main@rm
00000010  1c00                  call      main@rl
                                .loc      14
00000012  1e00                  jp        BOOT      ;         xjp       BOOT      ; infinity loop

                                .data
00000000  00000001              .word 1 2
          00000002

00000000  00                    .comm tmp 4
                                .endfile
```

アドレスの内容

アブソリュートモジュールの場合: 絶対アドレスが16進数で出力されます。
リロケータブルモジュールの場合: 各セクションの先頭からの相対アドレスが16進数で出力されます。

コードの内容

CODEセクション:　命令コードと定義したデータが16進数で出力されます。
DATAセクション:　データ定義擬似命令で定義したデータが16進数で出力されます。
BSSセクション:　確保した領域のサイズにかかわらず、すべて00が出力されます。

＊ BSSセクションのアドレスは、シンボルに定義されたアドレス(確保した領域の先頭アドレス)のみ出力されます。

注意事項

シンボルを参照しているコードの即値部分は、アセンブル時には基本的に0となります。
これらの即値はすべてリンク時に決定します。

11.10 エラー/ワーニングメッセージ

エラーおよびワーニングメッセージは標準出力(stdout)に表示/出力されます。-eオプションを指定した場合は、ファイル"as33.err"にも出力されます。
wb33の[AS33]ボタンでas33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

11.10.1 エラー

as33で発生するエラーは、システムエラーとアセンブラエラーの2つに分類されます。

表11.10.1.1 システムエラーメッセージ一覧

エラーメッセージ	内容
<file name>: Error: Out of memory	メモリが確保できません。
<file name>: Error: File open error	ファイルがオープンできません。
<file name>: Error: File access error	オープンしたファイルがリード/ライトできません。

アセンブラエラーは、ソース中にas33が処理できない構文や記述があった場合に発生します。このエラーが発生した場合、オブジェクトファイルは出力されません。アセンブリリストファイルはエラーメッセージを含めて出力されます。

表11.10.1.2 アセンブラエラーメッセージ一覧

エラーメッセージ	内容
Error: Invalid file name.	ソースファイルが出力ファイルと同じ拡張子(.o)を持っています。
Error: Filename length limit exceeded - 255.	ファイル名の長さが255文字を越えました。
Error: Directory path length limit exceeded - 255.	ディレクトリを含めたファイル名の長さが255文字を越えました。
Error: Line length limit exceeded - 255.	1行の文字数が255文字を越えました。
Error: Symbol name length limit exceeded - 32.	シンボル名が128文字を越えました。
Error: Token length limit exceeded - 64.	数値データが64桁を越えました。
Error: Multiple statements on the same line.	1行に複数のステートメントがあります。
Error: Invalid statement syntax.	不正な文字が使われています。
Error: Invalid instruction. - "<instruction>"	不正なインストラクション名が使われています。
Error: Invalid register. - "<register>"	不正なレジスタ名が使われています。
Error: Invalid directive. - "<directive>"	不正な擬似命令が使われています。
Error: Invalid symbol mask. - "<mask>"	不正なシンボルマスクが使われています。
Error: Invalid instruction syntax.	インストラクションの文法が不正です。
Error: Invalid directive syntax.	擬似命令の文法が不正です。
Error: Multiply declared symbol. - "<symbol>"	シンボルが二重に宣言されています。
Error: Multiply defined symbol. - "<symbol>"	シンボルが二重に定義されています。
Error: Incorrect section type for statement.	セクションタイプが不正です。
Error: Memory mapping conflict.	メモリ配置でアドレスが重複しています。
Error: Section count limit exceeded - 256.	セクション数が256を越えました。

* "Error"の前には"<file name>(<line No.>)"の形式で入力ファイル名と行番号が表示されます。また、デバッグ情報を含むソースファイルを入力した場合、メッセージの後ろに"near <file name>(<line No.>)"が表示されます。内容はデバッグ情報で示されるオリジナルのソースファイル名(*.c、*.s)と行番号です。

11.10.2 ワーニング

ワーニングの場合、アセンブラは処理を継続します。他のエラーが発生しなければ、ワーニングメッセージを表示して処理を終了します。オブジェクトファイル、アセンブリリストファイルも出力されます。

表11.10.2.1 ワーニングメッセージ一覧

ワーニングメッセージ	内容
Warning: Numeric range.	インストラクションオペランドの数値が不正です。
Warning: Unknown escape sequence.	.ascii 擬似命令に不正なエスケープシーケンスが使われています。
Warning: Escape sequence out of range for character.	.ascii 擬似命令中の 8/16 進で表現される文字コードが 0xff を越えています。
Warning: \x used with no following hex digits.	.ascii 擬似命令の 16 進数値指定に 16 進で表記される数値がありません。

* "Warning"の前には"<file name>(<line No.>)"の形式で入力ファイル名と行番号が表示されます。また、デバッグ情報を含むソースファイルを入力した場合、メッセージの後ろに"near <file name>(<line No.>)"が表示されます。内容はデバッグ情報で示されるオリジナルのソースファイル名(*.c、*.s)と行番号です。

11.11 注意事項

(1) リンク可能なオブジェクトファイル数はライブラリモジュールも含め最大4000個です。この制限を越えると、リンカでエラーとなります。

(2) db33でCソースレベルデバッグ、シンボリックデバッグを行う場合には必ず-gオプションを指定してas33を実行してください。
gcc33(pp33も同様)で-gオプションを指定しても、as33で-gオプションを指定しないとシンボル情報がすべてカットされます。ソース情報はカットされません。
gcc33(pp33も同様)で-gオプションを指定せずにas33で-gオプションを指定した場合、シンボル名とアドレスのみのシンボル情報がアセンブル時に付加されます。
また、lk33で-gオプションを指定しないとデバッグ情報はリンクの際にすべてカットされます。

ソースファイル内のデバッグ情報(デバッグ擬似命令)はgcc33およびpp33の-gオプションを指定する以外の方法で作成しないでください。また出力されたデバッグ情報を修正しないでください。as33、lk33、db33、dis33等の誤動作につながります。

12 リンカ

この章ではリンカlk33の機能を説明します。

12.1 機能

リンカlk33(以下lk33と記述)は実行可能なオブジェクトファイルを生成するソフトウェアで、以下の機能があります。

- ライブラリを含め、複数のオブジェクトモジュールをリンクし、1つの実行可能なアブソリュートオブジェクトファイルを生成
- モジュール間の外部参照を解決
- リロケータブルアドレスをアブソリュートアドレスに再配置
- リンク後のオブジェクトファイルに行番号やシンボル情報などのデバッグ情報を出力
- リンクマップファイル、シンボルファイル、クロスリファレンスファイルを出力可能

lk33 Ver. 3.0から、複数の同じ種類のセクションを1つのブロックとして扱えるようになっています。これにより、複数のセクションを一度に仮想・共用エリアに割り付け可能となるなど、以前のバージョンより機能が強化されています。ただし、これには新たに設けられたブロック設定用のコマンドを使用する必要があり、従来のセクション配置コマンドとは互換性がありません。-v3コマンドを指定するとVer. 3.0のコマンドが有効となり、未指定の場合は従来のコマンド体系で処理されます。したがって、以前のバージョンに対応したリンカコマンドファイルもそのまま使用可能です。
本章では、リンカコマンドとリンクの説明をVer. 2.1以前とVer. 3.0に分けて行います。旧コマンド体系による処理については"12.5 lk33 Ver. 2.1以前のリンク処理"を、新コマンド体系による処理については"12.6 lk33 Ver. 3.0のリンク処理"を参照してください。その他の説明はバージョンにかかわらず共通です。

12.2 入出力ファイル

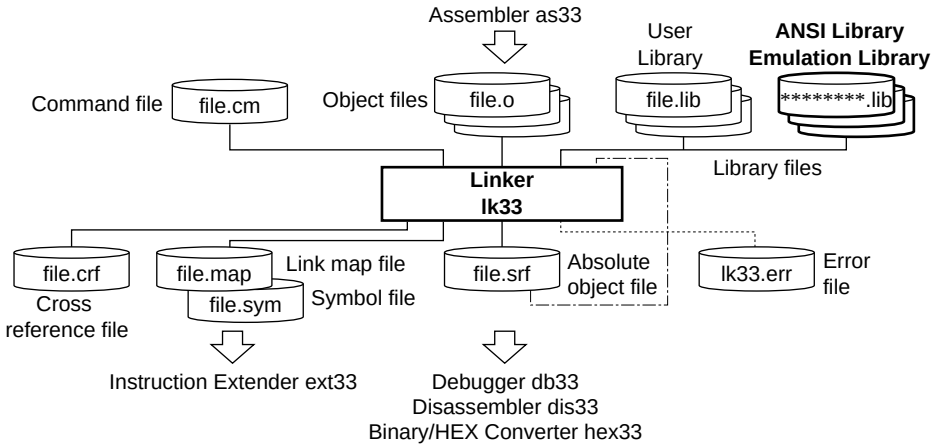


図12.2.1 **フローチャート**

12.2.1 入力ファイル

●オブジェクトファイル

ファイル形式: srf33形式のバイナリファイル
ファイル名: <ファイル名>.o
内容: アセンブラas33で生成したモジュール個々のオブジェクトファイルです。

●ライブラリファイル

ファイル形式: ライブラリ形式のバイナリファイル
ファイル名: <ファイル名>.lib
内容: ANSIライブラリ、エミュレーションライブラリおよびライブラリアンlib33で作成したユーザーのライブラリファイルです。

●リンカコマンドファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.cm
内容: 入力するオブジェクトファイル名や各セクションの開始アドレスなどを指定するファイルです。wb33の[Make edit]によってコマンドファイルのテンプレートが作成されますので、汎用エディタで修正して使用してください。
起動時オプションで-cが指定された場合にlk33に入力されます。
内容については12.5.1(Ver. 2.1以前)または12.6.1(Ver. 3.0)の"リンカコマンドファイル"を参照してください。

●アブソリュートオブジェクトファイル

ファイル形式: srf33形式のバイナリファイル
ファイル名: <ファイル名>.srf
内容: リンカで作成した実行形式のオブジェクトファイルです。-inclinkオプションを指定した場合にのみリンク可能です。".srf"ファイルはすでに絶対アドレスが決定しているため、アドレス指定および再配置は行えません。
また、-ldオプションにより作成したローダ用アブソリュートオブジェクトファイルはリンクできません。

12.2.2 出力ファイル

●アブソリュートオブジェクトファイル

ファイル形式: srf33形式のバイナリファイル
ファイル名: <ファイル名>.srf
出力先: カレントディレクトリ
内容: デバッガdb33に入力可能な実行形式のオブジェクトファイルです。1つのプログラムを構成するすべてのモジュールがリンクされ、すべてのコードが配置される絶対アドレスが決定した内容となっています。デバッグに必要な情報もsrf33形式で含まれています。
ファイル名は、次のように決定します。
単一モジュールの場合: 入力オブジェクトファイルと同じ名称
リンクコマンドファイルを入力する場合: 最初にリンクするファイルと同じ名称、またはコマンドで指定した名称

出力されるオブジェクトファイルの内容についてはAppendixの"srf33ファイルの構造"を参照してください。

●リンクマップファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.map
出力先: カレントディレクトリ
内容: 各入力ファイルが各セクションのどのアドレスから配置されたかを示すマップ情報ファイルです。起動時オプションで-mが指定された場合に出力されます。
命令エクステンダext33が入力して、コードの最適化を行います。

●シンボルファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.sym
出力先: カレントディレクトリ
内容: すべてのモジュール内で定義されているシンボルとそのアドレス情報が出力されます。起動時オプションで-s指定された場合に出力されます。
命令エクステンダext33が入力して、コードの最適化を行います。

●クロスリファレンスファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.crf
出力先: カレントディレクトリ
内容: すべてのグローバルシンボル名と、その定義アドレス、参照アドレス情報が出力されます。起動時オプションで-cref指定された場合に出力されます。

●エラーファイル

ファイル形式: テキストファイル
ファイル名: lk33.err
出力先: カレントディレクトリ
内容: 起動時オプション(-e)が指定された場合に出力されるファイルで、エラーメッセージなど、リンカlk33が標準出力(stdout)に対して出力する内容を記録します。

12.3 起動方法

12.3.1 起動フォーマット

●コマンドラインの一般形

形式1) マルチモジュールのリンク
lk33 ^ [**起動時オプション**] ^ -c ^ <**リンカコマンドファイル名**>

形式2) 単一モジュールのリンク
lk33 ^ [**起動時オプション**] ^ <**オブジェクトファイル名**>

^はスペースを示します。
[ ]は省略可能なことを示します。

ファイル名は拡張子も含めて指定します。
形式2でも複数のオブジェクトファイルを指定することは可能です。ただし、コマンドラインの文字数にはOSによる制限がありますので、注意してください。

●ワークベンチからの起動

形式1) マルチモジュールのリンク
起動オプションとコマンドファイルを選択し、[LK33]ボタンをクリックします。[use .cm file]をチェックしておくことが必要です。

形式2) 単一モジュールのリンク
[use .cm file]のチェックを外してオブジェクトファイル(.o)を選択し、[LK33]ボタンをクリックします。

12.3.2 起動時オプション

lk33には6種類の起動時オプションが用意されています。

-g

機能: デバッグ情報の付加
wb33上の指定: [debug info]をチェック
説明:
- デバッグ情報を含む出力ファイルが生成されます。
- ソース表示、シンボリックデバッグを行う場合は、必ず指定してください。指定しない場合は、Cコンパイラgcc33、プリプロセッサpp33、アセンブラas33で付加されたデバッグ情報はすべてカットされます。

-c <リンカコマンドファイル名>

機能: リンカコマンドファイルの指定
wb33上の指定: [use .cm file]をチェック
説明:
- リンカコマンドファイルを入力します。

-s

機能: シンボルファイルの出力
wb33上の指定: [symbol,map file]をチェック
説明:
- シンボルファイルを出力します。

-m

機能: リンクマップファイルの出力
wb33上の指定: [symbol,map file]をチェック
説明:
- リンクマップファイルを出力します。

-cref

機能: クロスリファレンスファイルの出力
wb33上の指定: なし
説明:
- クロスリファレンスファイルを出力します。

-e

機能: エラーファイルの出力
wb33上の指定: なし
説明:
- エラーメッセージなど、lk33が標準出力(stdout)に対して出力する内容をファイル(lk33.err)にも出力します。

-c以外のオプションはコマンドファイル内に記述することもできます。

コマンドラインにオプションを入力する場合、オプションの前後には1個以上のスペースが必要です。
例: c:¥cc33¥lk33 -g -e -s -m -cref -c test.cm

注: **-cオプションでコマンドファイルを指定する場合、他のオプションは-cオプションの前に入力するか、コマンドファイル内に記述してください。-cオプションの後ろに入力されたオプションは無視されます。**

12.4 メッセージ

lk33はメッセージをすべて標準出力(stdout)に対して出力します。
wb33の[LK33]ボタンでlk33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

●終了メッセージ

正常に終了した場合は次の終了メッセージのみを出力します。

```
Link Completed
```

●Usage出力

ファイル名が指定されなかったり、オプション指定が正しくない場合、次の使用方法のメッセージを出力して終了します。

```
Linker 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 1997-2001
Usage:
    lk33 [options] <filenames>
Options:
    -e : produce log file (lk33.err)
    -g : generate debug information in output file
    -s : generate symbol information (.sym)
    -m : generate map information (.map)
    -cref : generate cross reference information (.crf)
    -c CommandFile : execute lk33 commands from CommandFile (.cm)
Output:
    SRF33 object file (.srf)
Example:
    lk33 -e -g -s -m -c test.cm
```

●エラー・ワーニング発生時

エラーが発生した場合は、終了メッセージの前にエラーメッセージが表示されます。

例: `Error: Too many global bss symbol.`
`Link Completed`

エラーの場合、リンカlk33は出力ファイルを作成せずに終了します。

ワーニングが発生した場合は、終了メッセージの前にワーニングメッセージが表示されます。

例: `test.o: Warning: Unresolved external symbol 'SUB1'.`
`Link Completed`

ワーニングの場合、リンカlk33は出力ファイルを作成して終了します。ただし、その動作は保証されません。

エラーとワーニングの詳細は"12.11 エラー/ワーニングメッセージ"を参照してください。

12.5 lk33 Ver. 2.1以前のリンク処理

12.5.1 リンカコマンドファイル

lk33は起動時のコマンドライン指定を簡略化するため、必要な指定を記述したリンカコマンドファイルを入力してリンク処理を行うことができます。入力するリンカコマンドファイルは、lk33起動時に-cオプションで指定します。

リンカコマンドファイル例

```
;Map set
-code 0x0C00000             ; CODEセクション開始アドレス
-data 0x0C01000             ; DATAセクション開始アドレス
-bss  0x0000000             ; BSSセクション開始アドレス

-code 0x0C00100 {test2.o}   ; test2.oのCODEセクション開始位置を固定

-defsym BOOT = 0x0C00000    ; グローバルシンボルの設定

;Library path
-l C:\CC33\lib              ; ライブラリパスの指定

;Executable file
-o test.srf                 ; 出力ファイル名の指定

;Object files
test1.o                     ; 入力ファイルの指定
test2.o

;Library files
io.lib                      ; ライブラリファイルの指定
lib.lib
math.lib
string.lib
ctype.lib
fp.lib
idiv.lib
```

リンカコマンドファイルは以下の規則に沿って作成してください。

●ファイルの形式

リンカコマンドファイルは、例に示したとおり一般的なテキスト形式です。
汎用エディタで作成してください。または、wb33の[Make file editor]によりテンプレートが作成されますので、そのファイルを修正して使用してください。
ファイル名の拡張子は".cm"としてください。

●コマンドの記述

コマンドはすべてハイフン(-)で始まります。個々のコマンドは1個以上のスペース、タブ、あるいは改行で区切る必要があります。視認性の面から、個々のコマンドは行を変えて記述することを推奨します。
1行には2048文字まで記述可能です。

注: • コマンドはすべて小文字で記述してください。大文字は受け付けません。

• アドレス等を指定する数値は16進数形式(0x####)で記述してください。10進数、2進数表記は受け付けません。

• 単一の設定のみ許されるコマンドが重複して指定された場合、最後に指定したコマンドが有効となります。
例: -code 0x0C00000
-code 0x0C00100 ...-code 0x0C00100が有効
重複が危険なコマンド(-code{}、-ucode{})はエラーとなります。

• シンボル名、Uセクション名、ファイル名には以下の文字のみ受け付けます。
1文字目: a〜z, A〜Z
2文字目以降: a〜z, A〜Z, 0〜9, _
ファイル名(パス)には"."と"¥"と":"も使用可能です。

●入力ファイル、ライブラリファイルの指定

入力するオブジェクトファイル名、ライブラリファイル名は、リンクコマンドファイルの最後に記述してください。ライブラリファイルはオブジェクトファイルより後に記述します。
リンクによる配置は、特に指定した場合を除き、記述した順序で行われます。
ファイル名は拡張子(.o、.lib、.srf)まで記述してください。
オブジェクトファイルを記述せずに、ライブラリファイルのみを指定することはできません。

●コメント

リンカコマンドファイルにはコメントを記述することができます。
ソースファイルと同様に、セミコロン(;)からその行の終わりまでの文字列がコメントとみなされます。

●空白行

空白文字と改行のみの空白行は無視されます。セミコロンでコメントにする必要はありません。

12.5.2 リンカコマンド一覧(Ver. 2.1)

リンカlk33 Ver. 2.1以前に対応するリンカコマンドは次に示す22種類です(コマンドラインで指定可能な起動時オプションも含む)。

表12.5.2.1 リンカコマンド一覧(Ver. 2.1対応)

コマンド	機　能
-c	コマンドファイルの指定 *1, *2
-m	リンクマップファイルの出力 *1
-s	シンボルファイルの出力 *1
-cref	クロスリファレンスファイルの出力 *1
-g	デバッグ情報の出力 *1
-e	エラーファイルの出力 *1
-w	ワーニングレベルの設定
-l	ライブラリパスの指定
-o	出力ファイル名の設定
-defsym	グローバルシンボルの設定
-d	重複グローバルBBS領域の削除
-align	アライメント指定
-code	リロケータブルなCODEセクション開始位置の設定 *3
-data	リロケータブルなDATAセクション開始位置の設定 *3
-bss	リロケータブルなBSSセクション開始位置の設定 *3
-ucode	仮想・共用CODEセクション開始位置の設定 *3
-udata	仮想・共用DATAセクション開始位置の設定 *3
-ubss	共用BSSセクション開始位置の設定 *3
-objsym	セクションシンボルの作成
-section	出力セクション名の指定 *3
-ld	ローダ用srf33ファイルの出力
-inclink	.srfファイルのリンク許可

*1: 起動時オプション
*2: コマンドファイル内では使用できません。
*3: Ver. 2.1コマンド(-v3指定時は使用できません。)

以下、各リンカコマンドについて説明します。(起動時オプションについては12.3.2節参照)
実際の使用例とリンク結果については次節を参照してください。

-wコマンド

形式: -w
記述例: -w
説明:
・ -wコマンドが指定された場合、BSSセクション内のグローバルラベルの重複に対してはワーニングを出力しません。
デフォルト: 本コマンドが指定されない場合、すべてのワーニングメッセージを出力します。

-lコマンド

形式: -l <ライブラリ検索パス>
記述例: -l c:¥cc33¥lib
説明:
・ ライブラリを検索するディレクトリを指定します。
・ -lと<ライブラリ検索パス>の間には1個以上のスペースまたはタブが必要です。
・ ライブラリ検索パスは最大4つまで指定可能です。複数のディレクトリを指定する場合は、それぞれについて、-lオプションによる指定を行ってください。
デフォルト: 本コマンドが指定されない場合、カレントディレクトリの検索のみとなります。
個々のライブラリファイル名の記述にパス名を含めることも可能です。

-oコマンド

形式: -o <出力ファイル名>.srf
記述例: -o test.srf
説明:
・ 出力ファイル名を指定します。
・ -oと<出力ファイル名>の間には1個以上のスペースまたはタブが必要です。
デフォルト: 本コマンドが指定されない場合、コマンドファイルに記載された入力オブジェクトファイル名の中で、最初に現れたファイル名を使用して出力ファイルが生成されます。

-defsymコマンド

形式: -defsym <**シンボル名**> = <**値**>
記述例: -defsym BOOT = 0x0080000
説明:
- グローバルシンボルの値を、このコマンドで定義します。
- -defsymと<シンボル名>の間には1個以上のスペースまたはタブが必要です。
- -defsymコマンドで定義可能なシンボル数は最大256個です。

デフォルト: 本コマンドが指定されない場合、グローバルシンボルは何も設定されません。

-dコマンド

形式: -d
記述例: -d
説明:
- BSSセクションに同じグローバルシンボル名を持つ複数の領域が設定されている場合、1つの領域を除いて他の領域を削除します。
- 有効となるのは、領域サイズが最も大きく、入力オブジェクトファイル名の指定の中で最も早く現れた領域です。

デフォルト: 本コマンドが指定されない場合、無効となるシンボルの領域は削除されません。
注意: 同じ名称のグローバルシンボルが定義されている場合はワーニングとなります。
-dコマンドはBSSセクション以外のセクションに対しては無効です。

-alignコマンド

形式: -align <**アライメント値**>
記述例: -align 0x2
説明:
- -code、-data、-bss、-ucode、-udata、-ubssコマンドで設定するセクション開始アドレスのアライメントを定義します。
- アライメント値にnを指定すると、境界アドレスは2^nバイトに設定されます。
  例） -align 0x02 ...4バイト境界
  -align 0x03 ...8バイト境界
- -code、-data、-bss、-ucode、-udata、-ubssコマンドで、セクション開始アドレスを設定された境界アドレス以外に設定した場合はワーニングとなり、指定されたアライメント値だけ、下位ビットが無視されます。

デフォルト: 本コマンドが指定されない場合、各セクションは4バイト境界に配置されます。

-codeコマンド

形式1: -code <**アドレス**>
形式2: -code <**セクション名**>
記述例: -code 0x0c00000
-code EXTERNAL_ROM
説明:
- リロケータブルなCODEセクションを配置する領域の開始アドレスを設定します。形式3または4で指定したファイル、およびアブソリュートオブジェクトファイルのCODEセクションは影響を受けません。
- <セクション名>は-sectionコマンドでセクション名と開始アドレスを設定した場合にのみ指定可能です。
- -codeと<アドレス/セクション名>の間には1個以上のスペースまたはタブが必要です。
- <アドレス>は4バイト境界を指定してください。それ以外の指定はワーニングとなり、指定アドレスの下位2ビットは無視されます。

デフォルト: 本コマンドが指定されない場合、CODEセクションは0x0080000から始まります。

形式3: -code <**アドレス**> {<**ファイル名**> ... <**ファイル名**> }
形式4: -code <**セクション名**> {<**ファイル名**> ... <**ファイル名**> }
記述例: -code 0x0080100 {test1.o test2.o}
-code BLOCK2 {test1.o, test2.o}
説明:
- { }内に指定したリロケータブルオブジェクトファイルのCODEセクションを、指定のアドレスからファイルの指定順に配置します。
- 複数のファイルを指定する場合、<ファイル名>の間には1個以上のスペースまたはタブが必要です。
- その他は形式1または2と同様です。

-dataコマンド

形式1: -data <**アドレス**>
形式2: -data <**セクション名**>
記述例: -data 0x0081000
-data DATA1
説明:
- リロケータブルなDATAセクションを配置する領域の開始アドレスを設定します。形式3または4で指定したファイル、およびアブソリュートオブジェクトファイルのDATAセクションは影響を受けません。
- <セクション名>は-sectionコマンドでセクション名と開始アドレスを設定した場合にのみ指定可能です。
- -dataと<アドレス/セクション名>の間には1個以上のスペースまたはタブが必要です。
- <アドレス>は4バイト境界を指定してください。それ以外の指定はワーニングとなり、指定アドレスの下位2ビットは無視されます。

デフォルト: 本コマンドが指定されない場合、DATAセクションはアドレス最上位にあるCODEセクションの後ろに配置されます。

形式3: -data <**アドレス**> {<**ファイル名**> ... <**ファイル名**> }
形式4: -data <**セクション名**> {<**ファイル名**> ... <**ファイル名**> }
記述例: -data 0x0080100 {test1.o test2.o}
-data DATA1 {test1.o test2.o}
説明:
- { }内に指定したリロケータブルオブジェクトファイルのDATAセクションを、指定のアドレスからファイルの指定順に配置します。
- 複数のファイルを指定する場合、<ファイル名>の間には1個以上のスペースまたはタブが必要です。
- その他は形式1または2と同様です。

-bssコマンド

形式1: -bss <**アドレス**>
形式2: -bss <**セクション名**>
記述例: -bss 0x0000100
-bss VARIABLES
説明:
- リロケータブルなBSSセクションを配置する領域の開始アドレスを設定します。形式3または4で指定したファイル、およびアブソリュートオブジェクトファイルのBSSセクションは影響を受けません。
- <セクション名>は-sectionコマンドでセクション名と開始アドレスを設定した場合にのみ指定可能です。
- -bssと<アドレス/セクション名>の間には1個以上のスペースまたはタブが必要です。
- <アドレス>は4バイト境界を指定してください。それ以外の指定はワーニングとなり、指定アドレスの下位2ビットは無視されます。

デフォルト: 本コマンドが指定されない場合、BSSセクションは0x0000000から始まります。

形式3: -bss <**アドレス**> {<**ファイル名**> ... <**ファイル名**> }
形式4: -bss <**セクション名**> {<**ファイル名**> ... <**ファイル名**> }
記述例: -bss 0x0000100 {test1.o test2.o}
説明:
- { }内に指定したリロケータブルオブジェクトファイルのBSSセクションを、指定のアドレスからファイルの指定順に配置します。
- 複数のファイルを指定する場合、<ファイル名>の間には1個以上のスペースまたはタブが必要です。
- その他は形式1または2と同様です。

-ucodeコマンド

形式1: -ucode <**アドレス**>
形式2: -ucode <**セクション名**>
記述例: -ucode 0x1000
-ucode CACHE

説明:
- 仮想CODEセクションの開始アドレスを設定します。-codeコマンドの形式1と2の設定が適用されるリロケータブルオブジェクトファイルのCODEセクションを、指定のアドレスから配置して実行できるようにシンボルアドレスを解決してリンクします。rowデータの位置は-codeの指定のまま変更されません。アブソリュートオブジェクトファイルのCODEセクションは影響を受けません。ROMに書き込まれたプログラムをRAMに転送して実行させる場合に指定します。
- <セクション名>は-sectionコマンドでセクション名を設定した場合に指定可能です。
- -ucodeと<アドレス/セクション名>の間には1個以上のスペースまたはタブが必要です。
- <アドレス>は4バイト境界を指定してください。それ以外の指定はワーニングとなり、指定アドレスの下位2ビットは無視されます。

デフォルト: 本コマンドが指定されない場合、仮想CODEセクションは設定されません。

形式3: -ucode <**アドレス**> {<**ファイル名**> ... <**ファイル名**> }
形式4: -ucode <**セクション名**> {<**ファイル名**> ... <**ファイル名**> }
記述例: -ucode 0x1000 {test1.o test2.o}
記述例: -ucode CACHE {test1.o test2.o}
説明:
- 共用CODEセクションの開始アドレスを設定します。{ }内に指定したすべてのリロケータブルオブジェクトファイルのCODEセクションを、同じ領域(指定のアドレスまたは指定の共用セクションの先頭アドレスから始まる共用領域)に配置して実行できるようにシンボルアドレスを解決してリンクします。指定した複数のオブジェクトコードで1つのRAM領域を共用し、時分割で転送して実行を繰り返すような場合に有効です。
- <ファイル名>の間には1個以上のスペースまたはタブが必要です。
- その他は形式1または2と同様です。

-udata**コマンド**

形式1: -udata <**アドレス**>
形式2: -udata <**セクション名**>
記述例: -udata 0x1000
-udata INITDATA
説明:
- 仮想DATAセクションの開始アドレスを設定します。-dataコマンドの形式1と2の設定が適用されるリロケータブルオブジェクトファイルのDATAセクションを、指定のアドレスから配置して実行できるようにシンボルアドレスを解決してリンクします。rowデータの位置は-dataの指定のまま変更されません。アブソリュートオブジェクトファイルのDATAセクションは影響を受けません。ROMに書き込まれたDATA(初期化が必要な変数等)をRAMに転送して使用する場合に指定します。
- <セクション名>は-sectionコマンドでセクション名を設定した場合に指定可能です。
- -ucodeと<アドレス/セクション名>の間には1個以上のスペースまたはタブが必要です。
- <アドレス>は4バイト境界を指定してください。それ以外の指定はワーニングとなり、指定アドレスの下位2ビットは無視されます。

デフォルト: 本コマンドが指定されない場合、仮想DATAセクションは設定されません。

形式3: -udata <**アドレス**> {<**ファイル名**> ... <**ファイル名**> }
形式4: -udata <**セクション名**> {<**ファイル名**> ... <**ファイル名**> }
記述例: -udata 0x1000 {test1.o test2.o}
-udata INITDATA {test1.o test2.o}
説明:
- 共用DATAセクションの開始アドレスを設定します。{ }内に指定したすべてのリロケータブルオブジェクトファイルのDATAセクションを、同じ領域(指定のアドレスまたは指定の共用セクションの先頭アドレスから始まる共用領域)に配置して実行できるようにシンボルアドレスを解決してリンクします。指定した複数のオブジェクトデータで1つのRAM領域を共用し、時分割で転送して使用する場合に有効です。
- 複数のファイルを指定する場合、<ファイル名>の間には1個以上のスペースまたはタブが必要です。
- その他は形式1または2と同様です。

-ubssコマンド

形式1: -ubss <**アドレス**> {<**ファイル名**> ... <**ファイル名**> }
形式2: -ubss <**セクション名**> {<**ファイル名**> ... <**ファイル名**> }
記述例: -ubss 0x1000 {test1.o test2.o}
-ubss TMP {test1.o test2.o}
説明:
- 共用BSSセクションの開始アドレスを設定します。{ }内に指定したすべてのリロケータブルオブジェクトファイルのBSSセクションを、同じ領域(指定のアドレスまたは指定の共用セクションの先頭アドレスから始まる共用領域)に配置して使用できるようにシンボルアドレスを解決してリンクします。アブソリュートオブジェクトファイルのBSSセクションは影響を受けません。指定した複数のオブジェクトデータで1つのRAM領域を共用し、時分割で転送して使い分ける場合に有効です。
- <セクション名>は-sectionコマンドでセクション名を設定した場合に指定可能です。
- -ucodeと<アドレス/セクション名>の間には1個以上のスペースまたはタブが必要です。
- <ファイル名>の間には1個以上のスペースまたはタブが必要です。
- <アドレス>は4バイト境界を指定してください。それ以外の指定はワーニングとなり、指定アドレスの下位2ビットは無視されます。

デフォルト: 本コマンドが指定されない場合、共用BSSセクションは設定されません。

-objsymコマンド

形式: -objsym
記述例: -objsym
説明:
- 各入力ファイルのセクションごとに、配置後の開始アドレス、終了アドレス、サイズを示す3種類のシンボルを作成します("12.5.5 セクションシンボル"参照)。仮想・共用セクションへの転送ルーチンを作成する場合などにソースファイルで使用することができます。

デフォルト: 本コマンドが指定されない場合、シンボルは作成されません。

-sectionコマンド

形式1: -section <**セクション名**>
形式2: -section <**セクション名**> = <**アドレス**>
記述例: -section TMP
-section CACHE = 0x1000
説明:
- セクション名を定義します。
- 形式1は仮想・共用セクション用で、デフォルトのBSS領域(-bssコマンドで指定)の最終アドレスに続くアドレスが仮想・共用セクションの開始アドレスとなります。形式2は任意のセクションの指定に使用可能なセクション名とその開始アドレスを定義します。
- -sectionと<セクション名>の間には1個以上のスペースまたはタブが必要です。
- セクション指定コマンドの設定により領域の使用サイズが決まり、開始アドレス、配置後(最大使用時)の終了アドレス、サイズを示す3種類のシンボルが作成されます("12.5.5 セクションシンボル"参照)。仮想・共用セクションへの転送ルーチンを作成する場合など、このシンボルをソースファイルで使用することができます。

-ldコマンド

形式: -ld
記述例: -ld
説明:
- 通常のアブソリュートオブジェクトファイルの代わりに、ローダld33用のsrf33ファイルを作成します。ローダld33については、"utility¥ld33¥"ディレクトリ内の"readmeja.txt"(日本語)または"readme.txt"(英語)を参照してください。

デフォルト: 本コマンドが指定されない場合、通常のアブソリュートオブジェクトファイルを出力します。
注意: このファイルを作成する場合は、-ucode、-udata、-ubssコマンドによるUセクションの指定は行えません。また、2パスmakeによる最適化も行えません。

-inclinkコマンド

形式: -inclink

記述例: -inclink

説明: ・アブソリュートオブジェクトファイル(.srf)のリンクを許可します。
このコマンドが指定されると、.srfファイルもアセンブラで生成されたアブソリュートオブジェクトファイルと同様にリンクすることができます。ただし-ldコマンドにより作成したローダ用アブソリュートオブジェクトファイルはリンクできません。

デフォルト: 本コマンドが指定されない場合、.srfファイルはリンクできません。

12.5.3 セクションの配置

●デフォルト設定によるリロケータブルファイルの配置

各リロケータブルオブジェクトファイルは、それぞれ1つずつのCODEセクション、DATAセクション、BSSセクションを持ちます。
複数のリロケータブルオブジェクトファイルをリンクすると、特に指定したファイルを除き、同じ種類のセクションは1つにまとめられます。各セクション内の並びは、コマンドファイル内(またはコマンドライン)の入力ファイルの記述順となります。
各セクションの先頭アドレス(リンク後の絶対アドレス)はリンカコマンドで指定できます。指定のない場合は、次のようにアドレスが決定します。

CODE**セクション**: 0x0080000番地(エリア3、内蔵ROMの先頭)から配置されます。
DATA**セクション**: 入力ファイルのすべてのCODEセクションを配置後、最も上位アドレスに配置されたCODEセクションの直後に配置されます。
BSS**セクション**: 0x0000000番地(エリア0、内蔵RAMの先頭)から配置されます。

たとえば、2つのリロケータブルオブジェクトファイルsample1.oとsample2.oをセクションアドレスを指定せずにリンクすると、各セクションは次のように配置されます。
各オブジェクトの各セクションは4バイト境界アドレスからスタートします。

例: lk33 sample1.o sample2.o

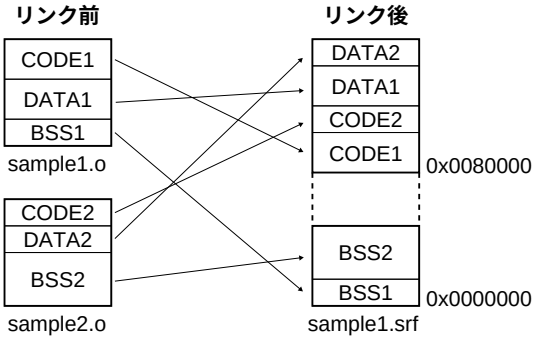


図12.5.3.1 **デフォルト設定でのリンク例**

lk33が生成するアブソリュートオブジェクトファイルには、CODEセクション、DATAセクション、BSSセクションの順に出力されます。
出力されるオブジェクトファイルの内容についてはAppendixの"srf33ファイルの構造"を参照してください。

●リロケータブルなセクションの開始アドレスを指定するには

セクションを前記のデフォルト以外のアドレスから配置するには、-code、-data、-bssコマンドによって開始アドレスを指定します。
コマンドの書式は次のとおりです。

-code **<アドレス>** CODEセクションの設定
-data **<アドレス>** DATAセクションの設定
-bss **<アドレス>** BSSセクションの設定

たとえば、図12.5.3.1のCODEセクションの開始アドレスを0x0c00000番地、BSSセクションの開始アドレスを0x0010000番地に変更するには次のようなコマンドファイルを入力してリンクします。
例: コマンドファイル

```
-code    0x0c00000
-bss     0x0010000
sample1.o
sample2.o
```

●特定ファイルのセクションを特定アドレスから配置するには

-code {}、-data {}、-bss {}コマンドでリロケータブルオブジェクトファイルを指定し、そのファイルのセクションのみを指定アドレスから配置することができます。複数のファイルが指定可能で、その場合は開始アドレスからファイルの指定順に配置されます。
コマンドの書式は次のとおりです。

-code **<アドレス>** {**<ファイル名>** ...} CODEセクションの設定
-data **<アドレス>** {**<ファイル名>** ...} DATAセクションの設定
-bss **<アドレス>** {**<ファイル名>** ...} BSSセクションの設定

例: コマンドファイル

```
-code    0x0080000
-bss     0x0000000
-code    0x0c00000 {sample2.o sample3.o}
sample1.o
sample2.o
sample3.o
```

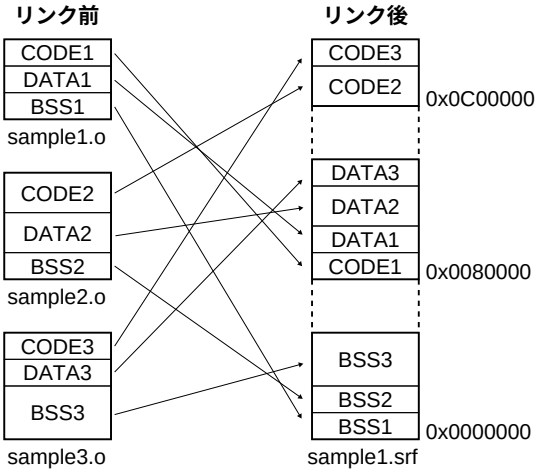


図12.5.3.2 **特定ファイルの配置例**

-code {}、-data {}、-bss {}コマンドで指定した以外のリロケータブルオブジェクトファイルは、デフォルトあるいは-code <アドレス/セクション名>、-data <アドレス/セクション名>、-bss <アドレス/セクション名>の設定が適用されます。

●セクション名による開始アドレスの指定

-sectionコマンドでセクション名と開始アドレスを定義することができます。
コマンドの形式は次のとおりです。

-section **<セクション名>** = **<アドレス>**

-sectionコマンドで定義した<セクション名>は、前述の-code、-data、-bss、-code {}、-data {}、-bss {}コマンドの<アドレス>の変わりに使用することができます。ただし、使用する前に-sectionコマンドによる定義が必要です。

例:
```
-section   CODE1 = 0x0c00100
-section   DATA1 = 0x0c01000
-code      CODE1 {test1.o}     ; test1.oのCODEセクションをセクションCODE1に配置
-data      DATA1 {test1.o}     ; test1.oのDATAセクションをセクションDATA1に配置
```

それぞれ、-sectionコマンドで指定したアドレスがセクション開始アドレスとなります。-sectionコマンドの<アドレス>は省略できません。
セクション名を定義するとlk33はその開始アドレス、終了アドレスおよびサイズを示す3種類のセクションシンボルを作成します("12.5.5 セクションシンボル"参照)。ソース内でグローバルシンボルとして使用できます。

●アブソリュートファイルの配置

アブソリュートオブジェクトファイルは、リンク前に各セクションの絶対アドレスが確定しているため、その番地に優先的に配置されます(lk33はリロケータブルオブジェクトファイルの前にアブソリュートオブジェクトファイルの配置を行います)。-code、-data、-bss等、アドレスを指定するコマンドの影響は受けません。

例: コマンドファイル

```
-code      0x0081000
sample1.o            ; アブソリュートオブジェクトファイル
sample2.o            ; リロケータブルオブジェクトファイル
```

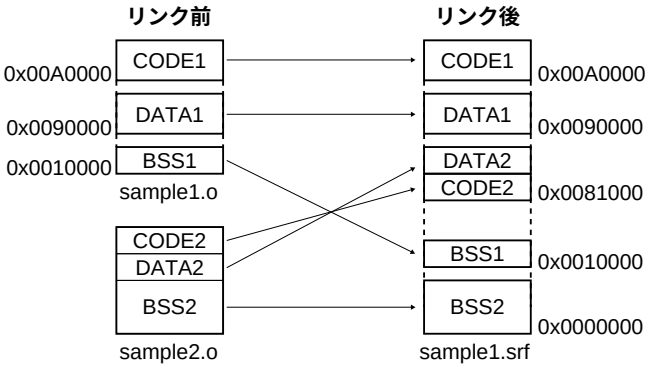


図12.5.3.3 **アブソリュートオブジェクトファイルの配置例**

●アドレス指定の優先順位

アドレス指定は次の優先順位で処理されます。

1 アブソリュートオブジェクトファイル内の設定
2 -code {}、-data {}、-bss {}コマンドによる設定
3 -code、-data、-bssコマンドによる設定
4 デフォルト設定

アブソリュートオブジェクトファイル内のセクションは、ソースで指定されているアドレスに最優先で配置されます。
リロケータブルオブジェクトファイルのセクションは、-code {}、-data {}、-bss {}コマンドによって指定されているものが先に配置され、残りのファイルのセクションは-code、-data、-bssコマンドで設定したアドレスまたはデフォルトのアドレスから配置されます。
同じ優先順位を持つ複数のファイルについては、コマンドファイル(またはコマンドライン)内の記述順にアドレスの上位方向へ配置されます。

●セクションのアライメント

各セクションはデフォルトで4バイト境界に配置されます。
-alignコマンドでアライメント境界を2^nに設定することもできます。
-code、-data、-bss、-ucode、-udata、-ubssコマンドで4バイト境界以外のアドレス、あるいは-alignコマンドで設定した境界アドレス以外のセクション配置アドレスを指定した場合はワーニングとなり、アドレスの下位ビットを0として最も指定アドレスに近いアライメント境界に配置されます。

12.5.4 仮想・共用(U)セクション

仮想・共用セクション(Uセクション)はリンカで作成される実データを持たないセクションで、別の領域にある実データをコピーして実際にアクセスあるいは実行させるための領域のことです。通常、UセクションはRAMに対応します。
たとえば、C言語で初期値を持つ変数を使用する場合や、外部メモリのプログラムをRAMにコピーして高速動作させる場合など、実データの置かれたアドレスでシンボル情報を作成するとRAMで動作させることができません。この場合は、実際に実行する領域をUセクションとして設定し、プログラムやデータをその領域に論理的に配置してリンクを行います。Uセクションに配置したモジュールのシンボル情報は、Uセクション内のアドレスとして生成されます。

注: **説明上、Uセクションへの配置という表現を使用しています。これはUセクションから実行アドレスを得るための論理的な配置を意味します。Uセクションへの論理的な配置は-ucode、-udata等のコマンドで行いますが、実データの置かれるアドレスは、ファイル名を指定しない-codeや -dataコマンドあるいはデフォルト設定によって決まります。**

Uセクションに配置したプログラムやデータは、実際に使用する前にその領域にコピーしておきます。
なお、実行時の再配置が必要となるため、アブソリュートオブジェクトファイルをUセクションに配置することはできません。
Uセクションには仮想セクションと共用セクションの2種類があります。

●仮想セクション

CODEセクション、DATAセクションの実データを配置する-code、-dataコマンドに相当するUセクション用のコマンドが-ucode、-udataです。
それぞれ次の書式で、仮想CODEセクションと仮想DATAセクションの開始アドレスが指定できます。

-ucode **<アドレス>** 仮想CODEセクションの設定
-udata **<アドレス>** 仮想DATAセクションの設定

これらのコマンドを指定すると、それぞれデフォルトのCODEセクション(-code <アドレス>で指定)、DATAセクション(-data <アドレス>で指定)が、指定のアドレスから始まる仮想セクションで実行されるものとしてシンボルの解明が行われます。

例: コマンドファイル
```
-code     0x0c00000
-udata    0x0001000
sample1.o
sample2.o
```

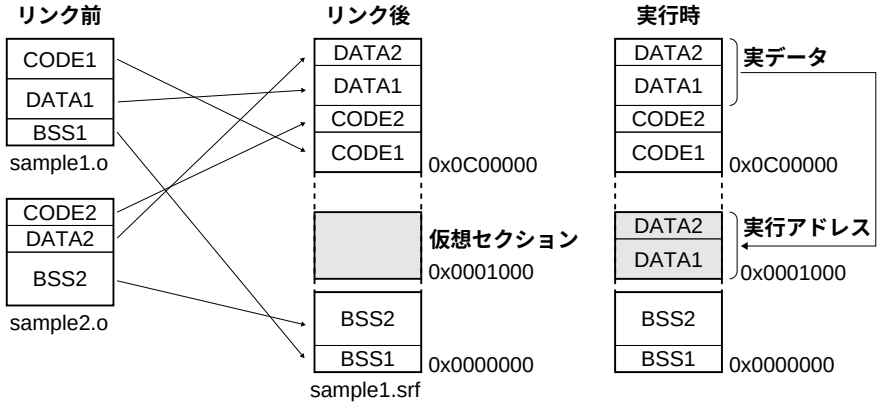


図12.5.4.1 **仮想セクションへの配置例**

●共用セクション

-ucode {}、-udata {}、-ubss {}コマンドでリロケータブルオブジェクトファイルを指定し、そのファイルのセクションのみを共用セクションに並列配置することができます。複数のファイルが同一の開始アドレスから配置されます。

-ucode <**アドレス**> {<**ファイル名**> ...} 共用CODEセクションの設定
-udata <**アドレス**> {<**ファイル名**> ...} 共用DATAセクションの設定
-ubss <**アドレス**> {<**ファイル名**> ...} 共用BSSセクションの設定

例: コマンドファイル
-code 0x0c00000
-ucode 0x000a000 {sample1.o sample2.o}
sample1.o
sample2.o

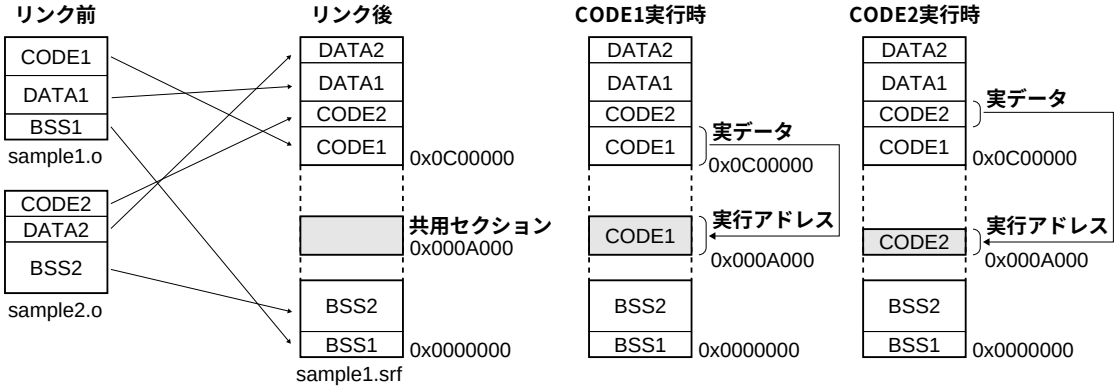


図12.5.4.2 **共用セクションへの配置例**

●名前付きUセクション

Uセクションには-sectionコマンドで名前を定義することができます。
コマンドの形式は次のとおりです。

形式1) -section <**セクション名**>
形式2) -section <**セクション名**> = <**アドレス**>

このコマンドにより指定の名前のUセクションが1つ作成されます。複数のUセクションを定義することもできます。その場合は、-sectionコマンドを必要数記述してください。
ここで定義した名前は、前述の-ucode、-udata、-ucode {}、-udata {}、-ubss {}コマンドの<アドレス>の変わりに使用することができます。ただし、使用する前に-sectionコマンドによる定義が必要です。

例: -section CACHE = 0x300
-section INITDATA = 0x100
-ucode CACHE {test1.o} ; test1.oのCODEセクションをUセクションCACHEに配置
-udata INITDATA {test1.o} ; test1.oのDATAセクションをUセクションINITDATAに配置

定義したUセクションの開始アドレスは、次のように設定されます。

形式1) デフォルトBSSセクション終了直後のアドレス。入力オブジェクトのすべてのBSSセクションを配置後に決定します。
形式2) 指定したアドレス

セクション名を定義するとlk33はその開始アドレス、終了アドレスおよびサイズを示す3種類のセクションシンボルを作成します(次節参照)。ソース内でグローバルシンボルとして使用できますので、実データのコピーに利用することができます。

●Uセクションの配置とサイズ

Uセクションはコマンドで設定されたアドレスから始まります。仮想セクションのサイズは配置したセクションの合計サイズです。共用セクションには、共用する複数のセクションの中で、最も大きなセクションのサイズが適用されます。Uセクションを定義する場合、Uセクション同士あるいは通常のセクションとのアドレスの重複も許されますので、注意して使用してください。

●セクションのアライメント

各セクションはデフォルトで4バイト境界に配置されます。
-alignコマンドでアライメント境界を2^nに設定することもできます。
-code、-data、-bss、-ucode、-udata、-ubssコマンドで4バイト境界以外のアドレス、あるいは-alignコマンドで設定した境界アドレス以外のセクション配置アドレスを指定した場合はワーニングとなり、アドレスの下位ビットを0として最も指定アドレスに近いアライメント境界に配置されます。

●Uセクションの使用方法

絶対アドレスを指定せずに、実際のプログラムや変数をUセクションに連続的に配置するには、以下のような順序で指定するのが効果的です。

↑RAM: 低アドレス側
(1) 初期値を持たない変数(BSS)
(2) 初期値を持つ変数(DATA)
(3) プログラムコード1(ROM上のCODEのコピー)
(4) プログラムコード2(ROM上のCODEのコピー)
:
↓RAM: 高アドレス側

上記のように配置するには、コマンドファイルで次のように指定します。

コマンドファイル例

```
-code    0x0c00000

-section    INITDATA
-section    CACHE1
-section    CACHE2

-udata    INITDATA
-ucode    CACHE1 {main.o}
-ucode    CACHE2 {sub.o}

main.o
sub.o
```

リンク後のリンクマップファイルを見ると、低アドレス側から高アドレス側に向かい(1)～(4)の順に配置されているのが分かります。

リンクマップファイル例

```
Code Section mapping
Address   Vaddress    Size       File              ID  Attr
00c00000  0000001c   00000040   main.o  ←(3)      0   REL
00c00040  0000005c   00000020   sub.o   ←(4)      0   REL

Data Section mapping
Address   Vaddress    Size       File              ID  Attr
00c00060  00000014   00000008   main.o  ←(2)      1   REL
00c00068  0000001c   00000000   sub.o   ←(2)      1   REL

Bss Section mapping
Address   Vaddress    Size       File              ID  Attr
00000000  --------   00000004   main.o  ←(1)      2   REL
00000004  --------   00000010   sub.o   ←(1)      2   REL
```

12.5.5 セクションシンボル

lk33は配置後のセクションのアドレス、サイズを示すセクションシンボル(グローバルシンボル)を生成することができます。それらのシンボルはソース中で使用することができます。

●デフォルトDATAセクションシンボル

-dataコマンドの指定がない場合、デフォルト(CODEセクションに続いて配置)のデータセクションには、自動的に次のセクションシンボルが生成されます。

__START_DEFAULT_DATA	デフォルトDATAセクションの開始アドレスが定義されます。
__END_DEFAULT_DATA	デフォルトDATAセクションの終了アドレスが定義されます。
__SIZEOF_DEFAULT_DATA	デフォルトDATAセクションのサイズ(バイト数)が定義されます。

注: __START_DEFAULT.DATA**、**__END_DEFAULT.DATA**、**__SIZEOF_DEFAULT.DATA**シンボルも互換性のために生成されます。ただし、これらのシンボル名は**"."**を含むため、C**ソース内で使えません。アセンブリソース内でのみ使用してください。**

●-objsymコマンドによるセクションシンボルの作成

コマンドファイルに-objsymコマンドを記述しておくと、lk33は各入力ファイルのセクションごとに、再配置後のセクション情報を示す以下のようなシンボルを生成します。

__START_<**ファイル名**>_<**セクション**>	セクションの開始アドレスが定義されます。
__END_<**ファイル名**>_<**セクション**>	セクションの終了アドレスの次のアドレスが定義されます。
__SIZEOF_<**ファイル名**>_<**セクション**>	セクションサイズ(バイト数)が定義されます。

例: 入力ファイル名が"test.o"、CODEセクションの開始アドレスが0x0c00000、サイズが0x100の場合

シンボル名	シンボルの値
__START_test_code	0x0c00000
__END_test_code	0x0c00100
__SIZEOF_test_code	0x100

<ファイル名>のパスと拡張子は、コマンドファイル内の記述にかかわらずカットされます。
シンボルに定義される開始アドレスとサイズは、リンクマップファイルの出力内容(実データアドレス)と同じで、終了アドレスは開始アドレス+サイズとなります。

●-sectionコマンドによるセクション名

-sectionコマンドでセクション名を定義すると、-objsymコマンドの有無にかかわらず、そのセクションの情報を示す以下のシンボルが作成されます。

__START_<**セクション名**>	セクションの開始アドレスが定義されます。
__END_<**セクション名**>	セクションの終了アドレスが定義されます。
__SIZEOF_<**セクション名**>	セクションサイズ(バイト数)が定義されます。

<セクション名>は-sectionコマンドで指定した名称です。
シンボルに定義される開始アドレスは-sectionコマンドで指定したアドレスまたはデフォルトBSSセクション終了直後のアドレスです。
サイズは、仮想セクション(-ucode、-udata)の場合は配置されるセクションの合計サイズ、共用セクション(-ucode { }、-udata { }、-ubss { })の場合は配置されるセクションの中で最大のものと同じサイズです。終了アドレスは開始アドレス+サイズとなります。

●アセンブリソースでのセクションシンボル使用例

例: test.oのCODEセクションをUセクション"CACHE1"に転送する場合

ソースファイル内の転送ルーチン

```
        xld.w   %r12, __START_CACHE1
        xld.w   %r13, __START_test1_code
        xld.w   %r14, __SIZEOF_test1_code

HCOPY_LOOP:
        ld.uh   %r4, [%r13]+        ; 命令コードの転送
        ld.h    [%r12]+, %r4
        sub     %r14, 2
        jrgt    HCOPY_LOOP
```

リンカコマンドによるUセクションの設定

```
-objsym
-section CACHE1 = 0x300
-ucode CACHE1 {test.o}
test.o
```

注: ソース内では、使用するセクションシンボルと同じ名称のシンボルは定義しないでください。

12.5.6　ライブラリのリンク

ライブラリのリンクは他の入力オブジェクトファイルの配置を終了後に行われます。

●ライブラリモジュールの検索

lk33は入力オブジェクト内に未解決の外部参照シンボルがある場合にのみ、コマンドファイル内に記述された順にライブラリファイルを検索し、最初に見つかったそのシンボルを持つモジュールのみを読み込んでリンクします。外部参照が解決すると、それ以降のモジュール、ライブラリファイルは検索しません。したがって、指定のライブラリファイルに検索対象のシンボルが複数定義されていても、最初に見つかったモジュール以外は無視されます。
ライブラリがカレントディレクトリにない場合は、-lコマンドで指定されたディレクトリを検索します。

ライブラリファイル間の外部参照は、現在処理中のライブラリ内のファイルあるいはそれ以降に検索するライブラリファイルでのみ解決できます。すでに検索したライブラリファイルで定義されていても解決されません。ライブラリを指定する順序に注意してください。

●ライブラリモジュールの配置(デフォルト)

リンクされるライブラリモジュールの各セクションは、同種のデフォルトセクション(指定のないリロケータブルセクションがすべて配置されるセクション)に配置されます。
例: コマンドファイル

```
-l c:¥cc33¥lib
-code    0x0c00000
sample1.o          ; リロケータブルオブジェクトファイル
sample2.o          ; リロケータブルオブジェクトファイル
sample.lib         ; ライブラリファイル(モジュールnを使用)
```

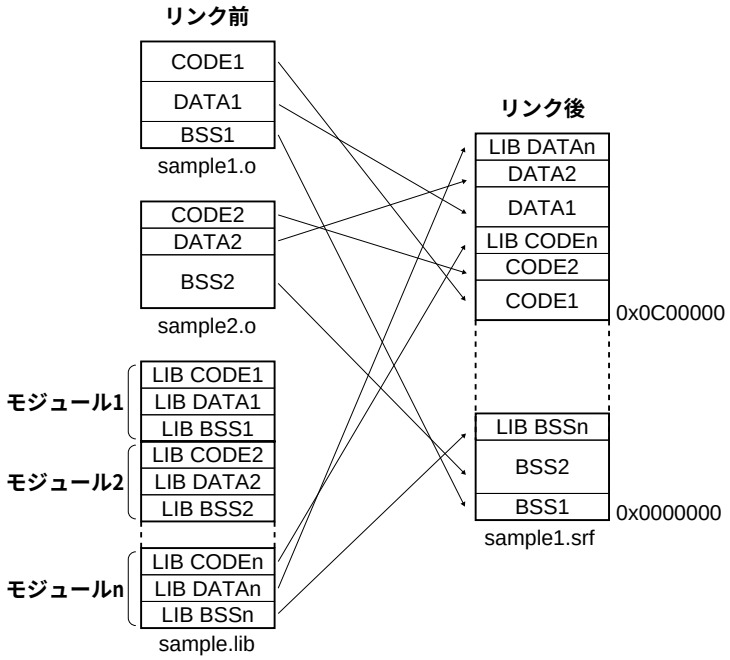


図12.5.6.1　**ライブラリモジュールの配置例**

●ライブラリモジュールの指定アドレスへの配置

-code {}、-data {}、-bss {}コマンドでライブラリモジュールを指定アドレスに配置することができます。ファイル名は"ライブラリ名:モジュール名"の形式で指定してください。この名称はリンクマップファイルのFile欄に表示される名前と同一になるようにしてください。

string.lib内のstrcpy.oを0xca0000番地から配置する例を以下に示します。

コマンドファイル例:

```
-l c:¥cc33¥lib
-code 0xc00000
-code 0xca0000 {c:¥cc33¥lib¥string.lib:strcpy.o}
test.o
string.lib
```

リンクマップファイル例:

Code Section mapping

Address	Vaddress	Size	File	ID	Attr
00c00000	--------	00000010	test.o	0	REL
00ca0000	--------	0000000c	c:¥cc33¥lib¥string.lib:strcpy.o	0	REL

注: ・ この機能を使い問題が発生した場合は、ライブラリモジュールをリロケータブルオブジェクトファイルに復元してリンクを行ってください。
・ ライブラリファイル自身を指定アドレスに配置することはできません。以下の使い方は誤った例です。

誤ったコマンドファイル例:

```
-l c:¥cc33¥lib
-code 0xc00000
-code 0xca0000 {c:¥cc33¥lib¥string.lib}        ←ライブラリ直接指定はできません。
test.o
string.lib
```

●ライブラリモジュールのUセクションへの配置

-ucode {}、-udata {}、-ubss {}コマンドではライブラリファイルを指定することができません。
使用したい場合は、そのライブラリモジュールをリロケータブルオブジェクトファイルに復元してリンクを行ってください。

12.6 lk33 Ver. 3.0のリンク処理

12.6.1 リンカコマンドファイル

lk33は起動時のコマンドライン指定を簡略化するため、必要な指定を記述したリンカコマンドファイルを入力してリンク処理を行うことができます。入力するリンカコマンドファイルは、lk33起動時に-cオプションで指定します。

リンカコマンドファイル例

```
-v3                                          ; lk33 Ver. 3コマンドを使用許可
-addr 0x0C00000 {DEFAULT_CODE}               ; デフォルトCODEブロック開始アドレス
-addr 0x0C01000 {DEFAULT_DATA}               ; デフォルトDATAブロック開始アドレス
-addr 0x0000000 {DEFAULT_BSS}                ; デフォルトBSSブロック開始アドレス

-codeblock MAIN_OBJ {test2.o, test3.o}       ; CODEセクションのブロック化
-addr 0x0c00100 {MAIN_OBJ}                   ; ブロック開始アドレスの指定

-defsym BOOT = 0x0C00000                     ; グローバルシンボルの設定

;Library path
-l C:\CC33\lib                               ; ライブラリパスの指定

;Executable file
-o test.srf                                  ; 出力ファイル名の指定

;Object files
test1.o                                      ; 入力ファイルの指定
test2.o
test3.o

;Library files
io.lib                                       ; ライブラリファイルの指定
lib.lib
math.lib
string.lib
ctype.lib
fp.lib
idiv.lib
```

リンカコマンドファイルは以下の規則に沿って作成してください。

●ファイルの形式

リンカコマンドファイルは、例に示したとおり一般的なテキスト形式です。
汎用エディタで作成してください。または、wb33の[Make file editor]によりテンプレートが作成されますので、そのファイルを修正して使用してください。
ファイル名の拡張子は".cm"としてください。

注: wb33**は**lk33 Ver. 2.1**コマンドを使用してリンカコマンドファイルを作成します。**

●コマンドの記述

コマンドはすべてハイフン(-)で始まります。個々のコマンドは1個以上のスペース、タブ、あるいは改行で区切る必要があります。視認性の面から、個々のコマンドは行を変えて記述することを推奨します。
1行には2048文字まで記述可能です。

注: lk33 Ver. 3.0**コマンドを使用する場合は、必ずファイルの先頭に**-v3**コマンドを記述してください。**

注: ・ コマンドはすべて小文字で記述してください。大文字は受け付けません。

・ アドレス等を指定する数値は16進数形式(0x####)で記述してください。10進数、2進数表記は受け付けません。

・ シンボル名、ブロック名、ファイル名には以下の文字のみ受け付けます。
1文字目: a〜z, A〜Z
2文字目以降: a〜z, A〜Z, 0〜9, _
ファイル名(パス)には"."と"¥"と":"も使用可能です。

●入力ファイル、ライブラリファイルの指定

入力するオブジェクトファイル名、ライブラリファイル名は、リンクコマンドファイルの最後に記述してください。ライブラリファイルはオブジェクトファイルより後に記述します。
リンクによる配置は、特に指定した場合を除き、記述した順序で行われます。
ファイル名は拡張子(.o、.lib、.srf)まで記述してください。
オブジェクトファイルを記述せずに、ライブラリファイルのみを指定することはできません。

●コメント

リンカコマンドファイルにはコメントを記述することができます。
ソースファイルと同様に、セミコロン(;)からその行の終わりまでの文字列がコメントとみなされます。

●空白行

空白文字と改行のみの空白行は無視されます。セミコロンでコメントにする必要はありません。

12.6.2 リンカコマンド一覧(Ver. 3.0)

リンカlk33 Ver. 3.0に対応するリンカコマンドは次に示す24種類です(コマンドラインで指定可能な起動時オプションも含む)。

表12.6.2.1 リンカコマンド一覧(Ver. 3.0対応)

コマンド	機能
-c	コマンドファイルの指定 *1, *2
-m	リンクマップファイルの出力 *1
-s	シンボルファイルの出力 *1
-cref	クロスリファレンスファイルの出力 *1
-g	デバッグ情報の出力 *1
-e	エラーファイルの出力 *1
-w	ワーニングレベルの設定
-l	ライブラリパスの指定
-o	出力ファイル名の設定
-defsym	グローバルシンボルの設定
-d	重複グローバルBBS領域の削除
-objsym	セクションシンボルの作成
-ld	ローダ用srf33ファイルの出力
-inclink	.srfファイルのリンク許可
-align	アライメント指定
-v3	lk33 Ver. 3コマンドイネーブル
-codeblock	CODEセクションのブロック化 *3
-datablock	DATAセクションのブロック化 *3
-bssblock	BSSセクションのブロック化 *3
-ucodeblock	CODEセクションの並列(ユニオン)ブロック化 *3
-udatablock	DATAセクションの並列(ユニオン)ブロック化 *3
-ubssblock	BSSセクションの並列(ユニオン)ブロック化 *3
-addr	ブロックの配置アドレス指定 *3
-defaddr	アドレス定義 *3

*1: 起動時オプション
*2: コマンドファイル内では使用できません。
*3: Ver. 3.0対応コマンド(-v3指定時以外は使用できません。)

以下、各リンカコマンドについて説明します。(起動時オプションについては12.3.2節参照)
実際の使用例とリンク結果については次節を参照してください。

-wコマンド

形式: -w
記述例: -w
説明:
- -wコマンドが指定された場合、BSSセクション内のグローバルラベルの重複に対してはワーニングを出力しません。

デフォルト: 本コマンドが指定されない場合、すべてのワーニングメッセージを出力します。

-lコマンド

形式: -l <ライブラリ検索パス>
記述例: -l c:¥cc33¥lib
説明:
- ライブラリを検索するディレクトリを指定します。
- -lと<ライブラリ検索パス>の間には1個以上のスペースまたはタブが必要です。
- ライブラリ検索パスは最大4つまで指定可能です。複数のディレクトリを指定する場合は、それぞれについて、-lオプションによる指定を行ってください。

デフォルト: 本コマンドが指定されない場合、カレントディレクトリの検索のみとなります。
個々のライブラリファイル名の記述にパス名を含めることも可能です。

-oコマンド

形式: -o <出力ファイル名>.srf
記述例: -o test.srf
説明:
- 出力ファイル名を指定します。
- -oと<出力ファイル名>の間には1個以上のスペースまたはタブが必要です。

デフォルト: 本コマンドが指定されない場合、コマンドファイルに記載された入力オブジェクトファイル名の中で、最初に現れたファイル名を使用して出力ファイルが生成されます。

-defsymコマンド

形式: -defsym <シンボル名> = <値>
記述例: -defsym BOOT = 0x0080000
説明:
- グローバルシンボルの値を、このコマンドで定義します。
- -defsymと<シンボル名>の間には1個以上のスペースまたはタブが必要です。
- -defsymコマンドで定義可能なシンボル数は最大256個です。

デフォルト: 本コマンドが指定されない場合、グローバルシンボルは何も設定されません。

-dコマンド

形式: -d
記述例: -d
説明:
- BSSセクションに同じグローバルシンボル名を持つ複数の領域が設定されている場合、1つの領域を除いて他の領域を削除します。
- 有効となるのは、領域サイズが最も大きく、入力オブジェクトファイル名の指定の中で最も早く現れた領域です。

デフォルト: 本コマンドが指定されない場合、無効となるシンボルの領域は削除されません。
注意: 同じ名称のグローバルシンボルが定義されている場合はワーニングとなります。
-dコマンドはBSSセクション以外のセクションに対しては無効です。

-objsymコマンド

形式: -objsym
記述例: -objsym
説明:
- 各入力ファイルのセクションごとに、配置後の開始アドレス、終了アドレス、サイズを示す3種類のシンボルを作成します("12.5.5 セクションシンボル"参照)。仮想・共用ブロックへの転送ルーチンを作成する場合などにソースファイルで使用することができます。

デフォルト: 本コマンドが指定されない場合、シンボルは作成されません。

-ldコマンド

形式: -ld
記述例: -ld
説明:
- 通常のアブソリュートオブジェクトファイルの代わりに、ローダld33用のsrf33ファイルを作成します。ローダld33については、"utility¥ld33¥"ディレクトリ内の"readmeja.txt"(日本語)または"readme.txt"(英語)を参照してください。

デフォルト: 本コマンドが指定されない場合、通常のアブソリュートオブジェクトファイルを出力します。
注意: このファイルを作成する場合は、-ucodeblock、-udatablock、-ubssblockコマンドによる共用ブロックの指定は行えません。また、2パスmakeによる最適化も行えません。

-inclinkコマンド

形式: -inclink
記述例: -inclink
説明:
- アブソリュートオブジェクトファイル(.srf)のリンクを許可します。
  このコマンドが指定されると、.srfファイルもアセンブラで生成されたアブソリュートオブジェクトファイルと同様にリンクすることができます。ただし-ldコマンドにより作成したローダ用アブソリュートオブジェクトファイルはリンクできません。

デフォルト: 本コマンドが指定されない場合、.srfファイルはリンクできません。

-alignコマンド

形式: -align <アライメント値>

記述例: -align 0x2

説明:
- -addrコマンドで設定するブロック開始アドレスのアライメントを定義します。
- アライメント値にnを指定すると、境界アドレスは2^nバイトに設定されます。
  例）-align 0x02 ...4バイト境界
  -align 0x03 ...8バイト境界
- -addrコマンドで、ブロック開始アドレスを設定された境界アドレス以外に設定した場合はワーニングとなり、指定されたアライメント値だけ、下位ビットが無視されます。

デフォルト: 本コマンドが指定されない場合、各ブロックは4バイト境界に配置されます。

-v3コマンド

形式: -v3

記述例: -v3

説明:
- lk33 Ver. 3.0以降のコマンドを使用する場合に指定します。
- Ver. 3.0モードでは以下のコマンドが有効になります。
  -codeblock, -datablock, -bssblock, -ucodeblock, -udatablock, -ubssblock, -addr, -defaddr
- 下記のコマンドは無効となり、指定時はエラーになります。
  -code, -data, -bss, -ucode, -udata, -ubss, -section
- 下記のコマンドは従来のリンカコマンドと共通です。
  -c, -m, -s, -cref, -g, -e, -w, -l, -o, -defsym, -d, -objsym, -ld, -inclink

デフォルト: 本コマンドが指定されない場合、Ver. 2.1対応のコマンドファイルとして処理されます。
上記のVer. 3.0対応コマンドは無効となり、指定時はエラーになります。

注意: -v3コマンドは、必ずコマンドファイルの1行目に指定してください。
1行目に記述されていない場合はエラーとなります。

-codeblockコマンド

形式: -codeblock <ブロック名> {<ファイル名>...<ファイル名>}

記述例: -codeblock MAIN_OBJ {main.o test.o}

説明:
- { }内に指定したリロケータブルオブジェクトファイルに含まれるCODEセクションをひとつにまとめて、新しいブロックを作成します。
- ブロックには、指定したファイル順にCODEセクションが連続して配置されます。
- -codeblockと<ブロック名>の間には1個以上のスペースまたはタブが必要です。
- 複数のファイルを指定する場合、<ファイル名>の間には1個以上のスペース、タブまたは改行が必要です。
- { }内は複数行に分けて記述できます。
  例) -codeblock MAIN_OBJ
  {
  boot.o
  main.o
  sub.o
  }
- ブロックは、最大256個定義可能です。
- 指定可能なブロック名の長さは最大130文字です。大文字と小文字は区別されます。
- 指定可能なオブジェクトファイル名の長さは、パスを含めて最大256文字です。
- { }内で指定可能なファイル数は、最大4000個です。

デフォルト: 本コマンドで指定されないオブジェクトファイルのCODEセクションは、すべてDEFAULT_CODEという名称のブロックに入れられます。

注意:
- 同じブロック名で重複定義した場合はワーニングとなります。
  この場合は最初の定義が有効になります。
  例) -codeblock OBJ1 {test1.o test2.o} ...有効
  -codeblock OBJ1 {test3.o test4.o} ...無効
- ブロック名の頭に'@'または'#'は使用できません。
- { }内で同じオブジェクトファイル名(パスが異なる場合を除く)を指定することはできません。

・{ }内には、必ず1個以上のオブジェクトファイルを指定する必要があります。
・{ }内で指定されたファイル名がリンクファイル(パスも含む)と一致しない場合はワーニングとなり、そのファイルは指定されなかったものとして処理されます。
結果的に{ }内の有効なファイル数が0になった場合はエラーとなります。
ただし、ライブラリのオブジェクトを指定している場合("xxx.lib:xxx.o"指定時)は、エラーになりません。
・ブロック名定義は、-addrコマンドで参照されなくてもエラーになりません。ただし、-addrでアドレスが指定されるまで、指定オブジェクトファイルのCODEセクションはDEFAULT_CODEブロックに置かれたままになります。
・ブロック名に、デフォルトブロック名(DEFAULT_CODE、DEFAULT_DATA、DEFAULT_BSS)を使用することはできません。ただし、すべてが大文字でなければ使用可能です(例: Default_Codeは使用可能)。

-datablockコマンド

形式: -datablock <**ブロック名**> {<**ファイル名**>...<**ファイル名**>}

記述例: -datablock MAIN_DATA {main.o test.o}

説明:
・{ }内に指定したリロケータブルオブジェクトファイルに含まれるDATAセクションをひとつにまとめて、新しいブロックを作成します。
・ブロックには、指定したファイル順にDATAセクションが連続して配置されます。
・-datablockと<ブロック名>の間には1個以上のスペースまたはタブが必要です。
・複数のファイルを指定する場合、<ファイル名>の間には1個以上のスペース、タブまたは改行が必要です。
・{ }内は複数行に分けて記述できます。

例)
```
-datablock MAIN_DATA
{
        boot.o
        main.o
        sub.o
}
```

・ブロックは、最大256個定義可能です。
・指定可能なブロック名の長さは最大130文字です。大文字と小文字は区別されます。
・指定可能なオブジェクトファイル名の長さは、パスを含めて最大256文字です。
・{ }内で指定可能なファイル数は、最大4000個です。

デフォルト: 本コマンドで指定されないオブジェクトファイルのDATAセクションは、すべてDEFAULT_DATAという名称のブロックに入れられます。

注意:
・同じブロック名で重複定義した場合はワーニングとなります。
この場合は最初の定義が有効になります。

例)
```
-datablock DATA1 {test1.o test2.o}      ...有効
-datablock DATA1 {test3.o test4.o}      ...無効
```

・ブロック名の頭に'@'または'#'は使用できません。
・{ }内で同じオブジェクトファイル名(パスが異なる場合を除く)を指定することはできません。
・{ }内には、必ず1個以上のオブジェクトファイルを指定する必要があります。
・{ }内で指定されたファイル名がリンクファイル(パスも含む)と一致しない場合はワーニングとなり、そのファイルは指定されなかったものとして処理されます。
結果的に{ }内の有効なファイル数が0になった場合はエラーとなります。
ただし、ライブラリのオブジェクトを指定している場合("xxx.lib:xxx.o"指定時)は、エラーになりません。
・ブロック名定義は、-addrコマンドで参照されなくてもエラーになりません。ただし、-addrでアドレスが指定されるまで、指定オブジェクトファイルのDATAセクションはDEFAULT_DATAブロックに置かれたままになります。
・ブロック名に、デフォルトブロック名(DEFAULT_CODE、DEFAULT_DATA、DEFAULT_BSS)を使用することはできません。ただし、すべてが大文字でなければ使用可能です(例: Default_Dataは使用可能)。

-bssblockコマンド

形式: -bssblock <**ブロック名**> {<**ファイル名**>...<**ファイル名**>}

記述例: -bssblock RAM1 {main.o test.o}

説明:
- { }内に指定したリロケータブルオブジェクトファイルに含まれるBSSセクションをひとつにまとめて、新しいブロックを作成します。
- ブロックには、指定したファイル順にBSSセクションが連続して配置されます。
- -bssblockと<ブロック名>の間には1個以上のスペースまたはタブが必要です。
- 複数のファイルを指定する場合、<ファイル名>の間には1個以上のスペース、タブまたは改行が必要です。
- { }内は複数行に分けて記述できます。

  例)
  ```
  -bssblock RAM1
  {
          boot.o
          main.o
          sub.o
  }
  ```
- ブロックは、最大256個定義可能です。
- 指定可能なブロック名の長さは最大130文字です。大文字と小文字は区別されます。
- 指定可能なオブジェクトファイル名の長さは、パスを含めて最大256文字です。
- { }内で指定可能なファイル数は、最大4000個です。

デフォルト: 本コマンドで指定されないオブジェクトファイルのBSSセクションは、すべてDEFAULT_BSSという名称のブロックに入れられます。

注意:
- 同じブロック名で重複定義した場合はワーニングとなります。
  この場合は最初の定義が有効になります。

  例)
  ```
  -bssblock RAM1 {test1.o test2.o}      ...有効
  -bssblock RAM1 {test3.o test4.o}      ...無効
  ```
- ブロック名の頭に'@'または'#'は使用できません。
- { }内で同じオブジェクトファイル名(パスが異なる場合を除く)を指定することはできません。
- { }内には、必ず1個以上のオブジェクトファイルを指定する必要があります。
- { }内で指定されたファイル名がリンクファイル(パスも含む)と一致しない場合はワーニングとなり、そのファイルは指定されなかったものとして処理されます。
  結果的に{ }内の有効なファイル数が0になった場合はエラーとなります。
  ただし、ライブラリのオブジェクトを指定している場合("xxx.lib:xxx.o"指定時)は、エラーになりません。
- ブロック名定義は、-addrコマンドで参照されなくてもエラーになりません。ただし、-addrでアドレスが指定されるまで、指定オブジェクトファイルのBSSセクションはDEFAULT_BSSブロックに置かれたままになります。
- ブロック名に、デフォルトブロック名(DEFAULT_CODE、DEFAULT_DATA、DEFAULT_BSS)を使用することはできません。ただし、すべてが大文字でなければ使用可能です(例: Default_Bssは使用可能)。

-ucodeblockコマンド

形式: -ucodeblock <**ブロック名**> {<**ファイル名**>...<**ファイル名**>}

記述例: -ucodeblock CACHE_OBJ {speed.o cache.o}

説明:
- 指定したオブジェクトファイルに含まれるCODEセクションを同一領域(共用ブロック)に割り当てます。指定した複数のオブジェクトコードで1つのRAM領域を共用し、時分割で転送して実行を繰り返すような場合に有効です。オブジェクトコードが実際に置かれる場所は-codeblockで指定します(デフォルトはDEFAULT_CODEブロック内)。
- ブロックのサイズは、割り当てるセクションの中でもっとも長いサイズが採用されます。
- -ucodeblockと<ブロック名>の間には1個以上のスペースまたはタブが必要です。
- 複数のファイルを指定する場合、<ファイル名>の間には1個以上のスペース、タブまたは改行が必要です。

・{ }内は複数行に分けて記述できます。
例) -ucodeblock CACHE_OBJ
{
boot.o
main.o
sub.o
}
・ブロックは、最大256個定義可能です。
・指定可能なブロック名の長さは最大130文字です。大文字と小文字は区別されます。
・指定可能なオブジェクトファイル名の長さは、パスを含めて最大256文字です。
・{ }内で指定可能なファイル数は、最大4000個です。

デフォルト: 本コマンドが指定されない場合、CODEセクションの共用ブロックは作成されません。

注意:
・同じブロック名で重複定義した場合はワーニングとなります。
この場合は最初の定義が有効になります。
例) -ucodeblock OBJ2 {test1.o test2.o} ...有効
-ucodeblock OBJ2 {test3.o test4.o} ...無効
・定義するブロック名の先頭に'@'または'#'は使用できません。
・本コマンドで定義したブロックのアドレスを-addrコマンドで指定するときは、必ずブロック名の先頭に"@"を付けて指定してください。"@"が付いていない場合はエラーとなります。
・{ }内で同じオブジェクトファイル名(パスが異なる場合を除く)を指定することはできません。
・{ }内には、必ず1個以上のオブジェクトファイルを指定する必要があります。
・{ }内で指定されたファイル名がリンクファイル(パスも含む)と一致しない場合はワーニングとなり、そのファイルは指定されなかったものとして処理されます。
結果的に{ }内の有効なファイル数が0になった場合はエラーとなります。
ただし、ライブラリのオブジェクトを指定している場合("xxx.lib:xxx.o"指定時)は、エラーになりません。
・ブロック名定義は、-addrコマンドで参照されなくてもエラーになりません。
・ブロック名に、デフォルトブロック名(DEFAULT_CODE、DEFAULT_DATA、DEFAULT_BSS)を使用することはできません。ただし、すべてが大文字でなければ使用可能です(例: Default_Codeは使用可能)。

-udatablockコマンド

形式: -udatablock <**ブロック名**> {<**ファイル名**>...<**ファイル名**>}

記述例: -udatablock DATA {speed.o cache.o}

説明:
・指定したオブジェクトファイルに含まれるDATAセクションを同一領域(共用ブロック)に割り当てます。指定した複数のオブジェクトのデータで1つのRAM領域を共用し、時分割で転送して使用する場合に有効です。データが実際に置かれる場所は-datablockで指定します(デフォルトはDEFAULT_DATAブロック内)。
・ブロックのサイズは、割り当てるセクションの中でもっとも長いサイズが採用されます。
・-udatablockと<ブロック名>の間には1個以上のスペースまたはタブが必要です。
・複数のファイルを指定する場合、<ファイル名>の間には1個以上のスペース、タブまたは改行が必要です。
・{ }内は複数行に分けて記述できます。
例) -udatablock DATA
{
boot.o
main.o
sub.o
}
・ブロックは、最大256個定義可能です。
・指定可能なブロック名の長さは最大130文字です。大文字と小文字は区別されます。
・指定可能なオブジェクトファイル名の長さは、パスを含めて最大256文字です。
・{ }内で指定可能なファイル数は、最大4000個です。

デフォルト: 本コマンドが指定されない場合、DATAセクションの共用ブロックは作成されません。

注意:
- 同じブロック名で重複定義した場合はワーニングとなります。
  この場合は最初の定義が有効になります。
  例) -ucodeblock DATA2 {test1.o test2.o} ...有効
  -ucodeblock DATA2 {test3.o test4.o} ...無効
- 定義するブロック名の先頭に'@'または'#'は使用できません。
- 本コマンドで定義したブロックのアドレスを-addrコマンドで指定するときは、必ずブロック名の先頭に"@"を付けて指定してください。"@"が付いていない場合はエラーとなります。
- { }内で同じオブジェクトファイル名(パスが異なる場合を除く)を指定することはできません。
- { }内には、必ず1個以上のオブジェクトファイルを指定する必要があります。
- { }内で指定されたファイル名がリンクファイル(パスも含む)と一致しない場合はワーニングとなり、そのファイルは指定されなかったものとして処理されます。
  結果的に{ }内の有効なファイル数が0になった場合はエラーとなります。
  ただし、ライブラリのオブジェクトを指定している場合("xxx.lib:xxx.o"指定時)は、エラーになりません。
- ブロック名定義は、-addrコマンドで参照されなくてもエラーになりません。
- ブロック名に、デフォルトブロック名(DEFAULT_CODE、DEFAULT_DATA、DEFAULT_BSS)を使用することはできません。ただし、すべてが大文字でなければ使用可能です(例: Default_Dataは使用可能)。

-ubssblockコマンド

形式: -ubssblock <**ブロック名**> {<**ファイル名**>...<**ファイル名**>}

記述例: -ubssblock RAM2 {speed.o cache.o}

説明:
- 指定したオブジェクトファイルに含まれるBSSセクションを同一領域(共用ブロック)に割り当てます。指定した複数のオブジェクトのデータで1つのRAM領域を共用し、時分割で転送して使い分ける場合に有効です。
- ブロックのサイズは、割り当てるセクションの中でもっとも長いサイズが採用されます。
- -ubssblockと<ブロック名>の間には1個以上のスペースまたはタブが必要です。
- 複数のファイルを指定する場合、<ファイル名>の間には1個以上のスペース、タブまたは改行が必要です。
- { }内は複数行に分けて記述できます。

```
例) -ubssblock RAM2
    {
            boot.o
            main.o
            sub.o
    }
```

- ブロックは、最大256個定義可能です。
- 指定可能なブロック名の長さは最大130文字です。大文字と小文字は区別されます。
- 指定可能なオブジェクトファイル名の長さは、パスを含めて最大256文字です。
- { }内で指定可能なファイル数は、最大4000個です。

デフォルト: 本コマンドが指定されない場合、BSSセクションの共用ブロックは作成されません。

注意:
- 同じブロック名で重複定義した場合はワーニングとなります。
  この場合は最初の定義が有効になります。
  例) -ubssblock RAM2 {test1.o test2.o} ...有効
  -ubssblock RAM2 {test3.o test4.o} ...無効
- 定義するブロック名の先頭に'@'または'#'は使用できません。
- 本コマンドで定義したブロックのアドレスを-addrコマンドで指定するときは、必ずブロック名の先頭に"@"を付けて指定してください。"@"が付いていない場合はエラーとなります。
- { }内で同じオブジェクトファイル名(パスが異なる場合を除く)を指定することはできません。
- { }内には、必ず1個以上のオブジェクトファイルを指定する必要があります。
- { }内で指定されたファイル名がリンクファイル(パスも含む)と一致しない場合はワーニングとなり、そのファイルは指定されなかったものとして処理されます。

結果的に{ }内の有効なファイル数が0になった場合はエラーとなります。
ただし、ライブラリのオブジェクトを指定している場合("xxx.lib:xxx.o"指定時)は、エラーになりません。

・ ブロック名定義は、-addrコマンドで参照されなくてもエラーになりません。
・ ブロック名に、デフォルトブロック名(DEFAULT_CODE、DEFAULT_DATA、DEFAULT_BSS)を使用することはできません。ただし、すべてが大文字でなければ使用可能です(例: Default_Bssは使用可能)。

-addrコマンド

形式1: -addr <**アドレス**> {<**ブロック名**>...<**ブロック名**>}
形式2: -addr <**シンボル**> {<**ブロック名**>...<**ブロック名**>}
記述例: -addr 0x100000 {DEFAULT_CODE MAIN_OBJ}
-addr EXT_ROM {DEFAULT_CODE MAIN_OBJ}
説明:

・ 指定したブロックを指定したアドレスに配置します。形式1ではアドレスを直接指定します。また、形式2のように、-defaddrコマンドで定義したシンボルをアドレスの代わりに使用することもできます。
・ 指定可能なアドレス範囲は0x0～0xffffffffです。
・ ブロック名の先頭に"@"を付けた場合、仮想ブロックとしての配置を意味します。
仮想ブロックはRAMなどに割り当てるブロックで、実行時はこのアドレスに置かれるものとして処理されます。たとえば、ROM内のプログラムをRAMに転送して実行させる場合に指定します。
・ 標準設定では、仮想ブロックのシンボリックデバッグ情報は仮想アドレスに、ソースレベルデバッグ情報は実アドレスに割り当てられます(ブロック名の先頭に"@"のみを付けた場合)。
・ ブロック名の先頭に"@#"を付けた場合、ソースレベルデバッグ情報を仮想ブロックに割り当てます。-uXXXblockコマンドでブロック化したセクションのソースレベルデバッグ情報は一番最初に指定したファイルのみ有効となります。
ただし、アブソリュートファイルを含むブロックの場合、アブソリュートファイルはデバッグ情報添付の対象になりません。次の例で、abs.oがアブソリュートファイルの場合、abs.oは仮想ブロックにはなりません。rel1.o、rel2.oは仮想ブロックです。

例) -codeblock BLK001 { rel1.o abs.o rel2.o }
-addr 0x1000 { @#BLK001 }

また、共有ブロックの最初のファイルがアブソリュートファイルのとき、デバッグ情報添付の対象は、最初のリロケートファイルになります。
次の例では、abs1.o、abs2.o、rel2.oは仮想ブロックになりません。rel1.oは仮想ブロックです。

例) -ucodeblock BLK001 { abs1.o abs2,o rel1.o rel2.o}
-addr 0x1000 { @#BLK001 }

"@#"は、-codeblock、-ucodeblockで定義したブロック名とDEFAULT_CODEに付加して使用します。これは、ソースデバッグ情報の文情報は、コードセクションについての情報であるためです。
DATAとBSSのブロックでは、"@#"と"@"が同じ意味になります。

例) -datablock BLOCK1 { boot.o main.o }
-addr 0x80000 { @BLOCK1 } ...(1)
-addr 0x80000 { @#BLOCK1 } ...(2)

(1)と(2)はどちらも同じ意味になります。

・ 仮想デフォルトブロック以外の仮想ブロックが-addrコマンド内に定義されているとき、仮想デフォルトブロックからはこの仮想ブロック内のオブジェクトファイルが除かれます。

例) -codeblock OBJ1 { boot.o }
-addr 0x100000 { @OBJ1 @DEFAULT_CODE }
boot.o
main.o
sub.o

この場合、DEFAULT_CODEには、boot.o、main.o、sub.oが含まれます。
@OBJ1はboot.oを含みます。@DEFAULT_CODEは、main.oとsub.oを含みます。

・-addrと<アドレス>または<シンボル>の間には1個以上のスペースまたはタブが必要です。
・ 複数のブロックを指定する場合、<ブロック名>の間には1個以上のスペース、タブまたは改行が必要です。
・ { }内は複数行に分けて記述できます。

例) -addr IN_ROM
{
MAIN_BLOCK
SUB_BLOCK
}

・ ブロックは、最大256個定義可能です。
・ 指定可能なブロック名の長さは最大130文字です。大文字と小文字は区別されます。
・ 指定可能なオブジェクトファイル名の長さは、パスを含めて最大256文字です。
・ { }内で指定可能なブロック数は、最大256個です。

デフォルト: 本コマンドで指定されないブロックは、次のように配置されます。
CODEブロック: 0x0c00000を先頭に配置
DATAブロック: 最上位のCODEブロックの直後
BSSブロック: 内蔵RAMの先頭(0x0000000)

注意:
・ 形式2のシンボル名は、使用する前に-defaddrコマンドによる定義が必要です。
・ -addrで指定するブロック名は、デフォルトブロック名を除き、前述の-XXXblockまたは-uXXXblockコマンドであらかじめ定義されている必要があります。
・ アドレスには、-alignコマンドで指定されたアラインメント境界を指定します。
それ以外の指定ではワーニングとなり、下位nビットは無視されます。
デフォルトのアライメントは4バイト境界です。
・ 次のように同じアドレス(または、シンボル)を重複して定義した場合はワーニングとなり、最初のコマンドのみ有効になります。

例1) -addr 0x50000 { BLOCK1 BLOCK2 } ...有効
-addr 0x50000 { DEFAULT_CODE } ...無効

・ ブロック名を1つの-addrコマンド内または他の-addrコマンドで重複して指定するとエラーになります。ただし、"@"の有無によって異なるブロック名とみなされます。

例1) -addr IN_ROM { BLOCK1 @BLOCK1 } ...OK "@"なしと"@"ありで異なる
例2) -addr IN_ROM { BLOCK1 BLOCK2 BLOCK1 } ...エラー BLOCK1が重複
例3) -addr IN_ROM { @BLOCK1 BLOCK2 @BLOCK1 } ...エラー @BLOCK1が重複
例4) -addr IN_RAM { BLOCK1 BLOCK2 BLOCK3 }
-addr EXT_RAM { BLOCK4 BLCOK1 } ...エラー BLOCK1がIN_RAMにもある

・ 同一のオブジェクト定義を含む複数のブロックを、同じ-addr定義内、または他の-addr定義内で指定することはできません。

例1) -codeblock BLOCK1 { boot.o main.o sub.o }
-codeblock BLOCK2 { main.o }
-addr 0x800
{
BLOCK1
BLOCK2 ...エラー main.oが重複
}

例2) -codeblock BLOCK1 { boot.o main.o sub.o }

```
-codeblock BLOCK2 { comp.o }
-codeblock BLOCK3 { main.o }
-addr 0x800
{
        BLOCK1
        BLOCK2
}
-addr 0x1000
{
        BLOCK2
        BLOCK3                          ...エラー  main.oが重複
}
```

- 指定された仮想ブロック(デフォルトのブロック名は除く)の中に"xxx.lib:xxx.o"形式のファイル名があるとワーニングになり、このブロック定義は無視されます。ただし、実ブロックのときはマッピングされます。
- "@"のみまたは"@#"のみのブロック名は使用できません。
- { }内には、必ず1個以上のブロックを指定する必要があります。
- ブロックを配置するアドレスは、他の-addrで定義されているアドレスと重複することはできません。

-defaddrコマンド

形式: -defaddr <**シンボル**>=<**アドレス**>

記述例: -defaddr DRAM_AREA=0x800000

説明:
- シンボルにアドレスを定義します。定義したシンボルは、前述の-addrコマンドのアドレスの代わりに使用することができます。

  例)
  ```
  -defaddr IN_RAM=0x000100
  -addr IN_RAM { BLOCK1,BLOCK3 }
  ```
- 指定可能なアドレス範囲は0x0～0xffffffffです。
- シンボルは、最大256個定義可能です。
- 指定可能なシンボル名の長さは最大130文字です。大文字と小文字は区別されます。

注意:
- アドレスには、4バイト境界の値を指定してください。それ以外の指定はワーニングとなり、指定アドレスの下位2ビットは無視されます。
- 次のように同じシンボル名を定義した場合はワーニングとなり、最初のコマンドが有効となります。

  例)
  ```
  -defaddr IN_RAM=0x000000        ...有効
  -defaddr IN_RAM=0x050000        ...無効
  ```

12.6.3 ブロックの配置

●デフォルト設定によるブロックの配置

各リロケータブルオブジェクトファイルは、それぞれ1つずつのCODEセクション、DATAセクション、BSSセクションを持ちます。
lk33 Ver. 3.0モードでは、指定した複数のリロケータブルオブジェクトファイル内の同じ種類のセクションを1つのブロックとして定義し、そのブロックごとにアドレスを指定して配置することができます。特にブロックへの割り当てを行わないリロケータブルオブジェクトファイルおよびブロック定義はされていてもアドレスが指定されていないリロケータブルオブジェクトファイルのセクションは、以下の名称のデフォルトブロックに割り当てられます。
DEFAULT_CODE: CODEセクションのデフォルトブロック
DEFAULT_DATA: DATAセクションのデフォルトブロック
DEFAULT_BSS: BSSセクションのデフォルトブロック

ブロック内の配置は、コマンドファイル内の入力ファイルの記述順となります。

デフォルトブロックも含め、各ブロックの先頭アドレス(リンク後の絶対アドレス)はリンカコマンドで指定できます。指定のない場合は、次のようにアドレスが決定します。
CODEブロック: 0x0c00000番地から配置されます。
DATAブロック: すべてのCODEブロックを配置後、最も上位アドレスに配置されたCODEブロックの直後に配置されます。
BSSブロック: 0x0000000番地から配置されます。

たとえば、2つのリロケータブルオブジェクトファイルsample1.oとsample2.oをブロック定義およびアドレス指定なしでリンクすると、各セクションは次のように配置されます。

例: コマンドファイル
-v3
sample1.o
sample2.o

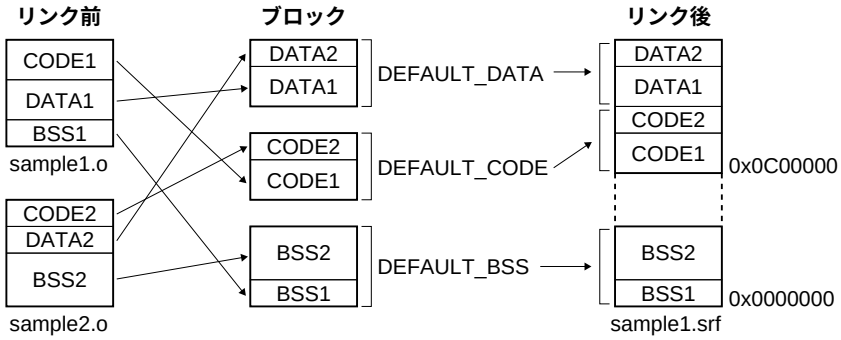


図12.6.3.1 **デフォルト設定でのリンク例**

デフォルトブロックを前記のデフォルト以外のアドレスから配置するには、-addrコマンドによって開始アドレスを指定します。
たとえば、図12.6.3.1のCODEブロックの開始アドレスを0x0080000番地、BSSブロックの開始アドレスを0x0010000番地に変更するには次のようなコマンドファイルを入力してリンクします。
例: コマンドファイル
-v3
-addr 0x0010000 {DEFAULT_BSS}
-addr 0x0080000 {DEFAULT_CODE DEFAULT_DATA}
sample1.o
sample2.o

DEFAULT_CODE**ブロック**: 0x0080000番地から配置されます。
DEFAULT_DATA**ブロック**: DEFAULT_CODEブロックを配置後、その直後に配置されます。
DEFAULT_BSS**ブロック**: 0x0010000番地から配置されます。

●特定ファイルを特定アドレスから配置するには

必要なファイルを-codeblock、-datablock、-bssblockコマンドでブロック化し、-addrコマンドでアドレスを指定することにより、指定ファイルの特定セクションを指定のアドレスに配置することができます。複数のファイルが指定可能で、その場合は開始アドレスからファイルの指定順に配置されます。

例: コマンドファイル

```
-v3
-codeblock EXTRA_BLOCK
{
        sample2.o
        sample3.o
}
-addr 0xc10000 {EXTRA_BLOCK}
sample1.o
sample2.o
sample3.o
```

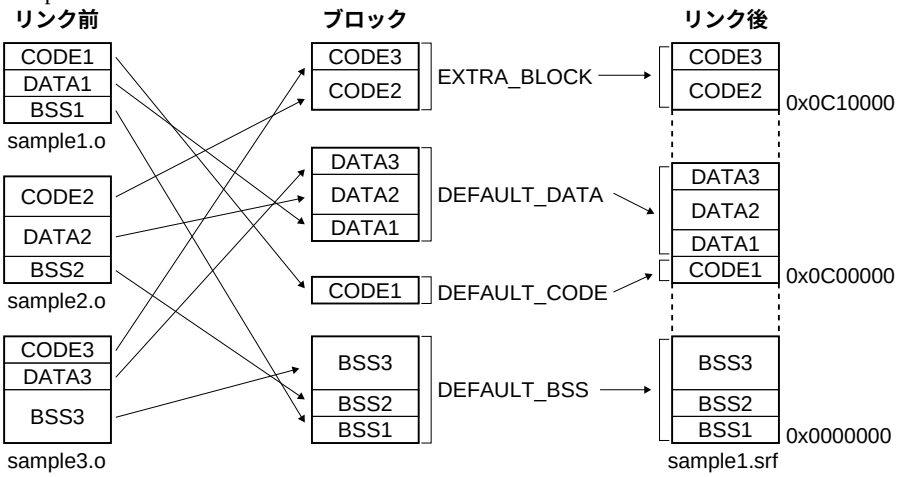


図12.6.3.2 **特定ファイルの配置例**

この場合、sample2.o、sample3.oのCODEセクションはEXTRA_BLOCKに定義されていますので、0x00c10000から配置され、sample1.oの全セクションおよびsample2.o、sample3.oのDATAとBSSセクションはデフォルトブロックに割り当てられた状態で配置されます。

-addrコマンドでは、-defaddrコマンドで定義したシンボルを使用してアドレスを指定することもできます。

例: -defaddr EXT_ROM=0xc10000
    -addr EXT_ROM {EXTRA_BLOCK}

●アブソリュートファイルの配置

アブソリュートオブジェクトファイルは、リンク前に各セクションの絶対アドレスが確定しているため、その番地に優先的に配置されます。ブロック内に他のリロケータブルオブジェクトと共に定義されアドレスを指定されたとしても、その影響を受けません。

例: コマンドファイル

```
-v3
-codeblock EXTRA_BLOCK
{
        abs.o           ; アブソリュートオブジェクトファイル
        rel.o           ; リロケータブルオブジェクトファイル
}
-addr 0xc10000 {EXTRA_BLOCK}
abs.o
rel.o
```

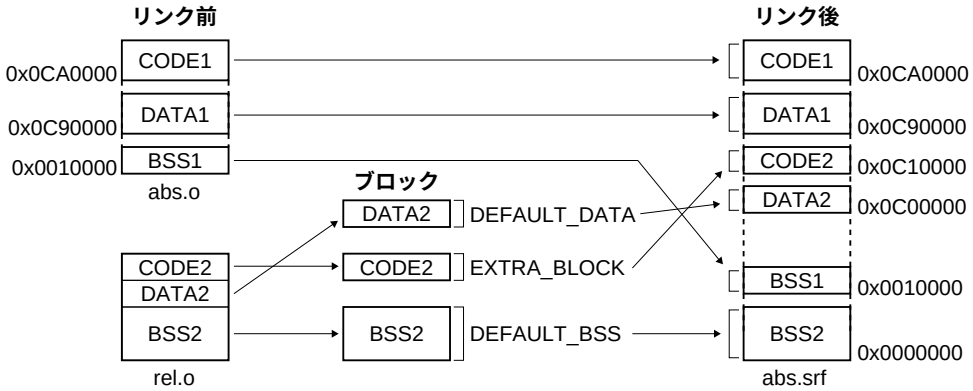


図12.6.3.3　**アブソリュートオブジェクトファイルの配置例**

この例では、アブソリュートファイルであるabs.oはリロケータブルファイルrel.oと共にブロック定義されていますがabs.oはファイルの持つ本来のアドレスに配置されます。rel.oのCODEセクションは0x00c10000から配置され、rel.oのその他のセクションはデフォルトブロックとして配置されます。

●ブロックのアライメント

各ブロックはデフォルトで4バイト境界に配置されます。

-alignコマンドでアライメント境界を2^nに設定することもできます。

-addrコマンドで4バイト境界以外のアドレスあるいは-alignコマンドで設定した境界アドレス以外のブロック配置アドレスを指定した場合はワーニングとなり、アドレスの下位ビットを0として最も指定アドレスに近いアライメント境界に配置されます。

12.6.4 仮想ブロックと共用ブロック

仮想ブロックと共用ブロックはともに実データを持たないブロックです。
仮想ブロックは配置先に実態は存在しませんが、実行時にオブジェクトを転送して使用することができます。たとえば、RAM上でのプログラムの実行等に有効です。
共用ブロックは異なるいくつかのオブジェクトを並列に配置する機能で、仮想ブロックとして配置する必要があります。実行時に共有エリアにオブジェクトを転送して使用します。

●仮想ブロック

-addrコマンド内で、"@"を付加したブロック名(-codeblock、-datablock、-bssblockコマンドで定義)を指定すると、そのブロックは仮想ブロックとして配置されます。指定ブロック内のオブジェクトが指定アドレスで始まる仮想ブロックで実行されるものとして、シンボルの解決が行われます。シンボリックデバッグ情報は仮想アドレスに、ソースレベルデバッグ情報は実アドレスに割り当てられます。ソースレベルデバッグ情報も仮想ブロックに割り当てるためには、ブロック名の頭に"@#"を付けて指定します。

```
例: -addr IN_RAM
    {
            @OBJ1     ... "OBJ1"を仮想ブロックとして配置
            @#OBJ2    ... "OBJ2"を仮想ブロックとして配置。ソースレベルデバッグ情報も仮想
    }                     ブロックに割り当てる
```

この例では、IN_RAMというRAM内のアドレスを指定して、そこからOBJ1を、続けてOBJ2を配置しています。これにより、これらのオブジェクトのシンボル情報はIN_RAMから始まる仮想ブロック内のアドレスで生成されます。実際のオブジェクトは不揮発性メモリに配置しておく必要がありますが、そのアドレスは"@"または"@#"を付けずに別途-addrコマンドで指定します。特に実際のオブジェクトの配置を指定しなかった場合は、デフォルトブロックに入れられて処理されます。OBJ1のソースレベルデバッグ情報は、この実アドレスの方で生成されます。OBJ2は"@#"付きのため、仮想ブロック内のアドレスが割り当てられます。したがって、デバッグ時はOBJ1のソースが実アドレスの位置に、OBJ2のソースがRAM内の割り付け位置に表示されます。
なお、実際にOBJ1とOBJ2を実行する前には、ユーザプログラムで実アドレス位置から仮想アドレス位置にコードを転送しておく必要があります。

●共用ブロック

共用ブロックの定義コマンド(-ucodeblock、-udatablock、-ubssblock)内で指定されたオブジェクトは、ブロック内に並列に配置されます。つまり、複数のオブジェクトが同一の開始アドレスから配置されます。
共用ブロックの定義コマンドでブロック化したセクションのソースレベルデバッグ情報は、一番最初に指定したファイルのみ有効となります。-addrコマンドでアドレスを指定するときは、必ず共用ブロック名の先頭に"@"を付けてください。

```
例: -ucodeblock OBJ1      ... 共有ブロックの定義
    {                         sample1.oとsample2.oのCODEセクションを同一エリア
            sample1.o         (同一アドレス上)に設定
            sample2.o
    }
    -addr IN_RAM
    {
            @OBJ1     ... 共有ブロック"OBJ1"は必ず仮想ブロックとして配置します。
    }
```

この例では、OBJ1という共有ブロックを定義してIN_RAMというRAM内のアドレスから配置しています。sample1.oとsample2.oのCODEセクションが、どちらもこの領域で実行されるものとしてシンボル情報が生成されます。ソースレベルデバッグ情報はsample1.oのみ生成されます。実際のオブジェクトは不揮発性メモリに配置しておく必要がありますが、それには別途-codeblockコマンドで実ブロックを定義して、そのアドレスを-addrコマンドで指定します。特に指定しなかった場合は、デフォルトブロックに入れられて処理されます。
なお、実際にsample1.oとsample2.oのコードを実行する前には、ユーザプログラムで実アドレス位置から共用アドレス位置にコードを転送しておく必要があります。

●仮想ブロック、共有ブロックの配置例

以下に、仮想ブロック、共有ブロックを使用した配置例を示します。

例: コマンドファイル

```
-v3                          ... lk33 Ver. 3.0以降のコマンドを使う

-defaddr IN_RAM=0x0          ... アドレスのシンボル定義
-defaddr EXT_RAM=0x600000
-defaddr EXT_ROM=0xc00000

-codeblock OBJ1              ... ブロックの定義
{
        sample2.o
        sample3.o
}

-ucodeblock OBJ2             ... 共有ブロックの定義
{                                sample1.oとsample4.oのCODEセクションを同一エリア
        sample1.o                (同一アドレス上)に設定
        sample4.o
}

                             ... ここよりブロックの配置
-addr IN_RAM                     (オブジェクトは直接配置できない)
{
        DEFAULT_BSS
        @DEFAULT_DATA
        @#OBJ1               ... "OBJ1"を仮想ブロックとして配置
        @OBJ2                ... 共有ブロック"OBJ2"を仮想ブロックとして配置
}

-addr EXT_ROM
{
        DEFAULT_CODE
        OBJ1
        DEFAULT_DATA
}

sample1.o                    ... オブジェクト、ライブラリの指定
sample2.o
sample3.o
sample4.o
sample5.o
sample.lib
```

注: ・**デフォルトコードブロック内**(DEFAULT_CODE)**のデバッグ情報付きのライブラリはソースレベルデバッグが可能です。**
ただし、仮想ブロック(@#DEFAULT_CODE)**の場合はデバッグ情報が仮想領域にマッピングされないため、仮想領域においてのソースレベルデバッグはできません。**

・**仮想ブロックのアブソリュートファイルのソースデバッグ情報は仮想メモリには配置されません。**

上記例の実際の配置は図12.6.4.1のようになります。

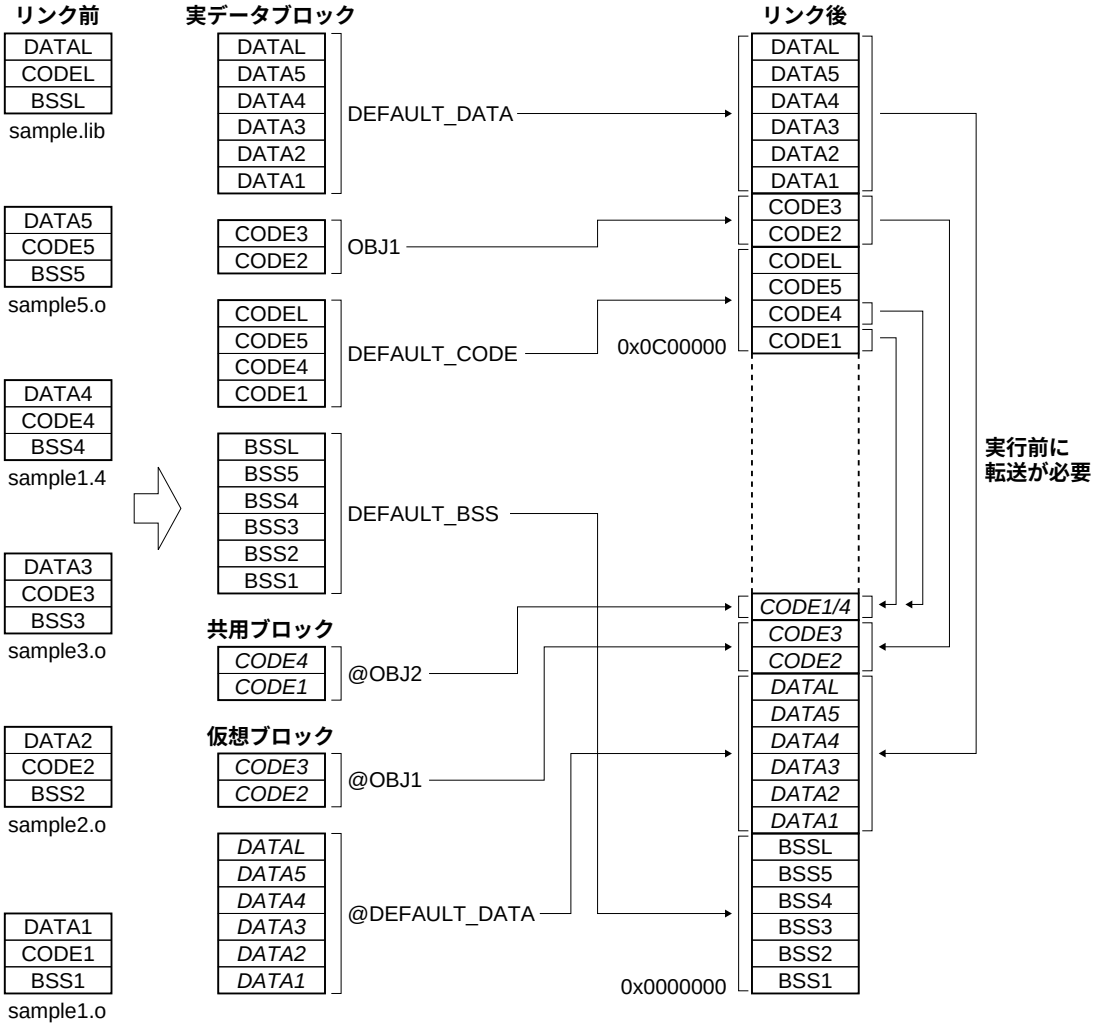


図12.6.4.1 **仮想ブロック、共用ブロックへの配置例**

上記例のリンクによって生成されるシンボルを以下に示します。これらのシンボルは仮想アドレスへの転送や各種アドレス指定に、ソースファイル内で使用することができます。生成されるシンボルの種類については、次節を参照してください。

__START_DEFAULT_BSS	DEFAULT_BSSブロック開始アドレス
__END_DEFAULT_BSS	DEFAULT_BSSブロック終了アドレス
__SIZEOF_DEFAULT_BSS	DEFAULT_BSSブロックサイズ
__START_V_DEFAULT_DATA	@DEFAULT_DATA仮想ブロック開始アドレス
__END_V_DEFAULT_DATA	@DEFAULT_DATA仮想ブロック終了アドレス
__START_V_OBJ1	@OBJ1仮想ブロック開始アドレス
__END_V_OBJ1	@OBJ1仮想ブロック終了アドレス
__START_V_OBJ2	@OBJ2共用ブロック開始アドレス
__END_V_OBJ2	@OBJ2共用ブロック終了アドレス
__START_DEFAULT_CODE	DEFAULT_CODEブロック開始アドレス
__END_DEFAULT_CODE	DEFAULT_CODEブロック終了アドレス
__SIZEOF_DEFAULT_CODE	DEFAULT_CODEブロックサイズ

__START_OBJ1	OBJ1ブロック開始アドレス
__END_OBJ1	OBJ1ブロック終了アドレス
__SIZEOF_OBJ1	OBJ1ブロックサイズ
__START_DEFAULT_DATA	DEFAULT_DATAブロック開始アドレス
__END_DEFAULT_DATA	DEFAULT_DATAブロック終了アドレス
__SIZEOF_DEFAULT_DATA	DEFAULT_DATAブロックサイズ
__START_IN_RAM	IN_RAM定義アドレス
__END_IN_RAM	-addr IN_RAM定義ブロック終了アドレス
__SIZEOF_IN_RAM	-addr IN_RAM定義ブロックサイズ
__STAR_EXT_ROM	EXT_ROM定義アドレス
__END_EXT_ROM	-addr EXT_ROM定義ブロック終了アドレス
__SIZEOF_EXT_ROM	-addr EXT_ROM定義ブロックサイズ

以下のシンボルは-objsymコマンドが指定されているときに出力されます。

__START_sample1_code	sample1.o CODEセクション開始アドレス
__END_sample1_code	sample1.o CODEセクション終了アドレス
__SIZEOF_sample1_code	sample1.o CODEセクションサイズ
__START_sample1_data	sample1.o DATAセクション開始アドレス
__END_sample1_data	sample1.o DATAセクション終了アドレス
__SIZEOF_sample1_data	sample1.o DATAセクションサイズ
__START_sample1_bss	sample1.o BSSセクション開始アドレス
__END_sample1_bss	sample1.o BSSセクション終了アドレス
__SIZEOF_sample1_bss	sample1.o BSSセクションサイズ
:	
__START_sample5_code	
__END_sample5_code	
__SIZEOF_sample5_code	
__START_sample5_data	
__END_sample5_data	
__SIZEOF_sample5_data	
__START_sample5_bss	
__END_sample5_bss	
__SIZEOF_sample5_bss	

12.6.5 Ver. 3.0モードで生成されるシンボル

lk33は配置後のブロックのアドレス、サイズを示すブロックシンボル(グローバルシンボル)を生成することができます。それらのシンボルはソース中で使用することができます。

●実ブロックのシンボル

ブロック指定時には以下のシンボルが生成されます。

__START_<**ブロック名**>	ブロックの開始アドレス
__END_<**ブロック名**>	ブロックの終了アドレス
__SIZEOF_<**ブロック名**>	ブロックのサイズ(バイト数)

●仮想ブロックのシンボル

仮想ブロック("@"付き)を配置すると、以下のシンボルが生成されます。仮想ブロックの指定だけでは、上記の実ブロックのシンボルは生成されません。

__START_V_<**ブロック名**>	仮想ブロックの開始アドレス
__END_V_<**ブロック名**>	仮想ブロックの終了アドレス

共用ブロック(-uXXXblock)の場合、割り当てた複数のセクションの中で最も大きなセクションのサイズで終了アドレスが設定されます。

●アドレス名

ブロックの配置アドレスを-addrコマンドで定義すると生成されます。

__START_<**アドレスまたはシンボル**>	開始アドレス
__END_<**アドレスまたはシンボル**>	終了アドレス
__SIZEOF_<**アドレスまたはシンボル**>	指定アドレスから配置したブロックの合計サイズ(バイト数)

●デフォルトシンボル

以下のシンボルはデフォルトDATAブロックの有無、"-v3"指定の有無にかかわらず出力されます。

__START_DEFAULT_DATA	デフォルトDATAブロック開始アドレス
__END_DEFAULT_DATA	デフォルトDATAブロック終了アドレス
__SIZEOF_DEFAULT_DATA	デフォルトDATAブロックサイズ

●シンボルファイルへの出力

ブロック名/アドレス名とも、シンボルファイルへの出力は、
File = $$lk33$$、Section = uscn、Type = globalに固定されます。

例: ブロック名=BLOCK1として出力されたシンボル

Symbol	File	Section	Type	Address
__START_BLOCK1	$$lk33$$	uscn	global	00000400
__END_BLOCK1	$$lk33$$	uscn	global	000009f0
__SIZEOF_BLOCK1	$$lk33$$	uscn	global	000005f0

●-objsymコマンドによるセクションシンボルの作成

コマンドファイルに-objsymコマンドを記述しておくと、lk33は各入力ファイルのセクションごとに、再配置後のセクション情報を示す以下のようなシンボルを生成します。

__START_<**ファイル名**>_<**セクション**>	セクションの開始アドレス
__END_<**ファイル名**>_<**セクション**>	セクションの終了アドレスの次のアドレス
__SIZEOF_<**ファイル名**>_<**セクション**>	セクションサイズ(バイト数)

例: 入力ファイル名が"test.o"、CODEセクションの開始アドレスが0x0c00000、サイズが0x100の場合

シンボル名	シンボルの値
__START_test_code	0x0c00000
__END_test_code	0x0c00100
__SIZEOF_test_code	0x100

<ファイル名>のパスと拡張子は、コマンドファイル内の記述にかかわらずカットされます。
シンボルに定義される開始アドレスとサイズは、リンクマップファイルの出力内容(実データアドレス)と同じで、終了アドレスは開始アドレス+サイズとなります。

●アセンブリソースでのブロックシンボル使用例

例: ブロック"OBJ1"を仮想ブロック"CACHE1"に転送する場合

ソースファイル内の転送ルーチン

```
        xld.w   %r12, __START_CACHE1
        xld.w   %r13, __START_OBJ1
        xld.w   %r14, __SIZEOF_OBJ1

HCOPY_LOOP:
        ld.uh   %r4, [%r13]+        ; 命令コードの転送
        ld.h    [%r12]+, %r4
        sub     %r14, 2
        jrgt    HCOPY_LOOP
```

12.6.6 ライブラリのリンク

ライブラリのリンクは他の入力オブジェクトファイルの配置を終了後に行われます。

●ライブラリモジュールの検索

lk33は入力オブジェクト内に未解決の外部参照シンボルがある場合にのみ、コマンドファイル内に記述された順にライブラリファイルを検索し、最初に見つかったそのシンボルを持つモジュールのみを読み込んでリンクします。外部参照が解決すると、それ以降のモジュール、ライブラリファイルは検索しません。したがって、指定のライブラリファイルに検索対象のシンボルが複数定義されていても、最初に見つかったモジュール以外は無視されます。
ライブラリがカレントディレクトリにない場合は、-lコマンドで指定されたディレクトリを検索します。

ライブラリファイル間の外部参照は、現在処理中のライブラリ内のファイルあるいはそれ以降に検索するライブラリファイルでのみ解決できます。すでに検索したライブラリファイルで定義されていても解決されません。ライブラリを指定する順序に注意してください。

●ライブラリモジュールの配置

特にブロック定義がされていないライブラリモジュールは、デフォルトブロックに割り当てられます。
特定のモジュールを特定のアドレスに配置するには、-XXXblock、-uXXXblockコマンドを使用し、ファイル名を"ライブラリ名:モジュール名"の形式で指定してください。この名称はリンクマップファイルのFile欄に表示される名前と同一になるようにしてください。定義したブロックは-addrコマンドで指定アドレスに配置することができます。
例: コマンドファイル

```
-v3
-codeblock OBJL {c:¥cc33¥lib¥string.lib:strcpy.o}
-addr 0xca0000
test.o
string.lib
```

12.7 シンボルの解明

●グローバルシンボルの解明手順

lk33は次の手順でシンボルを解明します。

(1) lk33は指定の入力オブジェクトファイルの順にグローバルシンボル情報を内部テーブルに追加していきます。未定義シンボルが参照されているとそのテーブルを検索し、合致するシンボル情報があればその内容を適用します。テーブルに情報がなければ、その後の入力ファイルで定義されるまで保留にします。

(2) 同じ名称のグローバルシンボルが多重定義されている場合、ワーニングを出力します。lk33は、入力ファイル順で最初に現れたシンボル情報を有効として処理を継続します。(BSSセクションに定義されたグローバルシンボルを除く)

(3) すべての入力オブジェクトファイルを検索後、未解決の外部参照がある場合に限り、ライブラリファイルを記述順に検索します。そのシンボル定義が現れると、そのモジュールをリンクします。複数のライブラリモジュールで同じ名称のシンボルが定義されている場合はワーニングとし、最初に現れたモジュールがリンクされます。

(4) リンクするライブラリモジュールに未解決の外部参照があり、入力オブジェクトファイル内あるいは現在処理中のライブラリファイルで定義されていれば、その内容を適用します。定義されていない場合は、それ以降のライブラリファイルで定義されるまで保留にします。
ライブラリファイル間の外部参照は、現在処理中のファイルあるいはそれ以降に検索するライブラリファイルでのみ解決できます。すでに検索したライブラリファイルで定義されていても解決されません。

(5) 全ライブラリファイルの検索後、まだ未定義の外部参照がある場合はワーニングを出力し、そのシンボルを参照する命令のリロケーションを行いません。命令中の変更を必要とするビットは0x0として処理されます。

●BSSセクション内で多重定義されているグローバルシンボル

.comm擬似命令で同じ名称のグローバルシンボルが多重定義されている場合、最もサイズが大きいシンボル定義が有効となります。同サイズの場合は入力ファイル順で最初に現れるシンボル定義を有効とします。lk33はワーニングを出力し、処理を継続します。-wコマンドが指定されている場合は、ワーニングを出力しません。

-dコマンドは、これによって無効となったシンボルの領域を削除します。BSSセクション内は、削除した領域をつめて再配置されます。
-dコマンドはBSSセクションにのみ有効で、他のセクションの無効シンボル領域は削除しません。

12.8 リンクマップファイル

リンクマップファイルは、セクションごとのモジュールの配置情報を参照するためのファイルです。また、シンボルファイルとともに命令エクステンダext33に入力し、コードの最適化を行うことができます。
リンクマップファイルは起動時オプションまたはコマンドファイルで-mが指定された場合に出力されます。
ファイル形式はテキストファイルで、ファイル名は"<ファイル名>.map"(<ファイル名>は出力オブジェクトファイルと同じ名称)です。

●リンクマップファイル例

```
Code Section mapping
Address   Vaddress    Size       File        ID   Attr
00080000  --------    00000028   boot.o      0    REL
00080028  000002c4    0000005c   main.o      0    REL
00080084  00000274    00000050   foo.o       0    REL
000800d4  00000274    00000040   bar.o       0    REL

Data Section mapping
Address   Vaddress    Size       File        ID   Attr
00080114  00000a00    00000000   boot.o      1    REL
00080114  00000a00    00000038   main.o      1    REL
0008014c  00000a38    0000000c   foo.o       1    REL
00080158  00000a44    00000028   bar.o       1    REL

Bss Section mapping
Address   Vaddress    Size       File        ID   Attr
00000000  --------    00000000   boot.o      2    REL
00000000  --------    00000230   main.o      2    REL
00000230  --------    00000024   foo.o       2    REL
00000254  --------    00000020   bar.o       2    REL
```

●リンクマップファイルの内容

Address	セクションの開始アドレスを示します。Vaddressに値を持つセクションは実データが置かれるアドレスを示します。
Vaddress	Uセクションでの開始アドレスを示します。
Size	セクションのサイズを示します。
File	リンクしたモジュールのファイル名を示します。
ID	各オブジェクトファイルにおけるセクションIDを示します。
Attr	セクション属性を示します。 REL: リロケータブルセクションであることを示します。 ABS: アブソリュートセクションであることを示します。

12.9 シンボルファイル

シンボルファイルはすべてのモジュール内で定義されているシンボルとそのアドレス情報を参照するためのファイルです。
また、リンクマップファイルとともに命令エクステンダext33に入力し、コードの最適化を行うことができます。
シンボルファイルは起動時オプションまたはコマンドファイルで-sが指定された場合に出力されます。
ファイル形式はテキストファイルで、ファイル名は"<ファイル名>.sym"(<ファイル名>は出力オブジェクトファイルと同じ名称)です。

●シンボルファイル例

```
Symbol            File      Section   Type      Address

DATA1             comm2.o   data      global    00080006
i                 comm2.o   bss       global    00000100
buffer            main.o    bss       global    00000030
BOOT              main.o    code      global    00080000
main              main.o    code      global    00080100
i                 lk21.o    data      local     00081008
Name              word.o    data      global    0008100c
:                 :         :         :         :
__START_CACHE1    $$lk33$$  uscn      global    00000400
__END_CACHE1      $$lk33$$  uscn      global    000009ff
__SIZEOF_CACHE1   $$lk33$$  uscn      global    000005ff
:                 :         :         :         :
```

●シンボルファイルの内容

Symbol　定義されているすべてのシンボルを処理順に示します。
lk33が作成するセクションシンボルは、ファイルの終わりに出力されます。

File　シンボルが定義されているオブジェクトファイル名を示します。
セクションシンボルは$$lk33$$となります。

Section　セクションの種類を示します。
code:　CODEセクション
data:　DATAセクション
bss:　BSSセクション
uscn:　Uセクション

Type　シンボルの属性を示します。
global:　グローバルシンボル
local:　ローカルシンボル

Address　シンボルに定義された絶対アドレスを示します。

12.10 クロスリファレンスファイル

クロスリファレンスファイルには、すべてのグローバルシンボルの定義アドレスと参照アドレス情報が出力されます。
クロスリファレンスファイルは起動時オプションまたはコマンドファイルで-crefが指定された場合に生成されます。
ファイル形式はテキストファイルで、ファイル名は"<ファイル名>.crf"(<ファイル名>は出力オブジェクトファイルと同じ名称)です。

●クロスリファレンスファイル例

```
Symbol          Address

BOOT
(00C00004)      00C00000 00C00020
DATA1
(00000106)      00C00104 00C00126 00C00148 00C00608 00C00616 00C01060 00C0106e 00C0201c
                00C02100 00C0216e
g_data1
(00000000)      00000718 00000732 0000073c
g_data2
(00000008)                                        …(＊)
main
(00C00004)      0008001e
sub
(00C00024)      00000708 00000710
     :              :
```

●シンボルファイルの内容

Symbol　定義されているすべてのグローバルシンボルがアルファベット順に出力されます。
( )内の16進数はシンボルに定義された絶対アドレスです。

Address　シンボルを参照しているアドレスが16進数で出力されます。
シンボルが参照されていない場合、この項目は空白になります。(＊)

12.11 エラー/ワーニングメッセージ

エラーおよびワーニングメッセージは標準出力(stdout)に表示/出力されます。-eオプションを指定した場合は、ファイル"lk33.err"にも出力されます。
wb33の[LK33]ボタンでlk33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

12.11.1 エラー

lk33で発生するエラーは、システムエラー、コマンドファイルエラー、リンカエラーの3つに分類されます。
エラーが発生した場合、リンカlk33はエラーメッセージを表示して、直ちに処理を終了します。オブジェクトファイルは出力されません。リンクマップファイル、シンボルファイルはエラー発生前までに処理された分が出力されます。

表12.11.1.1 システムエラーメッセージ一覧

エラーメッセージ	内容
<file name>: Error: Cannot open input file.	入力ファイル(*.o、*.lib)がオープンできません。
<file name>: Error: Cannot open output file.	出力ファイルが作成できません。
Error: Cannot open working file.	中間ファイルが作成できません。
Error: Cannot allocate memory.	メモリが確保できません。
Error: Cannot read a file.	ファイルが読み込めません。

表12.11.1.2 コマンドファイルエラーメッセージ一覧

エラーメッセージ	内容
link command: Error: Unknown option near line = #.	リンカコマンドファイル内の#行付近に認識できないリンカコマンドが記述されています。
link command: Error: Invalid parameter near line = #.	リンカコマンドファイル内、#行付近のリンカコマンドのパラメータの記述に誤りがあります。
link command: Error: Not define section name near line = #.	リンカコマンドファイル内の#行付近で指定されているセクション名が定義されていません。
link command: Error: Uninitialized section name near line = #.	リンカコマンドファイル内の#行付近で指定されているセクション名はアドレスが定義されていません。
<file name>: Error: -v3 command is not in the 1st line.	-v3コマンドはコマンドファイルの先頭になければなりません。
<file name>: Error: This command is invalid near line =#.	不正なコマンドが#行付近にあります。
<file name>: Error: Too long symbol length near line =#.	シンボル名、ブロック名の文字長は最大130文字です。
<file name>: Error: Too long file path length near line =#.	オブジェクトファイル名の長さは最大256文字です。
<file name>: Error: Use invalid block name '<symbol>' near line =#.	ブロック名が不正です。ブロック定義にデフォルトのブロック名や、先頭に"@"または"@#"を付けたブロック名は使用できません。
<file name>: Error: There is the file name same in an inside parenthesis near line =#.	{ }内に同じオブジェクトファイル名(フルパス)が複数使用されています。
<file name>: Error: '<symbol>' Not found symbol name near line =#.	シンボル名が見つかりません。シンボル名は使用する前に-defaddrコマンドによる定義が必要です。
<file name>: Error: '<symbol>' is invalid block name near line =#.	ブロック名が不正です。ブロック名が"@"のみ、または"@#"のみのときはエラーとなります。
<file name>: Error: '<symbol>' is not defined near line =#.	ブロックが定義されていません。使用する前に-XXXblock/-uXXXblockコマンドによる定義が必要です。
<file name>: Error: There is the block name same in an inside parenthesis near line =#.	{ }内で指定されたブロック名が、同じ-addr内または他の-addrで重複しています。
<file name>: Error: '<symbol>' already defined near line =#.	ブロックが多重定義されています。{ }内で指定されたブロック名が、同じ-addr内または他の-addrで重複しています。
<file name>: Error: The object file name defined as '<block name>' and '<block name>' overlaps.	同一のオブジェクト定義を含む複数のブロックを、同じ-addr定義内、または他の-addr定義内で指定することはできません。
<file name>: Error: '@' is required for the initial character of '<symbol>'.	-uXXXblockで指定した共用ブロックのアドレスを指定する場合、ブロック名の先頭には"@"が必要です。
<file name>: Error: '<symbol>': There is no definition of an effective object file name.	{ }内に有効なファイルがありません。

表12.11.1.3 リンカエラーメッセージ一覧

エラーメッセージ	内容
<file name>: Error: Code section map out of range.	CODEセクションの配置アドレスがリンク可能範囲外です。-codeコマンドによる指定が0x0〜0xffffffffの範囲を越えています。または、リンク処理中にコードの配置アドレスが上記の範囲を越えました。
<file name>: Error: Data section map out of range.	DATAセクションの配置アドレスがリンク可能範囲外です。-dataコマンドによる指定が0x0〜0xffffffffの範囲を越えています。または、リンク処理中にデータの配置アドレスが上記の範囲を越えました。
<file name>: Error: Bss section map out of range.	BSSセクションの配置アドレスがリンク可能範囲外です。-bssコマンドによる指定が0x0〜0xffffffffの範囲を越えています。または、リンク処理中にデータの配置アドレスが上記の範囲を越えました。
Error: Too many object files.	リンクするオブジェクトファイル数が4000個の制限を越えました。
Error: Too many output sections.	出力セクション数が256個の制限を越えました。
Error: Too many input sections.	デフォルトセクションに配置されるセクション数が4000個の制限を越えました。
Error: Too many library files.	ライブラリファイル数が256個の制限を越えました。
Error: Too many global bss symbol.	BSSセクションのグローバルシンボルが1024個の制限を越えました。
Error: Too many object symbols.	オブジェクトシンボル数が36000個の制限を越えました。
Error: Too many U sections.	Uセクション数が256個の制限を越えました。
Error: Too many section symbols.	セクションシンボル数が256個の制限を越えました。
Error: Too many U section symbols.	Uセクションシンボル数が256個の制限を越えました。
Error: No object files.	リンクするオブジェクトファイルの指定がありません。
<file name>: Error: Not SRF33 Object file or library file.	入力ファイルがsrf33のオブジェクトファイルまたはライブラリファイルではありません。
<file name>: Error: Too many defaddr commands.	アドレスの定義が256個を越えています。
<file name>: Error: Too many output blockes.	ブロック名の定義が256個を越えています。
<file name>: Error: Address out of range near line =#.	指定可能なアドレス範囲は0x0〜0xffffffffです。
<file name>: Error: Address out of range.	ブロックの配置アドレスが0xffffffffを越えています。
<file name>: Error: Too many specified files near line =#.	{ }内の指定ファイル数が4000個を越えています。
<file name>: Error: Too many addr commands.	-addrコマンドの定義が256個を越えています。
<file name>: Error: Too many specified blockes.	{ }内の指定ブロック数が256個を越えています。
Error: Chain information size is greater than file size.	srf33オブジェクトファイルに問題があります。再度Cコンパイラgcc33、プリプロセッサpp33から処理をやり直してください。
Error: Chain seek address is greater than file size.	
Error: Undefined symbol type.	
Error: Undefined relocation type.	

12.11.2 ワーニング

ワーニングの場合、リンカlk33は処理を継続します。他のエラーが発生しなければ、ワーニングメッセージを表示して処理を終了します。オブジェクトファイル、リンクマップファイル、シンボルファイルともに出力されますが、プログラムの動作は保証できません。

表12.11.2.1 ワーニングメッセージ一覧

ワーニングメッセージ	内容
<file name1>: Warning: '<symbol>' already defined in '<file name2>'.	<file name1>で定義している<symbol>はすでに<file name2>で定義済みです。ソースファイル上で修正してください。リンクするファイルの順序により、最初に現れたシンボル定義が有効となります。-wコマンドが指定された場合、BSSセクションで重複しているグローバルラベルに対してはこのワーニングを出力しません。
<file name>: Warning: unresolved external symbol '<symbol>'.	未定義シンボルが参照されています。そのシンボルを参照する命令のリロケーションを行いません。命令中の変更を必要とするビットは0x0として処理されます。
<file name>: Warning: Out of relocation range Address = <address>.	命令がリロケーション(再配置)可能範囲を越えています。リロケーション対象アドレスの計算を行い、命令中に挿入可能なビットのみを変更します。
<file name>: Warning: Section mapping conflict Address = <address>.	セクションが<address>から重複しています。
link command: Warning: Cannot mapping to USection '<file name>(<section>)'.	コマンドファイルに記述してある<file name>の<section>は再配置できません。
link command: Warning: Cannot find relocatable section '<file name>(<section>)'.	<file name>のリロケータブルセクション<section>が見つかりません。
link command: Warning: Alignment <address 1> -> <address 2>.	コマンドファイルで指定されている<address 1>を<address 2>としてアライメントを調整しました。
Warning: Cannot access section information.	セクション情報にアクセスできません。srf33オブジェクトファイルに問題があります。
Warning: Invalid loader information.	-dコマンド使用時に、-ucode、-udata、-ubssが指定されています。あるいは、入力ファイルにld33で処理できないリロケーションタイプが存在します。
<file name>: Warning: -v3 command is already defined.	-v3コマンドが2つ以上あります。
<file name>: Warning: '<symbol>' already address name defined.	同じシンボル名に異なったアドレスを定義しています。最初に定義された方が有効です。
(<file name>: Warning: '<symbol>' already block name defined.	同じブロック名を多重定義しています。最初に定義された方が有効です。
<file name>: Warning: address already defined near line =#.	同じアドレス(またはシンボル)が異なったブロックに定義されています。最初に定義された方が有効です。
<file name>: Warning: Cannot find link file '%s'.	指定されたファイル名がリンクファイルに見つかりません。このファイルは無視されます。
<file name>: Warning: Address mapping conflict.	ブロックの配置が他の-addrのブロックアドレスと重複しています。
<file name>: Warning: Cannot mapping to vertual block'<block name>'('<file name>').	仮想ブロック名(デフォルト以外)の中に"xxx.lib:xxx.o"形式のファイル名があります。

12.12 注意事項

(1) セクションを配置可能なアドレス範囲は0x0～0xffffffffまでです。
この範囲内にすべてのモジュールが配置できれば、リンカではエラーとなりません。ただし、開発する機種によりメモリ容量が制限されますので注意が必要です。

(2) リンク可能なオブジェクトファイル数は最大4000ファイルです。これを越えた場合、エラーとなります。

(3) 出力可能なセクション数およびUセクション数はそれぞれ最大256個です。それ以上の出力セクションが存在するとエラーとなります。

(4) リンク時に指定可能なライブラリファイル数は最大256ファイルです。これを越えた場合、エラーとなります。また、-lコマンドで指定可能なライブラリ検索パスは最大4つです。

(5) セクションシンボルおよびUセクションシンボルはそれぞれ最大256個まで生成可能です。これを越えるとエラーとなります。

13 ディスアセンブラ

この章ではディスアセンブラdis33の機能を説明します。

13.1 機能

ディスアセンブラdis33(以下dis33と記述)はsrf33形式のオブジェクトファイルを入力し、コード部のディスアセンブル内容をソースに対応させてリスト出力します。データ部のダンプ出力も可能です。リンク後あるいはデバッグ後のプログラムを確認するのに有効です。
起動時のオプション指定により、出力内容を選択することができます。

13.2 入出力ファイル

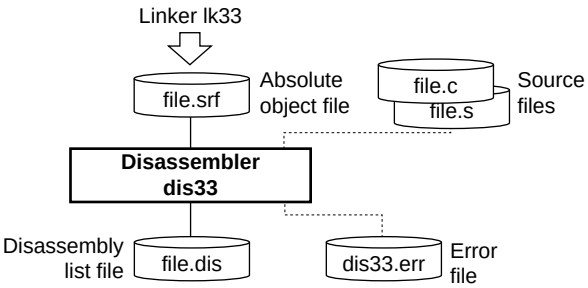


図13.2.1 フローチャート

13.2.1 入力ファイル

●オブジェクトファイル

ファイル形式: srf33形式のバイナリファイル
ファイル名: <ファイル名>.srf または <ファイル名>.o
内容: リンカlk33、アセンブラas33で生成したオブジェクトファイルです。ソースをミックス出力する場合は、デバッグ情報が含まれている必要があります。

●ソースファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.c および <ファイル名>.s
内容: ソースをミックス出力する場合は、上記オブジェクトファイルに含まれるソースファイル名情報に従ってソースファイルも入力されます。

13.2.2 出力ファイル

●ディスアセンブリファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.dis (<ファイル名>は入力ファイルと同じです。)
出力先: カレントディレクトリ
内容: 入力ファイルをディスアセンブルした内容が出力されます。
出力内容については"13.5 ディスアセンブル出力"を参照してください。

●エラーファイル

ファイル形式: テキストファイル
ファイル名: dis33.err
出力先: カレントディレクトリ
内容: 起動時オプション(-e)が指定された場合に出力されるファイルで、エラーメッセージなど、dis33が標準出力(stdout)に対して出力する内容を記録します。

13.3 起動方法

13.3.1 起動フォーマット

●コマンドラインの一般形

dis33 ^ [<起動オプション>] ^ <ファイル名>

^はスペースを示します。
[ ]は省略可能なことを示します。

<ファイル名>: srf33オブジェクトファイルを拡張子も含めて指定します。

●ワークベンチからの起動

起動オプションとオブジェクトファイルを選択し、[DIS33]ボタンをクリックします。

13.3.2 起動オプション

dis33には5種類の起動時オプションが用意されています。

-m
機能: ソースミックス出力
wb33上の指定: [src mix]をチェック
説明:
- コード部の出力時にソースファイルを読み込み、ディスアセンブルしたコードにソースコードを対応させて出力します。
- このオプションを指定する場合は、入力するオブジェクトファイルにデバッグ情報が含まれている必要があります。

-c
機能: コード部のみの出力
wb33上の指定: [code only]をチェック
説明:
- コード部のディスアセンブル結果のみを出力します。データダンプは行いません。

-d
機能: データ部のみの出力
wb33上の指定: [data only]をチェック
説明:
- データ部のダンプ結果のみを出力します。コード部のディスアセンブル出力は行いません。

-a **<開始アドレス> <終了アドレス>**
機能: アドレス範囲の指定
wb33上の指定: [addr range]をチェックし、開始/終了アドレスをテキストボックスに入力
説明:
- ディスアセンブルするアドレス範囲を指定します。
- 指定可能なアドレス範囲は連続する1つの領域のみで、開始および終了アドレスは16進数で指定します。-a、開始アドレス、終了アドレスの間には1個以上のスペースが必要です。
- アドレスに28ビット(0x0fffffff)を越える値を指定した場合は、0x0fffffffが指定されたものとして処理されます。開始アドレスが終了アドレスより大きい場合、16進数以外の指定が行われた場合はエラーとなり、Usageが出力されます。
- 本オプションの指定がない場合は、指定ファイルの全アドレスがディスアセンブルされます。

-e
機能: エラーファイルの出力
wb33上の指定: なし
説明:
- エラーメッセージなど、dis33が標準出力(stdout)に対して出力する内容をファイル(dis33.err)にも出力します。

コマンドラインにオプションを入力する場合、オプションの前後には1個以上のスペースが必要です。
例: c:¥cc33¥dis33 -e -m -a 0x80000 0x80100 test.srf

●-m, -c, -dの組み合わせ

-m、-c、-dオプションの説明はいずれか1つのみを指定した場合の機能で、指定の組み合わせによって機能が以下のように変わります。なお、組み合わせの順序に規定はありません。

オプション指定なし	コード部のディスアセンブル出力とデータダンプ出力を行います。
-c -m	コード部のミックス出力のみを行います。
-d -m	データ部のダンプ出力のみを行います。
-c -d	空のファイルが出力されます。
-c -d -m	空のファイルが出力されます。

13.4 メッセージ

dis33はメッセージをすべて標準出力(stdout)に対して出力します。
wb33の[DIS33]ボタンでdis33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

●終了メッセージ

正常に終了した場合は次の終了メッセージのみを出力します。

```
Disassemble complete
```

●Usage出力

ファイル名が指定されなかったり、オプション指定が正しくない場合、次の使用方法のメッセージを出力して終了します。

```
Disassembler 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x.
Usage:
    dis33 [options] <file name>
Options:
    -e        : produce log file (dis33.err)
    -m        : generate disassemble code with source mix
    -c        : generate disassemble code only
    -d        : generate data dump only
    -a address1 address2
              : specify disassemble area
                address1 - start address, hexadecimal number
                address2 - end address, hexadecimal number
Output:
    Disassemble file (.dis)
    Log file (dis33.err)
Example:
    dis33 -e -m -a 0x80000 0x8ffff sample.srf
```

●エラー・ワーニング発生時

エラーが発生した場合は、終了メッセージの前にエラーメッセージが表示されます。

例: `Error: Can't open file, test.srf.`
`Disassemble complete`

エラーの場合、dis33は出力ファイルを作成せずに終了します。

ワーニングが発生した場合は、終了メッセージの前にワーニングメッセージが表示されます。

例: `Warning: No debug information.`
`Disassemble complete`

ワーニングの場合、dis33は出力ファイルを作成して終了します。

エラーとワーニングの詳細は"13.6 エラー/ワーニングメッセージ"を参照してください。

13.5 ディスアセンブル出力

13.5.1 ミックス出力

ミックス出力(-mオプション)が指定されると、コード出力にソースファイルの内容が付加されます。
dis33は入力したsrf33ファイルのデバッグ情報データからソースファイル名を取得し、読み込みます。したがって、Cコンパイラgcc33、プリプロセッサpp33、アセンブラas33、リンカlk33の処理においてデバッグ情報を付加する-gオプションを指定しなかった場合は、ソース部は出力されません。
-cオプションが指定されていない場合は、コード部の出力後、データ部があれば16進ダンプ出力も行います(詳細は13.5.3項「データ出力」参照)。

●出力形式

```
  ***** Disassemble code and source code *****
Addr   Code  Unassemble        Line       Source
                        -- <ソースファイル名※> --
:    :      :       :        :
(逆アセンブルコード)                    (ソースコード)
:    :      :       :        :
  ***** Data *****
Addr     +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00800000         (データ)
:     : : : : : : : : : : : : : : : :
```

Addr	コード/データのアドレスを16進数で示します。
Code	オブジェクトコードを16進数で示します。
Unassemble	逆アセンブルによるニーモニックコードを示します。
Line	ソースファイルの行番号を10進数で示します。
Source	ソースファイル内のステートメントを示します。

※参照したソースファイル名/インクルードファイル名が、それぞれの開始位置に出力されます。

●出力例

ソースファイル"sample1.s"

```
#define SP_INI 0x0800    ; sp is in end of 2KB RAM
#define GP_INI 0x0000    ; global pointer %r8 is 0x0

    .code
    .word BOOT           ; BOOT VECTOR
BOOT:
    xld.w      %r8,SP_INI
    ld.w       %sp,%r8   ; set SP
    ld.w       %r8,GP_INI ; set global pointer
    xcall      main      ; goto main
    xjp        BOOT      ; infinity loop

#include sample2.s
```

ソースファイル"sample2.s"

```
#include sample3.s
```

ソースファイル"sample3.s"

```
    .data
    .word 1 2
```

ソースファイル"sample4.c"

```
int i;

main()
    {
    int j;

    i = 0;
    for (j=0 ; ; j++)
        {
                i++;
        }
    }
```

ディスアセンブル出力

```
**** Disassemble code and source code ****
  Addr   Code  Unassemble                Line        Source
00C00000 0004 ***
00C00002 0008 ***
                                             --- sample1.s ---
                                        00001  #define SP_INI 0x0800     ; sp is in end of 2KB RAM
                                        00002  #define GP_INI 0x0000     ; global pointer %r8 is 0x0
                                        00003
                                        00004       .code
                                        00005       .word BOOT            ; BOOT VECTOR
                                        00006  BOOT:
00C00004 C020 ext     0x20              00007       xld.w      %r8,SP_INI
00C00006 6C08 ld.w    %r8,0x0
00C00008 A081 ld.w    %sp,%r8           00008       ld.w       %sp,%r8    ; set SP
00C0000A 6C08 ld.w    %r8,0x0           00009       ld.w       %r8,GP_INI ; set global pointer
00C0000C C000 ext     0x0               00010       xcall      main       ; goto main
00C0000E 1C03 call    0x3
00C00010 1EFA jp      0xfa              00011       xjp        BOOT       ; infinity loop
00C00012 0000 nop
                                             --- sample4.c ---
                                        00001  int i;
                                        00002
                                        00003  main()
                                        00004      {
                                        00005       int j;
                                        00006
00C00014 6C0B ld.w    %r11,0x0          00007       i = 0;
00C00016 C004 ext     0x4
00C00018 3C8B ld.w    [%r8],%r11
                                        00008       for (j=0 ; ; j++)
                                        00009           {
00C0001A C004 ext     0x4               00010                 i++;
00C0001C 308A ld.w    %r10,[%r8]
00C0001E 601A add     %r10,0x1
00C00020 C004 ext     0x4
00C00022 3C8A ld.w    [%r8],%r10
00C00024 1EFB jp      0xfb              00008       for (j=0 ; ; j++)
                                        00011           }
00C00026 0640 ret                       00012      }

***** Data *****
  Addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00C00020                          01 00 00 00 02 00 00 00
```

コードに対応するソース行とそれ以前のソースの内容が出力されます。
行番号情報がないソース、実コードのないインクルードファイルは出力されません。
forステートメントのように1行のソースが複数個所のコードに展開されるものは、そのソース行が複数個所に現れることがあります。
ソースファイルがない場合はワーニングメッセージを出力し、Source欄には"no source"を出力します。

13.5.2 コード出力

ミックス出力(-mオプション)が指定されていない場合、入力したsrf33ファイルのコード部はアドレス順にディスアセンブルされ、以下の形式で出力されます。

●出力形式

```
***** Disassemble code *****
Addr Code  Unassemble
:   :    :
(逆アセンブルコード)
:   :    :
```

Addr	コードのアドレスを16進数で示します。
Code	オブジェクトコードを16進数で示します。
Unassemble	逆アセンブルによるニーモニックコードを示します。

●出力例

ソースファイル

```
.abs
    .code
    .org 0xc00000
    .word BOOT

    .org 0xc00080
    xjp        BOOT

    .org 0xc00100
BOOT:
    xld.w      %r8,0x800
    ld.w       %sp,%r8
    ld.w       %r8,0x0
STNDBY:
    halt
    jp         STANDBY
```

ディスアセンブル出力

```
 ***** Disassemble code *****
   Addr   Code  Unassemble
 00C00000 0100 ***
 00C00002 0008 ***

 00C00080 1E40 jp      0x40

 00C00100 C020 ext     0x20
 00C00102 6C08 ld.w    %r8,0x0
 00C00104 A081 ld.w    %sp,%r8
 00C00106 6C08 ld.w    %r8,0x0
 00C00108 0080 halt
 00C0010A 1E00 jp      0x0
```

アドレスが離れている箇所は改行します。

13.5.3 データ出力

データ部は、そのサイズ分がアドレス順に16進数でダンプ出力されます。入力ファイルにデータがない場合はデータ出力は行いません。-dオプションのみを指定し、データのないファイルを入力した場合は、空のファイルが出力されます。

●出力形式

```
  ***** Data *****
Addr      +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00800000          (データ)
:     : : : : : : : : : : : : : : : :
```

Addr　データのアドレスを16進数で示します。その行(16アドレス分)の開始アドレスです。

+0～+F　16アドレス分のデータを16進数で示します。データが定義されていないアドレスは空白となります。

●出力例

ソースファイル(データ定義部)

```
    .data
    .org      0x8008
CHAR1:
    .half     0x9
    .half     0xa
    .half     0xb

    .org      0x8104
CHAR2:
    .word     0x3
    .word     0x4
    .word     0x5
```

ディスアセンブル出力

```
***** Data *****
   Addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
 00008000                          09 00 0A 00 0B 00

 00008100              03 00 00 00 04 00 00 00 05 00 00 00
```

アドレスが離れている箇所は改行します。また、データのないアドレスにはスペースが入ります。

13.6 エラー/ワーニングメッセージ

エラーおよびワーニングメッセージは標準出力(stdout)に表示/出力されます。-eオプションを指定した場合は、ファイル"dis33.err"にも出力されます。
wb33の[DIS33]ボタンでdis33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

13.6.1 エラー

dis33で発生するエラーは、システムエラー、ディスアセンブラエラーの2つに分類されます。
エラーが発生した場合、dis33はエラーメッセージを表示して、直ちに処理を終了します。ディスアセンブリファイルは出力されません。

表13.6.1.1 システムエラーメッセージ一覧

エラーメッセージ	内容
Error: Cannot open file, <file name>.	ファイルがオープンできません。
Error: Cannot close file, <file name>.	ファイルがクローズできません。
Error: Cannot write to file, <file name>.	ファイルへの書き込みができません。
Error: Cannot close SROFF file.	オブジェクトファイルがクローズできません。
Error: Cannot load data, memory allocation failure.	メモリを確保できないため、セクション情報がロードできません。
Error: Cannot load debug information, memory allocation failure.	メモリを確保できないため、デバッグ情報がロードできません。
Error: Cannot load data, file read failure, <file name>.	ファイルが読み込めないため、セクション情報をロードできません。
Error: Cannot load debug information, file read failure, <file name>.	ファイルが読み込めないため、デバッグ情報がロードできません。

表13.6.1.2 ディスアセンブラエラーメッセージ一覧

エラーメッセージ	内容
Error: Invalid file name <file name>.	入力ファイル名に".dis"が指定されました。
Error: Too many sections. Error: Cannot load data, please check SROFF file. Error: Cannot load debug information, please check SROFF file.	これらのエラーは、入力した srf33 オブジェクトファイル中の情報に誤りがある場合に発生します。ソースからリンクまでのファイルに整合性があるか確認し、最終的なソースファイルを使用して C コンパイラ、プリプロセッサから処理をやり直してください。

13.6.2 ワーニング

ワーニングの場合、dis33は処理を継続します。他のエラーが発生しなければ、ワーニングメッセージを表示して処理を終了します。ディスアセンブリファイルも出力されます。

表13.6.2.1 ワーニングメッセージ一覧

ワーニングメッセージ	内容
Warning: No debug information.	入力ファイルにデバッグ情報がありません。このワーニングは-m オプションを指定してミックス出力を行った場合のみ発生します。デバッグ情報がないため、ソースの出力が行えません。逆アセンブル出力のみを行います。
Warning: Cannot open file, <file name>.	ミックス出力時にソースファイルがオープンできません。ソース出力部を "no source"として処理します。
Warning: Line number of source file is invalid.	ソースファイルが更新され、行数が足りません。

13.7 注意事項

dis33でソースミックス出力を行うためには、以下の点に注意してください

(1) ソースを記述する際には、できるだけタブを8文字間隔に設定してください。それ以外のタブ設定では、出力位置がずれる場合があります。

(2) dis33は、入力したsrf33ファイルのデバッグ情報データからソースファイル名を取得し、読み込みます。したがって、Cコンパイラgcc33、プリプロセッサpp33、アセンブラas33、リンカlk33の処理において、デバッグ情報を付加する-gオプションを指定して生成したオブジェクトファイルを入力してください。
ソース表示用デバッグ情報のあるオブジェクトと、ないオブジェクトがリンクされている場合、デバッグ情報のあるファイルのみソースミックス出力を行います。

(3) 入力するオブジェクトファイルに対応したソースファイルの整合性に注意してください。オブジェクトファイル生成後にソースファイルが変更されていると、コードとソースが正しく対応した出力を得ることはできません。あるいはエラーとなり、出力することができません。

14 バイナリ/HEXコンバータ

この章ではバイナリ/HEXコンバータhex33の機能を説明します。

14.1 機能

バイナリ/HEXコンバータhex33(以下hex33と記述)はsrf33形式のオブジェクトファイルを入力し、指定したアドレス範囲をモトローラS3形式のデータに変換してファイルを生成します。指定したアドレス範囲内のデータが存在しない領域には0xFFを埋め込みます。変換後のアドレスに絶対アドレスまたは指定した開始アドレスからのオフセットアドレスを指定可能です。
hex33は以下の場合に使用します。

- 完成したsrf33オブジェクトファイルから内蔵ROMデータを取り出して、マスクデータファイルを生成する場合
- ターゲットボードや製品の外部ROMに書き込むデータを作成する場合
- 完成したプログラムをデバッガで最終的に検収する場合

14.2 入出力ファイル

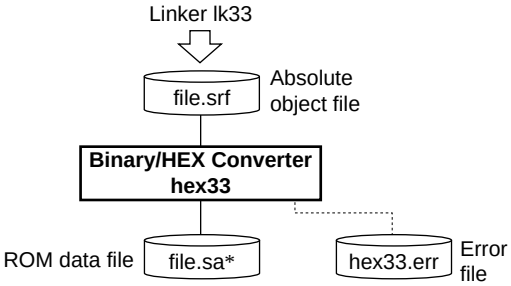


図14.2.1 フローチャート

14.2.1 入力ファイル

●オブジェクトファイル

ファイル形式: srf33形式のバイナリファイル
ファイル名: <ファイル名>.srf
内容: リンカlk33で生成したオブジェクトファイルです。

14.2.2 出力ファイル

●HEXファイル

ファイル形式: モトローラS3形式のHEXファイル
ファイル名: <ファイル名>.sa* (<ファイル名>は入力ファイルと同じです。)
出力先: カレントディレクトリ
内容: 入力ファイルの指定アドレス範囲がモトローラS形式に変換されて出力されます。

●エラーファイル

ファイル形式: テキストファイル
ファイル名: hex33.err
出力先: カレントディレクトリ
内容: 起動時オプション(-e)が指定された場合に出力されるファイルで、エラーメッセージなど、hex33が標準出力(stdout)に対して出力する内容を記録します。

14.3 起動方法

14.3.1 起動フォーマット

●コマンドラインの一般形

hex33 ^ [<起動オプション>] ^ <開始アドレス> ^ <終了アドレス> ^ <ファイル名>

^はスペースを示します。
[ ]は省略可能なことを示します。

<開始アドレス>: 変換範囲の開始アドレスを16進数で指定します。
<終了アドレス>: 変換範囲の終了アドレスを16進数で指定します。
<ファイル名>: 変換するsrf33オブジェクトファイルを拡張子も含めて指定します。

※<開始アドレス>と<終了アドレス>には32バイト単位の境界アドレスを指定してください。
指定する数値は"0x"の有無にかかわらず16進数として扱われます。

●ワークベンチからの起動

起動オプション、変換範囲およびオブジェクトファイルを指定し、[HEX33]ボタンをクリックします。

14.3.2 起動オプション

hex33には4種類の起動時オプションが用意されています。

-x
機能: 指定アドレスを拡張子に追加
wb33上の指定: [addr to name]をチェック
説明: ・ 生成するファイルの拡張子に指定した開始アドレスと終了アドレスを付加します。
例: test.sa_c00000_c0ffff
どの部分のデータであるか一目で分かり便利です。

-z
機能: 絶対アドレス出力
wb33上の指定: [abs addr]をチェック
説明: ・ 変換後のアドレス部分を絶対アドレスとして出力ファイルを生成します。
このオプションを指定しない場合は、指定した開始アドレスを0番地とするオフセットアドレスに変換されます。
注意: ・ マスクデータファイルを作成する場合は、このオプションを必ず指定してください。

-r
機能: セクションのチェック
wb33上の指定: [abs addr]をチェック
説明: ・ すべての変換セクションが指定アドレス範囲に収まっているかチェックし、範囲を越えるセクションがあればエラーとします。

-e
機能: エラーファイルの出力
wb33上の指定: なし
説明: ・ エラーメッセージなど、hex33が標準出力(stdout)に対して出力する内容をファイル(hex33.err)にも出力します。

コマンドラインにオプションを入力する場合、オプションの前後には1個以上のスペースが必要です。
例: c:¥cc33¥hex33 -e -x -z test.srf

14.4 メッセージ

hex33はメッセージをすべて標準出力(stdout)に対して出力します。
wb33の[HEX33]ボタンでhex33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

●終了メッセージ

正常に終了した場合は次の終了メッセージのみを出力します。

```
Conversion Completed
```

●Usage出力

ファイル名が指定されなかったり、オプション指定が正しくない場合、次の使用方法のメッセージを出力して終了します。

```
HEX Data Converter 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    hex33 [options] <start address> <end address> <file name>
Options:
    -e : produce log file (hex33.err)
    -x : add start and end addresses to the file name extension
    -z : make converted address absolute
    -r : check all data within start and end area
Output:
    hex file  (.sa, .sa_<start address>_<end address>)
    log file  (hex33.err)
Example:
    hex33 -x -z 80000 80fff sample.srf
```

●エラー・ワーニング発生時

エラーが発生した場合は、終了メッセージの前にエラーメッセージが表示されます。
例: `Error: Input file is not SRF33 file.`
`Conversion Completed`

エラーの場合、hex33は出力ファイルを作成せずに終了します。

ワーニングが発生した場合は、終了メッセージの前にワーニングメッセージが表示されます。
例: `Warning: Section information chain is not found.`
`Conversion Completed`

ワーニングの場合、hex33は出力ファイルを作成して終了します。

エラーとワーニングの詳細は"14.6 エラー/ワーニングメッセージ"を参照してください。

14.5 HEXファイルの内容

14.5.1 モトローラS3形式

hex33はsrf33オブジェクトファイルを、32ビットアドレスが使用可能なモトローラS3形式に変換します。出力ファイルの各行は、次の形式で生成されます。

S3	length(1)	address(4)	data(1)	-----	data(1)	sum(1)	¥n

:

S7	length(1)	00000000	sum(1)	¥n

()内はバイト数です。

S3: その行がデータレコードであることを示します。
S7: その行がエンドレコードであること(データの終了)を示します。
length: address+data+sumのレコード長を示します。
hex33が出力するデータレコードは常に0x25、エンドレコードは0x05となります。
address: レコード内の先頭データが置かれるアドレスを示します。
data: 32バイト分のデータです。エンドレコードにはありません。
sum: lengthから最終dataまでのチェックサムです(1の補数)。
¥n: 改行コードです。

エンドレコードは常にS70500000000FAとなります。

14.5.2 絶対アドレス出力

-zオプションを指定するとaddress部分には絶対アドレスが入ります。

出力ファイルのダンプ例

```
S32500080000400080020C0086C81A0086C00C000C0021CF91E00020B6C00C000C08B3C006CFC
S325000800200C2E051C1060FD1E400240061C70081800C000C08A301A6000C000C08A3C400658
S32500080040FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFB2
S32500080060FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF92
                                        :
S32500080FA0FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF43
S32500080FC0FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF23
S32500080FE0FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF03
S70500000000FA
```

これは開始アドレスに0x80000、終了アドレスに0x80FFF、-zオプションを指定して作成したファイルの例です。
アドレス部分が00080000から00080FE0まで、32アドレスずつのデータレコードが生成されます。
データのない領域には0xFFが埋め込まれます。

14.5.3 オフセットアドレス出力

-zオプションを省略すると、address部分には指定した開始アドレスからのオフセットアドレスが入ります。

出力ファイルのダンプ例

```
S32500000000400080020C0086C81A0086C00C000C0021CF91E00020B6C00C000C08B3C006C04
S325000000200C2E051C1060FD1E400240061C70081800C000C08A301A6000C000C08A3C400660
S32500000040FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFBA
S32500000060FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF9A
                                        :
S32500000FA0FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF4B
S32500000FC0FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF2B
S32500000FE0FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF0B
S70500000000FA
```

前記の例を-zオプションを指定せずに変換したものです。アドレス部分が0x80000番地からのオフセットになります。

14.6 エラー/ワーニングメッセージ

エラーおよびワーニングメッセージは標準出力(stdout)に表示/出力されます。-eオプションを指定した場合は、ファイル"hex33.err"にも出力されます。
wb33の[HEX33]ボタンでhex33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

14.6.1 エラー

hex33で発生するエラーは、システムエラー、入力ファイルフォーマットエラーの2つに分類されます。エラーが発生した場合、hex33はエラーメッセージを表示して、直ちに処理を終了します。HEXファイルは出力されません。

表14.6.1.1 システムエラーメッセージ一覧

エラーメッセージ	内容
Error: File open error.	ファイルがオープンできません。
Error: File write error.	ファイルへの書き込みができません。
Error: File read error.	ファイルの読み込みができません。
Error: Memory allocation error.	メモリが確保できません。

表14.6.1.2 入力ファイルフォーマットエラーメッセージ一覧

エラーメッセージ	内容
Error: "<file name>" file could not be opened.	ファイル<file name>を開くことができません。
Error: Input file is not SRF33 file.	入力ファイルが srf33 形式のオブジェクトファイルではありません。
Error: Start address error.	変換開始アドレスが不正です。0x00000000〜0x0fffffff の範囲外のアドレス、または 32 バイト境界以外のアドレスが指定されました。
Error: End address error.	変換終了アドレスが不正です。0x00000000〜0x0fffffff の範囲外のアドレス、または 32 バイト境界以外のアドレスが指定されました。
Error: Out of area in address <address>.	変換データが指定アドレス範囲を越えました。
Error: Chain information size is greater than file size. Error: Chain seek address is greater than file size. Error: File control flag error. Error: Section address error. Error: Section ID error. Error: Data conflicted at <address>.	これらのエラーは、入力した srf33 オブジェクトファイル中の情報に誤りがある場合に発生します。ソースからリンクまでのファイルに整合性があるか確認し、最終的なソースファイルを使用して C コンパイラ、プリプロセッサから処理をやり直してください。

14.6.2 ワーニング

ワーニングの場合、hex33は処理を継続します。他のエラーが発生しなければ、ワーニングメッセージを表示して処理を終了します。HEXファイルも出力されます。

表14.6.2.1 ワーニングメッセージ一覧

ワーニングメッセージ	内容
Warning: Section information chain is not found.	入力ファイルにセクション情報チェーンがありません。このワーニングは入力した srf33 オブジェクトファイル中の情報に誤りがある場合に発生します。ソースからリンクまでのファイルに整合性があるか確認し、最終的なソースファイルを使用して C コンパイラ、プリプロセッサから処理をやり直してください。

14.7 注意事項

(1) 変換開始アドレスと終了アドレスには32バイトの境界アドレスを16進数で指定してください。

(2) マスクデータ作成用に内蔵ROM領域を変換する場合、変換開始アドレスと変換終了アドレスには開発機種の内蔵ROMの開始/終了アドレスを指定し、絶対アドレスの出力(-zオプションを指定)を行ってください。プログラムサイズが小さい場合でも、内蔵ROMの終了アドレスまでのHEXファイルを作成する必要があります。

15 ライブラリアン

この章ではライブラリアンlib33の機能を説明します。

15.1 機能

ライブラリアンlib33(以下lib33と記述)はsrf33形式ライブラリファイルの編集ツールです。
アセンブラas33が出力するリロケータブルオブジェクトファイルでライブラリを作成することができます。汎用モジュールをライブラリ化しておくことにより、将来のS1C33 Familyを使用する製品の開発時間、コストを削減できます。
lib33は以下の機能を持っています。

・ アセンブラas33が出力するリロケータブルオブジェクトファイルを既存のライブラリファイルに追加
・ アセンブラas33が出力するリロケータブルオブジェクトファイルで新規ライブラリファイルを作成
・ ライブラリファイル内のモジュール一覧の出力
・ ライブラリファイルから指定モジュールの削除
・ ライブラリファイル内の指定モジュールをリロケータブルオブジェクトファイルに復元

作成・編集したライブラリファイルはリンク時に指定しておくことで、その中の必要なモジュールのみが他のオブジェクトファイルとリンクされます。

15.2 入出力ファイル

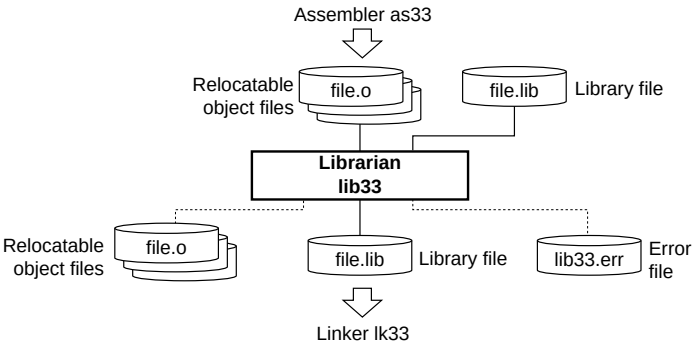


図15.2.1 **フローチャート**

15.2.1 入力ファイル

●ライブラリファイル(新規作成時を除く)

ファイル形式: srf33ライブラリ形式のバイナリファイル
ファイル名: <ファイル名>.lib
内容: パッケージに含まれるライブラリファイル、およびlib33で作成したライブラリファイルです。

●リロケータブルオブジェクトファイル(ライブラリ作成時、ライブラリへの追加時)

ファイル形式: srf33形式のバイナリファイル
ファイル名: <ファイル名>.o
内容: アセンブラas33で作成したリロケータブルオブジェクトファイルです。

15.2.2 出力ファイル

●ライブラリファイル(新規作成時、モジュールの追加/削除時)

ファイル形式: srf33ライブラリ形式のバイナリファイル
ファイル名: <ファイル名>.lib
出力先: カレントディレクトリ
内容: 複数のリロケータブルオブジェクトモジュールで構成されます。
ライブラリファイルの内容については、Appendixの"srf33ファイルの構造"を参照してください。

●リロケータブルオブジェクトファイル(モジュールの復元時)

ファイル形式: srf33形式のバイナリファイル
ファイル名: <ファイル名>.o
内容: 指定のモジュールが元のリロケータブルオブジェクトファイルに復元されます。

●エラーファイル

ファイル形式: テキストファイル
ファイル名: lib33.err
出力先: カレントディレクトリ
内容: 起動時オプション(-e)が指定された場合に出力されるファイルで、モジュール一覧やエラーメッセージなど、lib33が標準出力(stdout)に対して出力する内容を記録します。

15.3 起動方法

15.3.1 起動フォーマット

●コマンドラインの一般形

lib33 ^ [<起動オプション>] ^ <ライブラリファイル名> ^ [<オブジェクトファイル名>]

^はスペースを示します。
[ ]は省略可能なことを示します。

<ライブラリファイル名>: 編集するライブラリファイルを拡張子も含めて指定します。
<オブジェクトファイル名>: 登録、削除、復元するファイルを拡張子も含めて指定します。複数のオブジェクトファイルが指定可能です。

●ワークベンチからの起動

起動オプションおよび入力ファイルを指定し、[LIB33]ボタンをクリックします。

15.3.2 起動オプション

lib33には5種類の起動時オプションが用意されています。
実際の使用方法は後述します。

-a
機能: ライブラリへオブジェクトファイルを追加
wb33上の指定: [add]をチェック(追加)
説明:
・ 既存のライブラリが指定された場合は、指定のオブジェクトファイルをライブラリに追加します。
・ 新規のライブラリファイル名が指定された場合は、指定のオブジェクトファイルでライブラリファイルを作成します。

-d
機能: ライブラリからオブジェクトファイルを削除
wb33上の指定: [del]をチェック
説明:
・ 指定のライブラリから指定のオブジェクトファイルを削除します。

-l
機能: ライブラリに登録されているオブジェクトファイルを表示
wb33上の指定: [list]をチェック
説明:
・ 指定のライブラリ内のオブジェクトファイルを格納順に表示します。
・ 表示は標準出力(stdout)に対して行われます。

-x
機能: ライブラリモジュールをオブジェクトファイルに復元
wb33上の指定: [extract]をチェック(指定モジュールのみを復元)、または[extract all](全モジュールを復元)

説明:
- オブジェクトファイル名を指定した場合は、ライブラリ内の指定のモジュールのみをオブジェクトファイルに復元します。
- オブジェクトファイル名が指定されない場合は、ライブラリ内の全モジュールをオブジェクトファイルに復元します。

-a、-d、-l、-xオプションは複数を同時に指定することはできません。この中のオプションが1つも指定されなかった場合は、指定のオブジェクトファイルで新規ライブラリを作成します(wb33上では[new]をチェック。)

-e
機能: エラーファイルの出力
wb33上の指定: なし
説明:
- ライブラリモジュールのリストやエラーメッセージなど、lib33が標準出力(stdout)に対して出力する内容をファイル(lib33.err)にも出力します。

コマンドラインにオプションを入力する場合、オプションの前後には1個以上のスペースが必要です。
例: c:¥cc33¥lib33 -e -a sample.lib test1.o test2.o

15.4 メッセージ

lib33はメッセージをすべて標準出力(stdout)に対して出力します。
wb33の[LIB33]ボタンでlib33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

●終了メッセージ

正常に終了した場合は次の終了メッセージのみを出力します。

```
Librarian Completed
```

●Usage出力

ファイル名が指定されなかったり、オプション指定が正しくない場合、次の使用方法のメッセージを出力して終了します。

```
Librarian 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    lib33 [option] <library-file> <files>
Options:
    -e : produce log file (lk33.err)
    -a : add new files after an existing member of the library
    -d : delete members from the library
    -l : display a table listing the contents of library
    -x : extract members from the library
Output:
    EOC33 library file (.lib)
Example:
    lib33 -a libc.lib putc.o getc.o
```

●エラー・ワーニング発生時

エラーが発生した場合は、終了メッセージの前にエラー/ワーニングメッセージが表示されます。
例:
```
test.lib: Error: Cannot open file
Librarian Completed
```

エラーの場合、lib33は出力ファイルを作成せずに終了します。
ワーニングの場合、lib33は出力ファイルを作成して終了します。
エラーの詳細は"15.6 エラー/ワーニングメッセージ"を参照してください。

15.5 ライブラリの編集機能

15.5.1 新規作成

ライブラリを新規作成する場合は、-e以外のオプションを指定せずにlib33を実行します。

lib33 <ライブラリファイル名> <オブジェクトファイル名> ... <オブジェクトファイル名>

例: c:¥cc33¥lib33 test.lib test1.o test2.o test3.o

指定した名称でライブラリファイルが作成され、オブジェクトファイルが指定順に格納されます。

wb33で実行する場合、[other options]ウィンドウの[new]ボタンをチェックし、ライブラリファイル名はその下にあるテキストボックスに入力してください(拡張子".lib"は不要)。登録するオブジェクトファイルは[other options]ウィンドウのファイルリストボックスから選択してください。

注:
- **ライブラリに登録できるのは、srf33形式のリロケータブルオブジェクトファイル(*.o)に限られます。アブソリュートオブジェクトファイルは登録することができません。**
- **既存のライブラリ名を指定すると、新しく指定したオブジェクトファイルの構成で上書きされます。**
- **1つのライブラリファイルに登録可能なオブジェクト数は最大4000個です。これを越えた場合、エラーとなります。**

15.5.2 モジュールの追加

既存のライブラリにオブジェクトファイルを追加する場合は次の起動コマンドでlib33を実行します。

lib33 -a <ライブラリファイル名> <オブジェクトファイル名> ... <オブジェクトファイル名>

例: c:¥cc33¥lib33 -a test.lib test1.o test2.o test3.o

-aオプションと既存のライブラリファイル名を指定すると、そのライブラリに指定のオブジェクトファイルを追加することができます。オブジェクトファイルは指定した順序でライブラリファイルの最後に登録されます。

wb33で実行する場合、ライブラリファイル名は実行ウィンドウのファイルリストボックスから、追加するオブジェクトファイル名は[other options]ウィンドウのファイルリストボックスから選択してください。オブジェクトファイルはディレクトリが指定できませんので、ライブラリファイルと同じディレクトリに用意しておく必要があります。[add]ボタンをチェックして実行すると、選択したオブジェクトファイルがライブラリに登録されます。

注:
- **存在しないライブラリ名を指定すると、そのライブラリ名で新規ライブラリを作成します。**
- **ライブラリに登録できるのは、srf33形式のリロケータブルオブジェクトファイル(*.o)に限られます。アブソリュートオブジェクトファイルは登録することができません。**
- **すでに登録されているモジュールと同じ名称のオブジェクトファイルを指定すると、ワーニングとなりますが、ライブラリの最後に同じ名称で追加されます。その後、削除でそのオブジェクトファイル名を指定すると、モジュールが両方とも削除されます。**
- **1つのライブラリファイルに登録可能なオブジェクト数は最大4000個です。これを越えた場合、エラーとなります。**

15.5.3 登録モジュールの一覧表示

ライブラリに登録されたオブジェクトファイルの一覧を参照するには、次の起動コマンドでlib33を実行します。

lib33 -l <ライブラリファイル名>

```
例:  c:¥cc33¥lib33 -l string.lib
     The files list in string.lib
     strerror.o
     strcat.o
     strchr.o
       :
     memset.o
     strtok.o
     Librarian Completed
```

指定したライブラリに含まれるすべてのオブジェクトファイル名を格納順に表示します。
一覧は標準出力(stdout)に表示/出力されます。-eオプションを指定した場合は、ファイル"lib33.err"にも出力されます。

wb33の場合、<ライブラリファイル名>を実行ウィンドウのファイルリストボックスから選択し、[list]ボタンをチェックして実行してください。

15.5.4 モジュールの削除

不要となったライブラリモジュールはライブラリファイルから削除することができます。

lib33 -d <ライブラリファイル名> <オブジェクトファイル名> ... <オブジェクトファイル名>

例: c:¥cc33¥lib33 -d test.lib test1.o test2.o

指定ライブラリ内に同じ名称のオブジェクトファイルが複数登録されている場合は、すべてが削除されます。

wb33で実行する場合、<ライブラリファイル名>は実行ウィンドウのファイルリストボックスから選択し、削除する<オブジェクトファイル名>は[other options]ウィンドウの[del]ボタンの下のテキストボックスに入力してください(拡張子".o"は不要)。このテキストボックスでは1つのオブジェクトファイル名のみ指定可能です。複数のオブジェクトファイルを削除する場合は、複数回lib33を実行するか、コマンドラインへの追加入力で対応してください。
[del]ボタンをチェックして実行すると、指定したオブジェクトファイルがライブラリから削除されます。

15.5.5 オブジェクトファイルの復元

ライブラリ内に登録されたモジュールは、元のsrf33形式リロケータブルオブジェクトファイルに復元することができます。
オブジェクトファイルへの復元は次の起動コマンドで行います。

lib33 -x <ライブラリファイル名> <オブジェクトファイル名> ... <オブジェクトファイル名>

例: c:¥cc33¥lib33 -x test.lib test1.o test2.o

<オブジェクトファイル名>を省略して実行すると、ライブラリ内のすべてのモジュールがオブジェクトファイルに復元されます。
モジュールを復元しても、ライブラリファイルの内容は変更されません。
ライブラリファイル内に指定の名称のモジュールが複数登録されている場合は、登録順で最後のモジュールが復元されます。

wb33で実行する場合、<ライブラリファイル名>は実行ウィンドウのファイルリストボックスから選択し、復元する<オブジェクトファイル名>は[other options]ウィンドウの[extract all]ボタンの右側にあるテキストボックスに入力してください(拡張子".o"は不要)。このテキストボックスでは1つのオブジェクトファイル名のみ指定可能です。複数のオブジェクトファイルを復元する場合は、複数回lib33を実行するか、コマンドラインへの追加入力で対応してください。
[extract]ボタンをチェックして実行すると、指定したオブジェクトファイルが復元されます。
すべてのモジュールを復元する場合は、[extract all]ボタンをチェックして実行してください。

15.6 エラー/ワーニングメッセージ

エラー/ワーニングメッセージは標準出力(stdout)に表示/出力されます。-eオプションを指定した場合は、ファイル"lib33.err"にも出力されます。
wb33の[LIB33]ボタンでlib33を起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

15.6.1 エラー

lib33でエラーが発生した場合、エラーメッセージを表示して直ちに処理を終了します。出力ファイルは生成されません。

表15.6.1.1 エラーメッセージ一覧

エラーメッセージ	内容
<file name>: Error: Cannot open file	ファイルがオープンできません。
<file name>: Error: Not srf33 library file	指定したライブラリファイルがsrf33ライブラリ形式ではありません。
Error: Max object files	ライブラリ内のオブジェクト数が上限を越えました。1つのライブラリファイルに登録可能なオブジェクト数は最大4000個です。

15.6.2 ワーニング

ワーニングの場合、lib33は処理を継続します。他のエラーが発生しなければ、ワーニングメッセージを表示して処理を終了します。出力ファイルも生成されますが、結果は保証されません。

表15.6.2.1 ワーニングメッセージ一覧

ワーニングメッセージ	内容
<file name>: Warning: Not srf33 object file	指定オブジェクトがsrf33形式のリロケータブルオブジェクトファイルではありません。
Warning: Multiple object file '<file name>'	指定したオブジェクトファイルはすでに登録されています。この場合でも、指定のオブジェクトファイルはライブラリの最後に追加されます。
Warning: Cannot find '<file name>' in library	削除または復元で指定したオブジェクトファイルがライブラリ内に見つかりません。

15.7 注意事項

(1) ライブラリに登録できるのは、srf33形式のリロケータブルオブジェクトファイル(*.o)に限られます。アブソリュートオブジェクトファイルは登録することができません。また、1つのライブラリに登録可能なオブジェクト数は最大4000個です。

(2) すでに登録されているモジュールと同じ名称のオブジェクトファイルを指定してライブラリへの追加を行うとワーニングになりますが、ライブラリの最後に同じ名称で追加されます。その後、削除でそのオブジェクトファイル名を指定すると、モジュールが両方とも削除されますので注意してください。

16 デバッガ

この章ではデバッガdb33の操作方法を説明します。

16.1 特長

デバッガdb33(以下db33と記述)はリンカによって生成したsrf33形式のオブジェクトファイルまたはモトローラS3形式のROMデータファイルを読み込み、プログラムのデバッグを行うためのソフトウェアです。以下の特長と機能を持ちます。

- マルチウィンドウにより各種のデータを一度に参照可能
- 使用頻度の高いコマンドはツールバーおよびメニューからマウス操作で実行可能
- S5U1C33104H、S5U1C33000H、デバッグモニタを使用したデバッグのほかに、パソコン上でデバッグ可能なソフトウェアシミュレータ機能を搭載
- Cソースコードおよびアセンブリソースコードに対応したソースレベルデバッグ機能
- プログラムの連続実行とCソースレベルに対応した2種類のステップ実行が可能
- ハードウェア、ソフトウェアブレーク機能およびメモリアクセス条件が指定可能なデータブレーク機能
- フラグや監視アドレスのオンザフライによるリアルタイム表示機能
- 時間とサイクルによる実行時間の表示機能
- データ検索、保存が可能なトレース機能
- コマンドファイルによるコマンドの自動実行機能
- デバッガ上で入出力の評価が可能なシミュレーテッドI/O機能

16.2 入出力ファイル

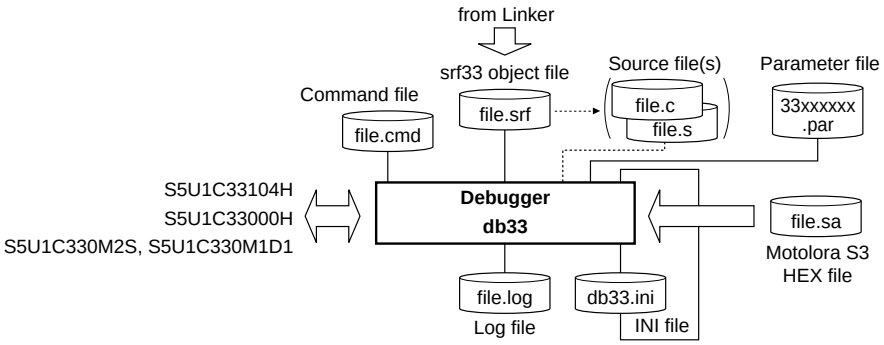


図16.2.1 **フローチャート**

16.2.1 入力ファイル

●パラメータファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.par
内容: 各機種のメモリ情報などが記録された、デバッガを起動するために必須のファイルです。 ワークベンチwb33の[Par gen]ボタンによって基本的な内容のパラメータファイルを作成することができます。
詳細については、"16.11 パラメータファイルについて"を参照してください。

●INIファイル

ファイル形式: テキストファイル
ファイル名: db33.ini
内容: デバッガの終了時にmdコマンドの設定内容、ウィンドウ表示情報が保存されます。また、内容を一部書き換えることにより、ファンクションキーの定義を変更することができます。このファイルはデバッガ起動時に読み込まれ、デバッガを初期設定します。これらの情報を出荷時の状態に初期化したい場合は、このファイルを削除します。詳細については、"16.12 INIファイル"を参照してください。

以下のファイルはすべてコマンドの指定によってデバッガが読み込みます。

●オブジェクトファイル

ファイル形式: srf33形式のバイナリファイル
ファイル名: <ファイル名>.srf
内容: リンカlk33で生成したアブソリュートオブジェクトファイルです。lfコマンドにより読み込みます。デバッグ情報を含んだsrf33形式のファイルを読み込むことで、ソース表示およびシンボリックデバッグが行えます。

●ROMデータHEXファイル

ファイル形式: モトローラS3形式のHEXファイル
ファイル名: <ファイル名>.sa
内容: バイナリ/HEXコンバータhex33で生成したROMデータファイルです。lhコマンドにより読み込みます。デバッグ情報を含まないため、ソース表示およびシンボリックデバッグは行えません。最終プログラムデータの動作確認を行う場合などに読み込みます。

●ソースファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.c(Cソース)、<ファイル名>.s(アセンブリソース)
内容: 上記オブジェクトファイルのソースファイルで、ソース表示を行うときに読み込まれます。

●コマンドファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.cmd
内容: 連続して実行させるデバッグコマンドを記述したファイルです。頻繁に使用する一連のコマンドを書き込んでおくことで、キーボードからのコマンド入力の手間を省くことができます。このファイルはcomまたはcmwコマンドにより読み込み、実行させます。

16.2.2 出力ファイル

●ログファイル

ファイル形式: テキストファイル
ファイル名: <ファイル名>.log
内容: 実行したコマンドと実行結果が出力されます。このファイル出力はlogコマンドによって制御できます。

●INIファイル

ファイル形式: テキストファイル
ファイル名: db33.ini
内容: デバッガの終了時にmdコマンドの設定内容、ウィンドウ表示情報が保存されます。また、内容を一部書き換えることにより、ファンクションキーの定義を変更することができます。このファイルはデバッガ起動時に読み込まれ、デバッガを初期設定します。これらの情報を出荷時の状態に初期化したい場合は、このファイルを削除します。詳細については、"16.12 INIファイル"を参照してください。

16.3 起動方法

16.3.1 起動フォーマット

●コマンドラインの一般形

db33 ^ [<起動時オプション>] ^ -p ^ <パラメータファイル名>

^はスペースを示します。
[ ]は省略可能なことを示します。

●ワークベンチの操作

起動オプションとパラメータファイルを選択し、[DB33]ボタンをクリックします。

16.3.2 起動時オプション

db33には9種類の起動時オプションが用意されています。

-p <**ファイル名**>.par

機能: パラメータファイルの指定(必須設定項目)
wb33上の指定: 実行ウィンドウのファイルリストボックスから選択
説明:
- 開発機種の仕様に合わせて作成したパラメータファイル(wb33で作成し、必要によりカスタマイズ)を指定します。
- このファイルがないと、デバッグは行えません。

-sim/-simdll/-mon/-icd/-ice

機能: デバッガモードの指定
wb33上の指定: リストボックス(ICD、MON、ICE、SIM)から選択
説明:
- db33の動作モードを指定します。
  - -sim シミュレータモード
  - -simdll シミュレータDLLモード("S5U1C330C2S Manual"参照)
  - -mon デバッグモニタモード
  - -icd ICDモード
  - -ice ICEモード
- 本オプションを省略した場合、db33はICEモードで起動します。

-c <**ファイル名**>.cmd

機能: コマンドファイルの指定
wb33上の指定: [db33 *.cmd file]をチェックし、リストボックスでコマンドファイルを選択
説明:
- db33起動直後に一連のコマンドを実行させたい場合に、それらを書き込んだコマンドファイルを指定します。

-w

機能: 起動時のウィンドウ指定
wb33上の指定: [1 win]をチェック
説明:
- このオプションを指定すると、db33起動時に[Command]ウィンドウのみが開きます。ログを保存する場合などに指定します。
- このオプションが指定されないときは、[Command]ウィンドウ、[Source]ウィンドウ、および[Register]ウィンドウが同時に開きます。

-comX

機能: シリアルポートの指定
wb33上の指定: リストボックス(com1、com2、com3、com4、com5、com6、com7、com8)から選択
説明:
- S5U1C33104H、S5U1C33000H、S5U1C330M2Sと通信を行うパソコンのシリアルポートを指定します。Xの部分にはポート番号(1～8)を指定してください。使用可能なポートはコンピュータによって異なります。
- 本オプションが指定されない場合、com1ポートを使用します。

-b <**ボーレート**>

機能: 通信速度の指定

wb33上の指定: リストボックス(4800、9600、19200、38400、57600、115200)から選択

説明:
- パソコン側の通信速度を指定します。<ボーレート>には4800、9600、19200、38400、57600、115200のいずれか1つを指定してください。設定可能な通信速度はコンピュータによって異なります。
- 本オプションが指定されない場合、ICEモードとデバッグモニタモードでは38400bps、ICDモードでは115200bpsに設定されます。
- S5U1C33104H/S5U1C33000H側の設定はDIPスイッチで切り換えます。

-sf

機能: スモールフォントの指定

wb33上の指定: [small font]をチェック

説明:
- db33のウィンドウに表示される文字をTerminal 10ptに設定します。
- 本オプションが指定されない場合、FixedSys 14ptを使用します。

-lptX

機能: パラレルポートの指定

wb33上の指定: リストボックス(no lpt、lpt1、lpt2)から選択

説明:
- S5U1C33104H/S5U1C33000Hと通信を行うパソコンのパラレルポートを指定します。Xの部分にはポート番号(1または2)を指定してください。使用可能なポートはコンピュータによって異なります。
- 本オプションが指定されると、lfおよびlhコマンドはパラレルポートを使用してファイルを高速ダウンロードします。

-tX

機能: ポートオープン後のウェイト

wb33上の指定: なし

説明:
- パソコンのCOMポートおよびLPTポートをオープンした後に、X秒のウェイトを挿入します。Xには0～20の10進数を指定してください。
  ノートPCではCOM/LPTポートオープン後、すぐに使用可能にならないものがあります。このオプションにより、使用可能になるまでの待ち時間を設定できます。
- シミュレータモードでは無効です。

注: **db33で使用するCOMxポートおよびLPTxポートは、他のドライバやアプリケーションで使用しないでください。また、db33を起動する前に使用できる状態になっていることを確認してください。**

コマンドラインにオプションを入力する場合、オプションの前後には1個以上のスペースが必要です。

例: c:¥cc33¥db33 -ice -p 33104_1.par -c startup.cmd -sf -com1 -b 38400

16.3.3 起動メッセージ

デバッガdb33は起動時に次のメッセージを[Command]ウィンドウに出力します。

●ICEモードでの起動時(デフォルトまたは-iceオプション指定時)

```
Debugger 33  Ver x.xx
Copyright (C)  SEIKO EPSON CORP. 1997-200x

Connecting with ICE ............. done
Reading parameter file .......... done
Initializing .................... done
Parameter file name      : xxxxxxxx.par
               Version   : xx
               Chip name : xxxxx
BOOT address             : xxxxxxx
PRC board version        : x.x
PRC board status         : xxxxxxxxxxxxxxxx
ICE hardware version     : x.x
ICE software version     : x.x
DIAG test                : omitted
Emulation memory size    : xMB
Mapping.........................done
>
```

ICEモードでの起動時は以下のテストとS5U1C33104Hの初期化を行い、上記のメッセージを表示します。

1. S5U1C33104Hの接続テスト
2. パラメータファイルの読み込みおよび内容のチェック
3. ハードウェアバージョンの読み込みおよびチェック
4. S5U1C33104Hの初期化
5. マッピング
6. S5U1C33104Hのリセット
7. デバッグ情報の初期化

これらの処理の中でエラー/ワーニングが発生した場合は、エラー/ワーニングメッセージを表示して続く処理を中止します。その場合は一度db33を終了し、エラー要因を解除後に再起動してください。

なお、ICEモードでdb33を起動する前に以下の内容を確認しておいてください。

・S5U1C33104Hが正規のRS232Cケーブルおよびパラレルケーブルでパソコンに接続されていること
・ポートの指定が正しく行われていること
・ボーレートが合っていること
・S5U1C33xxxPボードが正しく装着されていること
・S5U1C33104Hの電源がONになっていること
・S5U1C33104HおよびS5U1C33xxxPボードがリセット状態になっていないこと
・S5U1C33104Hがフリーランまたは自己診断中ではないこと
・指定したCOMポート、LPTポートが他のアプリケーションで使用されていないこと

●ICDモードでの起動時(-icdオプション指定時)

```
Debugger 33  Ver x.xx
Copyright (C)  SEIKO EPSON CORP. 1997-200x

Connecting with ICD ............. done
Reading parameter file .......... done
Initializing .................... done
Parameter file name      : xxxxxxxx.par
               Version   : xx
               Chip name : xxxxx
ICD software version     : x.x
Target connection test   : OK
Mapping...........................done
Connecting with MEM33............done (パラメータファイルにS5U1C33001M1の設定があるときのみ)
>
```

ICDモードでの起動時は以下のテストとS5U1C33000Hの初期化を行い、上記のメッセージを表示します。

1. S5U1C33000Hの接続テスト
2. パラメータファイルの読み込みおよび内容のチェック
3. S5U1C33000Hの初期化
4. デバッグ情報の初期化

これらの処理の中でエラー/ワーニングが発生した場合は、エラー/ワーニングメッセージを表示して続く処理を中止します。その場合は一度db33を終了し、エラー要因を解除後に再起動してください。

なお、ICDモードでdb33を起動する前に以下の内容を確認しておいてください。

・S5U1C33000Hが正規のRS232Cケーブルおよびパラレルケーブルでパソコンに接続されていること
・ポートの指定が正しく行われていること
・ボーレートが合っていること
・ターゲットボードが正しく接続されていること
・S5U1C33000Hおよびターゲットボードの電源がONになっていること
・指定したCOMポート、LPTポートが他のアプリケーションで使用されていないこと

●デバッグモニタモードでの起動時(-monオプション指定時)

```
Debugger 33  Ver x.xx
Copyright (C)  SEIKO EPSON CORP. 1997-200x

Connecting with MON ............. done
Reading parameter file .......... done
Initializing .................... done
Parameter file name      : xxxxxxxx.par
               Version   : xx
               Chip name : xxxxx
MON software version     : x.x
Mapping...........................done
>
```

デバッグモニタモードでの起動時は以下のテストを行い、上記のメッセージを表示します。

1. S5U1C330M2Sとの接続テスト
2. パラメータファイルの読み込みおよび内容のチェック
3. デバッグ情報の初期化

これらの処理の中でエラー/ワーニングが発生した場合は、エラー/ワーニングメッセージを表示して続く処理を中止します。その場合は一度db33を終了し、エラー要因を解除後に再起動してください。

なお、デバッグモニタモードでdb33を起動する前に以下の内容を確認しておいてください。
・S5U1C330M1D1が正規のRS232Cケーブルでパソコンに接続されていること
・ポートの指定が正しく行われていること
・ボーレートが合っていること
・ターゲットボードが正しく装着されていること
・ターゲットボード上でS5U1C330M2Sが起動していること

●シミュレータモードでの起動時(-simオプション指定時)

```
Debugger 33  Ver x.xx
Copyright (C)  SEIKO EPSON CORP. 1997-200x

Connecting with simulator ....... done
Reading parameter file .......... done
Initializing .................... done
Parameter file name      : xxxxxxxx.par
               Version   : xx
               Chip name : xxxxx
BOOT address             : xxxxxxx
Mapping.........................done
>
```

シミュレータモードでの起動時は以下のテストと初期化を行い、上記のメッセージを表示します。
1. パラメータファイルの読み込みおよび内容のチェック
2. シミュレータの初期化
3. マッピング
4. シミュレータのリセット
5. デバッグ情報の初期化

これらの処理の中でエラー/ワーニングが発生した場合は、エラー/ワーニングメッセージを表示して続く処理を中止します。その場合は一度db33を終了し、正しいパラメータファイルを作成/選択して再起動してください。

●Usage出力

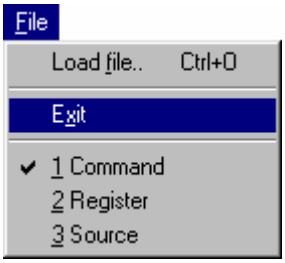
オプションが不正な場合またはパラメータファイルを指定せずにdb33を起動すると、次の使用方法のメッセージを出力します。

```
Debugger 33  Ver x.xx
Copyright (C)  SEIKO EPSON CORP. 1997-200x

 - usage -
db33.exe -p parameter_file [-MODE] [-c command_file] [-w] [-comX] [-b baud_rate] [-sf] [-lptX] [-tX]
    -p            ... parameter file
    -MODE         ... select debugger mode
      -sim          ... simulator mode
      -simdll       ... simulator + dll mode
      -mon          ... debug monitor mode
      -icd          ... ICD mode
      -ice          ... ICE mode
      default       ... ICE mode
    -c            ... command file
    -w            ... open only command window
    -comX (X:1-8) ... com port, default com1
    -b            ... baud rate, 4800, 9600, 19200, 38400(default), 57600, 115200 [bps]
                                 default baud rate   ICE mode 38400
                                 default baud rate   ICD mode 115200
                                 default baud rate   debug monitor mode 38400
    -sf           ... display with small font
    -lptX (X:1-2) ... parallel port
    -tX (X:1-20)  ... waiting time after port open
>
```

このメッセージが表示された場合は一度db33を終了し、再度正しい方法で起動してください。

16.3.4 終了方法

デバッガを終了するには[File]メニューから[Exit]を選択してください。



[File]**メニュー**

[Command]ウィンドウ上でexitコマンドを入力することによっても終了します。

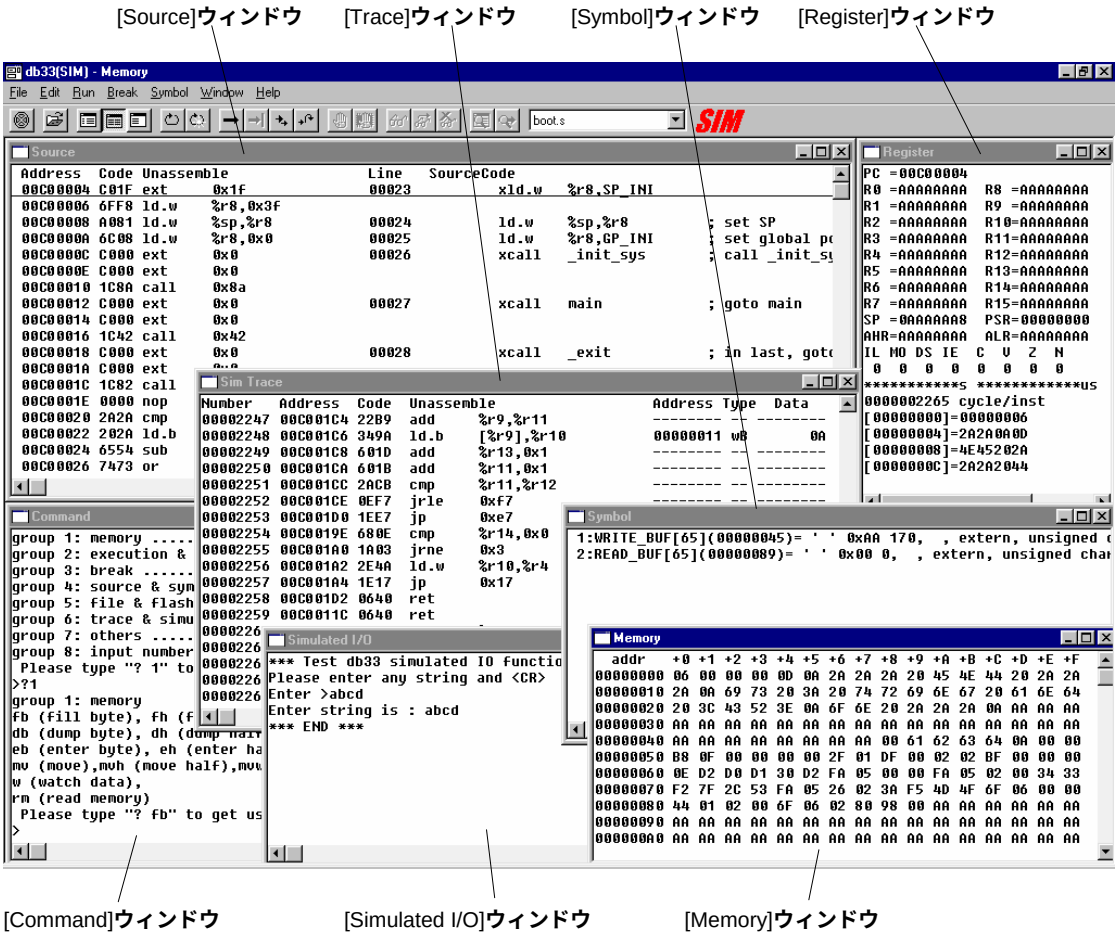
>exit

16.4 ウィンドウ

ここでは、db33で使用するウィンドウの種類を説明します。

16.4.1 ウィンドウの基本構成

デバッガdb33のウィンドウ構成は次のとおりです。



[Command]ウィンドウ以外は、メニューコマンドによって表示/非表示を切り換えられます。
起動時は"db33.ini"ファイルが読み込まれ、各ウィンドウの表示/非表示、位置とサイズが前回の終了時と同じ状態に設定されます。最初の起動時、または"db33.ini"ファイルを削除した場合は、デフォルトで[Command]、[Source]、[Register]ウィンドウが表示されます。また、-wオプションを指定して起動した場合は、[Command]ウィンドウのみが表示されます。

●全ウィンドウの共通な操作

(1) サイズの変更と移動

それぞれのウィンドウサイズは、ウィンドウの境界をドラッグすることによって任意の大きさに変更できます。[最小化]ボタン、[最大化]ボタン等も一般のWindowsアプリケーションと同様です。各ウィンドウはタイトルバーをドラッグすることによって、表示位置を変更できます。ただし、サイズ変更、移動ともに、アプリケーションウィンドウの範囲内に限られます。
[Command]ウィンドウ以外はすべて閉じることもできます。

(2) スクロール

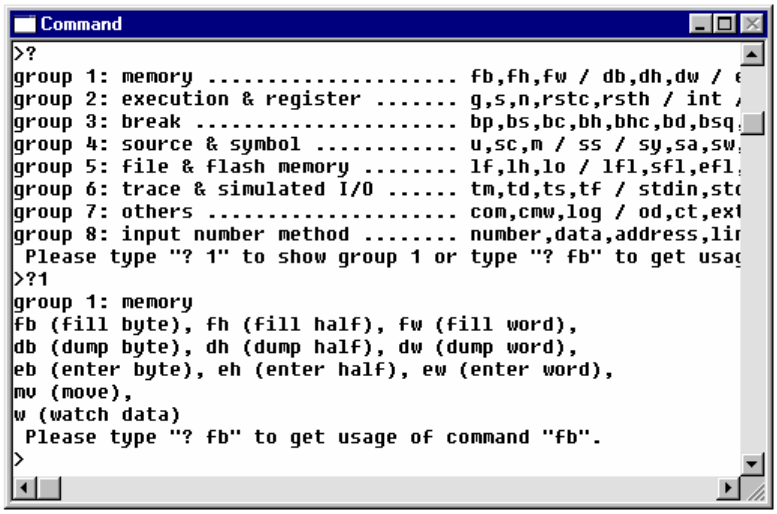
各ウィンドウはすべてスクロールできるようになっています。スクロールは次の3つの方法のいずれかによって行います。

1. 矢印ボタンのクリック、または矢印キーの入力(カーソルの移動)による1行単位のスクロール([Command]ウィンドウを除く)
2. スクロールバーのクリックによるページ(現在のウィンドウサイズ)単位のスクロール
3. スクロールバーハンドルのドラッグによる任意のエリアへの移動

(3) その他

矢印キーが使えるのは、[Command]ウィンドウ、[Source]ウィンドウ、[Register]ウィンドウ、[Memory]ウィンドウおよび[Simulated I/O]ウィンドウです。また、全ウィンドウともカット＆ペーストなどの編集は行えませんが、[Source]ウィンドウおよび[Memory]ウィンドウからのコピーと、[Command]ウィンドウへのテキストのペーストに対応しています。[Source]ウィンドウおよび[Memory]ウィンドウからコピーできるのは1行以内です。複数行のコピーには対応していません。

16.4.2 [Command]ウィンドウ



●表示内容

db33のメッセージ、ブレーク情報、マップ情報、トレースサーチの検索件数等、コマンドの実行結果を表示します。ただし、コマンド実行結果の中には、他のウィンドウに表示される内容もあります。これらの内容は対応するウィンドウが開いていれば、その中に表示されます。ウィンドウが閉じている場合は、[Command]ウィンドウに表示されます。
ログファイルには、[Command]ウィンドウに表示される内容のみが出力されます。(logコマンド参照)

表示されるコマンドの実行結果については各コマンド説明を参照してください。

●操作

<u>オープン/クローズ</u>

[Command]ウィンドウはdb33起動時に必ず開きます。[Command]ウィンドウを閉じることはできません。最小化は可能です。

<u>コマンド入力</u>

[Command]ウィンドウでは、すべてのデバッグコマンドを入力し実行させることができます。
[Command]ウィンドウの最終行にプロンプト">"が表示され、キーボードからのコマンド入力を受け付けます。
他のウィンドウが選択されているときは、[Command]ウィンドウをクリックすることによりプロンプトの後ろにカーソルが点滅し、コマンドが入力可能な状態となります。

<u>コマンドヒストリ</u>

矢印キー(↑、↓)により直前に実行した最大20個前までのコマンドを表示させ、再実行することもできます。

コマンドの入力方法については"16.8.1 コマンドのキーボード入力"を参照してください。

●制限事項

- [Command]ウィンドウ1行に表示/入力可能な文字数は空白文字も含め最大255文字です。
- 最大スクロール行数は256行です。それ以前の表示/入力内容は消去されます。

16.4.3 [Source]ウィンドウ

●表示内容

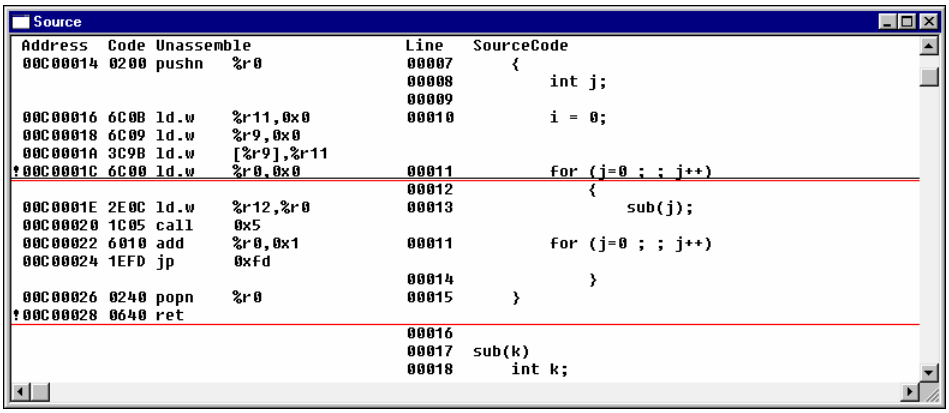
プログラムの表示

[Source]ウィンドウは、ロードしたプログラムの内容を表示します。
3種類の表示形式が選択できます。

1. **ミックス表示**([Mix]**ボタンまたは**m**コマンド入力**)

[Mix]**ボタン**

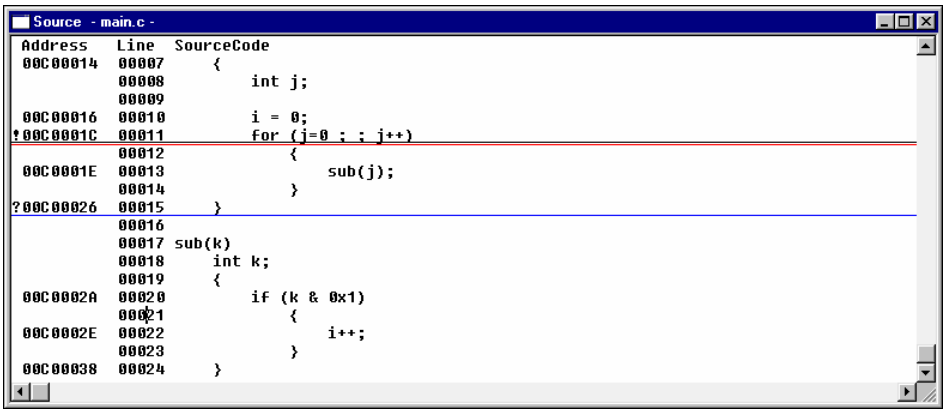
アドレス、コード、逆アセンブル内容、対応するソース行番号とソースを表示します。



2. **ソース表示**([Source]**ボタンまたは**sc**コマンド入力**)

[Source]**ボタン**

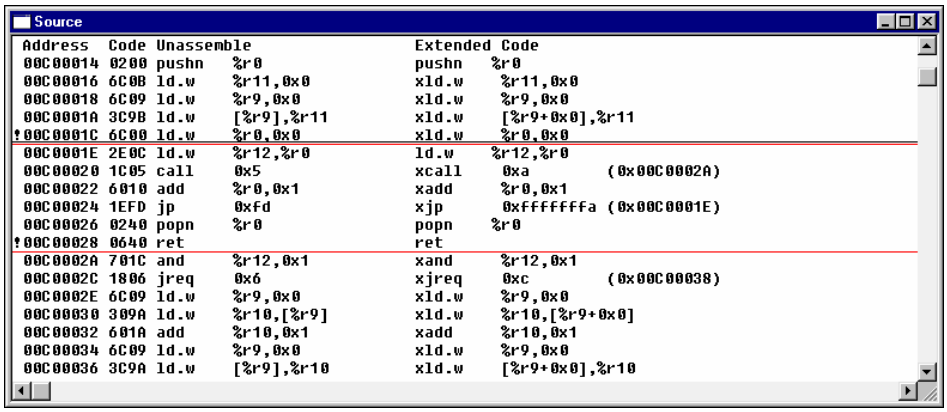
アドレス、対応するソース行番号とソースを表示します。表示されるアドレスはソースの先頭命令のアドレスです。現在のPCが属しているソースのみが表示されます。



3. **逆アセンブル表示([Unassemble]ボタンまたはuコマンド入力)**

[Unassemble]**ボタン**

db33起動時の表示形式で、アドレス、コード、逆アセンブル内容および拡張命令コードを表示します。



ソース行番号とソースの表示

ソース行番号とソースは、ソース表示用のデバッグ情報を含むsrf33オブジェクトファイルを読み込んだ場合に表示可能です。その中でも、Cコンパイラまたはプリプロセッサで-gオプションを指定したソースのみが表示されます。
db33はコードごとにそのアドレスに対応するソース行と、その直前のコードまでの間のソース行情報を持つソースを表示するようになっています。したがって、コードが終了した後に記述されたコード化されないソースなどは表示されません。

カレントPCの表示

黒の下線付きで表示される行は次に実行される行(カレントPCアドレスの行)を示します。
ミックス表示モードおよび逆アセンブル表示モードでは行全体に下線が表示されます。
ソース表示モードではカレントPCがソースの先頭命令のアドレスの場合にのみ行全体に下線が表示され、それ以外の場合、アドレス部分には下線が表示されません。

ブレークポイントの表示

アドレスの直前または直後に表示される"!"は、そのアドレスがブレークポイントに設定されていることを示します。アドレスの直前の"!"はソフトウェアブレーク、アドレスの直後の"!"はハードウェアブレークを示します。また、ブレークポイントに設定された行には赤の下線も表示されます。
ソース表示モードで表示される先頭命令のアドレス以外にブレークポイントが設定されている場合、"!"の代わりに"?"が使用されます。この行には青の下線も表示されます。

未使用領域の表示

パラメータファイルで指定されるメモリがマッピングされていない領域は、"no map"が表示されます。

その他

ミックス表示モードで、ソースファイルがない場合や行数が足りない場合、"no source"が表示されます。

●操作

オープン/クローズ

[Source]ウィンドウは、db33起動時に前回終了したときの位置と大きさで開きます。ただし、-wオプションを指定した場合は開きません。
[Source]ウィンドウは[Window]メニューの[Source]コマンドで開くことや、クローズボックスで閉じることができます。

表示形式の変更

前述のとおり、m、sc、uコマンドを実行してください(現在のPCから表示されます)。
またはそれぞれのボタンをクリックしてください(現在表示されているアドレスや行番号を基準に表示が切り換わります)。

表示内容の更新

[Source]ウィンドウは以下の場合に表示内容を更新します。
・プログラムをロードした場合(lf、lh、lflコマンド)
・プログラムを実行した場合(g、s、nコマンド)
・表示形式を変更した場合(m、sc、uコマンド)
・CPUをリセットした場合(rstc、rsthコマンド)
再表示の開始位置は現在のPCが示すアドレスからとなります。リセットを行った場合はブートアドレスから表示されます。
これらの要因以外では、プログラムメモリの内容をコマンド(e、f、mv)で変更しても、[Source]ウィンドウの表示内容を更新しません。
最新の内容を表示させるには、次の操作の1つを行ってください。

1. m、sc、uコマンドのいずれか1つを実行
2. 垂直スクロールバーをクリック

ブレークポイントの設定

ソフトウェアブレークポイント、ハードウェアブレークポイントおよびテンポラリブレークポイントの指定がウィンドウ上で行えます。
ブレークポイントを設定したいアドレスの行(ソースのみの行は不可)にカーソルを置きます。そこで、

・ [Soft PC break]ボタンをクリックすると、そのアドレスがソフトウェアブレークポイントに設定されます。[Hard PC break]ボタンをクリックすると、そのアドレスがハードウェアブレークポイントに設定されます。ブレークポイントに設定されたアドレスの行で同じ操作をすると、そのブレークポイントは解除されます。

[Soft PC break]**ボタン** [Hard PC break]**ボタン**

・ [Go to]ボタンをクリックすると、プログラムが現在のPCから実行を開始し、カーソルのある行の実行前にブレークします。

[Go to]**ボタン**

シンボルの表示と登録

内容を表示させるシンボル、および[Symbol]ウィンドウに登録するシンボルを[Source]ウィンドウ上で選択することができます。
ミックス表示モードまたはソース表示モードで、選択するシンボル名の中あるいは直前・直後にカーソルを置きます。("*"、"."、"->"、"["、"]"も含め、シンボルを切り出します。)
そこで、

・ [Symbol watch]ボタンをクリックすると、シンボルの内容を[Command]ウィンドウに表示します。

[Symbol watch]**ボタン** ([Symbol]メニュー/[Watch]コマンドでも可能)

・ [Symbol add]ボタンをクリックすると、シンボルを[Symbol]ウィンドウに登録します。

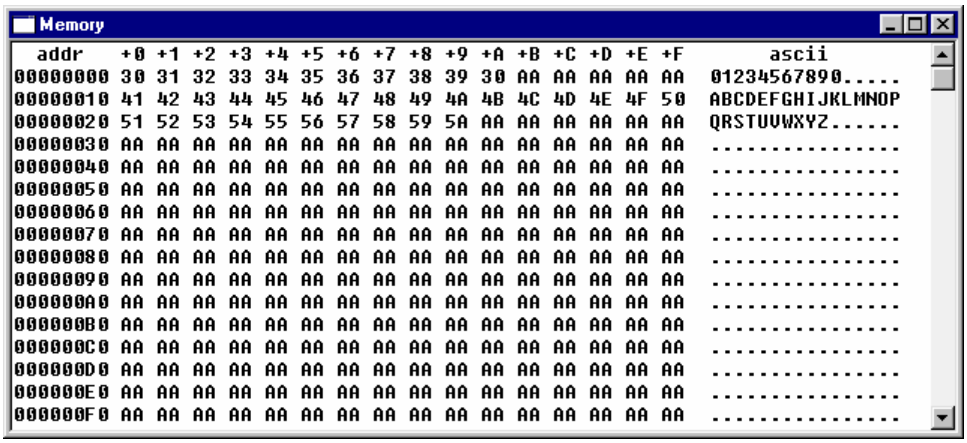
[Symbol add]**ボタン** ([Symbol]メニュー/[Add]コマンドでも可能)

●制限事項

・ [Source]ウィンドウ1行に表示可能な文字数は最大255文字です(先頭の!、?を含む)。

・ [Source]ウィンドウは表示のみで、キー入力、表示内容の編集は行えません。ただし、[Source]ウィンドウから1行以内のテキストをコピーすることは可能です。

16.4.4 [Memory]ウィンドウ



●表示内容

[Memory]ウィンドウは全メモリ領域のダンプ結果を16進数とASCII文字で表示します。必要個所の内容をスクロールにより表示可能です。各機種のマップされないアドレスの内容は"*"となります。

表示形式

データの表示形式はdb、dh、dwコマンドによってそれぞれ、バイト単位、ハーフワード単位、ワード単位に設定できます。ハーフワードおよびワード単位の表示は、その領域に設定されているエンディアン形式(パラメータファイルで指定)で行われます。

例: **バイト単位(dbコマンド)**

```
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F       ascii
00000000 30 31 32 33 34 35 36 37 38 39 30 AA AA AA AA AA  01234567890.....
    :                          :                                 :
```

ハーフワード単位(dhコマンド)

```
  addr    +0   +2   +4   +6   +8   +A   +C   +E        ascii
00000000 3130 3332 3534 3736 3938 AA30 AAAA AAAA  1032547698.0....
    :                     :                              :
```

ワード単位(dwコマンド)

```
  addr      +0       +4       +8       +C          ascii
00000000 33323130 37363534 AA303938 AAAAAAAA  32107654.098....
    :                     :                          :
```

●操作

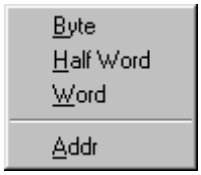
オープン/クローズ

[Memory]ウィンドウが閉じている場合は、[Window]メニュー/[Memory]コマンドを選択することによって開きます。また、同時に複数のウィンドウを開くことができます。閉じるには、クローズボックスをクリックしてください。

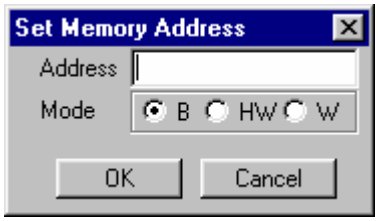
表示形式・表示開始アドレスの変更

1. コマンドによる変更
   前述のとおり、db、dh、dwコマンドを実行してください。これらのコマンドは表示開始アドレスの指定も行えます。

2. ポップアップメニューからの変更



[Memory]ウィンドウ内でマウスの右ボタンをクリックするとポップアップメニューが表示されます。メニューからデータサイズを選択すると、ウィンドウの表示形式が変更されます。



ポップアップメニューより[Addr]を選ぶと、ダイアログボックスが表示されます。テキストボックスに表示開始アドレスを入力し、ラジオボタンで表示形式を選択して[OK]ボタンをクリックしてください。アドレスの入力は省略できます。省略した場合、[Memory]ウィンドウの表示位置は変更されません。
このダイアログボックスは、[Memory]ウィンドウ内をダブルクリックすることによっても表示されます。

メモリデータの書き換え

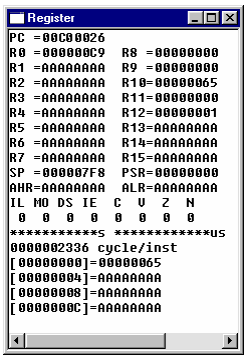
ウィンドウ内をクリックするか矢印キーを使用して、変更したいデータの直前にカーソルを移動します。そこで、16進数のデータ(asciiフィールドの場合はASCII文字)を入力します。データが変更され、カーソルが次のデータに移動しますので、連続的にデータを入力できます。ただし、バックスペースおよびデリートキーは使用できません。誤って入力した場合は、カーソルを戻してから入力し直してください。

表示内容の更新

メモリ内容をコマンド(e、f、mv)で変更しても、その時点では[Memory]ウィンドウの表示内容を更新しません。また、ターゲットプログラムのロード(lf、lh、lflコマンド)により、[Memory]ウィンドウの内容がクリアされます。
最新の内容を表示させるには、次の操作の1つを行ってください。
1. db、dh、dwコマンドのいずれか1つを実行
2. ウィンドウ内をクリック

ターゲットプログラムの実行時(g、s、nコマンド)は、ステップごとに、あるいはブレーク時に[Memory]ウィンドウの内容が更新されます(デフォルト設定)。

16.4.5 [Register]ウィンドウ



●表示内容

[Register]ウィンドウは、以下の内容を表示します。

レジスタの内容

汎用レジスタ(R0～R15)および特殊レジスタ(PC、SP、PSR、AHR、ALR)の内容を表示します。PSRの内容は個別のフラグごとにも表示されます。

実行カウンタ

実行カウンタがリセットされてからの実行サイクル数および実行時間を積算し、表示します。実行カウンタについては"16.9.5 プログラムの実行"を参照してください。

監視メモリデータ

本デバッガでは、メモリ上の4個所のアドレスを指定して、その内容を監視することができます。この4つの監視データアドレス(指定アドレスから4バイトずつ)の内容を表示します。起動時は0、4、8、C番地が監視データアドレスとして初期設定されます。データはパラメータファイルで指定されるエンディアン形式で表示されます。また、監視データアドレスをシンボルで指定した場合は(wコマンド)、データの後にシンボル名も表示されます。

ICE CPUステータス(ICEモード時)

ICEモードでは、S5U1C33104H上のCPUの状態をPCの表示データの右に表示します。表示内容は以下のとおりです。その他のモードでは表示されません。

SLP	CPUがSLEEPモードにあることを示します。
HLT	CPUがHALTモードにあることを示します。
OSC3	CPUが高速クロック(OSC3)で動作していることを示します。
OSC1	CPUが低速クロック(OSC1)で動作していることを示します。
RESET	CPUがリセット状態にあることを示します。
NO VCC	電源電圧が供給されていないことを示します。
NO CLK	動作クロックが供給されていないことを示します。
WAIT	CPUがウェイト状態にあることを示します。

表示内容の更新

[Register]ウィンドウは以下の場合に表示内容を更新します。
・レジスタ内容をダンプした場合(rdコマンド)
・CPUをリセットした場合(rstc、rsthコマンド)
・レジスタ値を変更した場合(rsコマンド)
・監視データアドレスを設定した場合(wコマンド)
・実行カウンタの表示モードを変更した場合(mdコマンド)
・オプションデータまたはフラッシュメモリの内容をロードした場合(lo、lflコマンド)
・プログラムの実行(g、s、nコマンド)終了後

オンザフライモード以外の連続実行中(gコマンド)、および最終ステップ表示モードでのステップ実行中(s、nコマンド)は数値表示部分が空白となります。
ICEモードでオンザフライ機能をONにしてプログラムを実行(gコマンド)した場合は、1～0.1秒ごとにPC、PSRのフラグ、監視データの表示内容がリアルタイムで更新されます。他の内容はブレークするまで空白となります。

●操作

オープン/クローズ

[Register]ウィンドウは、db33起動時に前回終了したときの位置と大きさで開きます。ただし、-wオプションを指定した場合は開きません。[Register]ウィンドウは[Window]メニューの[Register]コマンドで開くことや、クローズボックスで閉じることができます。

レジスタデータの書き換え

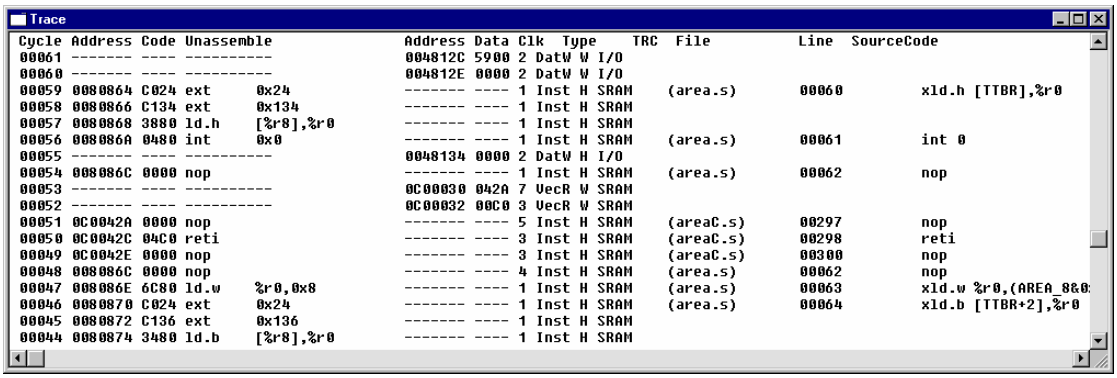
汎用レジスタ(R0～R15)、特殊レジスタ(PC、SP、PSR、AHR、ALR)およびPSRの内容を直接変更することができます。ウィンドウ内をクリックするか矢印キーを使用して、変更したいデータの直前にカーソルを移動します。そこで、16進数のデータ(PSRの各フラグは1か0)を入力します。バックスペースおよびデリートキーは使用できません。

16.4.6 [Trace]ウィンドウ

●表示内容

[Trace]ウィンドウは、各命令の実行結果を示すトレースデータを表示します。
トレースモード、トレース内容の詳細は"16.9.7 トレース機能"を参照してください。
デバッグモニタモードにはトレース機能はありません。

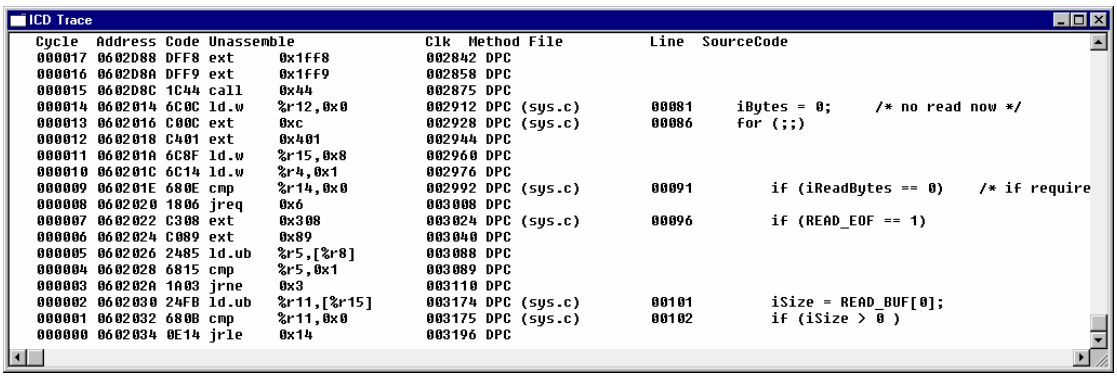
(1) ICEモード



ICEモードでは、S5U1C33104Hのトレースメモリから最大32,768サイクルのトレース結果を読み出して表示できます。トレース方式はノーマル、シングルディレイの2種類に対応しています。
表示内容は、次のとおりです。

・実行サイクル番号
・実行アドレス、コード、逆アセンブル内容
・メモリのアクセス内容(アドレス、R/W、データ)
・TRCIN端子への入力
・バスオペレーションタイプ
・ソースコード(tmコマンドのオプション)

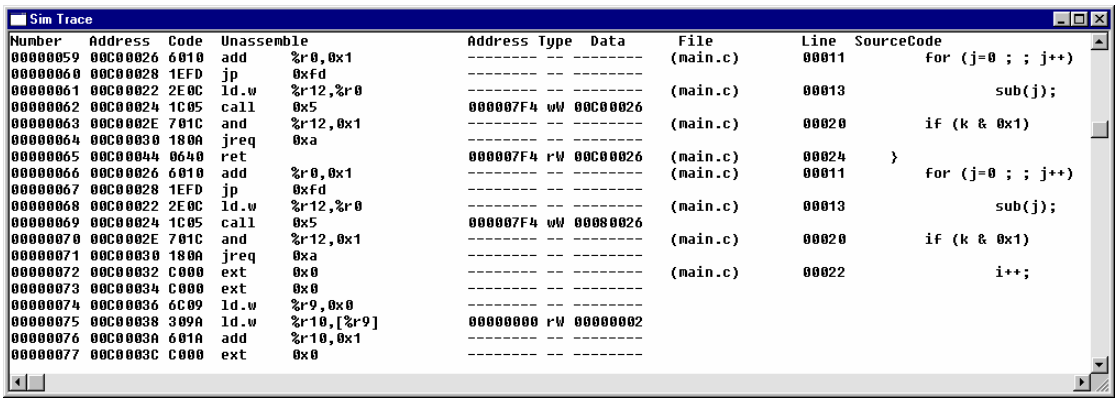
(2) ICDモード



ICDモードでは、S5U1C33000Hのトレースメモリからトレース結果を読み出して表示できます。
トレース方式は全トレース、範囲トレースの2種類に対応しています。
表示内容は、次のとおりです。

・実行サイクル番号
・実行アドレス、コード、逆アセンブル内容
・クロック数
・PC判定方式
・ソース行番号、ソースコード

(3) シミュレータモード



シミュレータモードでは、トレース機能をON(tmコマンド)にすると、それ以降のプログラム実行がすべてトレース表示されます(ファイル出力を選択した場合を除く)。表示内容は、次のとおりです。

・実行命令数
・実行アドレス、コード、逆アセンブル内容
・メモリのアクセス内容(アドレス、R/W、データ)
・ソースコード(tmコマンドのオプション)
・レジスタの内容(tmコマンドのオプション)

●操作

<u>オープン/クローズ</u>

[Trace]ウィンドウが閉じている場合は、[Window]メニュー/[Trace]コマンドを選択することによって開きます。
閉じるには、クローズボックスをクリックしてください。

<u>表示の更新</u>

ICEモード

ターゲットプログラムの実行により、[Trace]ウィンドウの内容がクリアされます。
最新の内容を表示させるには、tdコマンドを実行するか、[Trace]ウィンドウを一度閉じてから、再度開いてください。

ICDモード

ターゲットプログラムの実行により、[Trace]ウィンドウの内容がクリアされます。
プログラム中断後に最新の内容を表示させるには、tdコマンドを実行するか、[Trace]ウィンドウを一度閉じてから、再度開いてください。
また、プログラム実行中に[Display trace]ボタンをクリックするとトレースデータの更新を中止し、トレースデータを[Trace]ウィンドウに表示します。その状態で、[Resume trace]ボタンをクリックすると[Trace]ウィンドウはクリアされ、トレースを再開します。

[Display trace]**ボタン** [Resume trace]**ボタン**

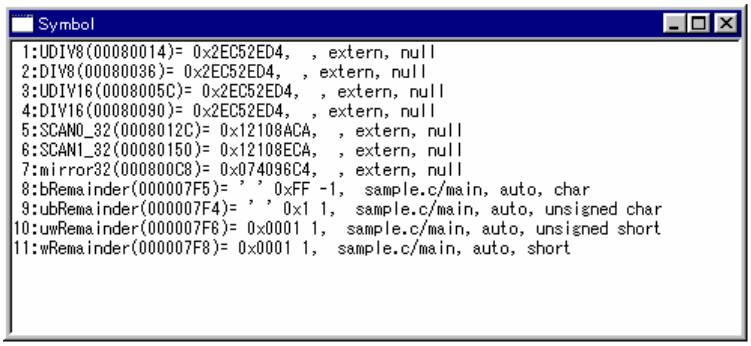
シミュレータモード

トレース機能をON(tmコマンド)にすると、プログラム実行(g、s、nコマンド)により逐次トレース結果を表示し、実行の停止とともに表示も止まります。
トレース機能をOFF(デフォルト)にすると、プログラムを実行しても表示は更新されません。

●制限事項

・[Trace]ウィンドウは表示のみで、キー入力、表示内容の編集は行えません。
・シミュレータモードでは最大255行までのスクロールに対応しています。
ICE/ICDモードではS5U1C33104H/S5U1C33000Hのトレースメモリの内容をすべて表示させることができます。

16.4.7 [Symbol]ウィンドウ



●表示内容

監視テーブルに登録されたシンボルの内容を表示します。最大99個のシンボルが登録できます。
シンボルの表示形式についてはsaコマンドの説明を参照してください。

表示の更新

[Symbol]ウィンドウは以下の場合に表示内容を更新します。

・プログラムの実行(g、s、nコマンド)終了後
　最終ステップ表示モードでのステップ実行中(s、nコマンド)は更新されません。
・シンボルを登録した場合(saコマンド)
　登録済みシンボルをsaコマンドで指定した場合も再表示されます。
・シンボルを削除した場合(sdコマンド)

シンボルのアドレスの内容をコマンド(e、f、mv)で変更しても、[Symbol]ウィンドウの表示内容を更新しません。

●操作

オープン/クローズ

[Symbol]ウィンドウが閉じている場合は、[Window]メニュー/[Symbol]コマンドを選択することによって開きます。
閉じるには、クローズボックスをクリックしてください。

表示させるシンボルの登録

[Symbol]ウィンドウにシンボルの内容を表示させるには、シンボルの登録が必要です。
登録は次のどちらかの方法で行います。

1. saコマンドを実行
2. [Source]ウィンドウをミックス表示モードまたはソース表示モードにし、登録するシンボル名の中あるいは直前・直後にカーソルを置きます。次に[Symbol add]ボタンをクリックします。

[Symbol add]**ボタン**　　([Symbol]メニュー/[Add]コマンドでも可能)

登録したシンボルの削除

監視が不要となったシンボルは次のどちらかの方法でウィンドウから削除します。

1. sdコマンドを実行
2. [Symbol]ウィンドウの削除するシンボルの行にカーソルを置きます。次に[Symbol delete]ボタンをクリックします。

[Symbol delete]**ボタン**　　([Symbol]メニュー/[Delete]コマンドでも可能)

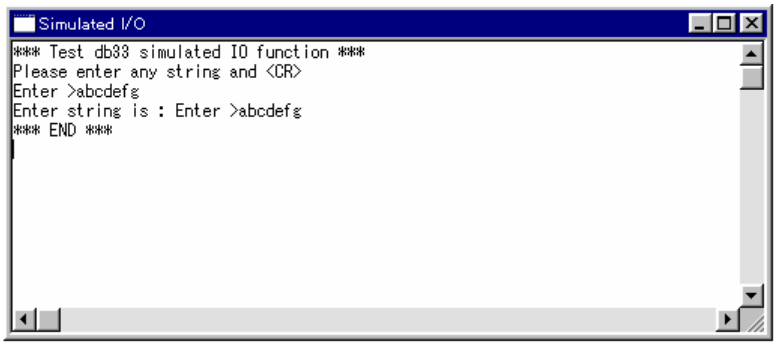
シンボル値の書き換え

シンボル値を、ウィンドウ上で直接変更することができます。変更入力できるのは16進数で表示されている部分だけです。ASCIIおよび10進数で表示されている部分では入力することができません。ウィンドウ内をクリックするか矢印キーを使用して、変更したい値の直前にカーソルを移動します。そこで、16進数を入力します。バックスペースおよびデリートキーは使用できません。

●注意

デバッガはシンボルの内容をターゲット上からバイト列として読み込み、シンボルに合わせてバイト、ハーフワード、ワードに再構成して表示します。このため、ビックエンディアン/リトルエンディアンの設定がターゲット上のBCUとデバッガが読み込んでいるパラメータファイルで異なると、データが正しく表示されませんので注意してください。

16.4.8 [Simulated I/O]ウィンドウ



●表示内容

シミュレーテッドI/O機能によりstdinから入力、stdoutへ出力する内容を表示します。

●操作

<u>オープン/クローズ</u>

[Simulated I/O]ウィンドウはデバッガ起動時には開きません。[Window]メニュー/[StdIO]コマンドを選択することによって開きます。
また、以下の場合にも開きます。

- stdinコマンドでデータの入力元がウィンドウに設定され、プログラムが指定したブレークポイントにヒットした場合
- stdoutコマンドでデータの出力先がウィンドウに設定され、プログラムが指定したブレークポイントにヒットした場合

閉じるには、クローズボックスをクリックしてください。

<u>データの入力</u>

stdinから取り込むデータを、このウィンドウから入力することができます。
詳細は"16.9.8 シミュレーテッドI/O"を参照してください。

●制限事項

[Simulated I/O]ウィンドウで表示可能な行数は最大256行です。

16.5 ツールバー

ここでは、ツールバーの概要を説明します。

16.5.1 ツールバーの構成

db33のツールバーには18個のボタンと1個のコンボボックスがあり、頻繁に使用するコマンドが設定されています。

16.5.2 [Key break]ボタン

[Key break]

ターゲットプログラムの実行を強制的にブレークします。ターゲットプログラムの実行によりCPUがスタンバイ(HALT、SLEEP)モードになった場合や、プログラムが永久ループ状態になった場合にブレークさせることができます。
このボタンはデバッグモニタモードでは無効です。

16.5.3 [Load file]ボタン

[Load file]

srf33形式のオブジェクトファイルを読み込みます。この選択はlfコマンドを実行するのと同じ働きがあります。[Load file]ボタンをクリックすると、ファイルオープン用のダイアログボックスが表示され、デバッグするファイルを選択することができます。

16.5.4 [Source], [Mix], [Unassemble]ボタン

[Source]

[Source]ウィンドウの表示をソース形式に切り換えます。

[Mix]

[Source]ウィンドウの表示をミックス形式(逆アセンブル&ソース)に切り換えます。

[Unassemble]

[Source]ウィンドウの表示を逆アセンブル形式に切り換えます。

表示の切り換えは、[Source]ウィンドウ上端に表示されているソース行番号、もしくはアドレスを基に行われます。これらのボタンは[Source]ウィンドウが開いているときのみ有効です。

16.5.5 [Reset cold], [Reset hot]ボタン

[Reset cold]

CPUをコールドリセットします。この選択はrstcコマンドを実行するのと同じ働きがあります。

[Reset hot]

CPUをホットリセットします。この選択はrsthコマンドを実行するのと同じ働きがあります。

16.5.6 [Go], [Go to], [Step], [Next]ボタン

[Go]

現在のPCからターゲットプログラムを実行します。この選択はgコマンドを実行するのと同じ働きがあります。

[Go to]

現在のPCから、[Source]ウィンドウのカーソル位置(その行のアドレス)までターゲットプログラムを実行します。この選択はg <address>コマンドを実行するのと同じ働きがあります。
このボタンを選択するには、[Source]ウィンドウを開き、ブレークするアドレスの行をクリックしておく必要があります。なお、クリックによるブレークアドレスの選択は、実コードのある行のみ有効で、ソースのみの行に対しては無効です。

[Step]

現在のPCからターゲットプログラムを1ステップ実行します。この選択はsコマンドを実行するのと同じ働きがあります。

[Next]

現在のPCからターゲットプログラムを1ステップ実行します。関数やサブルーチン内の命令は1ステップとしてすべて実行します。この選択はnコマンドを実行するのと同じ働きがあります。

16.5.7 [Soft PC break], [Hard PC break]ボタン

[Soft PC break]

[Source]ウィンドウ上のカーソルが置かれたアドレスに対し、ソフトウェアPCブレークポイントの設定と解除を行うのに使用します。[Source]ウィンドウが開いているときのみ有効です。(詳細については"16.9.6 ブレーク機能"を参照してください。)なお、クリックによるブレークアドレスの選択は、実コードのある行のみ有効で、ソースのみの行に対しては無効です。

[Hard PC break]

[Source]ウィンドウ上のカーソルが置かれたアドレスに対し、ハードウェアPCブレークポイントの設定と解除を行うのに使用します。[Source]ウィンドウが開いているときのみ有効です。(詳細については"16.9.6 ブレーク機能"を参照してください。)なお、クリックによるブレークアドレスの選択は、実コードのある行のみ有効で、ソースのみの行に対しては無効です。

16.5.8 [Symbol watch], [Symbol add], [Symbol delete]ボタン

[Symbol watch]

[Source]ウィンドウのカーソル位置にあるシンボル※の内容を[Command]ウィンドウに表示します。この選択はswコマンドを実行するのと同じ働きがあります。[Source]ウィンドウが開いている場合にのみ使用可能です。

[Symbol add]

[Source]ウィンドウのカーソル位置にあるシンボル※を[Symbol]ウィンドウに登録します。この選択はsaコマンドを実行するのと同じ働きがあります。[Source]ウィンドウが開いている場合にのみ使用可能です。

[Symbol delete]

[Symbol]ウィンドウのカーソルがある行のシンボルを、[Symbol]ウィンドウから削除します。この選択はsdコマンドを実行するのと同じ働きがあります。[Symbol]ウィンドウが開いている場合にのみ使用可能です。

※シンボル名の取得

[Symbol watch]および[Symbol add]ボタンは、[Source]ウィンドウ内のカーソルが置かれている文字列、またはカーソルの前後にある文字列を切り出してシンボル名として使用します。ただし、以下の文字で構成される範囲のみを切り出します。これら以外の文字はスペースなどの空白も含め文字列の区切りとみなされます。

a～z　A～Z　0～9　*　.　->　[　]

例: カーソル位置を"↓"で示します。

	切り出される文字列(シンボル名)
a↓ = b;	a
a = b*c↓;	c
a = b+*c↓;	*c
stru↓ct1->a = b;	struct1->a
stru↓ct1 -> a = b;	struct1

16.5.9 [Display trace], [Resume trace]ボタン

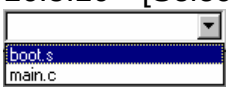
[Display trace]

ICDモードでプログラム実行中にクリックすると、トレースデータの更新を中止し、取得しているトレースデータを[Trace]ウィンドウに表示します。

[Resume trace]

ICDモードでプログラム実行中にクリックすると、中止していたトレースデータの更新を再開します。

16.5.10 [Select source]コンボボックス



[Select source]

[Source]ウィンドウに表示するプログラムのソースファイル名を選択します。
ここに表示するソースファイル名は、ロードしたオブジェクトファイルに含まれるデバッグ情報から得ています。したがって、ソース情報を持つsrf33オブジェクトファイルをロードした場合にのみ使用可能です。ソース情報がない場合、"no source"と表示されます。
また、[Source]ウィンドウがひとつも開いていない場合*1やコマンドのガイダンス途中は、使用できません。
ソースの選択は、[Source]ウィンドウの表示モードにかかわらず行えます。
ここでソースファイル名を選択すると、それに対応した[Source]ウィンドウが開き、ファイルの先頭のコードから表示されます。対応する[Source]ウィンドウがすでに開いている場合には、選択された[Source]ウィンドウは最上面に表示されます。
この選択はコマンド実行とはみなされず、コマンドヒストリやログ出力には現れません。
ロードしたオブジェクトファイルが複数のソース情報を持つ場合、ファイル名はアルファベット順にリストされます。また、同一のファイルを複数個所にインクルードしている場合、そのファイル名は1つのみ表示されます。
このボックスは選択専用で、ファイル名を入力することはできません。

*1 最初の[Source]ウィンドウの表示は、[Window]メニューより行ってください。("16.6.7 [Window]メニュー"参照)

16.6 メニュー

ここでは、メニューバーの概要を説明します。

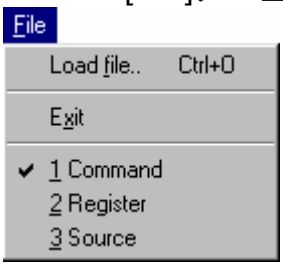
16.6.1 メニューの構成

db33のメニューバーには7つのメニュー項目があり、頻繁に使用するコマンドが設定されています。

File Edit Run Break Symbol Window Help

各メニューコマンドはマウスによる選択のほかに、キーボードからの選択([Alt]キーを押した後にメニューおよびコマンドの下線付きの文字を入力)も可能です。

16.6.2 [File]メニュー

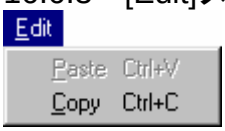


[Load file...]
ツールバーの[Load file]ボタンと同じ機能があります。[Ctrl]+[O]のキーボードショートカットも使用可能です。

[Exit]
db33を終了します。

ウィンドウリスト
現在開いている(最小化状態も含む)ウィンドウのリストが表示され、アクティブなウィンドウ名の前にチェックが付きます。このリストからウィンドウを選択すると、そのウィンドウがアクティブになります。

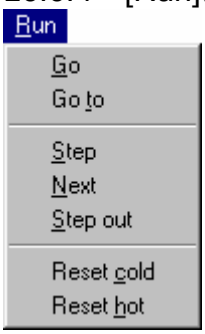
16.6.3 [Edit]メニュー



[Paste]
[Command]ウィンドウに対してのみ有効なペーストコマンドです。ログファイルなどからコマンドをコピー、ペーストして実行させたり、コマンドに必要なシンボル名などを[Source]ウィンドウよりコピー、ペーストすることができます。[Ctrl]+[V]のキーボードショートカットも使用可能です。

[Copy]
[Source]ウィンドウと[Memory]ウィンドウで使用可能なコピーコマンドです。シンボル名などをコピーして使用することができます。[Ctrl]+[C]のキーボードショートカットも使用可能です。

16.6.4 [Run]メニュー



[Go]
ツールバーの[Go]ボタンと同じ機能があります。

[Go to]
ツールバーの[Go to]ボタンと同じ機能があります。

[Step]
ツールバーの[Step]ボタンと同じ機能があります。

[Next]
ツールバーの[Next]ボタンと同じ機能があります。

[Step out]
現在のPCからターゲットプログラムを実行し、最初のリターン命令実行後に停止します。関数やサブルーチンを終了させるのに使用します。この選択はoコマンドを実行するのと同じ働きがあります。

[Reset cold]
ツールバーの[Reset cold]ボタンと同じ機能があります。

[Reset hot]
CPUをホットリセットします。この選択はrsthコマンドを実行するのと同じ働きがあります。

16.6.5 [Break]メニュー

[Soft PC...]
ソフトウェアPCブレークアドレスを設定/解除します。この選択はbpコマンドを実行するのと同じ働きがあります。[Soft PC...]コマンドを選択すると、ダイアログボックスが表示され、16ヵ所までのブレークアドレスを設定することができます。

[Hard PC...]
ハードウェアPCブレークアドレスを設定/解除します。この選択はbh/bh2コマンドを実行するのと同じ働きがあります。[Hard PC...]コマンドを選択すると、ダイアログボックスが表示され、ハードウェアPCブレークアドレスを設定することができます。

[Data...]
データブレーク条件を設定します。この選択はbdコマンドを実行するのと同じ働きがあります。[Data...]コマンドを選択すると、ダイアログボックスが表示され、ブレーク条件を設定することができます。

[Sequential...]
シーケンシャルブレーク条件を設定します。この選択はbsqコマンドを実行するのと同じ働きがあります。[Sequential...]コマンドを選択すると、ダイアログボックスが表示され、ブレーク条件を設定することができます。このメニューコマンドはICEモード時にのみ有効です。

[Area break]
エリアブレーク条件を設定します。この選択はbaコマンドを実行するのと同じ働きがあります。[Area break]コマンドを選択すると、ダイアログボックスが表示され、ブレーク条件を設定することができます。

[Bus break]
バスブレーク条件を設定します。この選択はbbコマンドを実行するのと同じ働きがあります。[Bus break]コマンドを選択すると、ダイアログボックスが表示され、ブレーク条件を設定することができます。

[All clear]
すべてのブレーク条件を解除します。この選択はbacコマンドを実行するのと同じ働きがあります。

各ブレーク機能については、"16.9.6 ブレーク機能"を参照してください。

16.6.6 [Symbol]メニュー

[Watch]
ツールバーの[Symbol watch]ボタンと同じ機能があります。[Source]ウィンドウが開いている場合にのみ選択可能です。

[Add]
ツールバーの[Symbol add]ボタンと同じ機能があります。[Source]ウィンドウが開いている場合にのみ選択可能です。

[Delete]
ツールバーの[Symbol delete]ボタンと同じ機能があります。[Symbol]ウィンドウが開いている場合にのみ選択可能です。

16.6.7 [Window]メニュー

[Command]
[Command]ウィンドウをアクティブにします。

[Source]
[Source]ウィンドウを開きます。すでに開いている場合は無効です。

[Memory]
[Memory]ウィンドウを開きます。[Memory]ウィンドウは複数開くことができます。

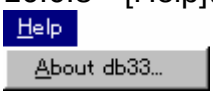
[Register]
[Register]ウィンドウを開きます。すでに開いている場合は無効です。

[Trace]
[Trace]ウィンドウを開きます。すでに開いている場合は無効です。

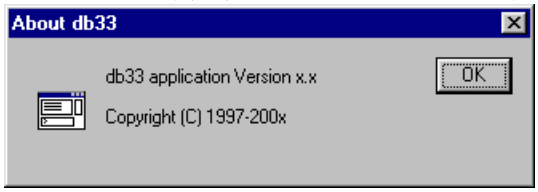
[Symbol]
[Symbol]ウィンドウを開きます。すでに開いている場合は無効です。

[StdIO]
[Simulated I/O]ウィンドウを開きます。すでに開いている場合は無効です。

16.6.8 [Help]メニュー



[About db33...]
デバッガdb33のバージョンを表示します。[OK]ボタンのクリックによってダイアログボックスが閉じ、デバッガのウィンドウに戻ります。



16.7 ファンクションキー

ファンクションキー[F1]～[F12]には、デフォルトで以下の機能が設定されており、ツールバーボタンと同様に使用することができます。

表16.7.1 ファンクションキーの割り当て(デフォルト)

キー	機能	
F1	**コールドリセット**	[Reset cold]**ボタンまたは**[Run]**メニューの**[Reset cold]**と同機能**
F2	**ホットリセット**	[Reset cold]**ボタンまたは**[Run]**メニューの**[Reset cold]**と同機能**
F3	**コマンドヒストリ**	**直前に実行したコマンドを再表示**(16.8.1**節参照**)
F4	**カーソル行まで実行**	[Go to]**ボタンまたは**[Run]**メニューの**[Go to]**と同機能**
F5	**実行**	[Go]**ボタンまたは**[Run]**メニューの**[Go]**と同機能**
F6	**ステップ実行**	[Step]**ボタンまたは**[Run]**メニューの**[Step]**と同機能**
F7	**ステップ＆スキップ**	[Next]**ボタンまたは**[Run]**メニューの**[Next]**と同機能**
F8	**シンボルサーチ**	**シンボルの検索と置き換え**(16.9.3**節参照**)
F9	**ロードファイル**	[Load file]**ボタンまたは**[File]**メニューの**[Load file]**と同機能**
F10	**ソース表示** [Source]**ボタンと同機能**	
F11	**ミックス表示**	[Mix]**ボタンと同機能**
F12	**逆アセンブル表示**	[Unassemble]**ボタンと同機能**

ファンクションキーへの機能の割り当ては、デバッガが起動時に読み込む"db33.ini"ファイルに定義されており、ユーザの書き換えにより変更が可能です。
上記以外に割り当て可能な機能は以下のとおりです。

キーブレーク	[Key break]ボタンと同機能
ソフトブレークポイント設定	[Soft PC break]ボタンと同機能
ハードブレークポイント設定	[Hard PC break]ボタンと同機能
シンボルの表示	[Symbol watch]ボタンまたは[Symbol]メニューの[Watch]と同機能
シンボルの登録	[Symbol add]ボタンまたは[Symbol]メニューの[Add]と同機能
シンボルの削除	[Symbol delete]ボタンまたは[Symbol]メニューの[Delete]と同機能
ステップアウト	oコマンドを実行
ICDオンチップトレース表示	[Display trace]ボタンと同機能
ICDオンチップトレース再開	[Resume trace]ボタンと同機能

"db33.ini"ファイルと定義内容の変更方法については、"16.12 INIファイル"を参照してください。

16.8 コマンド実行方法

デバッグ機能はすべてデバッグコマンドによって実行できます。ここでは、コマンドを実行させる方法を説明します。コマンドのパラメータなど、詳細については各コマンド説明を参照してください。

デバッグコマンドを実行させるには、[Command]ウィンドウをアクティブにし、キーボードからコマンドを入力して実行させます。頻繁に使用するコマンドはメニュー、ツールバー、ファンクションキーで実行させることもできます。

16.8.1 コマンドのキーボード入力

[Command]ウィンドウを選択してください([Command]ウィンドウ上をクリック)。その中の最終行にプロンプト">"が表示され、その後にカーソルが点滅していればコマンドが入力できる状態にあります。
そこに、デバッグコマンドを入力してください。コマンドは大文字でも、小文字でも受け付けます。

●コマンド入力の一般形

＞コマンド [パラメータ [パラメータ ... パラメータ]] ↵

・コマンドとパラメータ間にはスペースが必要です。
・パラメータ間にはスペース、カンマあるいはタブが必要です。

入力ミスの修正には矢印キー(←、→)、[Back Space]キー、[Delete]キーが使用できます。
最後に[Enter]キーを入力すると、そのコマンドを実行します(ガイダンス付きのコマンドは、表示に従って必要なデータを入力した時点で実行されます)。
入力例:

```
>g↵                    (コマンドのみの入力)
>com test.cmd↵         (コマンドとパラメータの入力)
```

●ガイダンス付きのコマンド入力

パラメータが指定されないと実行できないコマンドや、既存のデータを変更するコマンドはコマンドのみの入力でガイダンスモードとなります。ガイダンスが表示されますので、そこにパラメータを入力してください。
入力例:

```
>com↵
File name ? : test.cmd↵        ...ガイダンスに従ってデータ(アンダーライン部)を入力
>
```

- **パラメータ入力が必須のコマンド**
  上記例のcomコマンドはコマンドファイルを読み込むコマンドです。このような、パラメータの入力が必須のコマンドはパラメータを入力後、 [Enter]キーを押すと実行されます。複数のパラメータを持つコマンドでは、次のガイダンスが表示されますので、最後のパラメータまで順次入力してください。いずれかのガイダンスで[Enter]キーのみを入力すると、そのコマンドはキャンセルされ実行されません。

- **既存のデータを確認して置き換えるコマンド**
  メモリやレジスタを1つずつ書き換えるコマンドではガイダンスをスキップ(その内容を変更しない)、1つ前のガイダンスに戻す、および途中で入力を終了することができます。

[Enter]キー	入力をスキップ
[^] & [Enter]キー	1つ前のガイダンスに戻す
[q] & [Enter]キー	入力を終了する

入力例:

```
>eb↵                        ...データメモリを変更するコマンド
Enter address ? :0↵         ...開始アドレスを入力
00000000 02 :aa↵            ...0番地を0xAAに変更
00000001 00 :^↵             ...1つ前のアドレスに戻す
00000000 AA :10↵            ...0番地の入力をし直す
00000001 00 :↵              ...1番地をスキップ(内容は変更されない)
00000002 00 :ff↵
00000003 00 :q↵             ...入力を終了
>
```

●[Enter]キーによる連続実行

以下に示すコマンドは、一度実行した後に[Enter]キーのみを入力することにより、前回と同じ実行を繰り返す、あるいは前回の表示内容に引き続いて表示を行うことができます。

実行系: g、s、n
表示系: sc、m、u、db、dh、dw、od、td、sy、sw、com、cmw、ss

他のコマンドを実行すると、連続実行機能は終了します。

コマンドファイルを実行するcomおよびcmwコマンドの場合、コマンドファイルに記述されたすべてのコマンドが実行されます。複数のコマンドを連続実行させる場合などに利用できます。
たとえば、sコマンドとdbコマンドが記述されたコマンドファイルをcomコマンドで一度実行すると、その後は[Enter]キーのみで、ステップ動作とメモリダンプ(バイト)を繰り返し実行することができます。

●コマンドヒストリ

[Command]ウィンドウでは、矢印キー(↑、↓)の入力により、それまでに実行した最新の20コマンドを1つずつプロンプト位置に再表示させることができます。そこで[Enter]キーを入力すると、表示させたコマンドを再実行します。また、直前に実行した1個のコマンドについては[F3]キーで再表示させることができます(デフォルト)。

●コマンドラインのエイリアス

aliasコマンドにより、使用頻度の高いコマンドをパラメータも含めて他の名称で定義しておくことができます。最大100個のコマンドが登録でき、実行時は定義した名称に"@"を付けて入力します。
例:

```
>alias tdmp call task_dump 8↵   ..."call task_dump 8"をtdmpとして登録
      :
>@tdmp                          ...登録したコマンドの実行
```

16.8.2 パラメータの入力形式

●数値入力

コマンドでアドレスやデータを指定するパラメータは、デフォルトで16進数を入力するように設定されています。この入力には16進数の先頭の0xは省略できます。16進数として認められる文字は0～9、a～f、A～Fのみです。

コマンドにより、実行ステップ数や番号などを指定するパラメータがデフォルトで10進数に設定されているものもあります。そのパラメータに使用可能な文字は0～9のみです。
これらのパラメータについては各コマンドの説明を参照してください。

なお、以下の形式の数値は、上記デフォルト設定にかかわらず受け付けられます。

・先頭が0xで始まる数値: 16進数とみなされます。xは小文字のみ有効で0Xは数値とはみなされません。0xの後に使用可能な文字は0～9、a～f、A～Fのみです。
・先頭が+で始まる数値: 10進数とみなされます。-(負の数)はエラーとなります。+の後に使用可能な文字は0～9のみです。

注: **16進数がデフォルトのアドレス指定パラメータに"0x"を省略した16進数を入力した場合、db33は、はじめにシンボルが指定されたものとしてシンボル情報を検索します。したがって、16進数/10進数表記に該当するシンボルを使用している場合に数値でアドレスを指定するには、"0x"を付けて入力してください。**

●行番号によるアドレス指定

アドレスの指定には、ソースファイル内の行番号を使用することができます。ただし、ソース行番号情報を含むsrf33形式のオブジェクトファイルをデバッグする場合に限られます。
行番号の指定は次の形式で行います。

行番号指定: [<file name>] # <line No.>

<file name>: ソースファイル名
ファイル名<file name>は、カレントファイル(現在のPCに対応するコードを含むファイル)内のシンボルを指定する場合は省略可能です。ファイル名を省略してカレントファイルにないシンボルを指定するとエラーとなります。

<line No.>: 行番号
行番号<line number>は10進数でのみ指定可能です。"+"や"-"を付けるとエラーとなります。

例: main.c#100
#100

●シンボルによるアドレス指定

アドレスの指定には、シンボルを使用することもできます。ただし、シンボル情報を含むsrf33形式のオブジェクトファイルをデバッグする場合に限られます。
シンボルの指定は次の2つの形式で行えます。

注: 以下の説明中で、カレントソースファイル、カレント関数などの"カレント"は、現在のPCに対応するコードを含んでいることを意味します。

形式1: <symbol>

<symbol>: シンボル名
- ポインタ(*)、構造体のメンバー(->、.)、および配列([、])も指定可能です。表記方法はC言語の文法に準拠します。*は3個、[ ]は4個まで記述可能です。
- 使用可能な文字は、0〜9、a〜z、A〜Z、->、.、*に限られます。大文字と小文字は区別されます。

例: i
*message1
struct1->member1
struct2[5]

この形式でシンボルが指定されると、db33は次の順序でシンボルを検索し、そのアドレスを取得します。
1. カレントブロック
2. カレント関数
3. カレントソースファイル内のスタティックシンボル
4. 外部シンボル

該当のシンボルが見つからなかった場合は、16進数とみなされます。この場合、0〜9、a〜f、A〜F以外の文字が使われているとエラーとなります。

形式2: [<file>] / [<function>] / <symbol>

[ ]内のパラメータは省略可能です。ただし"/"は省略できません。

<file>: ソースファイル名
- カレントソースファイルを指定する場合は"."(ピリオド)を入力してください。
- 省略時の指定内容は下記の"指定の種類"を参照してください。

<function>: 関数名
- カレントソースファイルを指定する場合は"."(ピリオド)を入力してください。
- 省略時の指定内容は下記の"指定の種類"を参照してください。

<symbol>: シンボル名
- シンボル名の先頭には"*"が最大3つまで指定可能です。
- 構造体のメンバー(->、.)は10階層まで指定可能です。
- 配列([ ])は4次元まで指定可能です。
- 使用可能な文字は、0〜9、a〜z、A〜Z、->、.、*、[ ]に限られます。大文字と小文字は区別されます。( )は指定できません。

指定の種類:

//symbol	グローバルシンボル
/./symbol	カレント関数内のオート/スタティックシンボル
.//symbol	カレントソースファイル内のスタティックシンボル
file//symbol	指定ソースファイル内のスタティックシンボル
/function/symbol	指定外部関数内のオート/スタティックシンボル
./function/symbol	カレントソースファイル内の指定関数のオート/スタティックシンボル
file/function/symbol	指定ソースファイル内の指定関数のオート/スタティックシンボル

配列指定時の注意事項

1) 型情報がある場合

1次元配列では、指定した要素番号が実際の要素数より大きい場合でも、型のサイズに従ってオフセット計算を行います。エラーにはなりません。
2次元以上の配列ではこれをチェックし、指定した要素番号が実際の要素数より大きい場合はエラーとなります。

2) 型情報がない場合

1次元配列では、バイト単位でオフセット計算を行います。エラーにはなりません。
2次元以上の配列は指定することができません。

その他の制限事項

次のような指定を行った場合は、アドレスが取得できないためエラーとなります。
1) レジスタ変数を指定した場合(アドレスが割り当てられていないため)
2) 指定したポインタ変数がマップされていない領域を指している場合

●ファイル名の入力

ファイル名はパスを含めて127文字まで指定可能です。カレントディレクトリ以外のファイル名は必ずパスを指定してください。
使用可能な文字は、a〜z、A〜Z、0〜9、_、.、¥のみです。大文字と小文字は区別されます。

16.8.3 メニュー/ツールバー/ファンクションキーによる実行

16.5～16.7節に示したように、メニュー、ツールバー、ファンクションキーには頻繁に使用するコマンドが登録されています。メニューコマンドの選択、ツールバーボタンのクリック、あるいはファンクションキーを押すだけで指定のコマンドを実行できます。表16.8.3.1に登録されているコマンドの一覧を示します。

表16.8.3.1 メニュー、ツールバー、ファンクションキーで指定可能なコマンド

コマンド	機能	メニュー	ボタン	Fキー	ウィンドウ
lf	srf33ファイルのロード	[File]-[Load File...]		F9	–
m	ミックス表示	–		F11	[Source]
sc	ソース表示	–		F10	[Source]
u	逆アセンブル表示	–		F12	[Source]
g	実行	[Run]-[Go]		F5	–
g <address>	<address>まで実行	[Run]-[Go to]		F4	[Source]
s	ステップ実行	[Run]-[Step]		F6	–
n	ステップ&スキップ	[Run]-[Next]		F7	–
o	ステップアウト	[Run]-[Step out]		*	–
rstc	コールドリセット	[Run]-[Reset cold]		F1	–
rsth	ホットリセット	[Run]-[Reset hot]		F2	–
bp	ソフトウェアブレークポイント設定	[Break]-[Soft PC...]		*	[Source]
bh, bh2, bph	ハードウェアブレークポイント設定	[Break]-[Hard PC...]		*	[Source]
bd	データブレーク条件設定	[Break]-[Data...]	–	–	–
bsq	シーケンシャルブレーク条件設定	[Break]-[Sequential...]	–	–	–
bac	全ブレーク条件の解除	[Break]-[All clear]	–	–	–
ba	エリアブレーク条件設定	[Break]-[Area break]	–	–	–
bb	バスブレーク条件設定	[Break]-[Bus break]	–	–	–
sw	シンボルの表示	[Symbol]-[Watch]		*	[Source]
sa	モニタシンボルの登録	[Symbol]-[Add]		*	[Source]
sd	モニタシンボルの削除	[Symbol]-[Delete]		*	[Symbol]
–	シンボルサーチ	–	–	F8	–
–	ICDオンチップトレースデータ表示	–		*	–
–	ICDオンチップトレース再開	–		*	–
–	[Command]ウィンドウを有効化	[Window]-[Command]	–	–	–
–	[Source]ウィンドウを開く	[Window]-[Source]	–	–	–
–	[Memory]ウィンドウを開く	[Window]-[Memory]	–	–	–
–	[Register]ウィンドウを開く	[Window]-[Register]	–	–	–
–	[Trace]ウィンドウを開く	[Window]-[Trace]	–	–	–
–	[Symbol]ウィンドウを開く	[Window]-[Symbol]	–	–	–
–	[Simulated I/O]ウィンドウを開く	[Window]-[StdIO]	–	–	–
–	強制ブレーク	–		*	–
–	コマンドヒストリ	–	–	F3	–

Fキー(ファンクションキー)はデフォルトの定義内容を示します。INIファイルの書き換えにより、*の機能に変更することもできます(16.12節参照)。
ウィンドウ欄は、機能を実行する際に、開いていることが必要なウィンドウを示しています。
メニュー、ツールバー、ファンクションキーで実行したコマンドは、[Command]ウィンドウのプロンプト部分には表示されません。実行結果は対応するウィンドウに表示されます。

16.8.4 コマンドファイルによる実行

一連のデバッグコマンドを記述したコマンドファイルを読み込んで、それらのコマンドを実行させることができます。

●コマンドファイルの作成

コマンドファイルは汎用エディタ等でテキストファイルとして作成してください。
ファイル名の拡張子は".cmd"としてください。

●コマンドファイル例

次の例は、シミュレーテッドI/Oのサンプルに含まれているコマンドファイル例です。

例: ファイル名=simIO.cmd

```
lf simIO.srf        ...ファイルのロード
rstc                ...コールドリセット
stdout              ...stdoutの設定
1
WRITE_FLASH
WRITE_BUF
1
stdin               ...stdinの設定
1
READ_FLASH
READ_BUF
1
bs _exit            ...ソフトウェアブレークの設定
```

コマンドファイルにはガイダンスモードに対応した記述も可能です。その場合は、ガイダンス入力の各項目ごとに改行して記述してください。上記例では、stdout、stdinに続く内容がガイダンス項目です。

●コマンドファイルの読み込み/実行

コマンドファイルを読み込み、実行させる方法は次の2通りです。

(1) 起動時オプションによる実行

デバッガdb33の起動コマンドで-cオプションを指定すると、1つのコマンドファイルをデバッガの起動時に実行させることができます。

例: デバッガdb33の起動コマンド

```
db33 -c startup.cmd -p 33104_1.par
```

(2) コマンドによる実行

コマンドファイルの実行用に、comコマンドとcmwコマンドが用意されています。
comコマンドは指定されたファイルを読み込み、その中のコマンドを記述順に実行します。
cmwコマンドも同様ですが、個々のコマンドはmdコマンドで指定された間隔(1～256秒)で実行されます。

例: com startup.cmd

```
cmw test.cmd
```

コマンドファイルに記述されたコマンドは、[Command]ウィンドウに表示されます。

●コマンドファイルの繰り返し実行

com、cmwコマンドで一度コマンドファイルを実行させると、その後は[Enter]キーのみの入力で繰り返し実行させることができます。この機能は、他のコマンドを実行するまで有効です。

●制限事項

コマンドファイル内から別のコマンドファイルを読み込むことも可能です。ただし、最大5階層までに制限されます。6階層目のcomまたはcmwコマンドが現れるとエラーとなり、それ以後の実行を中止します。

16.8.5 ログファイル

実行したコマンドと実行結果を、テキスト形式のログファイルとして保存することができます。これによって、後からデバッグの手順と内容を確認することができます。

●コマンド例

>log test.log

logコマンドでログモードに設定後(出力開始後)は、次にlogコマンドが実行されるまでログの保存を行います。

●ログの保存内容

保存の対象となるのは[Command]ウィンドウに表示された内容です。
ツールバーやメニューで実行したコマンドや、他のウィンドウに表示される実行結果など、[Command]ウィンドウに表示されない内容は保存されません。
したがって、ログ管理をする場合は、-wオプションを指定してデバッガdb33を起動し、[Command]ウィンドウのみで実行ログを取ることをお勧めします。

16.9 デバッグ機能

ここでは、db33のデバッグ機能の概要を機能別に説明します。
各デバッグコマンドの詳細については"16.10 コマンドリファレンス"を参照してください。

16.9.1 デバッグモード

db33は4種類のデバッグモードに対応しており、起動時オプションで使用するモードを設定します。

注: **接続しているデバッグターゲットとデバッグモードを指定する起動時オプションが合っていないとワーニングダイアログが表示されます。その場合は一度db33を終了し、正しいオプションを指定して再度db33を起動してください。**

●ICEモード

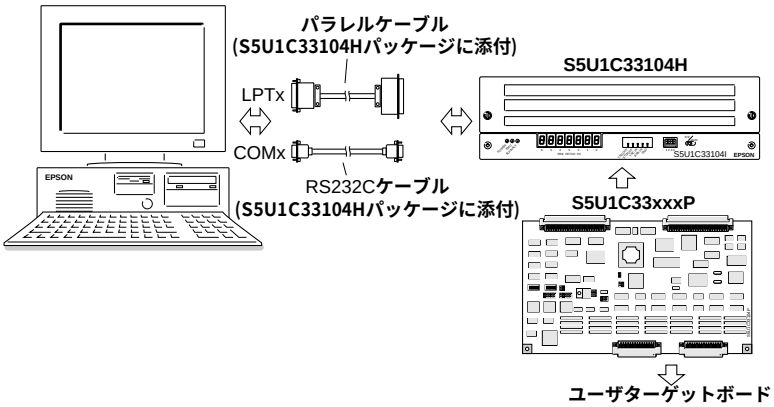


図16.9.1.1 S5U1C33104H**を使用するデバッグシステム**

起動時の指定方法

起動時オプション: -iceを指定または指定なし
wb33上の指定方法: [ICE]を選択

ICEモードで起動すると、ツールバー上に"ICE"と表示されます。

ICEモードは、インサーキットエミュレータS5U1C33104Hを使用してデバッグを行うモードです。プログラムの実行やトレースはS5U1C33104H上のメモリを使用します。S5U1C33104Hに備わる機能をすべて使用することができ、またターゲットボードを接続してハードウェアデバッグも併せて行えます。

ICEモードで起動する場合、S5U1C33104Hが正しく接続され、その電源も投入されている必要があります。
S5U1C33104H上では、各領域が次のように初期化されます。
・内蔵ROM、エミュレーションメモリに割り当てられたROM(EROM): 0xFF
・エミュレーションメモリに割り当てられたRAM(ERAM): 0xAA
・上記以外のI/O、RAM、ROM領域: 初期化されません。

S5U1C33104Hの取り扱い等については、"S1C33 Family In-Circuit Emulator Manual"を参照してください。

●ICDモード

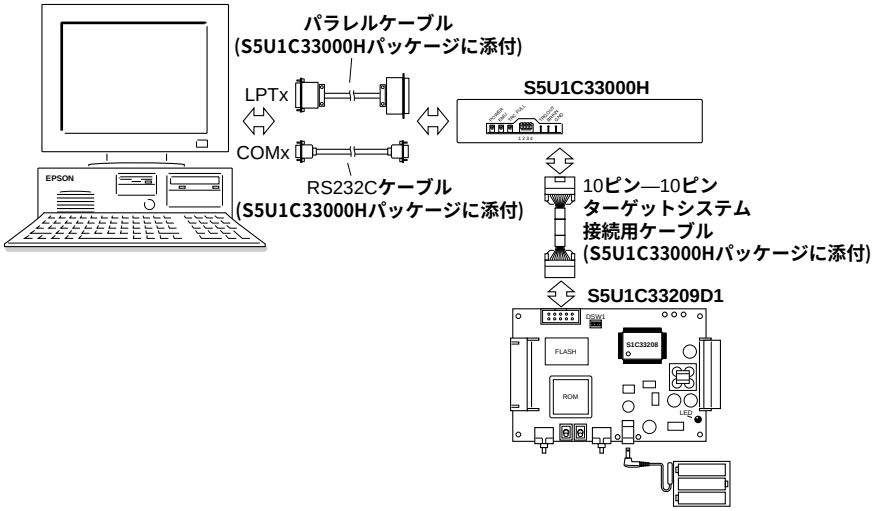


図16.9.1.2　S5U1C33000H**と**S5U1C33209D1**ボードによるデバッグシステム例**

起動時の指定方法

起動時オプション:　-icdを指定
wb33上の指定方法:　[ICD]を選択

ICDモードで起動すると、ツールバー上に"ICD"と表示されます。

ICDモードは、インサーキットデバッガS5U1C33000Hを使用してデバッグを行うモードです。プログラムはターゲットボード上で実行され、トレースデータはS5U1C33000H上のメモリに取得します。
ICDモードでは、以下のデバッグ機能を使用することはできません。
・オプションデータロード/ダンプ
・シーケンシャルブレーク
・マップブレーク

ICDモードで起動する場合、S5U1C33000Hおよびターゲットボードが正しく接続され、それらの電源も投入されている必要があります。
また、トレース機能を使用する場合はS5U1C33000HのディップスイッチSW4をOPENとしておきます。
S5U1C33000Hの取り扱い等については、"S1C33 Family In-Circuit Debugger Manual"を参照してください。

●ICD+MEM33モード

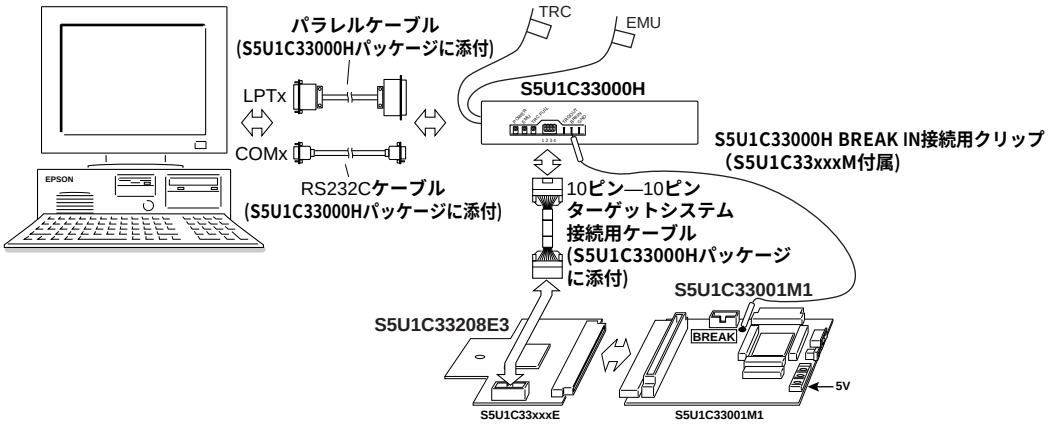


図16.9.1.3　S5U1C33000H、S5U1C33208E3、S5U1C33001M1によるデバッグシステム

起動時の指定方法

起動時オプション:　-icdを指定
wb33上の指定方法:　[ICD]を選択

注:　**起動時の指定は、ICDモードと同様です。**

ICDモードまたはICD+MEM33モードの指定は、パラメータファイルで行います。
パラメータファイルの説明は"16.11 パラメータファイルについて"を参照してください。

ICD+MEM33モードで起動するとツールバー上に"ICD+MEM33"と表示されます。

ICD+MEM33モードは、インサーキットデバッガS5U1C33000Hを使用してデバッグをするモードです。S5U1C33001M1を併用すると以下のブレーク機能が拡張されます。
・バスブレーク機能
・エリアブレーク機能
・CEブレーク機能
・マップブレーク機能(32KB単位)

ICD+MEM33モードで起動する場合、S5U1C33000H、S5U1C33001M1、S5U1C33208E3が正しく接続され、それらの電源も投入されている必要があります。
接続方法等については、"3.3.4 メモリボードS5U1C33001M1"を参照してください。

S5U1C33000H、S5U1C33001M1、S5U1C33208E3の取り扱い等については、それぞれ"S1C33 Family In-Circuit Debugger Manual"、"S1C33 Family S5U1C33001M1 Manual"、"S1C33 Family S5U1C33208E3 Manual"を参照してください。

●デバッグモニタモード

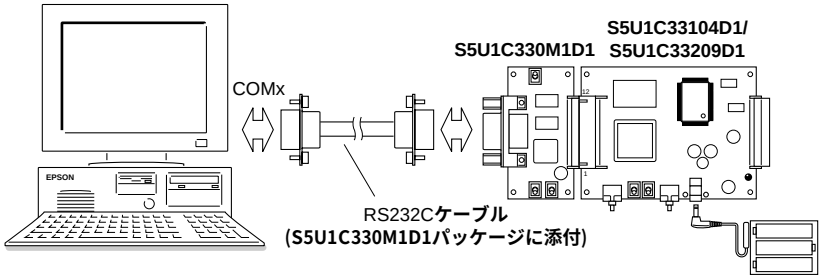


図16.9.1.4 S5U1C330M1D1とS5U1C33104D1/33209D1ボードを使用するデバッグシステム

起動時の指定方法

起動時オプション: -monを指定
wb33上の指定方法: [MON]を選択

デバッグモニタモードで起動すると、ツールバー上に"MON"と表示されます。

デバッグモニタモードは、S5U1C330M1D1ボードを介して接続されたターゲットボード上でプログラムのデバッグを行うモードです。ターゲットボードにはデバッグモニタS5U1C330M2Sが実装されている必要があります。
デバッグモニタモードでは、以下のデバッグ機能を使用することはできません。
・パラレルポートを使用したデータ書き込み
・オンザフライモード
・オプションデータロード/ダンプ
・シーケンシャルブレーク
・マップブレーク
・トレース
・実行時間/サイクル数測定
・キーブレーク

デバッグモニタモードで起動する場合、S5U1C330M1D1およびターゲットボードが正しく接続され、それらの電源も投入されている必要があります。
また、ターゲットボードのデバッグモニタを起動しておきます。
デバッグモニタの詳細については、"S1C33 Family Debug Monitor Operation Manual"を参照してください。

●シミュレータモード

起動時の指定方法

起動時オプション: -simを指定
wb33上の指定方法: [SIM]を選択

シミュレータモードで起動すると、ツールバー上に"SIM"と表示されます。

シミュレータモードは、パソコンのメモリ上でターゲットプログラムの実行をシミュレートするモードで、他のツールを必要としません。ただし、S5U1C33104Hに依存した機能や、I/O機能の評価は行えません。コアCPU、メモリモデルと割り込みのシミュレーションが可能です。
ICEモードで可能なシーケンシャルブレーク機能などがなく、トレース方法も他のモードとは異なります。詳細については、個々のコマンドの説明を参照してください。
シミュレータモードでは、各領域が次のように初期化されます。

・RAM: 0xAA
・ROM: 0xFF
・I/O: 0x00

●シミュレータDLLモード

起動時の指定方法

起動時オプション: -simdllを指定
wb33上の指定方法: [SIMDLL]を選択

シミュレータDLLモードで起動すると、ツールバー上に"SIMDLL"と表示されます。

シミュレータDLLモードについては、"S5U1C330C2S Manual"を参照してください。

●ICE、ICD、デバッグモニタモードにおける注意事項

S5U1C33104H、S5U1C33000Hでプログラムの実行をブレークするとCPUの動作クロックが高速クロック(OSC3)に切り換わり、ブレーク中はDRAMリフレッシュを除き、S1C33チップ内の全周辺機能に対するクロック供給が停止します。
S5U1C330M2Sでのプログラム実行時およびブレーク時は、瞬間的に上記と同じ状態になり、すぐに元の状態に戻ります。
このため、OSC3クロックを使用しないシステムはデバッグできません。
また、OSC3発振回路を停止して低速クロック(OSC1、32kHz)で動作している場合は、プログラムの実行をブレークした瞬間にOSC3が発振を開始しますが、発振が不安定なためCPUが誤動作します。SLEEP時を含み、OSC3発振が停止状態でのブレークは避けてください。

16.9.2　ファイルの読み込み

●ファイルの種類

db33はsrf33形式、モトローラS3形式のファイルを読み込んでデバッグすることができます。
表16.9.2.1に読み込み可能なファイルの一覧と読み込みコマンドを示します。

表16.9.2.1　ファイルと読み込みコマンド一覧

ファイル形式	拡張子	生成ツール	コマンド	メニュー	ボタン
srf33	.srf	**リンカ**	lf	[File]-[Load File..]	
モトローラ S3	.sa	**バイナリ**/HEX **コンバータ**	lh	–	–

●ソースレベルデバッグを行うためには

ソース表示とシンボルを使用してデバッグを行うためには、srf33形式のオブジェクトファイルを読み込む必要があります。モトローラS3形式のプログラムファイルを読み込んだ場合は、逆アセンブル表示のみとなります。
ターゲットのROMに書き込んだプログラムのソースレベルデバッグ用に、srf33形式のオブジェクトファイルからデバッグ情報のみを読み込むldコマンドも用意されています。

●注意点

lfおよびlhコマンドは、コード、データが存在する部分のみを読み込みます。それ以外は以前のデータが残りますので注意してください。
srf33オブジェクトファイルをロードしてソース表示を行う場合は、デバッグ情報に従ってソースファイルが読み込まれます。このため、ソースファイルの内容、保存場所(ディレクトリ)ともsrf33オブジェクトファイルを生成したときと同じ状態になっている必要があります。また、1つのソースファイルとしては最大32767行まで読み込みが可能です。

16.9.3 ソース表示およびシンボリックデバッグ機能

db33では、Cソース、アセンブリソースを表示させてデバッグを行うことができます。また、シンボルによるアドレスの指定やシンボル内容の表示も可能です。

●プログラムコードの表示

[Source]ウィンドウを開いておくと、その中にデバッグするプログラムを表示させることができます。表示形式は以下の3種類に切り換えることができます。

(1) **逆アセンブル表示**

```
Source
 Address  Code Unassemble                Extended Code
 00C00014 0200 pushn   %r0               pushn   %r0
 00C00016 6C0B ld.w    %r11,0x0          xld.w    %r11,0x0
 00C00018 6C09 ld.w    %r9,0x0           xld.w    %r9,0x0
 00C0001A 3C9B ld.w    [%r9],%r11        xld.w    [%r9+0x0],%r11
!00C0001C 6C00 ld.w    %r0,0x0           xld.w    %r0,0x0
 00C0001E 2E0C ld.w    %r12,%r0          ld.w    %r12,%r0
 00C00020 1C05 call    0x5               xcall    0xa         (0x00C0002A)
 00C00022 6010 add     %r0,0x1           xadd     %r0,0x1
 00C00024 1EFD jp      0xfd              xjp      0xfffffffa (0x00C0001E)
 00C00026 0240 popn    %r0               popn    %r0
!00C00028 0640 ret                       ret
 00C0002A 701C and     %r12,0x1          xand     %r12,0x1
 00C0002C 1806 jreq    0x6               xjreq    0xc         (0x00C00038)
 00C0002E 6C09 ld.w    %r9,0x0           xld.w    %r9,0x0
 00C00030 309A ld.w    %r10,[%r9]        xld.w    %r10,[%r9+0x0]
 00C00032 601A add     %r10,0x1          xadd     %r10,0x1
 00C00034 6C09 ld.w    %r9,0x0           xld.w    %r9,0x0
 00C00036 3C9A ld.w    [%r9],%r10        xld.w    [%r9+0x0],%r10
```

(2) **ミックス表示**

```
Source
 Address  Code Unassemble                Line   SourceCode
 00C00014 0200 pushn   %r0               00007      {
                                         00008          int j;
                                         00009
 00C00016 6C0B ld.w    %r11,0x0          00010          i = 0;
 00C00018 6C09 ld.w    %r9,0x0
 00C0001A 3C9B ld.w    [%r9],%r11
!00C0001C 6C00 ld.w    %r0,0x0           00011          for (j=0 ; ; j++)
                                         00012              {
 00C0001E 2E0C ld.w    %r12,%r0          00013                  sub(j);
 00C00020 1C05 call    0x5
 00C00022 6010 add     %r0,0x1           00011          for (j=0 ; ; j++)
 00C00024 1EFD jp      0xfd
                                         00014              }
 00C00026 0240 popn    %r0               00015      }
!00C00028 0640 ret
                                         00016
                                         00017  sub(k)
                                         00018      int k;
```

(3) **ソース表示**

```
Source - main.c -
 Address   Line  SourceCode
 00C00014  00007     {
           00008         int j;
           00009
 00C00016  00010         i = 0;
!00C0001C  00011         for (j=0 ; ; j++)
           00012             {
 00C0001E  00013                 sub(j);
           00014             }
?00C00026  00015     }
           00016
           00017 sub(k)
           00018     int k;
           00019     {
 00C0002A  00020         if (k & 0x1)
           00021             {
 00C0002E  00022                 i++;
           00023             }
 00C00038  00024     }
```

ソース表示モードでは、現在のPCに対応するコードを含むカレントソースまたは[Select source]ボックスで選択されているソースのみを表示します。

表16.9.3.1 表示切り換えコマンド/ツールバーボタン

表示形式	コマンド	ボタン
逆アセンブル	u	
ミックス	m	
ソース	sc	

これらのコマンドの実行により、[Source]ウィンドウはカレントPCアドレスがウィンドウ内に表示されるように表示内容を更新します。表示開始アドレスをこれらのコマンドによって指定することもできます。
[Source]ウィンドウが開いていない場合、[Command]ウィンドウに表示します。
各ボタンは[Source]ウィンドウが開いているときにのみ使用可能です。なお、ツールバーボタンで表示モードを変更した場合は、直前の表示と同じ個所が表示され、カレントPCアドレスへの更新は行われません。
[Select source]ボックスでソースファイルを変更した場合は、選択したソースに対応する[Source]ウィンドウを最上面に表示します。このとき対応する[Source]ウィンドウが開いていなければ、新たにウィンドウを開き、選択したソースの先頭アドレスから、それぞれの表示モードに従って表示されます。

ソース表示モードでは、検索文字列を指定して見つかった位置から表示させることもできます。

表16.9.3.2 ソース文字列検索コマンド

機　能	コマンド
文字列検索	ss

●シンボルの操作

srf33形式のオブジェクトファイルを読み込んでデバッグを行う場合、ソースファイルで定義されたシンボルを使用したデバッグが可能となります。

(1) シンボルによるアドレス指定

パラメータに<address>を持つコマンドを[Command]ウィンドウ上で入力する場合、およびダイアログボックスにアドレスを入力する場合はアドレスをシンボルで指定することができます。指定方法については"16.8.2 パラメータの入力形式"を参照してください。

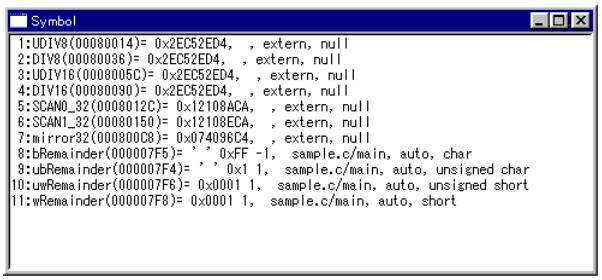
(2) シンボル情報の表示

デバッグ中のプログラムで使用しているシンボルの情報(アドレス、内容、スコープ、クラス、型等)を、[Command]ウィンドウに表示させることができます。条件を指定した一覧表示や、プログラム実行後の変数の確認などに対応したコマンドが用意されています。表示内容の詳細はsyコマンドの説明を参照してください。

表示例:
```
BOOT, 00080004, boot.s/, static, null
i, 00000000,  , extern, int
j, R0, main.c/main, register, int
k, R12, main.c/sub, reg parameter, int
main, 00080014,  , extern, int  ()
sub, 00080028,  , extern, int  ()
```

注: デバッガはシンボルの内容をターゲット上からバイト列として読み込み、シンボルに合わせてバイト、ハーフワード、ワードに再構成して表示します。このため、ビックエンディアン/リトルエンディアンの設定がターゲット上のBCUとデバッガが読み込んでいるパラメータファイルで異なると、データが正しく表示されませんので注意してください。

(3) [Symbol]ウィンドウによるシンボルの監視



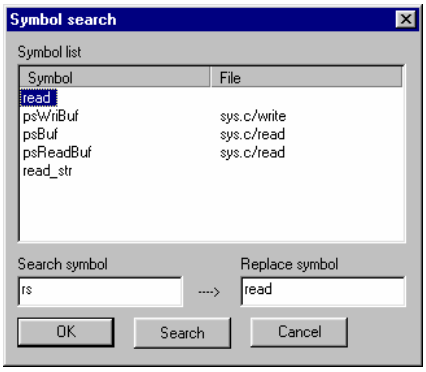
[Symbol]ウィンドウ(監視シンボルテーブル)には最大99個のシンボルを登録しておくことができ、プログラムの実行により変更される変数の内容などを監視することができます。

表16.9.3.3 シンボルリスト表示コマンド/メニューコマンド/ツールバーボタン

機　能	コマンド	メニュー	ボタン
シンボルの表示	sw	[Symbol]-[Watch]	
シンボルリスト の表示	sy	—	—
監視シンボルの登録	sa	[Symbol]-[Add]	
監視シンボルの削除	sd	[Symbol]-[Delete]	

(4) シンボル検索機能

swコマンドなどのシンボル名を入力するときの補助機能として、シンボルの検索機能があります。デフォルトでは、ファンクションキー[F8]を押すとダイアログボックスが開き、入力中のシンボルに対してあいまい検索ができます。また、検索したシンボル名を選択してコマンドラインに貼り付けることができます。



たとえば、

>sw rs

と入力した時点で[F8]キーを押すと、'rs'に近いシンボルをリスト表示します。その中からシンボル名(たとえば'read')を選択しOKをクリックするとコマンドラインは、

>as read

となります。

検索される文字は、検索対象文字列に50%以上一致している文字列です。

例: 検索文字列　候補

a1c	abc.efd	リストに登録する
a1c	abb.efd	リストに登録しない
a	abc.efd	リストに登録する

ダイアログが開いている状態では、[Search symbol]テキストボックスに検索文字を入力して[Search]ボタンを押すことにより再検索ができます。
リストに登録される文字列は最大500個です。

注: シンボル検索機能は、ファンクションキーでのみ実行可能です。したがって、"db33.ini"**ファイル内からシンボル検索機能の定義を削除すると、本機能を実行することはできません。**"db33.ini"**ファイルの定義内容については、**"16.12 INI**ファイル**"を参照してください。**

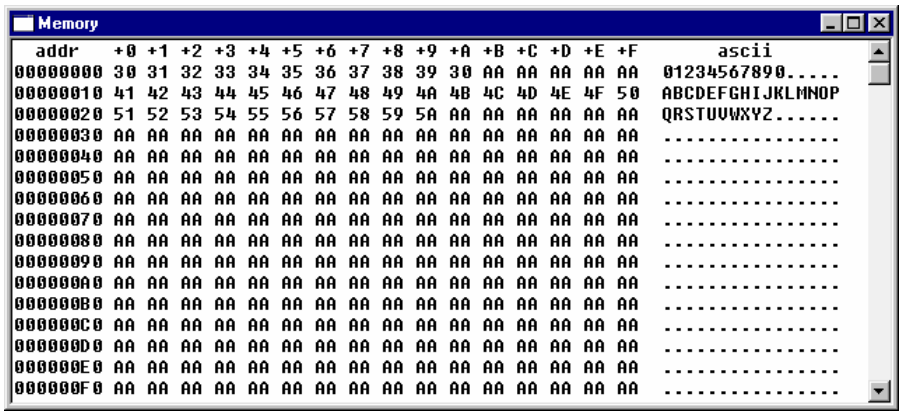
16.9.4 メモリデータ、レジスタの表示と変更

db33はメモリおよびレジスタに対する操作を行う機能を持っています。各メモリ領域はパラメータファイルで与えられるマップ情報に従ってデバッガに設定されます。ハーフワードおよびワードデータのアクセスと表示はデフォルトでリトルエンディアン形式で行われますが、パラメータファイルで特定の外部メモリ領域をビッグエンディアン形式に設定できます。

●メモリ領域の操作

メモリ領域に対しては以下の操作が行えます。

(1) **メモリダンプ**



メモリの内容を16進数とASCII文字で表示します。[Memory]ウィンドウが開いていれば最上面の[Memory]ウィンドウの内容を更新し、開いていなければ[Command]ウィンドウに表示します。(db、dh、dwコマンド)
[Memory]ウィンドウの表示開始アドレスと表示形式(バイト、ハーフワード、ワード表示)は、ウィンドウ上を右クリックすることにより表示されるポップアップメニューから変更することもできます("16.4.4 [Memory]ウィンドウ"参照)。

(2) **データの入力/変更**

データを入力して、指定アドレスの内容を書き換えます。(eb、eh、ewコマンド)
データの入力/変更は[Memory]ウィンドウ上でも行えます(16.4.4節参照)。

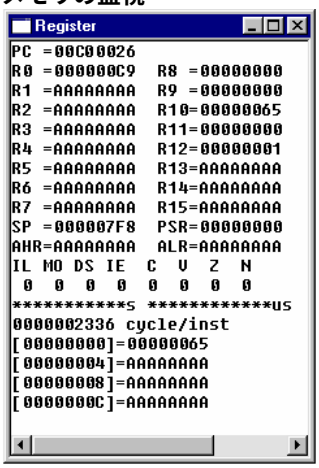
(3) **指定領域の書き換え**

指定した領域を指定したデータですべて書き換えます。(fb、fh、fwコマンド)

(4) **指定領域のコピー**

指定した領域の内容を、別の領域にコピーします。(mv、mvh、mvwコマンド)

(5) **メモリの監視**



ワード(4バイト)サイズのメモリ領域を4ヵ所、監視データアドレスとして登録することができます。(wコマンド)
登録した監視データは[Register]ウィンドウ上で確認できます。ICEモードでは、オンザフライ機能を設定することにより1〜0.1秒間隔でその内容がリアルタイムに更新されます。初期設定では、0番地、4番地、8番地、C番地が監視データアドレスとなっています。
なお、内蔵RAM領域へのデータアクセスはチップの外部から検出できないため、リアルタイムには更新されません。ブレーク時にのみ更新されます。

←監視データ

(6) **メモリデータの保存**

指定した領域の内容を、バイナリ、テキスト(16進ダンプ形式)、またはモトローラS3形式でファイルに保存することができます。(dfコマンド)

(7) **メモリとオブジェクトファイルの比較**

指定したsrf33形式のオブジェクトファイルとメモリの内容を比較します。(mcコマンド)

表16.9.4.1 メモリ操作コマンド

機　能	コマンド
メモリダンプ	db(**バイト単位**), dh(**ハーフワード単位**), dw(**ワード単位**)
データの入力/変更	eb(**バイト単位**), eh(**ハーフワード単位**), ew(**ワード単位**)
指定領域の書き換え	fb(**バイト単位**), fh(**ハーフワード単位**), fw(**ワード単位**)
指定領域のコピー	mv(**バイト単位**), mvh(**ハーフワード単位**), mvw(**ワード単位**)
監視データアドレス設定	w(**ワード単位**)
メモリデータの保存	df
メモリとオブジェクトファイルの比較	mc

※ [Memory]**ウィンドウの更新**

[Window]メニュー/[Memory]コマンドで[Memory]ウィンドウを開くと、メモリの内容が表示されます。ウィンドウをスクロールさせて、全アドレスのデータを確認することができます。d*(db、dh、dw)コマンドまたはウィンドウ上のダブルクリックで表示されるダイアログボックスでアドレスを指定し、そのアドレスから表示させることも可能です。

[Memory]ウィンドウが開き、変更するアドレスが表示されている状態でその内容を変更しても、[Memory]ウィンドウの表示内容は更新されません。表示内容を更新するには、d*コマンドを実行するか、[Memory]ウィンドウ上をクリックする必要があります。

また、ファイルを読み込むと[Memory]ウィンドウがクリアされますので、上記と同様に再表示させてください。

プログラムを連続実行させた場合は、プログラムの実行がブレークした時点で[Memory]ウィンドウが更新されます。ステップ実行時は各ステップ実行ごとに[Memory]ウィンドウが更新されます。これを、自動更新されないようにmdコマンドで設定することも可能です。

※ [Source]**ウィンドウの更新**

[Source]ウィンドウが開き、変更するアドレスが表示されている状態でその内容を変更しても、[Source]ウィンドウの表示内容は更新されません。表示内容を更新するには、コマンドで[Source]ウィンドウの表示形式を一度変更するか、[Source]ウィンドウを垂直方向にスクロールさせる必要があります。なお、コードを変更すると逆アセンブル結果は変わりますが、ソースの表示内容は変わりません。

注:
- **内蔵**I/O**領域の中でレジスタ等が割り付けられていないアドレスを読み出すと、チップの仕様上**CPU**が最後に読み出したデータが表示されます。**
- S5U1C33000H**は、メモリからの読み出しを**1**回につき**8**バイト単位で行います。このため、コマンドで指定したアドレスより、最大**7**バイト先まで読み出す可能性があります。**I/O**メモリをアクセスする際は、読み出しにより状態が変更されるレジスタに注意してください。**
- S5U1C33000H**あるいは**S5U1C330M2S**により**S5U1C33xxxE**上の内蔵**ROM**エミュレーションメモリにデータを書き込む場合はバイト書き込みのコマンド**(eb、fb、mv)**は使用しないでください。必ず、ハーフワード以上のコマンド**(eh、ew、fh、fw、mvh、mvw)**を使用してください。ファイルロードコマンド**(lf、lh)**はハーフワード書き込みを行っています。**

●レジスタの操作

レジスタに対しては以下の操作が行えます。

(1) レジスタの表示

レジスタの内容は[Register]ウィンドウまたは[Command]ウィンドウに表示させることができます。
汎用レジスタ: R0～R15
特殊レジスタ: PC、SP、PSR、AHR、ALR

ICEモードでは、プログラム実行中もオンザフライ機能("16.9.5 プログラムの実行"参照)によってPCとPSRの内容をリアルタイムに更新させることができます。

(2) レジスタ値の変更

上記レジスタの内容を任意の値に設定できます。

表16.9.4.2 レジスタ操作コマンド/メニューコマンド

機　能	コマンド	メニュー
レジスタの表示	rd	[Window]-[Register]
レジスタ値の変更	rs	―

レジスタ値やフラグは[Register]ウィンドウ上で直接書き換えることもできます("16.4.5 [Register]ウィンドウ"参照)。

16.9.5 プログラムの実行

デバッガにはターゲットプログラムを連続実行およびシングルステップ実行させる機能があります。

●連続実行

(1) **連続実行コマンド**

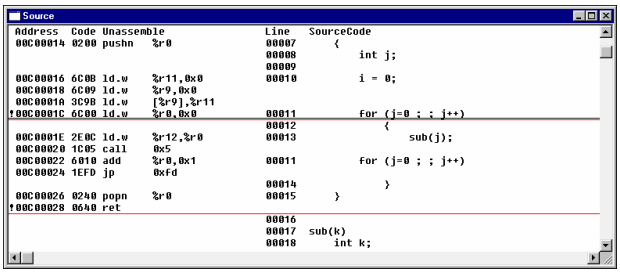
連続実行コマンドは、ロードされているプログラムを現在のPCアドレスから連続実行します。

表16.9.5.1 連続実行コマンド/メニューコマンド/ツールバーボタン

機 能	コマンド	メニュー	ボタン
連続実行	g	[Run]-[Go]	
指定アドレスまでの連続実行	g <address>	[Run]-[Go to]	

(2) **連続実行の停止**

連続実行コマンドによって、その実行中にのみ有効なテンポラリブレークアドレスを1ヵ所指定することができます。
テンポラリブレークアドレスは[Source]ウィンドウで指定することもできます。



たとえば、[Source]ウィンドウ上のアドレス0xC0001Cの行をクリック(カーソルをその行に移動)して[Go to]ボタンをクリックすると、プログラムは現在のPCから実行を開始し、0xC0001Cの命令を実行する前にブレークします。

このテンポラリブレーク以外では、実行中のプログラムは次のいずれかの要因によってブレークするまで停止しません。
・ブレーク設定コマンドで設定したブレーク条件が成立
・[Key break]ボタンのクリック(デバッグモニタモードを除く)
・その他ブレーク要因の発生

[Key break]**ボタン**
※プログラムが停止しない場合は、このボタンで強制ブレークさせることができます。

[Source]ウィンドウがソースモードで表示されている場合、実行が停止すると停止したアドレスのある[Source]ウィンドウに切り替ります。ミックスモード、逆アセンブルモードで表示されている場合には、最上面の[Source]ウィンドウが更新されます。

(3) **オンザフライ機能**

ICEモードには、連続実行中にPC、PSR、監視データの値を1～0.1秒ごとに表示するオンザフライ機能があります。これらの内容は[Register]ウィンドウの該当個所に、[Register]ウィンドウが閉じている場合は[Command]ウィンドウに表示されます。デバッガの初期設定でオンザフライ機能はOFFとなっていますが、mdコマンドによって1秒間に0回～10回まで設定することができます。オンザフライはS5U1C33104Hハードウェアにより実現される、完全なリアルタイム表示です。
ICDモードのオンザフライ機能は、トレースメモリがいっぱいになったことを示す[Register]ウィンドウ(または[Command]ウィンドウ)上への表示、およびリアルタイムトレースの表示を行います。表示間隔はICEモードと同様にmdコマンドで設定します。
ICEとICDモード以外では、オンザフライ機能は働きません。連続実行中、[Register]ウィンドウ以外のウィンドウは表示が変わらないか、内容がクリアされます。

●シングルステップ実行

(1) シングルステップ実行の種類

2種類のシングルステップ実行機能があります。

・ 全コードをシングルステップ実行(STEP)
ステップ実行の単位は[Source]ウィンドウの表示モードに従って次のようになります。
逆アセンブル表示モード: アドレス単位
ミックス表示モード: アドレス単位
ソース表示モード: ソースコード単位

・ 関数/サブルーチン以外をシングルステップ実行(NEXT)
Cソースの関数コール、アセンブリソースのサブルーチンコールやソフトウェア割り込みは、呼び出される関数/サブルーチン/ソフトウェア割り込みルーチンを1ステップとして実行します。カレント関数/サブルーチン内のコール以外のコードは、通常のシングルステップ実行と同様です。

どちらの場合も現在のPCから実行します。

表16.9.5.2 シングルステップコマンド/メニューコマンド/ツールバーボタン

機 能	コマンド	メニュー	ボタン
全コードをシングルステップ実行	s	[Run]-[Step]	
関数/サブルーチン以外をシングルステップ実行	n	[Run]-[Next]	

コマンド入力による実行では、実行するステップ数を最大65535ステップまで指定することができます。メニューコマンド、ツールバーでは1ステップずつ実行します。

以下の場合には、シングルステップ実行が指定のステップ数の実行前に終了します。
・[Key break]ボタンのクリック(デバッグモニタモードを除く)
・ユーザ設定ブレーク以外のブレーク要因の発生

ブレークポイント、データブレーク等のユーザ設定ブレークでは停止しません。

[Key break]**ボタン**
※プログラムが停止しない場合は、このボタンで強制ブレークさせることができます。

(2) ステップ動作中の表示

デバッガの初期設定では、次のように各ステップごとに表示を更新します。
[Source]ウィンドウを開いている場合は、次の実行アドレスを示す下線がステップ実行により1ステップずつ移動します。[Source]ウィンドウがソースモードで表示されている場合に実行が他のソースに移ると、そのソースに対応する[Source]ウィンドウに切り替わります。また、対応する[Source]ウィンドウが開いていない場合は新たに[Source]ウィンドウを開きます。
[Register]ウィンドウと[Memory]ウィンドウの表示内容も各ステップごとに更新されます。この表示モードを、[Register]ウィンドウは指定ステップ数の最終ステップでのみ更新するように、[Memory]ウィンドウは自動更新されないようにmdコマンドで切り換えることもできます。

(3) HALT、SLEEP状態と割り込み

S5U1C33104Hでは、シングルステップ動作時の割り込みを禁止します。
halt命令およびslp命令はシングルステップ動作時でも実行され、それによってスタンバイモードになります。
スタンバイモードは、外部割り込みを発生させることによって解除されます。また、[Key break]ボタンで解除することもできます。

●関数/サブルーチンからの脱出

プログラム実行機能のひとつに、ステップアウト機能があります。
この機能は、プログラムを現在のPCアドレスから連続実行し、関数/サブルーチンを抜けたところで停止します。なお、プログラムは、最初に現れたリターン命令を実行後、次のステップで停止します。したがって、必ずしも現在の関数/サブルーチンを抜けて停止するものではありません。

表16.9.5.3 ステップアウトコマンド/メニューコマンド

機 能	コマンド	メニュー
関数/サブルーチンからの脱出	o	[Run]-[Step out]

リターン命令を実行するまで、あるいはリターン命令がない場合はgコマンドと同様に動作します。したがって、ウィンドウの表示や、リターン命令実行前の停止方法もgコマンドと同様です。

●実行サイクル/実行時間の測定

(1) 実行カウンタと測定モード

S5U1C33104Hは31ビットの実行カウンタを3個内蔵しており、プログラムの実行時間(2種類)およびバスクロック数(1種類)を測定することができます。
S5U1C33000Hは29ビットの実行カウンタを1個内蔵しており、カウンタの測定単位をmdコマンドで実行時間(secまたはμsec)またはCPUクロック数に設定可能です。
シミュレータモードの実行カウンタは、命令数のカウントのみを行います。
デバッグモニタモードには、この測定機能はありません。

表16.9.5.4 実行カウンタの測定単位と精度

実行カウンタ	ICEモード	ICDモード	シミュレータモード
実行時間1	0〜2μsec	-1〜+1μsec *1	–
実行時間2	0〜100nsec	-0〜+100nsec *1	–
クロック	0〜2cycle	+0〜+8cycle *1	–
命令	–	–	1±0**命令**

*1) go〜ブレークのオーバーヘッドによりさらに2命令分のプリフェッチサイクルも加算されます。

実行カウンタで測定可能な最大値は次のとおりです。

	ICEモード	ICDモード
実行時間1(秒単位):	2147483647μsec = 約36分	536870911μsec = 約9分
実行時間2(μsec単位):	2147483647 × 50nsec = 約107秒	536870911 × 50nsec = 約27秒
クロック数:	2147483647 バスクロック	536870911 × 4 =2147483644 CPUクロック

(2) 測定結果の表示

測定結果は[Register]ウィンドウに表示されます。プログラム実行中はクリアされ、実行終了後に更新されます。
[Register]ウィンドウが閉じている場合は、rdコマンドによって[Command]ウィンドウに表示することができます。シングルステップの実行結果としても表示されます。
カウンタの最大値を越えた場合は"over flow"を表示します。

(3) 積算モードとリセットモード

デバッガの初期設定では、実行サイクルカウンタは積算モードに設定されます。このモードでは、カウンタがリセットされるまで測定値を積算します。
リセットモードはmdコマンドで設定することができ、プログラムの実行ごとにカウンタがリセットされます。
連続実行ではgコマンドの入力によるプログラムの実行開始時にリセットされ、その実行終了(ブレーク)までを測定します。
シングルステップ実行では、sまたはnコマンドの入力による実行開始時にリセットされ、指定ステップ数の実行終了までを測定します。1ステップのみの指定またはツールバーボタン/メニューコマンドによる実行では、1ステップごとにリセットされます。

(4) 実行カウンタのリセット

実行カウンタは次の場合にリセットされます。
- 実行カウンタのモードをmdコマンドにより切り換えた場合(積算モード→リセットモード)
- リセットモードでプログラムの実行を開始した場合

●CPUのリセット

CPUはrstcコマンド(または[Run]メニュー/[Reset Cold]コマンド、[Reset cold]ボタン)によりコールドリセット、rsthコマンド(または[Run]メニュー/[Reset Hot]コマンド、[Reset hot]ボタン)によりホットリセットされます。
CPUのリセットによる初期設定内容は以下のとおりです。

(1) CPUの内部レジスタ

R0〜R15: 0xAAAAAAAA
PC: ブートアドレス (*)
SP: 0x0AAAAAA8
PSR: 0x00000000
AHR, ALR: 0xAAAAAAAA

* ブートアドレスはTTBRレジスタに設定されているアドレスから始まるベクタテーブルの先頭4バイトの値です。コールドリセット時はTTBRレジスタが0x80000または0xc00000に初期化され、ホットリセット時はリセット前の値を保持します。

(2) 実行カウンタを0に設定

(3) [Source]ウィンドウ、[Register]ウィンドウを再表示

PCがブートアドレスに設定されるため、そのアドレスから再表示します。
[Register]ウィンドウを上記の設定で再表示します。

メモリの内容は変更されません。

注: 各モードによりrstcコマンドの動作が異なります。

ICEモード
上記の処理を行い、S1C33チップもリセットします。ターゲットボードはリセットされません。

ICDモード
上記の処理を行い、S1C33チップもリセットします。ターゲットボードはリセットされません。また、rstcコマンド実行時にターゲットがフリーランしている場合は、強制ブレークをかけてからリセットを行います。S5U1C33000Hに接続されたターゲットをリセットするとターゲットシステムはフリーラン状態となりますが、rstcコマンドでこれを停止させることができます。

デバッグモニタモード
rstcコマンドはrsthコマンドと同じ動作をします。S1C33チップのリセットやTTBRレジスタの初期化は行いません。

シミュレータモード
パラメータファイルのMCU/MPU指定によりブートアドレスが決定します。

16.9.6 ブレーク機能

ターゲットプログラムは次の要因により実行を中断します。
・ブレークコマンドの条件成立
・[Key break]ボタン(デバッグモニタモードを除く)
・S5U1C33104H/S5U1C33000HのBRKIN端子へのLowレベル入力
・不正なメモリアクセス等の発生

●コマンドによるブレーク機能

db33はコマンドによってブレーク条件が設定できる4種類のブレーク機能を持っています。それぞれ、条件が成立すると実行中のプログラムはブレークします。

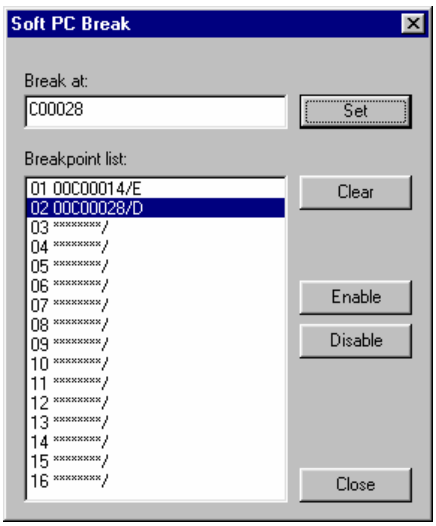
(1) **ソフトウェアPCブレーク**

実行PCアドレスがコマンドで設定したアドレスに一致するとブレークする機能です。プログラムは、そのアドレスの命令を実行する前にブレークします。最大16個所のアドレスをブレークポイントとして設定できます。
このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

```
Break by software PC break.
>
```

表16.9.6.1 ソフトウェアPCブレークポイント設定コマンド/メニューコマンド/ツールバーボタン

機　能	コマンド	メニュー	ボタン
ブレークポイント設定/解除	bp	[Break]-[Soft PC...]	



このダイアログは[Break]メニュー/[Soft PC...]コマンドを選択した場合に表示されます。
16個所までのアドレスをブレークポイントリストに登録することができます。

ブレークアドレスの登録

アドレスを[Break at]テキストボックスに入力し、[Enter]キーを押すか、[Set]ボタンをクリックします。アドレスはシンボルでも入力可能です。

ブレークポイントの解除

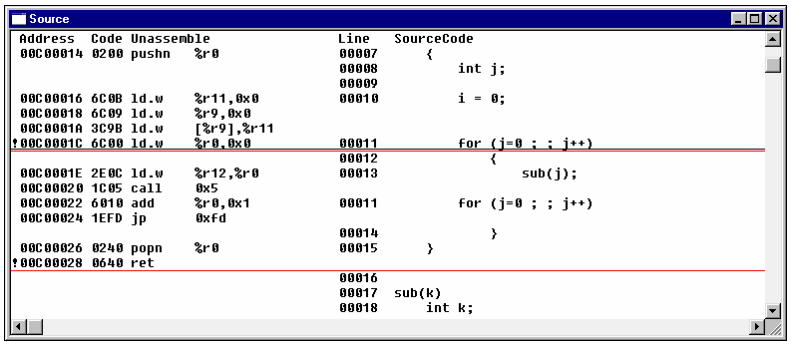
[Breakpoint list]から解除するアドレスを選択し、[Clear]ボタンをクリックします。

有効/無効の切り換え

ブレークアドレスを登録すると、そのアドレスはブレークポイントとして有効に設定されます。有効なブレークポイントはリスト内のアドレスに/Eが付きます。
登録したアドレスをクリアせずに無効に設定することもできます。この場合は、リストから無効にするアドレスを選択し、[Disable]ボタンをクリックします。アドレスに付いた/Eが/Dに変わります。[Enable]ボタンは無効(/D)に設定されたブレークポイントを有効(/E)に切り換えます。

bpコマンドを入力した場合は、[Command]ウィンドウ上に表示されるガイダンスに従って入力を行います。ブレークポイントとして有効に設定されたアドレスは、[Source]ウィンドウ上のアドレスの前に"!"または"?"のマークが付き、その行には赤または青の下線も表示されます。
"**!**" + 赤の下線:　表示されているアドレスがブレークポイントに設定されている場合
"**?**" + 青の下線:　ソース表示モードでそのソースコードの先頭アドレス以外にブレークポイントが設定されている場合

[Soft PC Break]ボタンを使用して簡単にブレークポイントの設定と解除を行うこともできます。



[Source]ウィンドウ上で、ブレークさせるアドレスの行をクリック(カーソルを移動)し、[Soft PC Break]ボタンをクリックします。その行の先頭に"!"が付き、ブレークポイントに設定されます。そのアドレスはブレークポイントリストに登録されます。"!"付きの行をクリックして[Soft PC Break]ボタンをクリックすると、ブレーク設定が解除され、アドレスがブレークポイントリストから削除されます。

※ソフトウェアPCブレークは、BRK命令の埋め込みにより実現しています。そのため、命令の埋め込みができないターゲットボード上のROMに対しては使用できません。その際は、ハードウェアPCブレークを使用してください。

注: ソフトウェアPCブレークおよびハードウェアPCブレークを、extによる拡張命令または遅延分岐命令のアドレスに設定する場合、先頭アドレス以外には設定できません。

ext xxxx	**... ○ 設定可**	**jr*.d xxxx**	**... ○ 設定可**
ext xxxx	**... × 設定不可**	**遅延命令**	**... × 設定不可**
拡張される命令	**... × 設定不可**		

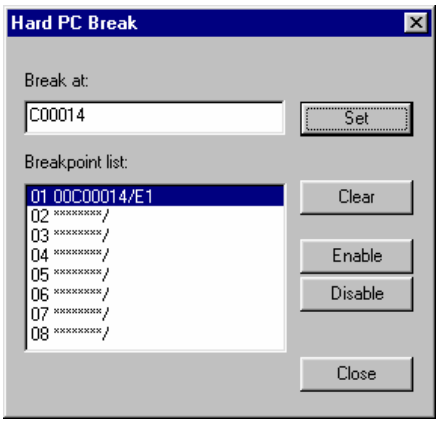
(2) **ハードウェアPCブレーク**

ハードウェアPCブレークはS1C33000コアCPUが持つデバッグモードを使用したブレーク機能です。シミュレータモードでもシミュレーション可能です。実行PCアドレスがコマンドで設定したアドレスに一致すると、そのアドレスの命令を実行する前にブレークします。2個所までのアドレスをハードウェアブレークポイントとして設定できます。このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

Break by hardware PC break. または **Break by hardware PC break2.**
>

表16.9.6.2 **ハードウェアPCブレークポイント設定コマンド/メニューコマンド/ツールバーボタン**

機　能	コマンド	メニュー	ボタン
ブレークポイント設定	bph, bh, bh2	[Break]-[Hard PC...]	
ブレークポイント解除	bph, bhc, bhc2		



このダイアログは[Break]メニュー/[Hard PC...]コマンドを選択した場合に表示されます。
8個所までのアドレスをハードウェアPCブレークポイントとして登録可能で、この内の2個所を有効なブレークポイントとして設定できます。

ブレークアドレスの登録

アドレスを[Break at]テキストボックスに入力し、[Enter]キーを押すか、[Set]ボタンをクリックします。このとき、有効なブレークポイントがすでに2個所あるときは、無効なブレークポイントとして登録されます。同時に有効にできるブレークポイントは2個所です。また、アドレスはシンボルでも入力可能です。

ブレークポイントの解除

[Breakpoint list]から解除するアドレスを選択し、[Clear]ボタンをクリックします。

有効/無効の切り換え

ブレークアドレスの登録で、有効なブレークポイントとなるのは最初の2個所までです。有効なブレークポイントはリスト内のアドレスの後に/E1または/E2が表示されます。無効なブレークポイントはアドレスの最後に/Dが表示されます。
有効/無効を切り換えるには、リストから有効あるいは無効にしたいアドレスを選択し、有効にする場合は[Enable]、無効にする場合は[Disable]のボタンをクリックします。3個所目以降のブレークポイントを有効にしようとしても[Enable]ボタンは機能しません。この場合は、先にブレークポイントを1個所無効にしてから、有効にするブレークポイントを設定してください。

※[Hard PC Break]ボタンは、[Soft PC Break]ボタンと同様に[Source]ウィンドウ上でブレークポイントを設定するのに使用します。ハードウェアPCブレークポイントに設定されたアドレスは、[Source]ウィンドウ上のアドレスの後ろに"!"または"?"のマークが付き、その行には赤または青の下線も表示されます。(ソフトウェアPCブレークを参照)

注: ICD**モードで範囲トレースを設定中は、ハードウェア**PC**ブレーク機能は無効となります。**

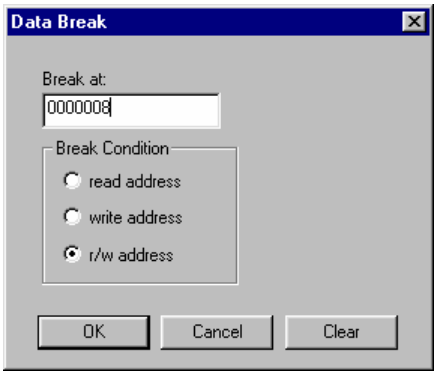
(3) データブレーク機能

データブレークは、指定したメモリアドレスをアクセスした場合にブレークを発生させる機能です。アクセス条件として、読み出しまたは書き込みのどちらか一方あるいは両方を指定することができます。両方を指定した場合は読み出し/書き込みのどちらが行われてもブレークします。
ブレークは上記の条件を満たすメモリアクセス終了後に発生します。
このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

```
Break by data break.
>
```

表16.9.6.3 データブレーク設定コマンド/メニューコマンド

機　能	コマンド	メニュー
データブレーク条件の設定/解除	bd	[Break]-[Data...]



このダイアログは[Break]メニュー/[Data...]コマンドを選択した場合に表示されます。
アドレスを[Break at]ボックスに入力し、アクセス条件をラジオボタンで選択します。
この例では、メモリの0x8番地からデータが読み出された場合あるいはデータが書き込まれた場合にブレークします。
bdコマンドを入力した場合は、[Command]ウィンドウ上に表示されるガイダンスに従って入力を行います。
アドレスにはシンボル入力も可能です。

(4) **シーケンシャルブレーク機能(ICEモードのみ)**

シーケンシャルブレークでは、1個所から3個所までのアドレス、データパターンとデータマスク、バスオペレーションのタイプを指定できます。プログラムが指定したアドレスの順序で指定のバスオペレーションを行うとブレークします。
データパターンとマスクは16ビットの16進数で指定します。
バスオペレーションタイプは以下の9種類から選択できます。

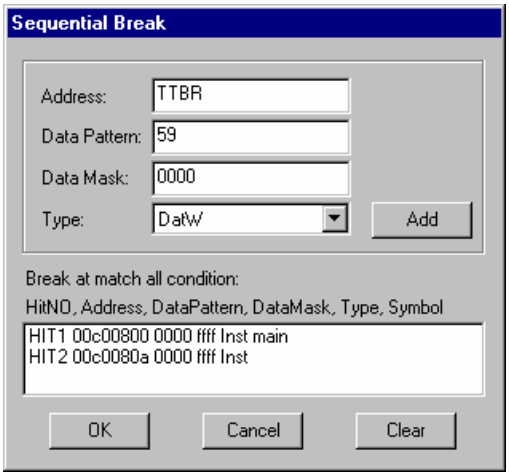
0. All　すべてのバスオペレーション
1. Inst　命令のフェッチ
2. VecR　ベクタのフェッチ
3. DatR　データの読み出し
4. DatW　データの書き込み
5. StkR　スタックからの読み出し
6. StkW　スタックへの書き込み
7. DmaR　DMAによる読み出し
8. DmaW　DMAによる書き込み

シーケンシャルブレーク機能はICEモード時にのみ使用可能です。
このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

```
Break by sequential break.
>
```

表16.9.6.4　シーケンシャルブレーク設定コマンド/メニューコマンド

機　能	コマンド	メニュー
シーケンシャルブレーク条件の設定/解除	bsq	[Break]-[Sequential...]



[Break]メニュー/[Sequential...]コマンドを選択すると、このダイアログが表示されます。
アドレス、データパターン、データマスクを各テキストボックスに入力し、バスオペレーションタイプをコンボボックスのリストから選択します。
[Add]ボタンをクリックすると、Hit No.1～3の順にブレーク条件が設定されます。
アドレスにはシンボル入力も可能です。

bsqコマンドを入力した場合は、[Command]ウィンドウ上に表示されるガイダンスに従って入力を行います。

設定例:

No.	アドレス	データパターン	データマスク	バスオペレーション
1	0x00c80000	0x0000	0xffff	Inst
2	0x00e00001	0x0001	0xff00	DatW

この例では、0xc80000番地の命令を実行後、0xe00001番地に1が書き込まれるとブレークします。1のデータマスク0xffffは、データパターンをすべてマスクする(条件に加えない)ことを指定します。2の0xff00はデータパターンの下位8ビットのみを実際のアクセスデータの下位8ビットと比較することを指定します。

注: **ICDモード、デバッグモニタモード、シミュレータモードにはシーケンシャルブレーク機能はありません。**

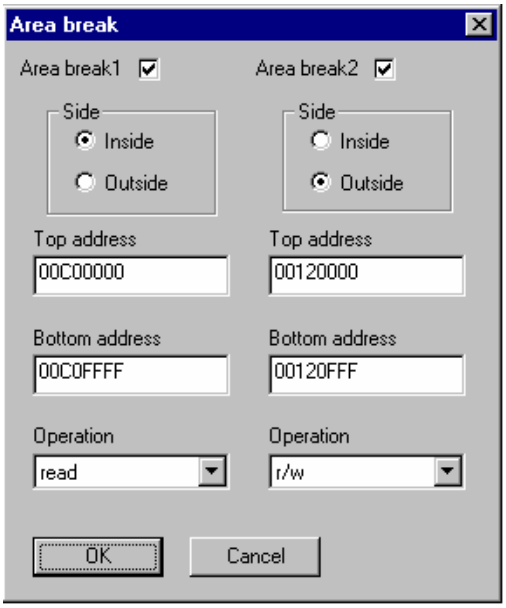
(5) **エリアブレーク機能**(ICD+MEM33**モードのみ**)

エリアブレークは、単一CEエリア内の指定アドレス範囲に対しメモリアクセスが発生した場合にブレークを発生させる機能です。指定アドレス範囲内でブレークさせるか、範囲外でブレークさせるか選択できます。複数のCEエリアの境界を越えるアドレス範囲の設定はできません。ただし、エリアブレークは2つ設定可能で、それぞれが別のCEエリアを使用することはできます。アクセス条件として、読み出しまたは書き込みの一方あるいは両方を指定することができます。
ブレークは、上記の条件を満たすメモリアクセス終了後に発生します。S5U1C33001M1は、S5U1C33000Hの外部ブレークを使用してブレークを発生させるため、実際にプログラムが停止するのはメモリアクセス直後ではありません。
このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

```
Break by external break.
Break by MEM33 area break. AreaX
>
```

表16.9.6.5 **エリアブレーク設定コマンド/メニューコマンド**

機　能	コマンド	メニュー
エリアブレーク条件の設定/解除	ba	[Break]-[Area break]



このダイアログは、[Break]メニュー/[Area break]コマンドを選択した場合に表示されます。
[Area breakX]チェックボックスで、使用するエリアブレーク番号を選択します。チェックを外したエリアブレークは設定が解除されます。
指定アドレスの範囲内でブレークさせるか、範囲外でブレークさせるかについては、[Side]ラジオボタンで選択します。アドレス範囲は、先頭アドレスを[Top address]テキストボックスに、終端アドレスを[Bottom address]テキストボックスに入力して指定します。アクセス条件は、[Operation]コンボボックスから選択します。
設定は、[OK]をクリックすると完了します。
[Cancel]をクリックすると、入力/選択内容は設定されずに終了します。
この例では、0xC00000番地から0xC0FFFF番地の内容が読み出されるとブレークします。
また、0x100000番地から0x11FFFF番地または0x121000番地から0x1FFFFF番地に対して読み出しまたは書き込みをするとブレークします。

baコマンドを入力した場合は、[Command]ウィンドウ上に表示されるガイダンスに従って入力を行います。アドレスの指定にはシンボルも使用可能です。

注:
- ICD**モード、**ICE**モード、デバッグモニタモード、シミュレータモードにはエリアブレーク機能はありません。**
- S5U1C33001M1**は、指定ブレーク条件によるアクセスの後、数回メモリアクセスされてから**S5U1C33000H**の外部ブレークを使用してブレークを発生させます。そのため、停止するのはメモリアクセス直後ではありません。**

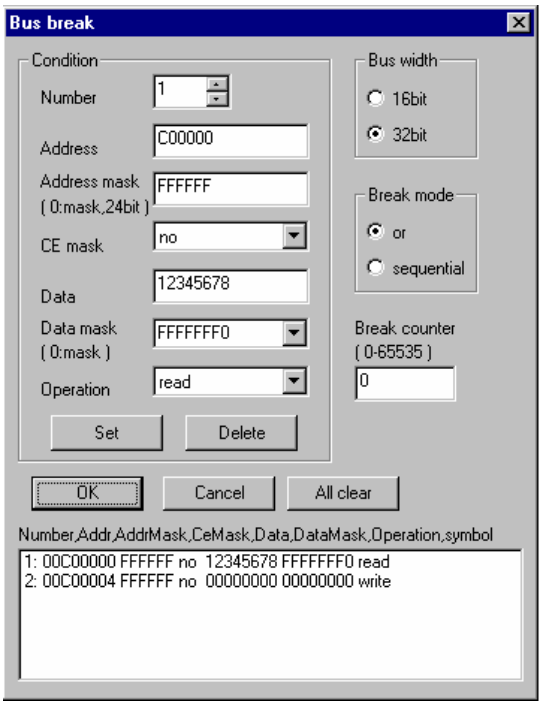
(6) **バスブレーク機能**(ICD+MEM33**モードのみ**)

バスブレークは、指定されたアドレス、アドレスマスク、CEマスク、データ、データマスク、パスオペレーション、バス幅、ブレークモード、ブレークカウンタのすべての条件が成立した場合にブレークを発生させる機能です。バス幅が16ビットの場合は6本、32ビットの場合は3本のブレーク条件を設定できます。このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

```
Break by external break.
Break by MEM33 bus break.
>
```

表16.9.6.6 **バスブレーク設定コマンド/メニューコマンド**

機　能	コマンド	メニュー
バスブレーク条件の設定/解除	bb	[Break]-[Bus break]



このダイアログは、[Break]メニュー/[Bus break]コマンドを選択した場合に表示されます。
はじめに、何本目のバスブレーク条件を設定/解除するのかを[Number]ボックスから選択してください。選択可能な番号は[Bus width]の設定により変わります(16bit: 6、32bit: 3)。なお、1から順番に設定してください。番号を飛ばして設定することはできません。番号を選んだら、設定の場合にはブレーク条件を入力し、[Set]をクリックして登録してください。削除の場合には[Delete]をクリックします。設定を終了するには、[OK]をクリックします。[Cancel]で終了した場合、入力内容は設定されません。

設定するブレーク条件は以下のとおりです。

[Bus width]　バス幅を16ビットと32ビットから選択します。

[Break mode]　ブレークモードをオアまたはシーケンシャルから選択します。
オアを選択すると、設定したバスブレークの中で1本でも条件が成立するとブレークします。シーケンシャルではバスブレーク1から最後の番号まで順番に条件が成立するとブレークします。

[Break counter]　ブレークする前に条件成立を何回スキップするか設定します。たとえば、0では最初の条件成立でブレークし、1では2回目の条件成立でブレークします。シーケンシャルモードでは最後の番号のブレーク条件成立のみをカウントの対象とします。

[Address]　アドレスを16進数で入力します。

[Address mask]　アドレスマスクを16進数で入力します。マスクしたいビットを0にします。マスクしたビットはアクセス条件の比較対象から外されます。

[CE mask]　CE信号をマスクするかしないか選択します。

[Data]　データを16進数で入力します。

[Data mask]　データマスクを16進数で入力します。マスクしたいビットを0にします。

[Operation]　アクセス条件を読み出し、書き込みあるいは両方から選択します。

上の例では、0xC00000番地から0x1234567Xが読み出されるとブレークします。
また、0xC00004番地にワードデータが書き込まれるとブレークします。

bbコマンドを入力した場合は、[Command]ウィンドウ上に表示されるガイダンスに従って入力を行います。アドレスの指定にはシンボルも使用可能です。

注: • ICD**モード、**ICE**モード、デバッグモニタモード、シミュレータモードにはバスブレーク機能はありません。**

• S5U1C33001M1**は、指定ブレーク条件によるアクセスの後、数回メモリアクセスされてから**S5U1C33000H**の外部ブレークを使用してブレークを発生させます。そのため、停止するのはメモリアクセス直後ではありません。**

●[Key break]**ボタンによる強制ブレーク**

[Key break]**ボタン**

[Key break]ボタンは、プログラムが永久ループまたはスタンバイ(HALT、SLEEP)状態から抜け出せない場合など、実行中のプログラムを強制的に終了させます。

注: **デバッグモニタモードにはこの強制ブレーク機能はありません。**

●S5U1C33104H**の**BRKIN**端子への**Low**レベル入力**(ICE/ICD**モードのみ**)

ICE/ICDモードにおいて、S5U1C33104H/S5U1C33000HのBRKIN端子にLowレベルのパルスが入力されるとプログラムはブレークします。
このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

```
Break by external break.
>
```

注: • **デバッグモニタモード、シミュレータモードには、この機能はありません。**

• S5U1C33000H Ver. 1**の場合、**BRKIN**端子にパルスが入力されてから実際にブレークが発生するまで、**1.5μsec**程度の遅れを生じます。**

●ICE**モード、シミュレータモードでのマップブレーク、不当命令実行によるブレーク**

プログラムが、無効な領域に対してアクセスを行った場合もブレークします。

注: ICD**モード、デバッグモニタモードには以下のブレーク機能はありません。**

(1) ROM**領域への書き込み**

パラメータファイルで設定されるROM領域に対して書き込みが行われた場合にブレークします。
このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

```
Break by writing ROM area.
>
```

(2) **未定義領域へのアクセス**

パラメータファイルで設定される、メモリマップ以外の未定義領域をアクセスした場合にブレークします。このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

```
Break by accessing no map area.
>
```

(3) **スタック領域外へのアクセス**(ICE**モードのみ**)

スタックポインタSPによってスタック領域外へのアクセスが行われた場合にブレークします。
このブレークはICEモード時にのみ発生します。このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

```
Break by out of SP area.
>
```

(4) **不当命令を実行(シミュレータモードのみ)**
シミュレータモードで、不当命令(アセンブラas33で生成されないコード。逆アセンブル表示に"*"が付きます)を実行した場合にブレークします。このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

```
Break by illegal instruction.
>
```

注: • S5U1C33104Hでは、内部RAM領域に対するバスアクセスはチップ外部から検出できないため、マップブレークとシーケンシャルブレークは発生しません。ただし、内部RAMでの命令実行時のみ、未定義領域へのアクセスによるブレークが発生します。

• ICE、ICD、デバッグモニタモードでターゲットプログラムの実行中にコールドリセットをかけると、チップ上でサポートされるハードウェアPCブレーク(goやnextで内部的に使用するテンポラリブレークも含む)およびデータブレークはクリアされ、ブレークが発生しなくなります。一度他の要因でブレークすると再設定されますが、それまでは無効となりますので注意してください。

●ICD+MEMモードでのCEブレークとマップブレーク

ICD+MEMモードでは、パラメータファイルのメモリマップ情報により、以下の2つのブレーク機能が設定されます。

(1) **CEブレーク**
S5U1C33001M1には、CEブレーク機能が搭載されています。これは、各CE信号に対して、読み出し/書き込み可、読み出しのみ、アクセス不可の属性を付け、不当なアクセスをした場合にブレークするものです。このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

```
Break by MEM33 ce break. CE5     (CE5でブレークした場合)
>
```

(2) **マップブレーク**
S5U1C33001M1には、マップブレーク機能が搭載されています。これは、各32KBのエリアに対してアクセス可/不可の属性を付け、不当なアクセスをした場合にブレークするものです。このブレークが発生したときは、次のメッセージを[Command]ウィンドウに表示してコマンド入力待ちとなります。

```
Break by MEM33 map break.
>
```

注: • バスの動作周波数が30MHzを越えると、no CEブレークが不当に発生しやすくなります。
no CEブレークは、CEが選択されずにRD/WRアクセスが行われると発生します。この場合は、パラメータファイルに;!MEM33_NOCE_DISABLEを記述して(wb33のパラメータファイルジェネレータのオプション選択で生成も可能)、このブレークを無効にしてください。

• アドレスまたはCE端子の一部が設けられていないマイコンにおいては、CEブレークとマップブレークを使用することはできません。パラメータファイルに;!MEM33_MAP_DISABLEと;!MEM33_CE_DISABLEを記述し、これらのブレークを無効にしてください。またエリアブレークも使用できません。バスブレークは、端子のないアドレスビットなどをマスクすると使用できます。

• CEブレーク、マップブレークとも、バスブレークやエリアブレークと同様に、ブレーク条件を満たすアクセスが発生してから2〜3回程度バスアクセスが行われると、実際にブレークが発生します。これは、S5U1C33001M1のブレーク機能がバスアクセスベースのパイプラインになっているためです。

• S5U1C33001M1のブレーク機能をマップしたエリア(デフォルトはCE9エリア)はブレーク用レジスタ専用エリアとなり、他のデバイスはマップできません。また、このエリアにはブレーク等も設定しないでください。

16.9.7 トレース機能

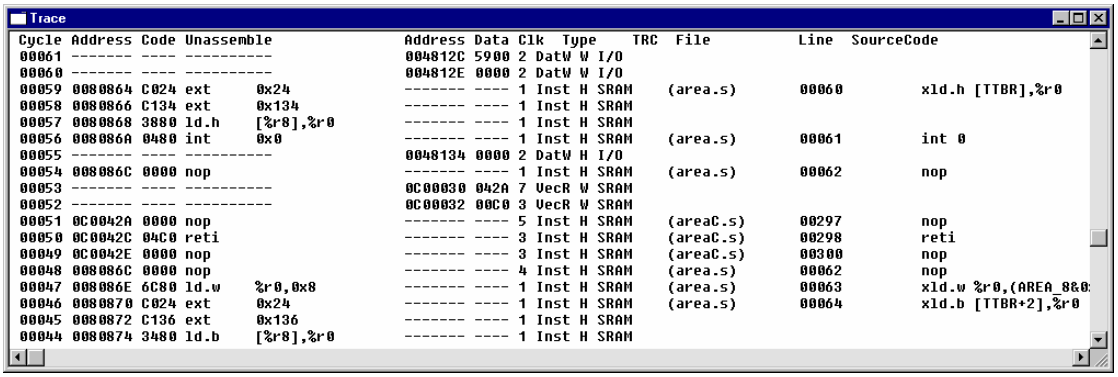
デバッガには、プログラムの実行をトレースする機能があります。
デバッグモードにより操作方法や機能が異なりますので、注意してください。

注: デバッグモニタモードにはトレース機能はありません。

●ICEモードのトレース機能

(1) トレースメモリとトレース情報

S5U1C33104Hはトレースメモリを内蔵しています。プログラムがトレースモードに従ったトレース範囲を実行すると、バスサイクルごとのトレース情報がこのメモリに取り込まれます。トレースメモリは32768サイクル分の容量があります。トレース情報がこの容量を越えると、シングルディレイトリガモード以外では古いデータから上書きしていきます。したがって、トレースメモリには常に32768サイクル以内のトレース情報が記録されています。プログラムの実行によりトレースメモリはクリアされ、新しい実行データをトレースします。



Trace

Cycle	Address	Code	Unassemble		Address	Data	Clk	Type	TRC	File	Line	SourceCode
00061	-------	----	----------		004812C	5900	2	DatW W I/O				
00060	-------	----	----------		004812E	0000	2	DatW W I/O				
00059	0080864	C024	ext 0x24		-------	----	1	Inst H SRAM		(area.s)	00060	xld.h [TTBR],%r0
00058	0080866	C134	ext 0x134		-------	----	1	Inst H SRAM				
00057	0080868	3880	ld.h [%r8],%r0		-------	----	1	Inst H SRAM				
00056	008086A	0480	int 0x0		-------	----	1	Inst H SRAM		(area.s)	00061	int 0
00055	-------	----	----------		0048134	0000	2	DatW H I/O				
00054	008086C	0000	nop		-------	----	1	Inst H SRAM		(area.s)	00062	nop
00053	-------	----	----------		0C00030	042A	7	VecR W SRAM				
00052	-------	----	----------		0C00032	00C0	3	VecR W SRAM				
00051	0C0042A	0000	nop		-------	----	5	Inst H SRAM		(areaC.s)	00297	nop
00050	0C0042C	04C0	reti		-------	----	3	Inst H SRAM		(areaC.s)	00298	reti
00049	0C0042E	0000	nop		-------	----	3	Inst H SRAM		(areaC.s)	00300	nop
00048	008086C	0000	nop		-------	----	4	Inst H SRAM		(area.s)	00062	nop
00047	008086E	6C80	ld.w %r0,0x8		-------	----	1	Inst H SRAM		(area.s)	00063	xld.w %r0,(AREA_8&0
00046	0080870	C024	ext 0x24		-------	----	1	Inst H SRAM		(area.s)	00064	xld.b [TTBR+2],%r0
00045	0080872	C136	ext 0x136		-------	----	1	Inst H SRAM				
00044	0080874	3480	ld.b [%r8],%r0		-------	----	1	Inst H SRAM				

各バスサイクルでトレースメモリに取り込まれるトレース情報は次のとおりです。[Trace]ウィンドウの左側から、表示に対応させて示します。

Cycle: トレースサイクル(10進数)
トレースメモリ内に最後に取り込まれた情報が00000となります。

Address: CPUの命令フェッチアドレス(16進数)
命令フェッチ以外のアクセスでは"--------"が表示されます。

Code: CPUがフェッチした命令コード(16進数)
命令フェッチ以外のアクセスでは"----"が表示されます。

Unassemble: フェッチした命令コードの逆アセンブル内容
命令フェッチ以外のアクセスでは"--------"が表示されます。

Address: CPUがアクセスしたアドレス(16進数)... 命令フェッチ以外
命令フェッチの場合は"--------"が表示されます。

Data: 読み出し/書き込みデータ(16進数)... 命令フェッチ以外
命令フェッチの場合は"----"が表示されます。

Clk: バスオペレーションに使用したクロック数(1～7)
8クロック以上の場合は"V"と表示されます。

Type: バスオペレーションタイプ
Inst: 命令フェッチ、VecR: ベクタリード、DatR: データリード、
DatW: データライト、StkR: スタックリード、StkW: スタックライト、
DmaR: DMAリード、DmaW: DMAライト
アクセスサイズ
B: バイトアクセス、H: ハーフワードアクセス、W: ワードアクセス
メモリタイプ
SRAM、DRAM、BROM(バーストROM)、IRAM(内蔵RAM)、I/O(内蔵I/O)、
DBUG(ICE開発用)、ERR(その他)

TRC: TRCIN端子の入力(Lowの信号入力時にのみ"L"を表示)

T: トレーストリガポイント(行の先頭に表示)
トレーストリガ条件が一致したバスサイクルにのみ表示されます。

File: ソースファイル名(tmコマンドでソース表示を選択した場合のみ)
Line: ソースの行番号(tmコマンドでソース表示を選択した場合のみ)
SourceCode: ソースコード(tmコマンドでソース表示を選択した場合のみ)

(2) **トレースモード**
トレース情報の採取方法の違いにより、2種類のトレースモードが設定されています。

表16.9.7.1 **トレースモード設定コマンド**

機　能	コマンド
トレースモードと条件の設定	tm

1. **ノーマルトレースモード**
プログラム実行中は、すべてのバスサイクルのトレース情報をトレースメモリに取り込みます。したがって、トレースメモリはブレークが起きるまで、直前に実行されたバスサイクルまでの最新情報を常に保持しています。

2. **シングルディレイトリガトレースモード**
シングルディレイトリガトレースモードの場合もプログラムの実行開始によってトレースを開始します。その後、コマンドで指定したトレーストリガ条件が成立すると、そこ(トレーストリガポイント)を基準に、やはりコマンドによる次の設定に従ってトレースを停止します。

- **トレーストリガポイントを**"start"**に設定した場合**
トレーストリガポイントから32768サイクル分のトレース情報を採取後にトレースを停止します。この場合、トレーストリガポイントのトレース情報が、トレースメモリ内の最も古い情報となります。
32768サイクル分のトレースの前にプログラムが停止した場合は、トレースメモリ容量内で、トレーストリガポイント以前のトレース情報も残ります。

図16.9.7.1 "start"**選択時のトレース範囲**

- **トレーストリガポイントを**"middle"**に設定した場合**
トレーストリガポイントから16384サイクル分のトレース情報を採取後にトレースを停止します。この場合、トレーストリガポイントの前後16384サイクルずつのトレース情報がトレースメモリに採取されます。
約16384サイクル分のトレースの前にプログラムが停止した場合は、トレースメモリ容量内で、トレーストリガポイントから16384サイクル以上前のトレース情報も残ります。

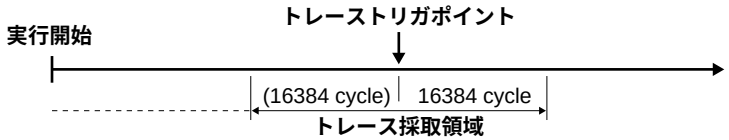


図16.9.7.2 "middle"**選択時のトレース範囲**

- **トレーストリガポイントを**"end"**に設定した場合**
トレーストリガポイントのトレース情報を採取後にトレースを停止します。この場合、トレーストリガポイントのトレース情報が、トレースメモリ内の最も新しい情報となります。
トレーストリガポイントのトレースの前にプログラムが停止した場合は、ノーマルモードと同様です。

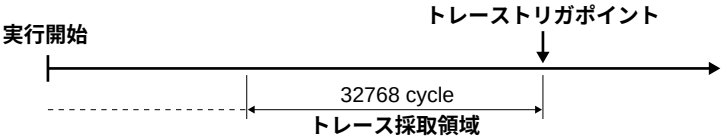


図16.9.7.3 "end"選択時のトレース範囲

シングルディレイトリガトレースの途中でプログラムを停止させた場合でも、次回の実行は新規のトレースを行います。

tmコマンドは上記のモード設定のほかに、トレーストリガ条件(アドレス、データパターン、バスオペレーションタイプ)を指定することができます。

(3) トレース情報の表示と検索

採取したトレース情報はコマンドによって[Trace]ウィンドウに表示させることができます。[Trace]ウィンドウが閉じている場合は[Command]ウィンドウに表示します。[Trace]ウィンドウではスクロールによってトレースメモリの全データを見ることができます。指定したサイクルから表示させることもできます。表示内容は前記のとおりです。ソース表示はオプションです。

表16.9.7.2 トレース情報表示コマンド

機　能	コマンド
トレース情報表示	td

検索条件を指定し、条件に合ったトレース情報のみを表示させることができます。
検索条件は次の2種類から選択できます。
1. アクセスしたメモリアドレス(全メモリ領域を対象にすることも可能)
2. バスオペレーションタイプ

上記条件を指定して検索を行います。条件に合ったトレース情報が見つかると、件数を[Command]ウィンドウに表示します。検索データは[Trace]ウィンドウ(閉じている場合は[Command]ウィンドウ)に表示します。

表16.9.7.3 トレース情報検索コマンド

機　能	コマンド
トレース情報検索	ts

[Trace]ウィンドウはプログラムの実行によりクリアされます。実行後は上記のコマンドで再表示させてください。

(4) トレース情報の保存

トレース情報を[Trace]ウィンドウに表示後(tdコマンド、もしくはtsコマンド実行後)、その中の指定サイクル範囲をファイルに保存することができます。

表16.9.7.4 トレース情報保存コマンド

機　能	コマンド
トレース情報保存	tf

●ICEモードのトレースにおける注意事項

(1) ステップ実行時やブレーク時は、まだ実行していないプリフェッチサイクルの情報も表示されます。また、ソフトウェアPCブレークによりプログラムの実行が中断された際には、ソフトウェアPCブレーク用に挿入したbrk命令のフェッチサイクルも表示されます。(下記の例を参照)

(2) ソフトウェアPCブレークポイントに設定されているアドレスからプログラムを連続実行させると、S5U1C33104Hは内部的にそのアドレスを一度ステップ実行してから連続実行に移ります。このため、ステップ実行時にプリフェッチしたトレースデータが余分に表示されることがあります。(下記の例を参照)

(3) ソースレベルのステップ実行の場合、S5U1C33104H内部ではシングルステップ動作を繰り返し行います。このため、ステップごとのプリフェッチ情報が多数表示されます。

(4) 上記の理由により、実行カウンタによる計測時間もプリフェッチ回数が増える分大きくなります。

(5) 内蔵RAMのリード/ライトは、バスアクセスが検出できないためトレースデータを参照できません。

(6) 高速DMAによるデータ転送中は、データは正しくトレースされません。

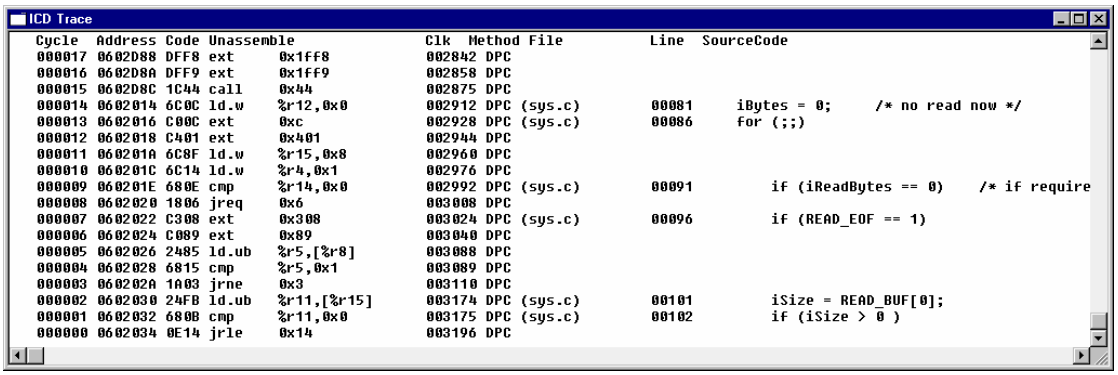
<u>ステップ実行時のプリフェッチデータ表示例</u>

```
>m                  実行プログラムサンプル(!のアドレスにソフトウェアPCブレークを設定)
!00080004 C020 ext     0x20            00010          xld.w      %r8,SP_INI
 00080006 6C08 ld.w    %r8,0x0
 00080008 A081 ld.w    %sp,%r8         00011          ld.w       %sp,%r8    ; set SP
 0008000A 6C08 ld.w    %r8,0x0         00012          ld.w       %r8,GP_INI ; set gp
!0008000C C000 ext     0x0             00013          xcall      main       ; goto main
 0008000E C000 ext     0x0
 00080010 1C02 call    0x2
 00080012 1EF9 jp      0xf9            00014          xjp        BOOT       ; infinity loop
                                            --- main.c ---
                                       00001  /* tst_main.c 1997.2.13 */
                                       00002  /* C main program */
                                       00003
                                       00004  int i;
                                       00005
                                       00006  main()
 00080014 0200 pushn   %r0             00007      {
>g                                 ...0x80004より連続実行
Break by software PC break         ...0x8000cでブレーク
>td                                ...トレースデータを表示
 Cycle Address Code Unassemble              Address Data Clk  Type    TRC
 00007 0080004 C020 ext     0x20            ------- ---- 1 Inst H SRAM
 00006 0080006 6C08 ld.w    %r8,0x0         ------- ---- 1 Inst H SRAM
 00005 0080008 A081 ld.w    %sp,%r8         ------- ---- 1 Inst H SRAM  ...ステップによる
 00004 008000A 6C08 ld.w    %r8,0x0         ------- ---- 1 Inst H SRAM  ...プリフェッチ(2)
 00003 0080008 A081 ld.w    %sp,%r8         ------- ---- 1 Inst H SRAM
 00002 008000A 6C08 ld.w    %r8,0x0         ------- ---- 1 Inst H SRAM
 00001 008000C C400 brk                     ------- ---- 1 Inst H SRAM  ...ソフトブレーク命令(1)
 00000 008000E C000 ext     0x0             ------- ---- 1 Inst H SRAM
>s
>td
 Cycle Address Code Unassemble              Address Data Clk  Type    TRC
 00005 008000C C000 ext     0x0             ------- ---- 1 Inst H SRAM
 00004 008000E C000 ext     0x0             ------- ---- 1 Inst H SRAM
 00003 0080010 1C02 call    0x2             ------- ---- 1 Inst H SRAM
 00002 0080012 1EF9 jp      0xf9            ------- ---- 1 Inst H SRAM
 00001 0080014 0200 pushn   %r0             ------- ---- 2 Inst H SRAM
 00000 0080016 6C0B ld.w    %r11,0x0        ------- ---- 1 Inst H SRAM
>s
>td
 Cycle Address Code Unassemble              Address Data Clk  Type    TRC
 00002 0080014 0200 pushn   %r0             ------- ---- 1 Inst H SRAM  ...実行
 00001 0080016 6C0B ld.w    %r11,0x0        ------- ---- 1 Inst H SRAM  ...プリフェッチ(1)
 00000 0080018 C000 ext     0x0             ------- ---- 1 Inst H SRAM  ...プリフェッチ(1)
>
```

●ICDモードのトレース機能

(1) トレースメモリとトレース情報

S5U1C33000Hは131072サイクル分のトレースメモリを内蔵しており、S1C33チップが出力するデバッグ用信号等を利用して命令実行サイクルの情報を取得します。



ICD Trace

Cycle	Address	Code	Unassemble		Clk	Method	File	Line	SourceCode
000017	0602D88	DFF8	ext	0x1ff8	002842	DPC			
000016	0602D8A	DFF9	ext	0x1ff9	002858	DPC			
000015	0602D8C	1C44	call	0x44	002875	DPC			
000014	0602014	6C0C	ld.w	%r12,0x0	002912	DPC	(sys.c)	00081	iBytes = 0; /* no read now */
000013	0602016	C00C	ext	0xc	002928	DPC	(sys.c)	00086	for (;;)
000012	0602018	C401	ext	0x401	002944	DPC			
000011	060201A	6C8F	ld.w	%r15,0x8	002960	DPC			
000010	060201C	6C14	ld.w	%r4,0x1	002976	DPC			
000009	060201E	680E	cmp	%r14,0x0	002992	DPC	(sys.c)	00091	if (iReadBytes == 0) /* if require
000008	0602020	1806	jreq	0x6	003008	DPC			
000007	0602022	C308	ext	0x308	003024	DPC	(sys.c)	00096	if (READ_EOF == 1)
000006	0602024	C089	ext	0x89	003040	DPC			
000005	0602026	2485	ld.ub	%r5,[%r8]	003088	DPC			
000004	0602028	6815	cmp	%r5,0x1	003089	DPC			
000003	060202A	1A03	jrne	0x3	003110	DPC			
000002	0602030	24FB	ld.ub	%r11,[%r15]	003174	DPC	(sys.c)	00101	iSize = READ_BUF[0];
000001	0602032	680B	cmp	%r11,0x0	003175	DPC	(sys.c)	00102	if (iSize > 0)
000000	0602034	0E14	jrle	0x14	003196	DPC			

各サイクルのトレース情報は次のとおりです。[Trace]ウィンドウの左側から、表示に対応させて示します。

Cycle: トレースサイクル(10進数)
トレースメモリ内に最後に取り込まれた情報が000000となります。
Address: CPUの命令実行アドレス(16進数)
Code: CPUが実行した命令コード(16進数)
Unassemble: 命令コードの逆アセンブル内容
Clk: 実行クロック数
デフォルトではトレース開始からのクロック数が累積されますが、各実行命令ごとのクロック数表示に変更することもできます。
Method: トレース解析(実行アドレス取得)方法
- SPC: 初期PCによる解析
- TRG: トリガアドレスによる解析
- DPC: DPCO信号による解析
- RET: call文、ret文による解析
- MAP: マップ情報による解析
- RTI: reti文による解析
- ---: 解析不能

File: 実行命令を含むソースファイル名
Line: ソース行番号
SourceCode: ソースコード

(2) トレースモードとトレース条件

トレース範囲により、2種類のトレースモードが設定されています。

表16.9.7.5 トレースモード設定コマンド

機 能	コマンド
トレースモードと条件の設定	tm

1. 全範囲トレースモード

プログラムの実行開始によりトレースを開始します。トレース条件を"オーバーライトあり"とした場合は、プログラムの実行を中断するまでトレース情報をトレースメモリに取り込みます。トレースメモリがいっぱいになった場合は古いデータから上書きされます。トレース条件を"オーバーライトなし"とした場合は、トレースメモリがいっぱいになった時点でトレースを終了します。

2. **範囲トレースモード**
トレース範囲を指定し、プログラムがその間を実行した場合にのみトレース情報を取り込みます。トレース終了アドレスでプログラムの実行をブレークさせることもできます。また、時間測定条件(全範囲測定または範囲測定)も指定可能です。

上記のトレースモードのほかに、トレース情報のクロック数(Clk)を命令個別にカウントするか、累計するかについても選択できます。

(3) トレース情報の表示と検索

採取したトレース情報はコマンドによって[Trace]ウィンドウに表示させることができます。[Trace]ウィンドウが閉じている場合は[Command]ウィンドウに表示します。

表16.9.7.6 トレース情報表示コマンド

機　能	コマンド
トレース情報表示	td

検索コマンドにより、指定のアドレスを実行したサイクルとその前後のトレース情報のみを表示させることもできます。検索データは[Trace]ウィンドウ(閉じている場合は[Command]ウィンドウ)に表示されます。

表16.9.7.7 トレース情報検索コマンド

機　能	コマンド
トレース情報検索	ts

[Trace]ウィンドウはプログラムの実行によりクリアされます。実行後は上記のコマンドで再表示させてください。

(4) プログラム実行中のトレース

S5U1C33000Hでは、プログラムの実行を中断することなくトレース表示ができます。プログラム実行中に[Display trace]ボタンをクリックするとトレース動作を中止し、そこまでのトレースデータを[Trace]ウィンドウに表示します。これにより中止したトレース動作は、[Resume trace]ボタンをクリックすることで再開します。

[Display trace]**ボタン** [Resume trace]**ボタン**

プログラム実行中にtsおよびtfコマンドを使用することはできません。
[Display trace]ボタンは、ブレーク中にtdコマンドの代わりに使用することもできます。

(5) トレース情報の保存

トレース情報を[Trace]ウィンドウに表示後(tdコマンド、もしくはtsコマンド実行後)、その中の指定サイクル範囲をファイルに保存することができます。

表16.9.7.8 トレース情報保存コマンド

機　能	コマンド
トレース情報保存	tf

●ICDモードのトレース動作と注意事項

ICDモードでは以下の方法でトレースを行っています。

S5U1C33000Hにはターゲット CPUから以下の4本の信号が入力されます。
DST0, DST1, DST2 .....逐次命令実行、相対分岐実行、絶対分岐実行、アイドルなど、CPUの実行状況を伝えるステータス信号です。
DPCO.....絶対分岐時、シリアルに分岐先のPC値を出力します。

S5U1C33000Hは、この4ビットの情報をCPUクロックに同期して最大128Kクロック分取り込みます。
db33はこの情報と、db33内部の逆アセンブル情報を基に次のようなフロー解析を行い、PC値を取得しています。
・DST0~2 = 逐次命令実行: +1命令
・DST0~2 = 相対分岐: 逆アセンブル情報から分岐命令数を計算
・DST0~2 = 絶対分岐: DPCO情報を参照し、分岐先を確定

ただし、トレースの起点とそれに対応したPC値が確定しないと解析ができません。
そこで、db33は以下の手法によりPC値を確定しています。使用された手法はトレース情報内のMethodの表示内容で知ることができます。

Method: SPC 実行開始時のPC値が分かっている場合(全範囲トレース&オーバーライトなし)
Method: DPC 絶対分岐で完全なDPCO情報が得られたことにより確定
Method: MAP 絶対分岐で不完全なDPCO情報入力した場合はマップ情報を補完して確定
なお、絶対分岐には以下の命令と割り込みへの分岐が該当します。
call %rb, call.d %rb, jp %rb, jp.d %rb, ret, ret.d, reti, int
Method: TRG 範囲トレースではトリガアドレスを使用して確定
Method: RET call文とret文が1対1に対応していることを前提として確定
Method: RTI 割り込み発生とretiが1対1に対応していることを前提として確定

このため、以下に示すような制約があります。

(1) オーバーライトありの場合の制約
相対分岐を繰り返すようなループでトレースを行うと、PC値が確定するまでトレース解析ができません。
解決策として、8ビットタイマなどで数msごとに割り込みを発生させ、DPCO情報を出力させて、トレースを解析可能にする方法があります。
("cc33¥sample¥icdtrc"のサンプル参照)

(2) 範囲トレースでの制約
通常、ハードウェアPCブレークは2本使用可能ですが、範囲トレースモードはそれらをトレーストリガとして使用するため、ハードウェアPCブレークとしては使用できなくなります。
範囲トレースを解除すると、使用できる状態に戻ります。

(3) オーバーライトなしの場合の制約
ソフトウェアPCブレークポイントからプログラムの実行を開始(go)すると、S5U1C33xxxHファームウェアは次のような処理を行います。
1. 実行開始アドレスのソフトウェアPCブレークを解除
2. 最初の命令をステップ実行
3. 再度ソフトウェアPCブレークを設定
4. 続く命令を連続実行
このため、db33は実行開始時の初期PC値を解析に使用できなくなくなります。
ソフトウェアPCブレークポイントで停止中のプログラムの実行を再開させる場合は、ステップ実行などでPC値を一旦ソフトウェアPCブレークポイントから外してから連続実行させてください。

以下は、全モード共通の制約です。

(4) 絶対分岐での制約

絶対分岐が27クロック以下で繰り返されると、PC値を完全に確定できません。マップ情報や、call～retのネスト情報によりリカバリ処理を試みますが、解析不能となる場合があります。

(5) 実行プログラムの制約

デバッガは、lfまたはlhコマンドでプログラムをロードする際に、内部にもその情報を保存し、その逆アセンブル情報をもとに解析を進めます。
以下のようなケースでは内部の解析情報とターゲットのプログラムに食い違いがでます。
・実行プログラムがROMにある場合
・プログラムが動的に実行ルーチンをコピー/移動する場合
・db33のコマンド等でプログラム領域を変更した場合
このような場合は、rmコマンドで必要な部分をターゲットからdb33に読み込んでください。

(6) シミュレーテッドI/O

シミュレーテッドI/Oは、ソフトウェアPCブレーク、ステップ実行、ソースステップ実行機能を内部で使用します。トレース実行とは併用しないでください。

以下は、S5U1C33000Hハードウェアにかかわる注意点です。

(7) S5U1C33000Hの設定

ICDモードでトレース機能を使用するには、S5U1C33000HのディップスイッチSW4をOPEN(上)に設定してください。また、ターゲットボードとS5U1C33000Hは、10ピンケーブルを使用してトレースに必要な信号(DST0、DST1、DST2、DPCO)をすべて接続しておく必要があります。

(8) S5U1C33000Hトレースの上限周波数

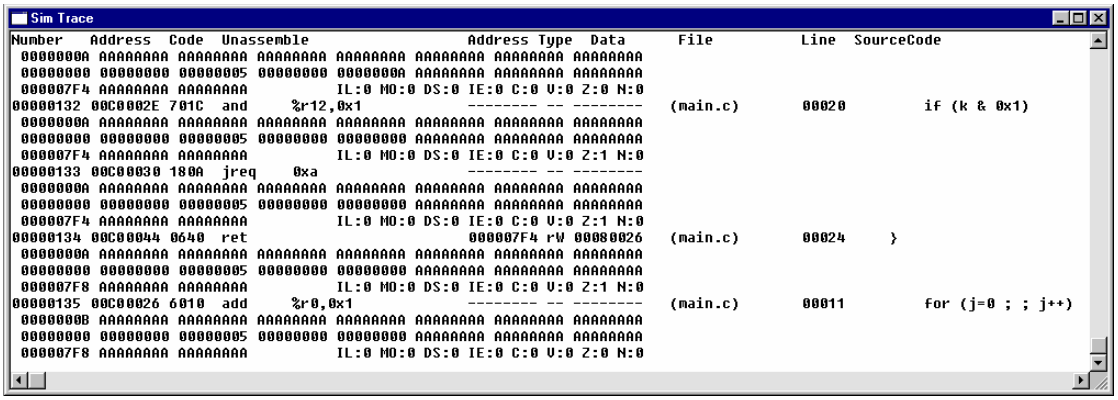
S5U1C33000Hによるトレースは50MHzが動作の上限となります。それ以上ではデータ化けを引き起こします。
50MHz以上のCPU動作クロックを使用する場合は、DIP SW4でトレース機能をOFFにしてください。また、S1C33チップのBCU(バス)の速度をCPUコアの1/2以下に設定して(#X2SPD端子で設定)ください。
トレース以外の機能はバスと同じ速度で動作し、40MHzが上限となります。

●シミュレータモードのトレース機能

シミュレータモードではトレース機能のON/OFF、表示方法およびファイル書き出しが指定できます。トレース機能をONすると、1命令実行ごとにトレース結果を表示あるいはファイルに保存します。

表16.9.7.9 トレースモード設定コマンド

機　能	コマンド
トレースモードの ON/OFF	tm



シミュレータモードで表示されるトレース情報は次のとおりです。

<各トレース情報の1行目>

Number: 実行命令No.(10進数)
CPUをリセットまたはトレースをONにしてからの実行命令数
Address: 実行命令アドレス(16進数)
Code: 命令コード(16進数)
Unassemble: 実行命令の逆アセンブル内容
Address: アクセスしたメモリアドレス(16進数)
Type: バスオペレーションタイプ
rB: バイトデータリード、rH: ハーフワードデータリード、rW: ワードデータリード
wB: バイトデータライト、wH: ハーフワードデータライト、wW: ワードデータライト
Data: 読み出し/書き込みデータ(16進数)
File: ソースファイル名(tmコマンドでソース表示を選択した場合のみ)
Line: ソースの行番号(tmコマンドでソース表示を選択した場合のみ)
SourceCode: ソースコード(tmコマンドでソース表示を選択した場合のみ)

<各トレース情報の2～4行目>

これらの行は、tmコマンドでレジスタ内容の表示を選択した場合にのみ表示されます。
各レジスタ値は次のような配置で表示されます。

R0	R1	R2	R3	R4	R5	R6	R7
R8	R9	R10	R11	R12	R13	R14	R15
SP	AHR	ALR		PSR(フラグ個別に表示)			

表示はtmコマンドでウィンドウへの表示を選択した場合に[Trace]ウィンドウに表示されます。
[Trace]ウィンドウが閉じている場合は[Command]ウィンドウに表示されます。
ファイルへの保存を選択した場合は、指定のファイルに出力され、ウィンドウには表示されません。
ICEモードとは異なり、特に表示のためのコマンドを入力する必要はありません。プログラムの連続実行、ステップ実行に従って自動的に表示されます。
なお、[Trace]ウィンドウでは最新情報から最大255サイクル前までのトレース情報が確認できます。それ以前のものは消去されます。

16.9.8 シミュレーテッドI/O

db33のシミュレーテッドI/O機能により、シリアルインタフェースなどの外部入出力機能を標準入出力(stdin、stdout)あるいはファイルの入出力により評価することができます。

表16.9.8.1 シミュレーテッドI/O設定コマンド

機　能	コマンド
入力設定	stdin
出力設定	stdout

●stdinによる入力

stdinコマンドでは、以下の条件を設定します。
・ブレークアドレス
・入力バッファアドレス(バッファサイズは65バイト固定)
・入力対象　[Simulated I/O]ウィンドウまたはファイル

これらの条件を設定後、プログラムを連続実行させます。

[Simulated I/O]ウィンドウを選択した場合

db33は設定したブレークアドレスになると[Simulated I/O]ウィンドウを開き、キーボードからデータが入力されるのを待ちます。データを入力し、[Enter]キーを押すと、db33は入力されたデータを指定されたバッファに書き込み、プログラムの実行をブレークしたアドレスから再開します。

ファイルを選択した場合

ファイルを選択した場合は、ブレーク時に指定のファイルからデータをバッファに入力します。その後、プログラムの実行をブレークしたアドレスから再開します。[Simulated I/O]ウィンドウは開きません。

●stdoutによる出力

stdoutコマンドでは、以下の条件を設定します。
・ブレークアドレス
・出力バッファアドレス(バッファサイズは65バイト固定)
・出力対象　[Simulated I/O]ウィンドウ、ファイルまたは両方

これらの条件を設定後、プログラムを連続実行させます。

[Simulated I/O]ウィンドウを選択した場合

db33は設定したブレークアドレスになると[Simulated I/O]ウィンドウを開き、バッファに設定されている内容を[Simulated I/O]ウィンドウに表示します。その後、プログラムの実行をブレークしたアドレスから再開します。

ファイルを選択した場合

ファイルを選択した場合は、ブレーク時にバッファの内容を指定のファイルに出力します。その後、プログラムの実行をブレークしたアドレスから再開します。[Simulated I/O]ウィンドウは開きません。

データを[Simulated I/O]ウィンドウとファイルの両方に出力させることもできます。

●プログラム上の対応

上記のシミュレーテッドI/O機能を使用するには、プログラム内に以下の定義が必要です。

入出力バッファの定義

db33がデータの入出力に使用するグローバルなバッファを次のような形式で定義しておきます。

入力バッファの定義: unsigned char READ_BUF[65] (.comm READ_BUF 65)
出力バッファの定義: unsigned char WRITE_BUF[65] (.comm WRITE_BUF 65)

バッファ名には、シンボル名の規定にそった任意の名称を使用することができます。
バッファサイズは65バイトに固定してください。
stdin、stdoutコマンド実行時には、このシンボル名でバッファアドレスを指定します。

データが入力されると、READ_BUF[0]には実際に入力されたデータのサイズ(1～64)が入ります。EOFが入力された場合READ_BUF[0]は0となります。入力データはREAD_BUF[1]以降に格納されます。
データを出力する場合はWRITE_BUF[0]に出力するデータのサイズ(1～64)を書き込み、出力データをWRITE_BUF[1]以降に書き込みます。EOFを出力する場合はWRITE_BUF[0]に0を書き込みます。
db33とプログラム間で、最大64バイトのデータ列として入出力を行えます。

データ更新用グローバルラベルの定義

db33が入力バッファにデータを取り込む位置、および出力バッファのデータを出力する位置にそれぞれ次のようなグローバルラベルを定義しておきます。

```
入力位置:  .global  READ_FLASH
           READ_FLASH:
出力位置:  .global  WRITE_FLASH
           WRITE_FLASH:
```

ラベルには任意の名称が使用できます。
stdin、stdoutコマンド実行時には、このシンボル名でブレークアドレスを指定します。

Cソースでは、標準入出力ライブラリ関数の下位レベル関数write、read(8.4節参照)内にこれらのラベルを定義します。

実際のプログラム例については、サンプルプログラムやデバッガのコマンドファイルがsample¥simio¥ディレクトリにインストールされていますので、参照してください。

db33はREAD_FLASHでブレークすると[Simulated I/O]ウィンドウに入力されるデータまたはファイルのデータを読み込み、定義されている入力バッファにロードします。その後、プログラムの実行を再開します。
WRITE_FLASHでブレークした場合、db33は出力バッファに書き込まれているデータを[Simulated I/O]ウィンドウまたはファイルに出力し、プログラムの実行を再開します。

●注意事項

stdin、stdoutコマンドで指定するブレークアドレスはソフトウェアブレークアドレスと重複することはできません。
内部的にはソフトウェアブレークを使用しているため、ターゲットボード上のROM領域は指定できません。
入出力にはASCIIキャラクタのみを使用してください。バイナリデータ(特に0x0や0x1A)では誤動作します。
stdin、stdoutにより入出力を行う部分はgoコマンドで連続実行させ、ステップ実行は行わないでください。また、その近辺ではブレークが発生しないようにしてください。
また、ICE、ICDで使う場合は、mdコマンドでオンザフライを使用しないと(0に設定)すると、より高速に表示されます。オンザフライを使う場合は表示間隔を10にした方が、デフォルトの5よりも高速に表示されます。

16.9.9 フラッシュメモリの操作

db33には、ターゲットボード上のフラッシュメモリを操作する機能およびS5U1C33104Hに搭載されているフリーラン用フラッシュメモリを操作する機能があります。

●ターゲットボード上のフラッシュメモリ操作

db33には、S1C33チップに内蔵あるいはターゲットボード上のフラッシュメモリに対してデータ書き込み/消去を行うためのユーティリティとコマンドが用意されています。
ICE(注)、ICD、デバッグモニタモードのデバッグ環境すべてで使用することができます。

表16.9.9.1 フラッシュメモリ操作コマンド

機　能	コマンド
フラッシュメモリの設定	fls
フラッシュメモリの消去	fle

注: S5U1C33104H**で使用する場合、ファームウェアが**Ver. 2.0**以上である必要があります。**
S5U1C33104H**の出荷時は、**Ver. 1.x**のファームウェアが実装されていますので、**"cc33¥utility¥ice33v20¥"**ディレクトリ内のプログラムで**S5U1C33104H**のファームウェアをアップデートしてください。手順についてはアップデータの**"readme.txt"**を参照してください。**

フラッシュメモリへの書き込みは以下の手順で行います。
以下の例は、"cc33¥sample¥dmt33004¥led2.cmd"より引用しています。
なお、より詳しい内容については、フラッシュサポートユーティリティfls33の"readme.txt"を参照してください。(fls33および"readme.txt"は"cc33¥utility¥fls33¥fls33vXX.exe"によりインストールできます。)

(1) **フラッシュルーチンのロード**

lfコマンドで、フラッシュルーチン(消去ルーチン、書き込みルーチン)を内蔵RAM等へロードします。
例:

```
lf flsh\am29f800.srf     ;load flash erase and write routine to IRAM area
```

このルーチンで、実際の消去、書き込みを行います。
セイコーエプソン提供のルーチンは、内蔵RAMの0x40～0x7ff(2KB)を使用します。

注: **フラッシュルーチンは、セイコーエプソン提供のものを使用するか、独自のものを作成してください。**
セイコーエプソン提供のルーチンは、"cc33¥utility¥fls33¥fls33.exe"**を実行してインストールします。**AMD**系フラッシュを中心に用意されています。ソースコード等も含まれていますので、必要により修正して使用してください。**

(2) **フラッシュの設定**

フラッシュメモリのスタートアドレス、エンドアドレス、(1)でロードした消去ルーチン、書き込みルーチンのエントリアドレスをdb33に設定します。
例:

```
fls           ;flash set command
1             ;1:set 2:clear
200000        ;flash area start address is 0x200000
2fffff        ;flash area end address is 0x2fffff
FLASH_ERASE   ;flash erase routine top address
FLASH_LOAD    ;flash load routine top address
```

(3) フラッシュの消去

フラッシュメモリを全消去、あるいはセクタ消去します。
フラッシュの内容は0xffに変わります。
例:

```
fle          ;flash erase command
200000       ;flash contorol regester is 0x200000
0            ;erase start block 0:all area 1-19:section
0            ;erase end block 1-19:sector if start block is 0 ,then this parameter is ignored.
```

まず、フラッシュメモリのコントロールレジスタのアドレスを指定します。通常は、フラッシュの先頭アドレスに一致します。
次に消去するセクタの範囲を指定します。0と0で全消去、1と3で1から3までのセクタ消去となります。セクタ数、サイズはデバイスにより異なります。
flsコマンドの次に必ずfleコマンドを実行してください。フラッシュを消去したくない場合は、セクタ範囲に-1と0を指定することにより消去動作以外の処理のみを行います。

(4) フラッシュへの書き込み

lfまたはlhコマンドで、フラッシュメモリにデータを書き込みます。
例:

```
lf led2.srf              ;load to 0x200000(flash)
```

(1)のflsコマンドで設定したスタート/エンドアドレスに入るデータは、書き込みルーチンに渡してフラッシュ書き込みを行い、それ以外は通常どおり、RAMへの書き込みとして処理されます。書き込みがタイムアウトとなった場合や、フラッシュメモリが消去されていない(0xffでない)場合はエラーが返ります。

lfやlh以外のewやehコマンドも書き込みに使用できます。8ビットデータ幅のフラッシュメモリでは、ebコマンドも使用できます。

●S5U1C33104Hフリーラン用フラッシュメモリの操作

S5U1C33104Hはフラッシュメモリを内蔵しています。このメモリはコマンドによってS5U1C33104Hの内蔵ROMエミュレーションメモリとデータのやり取りが行えるようになっています。
フラッシュメモリはS5U1C33104Hの電源をOFFにしてもデータが消えません。電源OFFの前にデバッグ中のプログラム、データ、オプションデータおよびマップ情報をフラッシュメモリに書き込んでおくことにより、次回はそれを読み出してデバッグを続けることができます。また、S5U1C33104Hをフリーランモード(S5U1C33104H単体でプログラムを実行するモード)で動作させる場合も、そのプログラムをフラッシュメモリに書き込んでおく必要があります。

フラッシュメモリに対しては以下の操作が行えます。

(1) **フラッシュメモリの読み出し**
フラッシュメモリのデータを内蔵ROMエミュレーションメモリにロードします。

(2) **フラッシュメモリへの書き込み**
内蔵ROMエミュレーションメモリのデータをフラッシュメモリにセーブします。また、パラメータファイルの内容も必要に応じて書き込みます。この書き込み以降、フラッシュメモリを読み出し/書き込み禁止に設定することもできます。

(3) **フラッシュメモリの消去**
フラッシュメモリの内容をすべて消去します。

(4) **フラッシュメモリのマップ情報の表示**
フラッシュメモリのマップ情報、チップ名、パラメータファイルのバージョンなどを表示します。

S5U1C33104Hフラッシュメモリの操作はICEモード時にのみ可能です。

表16.9.9.2　S5U1C33104H**フラッシュメモリ操作コマンド**

機　能	コマンド
フラッシュメモリの読み出し	lfl
フラッシュメモリへの書き込み	sfl
フラッシュメモリの消去	efl
フラッシュメモリマップ情報の表示	maf

※ S5U1C33104H**のフリーラン**

S5U1C33104Hのフリーラン(S5U1C33104H単独でプログラム実行)には、フラッシュメモリに書き込まれたデータが使用されます。フリーランを行う場合は、プログラム、データ、オプションデータをすべてフラッシュメモリに書き込んでください。なお、内蔵ROM以外のデータは保存されません。
S5U1C33104HはICE/RUNスイッチをRUN側にセットして電源をONするとフリーランを開始します。フリーラン中は、パラメータファイルで設定されているプログラム領域、データ領域に従ったマップブレークが有効です。マップブレーク時にはS5U1C33104H上のPC LEDが止まり、EMU LEDが消灯します。その他のブレーク設定はフラッシュメモリには書き込まれませんので無効です。

16.9.10 その他の機能

ここまで説明した主要な機能のほかに、db33は以下の機能をコマンドでサポートしています。詳細は各コマンドの説明を参照してください。

- **マップ表示**(ma**コマンド**)
  マップ情報、チップ名、パラメータファイルのバージョン等を表示します。
- **形式変換表示**(ct**コマンド**)
  入力した数値/文字列を異なる形式に変換して返します。
- **拡張命令への逆変換**(ext**コマンド**)
  ext命令で即値拡張された命令のアドレスを指定することにより、拡張後の即値を含む命令エクステンダの拡張命令形式で表示させることができます。
- **ユーザ関数のコール**(call**コマンド**)
  db33からユーザ関数をコールすることができます。たとえば、S1C33 FamilyリアルタイムOS内の関数をコールして、各種デバッグ情報を取得することができます。その詳細については、"S1C33 Family Realtime OSオペレーションマニュアル"を参照してください。

16.9.11 ビッグエンディアンへの対応について

Cコンパイラからリンカまでのツールおよびライブラリはリトルエンディアンにのみ対応しています。
Cコンパイラでは、ビッグエンディアン領域にロードできるsrfファイルを作成できませんので注意してください。ただし、データについては、デバッガでビッグエンディアンに対応できます。

●ビッグエンディアンの指定方法

デバッガにビッグエンディアン領域の情報を設定するには、パラメータファイルのマップ情報で指定します。ビッグエンディアンに設定したい領域については、<終了アドレス>の後ろに"B"を記述します。この記述がない場合はリトルエンディアン領域になります。ただし、開発するS1C33チップがビッグエンディアンに対応した機種である必要があります。また、内蔵のROM、RAM、I/Oはビッグエンディアンに設定することはできません。
db33起動時にこのパラメータファイルを読み込むとともに、S1C33チップ上のエンディアンコントロールレジスタ(テクニカルマニュアル参照)も正しく設定してください。
シミュレータモードの場合はパラメータファイルのみでエンディアンが決定します。
パラメータファイルについては、16.11項を参照してください。

●デバッグコマンドの動作

(1) db, dh, dw, fb, fh, fw, mv, mvh, mvw, eb, eh, ew**コマンド**
コマンドのデータタイプに従ってバイト、ハーフワード、ワード単位の読み出し/書き込みを行いますので、エンディアンに対応した動作、表示を行います。

(2) sy, sa, sw(@)**コマンド**
内部的には読み出しをデータタイプにかかわらずバイト単位で行い、読み出し後に指定のデータタイプに構成します。このときに、エンディアンとデータタイプを調べ、必要であればデータをスワップして表示します。BCUとパラメータファイルの設定が異なっていると、正しく表示されません。

(3) lh, lf**コマンド**
エンディアンに対応したデータスワップを行い、ハーフワード単位でデータを書き込みます。
そのため、Cコンパイラで作成したプログラムをビッグエンディアン領域に正しくロードできません。

(4) sfl, lfl**コマンド**
sflコマンドは、エンディアン情報を保存しません。
lflコマンドはS5U1C33104Hから読み出したマップ情報にパラメータファイルのエンディアン情報を加え、実際のマップ情報を作成します。したがって、sflで保存したときと同じパラメータファイルでデバッガを起動する必要があります。

(5) **監視データ**
wコマンドで設定した監視データはワード単位で読み込みと表示が行われますので、エンディアンに対応して表示されます。

●シミュレータモードでの相違点

シミュレータモードでは、TTBRレジスタを含むアドレスをビッグエンディアンに設定できます。
その場合、トラップテーブルベースアドレスは次のように設定してください。
リトルエンディアンで0x48134番地に書き込む内容は0x48137番地に書き込むこと
リトルエンディアンで0x48135番地に書き込む内容は0x48136番地に書き込むこと
リトルエンディアンで0x48136番地に書き込む内容は0x48135番地に書き込むこと
リトルエンディアンで0x48137番地に書き込む内容は0x48134番地に書き込むこと

16.10 コマンドリファレンス

16.10.1 コマンド一覧

表16.10.1.1 コマンド一覧表

分　類	コマンド	機　能	モード対応				P No.
			ICD	ICE	SIM	MON	
メモリ操作	fb	領域のフィル (バイト単位)	●	●	●	●	331
	fh	領域のフィル (ハーフワード単位)	●	●	●	●	332
	fw	領域のフィル (ワード単位)	●	●	●	●	333
	db	メモリダンプ (バイト単位)	●	●	●	●	334
	dh	メモリダンプ (ハーフワード単位)	●	●	●	●	336
	dw	メモリダンプ (ワード単位)	●	●	●	●	338
	df	ファイルへのメモリダンプ	●	●	●	●	340
	eb	データ入力 (バイト単位)	●	●	●	●	341
	eh	データ入力 (ハーフワード単位)	●	●	●	●	342
	ew	データ入力 (ワード単位)	●	●	●	●	343
	mv	領域のコピー (バイト単位)	●	●	●	●	344
	mvh	領域のコピー (ハーフワード単位)	●	▲注	●	●	345
	mvw	領域のコピー (ワード単位)	●	▲注	●	●	346
	w	監視データアドレス設定	●	●	●	●	347
	rm	ターゲットメモリリード	●	—	—	—	348
	mc	srf33ファイルとメモリの比較	●	●	●	●	349
レジスタ操作	rd	レジスタ表示	○	○	○	○	350
	rs	レジスタ変更	●	●	●	●	351
プログラム実行	g	連続実行	●	●	●	●	352
	s	ステップ実行	●	●	●	●	354
	n	スキップ付きステップ実行	●	●	●	●	356
	o	ステップアウト	●	●	●	●	357
	call	ユーザ関数コール(引数: 10進数)	●	●	●	●	358
	callh	ユーザ関数コール(引数: 16進数)	●	●	●	●	358
	callmd	コールモードの設定	●	●	●	●	360
CPUリセット	rstc	コールドリセット	●	●	●	●	361
	rsth	ホットリセット	●	●	●	●	362
割り込み	int	割り込み発生	—	—	●	—	363
ブレーク	bp	ソフトウェアPCブレークポイントの設定/解除	●	●	●	●	364
	bs	ソフトウェアPCブレークポイントの設定	●	●	●	●	368
	bc	ソフトウェアPCブレークポイントの解除	●	●	●	●	369
	bh	ハードウェアPCブレークポイント1の設定	●	●	●	●	370
	bhc	ハードウェアPCブレークポイント1の解除	●	●	●	●	372
	bh2	ハードウェアPCブレークポイント2の設定	●	▲	●	●	373
	bhc2	ハードウェアPCブレークポイント2の解除	●	▲	●	●	375
	bph	ハードウェアPCブレークポイントの設定/解除	●	▲	●	●	376
	bd	データブレークの設定	●	●	●	●	380
	bsq	シーケンシャルブレークの設定	—	●	—	—	382
	ba	エリアブレークの設定	■	—	—	—	385
	bb	バスブレークの設定	■	—	—	—	387
	bl	全ブレーク条件の表示	○	○	○	○	389
	bac	全ブレーク条件の解除	○	○	○	○	390
プログラム表示	u	逆アセンブル表示	●	●	●	●	391
	sc	ソース表示	●	●	●	●	393
	m	逆アセンブル&ソース表示	●	●	●	●	395
	ss	ソース内の文字列検索	●	●	●	●	397
シンボル情報	sy	シンボル一覧	●	●	●	●	398
	sa	[Symbol]ウィンドウへの登録	●	●	●	●	403
	sd	[Symbol]ウィンドウからの削除	●	●	●	●	406
	sw	シンボル情報の表示	●	●	●	●	407
ファイル読み込み	lf	srf33形式ファイルの読み込み	●	●	●	●	410
	lh	モトローラS3形式ファイルの読み込み	●	●	●	●	412
	ld	デバッグ情報の読み込み	●	●	●	●	413
フラッシュメモリ操作	fls	ターゲットフラッシュメモリ設定	●	▲	—	●	414
	fle	ターゲットフラッシュメモリ消去	●	▲	—	●	415
	lfl	S5U1C33104Hフラッシュメモリの読み出し	—	●	—	—	416
	sfl	S5U1C33104Hフラッシュメモリへの書き込み	—	●	—	—	417
	efl	S5U1C33104Hフラッシュメモリ消去	—	●	—	—	418
	maf	S5U1C33104Hフラッシュメモリマップ情報の表示	—	●	—	—	419
トレース	tm	トレースモードの設定	○	○	○	—	420
	tm2	シミュレータトレースモードの設定	—	—	●	—	426
	td	トレース情報の表示	○	○	—	—	427
	ts	トレース情報の検索	○	○	—	—	431
	tf	トレース情報の保存	○	○	—	—	432
シミュレーテッドI/O	stdin	データ入力シミュレーション	●	●	●	●	433
	stdout	データ出力シミュレーション	●	●	●	●	434
その他	com	コマンドファイル実行	●	●	●	●	435
	cmw	コマンドファイルのウェイト付実行	●	●	●	●	436
	log	ログ出力ON/OFF	●	●	●	●	437
	od	オプションデータダンプ	—	●	—	—	438
	ct	データ形式変換表示	●	●	●	●	439
	ext	拡張命令への逆変換	●	●	●	●	441
	ma	マップ情報の表示	●	●	●	●	443
	md	モード設定	○	○	○	○	444
	alias	コマンドエイリアス	●	●	●	●	446
	exit	終了	●	●	●	●	448
	?	コマンドusageの表示	●	●	●	●	449

モード対応: ● = 使用可(モードによらず同一機能/表示)　○ = 使用可(モードにより機能/表示が異なる)
▲ = S5U1C33104HファームウェアVer. 2.0以上で対応　— = 使用不可　■ = S5U1C33xxxM使用の場合のみ
注: S5U1C33104HファームウェアVer. 2.0未満ではバイト単位で実行します。

16.10.2 メモリ操作コマンド

fb (fill byte) [ICD / ICE / SIM / MON]

■ 機　能

指定のメモリ領域を指定のバイトデータで書き換えます。

■ 書　式

(1) **fb** (ガイダンスモード)

(2) **fb <address1> <address2> <data>** (直接入力モード)

<address1>: 指定範囲先頭アドレス(16進数、またはシンボル)
<address2>: 指定範囲終了アドレス(16進数、またはシンボル)
<data>: 書き込みデータ(16進数)
条件: 0≦address1≦address2≦0xffffffff、0≦data≦0xff

■ 入力例

書式1)
```
>fb↵
Start address ? : 0000000↵   ...先頭アドレスを入力(シンボルも使用可能)
End   address ? : 000000f↵   ...終了アドレスを入力(シンボルも使用可能)
Data  pattern ? : 1↵         ...書き込みデータを入力
>
```
※[Enter]キーのみの入力でコマンドの実行を中止できます。

書式2)
```
>fb 0000000 000000f 1↵
>
```
どちらの例も、アドレス0x0～0xfのメモリ領域をすべて0x1のデータで書き換えます。
```
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00000000 01 01 01 01 01 01 01 01 01 01 01 01 01 01 01 01
```

シンボル使用例)
```
>fb LABEL1 LABEL2 0
```
アセンブリソースを元にしたコード領域を書き換える場合、ソース行番号でアドレスを指定することも可能です。

■ 注　意

- アドレスは、0～0xffffffffの領域内で指定してください。これを越えた場合、エラーとなります。書式1では再度アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
  Error: Address range (0-0xFFFFFFFF).
- 先頭アドレスが終了アドレスより大きい場合、エラーとなります。
  Error: address1 > address2
- 書き込み指定範囲内に未使用領域がある場合もエラーとはなりません。未使用領域以外はデータの書き換えが行われます。
- データは8ビット(0～0xff)の範囲内で入力してください。これを越えるとエラーとなります。書式1では再度データ入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
  Error: Data range (0-0xFF).
- fbコマンドは[Memory]ウィンドウや[Source]ウィンドウの表示内容を更新しません。それぞれ表示コマンドで再表示させるか、垂直スクロールを行ってください。
  また、プログラム領域を書き換えてもソースは書き換え前のまま表示されます。
- ICEモードで一度に広範囲の領域を書き換えると、時間がかかるためタイムアウトエラーが起きる場合があります。

fh (fill half) [ICD / ICE / SIM / MON]

■ 機　能

指定のメモリ領域を指定のハーフワードデータで書き換えます。デフォルトはリトルエンディアン形式ですが、パラメータファイルで指定領域をビッグエンディアン形式に設定することも可能です。

■ 書　式

(1) **fh** (ガイダンスモード)

(2) **fh <address1> <address2> <data>** (直接入力モード)

<address1>: 指定範囲先頭アドレス(16進数、またはシンボル)
<address2>: 指定範囲終了アドレス(16進数、またはシンボル)
<data>: 書き込みデータ(16進数)
条件: 0x0≦address1≦address2≦0xffffffe(ハーフワード境界)、0x0≦data≦0xffff

■ 入力例

書式1)
```
>fh↵
Start address ? : 0000000↵   ...先頭アドレスを入力(シンボルも使用可能)
End   address ? : 000000e↵   ...終了アドレスを入力(シンボルも使用可能)
Data  pattern ? : 1↵         ...書き込みデータを入力
>
```
※[Enter]キーのみの入力でコマンドの実行を中止できます。

書式2)
```
>fh 0000000 000000e 1↵
>
```
どちらの例も、アドレス0x0～0xf(0xe+1)のメモリ領域をすべて0x0001のデータで書き換えます。
(リトルエンディアンの場合)
```
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00000000 01 00 01 00 01 00 01 00 01 00 01 00 01 00 01 00
```

シンボル使用例)
```
>fh LABEL1 LABEL2 0
```
アセンブリソースを元にしたコード領域を書き換える場合、ソース行番号でアドレスを指定することも可能です。

■ 注　意

- データは16ビット単位で書き換えられるため、領域の先頭アドレスと終了アドレスにはハーフワード境界アドレス(偶数アドレス)を指定してください。奇数アドレスを指定した場合はワーニングとなり、アドレスの最下位ビットを0として書き換え領域が設定されます。
  Warning: Round down to multiple of 2.
- アドレスは、0～0ffffffffの領域内で指定してください。これを越えた場合、エラーとなります。書式1では再度アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
  Error: Address range (0-0xFFFFFFF).
- 先頭アドレスが終了アドレスより大きい場合、エラーとなります。
  Error: address1 > address2
- 書き込み指定範囲内に未使用領域がある場合もエラーとはなりません。未使用領域以外はデータの書き換えが行われます。
- データは16ビット(0～0xffff)の範囲内で入力してください。これを越えるとエラーとなります。書式1では再度データ入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
  Error: Data range (0-0xFFFF).
- fhコマンドは[Memory]ウィンドウや[Source]ウィンドウの表示内容を更新しません。それぞれ表示コマンドで再表示させるか、垂直スクロールを行ってください。
  また、プログラム領域を書き換えてもソースは書き換え前のまま表示されます。
- ICEモードで一度に広範囲の領域を書き換えると、時間がかかるためタイムアウトエラーが起きる場合があります。

fw (fill word) [ICD / ICE / SIM / MON]

■機　能

指定のメモリ領域を指定のワードデータで書き換えます。デフォルトはリトルエンディアン形式ですが、パラメータファイルで指定領域をビッグエンディアン形式に設定することも可能です。

■書　式

(1) **fw** (ガイダンスモード)

(2) **fw <address1> <address2> <data>** (直接入力モード)

<address1>: 指定範囲先頭アドレス(16進数、またはシンボル)
<address2>: 指定範囲終了アドレス(16進数、またはシンボル)
<data>: 書き込みデータ(16進数)
条件: 0x0≦address1≦address2≦0xfffffc(ワード境界)、0x0≦data≦0xffffffff

■入力例

書式1)

```
>fw↵
Start address ? : 0000000↵   ...先頭アドレスを入力(シンボルも使用可能)
End   address ? : 000000c↵   ...終了アドレスを入力(シンボルも使用可能)
Data  pattern ? : 1↵         ...書き込みデータを入力
>
```

※[Enter]キーのみの入力でコマンドの実行を中止できます。

書式2)

```
>fw 0000000 000000c 1↵
>
```

どちらの例も、アドレス0x0～0xf(0xc+3)のメモリ領域をすべて0x0001のデータで書き換えます。
(リトルエンディアンの場合)

```
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00000000 01 00 00 00 01 00 00 00 01 00 00 00 01 00 00 00
```

シンボル使用例)

```
>fw LABEL1 LABEL2 0
```

アセンブリソースを元にしたコード領域を書き換える場合、ソース行番号でアドレスを指定することも可能です。

■注　意

- データは32ビット単位で書き換えられるため、領域の先頭アドレスと終了アドレスにはワード境界アドレスを指定してください。それ以外のアドレスを指定した場合はワーニングとなり、アドレスの下位2ビットを0として書き換え領域が設定されます。
  Warning: Round down to multiple of 4.
- アドレスは、0～0xffffffffの領域内で指定してください。これを越えた場合、エラーとなります。書式1では再度アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
  Error: Address range (0-0xFFFFFFF).
- 先頭アドレスが終了アドレスより大きい場合、エラーとなります。
  Error: address1 > address2
- 書き込み指定範囲内に未使用領域がある場合もエラーとはなりません。未使用領域以外はデータの書き換えが行われます。
- データは32ビット(0～0xffffffff)の範囲内で入力してください。これを越えるとエラーとなります。書式1では再度データ入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
  Error: Data range (0-0xFFFFFFFF).
- fwコマンドは[Memory]ウィンドウや[Source]ウィンドウの表示内容を更新しません。それぞれ表示コマンドで再表示させるか、垂直スクロールを行ってください。
  また、プログラム領域を書き換えてもソースは書き換え前のまま表示されます。
- ICEモードで一度に広範囲の領域を書き換えると、時間がかかるためタイムアウトエラーが起きる場合があります。

db (dump byte) [ICD / ICE / SIM / MON]

■ 機　能

メモリの内容を16バイト/行の16進ダンプ形式で表示します。

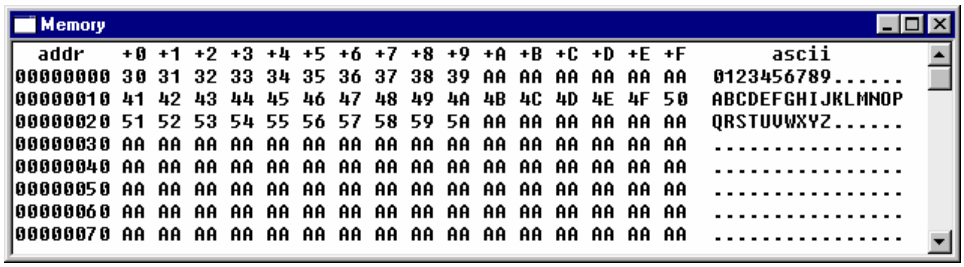
■ 書　式

(1) **db** (直接入力モード)
(2) **db <address1>** (直接入力モード)
(3) **db <address1> <address2>** (直接入力モード)

<address1>: 表示開始アドレス(16進数、またはシンボル)
<address2>: 表示終了アドレス(16進数、またはシンボル)
条件: 0≦address1≦address2≦0xffffff

■ 表　示

(1) [Memory]**ウィンドウを開いている場合**



書式1ではアドレス0x0から[Memory]ウィンドウを再表示します。
書式2および3では、<address1>が最上部の行に表示されるように[Memory]ウィンドウを再表示します。
<address1>が16アドレス/行の途中を指定していても、データはその行の先頭から表示されます。たとえば、<address1>に0x8番地を指定しても、図のように0x0番地からデータを表示します。ただし、<address1>に0xffffff0などのメモリ最上部付近のアドレスを指定した場合は、ウィンドウの最終行が0xffffff0となり、指定のアドレスがウィンドウの先頭部とはなりません。
[Memory]ウィンドウは、スクロールによってデータメモリ全体を表示可能なため、書式3の<address2>の指定は特に意味を持ちません。書式2と書式3は、まったく同じ表示となります。

(2) [Memory]**ウィンドウを閉じている場合**

[Command]ウィンドウにデータを表示します。
書式1では、アドレス0x0から16行分(デフォルト)のデータを表示してコマンド入力待ちとなります。

```
>db↵
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00000000 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
00000010 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
    :                                 :
000000F0 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
>
```

書式2では<address1>から16行分(デフォルト)のデータを表示してコマンド入力待ちとなります。
表示途中で最終アドレス0xffffff0の行を表示した場合は、16行未満であってもコマンド入力待ちとなります。行の途中のアドレスを指定した場合は、そのアドレスの前は空白となります。

```
>db 7fff8↵
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
0007FFF0                          ** ** ** ** ** ** ** **
00080000 04 00 08 00 20 C0 08 6C 81 A0 08 6C 00 C0 03 1C
    :                                 :
000800F0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
>
```

※マップされない領域は"**"で表示されます。

書式3では<address1>から<address2>までのデータを表示してコマンド入力待ちとなります。
16行(デフォルト)以上の表示範囲を指定しても、表示は16行で停止します(書式2と同じ)。

```
>db 0 17↵
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00000000 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
00000010 AA AA AA AA AA AA AA AA
>
```

[Command]ウィンドウには、[Memory]ウィンドウのようなASCII表示は行いません。

(3) [Command]ウィンドウの表示行数

dbコマンドによる[Command]ウィンドウへの表示は、1回の実行につきデフォルトで16行に設定されています。この表示行数はmdコマンドで1～1000行の範囲で変更することができます。

(4) ログ出力

logコマンドによってコマンド実行結果をログファイルに出力する場合は、[Memory]ウィンドウを閉じて[Command]ウィンドウにデータを表示させてください。
[Memory]ウィンドウが開いている場合、[Command]ウィンドウにはデータが表示されないため、表示内容がログファイルに出力されません。

(5) 連続表示機能

dbコマンドを一度実行すると、他のコマンドを実行するまでは[Enter]キーの入力のみでデータを連続して表示することができます。
[Enter]キーを入力すると、[Memory]ウィンドウは1画面分スクロールします。
[Command]ウィンドウにデータを表示している場合は、前回表示したアドレスに続く16行分(デフォルト)を表示します。

```
>db↵
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00000000 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
00000010 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
    :                           :
000000F0 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
>↵
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00000100 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
00000110 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
    :                           :
000001F0 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
>↵
```

(6) シンボル使用例

```
>db LABEL1↵
```

■ 注　意

・ 開始、終了アドレスともに、0～0xffffffの領域内で指定してください。これを越えた場合、エラーとなります。

Error: Address range (0-0xFFFFFFF).

・ 開始アドレスが終了アドレスより大きい場合、エラーとなります。

Error: address1 > address2

dh (dump half) [ICD / ICE / SIM / MON]

■ 機　能

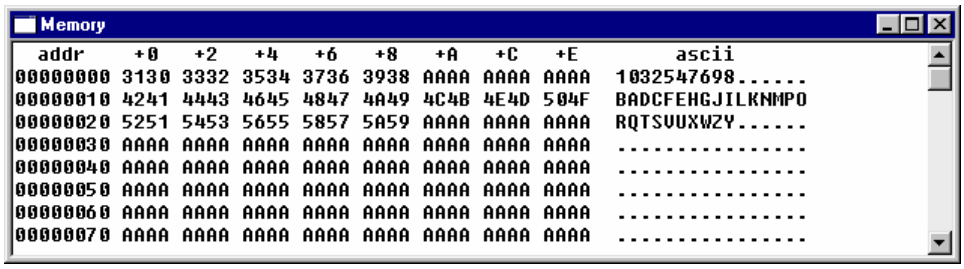
メモリの内容を8ハーフワード/行の16進ダンプ形式で表示します。デフォルトの表示形式はリトルエンディアンですが、パラメータファイルで指定領域をビッグエンディアンに設定することも可能です。

■ 書　式

(1) **dh** (直接入力モード)
(2) **dh <address1>** (直接入力モード)
(3) **dh <address1> <address2>** (直接入力モード)
<address1>: 表示開始アドレス(16進数、またはシンボル)
<address2>: 表示終了アドレス(16進数、またはシンボル)
条件: 0≦address1≦address2≦0xffffff

■ 表　示

(1) [Memory]**ウィンドウを開いている場合**



書式1ではアドレス0x0から[Memory]ウィンドウを再表示します。書式2および3では、<address1>が最上部の行に表示されるように[Memory]ウィンドウを再表示します。
<address1>が16アドレス/行の途中を指定していても、データはその行の先頭から表示されます。たとえば、<address1>に0x8番地を指定しても、図のように0x0番地からデータを表示します。ただし、<address1>に0xffffff0などのメモリ最上部付近のアドレスを指定した場合は、ウィンドウの最終行が0xffffff0となり、指定のアドレスがウィンドウの先頭部とはなりません。
[Memory]ウィンドウは、スクロールによってデータメモリ全体を表示可能なため、書式3の<address2>の指定は特に意味を持ちません。書式2と書式3は、まったく同じ表示となります。

(2) [Memory]**ウィンドウを閉じている場合**

[Command]ウィンドウにデータを表示します。
書式1では、アドレス0x0から16行分(デフォルト)のデータを表示してコマンド入力待ちとなります。

```
>dh↵
  addr    +0   +2   +4   +6   +8   +A   +C   +E
00000000 AAAA AAAA AAAA AAAA AAAA AAAA AAAA AAAA
00000010 AAAA AAAA AAAA AAAA AAAA AAAA AAAA AAAA
    :                             :
000000F0 AAAA AAAA AAAA AAAA AAAA AAAA AAAA AAAA
>
```

書式2では<address1>から16行分(デフォルト)のデータを表示してコマンド入力待ちとなります。表示途中で最終アドレス0xffffff0の行を表示した場合は、16行未満であってもコマンド入力待ちとなります。行の途中のアドレスを指定した場合は、そのアドレスの前は空白となります。

```
>dh 7fff8↵
  addr    +0   +2   +4   +6   +8   +A   +C   +E
0007FFF0                      **** **** **** ****
00080000 0004 0008 C020 6C08 A081 6C08 C000 1C03
    :                             :
000800F0 FFFF FFFF FFFF FFFF FFFF FFFF FFFF FFFF
>
```

※マップされない領域は"****"で表示されます。

書式3では<address1>から<address2>までのデータを表示してコマンド入力待ちとなります。
16行(デフォルト)以上の表示範囲を指定しても、表示は16行で停止します(書式2と同じ)。

```
>dh 0 17↵
  addr    +0   +2   +4   +6   +8   +A   +C   +E
00000000 AAAA AAAA AAAA AAAA AAAA AAAA AAAA AAAA
00000010 AAAA AAAA AAAA AAAA
>
```

[Command]ウィンドウには、[Memory]ウィンドウのようなASCII表示は行いません。

(3) [Command]**ウィンドウの表示行数**

dhコマンドによる[Command]ウィンドウへの表示は、1回の実行につきデフォルトで16行に設定されています。この表示行数はmdコマンドで1～1000行の範囲で変更することができます。

(4) **ログ出力**

logコマンドによってコマンド実行結果をログファイルに出力する場合は、[Memory]ウィンドウを閉じて[Command]ウィンドウにデータを表示させてください。
[Memory]ウィンドウが開いている場合、[Command]ウィンドウにはデータが表示されないため、表示内容がログファイルに出力されません。

(5) **連続表示機能**

dhコマンドを一度実行すると、他のコマンドを実行するまでは[Enter]キーの入力のみでデータを連続して表示することができます。
[Enter]キーを入力すると、[Memory]ウィンドウは1画面分スクロールします。
[Command]ウィンドウにデータを表示している場合は、前回表示したアドレスに続く16行分(デフォルト)を表示します。

```
>dh↵
  addr    +0   +2   +4   +6   +8   +A   +C   +E
00000000 AAAA AAAA AAAA AAAA AAAA AAAA AAAA AAAA
00000010 AAAA AAAA AAAA AAAA AAAA AAAA AAAA AAAA
    :                            :
000000F0 AAAA AAAA AAAA AAAA AAAA AAAA AAAA AAAA
>↵
  addr    +0   +2   +4   +6   +8   +A   +C   +E
00000100 AAAA AAAA AAAA AAAA AAAA AAAA AAAA AAAA
00000110 AAAA AAAA AAAA AAAA AAAA AAAA AAAA AAAA
    :                            :
000001F0 AAAA AAAA AAAA AAAA AAAA AAAA AAAA AAAA
>↵
```

(6) **シンボル使用例**

```
>dh LABEL1↵
```

■ **注　意**

・ ハーフワード境界以外のアドレスを指定した場合、最下位ビットを0として表示範囲が設定されます。

・ 開始、終了アドレスともに、0～0xffffffffの領域内で指定してください。これを越えた場合、エラーとなります。

Error: Address range (0-0xFFFFFFF).

・ 開始アドレスが終了アドレスより大きい場合、エラーとなります。

Error: address1 > address2

dw (dump word) [ICD / ICE / SIM / MON]

■ 機　能

メモリの内容を4ワード/行の16進ダンプ形式で表示します。デフォルトの表示形式はリトルエンディアンですが、パラメータファイルで指定領域をビッグエンディアンに設定することも可能です。

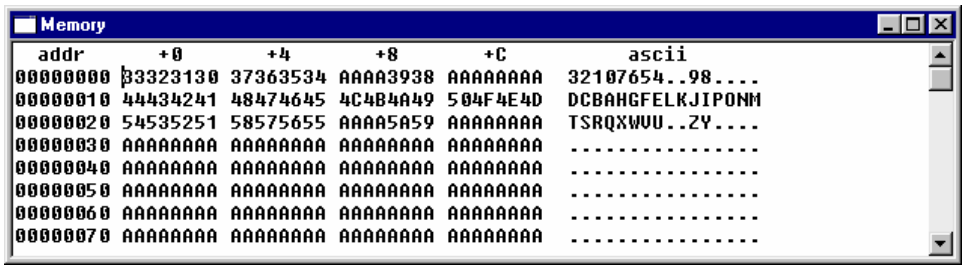
■ 書　式

(1) **dw** (直接入力モード)
(2) **dw <address1>** (直接入力モード)
(3) **dw <address1> <address2>** (直接入力モード)

<address1>: 表示開始アドレス(16進数、またはシンボル)
<address2>: 表示終了アドレス(16進数、またはシンボル)
条件: 0≦address1≦address2≦0xffffff

■ 表　示

(1) [Memory]**ウィンドウを開いている場合**



書式1ではアドレス0x0から[Memory]ウィンドウを再表示します。書式2および3では、<address1>が最上部の行に表示されるように[Memory]ウィンドウを再表示します。
<address1>が16アドレス/行の途中を指定していても、データはその行の先頭から表示されます。たとえば、<address1>に0x8番地を指定しても、図のように0x0番地からデータを表示します。ただし、<address1>に0xffffff0などのメモリ最上部付近のアドレスを指定した場合は、ウィンドウの最終行が0xffffff0となり、指定のアドレスがウィンドウの先頭部とはなりません。
[Memory]ウィンドウは、スクロールによってデータメモリ全体を表示可能なため、書式3の<address2>の指定は特に意味を持ちません。書式2と書式3は、まったく同じ表示となります。

(2) [Memory]**ウィンドウを閉じている場合**

[Command]ウィンドウにデータを表示します。
書式1では、アドレス0x0から16行分(デフォルト)のデータを表示してコマンド入力待ちとなります。

```
>dw↵
  addr      +0       +4       +8       +C
00000000 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
00000010 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
    :                           :
000000F0 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
>
```

書式2では<address1>から16行分(デフォルト)のデータを表示してコマンド入力待ちとなります。
表示途中で最終アドレス0xffffff0の行を表示した場合は、16行未満であってもコマンド入力待ちとなります。行の途中のアドレスを指定した場合は、そのアドレスの前は空白となります。

```
>dw 7fff8↵
  addr      +0       +4       +8       +C
0007FFF0                   ******** ********
00080000 00080004 6C08C020 6C08A081 1C03C000
    :                           :
000800F0 FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF
>
```

※マップされない領域は"********"で表示されます。

書式3では<address1>から<address2>までのデータを表示してコマンド入力待ちとなります。
16行(デフォルト)以上の表示範囲を指定しても、表示は16行で停止します(書式2と同じ)。

```
>dw 0 17↵
  addr      +0       +4       +8       +C
00000000 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
00000010 AAAAAAAA AAAAAAAA
>
```

[Command]ウィンドウには、[Memory]ウィンドウのようなASCII表示は行いません。

(3) [Command]ウィンドウの表示行数

dwコマンドによる[Command]ウィンドウへの表示は、1回の実行につきデフォルトで16行に設定されています。この表示行数はmdコマンドで1～1000行の範囲で変更することができます。

(4) ログ出力

logコマンドによってコマンド実行結果をログファイルに出力する場合は、[Memory]ウィンドウを閉じて[Command]ウィンドウにデータを表示させてください。
[Memory]ウィンドウが開いている場合、[Command]ウィンドウにはデータが表示されないため、表示内容がログファイルに出力されません。

(5) 連続表示機能

dwコマンドを一度実行すると、他のコマンドを実行するまでは[Enter]キーの入力のみでデータを連続して表示することができます。
[Enter]キーを入力すると、[Memory]ウィンドウは1画面分スクロールします。
[Command]ウィンドウにデータを表示している場合は、前回表示したアドレスに続く16行分(デフォルト)を表示します。

```
>dw↵
  addr      +0       +4       +8       +C
00000000 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
00000010 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
    :                           :
000000F0 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
>↵
  addr      +0       +4       +8       +C
00000100 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
00000110 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
    :                           :
00000200 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
>↵
```

(6) シンボル使用例

```
>dw LABEL1↵
```

■ 注　意

- ワード境界以外のアドレスを指定した場合、下位2ビットを0として表示範囲が設定されます。
- 開始、終了アドレスともに、0～0xffffffffの領域内で指定してください。これを越えた場合、エラーとなります。

  Error: Address range (0-0xFFFFFFF).

- 開始アドレスが終了アドレスより大きい場合、エラーとなります。

  Error: address1 > address2

df (dump file) [ICD / ICE / SIM / MON]

■ 機　能

メモリの内容をテキスト形式、バイナリ形式、またはモトローラS3形式でファイルに出力します。

■ 書　式

df (ガイダンスモード)

■ 入力例

ガイダンスに従ってファイルに書き出すアドレス範囲、ファイル形式、ファイル名を入力します。

```
>df↵
Start address :00600000↵                          ...開始アドレスを入力
End   address :0061ffff↵                          ...終了アドレスを入力
File type (1.Binary 2.Text 3.Motorola-S3) ...? 2↵ ...ファイル形式を選択
File name ? : df.txt↵                             ...出力ファイル名を入力
Processing 00600000-00607FFF address.
Processing 00608000-0060FFFF address.
Processing 00610000-00617FFF address.
Processing 00618000-0061FFFF address.
>
```

次のようなファイルが作成されます(テキスト形式の例)。

```
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00600000 30 31 32 33 34 35 36 37 38 39 41 42 43 44 45 46
00600010 30 31 32 33 34 35 36 37 38 39 41 42 43 44 45 46
00600020 30 31 32 33 34 35 36 37 38 39 41 42 43 44 45 46
    :                           :
```

■ 注　意

・ 開始、終了アドレスともに、0～0xffffffffの領域内で指定してください。これを越えた場合、エラーとなります。

Error: Address range (0-0xFFFFFFF).

・ 開始アドレスが終了アドレスより大きい場合、エラーとなります。

Error: address1 > address2

・ テキスト形式は16バイト/行の出力です。
モトローラS3形式では1行に32バイト分が出力されます。最終行が32バイトに満たない場合、32バイトになるように終了アドレスが補正されます。

例)
```
>df↵
Start addess  : 80000↵                            ...開始アドレスを入力
End   address : 80103↵                            ...終了アドレスを入力
File type (1.Binary 2.Text 3.Motorola-S3) ...? 3↵ ...モトローラS3形式を指定
File name ? : df.mot↵                             ...出力ファイル名を入力
Processing 00080000-0008011F address.             ...終了アドレスが補正される
>
```

eb (enter byte) [ICD / ICE / SIM / MON]

■ 機　能

バイトデータ(16進数)を入力してメモリの内容を書き換えます。
ガイダンスモードにより、指定したアドレスから連続してデータの書き込みが行えます。

■ 書　式

(1) **eb** (ガイダンスモード)
(2) **eb <address>** (ガイダンスモード)
<address>: 書き込み開始アドレス(16進数、またはシンボル)
条件: 0≦address≦0xffffff、0≦入力データ≦0xff

■ 入力例

ガイダンスとして、アドレスと現在のデータが表示されます。

書式1)
```
>eb↵
Enter address ? :0↵          ...開始アドレスを入力
00000000 AA :00↵             ...データを16進数で入力
00000001 AA :q↵              ...コマンドを終了
>
```

書式2)
```
>eb 0↵
00000000 AA :00↵
00000001 AA :^↵              ...前のアドレスに戻す
00000000 00 :↵               ...入力をスキップ
00000001 AA :q↵
>
```
どちらの例も、アドレス0x0を0x00のデータで書き換えます。
```
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00000000 00 AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
```

シンボル使用例)
```
>eb CAHR1↵
```

■ 注　意

- 開始アドレスは、0～0ffffffffの領域内で指定してください。これを越えた場合、エラーとなります。書式1では再度アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
  Error: Address range (0-0xFFFFFFF).
- データメモリマップ中の未使用アドレスはデータが"**"として表示されます。未使用アドレスの指定やガイダンスの表示は行えますが、データを入力するとエラーとなります。"**"の表示されたアドレスは、[Enter]キーでスキップするか、コマンドを終了させてください。
  Error: No map area.
- データは8ビット(0～0xff)の範囲内の16進数を入力してください。これを越えるとエラーとなり、再度、同じアドレスに入力するためのガイダンスが表示されます。
  Error: Data range (0-0xFF).
- ガイダンス入力中に0番地の前のアドレスに移動する操作、または0ffffffff番地を越える操作を行ってもアドレスは変わりません。
- ebコマンドは[Memory]ウィンドウや[Source]ウィンドウの表示内容を更新しません。それぞれ表示コマンドで再表示させるか、垂直スクロールを行ってください。
  また、プログラム領域を書き換えてもソースは書き換え前のまま表示されます。

eh (enter half) [ICD / ICE / SIM / MON]

■ 機　能

ハーフワードデータ(16進数)を入力してメモリの内容を書き換えます。ガイダンスモードにより、指定したアドレスから連続してデータの書き込みが行えます。デフォルトはリトルエンディアン形式ですが、パラメータファイルで指定領域をビッグエンディアン形式に設定することも可能です。

■ 書　式

(1) **eh** (ガイダンスモード)

(2) **eh <address>** (ガイダンスモード)

<address>: 書き込み開始アドレス(16進数、またはシンボル)

条件: 0≦address≦0xffffffe(ハーフワード境界)、0≦入力データ≦0xffff

■ 入力例

ガイダンスとして、アドレスと現在のデータが表示されます。

書式1)
```
>eh↵
Enter address ? :0↵          ...開始アドレスを入力
00000000 AAAA :1234↵         ...データを16進数で入力
00000002 AAAA :q↵            ...コマンドを終了
>
```

書式2)
```
>eh 0↵
00000000 AAAA :1234↵
00000002 AAAA :^↵            ...前のアドレスに戻す
00000000 1234 :↵             ...入力をスキップ
00000002 AAAA :q↵
>
```

どちらの例も、アドレス0x0(～0x1)を0x1234のデータで書き換えます。(リトルエンディアンの場合)

```
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00000000 34 12 AA AA AA AA AA AA AA AA AA AA AA AA AA AA
```

シンボル使用例)
```
>eh VAR1↵
```

■ 注　意

- データは16ビット単位で書き換えられるため、書き込み開始アドレスにはハーフワード境界アドレス(偶数アドレス)を指定してください。奇数アドレスを指定した場合はワーニングとなり、アドレスの最下位ビットを0として処理されます。
  Warning: Round down to multiple of 2.
- 開始アドレスは、0～0ffffffffの領域内で指定してください。これを越えた場合、エラーとなります。書式1では再度アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
  Error: Address range (0-0xFFFFFFF).
- データメモリマップ中の未使用アドレスはデータが"****"として表示されます。未使用アドレスの指定やガイダンスの表示は行えますが、データを入力するとエラーとなります。"****"の表示されたアドレスは、[Enter]キーでスキップするか、コマンドを終了させてください。
  Error: No map area.
- データは16ビット(0～0xffff)の範囲内の16進数を入力してください。これを越えるとエラーとなり、再度、同じアドレスに入力するためのガイダンスが表示されます。
  Error: Data range (0-0xFFFF).
- ガイダンス入力中に0番地の前のアドレスに移動する操作、または0ffffffff番地を越える操作を行ってもアドレスは変わりません。
- ehコマンドは[Memory]ウィンドウや[Source]ウィンドウの表示内容を更新しません。それぞれ表示コマンドで再表示させるか、垂直スクロールを行ってください。
  また、プログラム領域を書き換えてもソースは書き換え前のまま表示されます。

ew (enter word) [ICD / ICE / SIM / MON]

■ 機　能

ワードデータ(16進数)を入力してメモリの内容を書き換えます。ガイダンスモードにより、指定したアドレスから連続してデータの書き込みが行えます。デフォルトはリトルエンディアン形式ですが、パラメータファイルで指定領域をビッグエンディアン形式に設定することも可能です。

■ 書　式

(1) **ew** (ガイダンスモード)

(2) **ew <address>** (ガイダンスモード)

<address>: 書き込み開始アドレス(16進数、またはシンボル)

条件: 0≦address≦0xfffffffc(ワード境界)、0≦入力データ≦0xffffffff

■ 入力例

ガイダンスとして、アドレスと現在のデータが表示されます。

書式1)
```
>ew↵
Enter address ? :0↵                    ...開始アドレスを入力
00000000 AAAAAAAA :12345678↵...データを16進数で入力
00000002 AAAAAAAA :q↵                  ...コマンドを終了
>
```

書式2)
```
>ew 0↵
00000000 AAAAAAAA :12345678↵
00000002 AAAAAAAA :^↵                  ...前のアドレスに戻す
00000000 12345678 :↵                   ...入力をスキップ
00000002 AAAAAAAA :q↵
>
```

どちらの例も、アドレス0x0(～0x3)を0x12345678のデータで書き換えます。(リトルエンディアンの場合)

```
  addr   +0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F
00000000 78 56 34 12 AA AA AA AA AA AA AA AA AA AA AA AA
```

シンボル使用例)
```
>ew i↵
```

■ 注　意

- データは32ビット単位で書き換えられるため、書き込み開始アドレスにはワード境界アドレスを指定してください。それ以外のアドレスを指定した場合はワーニングとなり、アドレスの下位2ビットを0として処理されます。
  Warning: Round down to multiple of 4.
- 開始アドレスは、0～0xffffffffの領域内で指定してください。これを越えた場合、エラーとなります。書式1では再度アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
  Error: Address range (0-0xFFFFFFF).
- データメモリマップ中の未使用アドレスはデータが"********"として表示されます。未使用アドレスの指定やガイダンスの表示は行えますが、データを入力するとエラーとなります。"********"の表示されたアドレスは、[Enter]キーでスキップするか、コマンドを終了させてください。
  Error: No map area.
- データは32ビット(0～0ffffffff)の範囲内の16進数を入力してください。これを越えるとエラーとなり、再度、同じアドレスに入力するためのガイダンスが表示されます。
  Error: Data range (0-0xFFFFFFFF).
- ガイダンス入力中に0番地の前のアドレスに移動する操作、または0xffffff番地を越える操作を行ってもアドレスは変わりません。
- ewコマンドは[Memory]ウィンドウや[Source]ウィンドウの表示内容を更新しません。それぞれ表示コマンドで再表示させるか、垂直スクロールを行ってください。
  また、プログラム領域を書き換えてもソースは書き換え前のまま表示されます。

mv (move) [ICD / ICE / SIM / MON]

■ 機　能

指定のメモリ領域の内容をバイト単位で別の領域にコピーします。

■ 書　式

(1) **mv** (ガイダンスモード)

(2) **mv <address1> <address2> <address3>** (直接入力モード)

<address1>: コピー元の先頭アドレス(16進数、またはシンボル)
<address2>: コピー元の終了アドレス(16進数、またはシンボル)
<address3>: コピー先のアドレス(16進数、またはシンボル)
条件: 0≦address1≦address2≦0xffffff、0≦address3≦0xffffff

■ 入力例

書式1)

```
>mv↵
Start address ? :000↵          ...コピー元の先頭アドレスを入力
End   address ? :0ff↵          ...コピー元の終了アドレスを入力
Destination address ? :300↵...コピー先のアドレスを入力
>
```

※[Enter]キーのみの入力でコマンドの実行を中止できます。

書式2)

```
>mv 0 ff 300↵
>
```

2つの例はともにアドレス0x0～0xffのメモリ領域の内容を、0x300以降の領域にコピーします。

シンボル使用例)

```
>mv LABEL1 LABEL2 LABEL3↵
```

■ 注　意

・ アドレスは、0～0ffffffffの領域内で指定してください。これを越えた場合、エラーとなります。書式1では再度アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
  Error: Address range (0-0xFFFFFFF).

・ コピー元の先頭アドレスが終了アドレスより大きい場合、エラーとなります。
  Error: address1 > address2

・ コピー元の指定範囲内にマップされていない領域がある場合、その領域のデータを0xf0としてコピーが行われます。

・ コピー先の範囲内にマップされていない領域がある場合、その領域を除いた有効領域に対してコピーが行われます。

・コピー先アドレスがコピー元の先頭アドレスより小さい場合は、先頭アドレスから順にコピーします。逆に、コピー先アドレスがコピー元の先頭アドレスより大きい場合は、終了アドレスから順にコピーします。したがって、コピー元の領域内にコピー先アドレスを設定した場合でもコピーされます。

・ コピー先の終了アドレスが0ffffffffを越える場合、0ffffffffまでがコピーの対象となります。

・ mvコマンドは[Memory]ウィンドウや[Source]ウィンドウの表示内容を更新しません。それぞれ表示コマンドで再表示させるか、垂直スクロールを行ってください。
  また、プログラム領域を書き換えてもソースは書き換え前のまま表示されます。

・ ICEモードで一度に広範囲の領域をコピーすると、時間がかかるためタイムアウトエラーが起きる場合があります。

mvh (move half) [ICD / (ICE) / SIM / MON]

■機　能

指定のメモリ領域の内容をハーフワード単位で別の領域にコピーします。コピー元とコピー先のエンディアン形式が異なる場合は、コピーの際にデータ形式が変換されます。

■書　式

(1) **mvh** (ガイダンスモード)

(2) **mvh <address1> <address2> <address3>**(直接入力モード)

<address1>: コピー元の先頭アドレス(16進数、またはシンボル)
<address2>: コピー元の終了アドレス(16進数、またはシンボル)
<address3>: コピー先のアドレス(16進数、またはシンボル)
条件: 0≦address1≦address2≦0xffffff、0≦address3≦0xffffff

■入力例

書式1)
```
>mvh↵
Start address ? :000↵        ...コピー元の先頭アドレスを入力
End   address ? :0ff↵        ...コピー元の終了アドレスを入力
Destination address ? :300↵ ...コピー先のアドレスを入力
>
```
※[Enter]キーのみの入力でコマンドの実行を中止できます。

書式2)
```
>mvh 0 ff 300↵
>
```
2つの例はともにアドレス0x0～0xffのメモリ領域の内容を、0x300以降の領域にコピーします。

シンボル使用例)
```
>mvh LABEL1 LABEL2 LABEL3↵
```

■注　意

- ハーフワード境界以外のアドレスを指定した場合、最下位ビットを0として処理されます。
- アドレスは、0～0xffffffffの領域内で指定してください。これを越えた場合、エラーとなります。書式1では再度アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
  Error: Address range (0-0xFFFFFFF).
- コピー元の先頭アドレスが終了アドレスより大きい場合、エラーとなります。
  Error: address1 > address2
- コピー元の指定範囲内にマップされていない領域がある場合、その領域のデータの各バイトを0xf0としてコピーが行われます。
- コピー先の範囲内にマップされていない領域がある場合、その領域を除いた有効領域に対してコピーが行われます。
- コピー先アドレスがコピー元の先頭アドレスより小さい場合は、先頭アドレスから順にコピーします。逆に、コピー先アドレスがコピー元の先頭アドレスより大きい場合は、終了アドレスから順にコピーします。したがって、コピー元の領域内にコピー先アドレスを設定した場合でもコピーされます。
- コピー先の終了アドレスが0xffffffffを越える場合、0xffffffffまでがコピーの対象となります。
- mvhコマンドは[Memory]ウィンドウや[Source]ウィンドウの表示内容を更新しません。それぞれ表示コマンドで再表示させるか、垂直スクロールを行ってください。
  また、プログラム領域を書き換えてもソースは書き換え前のまま表示されます。
- ICEモードで一度に広範囲の領域をコピーすると、時間がかかるためタイムアウトエラーが起きる場合があります。
- ICEモードで使用する場合、S5U1C33104Hのバージョンが2.0以上である必要があります。
  2.0未満のバージョンでは、バイト単位でコピーします。

mvw (move word) [ICD / (ICE) / SIM / MON]

■ 機　能

指定のメモリ領域の内容をワード単位で別の領域にコピーします。コピー元とコピー先のエンディアン形式が異なる場合は、コピーの際にデータ形式が変換されます。

■ 書　式

(1) **mvw** (ガイダンスモード)

(2) **mvw <address1> <address2> <address3>** (直接入力モード)

<address1>: コピー元の先頭アドレス(16進数、またはシンボル)
<address2>: コピー元の終了アドレス(16進数、またはシンボル)
<address3>: コピー先のアドレス(16進数、またはシンボル)
条件: 0≦address1≦address2≦0xffffffff、0≦address3≦0xffffffff

■ 入力例

書式1)
```
>mvw↵」
Start address ? :000↵」        ...コピー元の先頭アドレスを入力
End   address ? :0ff↵」        ...コピー元の終了アドレスを入力
Destination address ? :300↵」...コピー先のアドレスを入力
>
```
※[Enter]キーのみの入力でコマンドの実行を中止できます。

書式2)
```
>mvw 0 ff 300↵」
>
```
2つの例はともにアドレス0x0～0xffのメモリ領域の内容を、0x300以降の領域にコピーします。

シンボル使用例)
```
>mvw LABEL1 LABEL2 LABEL3↵」
```

■ 注　意

・ ワード境界以外のアドレスを指定した場合、下位2ビットを0として処理されます。

・ アドレスは、0～0xffffffffの領域内で指定してください。これを越えた場合、エラーとなります。書式1では再度アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
Error: Address range (0-0xFFFFFFF).

・ コピー元の先頭アドレスが終了アドレスより大きい場合、エラーとなります。
Error: address1 > address2

・ コピー元の指定範囲内にマップされていない領域がある場合、その領域のデータの各バイトを0xf0としてコピーが行われます。

・ コピー先の範囲内にマップされていない領域がある場合、その領域を除いた有効領域に対してコピーが行われます。

・コピー先アドレスがコピー元の先頭アドレスより小さい場合は、先頭アドレスから順にコピーします。逆に、コピー先アドレスがコピー元の先頭アドレスより大きい場合は、終了アドレスから順にコピーします。したがって、コピー元の領域内にコピー先アドレスを設定した場合でもコピーされます。

・ コピー先の終了アドレスが0xffffffffを越える場合、0xffffffffまでがコピーの対象となります。

・ mvwコマンドは[Memory]ウィンドウや[Source]ウィンドウの表示内容を更新しません。それぞれ表示コマンドで再表示させるか、垂直スクロールを行ってください。
また、プログラム領域を書き換えてもソースは書き換え前のまま表示されます。

・ ICEモードで一度に広範囲の領域をコピーすると、時間がかかるためタイムアウトエラーが起きる場合があります。

・ ICEモードで使用する場合、S5U1C33104Hのバージョンが2.0以上である必要があります。
2.0未満のバージョンでは、バイト単位でコピーします。

w (watch) [ICD / ICE / SIM / MON]

■ 機　能

メモリ領域の4個所を監視データアドレスとして登録します。それぞれのアドレスから4バイト分の内容が[Register]ウィンドウに表示されます。

■ 書　式

w　　(ガイダンスモード)

■ 入力例

ガイダンスとして、現在設定されてる4個所の監視データアドレスを下位アドレスから順に表示します。変更しないものは[Enter]キーのみの入力でスキップし、変更するものは新たなアドレスを入力してください。[^]キー(前に戻す)、[q]キー(終了)も使用できます。

```
>w↵
Address1 00000000          :100↵  ...監視アドレスを入力
Address2 00000004          :104↵
Address3 00000008          :i↵    ...変数iを指定
Address4 0000000C          :↵     ...設定を変更しない
>
```

■ 注　意

- db33起動時は0、4、8、0xC番地の4個所が監視データアドレスとして初期設定されます。
- アドレスは、0～0xfffffffの領域内で指定してください。これを越えた場合、エラーとなります。再度アドレス入力のガイダンスが表示されます。
  Error: Address range (0-0xFFFFFFF).
- 監視データアドレスは4バイト単位で設定されます。このワード境界以外のアドレスを指定するとワーニングとなり、4の倍数(下位2ビットを0)に切り捨てられます。
  Warning: Round down to multiple of 4.
- マップされていないアドレスを指定した場合も、[Register]ウィンドウには値が表示されますので注意してください。
- 表示は、パラメータファイルで指定されるエンディアン形式で行われます。

rm (read memory) [ICD]

■ 機　能

ターゲットボード上のプログラムをデバッガに読み込みます。読み込んだデータはS5U1C33000Hによるトレース解析に反映されます。ターゲットボード上のプログラムをメモリ操作コマンドなどで書き換えた場合などに使用してください。

■ 書　式

(1) **rm** (ガイダンスモード)

(2) **rm <address1> <address2>** (直接入力モード)

<address1>: 読み込む領域の先頭アドレス(16進数、またはシンボル)
<address2>: 読み込む領域の終了アドレス(16進数、またはシンボル)
条件: 0≦address1≦address2≦0xffffffff

■ 入力例

書式1)

```
>rm↵
Start address ? :0↵          ...先頭アドレスを入力
End   address ? :7fe↵        ...終了アドレスを入力
>
```

※[Enter]キーのみの入力でコマンドの実行を中止できます。

書式2)

```
>rm 0 7fe↵
>
```

2つの例はともにアドレス0x0～0x7feのターゲットボード上のメモリの内容を、デバッガに読み込みます。

シンボル使用例)

```
>rm LABEL1 LABEL2↵
```

■ 注　意

・ rmコマンドはICDモードでのみ有効です。

・ アドレスは、0～0xffffffffの領域内で指定してください。これを越えた場合、エラーとなります。書式1では再度アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
Error: Address range (0-0xFFFFFFF).

・ 先頭アドレスが終了アドレスより大きい場合、エラーとなります。
Error: address1 > address2

mc (memory compare) [ICD / ICE / SIM / MON]

■ 機　能

srf33形式のオブジェクトファイルとメモリとの比較を行います。

■ 書　式

(1) **mc** (ガイダンスモード)

(2) **mc <file name>** (直接入力モード)

<file name>: 比較するsrf33ファイル名

■ 入力例

書式1)
```
>mc↵
File name ? test.srf↵
>
```
...ファイル名を拡張子まで入力します。

書式2)
```
>mc test.srf↵
>
```

■ 注　意

・ mcコマンドは比較した結果、内容に相違がある場合にのみ以下のエラーメッセージを表示します。ファイルとメモリの内容が一致した場合には何も表示しません。

Error: memory verify failure.
Address=xxxxxxxx, Expect=xxxx, Read=xxxx

・ 指定したファイルがオープンできなかった場合、エラーとなります。

Error: Cannot compare data, file open failure, xxxxxxxx.xxx.

16.10.3 レジスタ操作コマンド

rd (register display) [ICD / ICE / SIM / MON]

■ 機　能

レジスタ、実行カウンタ、監視データの内容を表示します。

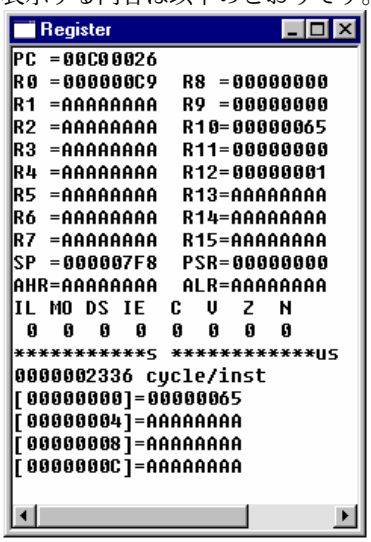
■ 書　式

rd

■ 表　示

(1) **表示内容**

表示する内容は以下のとおりです。



PC:	プログラムカウンタ
R0～R15:	汎用レジスタ
SP:	スタックポインタ
PSR:	プロセッサステータスレジスタ
AHR:	算術演算ハイレジスタ
ALR:	算術演算ローレジスタ
IL:	割り込みレベル
MO:	MACオーバーフローフラグ
DS:	被序数符号フラグ
IE:	割り込み許可フラグ
C:	キャリーフラグ
V:	オーバーフローフラグ
Z:	ゼロフラグ
N:	ネガティブフラグ
s, us:	実行時間(ICE/ICDモード時のみ有効)
cycle/inst:	実行サイクル数
[xxxxxxxx]:	4個所の監視データ

※監視データについては、未使用領域であっても値が表示されますので注意してください。

(2) [Register]**ウィンドウを開いている場合**

[Register]ウィンドウを開いている場合は、プログラムの実行に従って上記の内容がすべて[Register]ウィンドウに表示されます。rdコマンドは[Register]ウィンドウの表示を更新します。

(3) [Register]**ウィンドウが閉じている場合**

[Command]ウィンドウに[Register]ウィンドウと同じ形式でデータを表示します。

(4) **ログ出力**

logコマンドによってコマンド実行結果をログファイルに出力する場合は、[Register]ウィンドウを閉じて[Command]ウィンドウにデータを表示させてください。
[Register]ウィンドウが開いている場合、[Command]ウィンドウにはデータが表示されないため、表示内容がログファイルに出力されません。

rs (register set) [ICD / ICE / SIM / MON]

■ 機　能

レジスタの値を変更します。

■ 書　式

(1) **rs** (ガイダンスモード)

(2) **rs <name> <data>** (直接入力モード)

<name>: レジスタ名またはフラグ名

<data>: 設定データ(16進数)

■ 入力例

書式1) ガイダンスとして、各レジスタの名称と現在の値を表示します。変更しないものは[Enter]キーのみの入力でスキップし、変更するものは新たな値を16進数で入力してください。[^]キー(前に戻す)、[q]キー(終了)も使用できます。

```
>rs↵
 PC =00080004:↵
 R0 =AAAAAAAA:0↵
 R1 =AAAAAAAA:0↵
 R2 =AAAAAAAA:0↵
       :
 R14=AAAAAAAA:0↵
 R15=AAAAAAAA:0↵
 IL =       0:↵
 MO =       0:↵
 DS =       0:↵
 IE =       0:↵
 C  =       0:↵
 V  =       0:↵
 Z  =       0:↵
 N  =       0:↵
 SP =0AAAAAA8:1f00↵
 AHR=AAAAAAAA:0↵
 ALR=AAAAAAAA:0↵
>
```

コマンドを実行後、[Register]ウィンドウは入力した内容に更新されます。

[q]キーによって途中で入力を中止した場合、それまでに入力した内容は変更されます。

書式2)

```
>rs r0 0↵          ...R0レジスタを変更
>rs c 1↵           ...Cフラグをセット
```

■ 注　意

- レジスタ/フラグの有効ビット数を越える値を入力するとエラーとなります。書式1では再度その入力のガイダンスが表示され、書式2ではコマンド入力がキャンセルされます。

  Error: Invalid value.

- PCの設定値は強制的に16ビット境界(最下位ビットを0)に切り捨てられます。奇数アドレスを指定した場合でもエラーとはなりません。

- SPの設定値は強制的に32ビット境界(下位2ビットを0)に切り捨てられます。

16.10.4 プログラム実行コマンド

g (go) [ICD / ICE / SIM / MON]

■ 機　能

現在のPCからターゲットプログラムを実行します。

■ 書　式

(1) **g** (直接入力モード)

(2) **g <address>** (直接入力モード)

<address>: テンポラリブレークアドレス(16進数、シンボルまたはソース行番号)

条件: 0≦address≦0xffffffe

■ 動　作

(1) **書式1の動作**

>g↵

※ このコマンド入力と同じ機能は[Run]メニュー/[Go]コマンドまたは[Go]ボタンでも選択できます。

[Go]ボタン

PCが示すアドレスからターゲットプログラムを実行します。プログラムの実行は次のいずれかの要因によってブレークするまで継続します。

・すでに設定してあるブレーク条件が成立
・[Key break]ボタンのクリック(デバッグモニタモードを除く)
・マップブレーク等の発生

(2) **書式2の動作**

書式2ではテンポラリブレークアドレスを1個所指定することができます。ここで設定したブレークアドレスは他のコマンドを実行するまで有効です。

```
>g 80100↵
>g main.c#20↵          ...行番号による指定
>g SUB1↵               ...シンボルによる指定
```

※ このコマンド入力と同じ機能は[Run]メニュー/[Go to]コマンドまたは[Go to]ボタンでも選択できます。この場合、テンポラリブレークアドレスは、実行前に[Source]ウィンドウ上のブレークさせるアドレスの行をクリックして指定しておきます。実行時にカーソルが表示されている行のアドレスがテンポラリブレークアドレスとなります。

[Go to]**ボタン**

PCが示すアドレスからターゲットプログラムを実行します。プログラムの実行は上記(1)に示した要因により、または指定したテンポラリアドレスの命令をフェッチすると(その命令を実行する前に)ブレークします。

(3) **ブレーク後の**[Enter]**キー入力**

プログラムの実行がブレークすると、ブレークステータスメッセージ(16.13.1項参照)を表示後コマンド入力待ちとなります。

例:
```
Break by temporary break.
>
```

ここで[Enter]キーを入力すると、ブレーク後のPCアドレスからプログラムの実行を再開します。テンポラリブレークアドレスの設定も有効です。

(4) プログラム実行によるウィンドウの表示

＜ICEモード＞

db33の初期設定で、オンザフライ機能が働くようになっています。
プログラム実行中はオンザフライ機能により、[Register]ウィンドウ内のPC、フラグ、監視データの内容が1～0.1秒ごとにリアルタイムで更新されます。他の内容は空白となります。[Register]ウィンドウを閉じている場合は、上記の内容が[Command]ウィンドウに表示されます。オンザフライ機能はmdコマンドによって無効にすることもできます。その場合、プログラム実行中は[Register]ウィンドウの数値部分はすべて空白となり、ブレーク後に更新されます。

[Source]ウィンドウはブレーク後に、ブレークしたアドレスがウィンドウ内に表示されるように更新されます。
[Trace]ウィンドウが開いている場合は、プログラムの実行により表示内容がクリアされます。表示の更新はブレーク後にtdコマンドまたはtsコマンドによって行ってください。
[Memory]ウィンドウが開いている場合、プログラムの実行により表示内容がクリアされ、ブレーク後に更新されます。

＜その他のモード＞

ICEモード以外では上記のオンザフライ表示は行われません。このため、プログラム実行中は[Register]ウィンドウの数値部分はすべて空白となり、ブレーク後に更新されます。
[Source]ウィンドウはブレーク後に、ブレークしたアドレスがウィンドウ内に表示されるように更新されます。
シミュレータモードで[Trace]ウィンドウが開き、トレースモードがONになっている場合は、プログラムの実行によりトレース結果が逐次表示されます。[Trace]ウィンドウが閉じている場合は、トレース結果を[Command]ウィンドウに表示します。トレースモードがOFF、あるいはトレース情報をファイルに出力している場合、表示は行われません。
ICDモードで[Trace]ウィンドウが開いている場合は、プログラムの実行により表示内容がクリアされます。表示の更新はブレーク後にtdコマンドまたはtsコマンドによって行ってください。ICDモードでは、プログラム実行中に[Display trace]ボタンでトレースを中断し、トレースデータを表示させることもできます。
[Memory]ウィンドウが開いている場合、プログラムの実行により表示内容がクリアされ、ブレーク後に更新されます。

(5) オンザフライ情報のログ出力

オンザフライ情報をログファイルに出力させる場合は、[Register]ウィンドウを閉じて[Command]ウィンドウにオンザフライ情報が表示されるようにしてください。

(6) 実行カウンタ

[Register]ウィンドウに表示される実行カウンタは、ターゲットプログラムの実行サイクル数/実行時間を表示します。(ICEまたはICDモード時、詳細は16.9.5項参照)
db33の初期設定では、実行カウンタはリセットされるまで積算されるようになっています。
mdコマンドでこれをリセットモードにすると、gコマンドを実行するたびに実行カウンタが0にリセットされるようになります。[Enter]キーによって実行を再開させた場合も、その時点でリセットされます。

■注　意

- テンポラリブレークアドレスは、各機種のプログラムメモリ領域の範囲内で指定してください。これを越えた場合、エラーとなります。

  Error: Address range (0-0xFFFFFFF).　...0xfffffffを越えるアドレスを指定した場合
  Error: No map area.　...未使用アドレスを指定した場合

- 行番号や[Source]ウィンドウ上で実コードを持たないソース行をテンポラリブレークポイントに指定した場合、テンポラリブレークポイントはその行の直後にあるコードのアドレスに設定されます。
- カレントPCがブートアドレス(0x80000または0xc00000)の場合、プログラムの実行を開始する直前にCPUをコールドリセットします。

s (step) [ICD / ICE / SIM / MON]

■ 機　能

現在のPCからターゲットプログラムをステップ実行します。

■ 書　式

(1) **s** (直接入力モード)
(2) **s <step>** (直接入力モード)
<step>: 実行ステップ数(10進数)
条件: 0≦step≦65535

■ 動　作

(1) 実行ステップの単位

ステップ実行の単位は[Source]ウィンドウの表示モードに従って次のようになります。

逆アセンブル表示モード: アドレス単位
ミックス表示モード: アドレス単位
ソース表示モード: ソースコード単位

(2) 書式1の動作

>s↵

※ このコマンド入力と同じ機能は[Run]メニュー/[Step]コマンドまたは[Step]ボタンでも選択できます。

[Step]**ボタン**

PCが示すアドレスのプログラムを1ステップ実行します。

(3) 書式2の動作

>s 10↵

PCが示すアドレスから指定したステップ分のプログラムをステップ実行します。
プログラムの実行は、すべてのブレーク要因によって指定ステップ数の実行途中でも終了します。

(4) 実行終了後の[Enter]キー入力

プログラムのステップ実行が終了すると、コマンド入力待ちとなります。
ここで[Enter]キーを入力すると、再度同じステップ実行を行います。

(5) ステップ実行によるウィンドウの表示

デバッガの初期設定では、次のように各ステップごとに表示を更新します。
[Source]ウィンドウを開いている場合は、次の実行アドレスを示す下線がステップ実行により1ステップずつ移動します。[Register]ウィンドウの表示内容も各ステップごとに更新されます。
この表示モード(全ステップ表示モード)を、指定ステップ数の最終ステップでのみ更新するように(最終ステップ表示モード)、mdコマンドで切り換えることもできます。

連続実行(gコマンド)とは違い、実行がすぐに終了しない場合でも[Register]ウィンドウは空白にはなりません。

[Memory]ウィンドウの表示は各ステップ実行ごとに更新されます。

ICEまたはICDモードの[Trace]ウィンドウは、ステップ実行により表示内容がクリアされます。表示の更新はステップ実行終了後にtdコマンドまたはtsコマンドによって行ってください。

シミュレータモードで[Trace]ウィンドウが開き、トレースモードがONになっている場合は、プログラムの実行によりトレース結果が逐次表示されます。[Trace]ウィンドウが閉じている場合は、トレース結果を[Command]ウィンドウに表示します。トレース情報はステップ表示モードにかかわらず、各ステップごとに表示されます。トレースモードがOFF、あるいはトレース情報をファイルに出力している場合、表示は行われません。

(6) HALT、SLEEP状態と割り込み

S5U1C33104Hでは、シングルステップ動作時の割り込みを禁止します。
halt命令およびslp命令はシングルステップ動作時でも実行され、それによってスタンバイモードになります。
スタンバイモードは、外部割り込みを発生させることによって解除されます。また、[Key break]ボタンで解除することもできます。

(7) 実行カウンタ

[Register]ウィンドウに表示される実行カウンタは、ターゲットプログラムの実行サイクル数/実行時間を表示します。(ICEまたはICDモード時、詳細は16.9.5項参照)
db33の初期設定では、実行カウンタはリセットされるまで積算されるようになっています。
mdコマンドでこれをリセットモードにすると、sコマンドを実行するたびに実行カウンタが0にリセットされるようになります。[Enter]キーによって再実行させた場合も、その時点でリセットされます。

■ 注　意

- 書式2のステップ数は、0～65535の範囲内で指定してください。これを越えた場合、エラーとなります。

  Error: Step range (0-65535).

- カレントPCがブートアドレス(0x80000または0xc00000)の場合、ステップ実行を開始する直前にCPUをコールドリセットします。

- "jp 0x0"のような無限ループをソース表示モードでステップ実行させた場合、ステップ実行も終了しません。また、中身のないCのfor文をステップ実行すると、for文の処理が終わるまで制御ができません。このような場合は、ミックス表示モードまたは逆アセンブル表示モードでステップ実行してください。

n (next) [ICD / ICE / SIM / MON]

■ 機　能

現在のPCからターゲットプログラムをステップ実行します。

■ 書　式

(1) **n** (直接入力モード)

(2) **n <step>** (直接入力モード)

<step>: 実行ステップ数(10進数)

条件: 0≦step≦65535

※ 書式1のコマンド入力と同じ機能は[Run]メニュー/[Next]コマンドまたは[Next]ボタンでも選択できます。

[Next]**ボタン**

■ 動　作

基本的な動作はsコマンドと同様です。
ただし、Cソースの関数コールやアセンブリソースのサブルーチンコールは、呼び出される関数/サブルーチンを1ステップとして実行します。
この他の機能についてはsコマンドの説明を参照してください。

■ 注　意

・ 書式2のステップ数は、0～65535の範囲内で指定してください。これを越えた場合、エラーとなります。

Error: Step range (0-65535).

・ "jp 0x0"のような無限ループをソース表示モードでステップ実行させた場合、ステップ実行も終了しません。また、中身のないCのfor文をステップ実行すると、for文の処理が終わるまで制御ができません。このような場合は、ミックス表示モードまたは逆アセンブル表示モードでステップ実行してください。

o (step out) [ICD / ICE / SIM / MON]

■機　能

現在のPCからターゲットプログラムを実行し、現在の関数/サブルーチンから抜けたところで停止します。

■書　式

o (直接入力モード)

■動　作

現在のPCアドレスからターゲットプログラムを実行し、最初のリターン命令を実行後、次のステップを実行前に停止します。
実行中は[Source]ウィンドウ、[Register]ウィンドウ等の更新はしません。

■注　意

oコマンドは現在のPCの位置から"ret"命令を検索し、見つかるとプログラムを実行してその"ret"命令の実行後に停止させます。したがって、ニーモニックの"ret"命令がない場合は正しく動作しません。

call, callh (call) [ICD / ICE / SIM / MON]

■ 機　能

ユーザ関数をコールします。(エントリプログラムが必要)
callコマンドの引数は10進数なのに対して、callhコマンドの引数は16進数です。callとcallhの違いは引数の形式のみで、まったく同じ動作をします。

■ 書　式

(1) **call** (ガイダンスモード)
callh

(2) **call <function name> <arg1> [<arg2> [<arg3>]]** (直接入力モード)
callh <function name> <arg1> [<arg2> [<arg3>]]
<function name>: コールする関数
<arg1-3>: 引数(callは10進数またはシンボル、callhは16進数またはシンボル)

■ 入力例

書式1)
```
>call↵
 Function  ? : print_data↵   ...関数名を入力
 Argument1 ? : 1↵            ...引数1を入力
 Argument2 ? : 2↵            ...引数2を入力
 Argument3 ? : ↵             ...引数3がない場合は[Enter]のみを入力

 arg1=1, arg2=2
>
```
[^]キーの入力でひとつ上のパラメータ入力に戻ります。
[q]キーの入力で、コマンドの実行を中止できます。

書式2)
```
>user_func 1 2 3↵
 arg1=1, arg2=2, arg3=3
>
```

■ 動　作

指定のユーザプログラムを実行し、%r10から戻り値を受け取ります。戻り値が-1(0xffffffff)のときは何もせずに終了します。それ以外の値は関数から渡されるパケットの先頭アドレスと解釈し、パケット内のデータを[Command]ウィンドウにテキストとして表示、またはバイナリデータとしてファイルに書き出します。出力先はcallmdコマンドで指定します。デフォルトの出力先は[Command]ウィンドウです。

■ ユーザプログラム

ユーザ関数が返すパケットの構造は次のとおりです。

データサイズ(4バイト)	データ

パケットは必ず4バイト境界から始まらなければなりません。また、ユーザプログラムは、このパケットの先頭アドレスを%r10に設定してください。パケットを返せない場合は%r10に-1を設定します。

コールされるユーザプログラムの例を以下に示します。
ユーザプログラムの最後は、必ず"jp %r15"で終了していなければなりません。

・エントリプログラム(アセンブラ)

```
.global print_data
print_data:
    ld.w    %r9,%sp     ; save SP
    xld.w   %r8,SP_INI  ; new SP value
    ld.w    %sp,%r8     ; set SP
    ld.w    %r8,%r15    ; save return address
    pushn   %r9         ; save registers
    xld.w   %r8,GP_INI  ; set global pointer for safety
    xcall   iprint_data ; enter C program
    popn    %r9         ; restore registers
    ld.w    %r15,%r8    ; restore return address
    ld.w    %sp,%r9     ; restore SP
    jp      %r15        ; end of entry program
```

・ユーザ関数

```
struct {
    int  size;
    char buf[0x100];
} tmpbuf;

int *iprint_data(int arg1, int arg2, int arg3)
{
    int cnt;
    cnt = sprintf(tmpbuf.buf, "arg1=%d, arg2=%d, arg3=%d", arg1, arg2, arg3);
    tmpbuf.size = cnt+1;
    return (int*)&tmpbuf;
}
```

■注　意

・ 出力先がウィンドウの場合、表示されるのは255文字以内です。255文字を越えるデータは表示されません。

・ 上記のエントリプログラムがない関数をコールすると暴走します。

callmd (call mode) [ICD / ICE / SIM / MON]

■ 機　能

call、callhからの戻り値をウィンドウに表示するか、ファイルに出力するかを設定します。

■ 書　式

callmd (ガイダンスモード)

■ 入力例

```
>callmd↵
 Output mode 1.window 2.file 3.window &file ...? 1↵        ...ウィンドウを指定
>
```

ウィンドウを選択すると、戻り値は[Command]ウィンドウに表示されます。

```
>callmd↵
 Output mode 1.window 2.file 3.window &file ...? 2↵        ...ファイルを指定
 File name ? : retvalue.dat↵                               ...ファイル名を入力
>
```

3のウィンドウ&ファイル指定時もファイル名の入力があります。

■ 注　意

・ 本コマンドによる指定のない場合は、[Command]ウィンドウに表示されます。

・ 出力先がウィンドウの場合、表示されるのは255文字以内です。255文字を越えるデータは表示されません。

16.10.5 CPUリセットコマンド

rstc (cold reset CPU) [ICD / ICE / SIM / MON]

■ 機 能

CPUをコールドリセットします。

■ 書 式

rstc (直接入力モード)

※ このコマンド入力と同じ機能は[Run]メニュー/[Reset cold]コマンドまたは[Reset cold]ボタンでも選択できます。

[Reset cold]ボタン

■ リセット内容

CPUのリセットによる初期設定内容は以下のとおりです。

(1) CPUの内部レジスタ

R0～R15:	0xAAAAAAAA
PC:	ブートアドレス(0x80000、0xC00000の内容が示すアドレス)
SP:	0x0AAAAAA8
PSR:	0x00000000
AHR, ALR:	0xAAAAAAAA

(2) 実行カウンタを0に設定

(3) [Source]ウィンドウ、[Register]ウィンドウを再表示

PCがブートアドレスに設定されるため、そのアドレスから再表示します。
[Register]ウィンドウを上記の設定で再表示します。

メモリ内容、ブレークやトレースなどのデバッグステータスはリセットされません。
シミュレータモード以外で使用する場合、バスやI/Oの初期化については各機種の"テクニカルマニュアル"を参照してください。

■ 注 意

各モードによりrstcコマンドの動作が異なります。

ICEモード

上記の処理を行い、S1C33チップもリセットします。ターゲットボードはリセットされません。

ICDモード

上記の処理を行い、S1C33チップもリセットします。ターゲットボードはリセットされません。
また、rstcコマンド実行時にターゲットがフリーランしている場合は、強制ブレークをかけてからリセットを行います。S5U1C33000Hに接続されたターゲットをリセットするとターゲットシステムはフリーラン状態となりますが、rstcコマンドでこれを停止させることができます。

デバッグモニタモード

rstcコマンドはrsthコマンドと同じ動作をします。S1C33チップのリセットやTTBRレジスタの初期化は行いません。

シミュレータモード

パラメータファイルのMCU/MPU指定によりブートアドレスが決定します。

rsth (hot reset CPU) [ICD / ICE / SIM / MON]

■ 機　能

CPUをホットリセットします。

■ 書　式

rsth (直接入力モード)

※ このコマンド入力と同じ機能は[Run]メニュー/[Reset hot]コマンドまたは[Reset hot]ボタンでも選択できます。

[Reset hot]**ボタン**

■ リセット内容

レジスタ、実行カウンタ、ウィンドウの再表示についてはrstcコマンドと同様です。
PC値(ブートアドレス)はTTBRレジスタにより指定されます。
メモリ内容、ブレークやトレースなどのデバッグステータスはリセットされません。
バスやI/Oの状態は保持されます。

16.10.6 割り込みコマンド

int (interrupt) [SIM]

■ 機　能

割り込みの発生をシミュレートします。
本コマンドで割り込み番号を指定すると、次のプログラム実行開始時にその割り込みが発生します。

■ 書　式

(1) **int** (直接入力モード)
(2) **int <type> <level>** (直接入力モード)
<type>: 割り込みタイプ(10進数)
<level>: 割り込みレベル(10進数)
条件: 0≦type>≦215、0≦level>≦15

■ 入力例

書式1) **>int**↵」
パラメータを指定しない場合、NMIを発生します。

書式2) **>int 3 6**↵」
マスク可能な割り込みの番号と割り込みレベルが設定できます。

■ 注　意

・ intコマンドはシミュレータモード時のみ使用可能です。
・ 割り込みタイプは0～215の範囲内で指定してください。これを越えた場合、エラーとなります。
Error: Interrupt type (0-215).
・ 割り込みレベルは0～15の範囲内で指定してください。これを越えた場合、エラーとなります。
Error: Interrupt level (0-15).
・ シミュレータモードでもTTBRは有効です。

16.10.7 ブレーク設定コマンド

bp (break point set) [ICD / ICE / SIM / MON]

■ 機　能

ソフトウェアPCブレークの設定と解除、表示を行います。
プログラムの実行によりPCが設定したアドレスに一致すると、そのアドレスの命令を実行前にブレークします。
最大16個所のアドレスをブレークポイントとして設定可能です。さらに、個々のブレークポイントを有効、無効に設定可能です。

■ 書　式

(1) **bp** (ガイダンスモード)
(2) **bp <No.> <status> [<address>]** (直接入力モード)
<No.>: ブレークポイントNo.(10進数)
<status>: 設定する状態(1=set, 2=enable, 3=disable, 4=clear)
<address>: ブレークアドレス(16進数、シンボルまたはソース行番号)
(<status>=1の場合にのみ指定可能)
条件: 0≦No.≦15、0≦address≦0xffffffff(16ビット境界アドレス)

■ 入力例

(1) **ブレークポイントの表示**

書式1を実行すると現在の設定内容が表示されます。

```
>bp↵
 1 :00C00014/E main          9 :********/
 2 :00C00028/E sub          10 :********/
 3 :00C0002C/D              11 :********/
 4 :********/               12 :********/
 5 :********/               13 :********/
 6 :********/               14 :********/
 7 :********/               15 :********/
 8 :********/               16 :********/
Number ? :↵            ...[Enter]キーのみで終了
>
```

16個のブレークポイントの設定内容を表示します。"********"は未設定のブレークポイントを示します。設定されているアドレスに付加された"/D"はDisable(無効)、"/E"はEnable(有効)を示します。"/E"の付いたアドレスでブレークが発生します。アドレスが設定され、有効(/E)となっているブレークポイントは、[Source]ウィンドウ上のアドレスの前に"!"が表示されます(その行には赤の下線も付きます)。ソース表示モードで、ソースの先頭アドレス以外に設定されている場合は"!"の代わりに"?"が表示されます(その行には青の下線も付きます)。

(2) **新たなブレークアドレスの設定**

書式1) (1)のように現在の設定を表示させ、その後は次のように入力します。

```
Number ? :4↵                                  ...設定するブレークポイントNo.を入力
1.set  2.enable  3.disable  4.clear ...? 1↵   ..."1. set"を選択
Break address ? :C00030↵                      ...ブレークアドレスを16進数で入力
>
```

ブレークアドレスは、ソース行番号やシンボルで指定することもできます。

```
Number ? :5↵
1.set  2.enable  3.disable  4.clear ...? 1↵
Break address ? :main.c#24↵                   ...ブレークアドレスを行番号で入力
>
```

ガイダンス途中で終了するには[Enter]キーのみを入力してください。

書式2) 次のようにコマンドを入力します。

```
>bp 4 1 C00030↵      ...ブレークポイントNo.4をアドレス0xC00030に設定
>bp 5 1 main.c#24↵   ...ブレークポイントNo.5をmain.cの行番号24に設定
```

すでに設定されているブレークポイント番号も指定可能で、その場合は新たに入力したアドレスに変更されます。設定したブレークポイントはすべて有効(/E)となります。
他のブレークポイントNo.に設定されているブレークアドレスを指定することはできません。重複した場合は入力時にエラーとなります。

上記例を実行するとブレークポイントの一覧は次のように変更されます。

```
>bp↵
 1 :00C00014/E main            9 :********/
 2 :00C00028/E sub            10 :********/
 3 :00C0002C/D                11 :********/
 4 :00C00030/E                12 :********/
 5 :00C00032/E main.c#24      13 :********/
 6 :********/                 14 :********/
 7 :********/                 15 :********/
 8 :********/                 16 :********/
Number ? :
```

(3) **無効に設定されたブレークポイントを有効に変更**

/Dが付いたアドレスを持つブレークポイントはブレークを発生しません(上記例のNo.3)。ブレークを有効にするには次のようにコマンドを実行します。

書式1) 現在の設定を表示させ、その後は次のように入力します。

```
Number ? :3↵                                  ...有効にするブレークポイントNo.を入力
1.set  2.enable  3.disable  4.clear ...? 2↵   ..."2. enable"を選択
>
```

アドレスが設定されていないブレークポイントNo.を指定するとエラーとなります。
ガイダンス途中で終了するには[Enter]キーのみを入力してください。

書式2) 次のようにコマンドを入力します。

```
>bp 3 2↵              ...ブレークポイントNo.3を有効に設定
>bp↵
 1 :00C00014/E main            9 :********/
 2 :00C00028/E sub            10 :********/
 3 :00C0002C/E                11 :********/
 4 :00C00030/E                12 :********/
 5 :00C00032/E main.c#24      13 :********/
 6 :********/                 14 :********/
 7 :********/                 15 :********/
 8 :********/                 16 :********/
Number ? :            ...ブレークポイントNo.3は/Eが付き、有効となります。
```

(4) **有効に設定されたブレークポイントを無効に変更**

/Eが付いたアドレスを持つブレークポイントはブレークを発生します。アドレス設定を保持したままブレークを無効にするには次のようにコマンドを実行します。

書式1) 現在の設定を表示させ、その後は次のように入力します。

```
Number ? :2↵                                  ...無効にするブレークポイントNo.を入力
1.set  2.enable  3.disable  4.clear ...? 3↵   ..."3. disable"を選択
>
```

アドレスが設定されていないブレークポイントNo.を指定するとエラーとなります。
ガイダンス途中で終了するには[Enter]キーのみを入力してください。

書式2) 次のようにコマンドを入力します。

```
>bp 2 3↵              ...ブレークポイントNo.2を無効に設定
>bp↵
 1 :00C00014/E main            9 :********/
 2 :00C00028/D sub            10 :********/
 3 :00C0002C/E                11 :********/
 4 :00C00030/E                12 :********/
 5 :00C00032/E main.c#24      13 :********/
 6 :********/                 14 :********/
 7 :********/                 15 :********/
 8 :********/                 16 :********/
Number ? :            ...ブレークポイントNo.2は/Dが付き、無効となります。
```

(5) ブレークポイントの解除

書式1) 現在の設定を表示させ、その後は次のように入力します。

```
Number ? :4↵                                  ...解除するブレークポイントNo.を入力
1.set  2.enable  3.disable  4.clear ...? 4↵   ..."4. clear"を選択
>
```

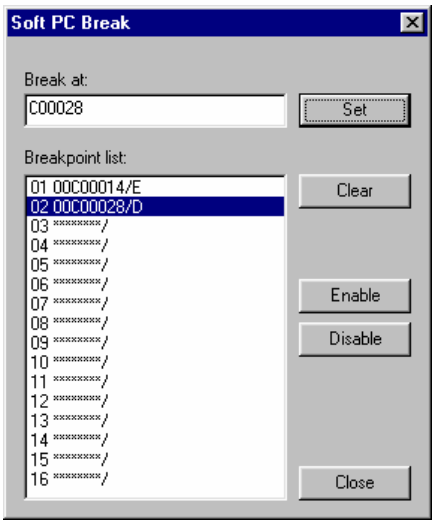
アドレスが設定されていないブレークポイントNo.を指定するとエラーとなります。
ガイダンス途中で終了するには[Enter]キーのみを入力してください。

書式2) 次のようにコマンドを入力します。

```
>bp 4 4↵                ...ブレークポイントNo.4の設定を解除
>bp↵
 1 :00C00014/E main          9 :********/
 2 :00C00028/D sub          10 :********/
 3 :00C0002C/E              11 :********/
 4 :********/               12 :********/
 5 :00C00032/E main.c#24    13 :********/
 6 :********/               14 :********/
 7 :********/               15 :********/
 8 :********/               16 :********/
Number ? :              ...ブレークポイントNo.4の設定が解除されます。
```

■ メニューによる設定

[Break]メニュー/[Soft PC...]コマンドを選択すると、次のダイアログボックスが表示されます。



ブレークアドレスの登録

アドレスを[Break at]テキストボックスに16進数、シンボルまたはソース行番号で入力し、[Enter]キーを押すか、[Set]ボタンをクリックします。ブレークポイント番号の昇順に登録されます。

ブレークポイントの解除

[Breakpoint list]から解除するアドレスを選択し、[Clear]ボタンをクリックします。

有効/無効の切り換え

登録したブレークアドレスを無効に設定するには、リストから無効にするアドレスを選択し、[Disable]ボタンをクリックします。アドレスに付いた/Eが/Dに変わります。[Enable]ボタンは無効(/D)に設定されたブレークポイントを有効(/E)に切り換えます。

■注　意

・ ブレークポイントNo.は1～16までが有効です。17以上を指定するとエラーとなります。
　　Error: Invalid value.

・ アドレス入力は"1. Set"を選択した場合にのみ行えます。直接入力(書式2)で"1. Set"以外にアドレスを入力した場合はエラーとなります。
　　Error: Invalid value.

・ PCアドレスがブレーク条件のため、ブレークポイントは16ビット境界アドレスに設定される必要があります。奇数アドレスを指定した場合は、強制的に最下位ビットを0として扱います。

・ 行番号などで実コードを持たないソース行をブレークポイントに指定した場合、ワーニングとなります。ブレークポイントはその行の直後にあるコードのアドレスに設定されます。
　　Warning: Invalid line, move to next valid line.

・ アドレスは、各機種のメモリ領域範囲内で指定してください。これを越えた場合、エラーとなります。書式1では再度、アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
　　Error: Address range (0-0xFFFFFFF).　　...0xfffffffを越えるアドレス
　　Error: No map area.　　...未使用アドレスを指定した場合

・ すでに設定されているアドレスを再設定するとエラーとなります。
　　Error: Already exist input address.

・ 設定されていないブレークポイントを有効化/無効化、あるいは解除しようとするとエラーとなります。書式1では再度、アドレス入力のガイダンスが表示されます。
　　Error: Invalid break number.

　書式2ではエラーとはなりませんが、コマンド入力がキャンセルされます。

・ ソフトウェアPCブレークは、BRK命令の埋め込みにより実現しています。そのため、命令の埋め込みができないターゲットボード上のROMに対しては使用できません。その際は、ハードウェアPCブレークを使用してください。

・ ソフトウェアPCブレークを、extによる拡張命令または遅延分岐命令のアドレスに設定する場合、先頭アドレス以外には設定できません。

ext xxxx	... ○ 設定可	jr*.d xxxx	... ○ 設定可
ext xxxx	... × 設定不可	遅延命令	... × 設定不可
拡張される命令	... × 設定不可		

bs (break software) [ICD / ICE / SIM / MON]

■ 機　能

ソフトウェアPCブレークアドレスを未設定のブレークポイントNo.に設定します。

■ 書　式

(1) **bs** (ガイダンスモード)
(2) **bs <address>** (直接入力モード)
<address>: ブレークアドレス(16進数、シンボルまたはソース行番号)
条件: 0≦address≦0xffffff(16ビット境界アドレス)

■ 入力例

書式1)
```
>bs↵
Break address ? :C00016↵          ...アドレスを16進数で指定
>bs
Break address ? :main.c#24↵       ...アドレスを行番号で指定
>
```

書式2)
```
>bs C00016↵                       ...アドレスを16進数で指定
>bs main.c#24↵                    ...アドレスを行番号で指定
>
```

未設定のブレークポイントの中で、番号の小さいものから順に割り当てられます。設定されたブレークポイントはブレークが有効(/E)となります。

■ ツールバーボタンによる設定

[Source]ウィンドウと[Soft PC break]ボタンによっても、bsコマンドと同様にソフトウェアPCブレークポイントの設定が行えます。

[Soft PC break]**ボタン**

[Source]ウィンドウ内で、ブレークポイントを設定するアドレスの行をクリック(カーソルを移動)して[Soft PC break]ボタンを押してください。その行の先頭に"!"(行には赤の下線)または"?"(行には青の下線)が付き、ブレークポイント(有効)に設定されます。

■ 注　意

- すでに16個のブレークポイントが設定されている場合はエラーとなります。
  Error: Cannot set address any more.
- PCアドレスがブレーク条件のため、ブレークポイントは16ビット境界アドレスに設定される必要があります。奇数アドレスを指定した場合は、強制的に最下位ビットを0として扱います。
- 行番号や[Source]ウィンドウ上で実コードを持たないソース行をブレークポイントに指定した場合、ワーニングとなります([Soft PC break]ボタンを使用した場合、ワーニングメッセージは表示されません)。ブレークポイントはその行の直後にあるコードのアドレスに設定されます。
  Warning: Invalid line, move to next valid line.
- アドレスは、各機種のメモリ領域範囲内で指定してください。これを越えた場合、エラーとなります。書式1では再度、アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
  Error: Address range (0-0xFFFFFFF). ...0xfffffffを越えるアドレス
  Error: No map area. ...未使用アドレスを指定した場合
- すでに設定されているアドレスを再設定するとエラーとなります。
  Error: Already exist input address.
- ソフトウェアPCブレークは、BRK命令の埋め込みにより実現しています。そのため、命令の埋め込みができないターゲットボード上のROMに対しては使用できません。その際は、ハードウェアPCブレークを使用してください。
- ソフトウェアPCブレークを、extによる拡張命令または遅延分岐命令のアドレスに設定する場合、先頭アドレス以外には設定できません。

ext xxxx	... ○ 設定可	jr*.d xxxx	... ○ 設定可
ext xxxx	... × 設定不可	遅延命令	... × 設定不可
拡張される命令	... × 設定不可		

bc (break clear) [ICD / ICE / SIM / MON]

■ 機　能

bpコマンド、bsコマンド、[Break]メニュー/[Soft PC...]コマンド、または[Soft PC break]ボタンで設定したソフトウェアPCブレークポイントを解除します。

■ 書　式

bc <address> (直接入力モード)
<address>: ブレークアドレス(16進数、シンボルまたはソース行番号)
条件: addressはブレークポイントに設定されているアドレスのみ指定可能

■ 入力例

```
>bc C00016↵          ...アドレスを16進数で指定
>bc main.c#24↵       ...アドレスを行番号で指定
>
```

■ ツールバーボタンによる解除

[Source]ウィンドウと[Soft PC break]ボタンによっても、ソフトウェアPCブレークポイントが解除できます。

[Soft PC break]**ボタン**

[Source]ウィンドウ内で、ブレークポイントに設定されている行(アドレスの先頭に"!"または"?"が表示されている行)をクリックして[Soft PC break]ボタンを押すと、そのブレークアドレスが解除されます。

■ 注　意

- ブレークポイントの解除はbpコマンド、[Break]メニュー/[Soft PC...]コマンドでも行えます。(bpコマンド参照)
- 奇数アドレスを指定した場合は、強制的に最下位ビットを0として扱います。
- 行番号や[Source]ウィンドウ上で実コードを持たないソース行を指定した場合、ワーニングとなります([Soft PC break]ボタンを使用した場合、ワーニングメッセージは表示されません)。その行の直後にあるブレークポイントが解除されます。
  Warning: Invalid line, move to next valid line.
- 設定されていないブレークアドレスを指定するとエラーとなります。
  Error: Invalid break number.

bh (break hardware) [ICD / ICE / SIM / MON]

■ 機　能

ハードウェアPCブレークアドレス1を表示または設定します。

■ 書　式

(1) **bh** ...表示(直接入力モード)

(2) **bh <address>** ...設定(直接入力モード)

<address>: ブレークアドレス(16進数、シンボルまたはソース行番号)

条件: 0≦address≦0xffffffff(16ビット境界アドレス)

■ 入力例

書式1)

```
>bh↵
Address  :********↵
```
...設定されていない場合

```
>bh↵
Address  :C00032 main.c#24↵
```
...行番号で設定されている場合

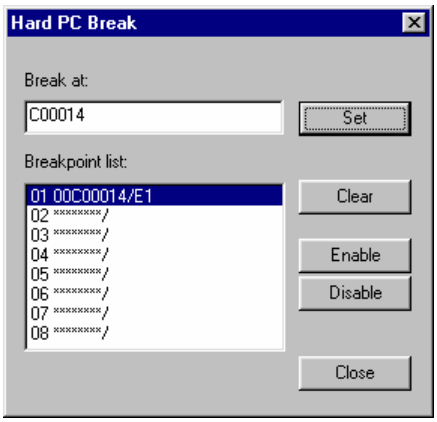
書式2)

```
>bh C00016↵
```
...アドレスを16進数で指定

```
>bh main.c#24↵
```
...アドレスを行番号で指定

設定可能なアドレスは1個所のみで、後から指定したアドレスが有効です。
[Source]ウィンドウが開いている場合、ハードウェアPCブレークポイントに設定されたアドレスの直後に"!"が表示されます(その行には赤の下線も付きます)。ソース表示モードで、ソースの先頭アドレス以外に設定された場合は"!"の代わりに"?"が表示されます(その行には青の下線も付きます)。

■ メニューによる設定



[Break]メニュー/[Hard PC...]コマンドを選択すると、ダイアログボックスが表示されます。
8個所までのアドレスをハードウェアPCブレークポイントとして登録可能で、この内の2個所を有効なブレークポイントとして設定できます。

ブレークアドレスの登録

アドレスを[Break at]テキストボックスに入力し、[Enter]キーを押すか、[Set]ボタンをクリックします。このとき、有効なブレークポイントがすでに2個所あるときは、無効なブレークポイントとして登録されます。同時に有効にできるブレークポイントは2個所です。また、アドレスはシンボルでも入力可能です。

ブレークポイントの解除

[Breakpoint list]から解除するアドレスを選択し、[Clear]ボタンをクリックします。

有効/無効の切り換え

ブレークアドレスの登録で、有効なブレークポイントとなるのは最初の2個所までです。有効なブレークポイントはリスト内のアドレスの後に/E1または/E2が表示されます。無効なブレークポイントはアドレスの最後に/Dが表示されます。
有効/無効を切り換えるには、リストから有効あるいは無効にしたいアドレスを選択し、有効にする場合は[Enable]、無効にする場合は[Disable]のボタンをクリックします。3個所目以降のブレークポイントを有効にしようとしても[Enable]ボタンは機能しません。この場合は、先にブレークポイントを1個所無効にしてから、有効にするブレークポイントを設定してください。

■ ツールバーボタンによる設定

[Source]ウィンドウと[Hard PC break]ボタンによっても、bhコマンドと同様にハードウェアPCブレークポイントの設定が行えます。

[Hard PC break]**ボタン**

[Source]ウィンドウ内で、ブレークポイントを設定するアドレスの行をクリック(カーソルを移動)して[Hard PC break]ボタンを押してください。そのアドレスがブレークポイントに設定されます。

■ 注　意

・ アドレスは、各機種のメモリ領域範囲内で指定してください。これを越えた場合、エラーとなります。書式1では再度、アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。

Error: Address range (0-0xFFFFFFF).　...0xffffffffを越えるアドレス
Error: No map area.　...未使用アドレスを指定した場合

・ ハードウェアPCブレークを、extによる拡張命令または遅延分岐命令のアドレスに設定する場合、先頭アドレス以外には設定できません。

ext xxxx	... ○ 設定可	jr*.d xxxx	... ○ 設定可
ext xxxx	... × 設定不可	遅延命令	... × 設定不可
拡張される命令	... × 設定不可		

・ ICDモードで範囲トレースを設定している場合、ハードウェアPCブレーク機能は無効となります。設定はクリアされず、範囲トレース機能を解除した時点で有効な状態に戻ります。

bhc (break hardware clear) [ICD / ICE / SIM / MON]

■ 機　能

bhコマンドまたは[Break]メニュー/[Hard PC...]コマンドで設定したハードウェアPCブレークポイント1を解除します。

■ 書　式

bhc (直接入力モード)

■ 入力例

```
>bhc↵」          ...ハードウェアPCブレークポイント1を解除
>
```

■ メニューによる解除

[Break]メニュー/[Hard PC...]コマンドを選択すると、ダイアログボックスが表示されます(bhコマンド参照)。
ハードウェアPCブレークポイント1を解除するには、01のブレークアドレスを選択し、[Clear]ボタンをクリックしてください。

■ ツールバーボタンによる解除

[Source]ウィンドウと[Hard PC break]ボタンによっても、ハードウェアPCブレークポイントが解除できます。

[Hard PC break]**ボタン**

[Source]ウィンドウ内で、ハードウェアPCブレークポイントに設定されている行(アドレスの後に"!"または"?"が表示されている行)をクリックして[Hard PC break]ボタンを押すと、そのブレークアドレスが解除されます。

bh2 (break hardware2) [ICD / (ICE) / SIM / MON]

■ 機　能

ハードウェアPCブレークアドレス2を表示または設定します。

■ 書　式

(1) **bh2** ...表示(直接入力モード)

(2) **bh2 <address>** ...設定(直接入力モード)

<address>: ブレークアドレス(16進数、シンボルまたはソース行番号)

条件: 0≦address≦0xffffffff(16ビット境界アドレス)

■ 入力例

書式1)

```
>bh2↵
Address  :********↵
```
...設定されていない場合

```
>bh2↵
Address  :C00032 main.c#24↵
```
...行番号で設定されている場合

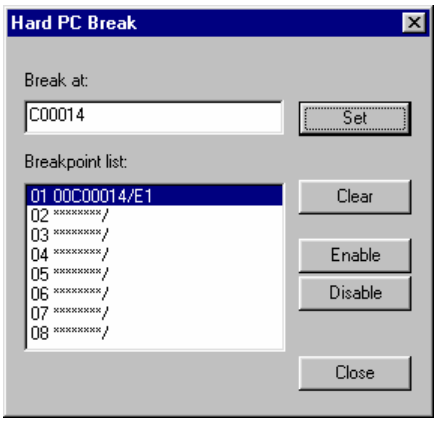
書式2)

```
>bh2 C00016↵
```
...アドレスを16進数で指定

```
>bh2 main.c#24↵
```
...アドレスを行番号で指定

設定可能なアドレスは1個所のみで、後から指定したアドレスが有効です。
[Source]ウィンドウが開いている場合、ハードウェアPCブレークポイントに設定されたアドレスの直後に"!"が表示されます(その行には赤の下線も付きます)。ソース表示モードで、ソースの先頭アドレス以外に設定された場合は"!"の代わりに"?"が表示されます(その行には青の下線も付きます)。

■ メニューによる設定



[Break]メニュー/[Hard PC...]コマンドを選択すると、ダイアログボックスが表示されます。
8個所までのアドレスをハードウェアPCブレークポイントとして登録可能で、この内の2個所を有効なブレークポイントとして設定できます。

ブレークアドレスの登録

アドレスを[Break at]テキストボックスに入力し、[Enter]キーを押すか、[Set]ボタンをクリックします。このとき、有効なブレークポイントがすでに2個所あるときは、無効なブレークポイントとして登録されます。同時に有効にできるブレークポイントは2個所です。また、アドレスはシンボルでも入力可能です。

ブレークポイントの解除

[Breakpoint list]から解除するアドレスを選択し、[Clear]ボタンをクリックします。

有効/無効の切り換え

ブレークアドレスの登録で、有効なブレークポイントとなるのは最初の2個所までです。有効なブレークポイントはリスト内のアドレスの後に/E1または/E2が表示されます。無効なブレークポイントはアドレスの最後に/Dが表示されます。
有効/無効を切り換えるには、リストから有効あるいは無効にしたいアドレスを選択し、有効にする場合は[Enable]、無効にする場合は[Disable]のボタンをクリックします。3個所目以降のブレークポイントを有効にしようとしても[Enable]ボタンは機能しません。この場合は、先にブレークポイントを1個所無効にしてから、有効にするブレークポイントを設定してください。

■ ツールバーボタンによる設定

[Source]ウィンドウと[Hard PC break]ボタンによっても、bh2コマンドと同様にハードウェアPCブレークポイントの設定が行えます。

[Hard PC break]**ボタン**

[Source]ウィンドウ内で、ブレークポイントを設定するアドレスの行をクリック(カーソルを移動)して[Hard PC break]ボタンを押してください。そのアドレスがブレークポイントに設定されます。

■ 注　意

・ アドレスは、各機種のメモリ領域範囲内で指定してください。これを越えた場合、エラーとなります。書式1では再度、アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。

Error: Address range (0-0xFFFFFFF).	...0xffffffffを越えるアドレス
Error: No map area.	...未使用アドレスを指定した場合

・ ハードウェアPCブレークを、extによる拡張命令または遅延分岐命令のアドレスに設定する場合、先頭アドレス以外には設定できません。

ext xxxx	... ○ 設定可	jr*.d xxxx	... ○ 設定可
ext xxxx	... × 設定不可	遅延命令	... × 設定不可
拡張される命令	... × 設定不可		

・ ICDモードで範囲トレースを設定している場合、ハードウェアPCブレーク機能は無効となります。設定はクリアされず、範囲トレース機能を解除した時点で有効な状態に戻ります。

・ ICEモードで使用する場合、S5U1C33104Hのバージョンが2.0以上である必要があります。

bhc2 (break hardware2 clear) [ICD / (ICE) / SIM / MON]

■ 機　能

bh2コマンドまたは[Break]メニュー/[Hard PC...]コマンドで設定したハードウェアPCブレークポイント2を解除します。

■ 書　式

bhc2 (直接入力モード)

■ 入力例

```
>bhc2↵
>
```

...ハードウェアPCブレークポイント2を解除

■ メニューによる解除

[Break]メニュー/[Hard PC...]コマンドを選択すると、ダイアログボックスが表示されます(bh2コマンド参照)。
ハードウェアPCブレークポイント2を解除するには、02のブレークアドレスを選択し、[Clear]ボタンをクリックしてください。

■ ツールバーボタンによる解除

[Source]ウィンドウと[Hard PC break]ボタンによっても、ハードウェアPCブレークポイントが解除できます。

[Hard PC break]**ボタン**

[Source]ウィンドウ内で、ハードウェアPCブレークポイントに設定されている行(アドレスの後に"!"または"?"が表示されている行)をクリックして[Hard PC break]ボタンを押すと、そのブレークアドレスが解除されます。

■ 注　意

ICEモードで使用する場合、S5U1C33104Hのバージョンが2.0以上である必要があります。

bph (break point set hardware) [ICD / ICE / SIM / MON]

■ 機　能

ハードウェアPCブレークの設定と解除、表示を行います。
最大8個所のアドレスが登録可能です。同時にブレークポイントとして設定可能なアドレスは2個所のため、必要に応じて有効あるいは無効に切り換えて使用します。

■ 書　式

(1) **bph** (ガイダンスモード)
(2) **bph <No.> <status> [<address>]** (直接入力モード)

<No.>: ブレークポイントNo.(10進数)
<status>: 設定する状態(1=set, 2=enable, 3=disable, 4=clear)
<address>: ブレークアドレス(16進数、シンボルまたはソース行番号)
(<status>=1の場合にのみ指定可能)
条件: 0≦No.≦8、0≦address≦0xffffff(16ビット境界アドレス)

■ 入力例

(1) **ブレークポイントの表示**

書式1を実行すると現在の設定内容が表示されます。

```
>bph↵
 1 :00C00300/E1 sys.c#100      5 :********/
 2 :00C00010/E2                6 :********/
 3 :00C00480/D                 7 :********/
 4 :********/                  8 :********/
Number ? :↵              ...[Enter]キーのみで終了
>
```

8個のアドレスの登録内容を表示します。"********"は未登録のアドレスを示します。登録されているアドレスに付加された記号の意味は、以下のとおりです。
/E1: ハードウェアPCブレークポイント1としてEnable(有効)
/E2: ハードウェアPCブレークポイント2としてEnable(有効)
/D: ハードウェアPCブレークポイントとしてDisable(無効)

[Source]ウィンドウが開いている場合、ハードウェアPCブレークポイントに設定されたアドレスの直後に"!"が表示されます(その行には赤の下線も付きます)。ソース表示モードで、ソースの先頭アドレス以外に設定された場合は"!"の代わりに"?"が表示されます(その行には青の下線も付きます)。

(2) **新たなブレークアドレスの設定**

(1)のように現在の設定を表示させ、その後は次のように入力します。

```
Number ? :4↵                                  ...設定するブレークポイントNo.を入力
1.set  2.enable  3.disable  4.clear ...? 1↵   ..."1. set"を選択
Break address ? :C00030↵                      ...ブレークアドレスを16進数で入力
>
```

有効(/Ex)の設定が2個所未満のときは有効(/Ex)として登録されます。すでに2個所が有効(/Ex)に設定されているときは、無効(/D)として登録されます。

ブレークアドレスは、ソース行番号やシンボルで指定することもできます。

```
Number ? :5↵
1.set  2.enable  3.disable  4.clear ...? 1↵
Break address ? :main.c#24↵                   ...ブレークアドレスを行番号で入力
>
```

ガイダンス途中で終了するには[Enter]キーのみを入力してください。

書式2では次のようにコマンドを入力します。

```
>bph 4 1 C00030↵        ...アドレスNo.4を0xC00030に設定
>bph 5 1 main.c#24↵     ...アドレスNo.5をmain.cの行番号24に設定
```

上記例を実行するとブレークポイントの一覧は次のように変更されます。

```
>bph↵
 1 :00C00300/E1 sys.c#100       5 :00C00032/D main.c#24
 2 :00C00010/E2                 6 :********/
 3 :00C00480/D                  7 :********/
 4 :00C00030/D                  8 :********/
Number ? :
```

すでに登録されているアドレス番号も指定可能で、その場合は新たに入力したアドレスに変更されます。変更前のアドレスが無効(/D)として登録されていた場合でも、有効(/Ex)の設定が2個所未満のときは有効(/Ex)として登録されます。変更前が有効(/Ex)の場合、あるいはすでに2個所が有効(/Ex)に設定されている場合、有効/無効の状態は変わりません。

(3) 有効に設定されたブレークポイントを無効に変更

すでに2個所のブレークポイントが有効(/Ex)に設定されている場合、他のアドレスを有効に設定することはできません。この場合は、先に1個所を無効(/D)に設定する必要があります。
アドレス設定を保持したままブレークを無効にするには次のようにコマンドを実行します。

書式1) 現在の設定を表示させ、その後は次のように入力します。

```
Number ? :2↵                                    ...無効にするブレークアドレスNo.を入力
1.set  2.enable  3.disable  4.clear ...? 3↵     ..."3. disable"を選択
>
```

アドレスが登録されていない番号を指定するとエラーとなります。
ガイダンス途中で終了するには[Enter]キーのみを入力してください。

書式2) 次のようにコマンドを入力します。

```
>bph 2 3↵               ...ブレークアドレスNo.2を無効に設定
>bph↵
 1 :00C00300/E1 sys.c#100       5 :00C00032/D main.c#24
 2 :00C00010/D                  6 :********/
 3 :00C00480/D                  7 :********/
 4 :00C00030/D                  8 :********/
Number ? :              ...ブレークアドレスNo.2は/Dが付き、無効となります。
```

(4) 無効に設定されたブレークアドレスを有効に変更

無効(/D)として登録されているアドレスをブレークポイントとして有効にするには、次のようにコマンドを実行します。ただし、有効(/Ex)なブレークポイントを1個所以下にしておく必要があります。

書式1) 現在の設定を表示させ、その後は次のように入力します。

```
Number ? :3↵                                    ...有効にするブレークアドレスNo.を入力
1.set  2.enable  3.disable  4.clear ...? 2↵     ..."2. enable"を選択
>
```

アドレスが登録されていない番号を指定するとエラーとなります。
すでに2個所を有効(/Ex)に設定済みのとき、"2. enable"を選択するとエラーとなります。
ガイダンス途中で終了するには[Enter]キーのみを入力してください。

書式2) 次のようにコマンドを入力します。

```
>bph 3 2↵               ...ブレークポイントNo.3を有効に設定
>bph↵
 1 :00C00300/E1 sys.c#100       5 :00C00032/D main.c#24
 2 :00C00010/D                  6 :********/
 3 :00C00480/E2                 7 :********/
 4 :00C00030/D                  8 :********/
Number ? :              ...ブレークポイントNo.3は/E2が付き、有効となります。
```

(5) 登録アドレスの抹消

書式1) 現在の設定を表示させ、その後は次のように入力します。

```
Number ? :1↵                                  ...登録を抹消するアドレスNo.を入力
1.set  2.enable  3.disable  4.clear ...? 4↵   ..."4. clear"を選択
>
```

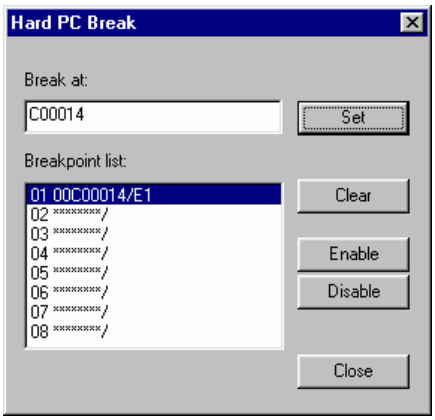
アドレスが登録されていない番号を指定するとエラーとなります。
ガイダンス途中で終了するには[Enter]キーのみを入力してください。

書式2) 次のようにコマンドを入力します。

```
>bph 1 4↵              ...アドレスNo.1の登録を抹消
>bph↵
 1 :********/                  5 :00C00032/D main.c#24
 2 :00C00010/D                 6 :********/
 3 :00C00480/E2                7 :********/
 4 :00C00030/D                 8 :********/
Number ? :              ...ブレークポイントNo.4の設定が解除されます。
```

■ メニューによる設定

[Break]メニュー/[Hard PC...]コマンドを選択すると、次のダイアログボックスが表示されます。



ブレークアドレスの登録

アドレスを[Break at]テキストボックスに入力し、[Enter]キーを押すか、[Set]ボタンをクリックします。このとき、有効なブレークポイントがすでに2個所あるときは、無効なブレークポイントとして登録されます。同時に有効にできるブレークポイントは2個所です。また、アドレスはシンボルでも入力可能です。

ブレークポイントの解除

[Breakpoint list]から解除するアドレスを選択し、[Clear]ボタンをクリックします。

有効/無効の切り換え

ブレークアドレスの登録で、有効なブレークポイントとなるのは最初の2個所までです。有効なブレークポイントはリスト内のアドレスの後に/E1または/E2が表示されます。無効なブレークポイントはアドレスの最後に/Dが表示されます。
有効/無効を切り換えるには、リストから有効あるいは無効にしたいアドレスを選択し、有効にする場合は[Enable]、無効にする場合は[Disable]のボタンをクリックします。3個所目以降のブレークポイントを有効にしようとしても[Enable]ボタンは機能しません。この場合は、先にブレークポイントを1個所無効にしてから、有効にするブレークポイントを設定してください。

■注　意

・すでに設定されているアドレスを再設定するとエラーとなります。
　　Error: Already exist input address.

・設定されていないブレークポイントを有効化/無効化、あるいは解除しようとするとエラーとなります。書式1では再度、アドレス入力のガイダンスが表示されます。
　　Error: Invalid break number.

　書式2ではエラーとはなりませんが、コマンド入力がキャンセルされます。

・ブレークアドレスNo.は1～8までが有効です。9以上を指定するとエラーとなります。
　　Error: Invalid value.

・アドレス入力は"1. Set"を選択した場合にのみ行えます。直接入力(書式2)で"1. Set"以外にアドレスを入力した場合はエラーとなります。
　　Error: Invalid value.

・PCアドレスがブレーク条件のため、ブレークアドレスは16ビット境界に設定される必要があります。奇数アドレスを指定した場合は、強制的に最下位ビットを0として扱います。

・行番号などで実コードを持たないソース行をブレークアドレスに指定した場合、ワーニングとなります。ブレークアドレスはその行の直後にあるコードのアドレスに設定されます。
　　Warning: Invalid line, move to next valid line.

・アドレスは、各機種のメモリ領域範囲内で指定してください。これを越えた場合、エラーとなります。書式1では再度、アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。
　　Error: Address range (0-0xFFFFFFF).　　...0xffffffffを越えるアドレス
　　Error: No map area.　　...未使用アドレスを指定した場合

・ハードウェアPCブレークを、extによる拡張命令または遅延分岐命令のアドレスに設定する場合、先頭アドレス以外には設定できません。

ext xxxx	... ○ 設定可	jr*.d xxxx	... ○ 設定可
ext xxxx	... × 設定不可	遅延命令	... × 設定不可
拡張される命令	... × 設定不可		

・ICDモードで範囲トレースを設定している場合、ハードウェアPCブレーク機能は無効となります。設定はクリアされず、範囲トレース機能を解除した時点で有効な状態に戻ります。

bd (data break) [ICD / ICE / SIM / MON]

■ 機　能

データブレークの設定と解除、表示を行います。
本コマンドでは次のブレーク条件が指定できます。
1. アクセスするメモリアドレス(1個所)
2. メモリのリード/ライト(リード、ライト、リードまたはライトの3条件)

プログラムは、上記条件を満たすメモリアクセスの終了後にブレークします。

■ 書　式

(1) **bd** (ガイダンスモード)
(2) **bd <mode> <address> {r | w | *}** (直接入力モード)
<mode>: 設定/解除の指定(1=set, 2=clear)
<address>: アドレス(16進数、またはシンボル)
r, w, *: アクセス条件(いずれか1つを入力)
r: リード
w: ライト
*: リードまたはライト
条件: 0≦address≦0xffffff

■ 入力例

(1) データブレーク条件の表示

書式1を実行すると現在の設定内容が表示されます。

```
>bd↵
Address  : ********
R/W/*    : -
1.set  2.clear ...? ↵          ...[Enter]キーのみの入力で終了
>
```

上記例はデータブレーク条件が未設定の場合の表示です。

(2) データブレーク条件の設定

書式1) (1)のように現在の設定を表示させ、その後は次のように入力します。

```
1.set  2.clear ...? 1↵         ..."1. set"を選択
Address      :100↵             ...アドレスを16進数で入力
R/W (R,W,*)  :*↵               ...アクセス条件を入力(*はR/W)
>
```

アドレスは、シンボルで指定することもできます。

```
1.set  2.clear ...? 1↵
Address      :i↵               ...アドレスをシンボルで入力
R/W (R,W,*)  :w↵
>
```

ガイダンス途中で終了するには[Enter]キーのみを入力してください。

書式2) 次のようにコマンドを入力します。

```
>bd 1 100 *↵          ...アドレスを0x100、アクセス条件をR/Wに設定
>bd 1 i w↵            ...アドレスを変数i、アクセス条件をWに設定
```

すでに条件が設定されている場合は、新たに入力した条件に変更されます。

(3) データブレーク条件の解除

書式1) 現在の設定を表示させ、その後は次のように入力します。

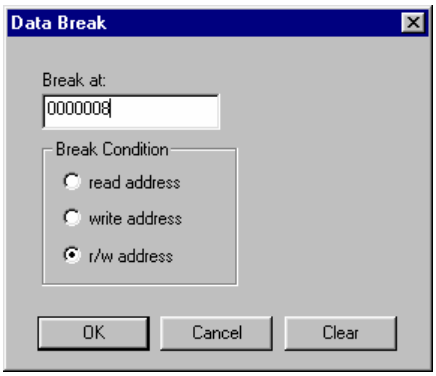
```
>bd↵
Address  : 00000000 i
R/W/*    : W
1.set  2.clear ...? 2↵          ..."2. clear"を選択
>
```

ガイダンス途中で終了するには[Enter]キーのみを入力してください。

書式2) 次のようにコマンドを入力します。

```
>bd 2↵
```

■ メニューによる設定



[Break]メニュー/[Data...]コマンドを選択すると、ダイアログボックスが表示されます。
アドレスは[Break at]テキストボックスに16進数またはシンボルで入力してください。
アクセス条件はラジオボタンで選択してください。

解除する場合は[Clear]ボタンをクリックします。

■ 注　意

・ アドレスは、各機種のメモリ領域範囲内で指定してください。これを越えた場合、エラーとなります。書式1では再度、アドレス入力のガイダンスが表示されます。書式2ではコマンド入力がキャンセルされます。

Error: Address range (0-0xFFFFFFF).　　...0xfffffffを越えるアドレス
Error: No map area.　　...未使用アドレスを指定した場合

・ プログラムはブレーク条件成立から1～数命令後に停止します。

bsq (break sequential) [ICE]

■ 機　能

シーケンシャルブレークの設定と解除、表示を行います。
アドレス、データ、データマスクおよびバスオペレーションタイプの組み合わせを3種類まで設定できます。CPUが設定した条件を満たすバスオペレーションを設定した順序で実行するとブレークします。

■ 書　式

(1) **bsq** (ガイダンスモード)

(2) **bsq <mode> <hit> [<condition1> [<condition2> [<condition3>]]]** (直接入力モード)

<mode>: 設定/解除の指定(1=set, 2=clear)
<hit>: シーケンス指定(1=Hit 1のみ, 2=Hit1～2, 3=Hit1～3)
<condition>: ブレーク条件(<address> <data> <mask> <bus type>)
<address>: アドレス(16進数、シンボルまたはソース行番号)
<data>: データ(16進数、指定アドレスアクセス時に比較する16ビットデータ)
<mask>: データマスク(16進数、データのマスクを指定する16ビットデータ)
"0"に設定されたビットがデータを比較するビットとなり、"1"に設定されたビットはマスクされます。
<bus type>: バスオペレーションタイプ(0～8の10進数)

0: All	すべてのバスオペレーション
1: Inst	命令フェッチ
2: VecR	ベクタリード
3: DatR	データリード
4: DatW	データライト
5: StkR	スタックリード
6: StkW	スタックライト
7: DmaR	DMAリード
8: DmaW	DMAライト

条件: 0≦address≦0xffffff

■ 入力例

(1) **シーケンシャルブレーク条件の表示**

書式1を実行すると現在の設定内容が表示されます。

```
>bsq↵
Hit1
 Address      : 00C80000
 Data pattern : 6C00
 Data mask    : 0000
 Type         : Inst
Hit2
 Address      : 00C80002
 Data pattern : 6C11
 Data mask    : 0000
 Type         : Inst
Hit3
 Address      : ********
 Data pattern : ****
 Data mask    : ****
 Type         : ****
1.set  2.clear ...? ↵          ...[Enter]キーのみの入力で終了
>
```

上記例は条件1と条件2が設定されている場合の表示です。条件3の"****"は条件が設定されていないことを示します。データマスクの"0000"は、指定データの16ビットをすべて比較対象とすることを示しています。この条件の場合、CPUがアドレス0xc80000から命令コード0x6c00をフェッチし、その後アドレス0xc80002から命令コード0x6c11をフェッチするとブレークが発生します。順序が逆、あるいは条件2のみの成立ではブレークしません。

(2) シーケンシャルブレーク条件の設定

書式1) (1)のように現在の設定を表示させ、その後は次のように入力します。

```
1.set  2.clear ...? 1↵                          ..."1. set"を選択
Number of sequential address (1-3) ? :3↵        ...シーケンス数を選択(3=Hit1～3)
Address      00C80000 :↵                        ...条件1のアドレス
Data pattern 6C00     :↵                        ...[Enter]キーのみでスキップ(変更なし)
Data mask    0000     :↵
Bus type  0: All 1:Inst 2:VecR 3:DatR 4:DatW 5:StkR 6:StkW 7:DmaR 8:DmaW...? Inst :↵
Address      00C80002 :↵                        ...条件2のアドレス
Data pattern 6C11     :↵
Data mask    0000     :↵
Bus type  0: All 1:Inst 2:VecR 3:DatR 4:DatW 5:StkR 6:StkW 7:DmaR 8:DmaW...? Inst :↵
Address      ******** :d000000↵                 ...条件3のアドレスを入力
Data pattern ****     :0012↵                    ...条件3のデータを入力
Data mask    ****     :^↵                       ...[^]キーで1つ前の項目へ
Data pattern 0012     :1200↵                    ...条件3のデータを再入力
Data mask    ****     :00ff↵                    ...条件3のデータマスクを入力
Bus type  0: All 1:Inst 2:VecR 3:DatR 4:DatW 5:StkR 6:StkW 7:DmaR 8:DmaW...? Inst :4↵
>                                               ...条件3のバスオペレーションタイプを入力
```

この例では、条件1と2を変更せずに、条件3を新たに設定しています。
条件3は、アドレス0xd000000にバイトデータ0x12が書き込まれた場合にブレークさせる設定です。データは16ビットで指定しますが、バイトアクセスに対応するためには、この指定のようにデータマスクで対応する必要があります。バイトアクセスの場合、偶数アドレスはデータバスの上位8ビット、奇数アドレスはデータバスの下位8ビットを使用します。したがって、この例のように偶数アドレスに0x12を書き込んだ場合の設定は、データを0x1200と指定し、下位8ビットをデータマスク0x00ffで無効にしておきます。奇数アドレスに対しては、データを0x0012、データマスクを0xff00と指定します。

アドレスは、シンボルあるいはソース行番号で指定することもできます。

```
Address      ******** :i↵                       ...シンボル入力例
Address      ******** :main.c#24↵               ...行番号入力例
```

ガイダンス途中でコマンドを終了するには[q] [Enter]を入力してください。途中で終了した場合、そこまでに設定した内容は変更されます。

複数の条件を設定する場合は、必ずシーケンシャルブレークの順序で入力してください。条件2や3から入力することはできません。
シーケンシャルブレークは、設定された順序で全条件が成立した場合にのみ発生します。

書式2) 次のようにコマンドを入力します。

```
>bsq 1 1 c80000 6c00 0000 1↵
```

...(1)の条件1と同じ設定です。各項目はスペースで区切って入力してください。
この設定の前に、条件2と3が設定されていた場合、それらの条件はクリアされます。

複数の条件を設定する場合も1行に入力してください。

```
>bsq 1 2 c80000 6c00 0000 1 c80002 6c11 0000 1↵
         条件1              条件2
```

(3) シーケンシャルブレーク条件の解除

書式1) 現在の設定を表示させ、その後は次のように入力します。

```
>bsq↵
  (現在の設定内容)
1.set  2.clear ...? 2↵                  ..."2. clear"を選択
>
```

設定条件がすべてクリアされます。

書式2) 次のようにコマンドを入力します。

```
>bsq 2↵
```

■注　意

・ bsqコマンドはICEモード時のみ使用可能です。

・ シーケンシャルブレークは、各条件が設定した順序ですべて成立した場合にのみ発生します。一部の条件のみ成立してもブレークしません。

・ ブレーク条件は1から連続して設定することが必要です。条件1と3または条件2と3のみを設定することはできません。なお、3個所すべてを設定する必要はありません。

・ アドレスは、各機種のメモリ領域範囲内で指定してください。これを越えた指定はエラーとなります。

Error: Address range (0-0xFFFFFFF).　　...0xffffffffを越えるアドレスを指定した場合
Error: No map area.　　...未使用アドレスを指定した場合

・ 直接入力モードで、入力パラメータ数に誤りがある場合はエラーとなります。

Error: Number of parameter.

・ 入力された数値に誤りがある場合、またはシンボルとして認識できない場合はエラーとなります。

Error: Invalid value.

・ 内蔵RAM領域はバスオペレーションが検出できないため、シーケンシャルブレークは発生しません。

ba (break area) [ICD+MEM33]

■ 機　能

エリアブレークの設定と解除、表示を行います。
本コマンドでは次のブレーク条件が指定できます。
1. メモリエリア(先頭/終端アドレス)
2. ブレーク位置(エリア内またはエリア外でブレーク)
3. バスオペレーション(リード、ライト、リードまたはライトの3条件)

プログラムは、上記条件を満たすメモリアクセスの終了後にブレークします。

■ 書　式

(1) **ba** (ガイダンスモード)
(2) **ba <number> <mode> [<side> <address1> <address2> <operation>]** (直接入力モード)

<number>: エリアブレーク番号(1 or 2)
<mode>: 設定/解除の指定(1=set, 2=clear)
<side>: エリアの内側/外側の指定(1=inside, 2=outside)
<address1>: エリアの先頭アドレス(16進数、シンボルまたはソース行番号)
<address2>: エリアの終端アドレス(16進数、シンボルまたはソース行番号)
<operation>: バスオペレーション(1=read, 2=write, 3=read/write)
条件: 0≦address1≦address2≦0xffffff

■ 入力例

(1) **エリアブレーク条件の表示**

```
>ba↵
Area break1:
Side           : inside
Top address    : 00100200
Bottom address : 001002FF
Operation      : r/w
Area break2:
Side           : ********
Top address    : ********
Bottom address : ********
Operation      : ********
Area break number(1-2)  ? :↵     ...[Enter]キーのみの入力で終了
>
```

上記例は、エリアブレーク1が設定されている場合の表示です。エリアブレーク2の"********"は、設定されていないことを示しています。

(2) **エリアブレーク条件の設定**

書式1) (1)のように現在の設定を表示させ、その後は次のように入力します。

```
Area break number(1-2)  ? :2↵        ...エリアブレーク2を指定
1.set  2.clear       ...? :1↵        ..."1.set"を選択
1.inside  2.outside  ...? :2↵        ..."2.outside"を選択
Top address            ? :210000↵    ...エリアの先頭アドレス
Bottom address         ? :21ffff↵    ...エリアの終端アドレス
1.read 2.write 3.r/w ...? :1↵        ..."1.read"を選択
>
```

この例では、エリアブレーク2を設定しています。0x210000番地から0x21ffff番地の範囲外のアドレスを読み出すとブレークします。

ガイダンスの途中でコマンドを終了させるには、[Enter]キーのみを入力してください。
この場合、エリアブレーク条件は変更されません。

書式2) 次のようにコマンドを入力します。

```
>ba 2 1 2 210000 21ffff 1↵
```

書式1と同じ設定です。各項目はスペースで区切って入力してください。

(3) エリアブレークの解除

書式1) 現在の設定を表示させ、その後は次のように入力します。

```
>ba↵
   (現在の設定内容)
Area break number(1-2)  ? :2↵          ...エリアブレーク2を指定
1.set  2.clear       ...? :1↵          ..."2.clear"を選択
>
```

エリアブレーク2がクリアされます。

書式2) 次のようにコマンドを入力します

```
>ba 2 2↵
>
```

エリアブレーク2がクリアされます。

■ 注　意

- baコマンドはICD+MEM33モード時のみ使用可能です。
- S5U1C33001M1を用いた拡張ブレークは、S5U1C33000Hの外部ブレークを使用して発生させるため、ブレーク発生直後にターゲットが停止しません。
- エリアブレークの先頭アドレスと終端アドレスが同一のCEエリアでない場合、エラーとなります。
  Top address and bottom address should be same CE.
- 内部メモリ領域を指定するとエラーとなります。
  MEM33 is not supported at area0,area1 and area2(0-0x7FFFF).
- エリアブレークの先頭アドレスと終端アドレスは、パラメータファイルで指定したCEFUNCでアクセスできるエリアでなければなりません。それ以外の場合はエラーとなります。
  Invalid address or invalid CEFUNC of parameter file.
- パラメータファイルの指定とBCUの設定が異なると、ブレークが正しく発生しません。
  コールドリセット後は、BCUの設定がデフォルトに戻るので注意が必要です。
- S5U1C33001M1のアドレスとターゲットのメモリ等のアドレスが競合しないように注意してください。

bb (break bus) [ICD+MEM33]

■ 機　能

バスブレークの設定と解除、表示を行います。
本コマンドでは次のブレーク条件が、16ビットバス選択時は最大6種類、32ビットバス選択時は最大3種類指定できます。
1. バス幅(16ビット、32ビット)
2. ブレークモード(複数条件のOR、シーケンシャル)
3. ブレークカウンタ(ブレーク条件成立回数)
4. アドレスおよびアドレスビットのマスク
5. CEマスク
6. リード/ライトデータおよびデータビットのマスク
7. バスオペレーション(リード、ライト、リードまたはライトの3条件)

プログラムは、上記条件を満たすメモリアクセスの終了後にブレークします。

■ 書　式

bb　　(ガイダンスモード)

■ 入力例

(1) **バスブレーク条件の表示**

```
>bb↵
Bus width      : 16
Bus break mode : or
Break counter  : 0
Number of bus break : 1
Address1       : 00100000
Address mask1  :   FFFFFF
CE mask1       : no
Data1          : 0002
Data mask1     : FFFF
Operation1     : write
1.set  2.clear ...? set    :↵   ...[Enter]キーのみの入力で終了
>
```

上記例は、16ビット幅、ORモード、ブレークカウント0のバスブレーク条件1のみが設定されている場合の表示です。0x100000番地に0x2が書き込まれるとブレークします。

(2) **バスブレーク条件の設定**

(1)のように現在の設定を表示させ、その後は次のように入力します。

```
1.set  2.clear ...? set    :1↵                ..."1.set"を選択
Bus width 1.16bit 2.32bit ? 16bit :↵          ...[Enter]キーのみでスキップ(変更なし)
Bus break mode 1.or 2.sequential ? or :↵
Bus break counter(0-65535) 0 :↵
Number of bus break(1-6)   1 :2↵              ...シーケンス数を入力
Address1      00200000 :↵                     ...条件1は入力をスキップ
Address mask1   FFFFFF :↵
CE mask1 1.yes  2.no   ? no  :↵
Data1         0012 :↵
Data mask1    FFFF :↵
Operation1 1.read 2.write 3.r/w ? read  :↵
Address2      00000000 :200000↵               ...条件2のアドレスを入力
Address mask2   FFFFFF :↵
CE mask2 1.yes  2.no   ? no  :↵
Data2         0000 :13↵                       ...条件2のデータを入力
Data mask2    FFFF :↵
Operation2 1.read 2.write 3.r/w ? read :1↵    ..."1.read"を選択
>
```

この例では、条件1は変更せずに、条件2を新たに追加しています。
変更前は、0x200000番地から0x12を読み出すとブレークする設定でしたが、この変更により0x200000番地から0x12または0x13を読み出すとブレークする設定となります。

各設定項目の内容は以下のとおりです。

Bus width 1.16bit 2.32bit ? 16bit :

データのバス幅を設定します。16ビット幅では6本、32ビット幅では3本の条件を設定できます。

Bus break mode 1.or 2.sequential ? or:

ブレークモードを設定します。ORモードでは、設定した条件に1つでも合致するとブレークします。シーケンシャルモードでは、若い番号の条件からその順番どおりにアクセスが発生するとブレークします。順番が逆、あるいは1つの条件に合致しただけではブレークしません。

Bus break counter(0-65535) 0 :

指定回数だけブレーク条件に合致した場合にブレークさせることができます。0を指定すると1回の条件合致でブレークします。1を指定すると2回の条件合致でブレークします。シーケンシャルモードでは最後のブレーク条件の合致のみをカウントします。

Number of bus break(1-6) 1 :

設定するバスブレーク条件の数を入力します。

以下の項目は、条件番号ごとに設定します。

AddressX 00000000 :

アドレスを16進数で入力します。

Address maskX FFFFFF :

アドレスマスクを16進数で入力します。マスクする(条件比較しない)ビットは0で指定してください。

CE maskX 1.yes 2.no ? no :

CE信号をマスクできます。マスクする場合は"1.yes"を選択してください。

DataX 0000 :

データを16進数で入力します。

Data maskX FFFF :

データマスクを16進数入力します。マスクする(条件比較しない)ビットは0で指定してください。

OperationX 1.read 2.write 3.r/w ? read :

バスオペレーションをリード、ライト、リードまたはライトの3条件から選択します。

(3) バスブレークの解除

現在の設定を表示させ、その後は次のように入力します。

```
>bb↵
   (現在の設定内容)
1.set  2.clear ...? set    :2↵          ..."2.clear"を選択
>
```

バスブレーク条件がクリアされます。

■ 注　意

- bbコマンドはICD+MEM33モード時のみ使用可能です。
- S5U1C33001M1を用いた拡張ブレークは、S5U1C33000Hの外部ブレークを使用して発生させるため、ブレーク発生直後にターゲットが停止しません。
- 設定するアドレス(マスク後の対象アドレス)は、パラメータファイルで指定したCEFUNCでアクセスできる範囲内になければなりません。それ以外の場合はエラーとなります。

  Invalid address or invalid CEFUNC of parameter file.
- パラメータファイルの指定とBCUの設定が異なると、ブレークが正しく発生しません。
  コールドリセット後は、BCUの設定がデフォルトに戻るので注意が必要です。
- S5U1C33001M1のアドレスとターゲットのメモリ等のアドレスが競合しないように注意してください。
- 内部メモリ領域を指定するとエラーとなります。

  MEM33 is not supported at area0,area1 and area2(0-0x7FFFF).
- 32ビットでシーケンシャルモードを使用する場合、32ビットアクセスの間には1つ以上のメモリアクセスが必要です。連続アクセスの場合、2つ目の32ビット判定がマッチしません。

bl (break list) [ICD / ICE / SIM / MON]

■機　能

すべてのブレーク条件を表示します。

■書　式

bl　　(直接入力モード)

■表　示

各ブレーク設定コマンドで個別に表示される内容をすべて表示します。
以下の表示はICEモード時の例です。

```
>bl↵
Soft PC break:
 1 :00F00000/E qa_cmain.c#1      9 :********/
 2 :********/                   10 :********/
 3 :********/                   11 :********/
 4 :********/                   12 :********/
 5 :********/                   13 :********/
 6 :********/                   14 :********/
 7 :********/                   15 :********/
 8 :********/                   16 :********/
Hard PC break:
 1 :00F00000/E1 qa_cmain.c#100   5 :********/
 2 :00C00010/E2                  6 :********/
 3 :00C00480/D                   7 :********/
 4 :00C0048a/D                   8 :********/
Data break:
Address  : 00000200
R/W/*    : R
Sequential break:
Hit1
 Address      : 00000000
 Data pattern : 0000
 Data mask    : 0000
 Type         : Inst
Hit2
 Address      : 00C00000
 Data pattern : 2323
 Data mask    : 0000
 Type         : VecR
Hit3
 Address      : ********
 Data pattern : ****
 Data mask    : ****
 Type         : ****
>
```

■注　意

表示内容はデバッグモードにより異なります。

bac (break all clear) [ICD / ICE / SIM / MON]

■ 機　能

コマンドで設定したブレーク条件(ソフトウェアPCブレーク、ハードウェアPCブレーク、データブレーク、シーケンシャルブレーク)をすべて解除します。

■ 書　式

bac (直接入力モード)

16.10.8 ソース表示コマンド

u (unassemble) [ICD / ICE / SIM / MON]

■機　能

プログラムを逆アセンブル表示します。

■書　式

(1) **u** (直接入力モード)

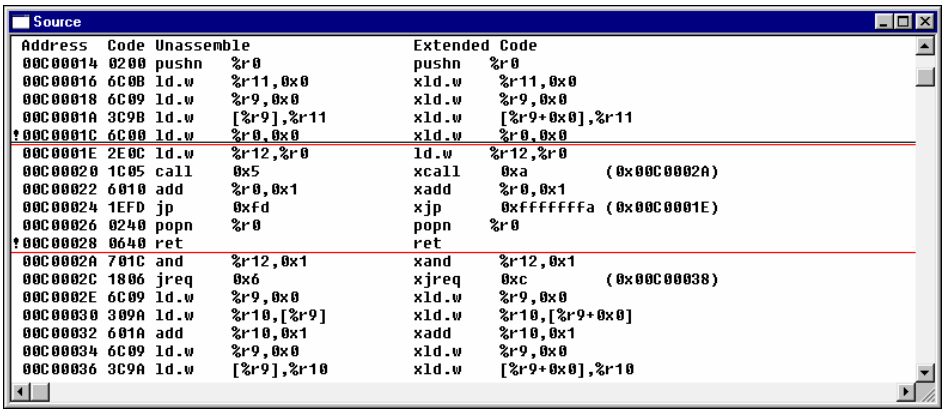
(2) **u <address>** (直接入力モード)

<address>: 表示開始アドレス(16進数、シンボルまたはソース行番号)

条件: 0≦address≦0xffffffff

■表　示

(1) [Source]**ウィンドウを開いている場合**



Address: メモリアドレス

Code: オブジェクトコード

Unassemble: 逆アセンブル内容

※現在のPCアドレスの行はアンダーラインで示されます。

書式1では[Source]ウィンドウを逆アセンブル表示モードに変更し、コードを現在のPC位置から表示します。

[Unassemble]ボタンも表示を逆アセンブル形式に変更します。ただし、表示位置は変更されません。

[Unassemble]**ボタン**

書式2では、[Source]ウィンドウを逆アセンブル表示モードに変更するとともに、<address>からコードの表示を行います。

(2) [Source]**ウィンドウを閉じている場合**

[Command]ウィンドウに16行分(デフォルト)の逆アセンブル結果を表示して、コマンド入力待ちとなります。

書式1では、現在のPC([Register]ウィンドウに表示)から、書式2では<address>から表示します。

```
>u C00014↵                    ...アドレスはシンボル、行番号でも指定可能です。
 00C00014 0200 pushn   %r0
 00C00016 6C0B ld.w    %r11,0x0
                 :
 00C00030 3C8A ld.w    [%r8],%r10
 00C00032 0640 ret
>
```

[Command]ウィンドウへの表示行数はmdコマンドで変更可能です。

(3) ログ出力

logコマンドによってコマンド実行結果をログファイルに出力する場合は、[Source]ウィンドウを閉じて[Command]ウィンドウにコードを表示させてください。
[Source]ウィンドウが開いている場合、[Command]ウィンドウにはコードが表示されないため、ログファイルには出力されません。

(4) 連続表示機能

uコマンドをキーボードより入力して実行すると、他のコマンドを実行するまでは[Enter]キーの入力のみでコードを連続して表示することができます。
[Enter]キーを入力すると、[Source]ウィンドウは1画面分スクロールします。
[Command]ウィンドウにコードを表示している場合は、前回表示したアドレスに続く16行分(デフォルト)を表示します。

■ 注　意

・ 表示開始アドレスは0～0xffffffffの領域内で指定してください。これを越えた場合、エラーとなります。
　　Error: Address range (0-0xFFFFFFF).

・ マップされていないメモリ領域は"no map"が表示されます。

sc (source code) [ICD / ICE / SIM / MON]

■機　能

ソースファイルの内容をアドレス、行番号と共に表示します。

■書　式

(1) **sc** (直接入力モード)

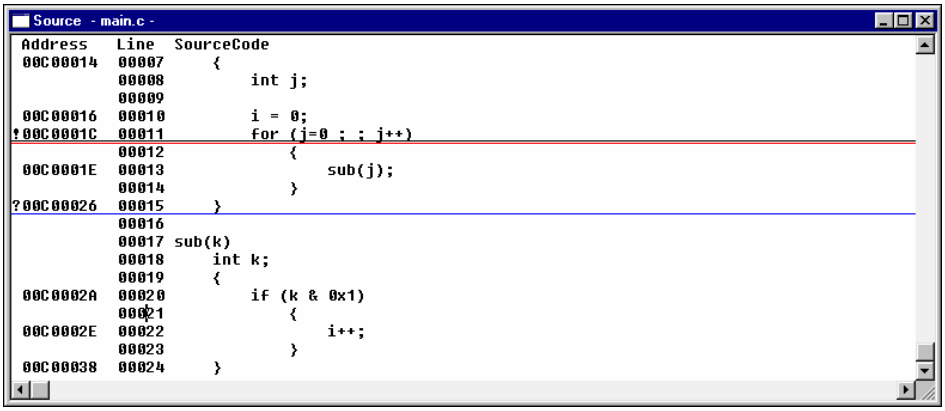
(2) **sc <address>** (直接入力モード)

<address>: 表示開始アドレス(16進数、シンボルまたはソース行番号)

条件: 0≦address≦0xffff

■表　示

(1) [Source]**ウィンドウを開いている場合**



Address: メモリアドレス

Line: ソースファイル内の行番号

SourceCode: ソースコード

※現在のPCアドレスの行はアンダーラインで示されます。

書式1では[Source]ウィンドウをソース表示モードに変更し、ソースを現在のPC位置から表示します。

[Source]ボタンも表示をソース形式に変更します。ただし、表示位置は変更されません。

[Source]**ボタン**

書式2では、[Source]ウィンドウをソース表示モードに変更するとともに、<address>からソースの表示を行います。

(2) [Source]ウィンドウを閉じている場合

[Command]ウィンドウに16行分(デフォルト)のソースを表示して、コマンド入力待ちとなります。書式1では、現在のPC([Register]ウィンドウに表示)から、書式2では<address>もしくは行番号から表示します。

```
>sc main.c#1↵                    ...アドレスは16進数、シンボルでも指定可能です。
           00001 /* tst_main.c 1997.2.13 */
           00002 /* C main program */
           00003
           00004 int i;
           00005
           00006 main()
 00C00014  00007     {
           00008      int j;
           00009
 00C00016  00010      i = 0;
 00C0001A  00011      for (j=0 ; ; j++)
           00012          {
 00C0001C  00013                sub(j);
           00014          }
 00C00024  00015     }
           00016
>
```

[Command]ウィンドウへの表示行数はmdコマンドで変更可能です。

(3) ログ出力

logコマンドによってコマンド実行結果をログファイルに出力する場合は、[Source]ウィンドウを閉じて[Command]ウィンドウにコードを表示させてください。
[Source]ウィンドウが開いている場合、[Command]ウィンドウにはコードが表示されないため、ログファイルには出力されません。

(4) 連続表示機能

scコマンドをキーボードより入力して実行すると、他のコマンドを実行するまでは[Enter]キーの入力のみでソースを連続して表示することができます。
[Enter]キーを入力すると、[Source]ウィンドウは1画面分スクロールします。
[Command]ウィンドウにコードを表示している場合は、前回表示したアドレスに続く16行分(デフォルト)を表示します。

■ 注　意

- ソースと行番号の表示は、ソース情報を持つsrf33形式のオブジェクトファイルを読み込んでいる場合にのみ可能です。
- ソース表示モードで表示可能なソースは一度に1つです。アドレスが連続している場合でも複数のソースを連続的に表示することはできません。
- 表示開始アドレスは0～0xffffffffの領域内で指定してください。これを越えた場合、エラーとなります。
  Error: Address range (0-0xFFFFFFF).
- マップされていないメモリアドレス、ソース情報のないアドレスを指定するとミックス表示モードに変更されます。ソース表示部分は空白となります。

m (mix) [ICD / ICE / SIM / MON]

■機　能

プログラムの逆アセンブル結果とソースファイルの内容を表示します。

■書　式

(1) **m** (直接入力モード)

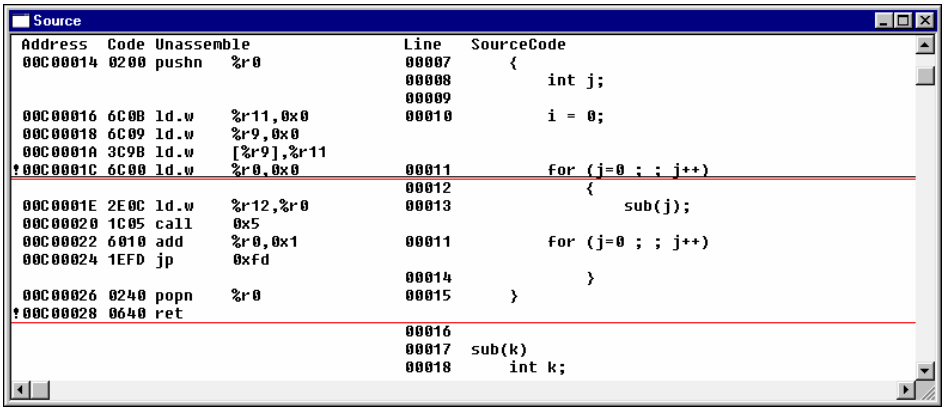
(2) **m <address>** (直接入力モード)

<address>: 表示開始アドレス(16進数、シンボルまたはソース行番号)

条件: 0≦address≦0xffffffff

■表　示

(1) [Source]**ウィンドウを開いている場合**



Address: プログラムメモリアドレス
Code: オブジェクトコード
Unassemble: 逆アセンブル内容
Line: ソースファイル内の行番号
SourceCode: ソースコード

※現在のPCアドレスの行はアンダーラインで示されます。

書式1では[Source]ウィンドウをミックス表示モードに変更し、逆アセンブル内容とソースを現在のPC位置から表示します。

[Mix]ボタンも表示をソース形式に変更します。ただし、表示位置は変更されません。

[Mix]**ボタン**

書式2では、[Source]ウィンドウをミックス表示モードに変更するとともに、<address>から逆アセンブル内容とソースの表示を行います。

ミックス表示モードでは複数のソースを連続的に表示可能です。ファイルが変わる部分にはソースファイル名が表示されます。

(2) [Source]ウィンドウを閉じている場合

[Command]ウィンドウに16行分(デフォルト)のミックス表示を行い、コマンド入力待ちとなります。
書式1では、現在のPC([Register]ウィンドウに表示)から、書式2では<address>から表示します。

```
>m main↵                          ...アドレスは16進数、行番号でも指定可能です。
 00C00014 0200 pushn   %r0                00007      {
                                          00008    int j;
                                          00009
 00C00016 6C0B ld.w    %r11,0x0           00010    i = 0;
 00C00018 3C8B ld.w    [%r8],%r11
 00C0001A 6C00 ld.w    %r0,0x0            00011    for (j=0 ; ; j++)
                                          00012        {
 00C0001C 2E0C ld.w    %r12,%r0           00013             sub(j);
 00C0001E 1C05 call    0x5
 00C00020 6010 add     %r0,0x1            00011    for (j=0 ; ; j++)
 00C00022 1EFD jp      0xfd
                                          00014        }
 00C00024 0240 popn    %r0                00015      }
 00C00026 0640 ret
                                          00016
                                          00017  sub(k)
>
```

[Command]ウィンドウへの表示行数はmdコマンドで変更可能です。

(3) ログ出力

logコマンドによってコマンド実行結果をログファイルに出力する場合は、[Source]ウィンドウを閉じて[Command]ウィンドウにコードを表示させてください。
[Source]ウィンドウが開いている場合、[Command]ウィンドウにはコードが表示されないため、ログファイルには出力されません。

(4) 連続表示機能

mコマンドをキーボードより入力して実行すると、他のコマンドを実行するまでは[Enter]キーの入力のみでソースを連続して表示することができます。
[Enter]キーを入力すると、[Source]ウィンドウは1画面分スクロールします。
[Command]ウィンドウにコードを表示している場合は、前回表示したアドレスに続く16行分(デフォルト)を表示します。

■ 注　意

- ソースと行番号の表示は、ソース情報を持つsrf33形式のオブジェクトファイルを読み込んでいる場合にのみ可能です。
- 表示開始アドレスは0～0xffffffffの領域内で指定してください。これを越えた場合、エラーとなります。
  Error: Address range (0-0xFFFFFFF).
- マップされていないメモリ領域は"no map"が表示されます。
- ソース行情報があるのにもかかわらず対応するソースファイルが読み込めない場合は、"no source"と表示されます。

ss (search strings) [ICD / ICE / SIM / MON]

■ 機　能

ソース内で指定の文字列を検索し、見つかった位置からソースを表示します。
[Source]ウィンドウをソース表示モードで開いている場合にのみ有効です。

■ 書　式

ss <string> (直接入力モード)
<string>: 検索文字列

■ 入力例

```
>ss main↵
>
```

カレントソース中(現在のPCアドレスに対応するコードを含むソース)で最初に見つかったmainの位置からソースを[Source]ウィンドウに表示します。この後、[Enter]キーの入力で次のmainを検索します。
指定の文字列が見つからなかった場合は、次のメッセージを表示します。
Error: Not found input strings.

■ 注　意

・ [Source]ウィンドウが閉じている場合にssコマンドを実行するとエラーとなります。
Error: Source window not opened.

・ [Source]ウィンドウをソース表示モード以外のモードに設定している場合もエラーとなります。
Error: Current mode is not source mode.

16.10.9 シンボル情報表示コマンド

sy (symbol list) [ICD / ICE / SIM / MON]

■機　能

シンボル情報、ソースファイル、関数、タグの一覧を[Command]ウィンドウに表示します。
シンボルは検索範囲や検索文字列を指定した条件付き検索も行えます。

■書　式

sy [<option>] (直接入力モード)

<option>: 表示項目、検索条件の指定

省略時: 全シンボル一覧
#: ファイル一覧
/: 関数一覧
@: タグ一覧
@<string>: 文字列<string>で始まるタグの一覧
[<file>]/[<function>>]/: 指定ファイル/関数内のシンボル一覧
[<file>]/[<function>]/<string>: 指定ファイル/関数内の文字列<string>で始まるシンボル一覧

<file>: ソースファイル名
<function>: 関数名
<string>: 検索文字列
条件: 1≦stringの文字数≦255

[<file>]/[function]/によって検索するシンボル

// グローバルシンボル
/./ カレント関数内のオート/スタティックシンボル
.// カレントソースファイル内のスタティックシンボル
file// 指定ソースファイル内のスタティックシンボル
/function/ 指定関数内のシンボル
./function/ カレントソースファイルの指定関数内のオート/スタティックシンボル
file/function/ 指定ソースファイルの指定関数内のシンボル

※ カレントソースファイル、カレント関数は現在のPCに対応するコードを含むソースファイル、関数を意味します。

■表　示

一覧は[Command]ウィンドウにアルファベット順に表示されます。表示行数はデフォルトで16行(mdコマンドで変更可)です。16行以上の表示項目がある場合は、16行表示したところでコマンド入力状態になりますので、[Enter]キーのみを入力して続く内容を表示させてください。

(1) シンボルの一覧

シンボル情報は次の形式で表示されます。
<symbol>, <address>, <scope>, <class>, <type>

<symbol>:　シンボル名を示します。

<address>:　シンボルのアドレスを示します。ストレージクラスにより表示内容が異なります。

ストレージクラス	表示内容
null、extern、static、label	8桁16進数のアドレス
auto、argument	SP+オフセット(0xXX)
register、reg parameter	レジスタ番号(R0～R15)
bit field	ビットフィールドのサイズ

<scope>:　ファイル名/関数名を示します。
externシンボルは空白となります。
staticシンボルはファイル名のみを表示します。
ブロック内で宣言した変数は、ブロックの開始アドレスと終了アドレスを表示します。(<start addr> ... <end addr>)

<class>:　ストレージクラスを示します。
null、auto、extern、static、register、label、argument、reg parameter

<type>:　シンボルの型を示します。
null、void、char、short、int、long、float、double、struct <tag name>、union <tag name>、enum <tag name>、unsigned char、unsigned short、unsigned int、unsigned long
シンボルがポインタ、配列、関数の場合は上記シンボルの後にそれぞれ"*"、"[ ]"(宣言内容を含む)、"( )"が表示されます。

全シンボルの表示

すべてのシンボルを表示させるには、次のように入力します。

```
>sy↵                                    ...オプションなしで実行
BOOT, 00C00004, boot.s/, static, null
LOOP, 00C00000,  , label, null
READ_BUF, 00800048,  , extern, unsigned char [65]
READ_EOF, 00800089, sys.c/, static, unsigned char
                    :
_init_sys, 00C00004,  , extern, void  ()
_iob, 0080008C,  , extern, struct __T2 [4]
>↵                                      ...[Enter]キーのみで続く一覧を表示
dfIn, R0, ansilib.c/main, register, double
dfOut, R10, ansilib.c/main, register, double
                    :
iSize, R12, sys.c/write, register, int
>
```

検索範囲を指定した一覧表示

オプションに示したとおり、ファイル名と関数名([<file>]/[<function>]/)が指定できます。

```
>sy //↵                         ...グローバルシンボルの一覧を指定
READ_BUF, 00800048,  , extern, unsigned char [65]
READ_FLASH, 00C0005C,  , extern, null
WRITE_BUF, 00800004,  , extern, unsigned char [65]
                    :
read, 00C00014,  , extern, int  ()
seed, 008000D0,  , extern, unsigned int
>
```

検索範囲と検索文字列を指定した一覧表示

指定範囲内から指定の文字列で始まるシンボルを検索し、表示します。

```
>sy sys.c//R↵                    ...sys.c内のRで始まるスタティックシンボルの一覧を指定
READ_EOF, 00800089, sys.c/, static, unsigned char
>
```

(2) **ソースファイル一覧**

ファイル一覧は次の形式で表示されます。

<file>(<line>), <start addr> . . <end addr> [, <start addr> . . <end addr>]

<file>: ソースファイル名を示します。
<line>: ソースファイル内の行数を示します。
ソースファイルが読み込まれていない場合は"not read"を表示します。
<start addr>: コードが配置された先頭アドレスを16進数で示します。
<end addr>: コードが配置された終了アドレスを16進数で示します。
コードが複数に分割された状態で配置されている場合は、<start addr>と<end addr>も複数表示されます。
また、インクルードファイルにコードがある場合は、<end addr>の後に(<インクルードファイル名>)が表示されます。

ソースファイルの一覧を表示させるには、次のように入力します。

```
>sy #↵
boot.s(29 lines) C00004..C00023
sys.c(180 lines) C00000..C000AF
lib.c(83 lines) C000B0..C00133
ansilib.c(not read) C00342..C00509
..\src\strlen.s(not read) C02B94..C02B9F
..\src\memcpy.s(not read) C02BA0..C02BAF
>
```

(3) **関数一覧**

関数一覧は次の形式で表示されます。

<file>
<function>( ) <start addr> . . <end addr>

<file>: ソースファイル名を示します。
各ソースファイルごとに1個ずつ表示されます。
関数を持たないファイル(アセンブリソース)も表示されます。
<function>: 関数名を示します。
関数はソースファイルごとに表示されます。
<start addr>: 関数が配置された先頭アドレスを16進数で示します。
<end addr>: 関数が配置された終了アドレスを16進数で示します。

関数の一覧を表示させるには、次のように入力します。

```
>sy /↵
boot.s
sys.c
    _exit() C00000..C00003
    _init_sys() C00004..C00013
    read() C00014..C00071
    write() C00072..C000AF
lib.c
    _init_lib() C000B0..C00133
ansilib.c
    main() C00342..C00509
>
```

(4) タグ一覧

タグ一覧は次の形式で表示されます。

```
<file>
    offset   <type> <tag> {
    <offset> <m type> <member>
                 :
                 } (total <size>)
```

<file>: ソースファイル名を示します。
各ソースファイルごとに1個ずつ表示されます。タグが宣言されていないファイルは表示されません。
<type>: タグの型を示します。
<tag>: タグ名を示します。
<offset>: メンバーのオフセットを16進数で示します。
<m type>: メンバーの型を示します。
<member>: メンバー名を示します。
<size>: サイズ(バイト数)を10進数で示します。

全タグの表示

すべてのタグを表示させるには、次のように入力します。

```
>sy @↵
lib.c
 offset  struct tm {
00000000     int  tm_sec
00000004     int  tm_min
           :
00000020     int  tm_isdst
         } (total 36 byte)
 offset  struct __T2 {
00000000     short  _flg
00000002     unsigned char  _buf
00000004     int  _fd
>↵                                  ...[Enter]キーのみで続く一覧を表示
         } (total 8 byte)
 offset  union __T1 {
00000000     struct  __T0 st
00000000     double  _D
         } (total 8 byte)
>
```

検索文字列を指定した一覧表示

指定の文字列で始まタグを検索し、表示します。

```
>sy @t↵                             ...tで始まるタグの一覧を指定
lib.c
 offset  struct tm {
00000000     int  tm_sec
00000004     int  tm_min
00000008     int  tm_hour
0000000C     int  tm_mday
00000010     int  tm_mon
00000014     int  tm_year
00000018     int  tm_wday
0000001C     int  tm_yday
00000020     int  tm_isdst
         } (total 36 byte)
ansilib.c
 offset  struct tm {
00000000     int  tm_sec
00000004     int  tm_min
>
```

ビットフィールドは以下のように表示されます。
offsetは先頭アドレスからのビットオフセットです。

```
 offset  struct bitSt {
00000000     unsigned int  b1   : 1
00000001     unsigned int  b11  : 11
0000000C     unsigned int  b3   : 3
0000000F     unsigned int  b17  : 17
00000020     unsigned int  b21  : 21
         } (total 8 byte)
```

enum型は以下のように表示されます。
valueは各メンバーの値です。

```
 value   enum iEnum {
00000000     enum member  iE1
00000001     enum member  iE2
00000002     enum member  iE3
0000000A     enum member  iE10
         }
```

■注　意

・ シンボル情報等の表示は、デバッグ情報を含むsrf33形式のオブジェクトファイルを読み込んでいる場合にのみ可能です。

・ 検索文字列<string>は大文字、小文字が区別されます。文字数は最大255文字まで指定可能です。

sa (symbol add) [ICD / ICE / SIM / MON]

■ 機　能

指定したシンボル(変数)をシンボル監視用のテーブルに登録します。
また、構造体や配列の一括登録ができます。
最大99個までのシンボルが登録可能で、登録されたシンボルの内容を[Symbol]ウィンドウで監視することができます。
併せて、表示形式を指定することもできます。

■ 書　式

(1) **sa** 表示(直接入力モード)
(2) **sa <symbol> [-<switch>]** 登録(直接入力モード)
(3) **sa @<symbol> [-<switch>]** 一括登録(直接入力モード)

<symbol>: シンボル名
-<switch>: 表示形式の指定
-b<size> 2進数
-d<size> 符号付き10進数
-u<size> 符号なし10進数
-h<size> 16進数
-c 8ビットの整数値
-f 32ビットの実数値
-df 64ビットの実数値
<size>は表示するビット数で8、16、32、64(例: -b8、-h32)を指定します。
省略した場合は、シンボルの型に従ったサイズで表示されます。
-dと-uには64ビットを指定することはできません。

デフォルト: -<switch>の指定を省略した場合
・型が不明なシンボル: 32ビットの16進数で表示
・int、short、long: 10進数と16進数の両方で表示
・char: アスキーコードと10進数および16進数で表示

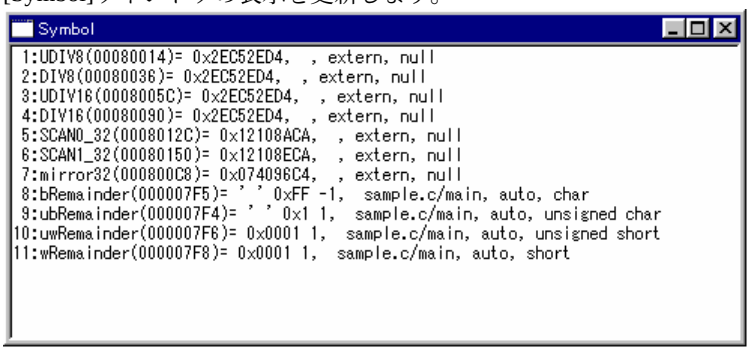
■ 入力例

(1) **表示**

saのみの入力(書式1)で、登録されているシンボルの内容を表示します。

[Symbol]ウィンドウが開いている場合

[Symbol]ウィンドウの表示を更新します。



[Symbol]ウィンドウが閉じている場合

[Command]ウィンドウに表示します。16行分(デフォルト)表示されますので、それ以降を表示させるには[Enter]キーを入力してください。表示行数はmdコマンドで変更可能です。

```
>sa↵
 1:UDIV8(00C00014)= 0x2EC52ED4,  , extern, null
 2:DIV8(00C00036)= 0x2EC52ED4,  , extern, null
 3:UDIV16(00C0005C)= 0x2EC52ED4,  , extern, null
 4:DIV16(00C00090)= 0x2EC52ED4,  , extern, null
 5:SCAN0_32(00C0012C)= 0x12108ACA,  , extern, null
 6:SCAN1_32(00C00150)= 0x12108ECA,  , extern, null
 7:mirror32(00C000C8)= 0x074096C4,  , extern, null
 8:bRemainder(000007F5)= ' ' 0xFF -1,  sample.c/main, auto, char
 9:ubRemainder(000007F4)= ' ' 0x1 1,  sample.c/main, auto, unsigned char
10:uwRemainder(000007F6)= 0x0001 1,  sample.c/main, auto, unsigned short
11:wRemainder(000007F8)= 0x0001 1,  sample.c/main, auto, short
>
```

表示内容

各シンボルの内容は次の形式で表示されます。

<No.> <symbol>(<address>) = <value>, <scope>, <class>, <type>

<No.>: 監視シンボルテーブルへの登録番号を示します。

<symbol>: シンボル名を示します。

<address>: シンボルのアドレスを8桁の16進数で示します。レジスタ変数の場合は、R0～R15で表示します。

<value>: 格納されている値を登録時の-<switch>の指定に従って表示します。
16進数の先頭には"0x"、2進数の先頭には"0b"が付きます。負の10進数には"-"が付きますが正の10進数には何も付きません。
スコープ外のものは"out of scope"を表示します。

<scope>: ファイル名/関数名を示します。
externシンボルは空白となります。
staticシンボルは関数名のみを表示します。

<class>: ストレージクラスを示します。
null、auto、extern、static、register、label、argument、reg parameter

<type>: シンボルの型を示します。
null、void、char、short、int、long、float、double、struct <tag name>、union <tag name>、enum <tag name>、unsigned char、unsigned short、unsigned int、unsigned long
シンボルがポインタ、配列、関数の場合は上記シンボルの後にそれぞれ"*"、"[ ]"(宣言内容を含む)、"( )"が表示されます。

(2) シンボルの登録

-<switch>を指定した、シンボルの登録例を示します。

```
>sa chChar -b8↵          ...chCharを8ビット、2進数表示として登録
>sa stA.ulCount -u32↵    ...stAのメンバーulCountを32ビット、符号なし10進数表示として登録
>sa iLoop -h↵            ...iLoopを型に従ったサイズの16進数表示として登録
>sa dDecimal -df↵        ...dDecimalをdouble型の表示として登録
>sa *plCount -u32↵       ...ポインタplCountを32ビット、符号付き10進数表示として登録
```

コマンドを実行すると、[Source]ウィンドウに登録したシンボルの情報が表示されます。[Symbol]ウィンドウが閉じている場合は[Command]ウィンドウに表示されます。登録されたシンボルには、登録順に番号(1～99)が付けられます。

※ シンボルの登録は[Symbol]メニュー/[Add]コマンド、または[Symbol add]ボタンでも行えます。

[Symbol add]**ボタン**

[Source]ウィンドウをミックス表示モードまたはソース表示モードにし、登録するシンボル名の中あるいは直前・直後にカーソルを置きます。そこで、[Symbol add]ボタンをクリックまたは[Symbol]メニュー/[Add]コマンドを選択すると、シンボルを監視シンボルテーブルに登録し、その情報を[Symbol]ウィンドウまたは[Command]ウィンドウに表示します。

これらのメニューコマンド、ボタンは[Source]ウィンドウがアクティブな場合にのみ選択可能です。また、表示形式は設定できません。シンボルの型に従ったデフォルトの表示形式となります。

(3) 構造体や配列の一括登録

構造体や配列は以下の書式で一括登録ができます。

```
>sa @stNagano.stMatumoto.stNorthSchool.stStudent.cName
1:stNagano.stMatumoto.stNorthSchool.stStudent.cName[0]
            (00001F08)= 's' 0x73 115,  main.c/FuzzySearchTest, struct, char [10]
2:stNagano.stMatumoto.stNorthSchool.stStudent.cName[1]
            (00001F09)= 'a' 0x61 97,  main.c/FuzzySearchTest, struct, char [10]
3:stNagano.stMatumoto.stNorthSchool.stStudent.cName[2]
            (00001F0A)= 's' 0x73 115,  main.c/FuzzySearchTest, struct, char [10]
4:stNagano.stMatumoto.stNorthSchool.stStudent.cName[3]
            (00001F0B)= 'a' 0x61 97,  main.c/FuzzySearchTest, struct, char [10]
5:stNagano.stMatumoto.stNorthSchool.stStudent.cName[4]
            (00001F0C)= 'k' 0x6B 107,  main.c/FuzzySearchTest, struct, char [10]
6:stNagano.stMatumoto.stNorthSchool.stStudent.cName[5]
            (00001F0D)= 'i' 0x69 105,  main.c/FuzzySearchTest, struct, char [10]
7:stNagano.stMatumoto.stNorthSchool.stStudent.cName[6]
            (00001F0E)= ' ' 0x00 0,  main.c/FuzzySearchTest, struct, char [10]
8:stNagano.stMatumoto.stNorthSchool.stStudent.cName[7]
            (00001F0F)= ' ' 0x00 0,  main.c/FuzzySearchTest, struct, char [10]
9:stNagano.stMatumoto.stNorthSchool.stStudent.cName[8]
            (00001F10)= ' ' 0x00 0,  main.c/FuzzySearchTest, struct, char [10]
10:stNagano.stMatumoto.stNorthSchool.stStudent.cName[9]
            (00001F11)= ' ' 0x00 0,  main.c/FuzzySearchTest, struct, char [10]
```

一括登録の機能は、メニューコマンドやボタンからは実行できません。

■ 注　意

- シンボルの登録および表示は、デバッグ情報を含むsrf33形式のオブジェクトファイルを読み込んでいる場合にのみ可能です。
- 構造体や配列の一括登録は、[Symbol]メニュー/[Add]コマンドや[Symbol add]ボタンからは実行できません。このため、"->"や"."の前にカーソルを置いてシンボルを選択することはできません。
- 指定したシンボルが見つからない場合エラーとなります。

  Error: Symbol not found.　　...シンボルが見つからない
- 指定したシンボルがスコープ範囲外、あるいはマップされていない領域を指している場合、以下のように表示されます。

  <symbol> = out of the scope, ...　　...スコープ範囲外

  <symbol> = symbol points to no map area, ...　　...マップされていない領域を指している場合
- 登録可能なシンボル数は最大99個です。これを越えるとエラーとなります。

  Error: Cannot add symbol any more.

sd (symbol delete) [ICD / ICE / SIM / MON]

■ 機　能

[Symbol]ウィンドウに表示(監視シンボルテーブルに登録)されているシンボルをウィンドウ(テーブル)から削除します。

■ 書　式

sd <number> (直接入力モード)

<number>: シンボルの登録番号(10進数)

条件: 1≦number≦シンボルの登録数(Max. 99)

■ 入力例

>sd 2.↵ ...シンボルNo.2を削除します。

>

シンボルを削除すると続くシンボルの番号が1つずつ繰り上がります。

※ シンボルの削除は[Symbol]メニュー/[Delete]コマンド、または[Symbol delete]ボタンでも行えます。

[Symbol delete]**ボタン**

[Symbol]ウィンドウの削除するシンボル情報の行にカーソルを置き、[Symbol delete]ボタンをクリックまたは[Symbol]メニュー/[Delete]コマンドを選択すると、そのシンボルが削除されます。

これらのメニューコマンド、ボタンは[Symbol]ウィンドウがアクティブな場合にのみ選択可能です。

■ 注　意

99を越える番号や登録されていない番号を指定するとエラーとなります。

Error: No symbol at the number.

sw (symbol watch) [ICD / ICE / SIM / MON]

■機　能

指定したシンボルの現在の値を[Command]ウィンドウに表示します。個々のシンボルあるいは指定範囲に含まれるシンボルを指定することができます。表示形式の指定も可能です。また、構造体や共用体のメンバー、配列の内容を表示させることもできます。

■書　式

(1) **sw <symbol> [-<switch>]** (直接入力モード)
(2) **sw @<symbol> [-<switch>]** (直接入力モード)
(3) **sw <scope>** (直接入力モード)

<symbol>: シンボル名

-<switch>: 表示形式の指定

-b<size> 2進数
-d<size> 符号付き10進数
-u<size> 符号なし10進数
-h<size> 16進数
-c 8ビットの整数値
-f 32ビットの実数値
-df 64ビットの実数値

<size>は表示するビット数で8、16、32、64(例: -b8、-h32)を指定します。
省略した場合は、シンボルの型に従ったサイズで表示されます。
-dと-uには64ビットを指定することはできません。

デフォルト: -<switch>の指定を省略した場合

・型が不明なシンボル: 32ビットの16進数で表示
・int、short、long: 10進数と16進数の両方で表示
・char: アスキーコードを10進数と16進数で表示

<scope>: シンボル一覧表示の範囲指定([<file>]/[<function>>]/)

// グローバルシンボル
/./ カレント関数内のオート/スタティックシンボル
.// カレントソース内のスタティックシンボル
file// 指定ソース内のスタティックシンボル
/function/ 指定関数内のシンボル
./function/ カレントソースの指定関数内のオート/スタティックシンボル
file/function/ 指定ソースファイルの指定関数内のシンボル

■表　示

(1) **個々のシンボルの表示**

書式1ではシンボルを指定して情報を表示させます。

```
>sw READ_BUF[0]↵                ...デフォルトの表示形式を指定
READ_BUF[0](00800048)= ' ' 0xAA 170,  , extern, unsigned char [65]
>

>sw WRITE_BUF -u8↵              ...8ビット、符号なし10進数による表示を指定
WRITE_BUF(00800004)= 170,  , extern, unsigned char [65]
>
```

表示内容についてはsaコマンドの説明を参照してください。

※ 書式1と同様の表示は、[Symbol]メニュー/[Watch]コマンド、または[Symbol watch]ボタンでも行えます。

[Symbol watch]**ボタン**

[Source]ウィンドウをミックス表示モードまたはソース表示モードにし、表示させるシンボル名の中あるいは直前・直後にカーソルを置きます。そこで、[Symbol watch]ボタンをクリックまたは[Symbol]メニュー/[Watch]コマンドを選択すると、そのシンボルの情報を[Command]ウィンドウに表示します。
これらのメニューコマンド、ボタンは[Source]ウィンドウがアクティブな場合にのみ選択可能です。また、表示形式は設定できません。シンボルの型に従ったデフォルトの表示形式となります。

(2) 構造体/共用体のメンバー、配列の表示

書式2では構造体/共用体のメンバー、配列を指定し、その内容を表示させることができます。
表示内容についてはsaコマンドの説明を参照してください。
16行分(デフォルト)表示されますので、それ以降を表示させるには[Enter]キーを入力してください。表示行数はmdコマンドで変更可能です。

配列の表示

例: char buf[2][2][2]

```
>sw @buf↵
buf[0][0][0](000006B0)= '#' 0x23 35, qa_cmain.c/main, auto, char [2][2][2]
buf[0][0][1](000006B1)= ' ' 0xF9 -7, qa_cmain.c/main, auto, char [2][2][2]
                                  :
buf[1][1][1](000006B7)= ' ' 0xE0 -32, qa_cmain.c/main, auto, char [2][2][2]
>

>sw @buf[0][1][0]↵
buf[0][1][0](000006B2)= ' ' 0x8F -113, qa_cmain.c/main, auto, char [2][2][2]
buf[0][1][1](000006B3)= ' ' 0x1C 28, qa_cmain.c/main, auto, char [2][2][2]
                                  :
buf[1][1][1](000006B7)= ' ' 0xE0 -32, qa_cmain.c/main, auto, char [2][2][2]
>
```

構造体メンバーの表示

例: struct STRUCT_UNION{

```
                int   iAddr
                char  cFlag;
                char  *pcFlag;
                char  cAry[3][3]
                union ExchangeType stSize;
    };
```

```
>sw @stStructTest1.cAry↵
stStructTest1.cAry[0][0](00000414)= ' ' 0x10 16, qa_cmain.c/main, struct, char [3][3]
stStructTest1.cAry[0][1](00000415)= ' ' 0x04 4, qa_cmain.c/main, struct, char [3][3]
                                  :
stStructTest1.cAry[2][2](0000041C)= ' ' 0x0C 12, qa_cmain.c/main, struct, char [3][3]
>

>sw @stStructTest1.stSize↵
stStructTest1.stSize(00000420)= 0xF4D18517, qa_cmain.c/main, struct, union ExchangeType
stStructTest1.stSize.ucChar(00000420)= ' ' 0x17 23, qa_cmain.c/main, union unsigned char [4]
stStructTest1.stSize.usShort(00000420)= 0x8517 34071, qa_cmain.c/main, union unsigned short [2]
stStructTest1.stSize.ulLong(00000420)= 0xF4D18517 4107371799, qa_cmain.c/main, union unsigned int [1]
>
```

構造体の場合、指定の1階層下のメンバーをすべて表示します。それより下の階層は表示しません。

(3) 指定範囲のシンボル一覧表示

書式3ではファイルおよび関数の範囲を指定し、その範囲に含まれるシンボルの一覧を表示させることができます。

```
>sw //↵                                    ...グローバルシンボルの一覧を指定
READ_BUF(00800048)= ' ' 0xAA 170,  , extern, unsigned char [65]
READ_FLASH(00C0005C)= 0x680A24FA,  , extern, null
WRITE_BUF(00800004)= ' ' 0xAA 170,  , extern, unsigned char [65]
                                  :
read(00C00014)= 0xC0106C0C -1072665588,  , extern, int  ()
seed(008000D0)= 0xAAAAAAAA 2863311530,  , extern, unsigned int
>
```

表示内容についてはsaコマンドの説明を参照してください。
16行分(デフォルト)表示されますので、それ以降を表示させるには[Enter]キーを入力してください。表示行数はmdコマンドで変更可能です。

■注　意

- シンボル情報の表示は、デバッグ情報を含むsrf33形式のオブジェクトファイルを読み込んでいる場合にのみ可能です。
- 指定したシンボルが見つからない場合エラーとなります。

  Error: Symbol not found.　　...シンボルが見つからない
- 指定したシンボルがスコープ範囲外、あるいはマップされていない領域を指している場合、以下のように表示されます。

  <symbol> = out of the scope, ...　　...スコープ範囲外

  <symbol> = symbol points to no map area, ...　　...マップされていない領域を指している場合
- "->"と"."は区別されません。
- ポインタ(*)のネストは3まで、配列要素は4次元まで指定可能です。
- 構造体、共用体、ビットフィールドは10階層まで指定可能です。ただし、sw @コマンドの場合は、メンバーを追加するため9階層までとなります。
- 配列要素を指定する[ ]内の数値は10進数のみ使用可能です。
- スコープを含むシンボルの文字列長は127文字までに制限されています。

16.10.10 ファイル読み込みコマンド

lf (load file) [ICD / ICE / SIM / MON]

■ 機　能

srf33形式のオブジェクトファイルを読み込みます。

■ 書　式

(1) **lf** (ガイダンスモード)
(2) **lf <file name>** (直接入力モード)
<file name>: 読み込みファイル名

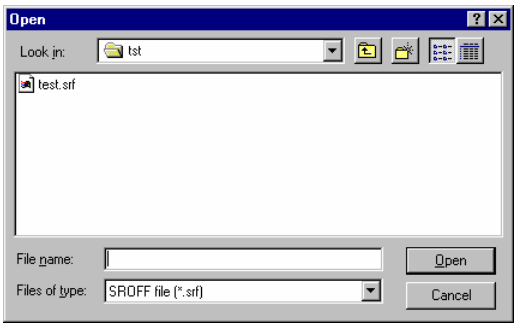
■ 入力例

書式1)
```
>lf↵
File name ? :test.srf↵
>
```
...ファイル名を拡張子まで入力します。

書式2)
```
>lf test.srf↵
>
```

※ [File]メニュー/[Load File..]コマンド、またはツールバーの[Load file]ボタンによってファイルを読み込むことも可能です。ファイルの選択は表示されるダイアログボックスによって行います。

[Load file]**ボタン**



ディレクトリとファイルを選択し、[Open]ボタンをクリック(またはファイル名をダブルクリック)してください。

ここで選択したディレクトリがカレントディレクトリとなります。

■ 注　意

- lfコマンドでは実行形式のsrf33オブジェクトファイル(リンカにより生成)以外は読み込めません。
  Error: cannot load data, please check SRF33 file
- デバッグにソースの表示やシンボルを使用する場合、デバッグ情報を含んだsrf33形式のオブジェクトファイルを読み込む必要があります。読み込んだファイルにデバッグ情報がない場合、ワーニングとなります。実データは有効です。
  Warning: No debug information, <file name>.
- srf33形式のオブジェクトファイルは、ディレクトリ構造も含めたソースファイル情報を持っています。このため、ソースファイルがカレントディレクトリから見てオブジェクトファイル内の指定ディレクトリ上にないと、ソースファイルが読み込めません。
  なお、メニュー選択、ボタン操作によるファイルの読み込み時は、ダイアログボックスによって選択したディレクトリがカレントディレクトリとなります。
  lfコマンドではカレントディレクトリは変更されません。
  基本的にはCコンパイラgcc33/プリプロセッサpp33～デバッガdb33までの操作を、同一のディレクトリ(カレントディレクトリにして)で行ってください。
- 1つのソースファイルとしては最大32767行まで読み込み可能です。
- ファイルの読み込み時に[Source]ウィンドウが開いていれば、その内容を更新します。プログラムの表示は現在設定されている形式で、現在のPCから行われます。PCはファイルの読み込みによっては変更されません。

- ファイルを読み込むと、現在のデバッグ情報および[Symbol]ウィンドウのシンボル登録情報は無効となります。ただし、ブレーク情報は残りますので、ブレーク情報をすべてクリア(bacコマンド)してから読み込んでください。また、[Memory]ウィンドウが開いている場合、内容をすべてクリアします。ダンプコマンド(db、dh、dw)を実行、または垂直スクロールバーをクリックして[Memory]ウィンドウを再表示させてください。
- ファイル読み込み中にエラーが発生した場合、すでに読み込まれた部分はエミュレーションメモリに残ります。ただし、ソース表示やシンボルを使用したデバッグは行えなくなります。また、どこまで読み込まれるかも不定です。この場合、[Source]ウィンドウは強制的に逆アセンブル表示モードになります。

lh (load hex) [ICD / ICE / SIM / MON]

■ 機　能

hex33より出力されたモトローラS3形式のファイルを読み込みます。

■ 書　式

(1) **lh** (ガイダンスモード)

(2) **lh <file name> <offset>** (直接入力モード)

<file name>: 読み込みファイル名(パスも指定可能)

<offset>: オフセットアドレス

■ 入力例

書式1)

```
>lh↵
File name ? :test.sa↵        ...ファイル名を拡張子まで入力します。
offset      : 80000↵         ...オフセットアドレスを指定します。
>
```

絶対アドレス付きで生成されたモトローラS3形式の場合はオフセットを0と入力してください。

書式2)

```
>lh test.sa 80000↵
>
```

■ 注　意

- モトローラS3形式のファイルでは、ソース表示やシンボルを使用したデバッグは行えません。
- ファイルの読み込み時に[Source]ウィンドウが開いていれば、その内容を更新します。プログラムの表示は逆アセンブル形式で、現在のPCから行われます。PCはファイルの読み込みによっては変更されません。
- ファイル読み込み中にエラーが発生した場合、すでに読み込まれた部分はエミュレーションメモリに残ります。
- マップされていないメモリ領域にロードされるようなオフセットアドレスを指定すると、エラーとなります。

  Error: No map area.

ld (load debug) [ICD / ICE / SIM / MON]

■ 機　能

srf33形式のオブジェクトファイルからデバッグ情報のみを読み込みます。

■ 書　式

(1) **ld** (ガイダンスモード)

(2) **ld <file name>** (直接入力モード)

<file name>: 読み込みファイル名(パスも指定可能)

■ 入力例

書式1)
```
>ld↵
File name ? :test.sa↵
>
```
...ファイル名を拡張子まで入力します。

書式2)
```
>ld test.sa↵
>
```

■ 注　意

- ldコマンドはデバッグ情報以外は変更しません。ROMのデバッグ等、あらかじめプログラムが書き込まれている場合にのみ使用してください。
- ldコマンドでは実行形式のsrf33オブジェクトファイル(リンカにより生成)以外は読み込めません。
  Error: cannot load data, please check SRF33 file.
- 読み込んだファイルにデバッグ情報がない場合、ワーニングとなります。
  Warning: No debug information, <file name>.
- srf33形式のオブジェクトファイルは、ディレクトリ構造も含めたソースファイル情報を持っています。このため、ソースファイルがカレントディレクトリから見てオブジェクトファイル内の指定ディレクトリ上にないと、ソースファイルが読み込めません。
  基本的にはCコンパイラgcc33/プリプロセッサpp33～デバッガdb33までの操作を、同一のディレクトリ(カレントディレクトリにして)で行ってください。

16.10.11 フラッシュメモリ操作コマンド

fls (flash memory set) [ICD / (ICE) / MON]

■ 機　能

ターゲットシステムのフラッシュメモリにデータを書き込むための設定を行います。

■ 書　式

fls (ガイダンスモード)

■ 入力例

```
>fls↵
 Flash start address   : ********                    ...設定されていない場合
 Flash end address     : ********
 Erase routine address : ********
 Write routine address : ********
1.set  2.clear ...? clear  :1↵                       ..."1. set"を選択
Flash start address   ? ******** : 200000↵           ...フラッシュ先頭アドレス
Flash end address     ? ******** : 2fffff↵           ...フラッシュ終端アドレス
Erase routine address ? ******** : FLASH_ERASE↵     ...消去プログラム先頭アドレス
Write routine address ? ******** : FLASH_LOAD↵      ...書き込みプログラム先頭アドレス
>
```

※[q]キーの入力でコマンドの実行を中止できます。

■ 注　意

- flsコマンドはターゲットシステム上のフラッシュメモリに対応したコマンドで、S5U1C33104Hに搭載のフラッシュメモリには適用されません。
  また、シミュレータモードでは使用できません。
- ターゲットシステムのフラッシュメモリを消去、あるいはフラッシュメモリにデータを書き込むためには、本コマンドを実行するとともに、データ書き込みおよび消去プログラムも指定のアドレスに書き込んでおく必要があります。フラッシュ書き込みプログラムについては、"utility¥fls33¥"の内容を参照してください。
- ICEモードで使用する場合、S5U1C33104Hのバージョンが2.0以上である必要があります。

fle (flash memory erase) [ICD / (ICE) / MON]

■機　能

ターゲットシステムのフラッシュメモリの内容を消去します。

■書　式

fle　　(ガイダンスモード)

■入力例

```
>fle↵
 Control Register : 00200000              ...flsで設定した先頭アドレス
 Start block      : ********              ...設定されていない場合
 End block        : ********
Control Register ? 00200000 : 200000↵     ...フラッシュ先頭アドレス
Start block      ? ******** : 0↵          ...消去するブロック範囲を指定
End block        ? ******** : 0↵            Start = End = 0で全領域を消去
Finish with 0x00000000          ...消去プログラムからの戻り値を表示
>
```

※[q]キーの入力でコマンドの実行を中止できます。

■注　意

- fleコマンドはターゲットシステム上のフラッシュメモリに対応したコマンドで、S5U1C33104Hに搭載のフラッシュメモリには適用されません。
  また、シミュレータモードでは使用できません。
- ターゲットシステムのフラッシュメモリを消去するためには、データ書き込みおよび消去プログラムをターゲットシステムメモリに書き込みflsコマンドを実行しておく必要があります。
  消去プログラムが設定されていない状態でfleコマンドを実行するとエラーとなります。
  Error: Erase routine is not set.
- ターゲットシステムのフラッシュメモリにデータを書き込む場合は、必ず先に本コマンドで消去を行ってください。
- ICEモードで使用する場合、S5U1C33104Hのバージョンが2.0以上である必要があります。

lfl (load from flash memory) [ICE]

■ 機　能

S5U1C33104H上のフラッシュメモリの内容をターゲットメモリに読み込みます。
前回フラッシュメモリに保存した内容に引き続いてデバッグを行うことができます。

■ 書　式

lfl　　(ガイダンスモード)

■ 入力例

```
>lfl↵」
Are you sure to load      1.yes 2.no ...? 1↵」        ...読み込みの確認
Loading from flash memory .............  done
>
```

読み込みを中止するには2を選択してください。

■ 注　意

- lflコマンドはS5U1C33104H上のフラッシュメモリに対応したコマンドで、ターゲットシステムのフラッシュメモリには適用されません。したがって、ICEモード以外では使用できません。
- フラッシュメモリにプロテクトがかかっている場合、または消去状態にあるときは読み込みは行えません。

  Error: Flash ROM is protected.　　...プロテクトされている場合
- フラッシュメモリとターゲットメモリのマップ状態が異なる場合でもロードされ、マップは書き換えられます。
- データ読み込み中にエラーが発生した場合、すでに読み込まれた部分はそのまま残ります。

sfl (save to flash memory) [ICE]

■ 機　能

S5U1C33104H上のターゲットメモリの内容をS5U1C33104Hフラッシュメモリに書き込みます。
フラッシュメモリへの書き込みにより、S5U1C33104Hのフリーランが行えます。また、次回のデバッグを現在の内容から引き続いて行うことも可能となります。
フラッシュメモリのプロテクトも設定できます。

■ 書　式

sfl　　(ガイダンスモード)

■ 入力例

ガイダンスに従って、フラッシュメモリのプロテクト、書き込みの確認を行います。

```
>sfl↵
Protect flash memory      1.yes 2.no ...? 2↵      ...プロテクトの選択
Are you sure to save      1.yes 2.no ...? 1↵      ...書き込みの確認
Saving to flash memory ................  done
>
```

書き込みを中止するには確認の入力で、2を選択してください。

■ 注　意

- sflコマンドはS5U1C33104H上のフラッシュメモリに対応したコマンドで、ターゲットシステムのフラッシュメモリには適用されません。したがって、ICEモード以外では使用できません。
- フラッシュメモリにプロテクトがかかっている場合、エラーとなり書き込みは行えません。
  Error: Flash ROM is protected.

  プロテクトはeflコマンドでフラッシュメモリを消去することによって解除されます。

efl (erase flash memory) [ICE]

■ 機　能

S5U1C33104H上のフラッシュメモリの内容をマップ情報も含めて消去し、プロテクトを解除します。

■ 書　式

efl　　(直接入力モード)

■ 入力例

```
>efl↵」
Are you sure to erase     1.yes 2.no ...? 1↵」        ...消去の確認
Erasing flash memory ..................  done
>
```

■ 注　意

eflコマンドはS5U1C33104H上のフラッシュメモリに対応したコマンドで、ターゲットシステムのフラッシュメモリには適用されません。したがって、ICEモード以外では使用できません。

maf (map flash memory) [ICE]

■ 機　能

S5U1C33104H上のフラッシュメモリのマップ情報、チップ名、パラメータファイルのバージョンなどを表示します。

■ 書　式

maf (直接入力モード)

■ 表示例

(1) プロテクトされていない場合

```
>maf↵
Chip name                : 33104
Parameter file version   : 01
Internal ROM area        : 80000 - 80FFF
Size of F0 area          : 1000
Emulation memory area
     00C00000 - 00CFFFFF ROM
     00800000 - 008FFFFF RAM
Map break information
     00000000 - 000007FF RAM
     00040000 - 000402FF IO
     00048000 - 000482FF IO
     00C00000 - 00CFFFFF ROM emulation   ...エミュレーションメモリの表示
     00800000 - 008F7FFF RAM emulation
     00000000 - 000007FF stack area      ...スタックエリアの表示
     00800000 - 008FFFFF stack area
>
```

(2) プロテクトされている場合

```
>maf↵
Flash memory is protected.               ...プロテクト情報を表示
Chip name                : 33104
Parameter file version   : 01
Internal ROM area        : 80000 - 80FFF
Size of F0 area          : 1000
Emulation memory area
     00C00000 - 00CFFFFF ROM
     00800000 - 008FFFFF RAM
Map break information
     00000000 - 000007FF RAM
     00040000 - 000402FF IO
     00048000 - 000482FF IO
     00C00000 - 00CFFFFF ROM emulation
     00800000 - 008F7FFF RAM emulation
     00000000 - 000007FF Stack area
     00800000 - 008FFFFF Stack area
>
```

(3) フラッシュメモリが初期化状態にある場合

```
>maf↵
Flash memory is not mapped.
>
```

■ 注　意

mafコマンドはS5U1C33104H上のフラッシュメモリに対応したコマンドで、ターゲットシステムのフラッシュメモリには適用されません。したがって、ICEモード以外では使用できません。

16.10.12 トレースコマンド

tm (trace mode) [ICD / ICE / SIM]

■ 機　能

(1) ICEモード

ICEモードではトレースモードとトレーストリガ条件の表示、設定を行います。

トレースモード

次の2種類のトレースモードが設定可能です。

1) ノーマルトレースモード

常に最新のトレース情報をトレースメモリに書き込みます。

2) シングルディレイトリガトレースモード

トレーストリガポイント(トレーストリガ条件の成立)を基準に次の3種類のトレース採取領域を指定できます。

1. start ...トレーストリガポイントからトレース情報を採取
2. middle ...トレーストリガポイントの前後のトレース情報を採取
3. end ...トレーストリガポイントまでトレース情報を採取

トレーストリガ条件

次の3種類のトレーストリガ条件が設定できます。

1) アドレス

メモリアドレスを1個所指定できます。CPUがこのアドレスをアクセスするすることがトレーストリガを発生する条件となります。

2) データパターンとマスク

CPUが上記アドレスから読み出す、または書き込むデータパターンを指定します。16ビットのパターンを指定できます。また、任意のビットあるいは全ビットをマスクしてトレーストリガ条件から外すこともできます。

3) バスオペレーションタイプ

CPUが上記アドレスをアクセスする際のバスオペレーションタイプを指定します。選択可能なタイプは次のとおりです。

0. All ...すべてのバスオペレーション
1. Inst ...命令のフェッチ
2. VecR ...ベクタのフェッチ
3. DatR ...データリード
4. DatW ...データライト
5. StkR ...スタックリード
6. StkW ...スタックライト
7. DmaR ...DMAリード
8. DmaW ...DMAライト

これらの条件が成立した時点(トレーストリガポイント)でシングルディレイトリガトレースやS5U1C33104HのTRGOUT端子からのパルス出力が制御されます。

(2) ICDモード

ICDモードではトレースモード、トリガアドレス等の表示、設定を行います。

トレースモード

次の2種類のトレースモードが設定可能です。

1) 全範囲トレースモード

プログラムの実行開始と同時にトレースも開始し、トレース情報をS5U1C33000Hのトレースメモリに書き込みます。

2) 範囲トレースモード

トリガアドレス1からトリガアドレス2までの範囲内を実行した場合にのみ、トレース情報をS5U1C33000Hのトレースメモリに書き込みます。

全範囲トレースモードの条件設定

全範囲トレースモードを選択した場合は、トレースメモリへの書き込み条件を指定できます。

1) オーバーライトあり

トレースメモリ(131072サイクル分)がいっぱいになった場合、古いデータから上書きされます。したがって、トレースメモリは常に最新のトレース情報を保持します。

2) オーバーライトなし

トレースメモリがいっぱいになるまでトレース情報を書き込み、それ以降は書き込みを行いません。

範囲トレースモードの条件設定

範囲トレースモードでは以下の条件を指定できます。

1) トリガアドレス2でのブレーク

指定範囲をトレース後、トリガアドレス2でプログラムの実行をブレークするか、継続するか指定することができます。

2) 時間測定条件

プログラムの実行時間を、実行開始からブレークするまで測定(全範囲測定モード)するか、トリガアドレス1からトリガアドレス2までの範囲内の実行時間のみ(範囲測定モード)を測定するか選択できます。

上記の条件のほかに、トレース情報のクロック数(Clk)を各命令ごとの実行クロック数として表示させるか、トレース開始から積算した値で表示させるか選択できます。この選択はトレースモードにかかわらず行えます。

例:

積算

```
Cycle  Address Code Unassemble            Clk     ...
000009 0610FFE 0000 nop                   000000  ...
000008 0611000 C000 ext     0x0           000008  ...
000007 0611002 D00A ext     0x100a        000016  ...
000006 0611004 6DF0 ld.w    %r0,0x1f      000024  ...
                          :
```

命令単位

```
Cycle  Address Code Unassemble            Clk     ...
000009 0610FFE 0000 nop                   000008  ...
000008 0611000 C000 ext     0x0           000008  ...
000007 0611002 D00A ext     0x100a        000008  ...
000006 0611004 6DF0 ld.w    %r0,0x1f      000008  ...
                          :
```

(3) **シミュレータモード**

シミュレータモードでは以下の選択のみが行えます。

1)**トレース機能の**ON/OFF

ONに設定すると、プログラムの実行に従ってトレース情報が採取されます。

2)**レジスタ値、ソースコードの表示**

基本的なトレース情報のほかにレジスタの内容やソースコードも表示するか選択できます。

3)**トレース情報の出力先**

採取したトレース情報の出力先として、ウィンドウまたはファイルを選択できます。

ウィンドウを選択した場合は、トレース情報が[Trace]ウィンドウ、または[Command]ウィンドウ([Trace]ウィンドウが閉じている場合)に表示されます。

ファイル選択時は、ファイル名も指定します。

■ 書　式

tm　　(ガイダンスモード)

■ ICEモードの入力例

(1) **現在の設定内容の表示**

tmコマンドを実行すると、はじめに現在の設定内容を表示します。

```
>tm↵
Trace mode    : normal          ...または"single delay"
Address       : 00000000
Data pattern  : 0000
Data mask     : ffff
Type          : Inst
Display option: normal
1.normal 2.single delay ...? normal :↵   ...設定を変更しない場合は
>                                          [Enter]キー入力のみで終了
```

(2) **設定の変更**

現在の設定内容の表示に続くガイダンスに従って、新たな設定内容を入力あるいは選択してください。

```
1.normal 2.single delay ...? normal :2↵          ..."2.single delay"を選択
Trigger address 00000000 :00C00100↵              ...トリガアドレスを入力
Data pattern    0000 :↵                          ...[Enter]のみでスキップ(変更なし)
Data mask       ffff :↵
Bus type  0:All 1:Inst 2:VecR 3:DatR 4:DatW 5:StkR 6:StkW 7:DmaR 8:DmaW ...? Inst :↵
1.start  2.middle  3.end ...? start :1↵          ...トリガ位置を選択
Display option 1.normal 2.source ...? normal :2↵ ...表示オプションを選択
>
```

この例では、シングルディレイトリガモードを選択し、アドレス0xc00100の命令フェッチからトレースを開始するように設定しています。
トリガ位置の選択は、シングルディレイモードを選択した場合にのみ表示され、ノーマルモードを選択した場合は表示されません。

データパターンは0x0000ですが、データマスクが0xffffですので、データはすべてのビットがマスクされ、トリガ条件には影響を与えません。したがって、この例の設定ではCPUが0xc00100番地から命令をフェッチすれば、命令コードにかかわらずトレーストリガ条件が成立します。
データのリード/ライトをトリガ条件にする場合、データは16ビットで指定しますので、バイトアクセス対してはデータマスクで対応する必要があります。たとえば、バイトアクセスでデータを0x12に設定する場合、偶数アドレスに対してはデータを0x1200と指定し、下位8ビットをデータマスク0x00ffで無効にしておきます。奇数アドレスに対しては、データを0x0012、データマスクを0xff00と指定します。

表示オプションは"1.normal"を選択すると、表示の際にS5U1C33104Hのトレース情報のみが表示され、"2.source"を選択すると、トレース情報に加えソースコードも表示されます。

アドレスは、シンボルあるいはソース行番号で指定することもできます。

```
Trigger address 00000000 :i↵            ...シンボル入力例
Trigger Address 00000000 :main.c#24↵    ...行番号入力例
```

ガイダンス途中でコマンドを終了するには[q] [Enter]を入力してください。途中で終了した場合、そこまでに設定した内容は変更されます。
また、[^] [Enter]で入力項目を1つ手前に戻すこともできます。

■ ICEモードの注意

トリガアドレスは、各機種のメモリ領域の範囲内で指定してください。これを越えた場合、エラーとなり、再度アドレス入力のガイダンスが表示されます。

Error: Address range (0-0xFFFFFFF).　　...0xffffffffを越えるアドレス
Error: No map area.　　...未使用アドレスを指定した場合

■ ICDモードの入力例

(1) 現在の設定内容の表示

tmコマンドを実行すると、はじめに現在の設定内容を表示します。

```
>tm↵
Trace mode           : area                ...または"all"
Over write           : no                  ...または"yes"
Measure mode         : all                 ...または"area"
Trigger address1     : 0000100
Trigger address2     : 0000000
Address2 break       : on                  ...または"off"
Counter display mode : accumulate          ...または"each instruction"
Trace mode 1.all 2.area ...? area :q↵      ...設定を変更しない場合は
>                                                [q]キー入力で終了
```

(2) 設定の変更

現在の設定内容の表示に続くガイダンスに従って、新たな設定内容を入力あるいは選択してください。

全範囲トレースモードの設定

```
Trace mode 1.all 2.area ...? area :1↵              ..."1.all"を選択
Over write 1.yes 2.no ...? no :1↵                  ...オーバーライト条件を選択
Counter display mode 1.accumulate 2.each instruction ...? accumulate :2↵
>                                                  ...カウンタ表示モードを選択
```

この例では、全範囲トレースモードを選択し、トレースメモリのオーバーライトありとクロック数の表示を命令単位に設定しています。

範囲トレースモードの設定

```
Trace mode 1.all 2.area ...?  all :2↵              ..."2.area"を選択
Measure mode 1.all 2.area ...?  all :2↵            ...時間測定条件を選択
Trigger address1 ? 0000100 :600000↵                ...トリガアドレス1を入力
Trigger address2 ? 0000000 :600100↵                ...トリガアドレス2を入力
Address2 break 1.on 2.off ...? on :1↵              ...アドレス2でのブレークを選択
Counter display mode 1.accumulate 2.each instruction ...? accumulate :1↵
                                                   ...カウンタ表示モードを選択
Warning: Hard PC break is not stopped at present mode.
>
```

この例では、範囲トレースモードを選択し、実行時間も指定範囲の測定を設定しています。トレース範囲は0x600000～0x600100で、0x600100でブレークします。トレース情報のクロック数は積算して表示されます。

ガイダンス途中でコマンドを終了するには[q] [Enter]を入力してください。途中で終了した場合、そこまでに設定した内容は変更されます。
また、[^] [Enter]で入力項目を1つ手前に戻すこともできます。
[Enter]キーのみを入力した場合、その項目は現在の設定のまま変更されません。

■ ICDモードの注意

- トリガアドレスは、各機種のメモリ領域の範囲内で指定してください。これを越えた場合、エラーとなり、再度アドレス入力のガイダンスが表示されます。

  Error: Address range (0-0xFFFFFFF).　...0xfffffffを越えるアドレス
  Error: No map area.　...未使用アドレスを指定した場合

- 範囲トレースモードを設定すると、ハードウェアPCブレーク機能は無効となります。ただし、ハードウェアPCブレークアドレスの設定はクリアされず、範囲トレースモードを解除した時点で有効になります。

■ シミュレータモードの入力例

(1) **トレースモードを**ON

```
>tm
Trace  1.on  2.off ...? 1↵                          ...トレースモードをON
Display option 1.normal 2.register 3.source 4.register,source ...? 1↵
Display mode 1.window 2.file ... ? 1↵               ...出力先をウィンドウに設定
>

>tm
Trace  1.on  2.off ...? 1↵                          ...トレースモードをON
Display option 1.normal 2.register 3.source 4.register,source ...? 4
Display mode 1.window 2.file ... ? 2↵               ...出力先をファイルに設定
File name ? : test.trc↵                             ...出力ファイル名を指定
R0  R1  R2  R3  R4  R5  R6  R7                      ...レジスタ表示形式
R8  R9  R10 R11 R12 R13 R14 R15
SP  AHR ALR
>
```

この設定以降にプログラムを実行すると、実行した命令ごとにトレース情報を表示/出力します。
ガイダンスの途中で[Enter]キーのみを入力した場合は、コマンドの実行を終了します。
表示オプションについては、次の項で説明します。

(2) **トレースモードを**OFF

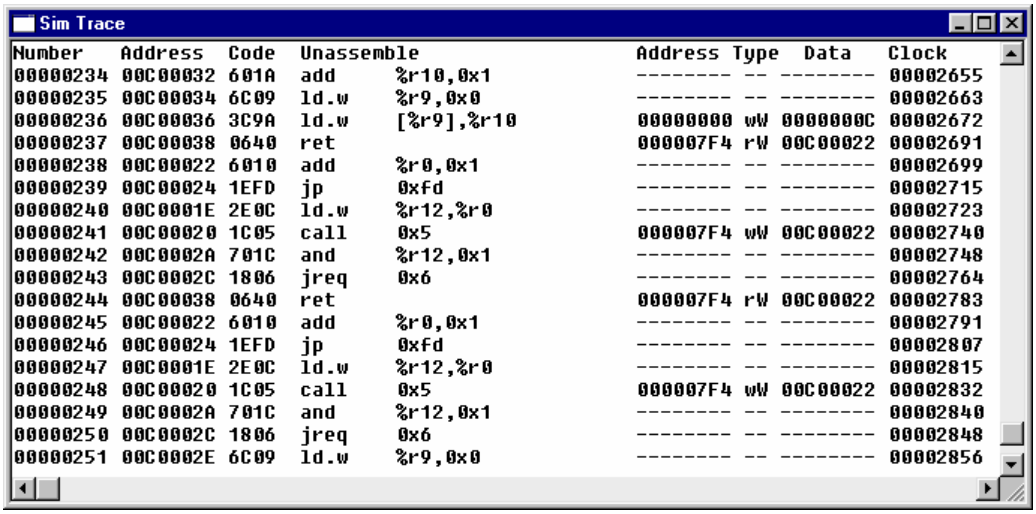
```
>tm↵
Trace  1.on  2.off ...? 2↵                          ...トレースモードをOFF
>
```

■ シミュレータモードのトレース情報表示

トレースモードをONにする際に出力先をウィンドウに設定してあると、トレース情報はプログラムを連続実行あるいはステップ実行させた場合に[Trace]ウィンドウに表示されます。[Trace]ウィンドウが閉じている場合には、[Command]ウィンドウに表示されます。
ファイルを選択した場合は指定のファイルに出力され、ウィンドウには表示されません。

[Trace]ウィンドウでは最新情報から最大255命令前までのトレース情報が確認できます。
表示オプションの選択による表示例を以下に示します。

"1.normal"**選択時**



Sim Trace

Number	Address	Code	Unassemble		Address	Type	Data	Clock
00000234	00C00032	601A	add	%r10,0x1	--------	--	--------	00002655
00000235	00C00034	6C09	ld.w	%r9,0x0	--------	--	--------	00002663
00000236	00C00036	3C9A	ld.w	[%r9],%r10	00000000	wW	0000000C	00002672
00000237	00C00038	0640	ret		000007F4	rW	00C00022	00002691
00000238	00C00022	6010	add	%r0,0x1	--------	--	--------	00002699
00000239	00C00024	1EFD	jp	0xfd	--------	--	--------	00002715
00000240	00C0001E	2E0C	ld.w	%r12,%r0	--------	--	--------	00002723
00000241	00C00020	1C05	call	0x5	000007F4	wW	00C00022	00002740
00000242	00C0002A	701C	and	%r12,0x1	--------	--	--------	00002748
00000243	00C0002C	1806	jreq	0x6	--------	--	--------	00002764
00000244	00C00038	0640	ret		000007F4	rW	00C00022	00002783
00000245	00C00022	6010	add	%r0,0x1	--------	--	--------	00002791
00000246	00C00024	1EFD	jp	0xfd	--------	--	--------	00002807
00000247	00C0001E	2E0C	ld.w	%r12,%r0	--------	--	--------	00002815
00000248	00C00020	1C05	call	0x5	000007F4	wW	00C00022	00002832
00000249	00C0002A	701C	and	%r12,0x1	--------	--	--------	00002840
00000250	00C0002C	1806	jreq	0x6	--------	--	--------	00002848
00000251	00C0002E	6C09	ld.w	%r9,0x0	--------	--	--------	00002856

"3.source"**選択時**

```
Sim Trace
Number   Address  Code  Unassemble                 Address Type  Data   Clock      File          Line  SourceCode
00000409 00C00032 601A  add     %r10,0x1           -------- -- -------- 00004639
00000410 00C00034 6C09  ld.w    %r9,0x0            -------- -- -------- 00004647
00000411 00C00036 3C9A  ld.w    [%r9],%r10         00000000 wW 00000008 00004656
00000412 00C00038 0640  ret                        000007F4 rW 00C00022 00004675  (main.c)       00024     }
00000413 00C00022 6010  add     %r0,0x1            -------- -- -------- 00004683  (main.c)       00011         for (j=0 ; ;
00000414 00C00024 1EFD  jp      0xfd               -------- -- -------- 00004699
00000415 00C0001E 2E0C  ld.w    %r12,%r0           -------- -- -------- 00004707  (main.c)       00013                 sub(
00000416 00C00020 1C05  call    0x5                000007F4 wW 00C00022 00004724
00000417 00C0002A 701C  and     %r12,0x1           -------- -- -------- 00004732  (main.c)       00020         if (k & 0x1)
00000418 00C0002C 1806  jreq    0x6                -------- -- -------- 00004748
00000419 00C00038 0640  ret                        000007F4 rW 00C00022 00004767  (main.c)       00024     }
00000420 00C00022 6010  add     %r0,0x1            -------- -- -------- 00004775  (main.c)       00011         for (j=0 ; ;
00000421 00C00024 1EFD  jp      0xfd               -------- -- -------- 00004791
00000422 00C0001E 2E0C  ld.w    %r12,%r0           -------- -- -------- 00004799  (main.c)       00013                 sub(
00000423 00C00020 1C05  call    0x5                000007F4 wW 00C00022 00004816
00000424 00C0002A 701C  and     %r12,0x1           -------- -- -------- 00004824  (main.c)       00020         if (k & 0x1)
00000425 00C0002C 1806  jreq    0x6                -------- -- -------- 00004832
00000426 00C0002E 6C09  ld.w    %r9,0x0            -------- -- -------- 00004840  (main.c)       00022                 i++;
```

"4.register, source"**選択時**

```
Sim Trace
Number   Address  Code  Unassemble                 Address Type  Data   Clock      File          Line  SourceCode
00000430 00C0000A 6C08  ld.w    %r8,0x0            -------- -- -------- 00004872  (boot.s)       00012         ld.w    %r8,
 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
 00000000 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
 00000800 AAAAAAAA AAAAAAAA          IL:0 MO:0 DS:0 IE:0 C:0 V:0 Z:0 N:0
00000431 00C0000C C000  ext     0x0                -------- -- -------- 00004880  (boot.s)       00013         xcall   main
 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
 00000000 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
 00000800 AAAAAAAA AAAAAAAA          IL:0 MO:0 DS:0 IE:0 C:0 V:0 Z:0 N:0
00000432 00C0000E 1C03  call    0x3                000007FC wW 00C00010 00004897
 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
 00000000 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
 000007FC AAAAAAAA AAAAAAAA          IL:0 MO:0 DS:0 IE:0 C:0 V:0 Z:0 N:0
00000433 00C00014 0200  pushn   %r0                000007F8 wW AAAAAAAA 00004906  (main.c)       00007     {
 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
 00000000 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
 000007F8 AAAAAAAA AAAAAAAA          IL:0 MO:0 DS:0 IE:0 C:0 V:0 Z:0 N:0
00000434 00C00016 6C0B  ld.w    %r11,0x0           -------- -- -------- 00004914  (main.c)       00010         i = 0;
 AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA AAAAAAAA
```

"2.register"を選択した場合は、"4"の表示からソース部分が削除されます。
[Command]ウィンドウへの表示、ファイルへの出力も上記の形式で行われます。

トレース情報の表示内容は次のとおりです。

<各トレース情報の1行目>

Number: 実行命令No.(10進数)
CPUをリセットまたはトレースをONにしてからの実行命令数
Address: 実行命令アドレス(16進数)
Code: 命令コード(16進数)
Unassemble: 実行命令の逆アセンブル内容
Address: アクセスしたメモリアドレス(16進数)
Type: バスオペレーションタイプ
rB: バイトデータリード、rH: ハーフワードデータリード、rW: ワードデータリード
wB: バイトデータライト、wH: ハーフワードデータライト、wW: ワードデータライト
Clock: 実行クロック数(10進数)
Data: 読み出し/書き込みデータ(16進数)
File: ソースファイル名(ソース表示を選択した場合のみ)
Line: ソースの行番号(ソース表示を選択した場合のみ)
SourceCode: ソースコード(ソース表示を選択した場合のみ)

<各トレース情報の2～4行目>

これらの行は、レジスタの表示を選択した場合にのみ表示されます。
各レジスタ値は次のような配置で表示されます。

R0	R1	R2	R3	R4	R5	R6	R7
R8	R9	R10	R11	R12	R13	R14	R15
SP	AHR	ALR		PSR(フラグ個別に表示)			

■ シミュレータモード時の注意事項

表示されるクロック数は、パラメータファイルに設定されたウェイト数情報を使い算出しています。パラメータファイルの設定に誤りがあると正しく表示されません。詳細については、"16.11 パラメータファイルについて"内の"ウェイト数の指定"を参照してください。

tm2 (trace mode 2) [SIM]

■ 機　能

シミュレータモードにおいて、[Register]ウィンドウに表示される実行カウンタの機能を、クロック数表示とするかインストラクション数表示とするか選択します。また、[Trace]ウィンドウのクロック表示を累積表示とするか命令単位とするか選択します。

■ 書　式

(1) **tm2** (ガイダンスモード)

(2) **tm2 <function> <display mode>** (直接入力モード)

<function>: 実行カウンタの表示(1=cycle, 2=instruction)

<display mode>: トレースのクロック表示モード(1=accumulate, 2=each instruction)

■ 入力例

書式1)
```
>tm2↵
Counter function 1.cycle 2.instruction ... ? 1↵
Counter display mode 1.accumulate 2.each instruction ...? 2↵
>
```

書式2)
```
>tm2 1 2↵
>
```

どちらの例も、[Register]ウィンドウの実行カウンタをクロック数表示に、[Trace]ウィンドウのクロック表示を命令単位に設定しています。

■ 注　意

シミュレータモードで表示されるクロック数は、パラメータファイルに設定されたウェイト数情報を使い算出しています。
パラメータファイルの設定に誤りがあると正しく表示されません。詳細については、"16.11 パラメータファイルについて"内の"ウェイト数の指定"を参照してください。

td (trace dump) [ICD / ICE]

■ 機　能

S5U1C33104H/S5U1C33000Hのトレースメモリに採取したトレース情報を表示します。

■ 書　式

td [<No.>] (直接入力モード)

<No.>: トレースサイクルNo.(10進数)
省略時は最新のトレースデータから表示

条件: ICEモード 0(最新データ)≦No≦32767(最旧データ)
ICDモード 0(最新データ)≦No≦131071(最旧データ)

■ 表示例

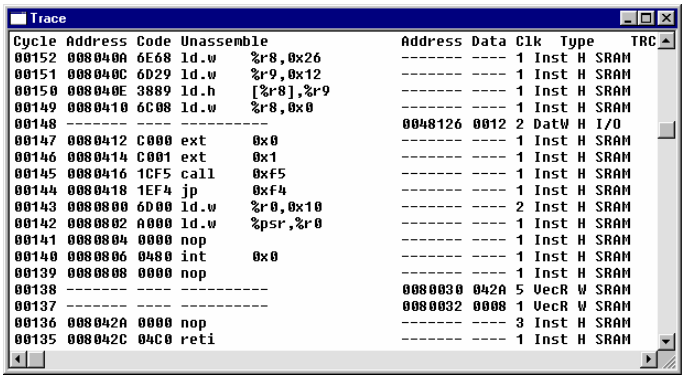
(1) [Trace]ウィンドウを開いている場合

```
>td↵
>
```

プログラムの実行により[Trace]ウィンドウがクリアされます。
実行終了後にtdコマンドを実行することによって、[Trace]ウィンドウが最新のデータを再表示します。ウィンドウの最下位行に最も新しいトレースデータが表示され、スクロールによってすべてのトレースデータを表示させることができます。
トレースサイクルNo.を指定した場合には、指定したサイクルのデータがウィンドウの最下位行に表示されます。この場合も、スクロールによってすべてのトレースデータを表示させることができます。

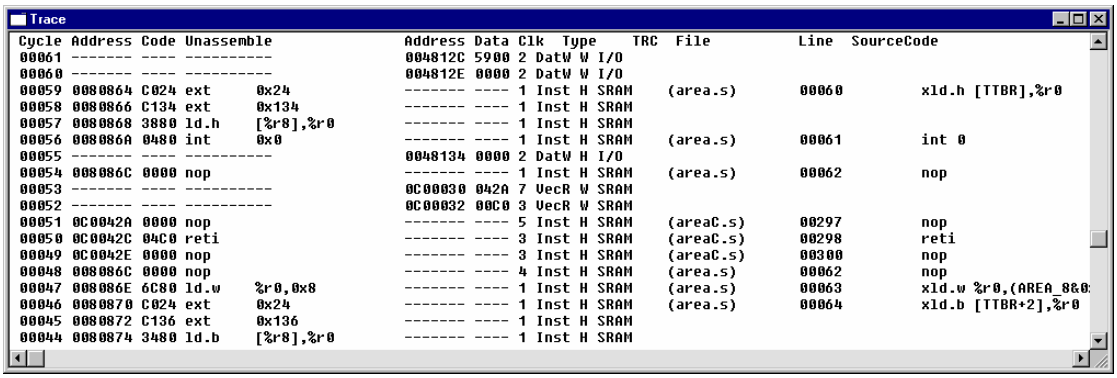
ICEモードの表示例

tmコマンドの表示オプションでノーマルを選択した場合の例



```
Trace
Cycle Address Code Unassemble                Address Data Clk  Type    TRC
00152 008040A 6E68 ld.w    %r8,0x26          ------- ---- 1 Inst H SRAM
00151 008040C 6D29 ld.w    %r9,0x12          ------- ---- 1 Inst H SRAM
00150 008040E 3889 ld.h    [%r8],%r9         ------- ---- 1 Inst H SRAM
00149 0080410 6C08 ld.w    %r8,0x0           ------- ---- 1 Inst H SRAM
00148 ------- ---- ----------                0048126 0012 2 DatW H I/O
00147 0080412 C000 ext     0x0               ------- ---- 1 Inst H SRAM
00146 0080414 C001 ext     0x1               ------- ---- 1 Inst H SRAM
00145 0080416 1CF5 call    0xf5              ------- ---- 1 Inst H SRAM
00144 0080418 1EF4 jp      0xf4              ------- ---- 1 Inst H SRAM
00143 0080800 6D00 ld.w    %r0,0x10          ------- ---- 2 Inst H SRAM
00142 0080802 A000 ld.w    %psr,%r0          ------- ---- 1 Inst H SRAM
00141 0080804 0000 nop                       ------- ---- 1 Inst H SRAM
00140 0080806 0480 int     0x0               ------- ---- 1 Inst H SRAM
00139 0080808 0000 nop                       ------- ---- 1 Inst H SRAM
00138 ------- ---- ----------                0080030 042A 5 VecR W SRAM
00137 ------- ---- ----------                0080032 0008 1 VecR W SRAM
00136 008042A 0000 nop                       ------- ---- 3 Inst H SRAM
00135 008042C 04C0 reti                      ------- ---- 1 Inst H SRAM
```

tmコマンドの表示オプションでソース表示を選択した場合の例



```
Trace
Cycle Address Code Unassemble                Address Data Clk  Type    TRC  File        Line  SourceCode
00061 ------- ---- ----------                004812C 5900 2 DatW W I/O
00060 ------- ---- ----------                004812E 0000 2 DatW W I/O
00059 0080864 C024 ext     0x24              ------- ---- 1 Inst H SRAM    (area.s)    00060       xld.h [TTBR],%r0
00058 0080866 C134 ext     0x134             ------- ---- 1 Inst H SRAM
00057 0080868 3880 ld.h    [%r8],%r0         ------- ---- 1 Inst H SRAM
00056 008086A 0480 int     0x0               ------- ---- 1 Inst H SRAM    (area.s)    00061       int 0
00055 ------- ---- ----------                0048134 0000 2 DatW H I/O
00054 008086C 0000 nop                       ------- ---- 1 Inst H SRAM    (area.s)    00062       nop
00053 ------- ---- ----------                0C00030 042A 7 VecR W SRAM
00052 ------- ---- ----------                0C00032 00C0 3 VecR W SRAM
00051 0C0042A 0000 nop                       ------- ---- 5 Inst H SRAM    (areaC.s)   00297       nop
00050 0C0042C 04C0 reti                      ------- ---- 3 Inst H SRAM    (areaC.s)   00298       reti
00049 0C0042E 0000 nop                       ------- ---- 3 Inst H SRAM    (areaC.s)   00300       nop
00048 008086C 0000 nop                       ------- ---- 4 Inst H SRAM    (area.s)    00062       nop
00047 008086E 6C80 ld.w    %r0,0x8           ------- ---- 1 Inst H SRAM    (area.s)    00063       xld.w %r0,(AREA_8&0
00046 0080870 C024 ext     0x24              ------- ---- 1 Inst H SRAM    (area.s)    00064       xld.b [TTBR+2],%r0
00045 0080872 C136 ext     0x136             ------- ---- 1 Inst H SRAM
00044 0080874 3480 ld.b    [%r8],%r0         ------- ---- 1 Inst H SRAM
```

ICEモード時のトレース情報の表示内容は次のとおりです。

Cycle: トレースサイクル(10進数)
トレースメモリ内に最後に取り込まれた情報が00000となります。
Address: CPUの命令フェッチアドレス(16進数)
命令フェッチ以外のアクセスでは"--------"が表示されます。
Code: CPUがフェッチした命令コード(16進数)
命令フェッチ以外のアクセスでは"----"が表示されます。
Unassemble: フェッチした命令コードの逆アセンブル内容
命令フェッチ以外のアクセスでは"--------"が表示されます。
Address: CPUがアクセスしたアドレス(16進数)... 命令フェッチ以外
命令フェッチの場合は"--------"が表示されます。
Data: 読み出し/書き込みデータ(16進数)... 命令フェッチ以外
命令フェッチの場合は"----"が表示されます。
Clk: バスオペレーションに使用したクロック数(1～7)
8クロック以上の場合は"V"と表示されます。
Type: バスオペレーションタイプ
Inst: 命令フェッチ、VecR: ベクタリード、DatR: データリード、
DatW: データライト、StkR: スタックリード、StkW: スタックライト、
DmaR: DMAリード、DmaW: DMAライト
アクセスサイズ
B: バイトアクセス、H: ハーフワードアクセス、W: ワードアクセス
メモリタイプ
SRAM、DRAM、BROM(バーストROM)、IRAM(内蔵RAM)、I/O(内蔵I/O)、
DBUG(ICE開発用)、ERR(その他)
TRC: TRCIN端子の入力(Lowの信号入力時にのみ"L"を表示)
T: トレーストリガポイント(行の先頭に表示)
トレーストリガ条件が一致したバスサイクルにのみ表示されます。
File: ソースファイル名(tmコマンドでソース表示を選択した場合のみ)
Line: ソースの行番号(tmコマンドでソース表示を選択した場合のみ)
SourceCode: ソースコード(tmコマンドでソース表示を選択した場合のみ)

ICDモードの表示例

```
ICD Trace
Cycle  Address Code Unassemble              Clk  Method File          Line  SourceCode
000017 0602D88 DFF8 ext      0x1ff8         002842 DPC
000016 0602D8A DFF9 ext      0x1ff9         002858 DPC
000015 0602D8C 1C44 call     0x44           002875 DPC
000014 0602014 6C0C ld.w     %r12,0x0       002912 DPC (sys.c)   00081    iBytes = 0;     /* no read now */
000013 0602016 C00C ext      0xc            002928 DPC (sys.c)   00086    for (;;)
000012 0602018 C401 ext      0x401          002944 DPC
000011 060201A 6C8F ld.w     %r15,0x8       002960 DPC
000010 060201C 6C14 ld.w     %r4,0x1        002976 DPC
000009 060201E 680E cmp      %r14,0x0       002992 DPC (sys.c)   00091        if (iReadBytes == 0)    /* if require
000008 0602020 1806 jreq     0x6            003008 DPC
000007 0602022 C308 ext      0x308          003024 DPC (sys.c)   00096        if (READ_EOF == 1)
000006 0602024 C089 ext      0x89           003040 DPC
000005 0602026 2485 ld.ub    %r5,[%r8]      003088 DPC
000004 0602028 6815 cmp      %r5,0x1        003089 DPC
000003 060202A 1A03 jrne     0x3            003110 DPC
000002 0602030 24FB ld.ub    %r11,[%r15]    003174 DPC (sys.c)   00101        iSize = READ_BUF[0];
000001 0602032 680B cmp      %r11,0x0       003175 DPC (sys.c)   00102        if (iSize > 0 )
000000 0602034 0E14 jrle     0x14           003196 DPC
```

ICDモード時のトレース情報の表示内容は次のとおりです。

Cycle: トレースサイクル(10進数)
トレースメモリ内に最後に取り込まれた情報が000000となります。
Address: CPUの命令実行アドレス(16進数)
Code: CPUが実行した命令コード(16進数)
Unassemble: 命令コードの逆アセンブル内容
Clk: 実行クロック数
デフォルトではトレース開始からのクロック数が累積されますが、各実行命令ごとのクロック数表示に変更することもできます。
Method: トレース解析(実行アドレス取得)方法
- SPC: 初期PCによる解析
- TRG: トリガアドレスによる解析
- DPC: DPCO信号による解析
- RET: call文、ret文による解析
- MAP: マップ情報による解析
- RTI: reti文による解析
- ---: 解析不能

File: 実行命令を含むソースファイル名
Line: ソース行番号
SourceCode: ソースコード

ICDモードでは、プログラム実行中に[Display trace]ボタンをクリックすることによりトレース動作を中止し、そこまでのトレースデータを[Trace]ウィンドウに表示させることもできます。これにより中止したトレース動作は、[Resume trace]ボタンをクリックすることで再開します。

[Display trace]**ボタン** [Resume trace]**ボタン**

(2) [Trace]ウィンドウを閉じている場合

[Command]ウィンドウに16行分(デフォルト)のトレースデータを表示します。表示行数はmdコマンドで変更可能です。
最も新しいデータが最下位行に、トレースサイクルNo.を指定した場合は指定のサイクルのデータが最下位行に表示されます。表示内容と形式は[Trace]ウィンドウと同様です。

(3) ログ出力

logコマンドによってコマンド実行結果をログファイルに出力する場合は、[Trace]ウィンドウを閉じて[Command]ウィンドウにトレースデータを表示させてください。
[Trace]ウィンドウが開いている場合、[Command]ウィンドウにはデータが表示されないため、表示内容がログファイルに出力されません。

(4) 連続表示機能

tdコマンドを実行すると、他のコマンドを実行するまでは[Enter]キーの入力のみでトレースデータを連続して表示することができます。
[Enter]キーを入力すると、[Trace]ウィンドウは1画面分前にスクロールします。
[Command]ウィンドウに表示している場合は、前回表示したサイクルの前の16行分(デフォルト)を表示します。

表示方向は、[Enter]キーの入力ごとに古い実行サイクル(FORWARD)に向かいますが、[B]キーによってこの方向を逆(BACKWORD)にすることができます。表示方向を戻すには[F]キーを入力します。[Trace]ウィンドウが開いている場合は、そのスクロール方向も変更されます。

```
>td 100↵」                    ...FORWORDで表示開始
(サイクルNo.115～100のデータを表示)
>b↵」                         ...BACKWORDに変更
(サイクルNo. 99～84のデータを表示)
>↵」                          ...BACKWORDで表示を継続
(サイクルNo.84～69のデータを表示)
>f↵」                         ...FORWORDに変更
(サイクルNo. 99～84のデータを表示)
>
```

■ ICEモード時の注意事項

- トレースサイクルNo.は0～32767の範囲内で指定してください。これを越えた場合、エラーとなります。
  Error: Trace range (0-32767).
- S5U1C33104Hの動作タイミングの関係上、トレース開始時のフェッチサイクル、トレース終了時の実行サイクル等、境界上のサイクルが記録されない場合があります。
- ステップ実行時やブレーク時は、まだ実行していないプリフェッチサイクルの情報も表示されます。また、ソフトウェアPCブレークによりプログラムの実行が中断された際には、ソフトウェアPCブレーク用に挿入したbrk命令のフェッチサイクルも表示されます。
- ソフトウェアPCブレークポイントに設定されているアドレスからプログラムを連続実行させると、S5U1C33104Hは内部的にそのアドレスを一度ステップ実行してから連続実行に移ります。このため、ステップ実行時にプリフェッチしたトレースデータが余分に表示されることがあります。
- ソースレベルのステップ実行の場合、S5U1C33104H内部ではシングルステップ動作を繰り返し行います。このため、各ステップごとのプリフェッチ情報が多数表示されます。
- 内蔵RAMのリード/ライトは、バスアクセスが検出できないためトレースデータを参照できません。
- 高速DMAによるデータ転送中は、データは正しくトレースされません。

■ ICDモード時の注意事項

- トレースサイクルNo.は0～131071の範囲内で指定してください。これを越えた場合、エラーとなります。
  Error: Trace range (0-131071).
- ICDモードでは、CPUからの4ビットの情報を元に、デバッガでプログラムの逆アセンブル情報を使用してトレースの解析を行います。このため、いくつかの制約があります。その詳細については、16.9.7項内の"ICDモードのトレース動作と注意事項"を参照してください。

ts (trace search) [ICD / ICE]

■ 機　能

トレースメモリの中から指定した条件でトレース情報を検索します。
ICEモードでは次の2種類の検索条件を指定することができます。

1. アドレス

指定のアドレスをアクセスしたサイクルを検索します。全メモリ領域を対象にすることもできます。

2. バスオペレーションタイプ

指定のバスオペレーションを行ったサイクルを検索します。選択可能なタイプは次のとおりです。

0. All	...すべてのバスオペレーション	5. StkR	...スタックリード
1. Inst	...命令のフェッチ	6. StkW	...スタックライト
2. VecR	...ベクタのフェッチ	7. DmaR	...DMAリード
3. DatR	...データリード	8. DmaW	...DMAライト
4. DatW	...データライト		

ICDモードではプログラムの実行アドレスにより検索を行います。

さらにどちらのモードの場合も、該当するサイクルの前、および後の情報をそれぞれ0～256行の範囲で表示させることができます。

■ 書　式

ts　　(ガイダンスモード)

■ 入力例

ICEモードの場合

```
>ts↵
Search address(* is all area) ? : 200↵      ...アドレスを指定(全領域の場合は*を入力)
Bus type  0:All 1:Inst 2:VecR 3:DatR 4:DatW 5:StkR 6:StkW 7:DmaR 8:DmaW ...? 2
Number of pre lines  (0-256)  ? : 2↵        ...該当行の前のデータを表示する行数
Number of post lines (0-256)  ? : 3↵        ...該当行の後のデータを表示する行数
Find 0 trace data. (0 lines)                ...該当件数を表示
>
```

ICDモードの場合

```
>ts↵
Search address                ? : 811038↵   ...アドレスを指定
Number of pre lines  (0-256)  ? : 1↵        ...該当行の前のデータを表示する行数
Number of post lines (0-256)  ? : 2↵        ...該当行の後のデータを表示する行数
Find 2001 trace data. (8003 lines)          ...該当件数を表示
>
```

■ 検索結果の表示

該当件数は[Command]ウィンドウに表示されます。
検索されたトレースデータの表示はトレースサイクル番号順となります。

(1) [Trace]ウィンドウを開いている場合

検索されたトレース情報は[Trace]ウィンドウに表示されます。
[Trace]ウィンドウはサーチモードに切り換わり、ウィンドウを垂直方向にスクロールすることにより検索データを連続的に表示します。このモードはtdコマンドが入力されるまで有効です。

(2) [Trace]ウィンドウを閉じている場合

[Command]ウィンドウに16行分(デフォルト)の検索データを表示します。表示行数はmdコマンドで変更可能です。
表示形式、および検索結果が16件以上の場合の表示方法はtdコマンドと同様です。

■ 注　意

- tsコマンドはICEモードまたはICDモード時のみ使用可能です。
- アドレスは、各機種のメモリ領域の範囲内で指定してください。これを越えた場合、エラーとなり、再度アドレス入力のガイダンスが表示されます。

  Error: Address range (0-0xFFFFFFFF).　...0xffffffffを越えるアドレス
  Error: No map area.　...未使用アドレスを指定した場合

tf (trace file) [ICD / ICE]

■ 機　能

トレースメモリ内のデータ、もしくはtsコマンドによる検索結果(tsコマンド実行直後の場合)の中から、指定範囲のデータをファイルに保存します。

■ 書　式

tf　　(ガイダンスモード)

■ 入力例

ICEモードの場合

```
>tf↵
Start cycle number (max 32767) ? : 2000↵  ...保存を開始するサイクル番号を入力
End cycle number   (min     0) ? : 0↵     ...保存を終了するサイクル番号を入力
File name ? : d:\trace.txt↵               ...保存するファイル名を入力
Processing 2000-1001 cycle.               ...1000サイクル単位で、進行状況を表示をします。
Processing 1000-1 cycle.
Processing 0-0 cycle.
>
```

ICDモードの場合

```
>tf↵
Start cycle number (max 131071) ? : 1000↵  ...保存を開始するサイクル番号を入力
End cycle number   (min      0) ? : 0↵     ...保存を終了するサイクル番号を入力
File name ? : d:\trace.txt↵                ...保存するファイル名を入力
Reading trace data.
Making file.
>
```

■ 注　意

・ tfコマンドはICEモードまたはICDモード時のみ使用可能です。

・ 既存のファイルを指定すると、データを上書きします。

・ tsコマンドによる検索結果は、表示どおりに若い番号から保存します。

16.10.13 シミュレーテッドI/O

stdin (standard input) [ICD / ICE / SIM / MON]

■ 機　能

データを[Simulated I/O]ウィンドウまたはファイルから入力して、プログラムに渡すための設定を行います。
stdinコマンドでは、以下の条件を設定します。

・ブレークアドレス　(db33がデータを取り込む位置)
・入力バッファアドレス　(65バイトのバッファ)
・入力対象　([Simulated I/O]ウィンドウまたはファイル)

プログラム側での対応については、"16.9.8 シミュレーテッドI/O"を参照してください。

■ 書　式

stdin　(ガイダンスモード)

■ 入力例

(1) 設定

```
>stdin↵
Break address  : ********                ...現在の設定内容(********は未設定状態)
Buffer address : ********
1.set  2.clear ...? 1↵                   ..."1. set"を選択
Break address ? :READ_FLASH↵             ...ブレークアドレスを設定
Buffer address ? :READ_BUF↵              ...入力バッファアドレスを設定
Input mode  1.window  2.file ...? 1↵     ...データの入力元を指定
>
```

この設定の後でプログラムを連続実行させると、db33はプログラム内のラベルREAD_FLASHの位置で実行を中断し、[Simulated I/O]ウィンドウを表示します。ウィンドウにデータを入力し、[Enter]キーを押すと、db33は入力したデータを入力バッファ(READ_BUF)に取り込み、プログラムの実行を再開します。

Input modeで"2. file"を選択した場合は、ファイル名の入力も行います。

```
Input mode  1.window  2.file ...? 2↵
File name ? :input.txt↵                  ...データを入力するファイル名を指定
>
```

ファイルを選択した場合はブレーク位置で[Simulated I/O]ウィンドウを開かずに、指定のファイルから1行分のデータを取り込みます。

ガイダンス途中で[Enter]キーのみを入力すると、コマンドの実行を終了します。

(2) 解除

```
>stdin↵
Break address  : 0008017C READ_FLASH     ...現在の設定内容
Buffer address : 00000048 READ_BUF
1.set  2.clear ...? 2↵                   ..."2. clear"を選択
>
```

データ入力機能が解除されます。

■ 注　意

・stdinコマンドで設定するブレークアドレスは、ソフトウェアPCブレークと重複することはできません。ソフトウェアPCブレークポイントを解除してからstdinコマンドを実行してください。ハードウェアPCブレークポイントとの重複は許されます。

・ブレークアドレスおよびバッファアドレスは、各機種のメモリ領域の範囲内で指定してください。これを越えた場合、エラーとなり、再度アドレス入力のガイダンスが表示されます。

```
Error: Address range (0-0xFFFFFFF).      ...0xfffffffを越えるアドレス
Error: No map area.                      ...未使用アドレスを指定した場合
```

・ICEモードでシミュレーテッドI/O機能を使用する場合、ボーレートを115200bpsに設定し、mdコマンドでオンザフライ表示間隔を0に設定すると、表示レスポンスが改善されます。

stdout (standard output) [ICD / ICE / SIM / MON]

■ 機　能

指定の出力バッファのデータを[Simulated I/O]ウィンドウまたはファイルに出力するための設定を行います。
stdoutコマンドでは、以下の条件を設定します。

・ブレークアドレス　(db33がデータを出力する位置)
・出力バッファアドレス　(65バイトのバッファ)
・出力対象　([Simulated I/O]ウィンドウ、ファイルまたは両方)

プログラム側での対応については、"16.9.8 シミュレーテッドI/O"を参照してください。

■ 書　式

stdout　(ガイダンスモード)

■ 入力例

(1) 設定

```
>stdout↵
Break address  : ********                ...現在の設定内容(********は未設定状態)
Buffer address : ********
1.set  2.clear ...? 1↵                   ..."1. set"を選択
Break address ? :WRITE_FLASH↵            ...ブレークアドレスを設定
Buffer address ? :WRITE_BUF↵             ...入力バッファアドレスを設定
Output mode  1.window  2.file 3.window & file ...? 3↵      ...データの出力先を指定
File name ? :output.txt↵                 ...データを出力するファイル名を指定
>
```

この設定の後でプログラムを連続実行させると、db33はプログラム内のラベルWRITE_FLASHの位置で実行を中断し、[Simulated I/O]ウィンドウを表示します。次に、指定のバッファ(WRITE_BUF)内のデータを[Simulated I/O]ウィンドウと指定のファイルに出力します。
出力先をファイルのみと指定した場合、[Simulated I/O]ウィンドウは開きません。

ガイダンス途中で[Enter]キーのみを入力すると、コマンドの実行を終了します。

(2) 解除

```
>stdout↵
Break address  : 000801D0 WRITE_FLASH    ...現在の設定内容
Buffer address : 00000004 WRITE_BUF
1.set  2.clear ...? 2↵                   ..."2. clear"を選択
>
```

データ入力機能が解除されます。

■ 注　意

・ stdoutコマンドで設定するブレークアドレスは、ソフトウェアPCブレークと重複することはできません。ソフトウェアPCブレークポイントを解除してからstdoutコマンドを実行してください。ハードウェアPCブレークポイントとの重複は許されます。

・ ブレークアドレスおよびバッファアドレスは、各機種のメモリ領域の範囲内で指定してください。これを越えた場合、エラーとなり、再度アドレス入力のガイダンスが表示されます。

Error: Address range (0-0xFFFFFFF).　...0xfffffffを越えるアドレス
Error: No map area.　...未使用アドレスを指定した場合

・ ICEモードでシミュレーテッドI/O機能を使用する場合、ボーレートを115200bpsに設定し、mdコマンドでオンザフライ表示間隔を0に設定すると、表示レスポンスが改善されます。

16.10.14 その他のコマンド

com (execute command file) [ICD / ICE / SIM / MON]

■ 機　能

コマンドファイルを読み込み、その中に記述されたデバッグコマンドを連続実行します。

■ 書　式

(1) **com** (ガイダンスモード)
(2) **com <file name>** (直接入力モード)
<file name>: コマンドファイル名

■ 入力例

ファイル名=startup.cmd

```
; Load file          ...";"から行の終わりまではコメントとみなされます。
lf test.srf
; Cold reset
rstc
; Display mode
m BOOT

>com↵
File name ? :startup.cmd↵      ...コマンドファイル名を入力
>lf test.srf
>rstc
>m BOOT
>
```

ファイルに記述されたコマンドを[Command]ウィンドウに表示し、実行します。

■ 注　意

- コマンドファイル内から別のコマンドファイルを読み込むことも可能です。ただし、最大5階層までに制限されます。6階層目のcom(またはcmw)コマンドが現れるとエラーとなり、それ以後の実行を中止します。
  Error: cannot open file, <ファイル名>.
- デバッガdb33の起動コマンドで-cオプションを指定すると、1つのコマンドファイルをデバッガの起動時に実行させることができます。
  db33 -c startup.cmd -p 88104_1.par
- comコマンドを入力して指定ファイルに記述されたコマンドを実行させた後は、g、s、nコマンドと同様に、他のコマンドを入力するまで[Enter]キーによってそのコマンドを繰り返し実行できます。
  たとえば、次の2行のコマンドが記述されたコマンドファイル"test.cmd"をcomコマンドで実行させます。

  ```
  s
  db  800100
  ```
  ... test.cmdの内容

  ```
  >com test.cmd↵   ..."s"および"db 800100"を実行します。
  >↵               ...上記を繰り返します。
  >↵               ...上記を繰り返します。
  ```

  これにより、複数のコマンドが[Enter]キーの入力のみで繰り返し実行できます。
- コマンドファイルを実行中も、[Key break]ボタンによってコマンドの実行を中止することができます。ただし、時間のかかるコマンド(広範囲のフィル等)を実行中は、入力が受け付けられるまでボタンを押し続けてください。

cmw (execute command file with wait) [ICD / ICE / SIM / MON]

■ 機　能

コマンドファイルを読み込み、その中に記述されたデバッグコマンドを一定時間ごとに実行します。
個々のコマンドの実行間隔は、mdコマンドにより1～256秒の範囲(1秒単位)で設定できます。デバッガの初期設定は1秒です。

■ 書　式

(1) **cmw** (ガイダンスモード)
(2) **cmw <file name>** (直接入力モード)
<file name>: コマンドファイル名

■ 入力例

```
>cmw↵
File name ? :infodisp.cmd↵          ...コマンドファイル名を入力
   :                                ...コマンドを実行
```

■ 注　意

- コマンドファイル内から別のコマンドファイルを読み込むことも可能です。ただし、最大5階層までに制限されます。6階層目のcmw(またはcom)コマンドが現れるとエラーとなり、それ以後の実行を中止します。
  Error: Cannot open file, <ファイル名>.
- comコマンドにより読み込むコマンドファイル内にcmwコマンドを記述すると、それ以降のファイル内のコマンドはすべて(comコマンドがあった場合でも)指定の時間ごとに実行されます。
- comコマンドと同様に、cmwコマンドでも[Enter]キーによる繰り返し実行が可能です。
- コマンドファイルを実行中も、[Key break]ボタンによってコマンドの実行を中止することができます。ただし、時間のかかるコマンド(広範囲のフィル等)を実行中は、入力が受け付けられるまでボタンを押し続けてください。

log (logging) [ICD / ICE / SIM / MON]

■ 機　能

入力したコマンドと、[Command]ウィンドウに表示されるコマンド実行結果をファイルに保存します。

■ 書　式

(1) **log** (ガイダンスモード)

(2) **log <file name>** (直接入力モード)

<file name>: ログファイル名

■ 保存内容

ログファイルには[Command]ウィンドウに表示された内容がそのままファイルに書き込まれます。
ツールバーやメニューから実行したコマンドや他のウィンドウに表示される実行結果は、[Command]ウィンドウに表示されないためファイルには出力されません。ログを保存する場合には、[Command]ウィンドウ以外のウィンドウを閉じてコマンドを実行してください。

■ 入力例

```
>log↵
File name ? :log1.log↵          ...ログファイル名を入力
log on                          ...ログ出力を開始
>
                : ログ出力は次回のlogコマンド実行まで有効
>log↵
log off                         ...ログ出力を終了
>
```

■ 注　意

- 既存のファイル名が指定された場合、logコマンドはファイルを上書きします。
- ログ出力を行う場合は、[Command]ウィンドウ以外のウィンドウを閉じ、[Command]ウィンドウへの実行結果の表示行数(デフォルト: 16行)をmdコマンドによって多く設定すると、キー操作の回数と時間を減らすことができます。

od (option data dump) [ICE]

■ 機　能

S5U1C33104Hからオプションデータを読み出して、[Command]ウィンドウに16進ダンプ形式で表示します。

■ 書　式

od (ガイダンスモード)

■ 入力例

ガイダンスに従って、表示範囲の先頭アドレス、終了アドレスを入力します。

```
>od↵
Start address ? :0↵
End   address ? :f↵
      0  1  2  3  4  5  6  7   8  9  A  B  C  D  E  F
0000: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
>
```

・ 開始アドレス、終了アドレスは[Enter]キーの入力のみで省略可能です。
開始アドレスを省略すると、アドレス0から表示されます。
終了アドレスを省略すると、オプション領域の最終アドレスが指定されたものとみなされます。

・ 一度に表示可能な行数は16行(デフォルト)です。終了アドレスを16行以上表示するように指定しても16行分の表示後、コマンド入力待ちとなります。続くデータは[Enter]キーの入力で表示されます。この表示行数はmdコマンドで変更可能です。

・ 未使用領域のデータは"*"として表示されます。

■ 注　意

・ odコマンドはICEモード以外では実行できません。
Error: Command is not supported at present mode.

・ 開始、終了アドレスともに、オプションの設定範囲内で指定してください。これを越えた場合、エラーとなります。
Error: Address range (0-0x3FFF).　...指定アドレスが範囲外

・ 開始アドレスが終了アドレスより大きい場合、エラーとなります。
Error: address1 > address2

・ オプションデータのデフォルト値は0です。

ct (change type) [ICD / ICE / SIM / MON]

■ 機　能

入力した数値、文字列を他の表示形式に変換して表示します。

■ 書　式

(1) **ct** (ガイダンスモード)
(2) **ct <value>** (直接入力モード)
(3) **ct <option><value>** (直接入力モード)

<value>: 変換する数値(10進数、16進数、実数)
<option>: **:** ...2進数を変換
. ...16進数をdouble型に変換
' ...16進数を文字列に変換
" ...文字列を16進数に変換

■ 入力例

(1) **ガイダンス入力**

```
>ct↵
Value ? :12345↵                                ...変換する数値を入力
bin    : 00000000000000000011000000111001
hex    : 00003039
>
```

(2) 2**進数の変換**

2進数の先頭に":"を付けて入力します。入力された2進数は、10進数、16進数、単精度実数形式に変換されます。

```
>ct :01000001↵
dec    : 65
hex    : 00000041
float  : 9.108440018e-044                      ...floatの表示は小数点以下9桁
>
```

(3) 10**進数の変換**

負の数値には"-"を付けて入力します。正の数値に"+"は付けないでください。
入力された10進数は、2進数、16進数に変換されます。

```
>ct 123456↵
bin    : 00000000000000011110001001000000
hex    : 0001E240
>ct -1↵
bin    : 11111111111111111111111111111111
hex    : FFFFFFFF
>
```

(4) 16**進数の変換**

<u>2進数、10進数、単精度実数形式への変換</u>

通常の16進数表記による入力は2進数は、10進数、単精度実数形式に変換されます。

```
>ct 0x41abcd↵
bin    : 00000000010000011010101111001101
dec    : 4303821
float  : 6.030937758e-039
>
```

文字列への変換

16進数を文字列に変換する場合は、先頭に" ' "を付け、1バイトずつスペース、タブまたはカンマで区切って入力してください。最大16バイトまで変換可能です。
表示可能なASCIIコード(0x20～0x7F)は文字に変換され、それ以外のコードはスペースとなります。入力する16進数に"0x"は不要です。

```
>ct '41 42 43 44 45 46↵
string : "ABCDEF"
>
```

倍精度実数形式への変換

倍精度実数形式に変換する場合は、先頭に" . "を付け、8バイトの16進数を入力します。上位4バイトと下位4バイトの間にはスペース、タブまたはカンマが必要です。入力する16進数に"0x"は不要です。

```
>ct .3fffffff ffffffff↵
double : 1.9999995231628416e+000          ...doubleの表示は小数点以下16桁
>
```

(5) **文字列の変換**

文字列の先頭に" " "を付けて入力します。最大16文字まで入力可能です。
入力された文字列は、16進数に変換されます。

```
>ct "abcd↵
hex    : 61 62 63 64
>
```

(6) **実数の変換**

実数は単精度実数サイズおよび倍精度実数サイズの2進数と16進数に変換されます。

```
>ct 1.75↵
bin(F) : 00111111111000000000000000000000
bin(D) : 0011111111111100000000000000000000000000000000000000000000000000
hex(F) : 3FE00000
hex(D) : 3FFC000000000000
>
```

■ 注　意

- 入力された2進数、10進数、16進数は32ビットの数値として扱われます。32ビットを越える数値を入力した場合は、変換結果が"--------"として表示されます。
- 10進数、実数の入力には負の数値(" - "付き)の指定も許されます。
- 変換後の10進数は符号付き32ビットデータです。

ext (extended instruction) [ICD / ICE / SIM / MON]

■ 機　能

ext命令により拡張された即値を計算し、結果を命令エクステンダの拡張命令形式で返します。

■ 書　式

(1) **ext** (ガイダンスモード)

(2) **ext <address>** (直接入力モード)

<address>: ターゲット命令のアドレス(16進数、シンボルまたはソース行番号)

条件: 0x0≦address≦0xffffffe(ハーフワード境界)

■ 入力例

ガイダンス入力

```
>ext↵
Address ? :80100↵      ...ターゲット命令のアドレスを入力
  :
```

直接入力

```
>ext 80100↵
    :
```

(1) **分岐命令**

ext命令がない場合

```
address  code  instruction
C00000   0000  nop
C00002   1E0F  jp     0xf

>ext c00002
xjp      0x1e          (0x00C00020)
>
```

分岐命令の場合には、即値の最下位ビットに0が拡張されます。( )内には分岐先のアドレスが表示されます。

ext命令がある場合

```
address  code  instruction
C00000   0000  nop
C00002   C010  ext    0x10
C00004   C000  ext    0x0
C00006   1E0F  jp     0xf

>ext c00006
xjp      0x800000      (0x00140006)
>
```

(2) **分岐命令以外**

```
address  code  instruction
C00000   0000  nop
C00002   C100  ext    0x100
C00004   3021  ld.w   %r1,[%r2]

>ext c00004
xld.w    %r1.[%r2+0x100]
>
```

■注　意

・ extによって拡張されない命令のアドレスを指定した場合は、エラーとなります。

Error: Target instruction cannot be extended.

・ ext命令は指定アドレスの直前にあるものが有効です。複数存在する場合は直前の2個までが有効となります。

・ アドレスは、各機種のメモリ領域の範囲内で指定してください。これを越えた場合、エラーとなります。

Error: Address range (0-0xFFFFFFF).　　...0xfffffffを越えるアドレス
Error: No map area.　　...未使用アドレスを指定した場合

・ アドレスにはハーフワード境界アドレス(偶数アドレス)を指定してください。奇数アドレスを指定した場合はワーニングとなり、アドレスの最下位ビットを0として処理されます。

Warning: Round down to multiple of 2.

ma (map information) [ICD / ICE / SIM / MON]

■機　能

パラメータファイルによって設定されたマップ情報を表示します。

■書　式

ma　　(直接入力モード)

■表　示

コマンド入力後、チップ名、パラメータファイルのバージョン、各領域のマップ情報を表示します。

例: 0x600000～0x6fffffをビッグエンディアンとした場合

```
>ma
Chip name                : 33208
Parameter file version   : 01
Internal ROM area        : 80000 - 80FFF
Size of F0 area          : 0000
Emulation memory area (not used in simulator mode)
     00C00000 - 00CFFFFF RAM
Memory map informatiopn
     00000000 - 000007FF RAM
     00040000 - 0004FFFF IO
     00200000 - 002FFFFF RAM
     00600000 - 006FFFFF RAM           ,Big endian
     00C00000 - 00CFFFFF ROM emulation
     00600000 - 006FFFFF Stack area
>
```

md (mode) [ICD / ICE / SIM / MON]

■ 機　能

以下のモード設定を行います。

1. **シングルステップ実行結果と**[Memory]**ウィンドウのステップ表示モード**
   このステップ表示モードをOnにすると、シングルステップ(s、nコマンド)動作時に、実行結果が各ステップごとに表示されます。Offにした場合は最終ステップのみを表示します。[Register]ウィンドウが開いている場合は、このモード設定に従って表示内容を更新し、閉じているときには[Command]ウィンドウに表示します。[Source]ウィンドウが開いている場合は、このモード設定に従って実行する行を下線で表示します。
   ステップ表示モードがOnの場合の[Memory]ウィンドウは、ステップ動作時は各ステップごとに、連続実行時はブレーク後に表示内容を更新します。ステップ表示モードがOffの場合、[Memory]ウィンドウは自動的には更新されません。表示の更新にはメモリ表示コマンドの実行やウィンドウのスクロール操作が必要になります。

2. **実行カウンタのモード**
   積算モードまたはリセットモードが選択できます。
   リセットモード時は、プログラム実行コマンドを入力するごとに([Enter]キーによる実行も含む)、カウンタの値がリセットされます。
   実行カウンタの値は、積算モードをリセットモードに変更することによってもリセットされます。

3. ICD**モードの実行カウンタ機能**(ICD**モード時のみ**)
   S5U1C33000Hの実行カウンタの測定単位を、サイクル数、秒単位、μsec単位から選択できます。

4. **コマンド実行結果の表示行数**
   以下のコマンドの実行結果を[Command]ウィンドウに表示する場合の、1回の表示行数が1～1000行の範囲で選択できます。
   対象コマンド: db、dh、dw、sc、m、u、sy、sw、sa、sd、od、td、ts

5. cmw**コマンドウェイト時間**
   cmwコマンドのウェイト時間を1～256秒の範囲内(1秒単位)で設定します。

6. TAB**ストップ**
   ソース表示などに使用するTAB間隔を2、4または8文字に設定できます。

7. **オンザフライ情報の表示**(ICE/ICD**モード時のみ**)
   オンザフライ情報の表示間隔を0～10回/秒に設定できます。
   0回を指定すると、表示を行いません。

各モードのデフォルト値

モード	ICDモード	ICEモード	SIMモード	MONモード
ステップ表示	ON	ON	ON	ON
実行カウンタ	積算	積算	積算	積算
ICD実行カウンタ機能	–	サイクル数	–	–
表示行数	16行	16行	16行	16行
cmwウェイト時間	1秒	1秒	1秒	1秒
TABストップ	8	8	8	8
ICEオンザフライ機能	OFF	OFF	OFF(固定)	OFF(固定)

mdコマンドの設定内容はファイルに保存されるため、デバッガを次回起動したときにも今回終了時点の設定が有効になります。

■ 書　式

md　　(ガイダンスモード)

■ 入力例

現在の設定内容が表示され、その後ガイダンスに従って以下の操作が行えます。

```
>md↵
Step and memory window each display mode : on
Counter mode          : hold
Counter function      : cycle
Number of display line:   16 line
Cmw wait time         :    1 s
Tab stop              :    8
On the fly interval   : 0 times/sec

Step and memory window each display mode  1.on 2.off ...? on :1↵
Counter mode           1.reset    2.hold    ...? hold       :2↵
Counter function  1.cycle 2.time[us] 3.time[s] ? cycle      :2↵
Number of display line 1 - 1000 line        ...?   16 line   :16↵
Cmw wait time          1 - 256 s            ...?    1 s      :1↵
Tab Stop               2, 4, 8 size         ...?    8        :8↵
On the fly interval    0 - 10 times/sec     ...? 0 times/sec :2↵
>
```

これはICDモードの例で、他のモードでは"Counter function"の表示と選択はありません。
シミュレータおよびデバッグモニタモードでは"On the fly interval"の選択は行えません。

ガイダンス途中で[Enter]キーのみを入力した場合は前回の設定データが変更されません。
途中で終了する場合は[q]キーを入力し[Enter]キーを押してください。それまでに入力された内容は変更されます。
[^]キーによって1つ前のガイダンスに戻すこともできます。

■ 注　意

ICEモードのオンザフライ機能による実際の表示間隔は次のようになります。
(1[sec]/設定回数) + (PC, RS232C, S5U1C33104Hのオーバーヘッド[sec]) = 表示間隔[sec]

オーバーヘッドは、PCの性能、RS232Cのボーレートによりますが、0.05～0.1秒ほどありますので注意してください。

また、デバッガはブレークの発生、シミュレーテッドI/Oをすべてオンザフライの間隔でチェックします。これらのレスポンスを向上させるには、オンザフライ間隔を10あるいは0(OFF)にしてください。

alias (alias) [ICD / ICE / SIM / MON]

■ 機　能

コマンドラインのエイリアスを作成します。使用頻度の高い文字列を登録して使用することができます。

■ 書　式

(1) **alias** (ガイダンスモード)

(2) **alias <key> <string>** (直接入力モード)

<key>: 登録キー

<string>: 登録する文字列

■ 入力例

(1) **エイリアスの登録**

書式1では次のように登録します。

```
>alias↵
twrite                  ---> task_write 8      ...すでに登録されているキー
tread                   ---> task_read 1 3 5   ...すでに登録されているキー
Key ? :tdmp↵                                   ...登録するキーを入力
String ?:call task_dump 8↵                     ...登録文字列の入力
>
```

以降、コマンドラインで"@tdmp"と"@"を先頭に付けて入力すると、"call task_dump 8"と入力するのと同じ意味になります。

```
>@tdmp↵                ...実行するには先頭に"@"を付けて入力
call task_dump 8       ...登録されている文字列が表示、実行される
>
```

ガイダンスモードでは、すでに登録されているキーがある場合には、すべてのキーを表示した後、設定に入ります。

書式2では次のように登録します。

```
>alias tdmp call task_dump 8↵
>
```

(2) **エイリアスの再定義**

エイリアスを再定義する場合は、登録と同じ手順で文字列を入力します。前の文字列は入力した文字列に変更されます。

```
>alias↵
twrite                  ---> task_write 8
tread                   ---> task_read 1 3 5
tdmp                    ---> task_dump 8
Key ?:tdmp↵                                    ...再定義したいキーを入力
String ?:task_dump2↵                           ...新規文字列を入力
>alias↵
twrite                  ---> task_write 8
tread                   ---> task_read 1 3 5
tdmp                    ---> task_dump2        ...新しい文字列に変更されている
Key ?:↵                                        ...[Enter]キーのみで終了
>
```

書式2でも同じように登録文字列を変更できます。

```
>alias tdump task_dump2↵                       ...新しい文字列で再定義
>
```

(3) エイリアスのクリア

書式1では削除したいキーを入力し、文字列に[Enter]のみを入力します。

```
>alias↵
twrite                  ---> task_write 8
tread                   ---> task_read 1 3 5
tdmp                    ---> task_dump 8
Key ?:tdmp↵                              ...削除したいキーを入力
String ?:↵                               ...[Enter]のみ入力
>
```

書式2では削除するキーのみを入力します。

```
>alias tdump↵                            ...tdumpを削除
>
```

(4) 実行例

エイリアスを実行するには登録したキーを先頭に"@"を付けて入力します。

```
>alias tdmp call task_dump 1 3 5↵        ...tdmpの定義
>alias↵
tdmp            --->call task_dump 1 3 5
Key ? :twrite↵
String ? :call task_write 8↵             ...twriteの定義
>@twrite↵                                ...実行には"@"を付加
call task_write 8
>@tdump 7↵                               ...引数の追加入力も可能
call task_dump 1 3 5 7
```

■注　意

登録可能なエイリアスは最大100個です。100個を越えるとエラーになります。

Error: Cannot set alias anymore.

exit (exit) [ICD / ICE / SIM / MON]

■ 機　能

デバッガを終了します。
COMポート、パラレルポート、ログファイル、コマンドファイルが開いているときはクローズします。

■ 書　式

exit (直接入力モード)

※ db33は[File]メニュー/[Exit]コマンドの選択によっても同様に終了します。

? (help) [ICD / ICE / SIM / MON]

■機　能

各コマンドの入力書式を表示します。

■書　式

(1) **?** (直接入力モード)
(2) **? <n>** (直接入力モード)
(3) **? <command>** (直接入力モード)

<n>: コマンドグループ番号(10進数)
<command>: コマンド名
条件: 0≦n≦8

■表　示

書式1、書式2のコマンド入力によって機能別のコマンド一覧を表示します。
個別のコマンドの入力書式は書式3のコマンド入力によって表示させます。

書式1)

```
>?↵
group 0: key function ............... F1-F12 / ctrl+o,ctrl+v,ctrl+c / arrow key
 Please type "? 0" to show group 0.
group 1: memory ..................... fb,fh,fw / db,dh,dw,df / eb,eh,ew / mv,mvh,mvw / w / rm
group 2: execution & register ....... g,s,n,o,rstc,rsth / int / rd,rs / call,callh,callmd
group 3: break ...................... bp,bs,bc,bph,bh,bhc,bd,bsq,bl,bac,bh2,bhc2,ba,bb
group 4: source & symbol ............ u,sc,m / ss / sy,sa,sw,sd
group 5: file & flash memory ........ lf,lh / lo / ld, mc, fls,fle / lfl,sfl,efl,maf(only for
                                      ICE flash memory)
group 6: trace & simulated I/O ...... tm,tm2,td,ts,tf / stdin,stdout
group 7: others ..................... com,cmw,log / od,ct,ext / ma,md, / alias,q,exit,?
group 8: input number method ........ number,data,address,linenum,symbol
 Please type "? 1" to show group 1 or type "? fb" to get usage of command "fb".
>
```

書式2)

```
>? 1↵
group 1: memory
fb (fill byte), fh (fill half), fw (fill word),
db (dump byte), dh (dump half), dw (dump word),
eb (enter byte), eh (enter half), ew (enter word),
mv (move),mvh (move half),mvw (move word),df(dump file),
w (watch data),
rm (read memory)
 Please type "? fb" to get usage of command "fb".
>
```

書式3)

```
>? fb↵
fb (fill byte): fill memory with byte data
usage: >fb addr1 addr2 data   ... fill data from addr1 to addr2
       >fb                    ... fill memory with guidance
       Start address ? :addr1 ... input start address
       End   address ? :addr2 ... input end address
       Data  pattern ? :data  ... input data pattern
                                  (data:0x0-0xFF)
>
```

ice (ice) [ICE]

■ 機　能

指定したデータを直接S5U1C33104Hに送ります。
送信後、S5U1C33104Hよりの応答データを16進数で表示します。

■ 書　式

ice　　(ガイダンスモード)

■ 入力例

```
>ice↵
1.OK 2.CAN 3.Message ...? 1↵
Control code = 70
>

>ice↵
1.OK 2.CAN 3.Message ...? 2↵
Control code = 40
Error code   = 08
Error address= 00000000
>

>ice↵
1.OK 2.CAN 3.Message ...? 3↵
Message ID ? :7↵
Send data ---[CR] : Quit guidance ---q[CR]
Data0001   ? :00↵
Send data ---[CR] : Quit guidance ---q[CR]
Data0002   ? :c0↵
Send data ---[CR] : Quit guidance ---q[CR]
Data0003   ? :00↵
Send data ---[CR] : Quit guidance ---q[CR]
Data0004   ? :30↵
Send data ---[CR] : Quit guidance ---q[CR]
Data0005   ? :00↵
Send data ---[CR] : Quit guidance ---q[CR]
Data0006   ? :21↵
Send data ---[CR] : Quit guidance ---q[CR]
Data0007   ? :↵
Data size = 0022
ID        = 07
Data      = 90 1C 00 C0 FF D7 E5 1C FD 1E AA AA AA AA AA AA
            AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA AA
            AA
BCC       = D3
>
```

ガイダンス途中でコマンドを終了するには[q] [Enter]を入力してください。
また、[^] [Enter]でガイダンスを1つ手前に戻すこともできます。
送信データは0x00～0xffの16進数、または+0～+255の10進数で入力します。
送信可能なメッセージは最大8192バイトです。
[Enter]キーのみの入力またはData8192へのデータ入力により、db33はサイズとBCCを付加して入力データを送信します。

■ 注　意

- iceはセイコーエプソン内のS5U1C33104H開発用のコマンドです。一般のユーザーは使用しないでください。
- 本コマンドはパラレルデータ送信には対応していません。
- 本コマンドの実行は[Key break]ボタンによっては停止できません。タイムアウトは150秒です。

16.11 パラメータファイルについて

db33を起動するには、S5U1C33xxxPボード(S5U1C33104Hに装着する各機種個別の周辺回路ボード)やメモリマップの情報を記述したパラメータファイルが必要です。db33およびS5U1C33104Hは、このパラメータファイルに記述された内容でメモリマップを構築し、有効領域外のアドレス指定に対するエラー処理やマップブレークの処理を行います。S5U1C33001M1に関する情報を指定することで、それらを使うことが可能となります。
基本となるパラメータファイルはwb33の[Par gen]ボタンによって作成できます。テキスト形式のファイルですので、これにユーザーの仕様に合わせた外部メモリ領域の指定などを追加して使用してください。
wb33で作成されるパラメータファイル名は次の形式です。

<チップ名>_<パラメータファイルのバージョン> .par

例: 33104_01.par

以下にパラメータファイル例を示します。

パラメータファイル例

```
CHIP        33104                   ; chip name (33XXX)                 ...(1)
IROM        1000                    ; internal ROM is 80000 to 80FFF    ...(2)
FOPT        0000                    ; f option size                     ...(3)
PRC VER     00 ff                   ; allow any PRC board               ...(4)
PRC STATUS  ****************        ; allow any PRC board status...(5)
MPU                                 ; 0xC00000 external boot address    ...(6)
VER         1                       ; this file version                 ...(7)

; Emulation memory allocation (max 8 areas, 1MB/area, 1MB boundary)     ...(8)

EMRAM       400000  4fffff          ; emulation RAM 1MB
EMROM       c800000 cfffff          ; emulation ROM 1MB

; Map allocation (max 31 areas, 256bytes boundary)                      ...(9)

RAM         0       1FFF    00W     ; internal RAM area 8KB
IO          40000   4ffff   11H     ; internal IO area 64KB
ROM         400000  4fffff  77H     ; external ROM 1MB
RA          600000  6fffff  77H     ; external RAM 1MB
ROM         c00000  cfffff  77H  B  ; external ROM 1MB

; Stack area except internal RAM area (max 8 areas, 256bytes boundary)  ...(10)

STACK       0       3ff             ; internal stack area 1KB
STACK       600000  6fffff          ; external stack area 1MB

; MEM33 configuration                                                   ...(11)

;!MEM33_CE          9               ; MEM33 address(CE number)
;!MEM33_CEFUNC      00              ; CEFUNC value
;!MEM33_DELAY       2               ; MEM33 delay
;!MEM33_CE10EX      c00000          ; Area10 external memory start address
;!MEM33_P30                         ; P30 is CE
;!MEM33_P34                         ; P34 is CE
;!MEM33_CE7_DRAM                    ; Area7(13) is DRAM
;!MEM33_CE8_DRAM                    ; Area8(14) is DRAM
;!MEM33_WRH_MASK                    ; WRH mask
;!MEM33_MAP_DISABLE                 ; Mem33 map break disable
;!MEM33_CE_DISABLE                  ; Mem33 ce break disable
;!MEM33_NOCE_DISABLE                ; Mem33 no ce break disable

END                                                                     ...(12)
```

※ ";"からその行の終わりまではコメントとみなされます。
";!"はコメントとなりません。
パラメータの(2)〜(6)、(8)、(10)は、デバッガdb33をICEモードで起動するときにのみ必要です。
その他のモードでは削除可能です。

●内容説明

デバッグモードによって必要な情報は異なります。wb33で作成される基本となるパラメータファイルを参照してください。

(1) **チップ名**

チップ名を記述します。

CHIP <チップ名>

(2) **内蔵**ROM**サイズ**

内蔵ROMサイズを記述します。

IROM <サイズ>

サイズは16進数のみ有効です(0xは不要)。
アドレス0x80000からここで指定したサイズ分が内蔵ROM領域としてマッピングされます。0の場合はサイズ0となります。
0xC00000からの内蔵ROMの場合、ここは0として、エリア10のROMとしてマップしてください。

(3) **オプションサイズ**

機種により設定されるファンクションオプションのサイズ(最大16KB)を記述します。

FOPT <サイズ>

サイズは16進数のみ有効です(0xは不要)。

(4) S5U1C33xxxP**ボードバージョン**

開発機種に対応するS5U1C33xxxPボードのバージョン範囲を記述します。

PRC VER <バージョン(1)> <バージョン(2)>

バージョン(1)には数値の小さい方、バージョン(2)には高い方を8ビットの16進数で記述してください。1つのバージョンのみに対応する場合は、両方に同じ数値を記述します。
db33をICEモードで起動した際にはS5U1C33xxxPボードのバージョンがチェックされ、バージョン(1)からバージョン(2)の範囲外のS5U1C33xxxPボードが装着されている場合はワーニングとします。
各機種に対応するS5U1C33xxxPボードのバージョンについてはS5U1C33xxxPボードのマニュアルを参照してください。"PRC VER 00 ff"と指定することで、すべてのバージョンを許します。

(5) S5U1C33xxxP**ステータス**

ICEモードでの起動時にチェックするS5U1C33xxxPボードのステータスビットを指定します。

PRC STATUS <ビット15><ビット14> ...<ビット0>

16ビットのステータスを1、0または*で指定します。
1または0を指定した場合、起動時のチェックでS5U1C33xxxPボードの対応するステータスビットがそれと異なるときにワーニングとします。*を指定した場合は、そのビットをチェックしません。
ステータスビットの設定については、S5U1C33xxxPボードのマニュアルを参照してください。
16ビットすべてを*とすることで、すべてのS5U1C33xxxPステータスを許します。

(6) MCU/MPU**モード**

CPUをMCUモード(内蔵ROMの0x80000からブート)で起動させるか、MPUモード(外部ROMの0xc00000からブート)で起動させるか指定します。

・MCUモードの指定　**MCU**
・MPUモードの指定　**MPU**

この指定はシミュレータモードで有効です。

(7) **パラメータファイルのバージョン**

パラメータファイルのバージョンを記述します。

VER <バージョン>

0～ffの16進数で記述してください。ユーザのバージョン管理用に使用してください。

(8) **エミュレーションメモリの割り付け**

S5U1C33104Hでは1Mバイト単位のエミュレーションメモリを最大8領域まで使用することができます。
外部メモリ領域をこのエミュレーションメモリに割り当ててデバッグすることが可能で、エミュレーションメモリを使用する場合は、各領域をRAMとして使用するか、ROMとして使用するかをここで指定します。

・RAMとして使用する場合
EMRAM <開始アドレス> <終了アドレス> (複数の記述、あるいは省略も可)
・ROMとして使用する場合
EMROM <開始アドレス> <終了アドレス> (複数の記述、あるいは省略も可)

開始アドレスはX00000、終了アドレスはXfffff(X=1～ff)の1Mバイト単位で指定してください。
EMROMで指定した領域はリードオンリーとなり、プログラムによるデータの書き込みはできません。EMRAMで指定した領域はプログラムによるリード/ライトが可能となります。

この指定はICEモード時にのみ有効です。
エミュレーションメモリを使用しない場合(内蔵メモリのみのシステム、またはターゲットボードに搭載したメモリで評価する場合)、これらの指定は省略します。

(9) **メモリマップの設定**

使用するメモリ領域を指定します。ここで設定した情報が、マップブレークに利用されます。

エミュレーションメモリのマッピング(S5U1C33104H使用時)

(8)で宣言したエミュレーションメモリ(各1Mバイト)内で使用する領域を以下の形式で設定します。

・RAMとして使用する領域の設定
ERAM <開始アドレス> <終了アドレス> (複数の記述、あるいは省略も可)
・ROMとして使用する領域の設定
EROM <開始アドレス> <終了アドレス> (複数の記述、あるいは省略も可)
・I/O領域の設定
EIO <開始アドレス> <終了アドレス> (複数の記述、あるいは省略も可)

ERAMで指定した領域はリード/ライト可能領域となり、0xaaで初期化されます。
EROMで指定した領域はライトオンリー領域となり、0xffで初期化されます。この領域に書き込みを行うと、ブレークが発生します。
EIOで指定した領域はリード/ライト可能領域となり、0x00で初期化されます。

アドレス範囲は、(8)で設定したエミュレーションメモリ領域内に限られます。
開始アドレスは256バイト境界となるように設定し、終了アドレスも領域サイズが256バイト単位となるように設定してください。(8)で指定したメモリ属性(ROM/RAM)と異なるメモリマップを指定してもエラーとはなりません。

エミュレーションメモリ以外のマッピング

内蔵RAM、内蔵I/O、ターゲットボードに搭載するメモリやI/Oのマップを以下の形式で設定します。

・RAM領域の設定(リード/ライト可能領域)
RAM <開始アドレス> <終了アドレス> (複数の記述、あるいは省略も可)
・ROM領域の設定(ライトオンリー領域)
ROM <開始アドレス> <終了アドレス> (複数の記述、あるいは省略も可)
・I/O領域の設定(リード/ライト可能領域)
IO <開始アドレス> <終了アドレス> (複数の記述、あるいは省略も可)

内蔵ROMについては、IROMの設定でマッピングされますので、ROMによる設定は不要です。
開始アドレスは256バイト境界となるように設定し、終了アドレスも領域サイズが256バイト単位となるように設定してください。

EMRAM、EMROMで指定したエミュレーションメモリの使用領域とアドレスが重複した場合は、RAM、ROM、IOの設定が有効となります。その領域へのアクセスはターゲットボードに対して行われ、エミュレーションメモリは使用されません。これにより、1Mバイトのエミュレーションメモリ領域の中で、エミュレーションメモリを使用する領域と使用しない領域を細かく指定することができます。

マップは、RAM、ROM、IO、ERAM、EROM、EIOの合計で最大31個まで設定可能です。

ウェイト数の指定

db33のシミュレータモードでは、アクセスウェイト数を記述する必要があります。
ここで指定した条件でCPUクロック数を計算していますので、設定に誤りがあると実行カウンタならびにトレース情報のクロック数が正しく表示されません。
ウェイト数の記述内容は、たとえば77Hの場合、最初の7がリードのウェイト数、次の7がライトのウェイト数、最後のHがデバイスサイズです。
ウェイト数は0～f(15)まで指定可能です。
ライトのウェイト数は省略可能です。その場合のデフォルトウェイト数は[リードのウェイト数+1]となります。
デバイスサイズは、"B"が8ビット、"H"が16ビット、"W"が32ビットです。デバイスサイズは、大文字で記述してください。
IROMで指定されているエリアは、77Hで計算します。指定なき場合のデフォルト値は77Hです。
旧バージョンのデバッガでは、これらの指定があるとエラーになります。

ビッグエンディアンの指定

マップの指定を行った各領域は、デフォルトでリトルエンディアン領域となります。ビッグエンディアンに設定したい領域については、<終了アドレス>の後ろに"B"を記述します(wb33で作成する場合は[Big]を選択)。ただし、開発するS1C33チップがビッグエンディアンに対応した機種である必要があります。また、内蔵ROM、RAM、I/Oはビッグエンディアンに設定することはできません。
db33起動時にこのパラメータファイルを読み込むとともに、S1C33チップ上のエンディアンコントロールレジスタ(テクニカルマニュアル参照)も正しく設定してください。
シミュレータモードの場合はパラメータファイルのみでエンディアンが決定します。
この設定は、ハーフワードまたはワード単位のメモリ操作やファイルのロードに影響を与えます。

(10) スタック領域の設定

スタックとして使用する領域を指定します。
STACK <開始アドレス> <終了アドレス> (複数の記述、あるいは省略も可)

最大8個所のスタック領域を指定することができます。
開始アドレスは256バイト境界となるように設定し、終了アドレスも領域サイズが256バイト単位となるように設定してください。

この設定はICEモード時に有効で、指定領域外へのスタック操作が行われた場合にブレークを発生します。
ただし、0番地から始まる内蔵RAMに対して行われるスタック操作はブレークの対象とはなりません。したがって、内蔵RAMに対するSTACK設定は省略可能です。
この設定は、プログラムによるSPの操作に影響を与えるものではありません。

(11)S5U1C33001M1を用いたデバッグ

S5U1C33001M1を使用する場合は以下の設定が必要です。S5U1C33001M1は、S5U1C33000Hと組み合わせて使用します。

1) S5U1C33xxxMボードのアドレス設定
例) ;!MEM33_CE 4 ; S5U1C33xxxMボードをCE4に設定
必ず設定してください。MEM33_CEは4～10で指定します。
(デバッガ内ではCEFUNCを00として、S5U1C33xxxMの設定をしています。)

2) CEFUNCの設定
例) ;!MEM33_CEFUNC 00 ;ユーザアプリケーションではCEFUNCを00固定で使用
必ず設定してください。MEM33_MAP_CEFUNCは、00、01、10、11で指定します。
CEFUNCの指定値でアクセスできないエリアをマップした場合、ワーニングを表示します。

3) エリア10の設定
例) ;!MEM33_CE10EX c08000 ;エリア10の外部アクセスエリア先頭番地を指定
指定のない場合(デフォルト)はc00000番地に設定されます。
この例では0xc00000～0xc07fffは内蔵ROMとなります。

4) DRAMエリア設定
例1) ;!MEM33_CE7_DRAM ;エリア7または13がDRAMのとき記述
例2) ;!MEM33_CE8_DRAM ;エリア8または14がDRAMのとき記述
指定のない場合(デフォルト)はSRAMに設定されます。

5) P30、P34をCEとして使う場合の設定
例1) ;!MEM33_P30 ;P30をCEとして使用
例2) ;!MEM33_P34 ;P34をCEとして使用
指定のない場合(デフォルト)はCEとしては使用しません。

6) メモリアクセスのディレイ設定
例) ;!MEM33_DELAY 0 ;ディレイを0に設定
0～3で指定します。
指定のない場合(デフォルト)は2に設定されます。通常は2以外にしないでください。

7) WRH信号の設定
例) ;!MEM33_WRH_MASK ;WRH信号を無視する
指定のない場合(デフォルト)はWRHによるブレークが有効となります。x16のSRAMでBSH、BSL方式を使用する場合、有効にしてください。

8) マップブレーク無効の設定
例) ;!MEM33_MAP_DISABLE ;マップブレークを無効にする
指定のない場合(デフォルト)はマップブレークが有効となります。

9) CEブレーク無効の設定
例) ;!MEM33_CE_DISABLE ;CEブレークを無効にする。
指定のない場合(デフォルト)はCEブレークが有効となります。

10) NO CEブレーク無効の設定
例) ;!MEM33_NOCE_DISABLE ;NO CEブレークを無効にする。
CEブレークとマップブレークに用意されているNO CEブレークを無効にします。
指定のない場合(デフォルト)はNO CEブレークが有効となります。
NO CEブレークとは、CEがアクティブにならない状態でRD、WRアクセスがあった場合に発生するブレークです。

(12)エンドマーク

パラメータファイルの最後には、必ずENDを記述してください。

●パラメータファイル作成上の注意

・各設定項目は、(1)～(12)の順に記述してください。

・ICEモードでは項目の(1)～(6)および(12)は省略できません。
その他のモードでは(1)および(12)を省略できません。

・項目名(CHIP、IROM等)は大文字で記述してください。

・各項目は行の先頭から記述し、パラメータ間には1個以上のスペースまたはタブを挿入してください。各項目のパラメータは省略できません。

・各メモリマップの指定は256バイト単位、エミュレーションメモリの設定は1Mバイト単位で行う必要があります。これ以外の境界アドレスが指定されているとワーニングとなります。

・S5U1C33001M1を使用する場合、メモリマップは32Kバイト単位で記述してください。

●wb33が作成するパラメータファイル

wb33の[Par gen]ボタンで作成したパラメータファイルを使用する場合、特に以下の点に注意してカスタマイズしてください。

- **オプションサイズ**(FOPT)
  オプションサイズは0に設定されます。ファンクションオプションを持つ機種を開発する場合は、サイズを設定してください。

- S5U1C33xxxP**ボードバージョン**(PRC VER)
  すべてのS5U1C33xxxPボードが使用可能な"00 ff"に設定されます。複数のS5U1C33xxxPボードをお持ちの場合は、バージョンを限定するように修正してください。

- **内部**I/O**領域**(RAM、IO)
  IOによる内部I/O領域の設定は0x40000～0x4ffffの64Kバイトとなります。開発する機種のI/O領域に合わせて修正してください。

- **エミュレーションメモリの割り当てと外部メモリ、外部**I/O**のマッピング**
  [Parameter file generator]ウィンドウで外部メモリ領域を選択した場合、次のような設定となります。

  [Emu]**ボタン選択時**

  [RAM]**ボタンを選択**: エミュレーションメモリとしてEMRAMが指定され、その全領域をRAMとして使用するようにERAMが設定されます。

  [ROM]**ボタンを選択**: エミュレーションメモリとしてEMROMが指定され、その全領域をROMとして使用するようにEROMが設定されます。

  [IO]**ボタンを選択**: エミュレーションメモリとしてEMRAMが指定され、その全領域がEIOとして設定されます。

  [Emu]**ボタン非選択時**

  [RAM]**ボタンを選択**: 全領域をターゲットのRAMとしてアクセスするようにRAMが設定されます。

  [ROM]**ボタンを選択**: 全領域をターゲットのROMとしてアクセスするようにROMが設定されます。

  [IO]**ボタンを選択**: 全領域をターゲットのI/O(RAMと同様)としてアクセスするようにIOが設定されます。

  マップは開発するシステムに合わせて修正、追加してください。

- **スタック領域(スタック領域)**
  [RAM]ボタンを選択して外部メモリを指定した場合に限り、指定の開始アドレスから512KB分のスタック領域が設定されます。
  必要に応じて修正してください。

- S5U1C33001M1**の設定**
  [Generate MEM33 information]チェックボックスをチェックした場合、作成されます。

16.12 INIファイル

デバッガは終了時に、各種情報を"db33.ini"という名前のテキストファイルに保存します。このファイルは"db33.exe"のあるディレクトリに作成されます。
ファイルに保存される内容は以下のとおりです。

ファンクションキー情報

[FUNCTION KEY]セクションに、F1～F12のファンクションキー定義が記述されています。
このセクションはユーザによるカスタマイズが可能で、各キーへの機能割り当てを自由に変更することができます。

ウィンドウ情報

[WINDOW]セクションには、ウィンドウ情報の内、各ウィンドウの位置と大きさが次回起動時に再現されるよう保存されます。ただし、複数オープンできるウィンドウ([Source]ウィンドウ、[Memory]ウィンドウ)は最初のウィンドウの情報のみ保存されます。

mdコマンド情報

[md]セクションには、mdコマンドで設定変更した情報が保存されます。デバッガを次に起動したときにも前回の設定で使用することができます。
mdコマンドについては"16.10 コマンドリファレンス"を参照してください。

注: **ファンクションキー以外の項目は、絶対に書き換えないでください。書き換えた場合の動作は保証されません。**

●ファンクションキー定義

ファンクションキーはデフォルトで以下のように定義されています。

INIファイルの例(デフォルト)

```
; db33 initialize file  - Debugger 33  Ver x.xx
;
[FUNCTION KEY]
F1=CR        ...コールドリセット
F2=HR        ...ホットリセット
F3=HIST      ...コマンドヒストリ
F4=GOTO      ...カーソル行まで実行
F5=GO        ...実行
F6=STP       ...ステップ実行
F7=NXT       ...ステップ＆スキップ
F8=SS        ...シンボルサーチ
F9=LF        ...ロードSRFファイル
F10=SRC      ...ソース表示
F11=MIX      ...ミックス表示
F12=UA       ...逆アッセンブル表示
;
; Don't change under from this line!!     ...これ以降の行は書き換え禁止
;
[WINDOW]
MAIN=MAX
     :
     :
```

各キーは、Fx=<予約語>の形式で定義されています。定義されている機能を変更するには、<予約語>を書き換えてください。割り当て可能な機能と<予約語>の一覧を表16.12.1に示します。書き換える場合、必ず表に記載されている予約語のみを使用してください。
また、各キーの定義は行の左端より左詰めで記述してください。

表16.12.1　ファンクションキー定義用予約語一覧

予約語		機　能	コマンド	ボタン
KB	(Key Break)	キーブレーク	–	
SB	(Soft Break)	ソフトブレークポイント設定	–	
HB	(Hard Break)	ハードブレークポイント設定	–	
HIST	(History)	コマンドヒストリ	–	–
GO	(Go)	実行	g	
GOTO	(Go To)	カーソル行まで実行	–	
SS	(Symbol Search)	シンボルのサーチ	–	–
SW	(Symbol Watch)	シンボルの表示	sw	
SA	(Symbol Add)	シンボルの登録	sa	
SD	(Symbol Delete)	シンボルの削除	sd	
HR	(Hot Reset)	ホットリセット	rsth	
CR	(Cold Reset)	コールドリセット	rstc	
NXT	(Next)	ステップ＆スキップ	n	
STP	(Step)	ステップ実行	s	
STO	(Step Out)	ステップアウト	o	–
LF	(Load File)	srf33ファイルのロード	lf	
DT	(Display Trace)	ICDオンチップトレース表示	–	
RT	(Resum Trace)	ICDオンチップトレース再開	–	
SRC	(Sorce Display)	ソース表示	sc	
MIX	(Mix Display)	ミックス表示	m	
UA	(UnAssemble Display)	逆アッセンブル表示	u	

各機能は、表中にあるコマンドあるいはボタンに割り当てられている機能と同じです。
機能の詳細は、各コマンドあるいはボタンの機能を参照してください。

●設定の初期化

ファンクションキー定義およびウィンドウの表示位置とサイズを初期状態に戻したい場合は、"db33.exe"と同じディレクトリに作成されている"db33.ini"を削除するか、リネームしてください。
ウィンドウ情報等を初期化せずにmdコマンドの設定情報のみを変更したい場合は、mdコマンドを実行して変更してください。
INIファイルを直接書き換えてウィンドウ情報とmdコマンド情報を変更することは禁止します。

16.13 ステータス/エラー/ワーニングメッセージ

デバッガは以下のメッセージをすべて[Command]ウィンドウに表示します。

16.13.1 ステータスメッセージ

ターゲットプログラムがブレークすると、コマンド入力待ちになる直前に以下のブレーク要因を示すメッセージを表示します。

表16.13.1.1 ステータスメッセージ

メッセージ	内　　容
Break by software PC break.	ソフトウェアPCブレークポイントによりブレーク
Break by hardware PC break.	ハードウェアPCブレークポイント1によりブレーク
Break by hardware PC break 2.	ハードウェアPCブレークポイント2によりブレーク
Break by temporary break.	gコマンドのテンポラリブレークポイントによりブレーク
Break by data break.	データブレーク条件によりブレーク
Break by read memory.	メモリリードによるデータブレーク
Break by write memory.	メモリライトによるデータブレーク
Break by sequential break.	シーケンシャルブレーク条件によりブレーク
Break by key break, xxxx.	[Key break]ボタンによりアドレスxxxxでブレーク
Break by accessing no map area.	マップされていない領域へのアクセスによりブレーク
Break by writing ROM area.	リードオンリー領域への書き込みによりブレーク
Break by out of SP area.	スタック領域外に対するスタック操作によりブレーク
Break by external break.	S5U1C33104H/S5U1C33000HのBRKIN端子への信号入力によりブレーク
Break by illegal instruction.	シミュレータモードで不当命令の実行によりブレーク
Break by MEM33 bus break.	S5U1C33xxxMのバスブレーク条件によりブレーク
Break by MEM33 ce break.	S5U1C33xxxMのCEブレーク条件によりブレーク
Break by MEM33 area break.	S5U1C33xxxMのエリアブレーク条件によりブレーク
Break by MEM33 map break.	S5U1C33xxxMのマップブレーク条件によりブレーク

16.13.2 エラーメッセージ

表16.13.2.1 エラーメッセージ

(アルファベット順)

エラーメッセージ	内　　容
Address is in no map area.	アドレス(シンボル)がマップ領域外です。
Address is not 2 byte boundary.	プログラムコードのアドレスが2バイト境界ではありません。
Address mask range (0-0xFFFFFF).	アドレスマスクが有効範囲外です。
Address range (0-0xFFFFFFF).	入力アドレスは有効範囲外です。
address1 > address2.	開始アドレスが終了アドレスを越えています。
Already exist input address.	設定済みのブレークアドレスを指定しました。
Argument1 range (0-0xFFFFFFF).	引数1が有効範囲外です。
Argument2 range (0-0xFFFFFFF).	引数2が有効範囲外です。
Argument3 range (0-0xFFFFFFF).	引数3が有効範囲外です。
Break number (1-16).	ソフトウェアPCブレークポイント番号が有効範囲外です。
Break number (1-8).	ハードウェアPCブレークポイント番号が有効範囲外です。
Cannot add symbol any more.	すでに99個のシンボルが登録済みです。
Cannot allocate memory.	メモリの割り付けができません。
Cannot close file.	ファイルがクローズできません。
Cannot compare data, file open failure, xxx.srf.	xxx.srfがオープンできません。
Cannot get file status.	ファイル情報が不正です。
Cannot get input, please check the system.	入力処理でエラーが発生しました。
Cannot get memory.	メモリ確保に失敗しました。
Cannot load data, file open failure srf33.	ファイルのロードに失敗しました。ファイルがオープンできません。
Cannot load data, file read failure srf33.	ファイルのロードに失敗しました。srf33ファイルが読み込めません。
Cannot load data, memory allocation failure srf33.	ファイルのロードに失敗しました。メモリが確保できません。
Cannot load data, please check SRF33 file srf33.	ファイルのロードに失敗しました。srf33実行形式以外のファイルが指定されました。
Cannot load debug information, debug information is wrong.	デバッグ情報のロードに失敗しました。デバッグ情報が不正です。 (プログラム/データの読み込みは成功)
Cannot load debug information, file open failure.	デバッグ情報のロードに失敗しました。ソースファイルがオープンできません。(プログラム/データの読み込みは成功)
Cannot load debug information, file read failure.	デバッグ情報のロードに失敗しました。ソースファイルが読み込めません。(プログラム/データの読み込みは成功)
Cannot load debug information, memory allocation failure.	デバッグ情報のロードに失敗しました。メモリが確保できません。 (プログラム/データの読み込みは成功)

エラーメッセージ	内　　容
Cannot load debug information, please check SRF33 file.	デバッグ情報のロードに失敗しました。srf33形式が不正です。 (プログラム/データの読み込みは成功)
Cannot load debug information, too many lines.	デバッグ情報のロードに失敗しました。ソースファイルの行数が多すぎます。 (プログラム/データの読み込みは成功)
Cannot open com port.	パソコンのCOMポートが開けません。またはボーレートの設定ができません。
Cannot open file any more.	これ以上ファイルをオープンできません。
Cannot open file.	ファイルをオープンできません。
Cannot open parallel port.	パソコンのパラレルポートが開けません。
Cannot open parameter file.	パラメータファイルがオープンできません。
Cannot open stdio file.	stdioのファイルがオープンできません。
Cannot read chip name.	パラメータファイルのチップ名情報が読み込めません。
Cannot read emulation memory map.	パラメータファイルのエミュレーションメモリマップ情報が読み込めません。
Cannot read FO area size.	パラメータファイルのFOPTサイズ情報が読み込めません。
Cannot read IROM size.	パラメータファイルの内蔵ROM領域サイズ情報が読み込めません。
Cannot read MCU/MPU information.	パラメータファイルのMCU/MPUモード情報が読み込めません。
Cannot read parameter file version.	パラメータファイルのバージョン情報が読み込めません。
Cannot read PRC board status.	パラメータファイルのS5U1C33xxxPボードステータス情報が読み込めません。
Cannot read PRC board version.	パラメータファイルのS5U1C33xxxPボードバージョン情報が読み込めません。
Cannot read stdin file.	stdinのファイルが読み込めません。
Cannot resist address any more.	すでに8個所のハードウェアPCブレークが登録されています。
Cannot set address any more.	すでに16個所のソフトウェアPCブレークポイントが設定されています。
Cannot set alias anymore.	すでに100件のエイリアスが登録されています。
Cannot set enable address anymore.	すでに2個所のハードウェアPCブレークが設定されています。
Cannot set hardware PC break.	ハードウェアPCブレークポイントが設定できません。
Cannot set software PC break.	ソフトウェアPCブレークポイントが設定できません。
Cannot set temporary break.	テンポラリブレークポイントが設定できません。
Cannot write flash memory.	フラッシュメモリに書き込めません。
Cannot write log file.	ログファイルに書き込めません。
Cannot write stdout file.	stdoutのファイルにデータが書き込めません。
Cannot write trace file.	トレースファイルに書き込めません。
Chip name should be 5 characters.	チップ名が5文字ではありません。
Chip name should be start with "33".	チップ名が"33"で始まっていません。
Command is not supported at present mode.	現モードで未対応のコマンドを実行しました。
Communication data size error.	通信データのサイズが不正です。
Communication error.	S5U1C33104Hとの通信中にオーバーラン、フレーミング、またはBCCエラーが発生しました。
CPU down.	S5U1C33xxxPボードの動作不良です。
CPU is not running.	S5U1C33104HのCPUが動作していません。
CPU is running.	S5U1C33104HのCPUが実行中です。
Current mode is not source mode.	ソース表示モード以外で文字列の検索はできません。
Data alignment error.	srf33ファイルのアライメントが不正です。
Data incomplete.	ファイルの構造が不正です。
Data mask range (0-0xFFFF).	データマスクが有効範囲外です。
Data mask range (0-0xFFFFFFFF).	データマスクが有効範囲外です。
Data range (0-0xFF).	入力データは有効範囲外です。
Data range (0-0xFFFF).	入力データは有効範囲外です。
Data range (0-0xFFFFFFFF).	入力データは有効範囲外です。
Debug data failure.	デバッグデータが異常です。
Diagnostic test failure.	S5U1C33104Hの自己診断でエラーが発生しました。
Duplicate input address.	ブレークアドレスを重複して指定しました。
Duplicate input break number.	ブレークポイント番号を重複して指定しました。
Empty file.	ファイルにデータがありません。
Erase routine is not set.	フラッシュメモリ消去ルーチンが設定されていません。
File end during guidance input.	コマンドファイル内のガイダンスの記述が途中で終了しています。
File not found.	指定ファイルが見つかりません。
Flash memory error.	フラッシュメモリへの書き込みまたは消去ができません。
Flash memory is not mapped.	フラッシュメモリがマップされていません。
Flash ROM is protected.	フラッシュメモリにプロテクトがかかっています。
FO address range (0-0x3FFF).	オプションダンプアドレスが有効範囲外です。
Format error.	フォーマットが異常です。
Function not found.	関数が見つかりません。
ICE is busy.	S5U1C33104Hが処理中です。
ICE is free run mode.	S5U1C33104Hがフリーランモードです。
ICE is maintenance mode.	S5U1C33104Hが保守モードです。
ICE is not mapped.	S5U1C33104Hがマップされていません。

エラーメッセージ	内　容
ICE system error.	S5U1C33104Hのシステムエラーです。
Interrupt level (0-15).	割り込みレベルが有効範囲外です。
Interrupt type (0-215).	割り込みタイプが有効範囲外です。
Invalid address or invalid CEFUNC of parameter file.	不正なアドレス、またはCEFUNCで指定したアドレスの有効範囲外です。
Invalid break address.	未設定のブレークアドレスを指定しました。
Invalid break number.	未設定のブレークポイント番号を指定しました。
Invalid command or parameter.	コマンドまたはパラメータが不正です。
Invalid emulation memory map.	パラメータファイルのエミュレーションメモリマップ情報が不正です。
Invalid file name.	ファイル名が不正です。
Invalid group number.	コマンドグループ番号が不正です。
Invalid map command or invalid sequence.	マップファイルに不正な文字がある、または順序が不適当です。
Invalid memory map.	パラメータファイルのメモリマップ情報が不正です。
Invalid parameter.	パラメータが不正です。
Invalid stack map.	パラメータファイルのスタックマップ情報が不正です。
Invalid value of MEM33_CE10EX (C00000-FFFFFF).	エリア10外部メモリスタートアドレスが有効範囲外です。
Invalid value of MEM33_CEFUNC (00,01,10,11).	CEFUNC値が有効範囲外です。
Invalid value of MEM33_DELAY (0-3).	ディレイ値が有効範囲外です。
Invalid value (0-65535).	有効範囲外の値です。
Invalid value.	入力数値が不正です。
IROM size is too long.	IROMサイズが大きすぎます。
MEM33 is not supported at area0, area1 and area2 (0-0x7FFFF).	S5U1C33xxxMは、エリア0, 1, 2をサポートしていません。
Memory verify failure.	メモリとの比較でエラーになりました。
No "END" in parameter file.	パラメータファイルにエンドマーク(END)がありません。
No "MEM33_CEFUNC" in parameter file.	MEM33_CEFUNCの指定がありません。
No map area.	入力アドレスはマップ領域外です。
No symbol at the number.	未登録のシンボル番号を指定しました。
Not ASCII character.	文字列の指定にASCIIキャラクタ以外の文字が含まれています。
Not defined ID.	S5U1C33104Hの応答IDが無効です。
Not found input strings.	入力文字列が見つかりません。
Number of emulation memory is wrong.	パラメータファイルのエミュレーションメモリ数が不正です。
Number of parameter.	パラメータの数が不正です。
On tracing.	S5U1C33104Hが実行データをトレース中です。
Parallel interface time out.	パラレルポートによるファイルのロード中にタイムアウトとなりました。
Parallel port is busy.	パラレルポートが使用中です。
Pointer pointed no map area.	ポインタ変数がマップ領域外を指しています。
Post line range (0-256).	トレース検索結果の後データ表示行数が有効範囲外です。
Pre line range (0-256).	トレース検索結果の前データ表示行数が有効範囲外です。
Register variable cannot be changed to address.	レジスタ変数シンボルでアドレスを指定しました。
Reset timeout.	S5U1C33104HのCPUがリセットできません。
Sequential break format error.	S5U1C33104Hに設定するシーケンシャルブレーク条件が不正です。
Shared RAM is busy.	S5U1C33104Hの内部エラーです。
Source window not opened.	[Source]ウィンドウが閉じています。
Start block > End block.	開始ブロック番号が終了ブロック番号を越えています。
Start cycle number > End cycle number.	開始サイクル番号が終了サイクル番号を越えています。
Stdout data size.	出力バッファのデータサイズが不正です。
Step range (1-65535).	シングルステップ数が有効範囲外です。
Symbol is too long.	シンボル名が長すぎます。
Symbol not found, xxx.	シンボルxxxが見つかりません。
Symbol not found.	シンボルが見つかりません。
Symbol not in scope.	シンボルがスコープ内にありません。
Target down.	S5U1C33xxxPボードが正常に動作していません。またはリセットが解除されません。
Target instruction cannot be extended.	EXTで拡張できない命令です。
Time out.	S5U1C33104Hとの通信中にタイムアウトが発生しました。*1
Too many include file.	インクルードファイルの数が制限を越えました。
Too many source file.	ソースファイルが長すぎます。
Top address > Bottom address.	先頭アドレスが終了アドレスを越えています。
Top address and bottom address should be same CE.	先頭アドレスと終了アドレスは同一エリアでありません。
Trace range (0-131071).	トレースサイクル番号が無効です。(ICDモード)
Trace range (0-32767).	トレースサイクル番号が無効です。(ICEモード)
Verify error.	フラッシュメモリ書き込み時にベリファイエラーが発生しました。
Wrong data.	ファイルのデータが不正です。
Wrong header.	ファイルのヘッダが不正です。

*1: タイムアウトエラーは以下の処理でS5U1C33104Hから一定時間応答がない場合に発生します。

・初期接続　2.5秒　・f、mv実行時　150秒　・その他 6秒
・Escape Break　1秒　・lfl、sfl実行時　150秒
・マップの初期化　150秒　・g、s、n実行時　タイムアウトしません。

16.13.3 ワーニングメッセージ

表16.13.3.1 ワーニングメッセージ

(アルファベット順)

ワーニングメッセージ	内　　容
Area10 external memory start address is not 32k byte boundary (XXX).	エリア10外部メモリスタートアドレスが32Kバイトバウンダリではありません。
Cannot set CE break (XXX-XXX).	CEブレークをS5U1C33xxxMに設定できませんでした。
Cannot set map break (XXX-XXX).	MAPブレークをS5U1C33xxxMに設定できませんでした。
Debugger mode does not match with a target.	起動オプション(-icd、-ice、-mon)で指定したモードと接続されているデバッグターゲットが異なります。
Duplicate MEM33 address (CE) with memory map (XXX-XXX).	S5U1C33xxxMのアドレスとメモリマップが重複しています。
Duplicate memory map attribute (CE break as RAM XXX-XXX).	同一エリアにROM, RAM, IOが混在して います。
Emulation memory address is not 1M byte boundary.	パラメータファイルのエミュレーションメモリ情報で1Mバイト境界以外のアドレスが指定されています。
FO size (0-0x4000), map as 0x4000.	FOサイズ情報が不正なため、0x4000としてマップしました。
FO size should be an even number, map as 0xXXXXXXXX.	FOサイズが奇数のため、0xXXXXXXXXとしてマップしました。
Invalid line, move to next valid line.	指定の行番号にはアドレスがありません。直後の有効アドレスを使用します。
IROM size (0-0x80000), map as 0x80000.	IROMサイズ情報が不正なため、0x80000としてマップしました。
Line number of source file is invalid.	ソースファイルに指定の行番号はありません。
MEM33 break information was over written by user program.	S5U1C33xxxMにユーザプログラムが書き込みをし、ブレーク情報を破壊した可能性があります。
MEM33 not found.	パラメータファイルで指定されたアドレスにS5U1C33xxxMがありません。
Memory map is not 256 byte boundary.	パラメータファイルのメモリマップ情報で256バイト境界以外のアドレスが指定されています。
Memory map is not 32k byte boundary (XXX-XXX).	メモリマップが32Kバイト単位ではありません。
No debug information.	srf33ファイルにデバッグ情報がありません。
No source, display on mix mode.	ソース情報がありません。プログラムはミックス表示されます。
Not found "Return Code".	プログラム中にリターンコードが見つかりません。
Number of source line exceeded 65535.	行番号が有効範囲外です。
PRC status does not match.	S5U1C33xxxPボードのステータスがパラメータファイルの指定と異なります。
PRC version does not match.	S5U1C33xxxPボードのバージョンがパラメータファイルの指定と異なります。
Round down to multiple of 16.	入力アドレスは16バイト境界にアライメントされました。
Round down to multiple of 2.	入力アドレスは2バイト境界にアライメントされました。
Round down to multiple of 4.	入力アドレスは4バイト境界にアライメントされました。
Stack map is not 256 byte boundary.	パラメータファイルのスタックマップ情報で256バイト境界以外のアドレスが指定されています。

17　その他のツール

この章では本パッケージに含まれるその他のツールについて説明します。

17.1　make

本パッケージはコンパイルからリンクまでを効率よく処理するメイクツール(以下makeと記述)を含んでいます。
makeは、makeファイルに記述されたソースや各ツールが出力するファイルの依存関係を元に必要なツールを実行し、ファイルを最新の状態に更新します。たとえば、1つのソースファイルのみを修正した場合、makeはそのファイルのみを対象にコンパイルあるいはプリプロセスからアセンブルまでの処理を実行します。他のモジュールはリンク時にオブジェクトファイルが読み込まれるのみで、アセンブルまでの処理は行いません。
本パッケージのmakeは、上記の処理を行うための依存リスト、サフィックス定義およびマクロ定義のみをサポートしています。
UNIXのmakeのサブセット機能を提供します。

17.1.1　起動方法

●起動フォーマット

make ^ [<オプション>] ^ [<ターゲット名>]

^はスペースを示します。
[ ]は省略可能なことを示します。

例: c:¥cc33¥make -f test.mak opt

ワークベンチからの起動
起動オプションおよびmakeファイル(.mak)を指定し、[MAKE]ボタンをクリックします。

●オプション
makeには3種類の起動時オプションが用意されています。

-f <ファイル名>
機能:　makeファイルの指定
wb33上の指定:　常時指定(ファイル名はリストボックスより選択)
説明:　makeは<ファイル名>(拡張子を含む)で指定されたmakeファイルを読み込み、その内容を処理します。
デフォルト:　-fオプションが指定されない場合は、"makefile"の名称を持つファイルをmakeファイルとして入力します。

-h
機能:　usage出力
wb33上の指定:　[usage]をチェック
説明:　標準出力(stdout)に使用方法のメッセージを出力して終了します。

-n
機能:　コマンド表示
wb33上の指定:　[no exe cmd]をチェック
説明:　make対象のコマンドを標準出力(stdout)に出力するだけで、実際には実行しません。ファイルの依存関係を確認する場合などに有効です。

●ターゲット名

実行させるコマンドのターゲット名を指定します。省略した場合は、makeファイル内で最初に現れるターゲットを実行します。

wb33の[Make file editor]ウィンドウで作成するmakeファイルには、ファイルの更新を行う依存リスト以外に2つ機能を実現するコマンドとターゲット名(opt、clean)が記録されています。
opt: 2パスmake用のコマンドを実行するターゲット名
clean: ソース以外のファイルを消去するコマンドのターゲット名
wb33からこれらのターゲットを実行するには、makeファイルを選択後、次のように操作してください。
optを実行するには: [Other option]ウィンドウ上で[2 pass]を選択し、[MAKE]ボタンをクリックします。
cleanを実行するには: [MAKE clean]ボタンをクリックします。

17.1.2 メッセージ

makeはメッセージをすべて標準出力(stdout)に対して出力します。
wb33の[MAKE]ボタンでmakeを起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

●実行メッセージ

実行中は、実行コマンドを出力し、最後に終了メッセージを出力します。
例:

```
C:\CC33\pp33 -g  sample1.s
Pre Processor Completed
C:\CC33\ext33 -gp 0x0  sample1.ps
Extend Completed
C:\CC33\as33 -g  sample1.ms
Assembly Completed
C:\CC33\lk33 -g -s -m -c  sample.cm
Link Completed
Make Completed
```

※ wb33による実行時はccapを使用してメッセージをファイル"wb33.err"に保存するとともに、各ツールで発生したエラー/ワーニングおよびコンパイラメッセージのカウントも行います。カウントした値はmakeの終了メッセージの後に表示されます。

```
Make Completed
0 error(s), 0 warning(s), 0 compiler message(s)
```

●Usage出力

ファイル名が指定されなかったり、オプション指定が正しくない場合、次の使用方法のメッセージを出力して終了します。

```
Make for 33/63 Ver x.x
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    make [options] [target]
Options:
    -f <file name> : makefile name
    -h  : output usage
    -n  : no command execution
Example:
    make -f test.mak
    make -f test.mak CLEAN
```

●エラー/ワーニング発生時

エラー/ワーニングが発生した場合は、終了メッセージの前にエラーメッセージが表示されます。
例:

```
Warning: sample.srf is up-to-date
Make Completed
```

エラーの詳細は"17.1.6 エラー/ワーニングメッセージ"を参照してください。

17.1.3　makeファイル

makeファイルはファイルの依存関係と実行するコマンドを記述したテキストファイルです。
wb33の[Make file editor]ウィンドウで基本的なmakeファイルを作成することができますので、必要に応じて追加や修正を行って使用してください。
makeファイルの例を次に示します。

makeファイル例(サフィックス定義を使用しない場合)

```
# make file made by wb33                          ...コメント

# macro definitions for tools & dir

TOOL_DIR = C:\CC33                                ...マクロ定義
GCC33 = $(TOOL_DIR)\gcc33
PP33  = $(TOOL_DIR)\pp33
EXT33 = $(TOOL_DIR)\ext33
AS33  = $(TOOL_DIR)\as33
LK33  = $(TOOL_DIR)\lk33
MAKE  = $(TOOL_DIR)\make
SRC_DIR =

# macro definitions for tool flags

GCC33_FLAG = -B$(TOOL_DIR)\ -S -g -O
PP33_FLAG  = -g
EXT33_FLAG =
AS33_FLAG  = -g
LK33_FLAG  = -g -s -m -c
EXT33_CMX_FLAG = -lk test -c

# dependency list

test.srf : test.cm boot.o main.o                  ...依存リスト
          $(LK33) $(LK33_FLAG) test.cm            ...コマンドライン

boot.ms : $(SRC_DIR)boot.s
          $(PP33) $(PP33_FLAG) $(SRC_DIR)boot.s
          $(EXT33) $(EXT33_FLAG) boot.ps
boot.o : boot.ms
          $(AS33) $(AS33_FLAG) boot.ms

main.ms : $(SRC_DIR)main.c
          $(GCC33) $(GCC33_FLAG) $(SRC_DIR)main.c
          $(EXT33) $(EXT33_FLAG) main.ps
main.o : main.ms
          $(AS33) $(AS33_FLAG) main.ms

# optimaization by 2 pass make

opt:
          $(MAKE) -f test.mak
          $(TOOL_DIR)\cwait 2
          $(EXT33) $(EXT33_CMX_FLAG) test.cmx
          $(MAKE) -f test.mak

# clean delete files except source

clean:
          del *.srf
          del *.o
          del *.ms
          del *.ps
```

●依存リスト

makeは次の形式で記述された依存リストに従って実行されます。

形式1: **<ターゲットファイル名>: <依存ファイル名1>** [**^ <依存ファイル名2>...**]
[　　**TAB　　<コマンド1>**
　　TAB　　<コマンド2>
　　**:　　**]

形式2: **<ターゲット名>:**
　　TAB　　<コマンド1>
[　　**TAB　　<コマンド2>**
　　**:　　**]

- ^はスペースを示します。
- [ ]は省略可能なことを示します。
- <コマンド>の前にはTAB(スペースは不可)が必要です。

形式1

形式1ではターゲットファイルを得るために必要な依存ファイルを指定し、ターゲットファイルが作成されていない場合、あるいは依存ファイルが更新されている場合に、続くコマンドを実行させます。

例: test.srf : test.cm boot.o main.o
　　$(LK33) $(LK33_FLAG) test.cm

この例ではターゲットファイル"test.srf"が"test.cm"、"boot.o"、"main.o"に依存していることを示します。
ターゲットファイル"test.srf"がない場合、あるいは依存ファイルに1つでもターゲットファイルより新しいものがある場合、コマンド"$(LK33) $(LK33_FLAG) test.cm"(lk33によるリンク)が実行されます。$(<名前>)は<名前>で定義したマクロと置き換えるための記述です。

- **依存ファイルが他のターゲットの場合**

依存ファイルが他の依存リストのターゲットとして指定されている場合は、そちらの依存リストを先に評価します。
たとえば、依存ファイル"boot.o"は次の2つの依存リストに関係していますので、それらのmakeを先に行います。

```
boot.ms : $(SRC_DIR)boot.s
        $(PP33) $(PP33_FLAG) $(SRC_DIR)boot.s
        $(EXT33) $(EXT33_FLAG) boot.ps
            ... ext33の出力ファイル"boot.ms"がない、あるいはソースファイル"boot.s"が
                "boot.ms"より新しい場合、pp33とext33を実行
boot.o : boot.ms
        $(AS33) $(AS33_FLAG) boot.ms
            ... as33の出力ファイル"boot.o"がない、あるいは"boot.ms"が"boot.o"より新し
                い場合、as33を実行
```

- **依存ファイルがない場合**

記述された依存ファイルがなく、他のターゲットとしても指定されていない場合はエラーとなります。

- **コマンドラインがない場合**

何も実行しません。ただし、ターゲットファイルと最初の依存ファイルの拡張子を持つサフィックスリスト(後述)が記述されていれば、そのコマンドを実行します。

形式2

依存ファイルが記述されていない場合、<ターゲット名>は単なるラベルとして使用されます。makeの起動コマンドでその<ターゲット名>を指定することにより、記述されたコマンドを実行させることができます。

例: make -f test.mak cleanで実行されるコマンド

```
clean:
        del *.srf
        del *.o
        del *.ms
        del *.ps
```

起動コマンドで<ターゲット名>を指定しなかった場合は、ファイル内で最初に記述されている依存リストによるmakeを行います。

●コマンドライン

S5U1C33000Cのツール名やDOSプロンプトのコマンドを記述することができます。
ただし、ドライブを示すコロン(:)は直接記述することはできません。コマンドラインでパスを指定する場合は、パスをマクロ定義して使用してください。
コマンドラインの先頭には、次の2つの記号を使用することができます。

@ そのコマンドラインのecho表示をoffします。
例: @copy test.s test.sbk
通常は、実行したコマンドラインがstdoutに出力されますが、@で始まるコマンドラインは出力されません。

- そのコマンドがエラー(exit(0)以外で終了)となった場合でも、エラーを無視して続くコマンドを実行します。
例: -make -f test.mak -n
通常は、エラーとなったコマンドでmakeを終了します。

コマンドライン内では、定義済みのマクロが参照可能です。また、次の2つのマクロ記号が使用できます。

$* 現在処理中のターゲットファイル名(拡張子を除く)に置き換わります。
例: test.dis: test.srf
$(TOOL_DIR)¥dis33 $*.dis

$@ 現在処理中のターゲットファイル名(拡張子を含む)に置き換わります。
例: 33xxxxx.sa: test.sa_80000_80fff
copy test.sa_80000_80fff $@

これらのマクロ記号は、コマンドライン以外では使用できません。

●マクロ定義

makeファイル内ではマクロを定義し、コマンドライン等から参照することができます。
マクロ定義と参照の形式は次のとおりです。

定義: **<マクロ名> = <マクロ本体>**

参照: **$(<マクロ名>)**

```
例: TOOL_DIR = C:¥CC33                          ... マクロ定義
    GCC33 = $(TOOL_DIR)¥gcc33                   ... マクロ定義および
    GCC33_FLAG = -B$(TOOL_DIR)¥ -S -g -O           マクロ定義内でのマクロ参照
    $(GCC33) $(GCC33_FLAG) $(SRC_DIR)main.c     ... コマンドライン内でのマクロ参照
    boot.ms : $(SRC_DIR)boot.s                  ... 依存ファイル名のマクロ参照
```

ドライブの指定に使用する":"はターゲット名の直後とコメント以外では、マクロ定義内でのみ使用可能です。したがって、コマンドラインや依存ファイルでパスを指定する場合は、上記例のようにマクロを定義して使用します。
マクロは以下の場所で参照することはできません。
・マクロ定義より前の行
・ターゲットファイル名
・サフィックス定義の行およびサフィックスリストの1行目

※ マクロの最後に"\"を記述する場合の注意事項

行の最後に¥<CR>が記述されていると、次の行に継続しているとみなされます。
したがって、¥で終了するパスなどをマクロ定義する場合は次のように記述してください。
例: "c:¥e0c33¥"をマクロ定義する場合

```
SRC_DIR = c:¥e0c33¥¥          ...¥を2個記述
                              ...空白行(改行のみ)を1つ挿入
<next statement>
```

●コメント

#からその行の終端まではコメントとみなされます。コメント内では、ASCIIキャラクタ以外の文字も記述できます。
例: # make file made by wb33

ただし、コマンドラインでは実行コマンドの一部とみなされるため、記述できません。

●サフィックス定義とサフィックスリスト

サフィックス定義とサフィックスリストを記述しておくと、依存リスト内のコマンドを省略することができます。wb33でmakeファイルを作成する際は、[Suffix type]オプションを選択しておくとサフィックス定義とサフィックスリストを含むmakeファイルが作成されます。

サフィックス未定義の依存リスト

```
# dependency list

test.srf : test.cm boot.o main.o
        $(LK33) $(LK33_FLAG) test.cm

boot.ms : $(SRC_DIR)boot.s
        $(PP33) $(PP33_FLAG) $(SRC_DIR)boot.s
        $(EXT33) $(EXT33_FLAG) boot.ps
boot.o : boot.ms
        $(AS33) $(AS33_FLAG) boot.ms

main.ms : $(SRC_DIR)main.c
        $(GCC33) $(GCC33_FLAG) $(SRC_DIR)main.c
        $(EXT33) $(EXT33_FLAG) main.ps
main.o : main.ms
        $(AS33) $(AS33_FLAG) main.ms
```

サフィックスを定義した例

```
# suffix & rule definitions

.SUFFIXES : .c .s .ps .ms .o .srf                ...サフィックス定義

.c.ms :                                          ...サフィックスリスト
        $(GCC33) $(GCC33_FLAG) $(SRC_DIR)$*.c
        $(EXT33) $(EXT33_FLAG) $*.ps

.s.ms :
        $(PP33)  $(PP33_FLAG)  $(SRC_DIR)$*.s
        $(EXT33) $(EXT33_FLAG) $*.ps

.ms.o :
        $(AS33)  $(AS33_FLAG)  $*.ms

# dependency list

test.srf : test.cm boot.o main.o
        $(LK33) $(LK33_FLAG) test.cm

boot.ms : $(SRC_DIR)boot.s                       ...依存リスト
boot.o : boot.ms

main.ms : $(SRC_DIR)main.c
main.o : main.ms
```

- **サフィックス定義**

サフィックスリストを使用する場合には、その前にサフィックスリストで使用するファイルの拡張子を定義しておく必要があります。サフィックス定義の形式は次のとおりです。

形式:　**.SUFFIXES : .xxx　.yyy　.zzz　.....**

例:　.SUFFIXES : .c .s .ps .ms .o .srf

- **サフィックスリスト**

サフィックスリストの形式は次のとおりです。

形式:　**<.依存ファイル1の拡張子><.ターゲットファイルの拡張子>:**
TAB　<コマンド1>
[　**TAB　<コマンド2>**
:　]

例:　main.o : main.ms　...依存リスト

.ms.o :　...サフィックスリスト
$(AS33)　$(AS33_FLAG)　$*.ms　...$*はターゲットファイル名(main)に置き換えられるマクロ記号です。

この例のサフィックスリストは"*.o"のターゲットファイルおよび"*.ms"の依存ファイル1(最初に記述された依存ファイル)を持つ依存リストに対応します。
その依存リストのコマンドラインを省略した場合に、サフィックスリストのコマンドが実行されるようになっています。

1つのサフィックスリストは、同じ拡張子の組み合わせを持つ複数の依存リストに対応しますので、ファイル数が多くなった場合に依存リストの記述が簡略化できます。

●文字の制限

各項目で使用可能な文字を以下に示します。それ以外の文字は使用しないでください。

表17.1.3.1　make**ファイル内で使用可能な文字**

項目		使用可能な文字
依存リスト	**ターゲット名**	a〜z　A〜Z　0〜9　_　-　.
	依存ファイル名	a〜z　A〜Z　0〜9　_　-　.　/　¥　$( )
マクロ	**マクロ名**	a〜z　A〜Z　0〜9　_　-
	マクロ本体	a〜z　A〜Z　0〜9　_　-　.　/　¥　:　$( )
サフィックス		a〜z　A〜Z　0〜9　_　-　.
コマンドライン		cc33**ツールや**DOS**プロンプトのコマンド** (**注**) $*　$@　$( )
コメント		**表示可能文字**

(注)　ドライブを示す":"はコマンドライン内で使用することができません。マクロを定義して使用してください。

・ ファイル名(パスを含む)、マクロ名、コマンド行は100文字以下で記述してください。
・ マクロ本体は、1000文字以内で作成してください。

17.1.4 2パスmake

wb33が作成するmakeファイルには、ext33でリンク後のシンボル/マップファイルを読み込んでコードの最適化を行うための2パスmakeのコマンドが記載されています。ターゲット名"opt"を指定(wb33では[2 pass]を選択)してmakeを行うと、それらのコマンドが実行されるようになっています。
記載されたコマンドは次のとおりです。

```
# optimization by 2 pass make

opt:
        $(MAKE) -f test.mak                          ...(1)
        $(TOOL_DIR)\cwait 2                          ...(2)
        $(EXT33) $(EXT33_CMX_FLAG) test.cmx          ...(3)
        $(MAKE) -f test.mak                          ...(4)
```

(1) 一度リンクまでの処理を行います。
(2) cwaitというツールで2秒の待ち時間を作ります。これは、2パス目のmakeを確実に行うための時間差を得るためのものです。
(3) リンクマップファイルおよびシンボルファイルを入力して最適化を行うオプション(-lk)を指定してext33を実行します。
(4) 再度makeを行い、アブソリュートオブジェクトファイルを作成します。

リンクマップファイルとシンボルファイルを使用した最適化については、"10.7.3 シンボル情報による最適化"を参照してください。

17.1.5 clean

wb33が作成するmakeファイルには、ソース以外の中間ファイルおよびオブジェクトファイルを削除するコマンドが記載されています。ターゲット名"clean"を指定してmakeを行うと(wb33ではmakeファイルを選択して[MAKE clean]ボタンをクリック)、それらのコマンドが実行されるようになっています。
記載されたコマンドは次のとおりです。

```
# clean files except source

clean:
        del *.srf
        del *.o
        del *.ms
        del *.ps
        del *.map
        del *.sym
```

カレントディレクトリ内の".srf"、".o"、".ms"、".ps"、".map"、".sym"の拡張子を持つファイルがすべて削除されます。

17.1.6　エラー/ワーニングメッセージ

エラー/ワーニングメッセージは標準出力(stdout)に表示/出力されます。
wb33の[MAKE]ボタンでmakeを起動すると"wb33.err"に出力され、実行終了時にメッセージが出力ウィンドウ(デフォルト)に表示されます。

make自体でエラー/ワーニングが発生した場合、makeはメッセージを表示して直ちに処理を終了します。makeファイル内の"-"で始まるコマンドにより実行したツールでエラーが発生した場合は処理を継続します。ツールのエラーメッセージについては、各ツールの章を参照してください。

makeのエラー/ワーニングメッセージを以下に示します。

表17.1.6.1　エラー/ワーニングメッセージ

エラー/ワーニングメッセージ	内容
Error: Cannot open XXXXXXXX	make ファイルがオープンできません。
Error: Cannot open tmp file	テンポラリファイルがオープンできません。
Error: Invalid syntax near line #	不正な文法で記述されています。#で示される行番号は目安として使用してください。¥<CR>で終わる行は次の行に続くものとして行数にはカウントされません。
Error: Invalid suffix .XXX	サフィックスが未定義です。
Error: Invalid macro name XXXX	マクロ名が未定義です。
Error: Abnormal termination in XXXX	モジュール XXXX の処理中にエラーが発生しました。
Error: Not enough memory	メモリが不足しています。
Error: Too long string	1 つの文字列が 1000 文字を越えています。
Error: No target found	ターゲットが見つかりません。
Error: Don't know how to make XXXX	ターゲットファイルがありません。
Error: Command exit with X	コマンドが異常終了しました。(X=exit コード)
Warning: XXXX is up-to-date	最終ターゲットはすでに更新されています。コマンドを実行せずに終了します。

17.1.7　注意事項

本パッケージのmakeは本書で説明した以外の機能(マクロやサフィックスのデフォルト設定、$<や$?などのマクロ記号)はサポートしていません。UNIXのmakeを常用されている方は注意してください。

makeはコマンドラインの実行で0のEXITコードが返るとそれ以降の実行を中断します。ただし、Windows95上でEXEファイルを実行すると、エラーの有無にかかわらずEXITコードが常に0として返ります。
makeはS5U1C33000Cツールに対しては特殊処理を行ってEXITコードを判断しますが、それ以外のEXEファイルのEXITコードは判別できず、処理を継続します。Windows95上で使用する場合は注意してください。
Windows NT4.0上では上記の問題は発生しません。

wb33の"Make file editor"にはファイルを追加/削除する機能があります。これには、makeファイル内のコメントや文字パターンが使用されますので、makeファイルをエディタ等で修正する場合は注意してください。必要なコメントや文字パターンを削除してしまうと、その後"Make file editor"による編集で誤動作してしまいます。この詳細については、5.2.6項内の"makeファイル編集上の注意事項"を参照してください。

ディレクトリのデリミタである"¥"のかわりに"/"も使用できます。ただし、連続した行であることを示す"¥"は代替できません。

17.2　cwait

17.2.1　機能

cwaitは指定の秒数の待ち時間を作るためのツールです。
wb33が作成するmakeファイル中で、2パスmakeの際に時間差を設けるために使用されます。

17.2.2　使用方法

●起動フォーマット

cwait ^ [<秒数>]

^はスペースを示します。
[ ]は省略可能なことを示します。

<秒数>: 待ち時間を秒数で指定します。
例: c:¥cc33¥cwait 2　　...2秒の待ち時間を作成

●Usage出力

cwaitの実行ではメッセージは表示されません。
ただし、秒数を指定せずに起動させた場合は、次のメッセージを表示します。

```
Cwait ver x.x
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    cwait <wait_second>
Example:
    cwait 2
```

17.3 ccap

17.3.1 機能

各ツールやコマンドがコンソール(標準出力あるいは標準エラー)に対して出力するメッセージをファイル化します。
wb33の実行ボタンで各ツールを実行する際は、ccapによってツールのメッセージをwb33.errというファイルに出力し、ツールの実行終了時に、wb33.errに出力された内容を出力ウィンドウ(あるいはエディタ)に表示させています。

17.3.2 使用方法

●起動フォーマット

ccap ^ [<オプション>] ^ <出力ファイル名> ^ "<実行コマンド>"

^はスペースを示します。
[ ]は省略可能なことを示します。

<出力ファイル>: メッセージを出力するファイル名を指定します。
<実行コマンド>: 実行するツールの起動コマンドを入力します。

●オプション

ccapには4種類の起動時オプションが用意されています。

-a
機能: 既存ファイルへの追加
説明: -aオプションを指定すると出力内容を既存のファイルの後に追加します。
ファイルが存在しない場合は、新規に作成します。
デフォルト: -aオプションが指定されない場合、指定のファイルに上書き(ファイルが存在する場合)、または新規作成(ファイルが存在しない場合)します。

-o
機能: ファイルのみ出力
説明: 実行コマンドのメッセージをファイルにのみ出力します。コンソールには出力しません。
デフォルト: -oオプションが指定されない場合、コンソールとファイルの両方に出力します。

-c
機能: 実行コマンドラインの出力を停止
説明: -cオプションが指定されるとccapは実行コマンドラインをコンソールおよびファイルに出力しません。
デフォルト: -cオプションが指定されない場合、実行コマンドラインも出力されます。

-e
機能: エラーカウント
説明: -eオプションが指定されると実行コマンドが出力したエラーメッセージの数をカウントして出力します。
カウントの対象は、次の文字列で始まるメッセージです。

Error	エラー数のカウント
Warning	ワーニング数のカウント
.c:	gcc33メッセージのカウント
.h:	gcc33メッセージのカウント

デフォルト: カウントは行いません。

オプションの前後には1個以上のスペースが必要です。
例: c:¥cc33¥ccap -a -o -e wb33.err "gcc33 -S test.c"

●Usage出力

ファイル名や実行コマンドが指定されなかったり、オプション指定が正しくない場合、次の使用方法のメッセージを出力して終了します。

```
ccap
Console Capture Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    ccap [options] <output-file> "command line"
Options:
    -a : append mode
    -o : disable console output
    -c : disable command echo
    -e : display error count
Example:
    ccap -a -o -c console.cap "gcc sample.c"
```

●エラーメッセージ

ccapのエラーメッセージは次のとおりです。

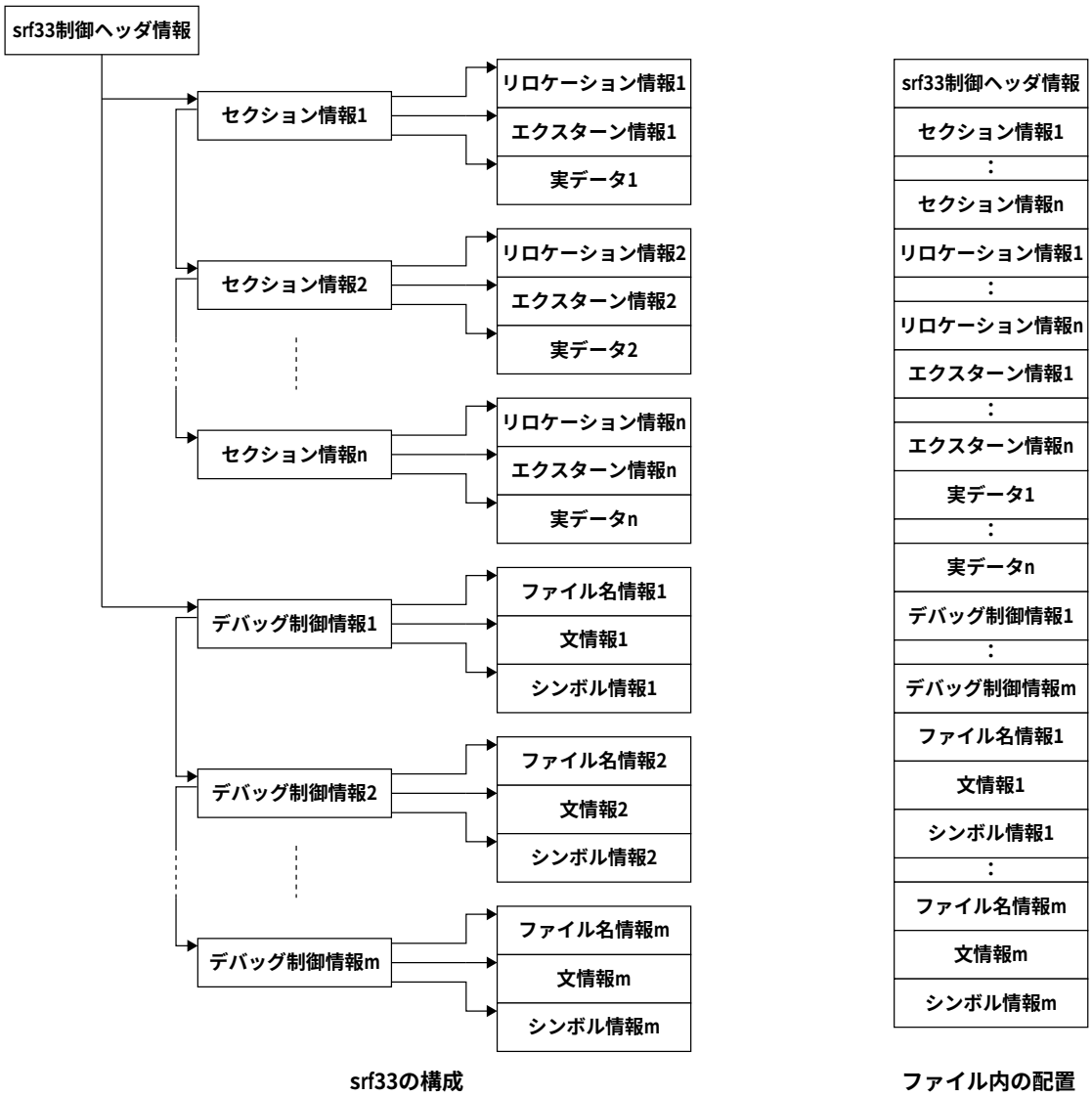
Error: Cannot execute	...指定した"実行コマンド"が実行できません。
Error: Cannot open output file	...出力ファイルがオープンできません。

Appendix　srf33ファイルの構造

A-1　srf33オブジェクトファイルの構造

アセンブラas33およびリンカlk33により生成されるsrf33形式ファイルの構造を以下に示します。
(srf33はSeiko Epson Relocatable File Format for S1C33の略称です。)

注: lk33の-ldコマンドを使用して作成したローダ用srf33ファイルの構造は、ここで説明する通常のsrf33ファイルとは異なります。ローダ用srf33ファイルについては、"utility¥ld33¥"ディレクトリ内の"readmeja.txt"(日本語)または"readme.txt"(英語)を参照してください。



図A.1　srf33オブジェクトファイルの構造

●srf33制御ヘッダ

srf33ファイルは必ずsrf33制御ヘッダを1つ、ファイルの先頭に持ちます。srf33制御ヘッダはセクション情報1、デバッグ制御情報1へのチェーン(ファイル内の位置情報)を持っています。

●セクション情報

アセンブラが生成するオブジェクトファイルでは、リロケータブルモジュールの場合は3個(CODE、DATA、BSS各1個ずつ)、アブソリュートモジュールの場合は1～3個(CODE、DATA、BSSそれぞれ0個または1個)のセクション情報を持ちます。
リンク後のモジュールは、リロケータブルな連続セクションはまとめられますが、それ以外のセクション(アドレスを個別に指定したセクションおよびアブソリュートセクション)はそのまま独立して存在します。リンク後のセクション情報は、CODEセクション、DATAセクション、BSSセクションごとに、アドレス順にまとめられます。実データを持たないCODEセクション、DATAセクション、BSSセクションもセクション情報は出力されます。

各セクション情報の内容は、そのセクションの属性(CODE、DATA、BSS)と配置アドレス、リロケーション情報/エクスターン情報/実データへのチェーン、実データのサイズなどです。また、次のセクション情報へのチェーンも持っています。

●リロケーション情報

リンカの再配置に必要な情報で、アセンブラが生成するオブジェクトファイルに含まれます。リンク後のオブジェクトファイルに、リロケーション情報は出力されません。

リロケーション情報の内容は、セクション内の命令コードに対するリロケーションタイプ、セクション内の位置、参照するエクスターン情報へのインデックスなどです。

●エクスターン情報

リンクに必要なシンボル情報で、アセンブラが生成するオブジェクトファイルに含まれます。リンク後のオブジェクトファイルに、エクスターン情報は出力されません。

エクスターン情報の内容は、セクション内で使用されているシンボルの名称と種類、セクション内の位置などです。

●実データ

各セクションごとの実際のデータ(BSSセクションはデータを持たない)です。CODEセクションの場合は、1つのコード(命令)に2バイトを使用します。DATAセクションの場合は、1つのデータに1、2または4バイトを使用します。

●デバッグ制御情報

デバッグ制御情報はリンクしたオブジェクトモジュール数だけ作られ、それぞれのモジュールのファイル名情報、文情報、シンボル情報のサイズと、チェーンを持ちます。また、次のデバッグ制御情報へのチェーンも持っています。
デバッグ制御情報はモジュールをリンクした順序で並びます。

デバッグ制御情報およびファイル名情報、文情報、シンボル情報は、Cコンパイラgcc33またはプリプロセッサpp33からリンカlk33までを-gオプションを指定して実行させた場合に出力されます。ソース表示およびシンボルの使用は、このデバッグ制御情報によって行います。デバッグ制御情報以降を持たない、あるいは読み出しに失敗した場合でも、実データまで正しく読み込まれれば、逆アセンブル表示によるデバッグは行えます。

●ファイル名情報

.file擬似命令で生成されるファイル名情報です。各モジュールのファイル名およびインクルードしたファイル名とそれぞれのディレクトリ構造の情報を持ちます。主にデバッガがソースコードを表示する際に参照します。

●文情報

.locおよび.endfile擬似命令で生成される行番号情報、ファイル切り換え情報です。主にデバッガが実データとソースコードを対応させるために参照します。

●シンボル情報

モジュール内で定義されているすべてのシンボルの情報を持ちます。シンボルの表示およびシンボルを使用したアドレス指定の際に参照されます。

＜参考: 各情報の内容＞

※　チェーン:　　ファイルの先頭からのバイト数で、続く情報のつながりを示します。0の場合は続く情報はありません。

　　インデックス:　セクションやシンボルを識別するための番号で、先頭が0となります。

(1)srf33制御ヘッダ

情　　報		Byte	内　　容
c_fatt	ファイル制御フラグ	2	以下のOR値 0x0001:リロケータブルファイル 0x0002:アブソリュートファイル 0x0004:実行形式(リンカ出力ファイル) 0x0008:デバッグ情報を含む 0x0010:ライブラリファイル
c_pentry	エントリーアドレス	2	0x0000:ブートアドレス
c_ver	srf33バージョン情報	2	0x3300(バージョン: 33, リビジョン: 00)
c_scncnt	セクション情報数	2	0x0000:セクション情報なし, 0以外:セクション情報数
c_scnptr	セクション情報チェーン	4	0x00000000:セクション情報なし, 0以外:チェーン
c_debptr	デバッグ制御情報チェーン	4	0x00000000:デバッグ制御情報なし, 0以外:チェーン

(2)セクション情報(最大65535個)

情　　報		Byte	内　　容
s_nxptr	次のセクション情報へのチェーン	4	0x00000000:セクション情報の終端, 0以外:チェーン
s_scntyp	セクションタイプ	2	0x0001:CODEセクション 0x0002:DATAセクション 0x0003:BSSセクション 0x0004:ダミーセクション
s_lnktyp	リンク方法	2	0x0000(未使用)
s_scnatt	セクション属性	2	0x0001:アブソリュート, 0x0002:リロケータブル
s_off	セクションのスタートアドレス	4	0x00000000～0x0fffffff
s_rcptr	リロケーション情報チェーン	4	0x00000000:リロケーション情報なし, 0以外:チェーン
s_rcsiz	リロケーション情報バイトサイズ	4	0x00000000:リロケーション情報チェーンが0の場合 0以外:バイトサイズ
s_exptr	エクスターン情報チェーン	4	0x00000000:エクスターン情報なし, 0以外:チェーン
s_exsiz	エクスターン情報バイトサイズ	4	0x00000000:エクスターン情報チェーンが0の場合 0以外:バイトサイズ
s_excnt	エクスターン情報の個数	4	0x00000000:エクスターン情報チェーンが0の場合 0以外:個数
s_rdptr	実データへのチェーン	4	0x00000000:実データなし(BSSセクションは常に0) 0以外:チェーン
s_dsiz	実データバイトサイズ	4	0x00000000: CODE/DATAセクションで実データへのチェーンが0の場合, BSSセクションではBSS領域がない場合 0以外: CODE/DATAセクションでは実データのバイトサイズ, BSSセクションではBSS領域のバイトサイズ
s_scnndx	セクションID	2	0x0000～0xffff

(3)リロケーション情報

情　　報		Byte	内　　容
r_rctyp	リロケーションタイプ	2	0x0001:8bit相対シンボル, SYMBOL<0x200 0x0002:32bit相対シンボル(31:22), SYMBOL@rh 0x0003:32bit相対シンボル(21:9), SYMBOL@rm 0x0004:32bit相対シンボル(8:1), SYMBOL@rl 0x0005:26bit相対シンボル(25:13), SYMBOL+sign32@ah 0x0006:26bit相対シンボル(12:0), SYMBOL+sign32@al 0x0007:32bit絶対シンボル(31:19), SYMBOL+imm32@h 0x0008:32bit絶対シンボル(18:6), SYMBOL+imm32@m 0x0009:32bit絶対シンボル(5:0), SYMBOL+imm32@l 0x000a:32bit絶対シンボル(31:0), SYMBOL
r_scnoff	セクション内オフセット	4	0x00000000～この情報が所属するセクションの実サイズ
r_exndx	参照するエクスターン情報のインデックス	4	0x00000000～参照するセクション内のエクスターン情報の数
r_scnndx	参照するエクスターン情報が所属するセクションのID	2	0x0000～同一ファイル内のセクション数
r_symoff	シンボルからのオフセット値	4	0x00000000(オフセット0)～0xffffffff

(4)エクスターン情報

情　　報		Byte	内　　容
e_scnoff	セクション内のオフセット	4	0x00000000～この情報が所属するセクションの実サイズ
e_size	シンボルのサイズ	4	0x00000000～0xffffffff(.comm, .lcommに対応)
e_scnndx	参照するエクスターン情報が所属するセクションのID	4	0x00000000(リンカ内部でのみ使用) リザーブエリア
e_extyp	エクスターンタイプ	2	0x0001:グローバルシンボル 0x0002:ローカルシンボル 0x0003:エクスターンシンボル
e_namsiz	シンボル名の長さ	1	0x00～0x20
e_exnam	シンボル名	*	シンボル名, *:最大32バイト

(5)実データ

CODEセクション: 1つのコードが2byte(下位－上位の順)で出力されます。(下位－上位の順)

DATAセクション: 1つのデータが1byte(byte)、2byte(half word)、または4byte(word)で出力されます(下位－上位の順)。

BSSセクション: 実データは持ちません。

(6)デバッグ制御情報

情　　報		Byte	内　　容
d_nxptr	デバッグ制御情報チェーン	4	0x00000000:デバッグ制御情報の終端, 0以外:チェーン
d_flptr	ファイル名情報チェーン	4	0x00000000:ファイル名情報なし, 0以外:チェーン
d_flsiz	ファイル名情報バイトサイズ	4	0x00000000:ファイル名情報チェーンが0の場合, 0以外:バイトサイズ
d_flcnt	ファイル名情報の個数	4	0x00000000:ファイル名情報チェーンが0の場合, 0以外:個数
d_stptr	文情報チェーン	4	0x00000000:文情報なし, 0以外:チェーン
d_stsiz	文情報バイトサイズ	4	0x00000000:文情報チェーンが0の場合, 0以外:バイトサイズ
d_syptr	シンボル情報チェーン	4	0x00000000:シンボル情報なし, 0以外:チェーン
d_sysiz	シンボル情報バイトサイズ (シンボル情報+特殊文情報)	4	0x00000000:シンボル情報チェーンが0の場合, 0以外:バイトサイズ
d_sycnt	シンボル情報の個数 (シンボル情報+特殊文情報)	4	0x00000000:シンボル情報チェーンが0の場合, 0以外:個数

(7)ファイル名情報

情　　報		Byte	内　　容
f_ftyp	種別	2	0x0000(未使用)
f_dirsiz	ディレクトリ名の長さ	1	0x00～0xff
f_fnamsiz	ソースファイル名の長さ	1	0x00～0xff
f_fnam	ソースファイル情報	*	パス&ファイル名, *:最大510バイト

(8)文情報

<一般の文情報>　※ソースの行情報(先頭が0xff以外の文情報)

情　　報		Byte	内　　容
t_line	ソースファイル内の行数	4	0x00000000～ソースファイル最終行
t_stat	文	2	0x0000(未使用)
t_scnoff	参照セクション内オフセット	4	0x00000000～参照セクションの実サイズ
t_scnndx	参照セクションID	2	0x0000～デバッグするsrf33ファイル内のセクション数

<特殊な文情報>　※ソースファイルの切り換え情報(先頭が0xff)

情　　報		Byte	内　　容
t_type	文情報のタイプ	2	0xffff:ソースファイル名参照範囲の開始 0cfffe:ソースファイル名参照範囲の終了
t_flndx	ファイル名情報へのインデックス	4	0x00000000～同一デバッグ制御情報内のファイル名情報数
t_fnoff	ソースファイルの最終CODEセクション内オフセット	4	0x00000000～参照セクションの実サイズ
t_scnndx	参照セクションID	2	0x0000～デバッグするsrf33ファイル内のセクション数

(9)シンボル情報

<一般のシンボル情報>

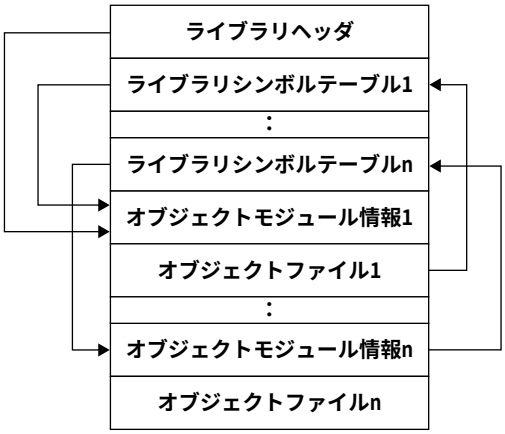
情　報		Byte	内　容
y_symndx	シンボルID	4	0x00000000～同一デバッグ制御情報内のシンボル情報数 0xffffffff: 無効のシンボル情報
y_value	シンボル値	4	0x00000000～0xffffffff
y_typ	シンボルタイプ	2	y_typ[0](3:0) 0: no type 1: void 2: char 3: short int 4: int 5: long int 6: float 7: double 8: structure 9: union 10: enum 11: member of enum 12: unsigned char 13: unsigned short 14: unsigned int 15: unsigned long y_typ[1] & y_typ[0](7:4) 0: no derived type 1: pointer 2: function 3: array (上記の値の組み合わせ)
y_scnndx	参照セクションID	2	0x0000～デバッグするsrf33ファイル内のセクション数
y_sclass	ストレージクラス	1	0: null 1: automatic variable 2: external symbol 3: static 4: register variable 5: external definition 6: label 7: undefined label 8: member of structure 9: function argument 10: structure tag 11: member of union 12: union tag 13: type definition 14: undefined static 15: enumeration tag 16: member of enumeration 17: register parameter 18: bit field 19: auto argument 20: dummy entry (end of block) 100: beginning of block ".begin" 101: beginning of function ".ent" 102: end of structure 103: file name 104: line # reformatted as symbol table entry 105: duplicate tag 106: ext symbol in dmert public lib 110: end of block ".bend" 111: end of function ".end"
y_auxcnt	AUXエントリ数	1	0x00～0xff
y_namsiz	シンボル名の長さ	1	0x00～0x20
y_symnam	シンボル名	*	シンボル名, *:最大32バイト

<AUXエントリ>

情　報		Byte	内　容
x_tagndx	構造体,共用体,enumタグ名へのインデックス	4	0x00000000～0xffffffff
x_size	構造体,共用体,配列のサイズ	4	0x00000000～0xffffffff
x_dimen	配列のディメンジョン	4*4	0x00000000～0xffffffff*0xffffffff

A-2　ライブラリファイルの構造

ライブラリアンlib33により生成されるライブラリファイルの構造を以下に示します。



ライブラリの構成

図A.2　ライブラリファイルの構造

●ライブラリヘッダ

ライブラリファイルは必ずライブラリヘッダを1つ、ファイルの先頭に持ちます。ライブラリヘッダはファイルサイズやライブラリファイル名の情報、最初のオブジェクトモジュールへのポインタ(ファイル内の位置情報)で構成されます。

●オブジェクトモジュール情報

オブジェクトモジュール個々に、オブジェクトモジュール情報が生成されます。オブジェクトモジュール情報はオブジェクトファイル名やサイズ、対応するライブラリシンボルテーブルへのポインタで構成されます。

●ライブラリシンボルテーブル

個々のオブジェクトモジュールに対応する、グローバルシンボル情報テーブルです。グローバルシンボルを持たないオブジェクトモジュールには生成されません。ライブラリシンボルテーブルは、シンボルテーブルのサイズ、個々のグローバルシンボル情報(シンボル名、対応するオブジェクトモジュール情報へのポインタ)で構成されます。

＜参考: 各情報の内容＞

(1)ライブラリヘッダ

情　　報		Byte	内　　容
l_att	ファイル制御フラグ	2	0x0010
l_size	ライブラリ全体のサイズ	4	ライブラリファイルのサイズ
l_ver	srf33バージョン情報	2	0x3300(バージョン: 33, リビジョン: 00)
l_objptr	最初のオブジェクトモジュールへのポインタ	4	ファイル先頭からオブジェクトモジュール情報までのオフセット
l_namsiz	ライブラリファイル名のサイズ	1	0x00〜0xff
f_fnam	ライブラリファイル名	*	パス&ファイル名, *:最大510バイト

(2)オブジェクトモジュール情報

情　　報		Byte	内　　容
o_att	ファイル制御フラグ	4	0xffffffff
o_size	オブジェクトファイルのサイズ	4	オブジェクトファイルのサイズ
o_lsymptr	ライブラリシンボルテーブルへのポインタ	4	ファイル先頭から対応するライブラリシンボルテーブルまでのオフセット
l_namsiz	オブジェクトファイル名のサイズ	1	0x00〜0xff
f_fnam	オブジェクトファイル名	*	パス&ファイル名, *:最大510バイト

(3)ライブラリシンボルテーブル

情　　報		Byte	内　　容
lst_size	テーブルサイズ	4	個々のライブラリシンボルテーブルの全体サイズ

テーブルサイズに続き、個々のグローバルシンボル情報が以下のとおり生成されます。

情　　報		Byte	内　　容
ls_objptr	オブジェクトモジュール情報へのポインタ	4	ファイル先頭から対応するオブジェクトモジュール情報までのオフセット
ls_namsiz	グローバルシンボル名のサイズ	1	0x00〜0x20
ls_glnam	グローバルシンボル名	*	シンボル名, *:最大32バイト

S1C33 Family C Compiler Package

Quick Reference

EPSON

CMOS 32-bit Single Chip Microcomputer

S1C33 Family C Compiler Package

Quick Reference for Development

メモリマップとトラップテーブル

S1C33000 Core CPU

メモリマップ

			サイズ
0xFFFFFFF	エリア18	外部メモリ	64MB
	エリア17	外部メモリ	64MB
	エリア16	外部メモリ	32MB
	エリア15	外部メモリ	32MB
	エリア14	外部メモリ	16MB
	エリア13	外部メモリ	16MB
	エリア12	外部メモリ	8MB
0x1000000	エリア11	外部メモリ	8MB
0x0C00000	エリア10	外部メモリ	4MB
	エリア9	外部メモリ	4MB
	エリア8	外部メモリ	2MB
	エリア7	外部メモリ	2MB
	エリア6	外部I/O	1MB
	エリア5	外部メモリ	1MB
0x0100000	エリア4	外部メモリ	1MB
0x0080000	エリア3	内蔵ROM	512KB
0x0060000	エリア2	Reserved	128KB
0x0030000	エリア1	内蔵I/O	128KB
0x0000000	エリア0	内蔵RAM	256KB

トラップテーブル

	ベクタアドレス
リセット	base + 0
Reserved	base + 4~12
ゼロ除算	base + 16
Reserved	base + 20
アドレス不整例外	base + 24
NMI	base + 28
Reserved	base + 32~44
ソフトウェア例外0	base + 48
:	:
ソフトウェア例外3	base + 60
マスク可能な外部割り込み0	base + 64
:	:
マスク可能な外部割り込み215	base + 924

base: トラップテーブル先頭アドレス
= 0x0080000 (内蔵ROMよりのブート時)
= 0x0C00000 (外部ROMよりのブート時)

レジスタ

S1C33000 Core CPU

汎用レジスタ (16)

31 … 0
R15
R14
R13
:
R4
R3
R2
R1
R0

特殊レジスタ (5)

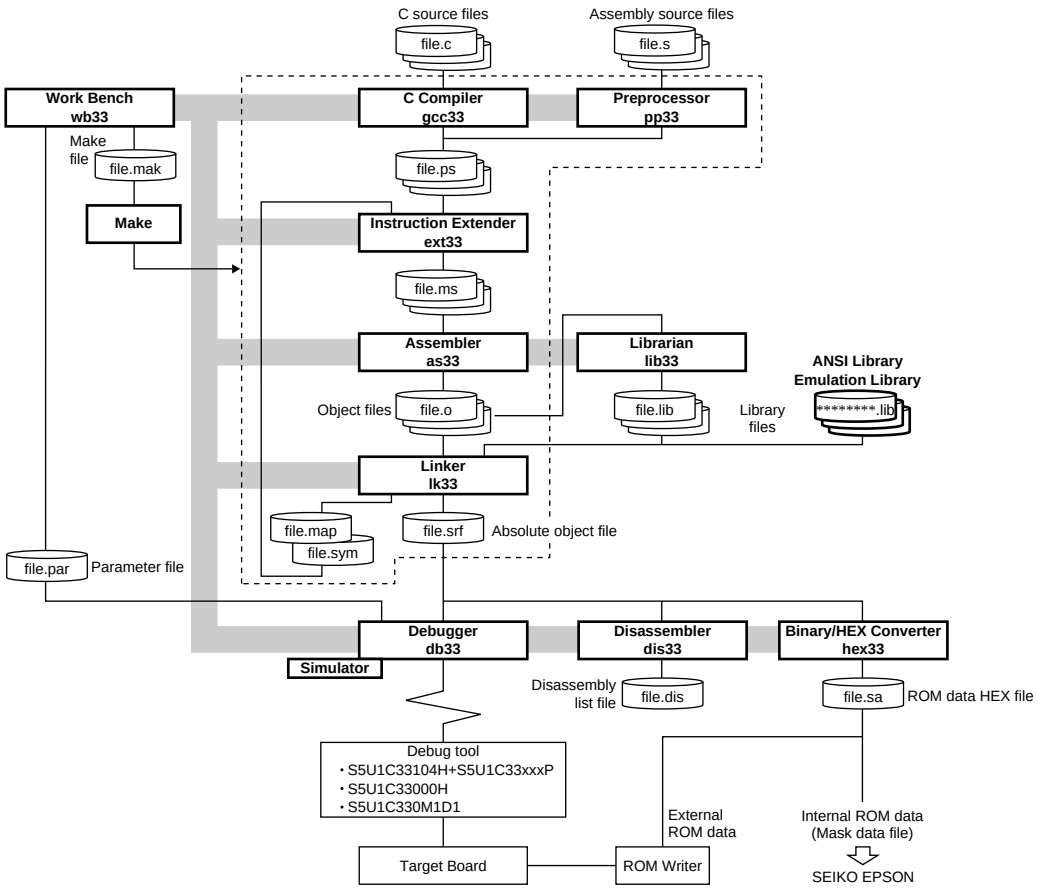
31 … 0	
PC	プログラムカウンタ
PSR	プロセッサステータスレジスタ
SP	スタックポインタ
ALR	算術演算ローレジスタ
AHR	算術演算ハイレジスタ

(AHR, ALR: 積和演算, 乗算, 除算用オプション)

PSR

31~12	11~8	7	6	5	4	3	2	1	0
Reserved	IL	MO	DS	–	IE	C	V	Z	N

IL: 割り込みレベル (0~15: 割り込み許可レベル)
MO: MACオーバーフローフラグ (1: オーバーフローあり, 0: なし)
DS: 被除数符号フラグ (1: 負, 0: 正)
IE: 割り込み許可 (1: 許可, 0: 禁止)
Z: ゼロフラグ (1: ゼロ, 0: ゼロ以外)
N: ネガティブフラグ (1: 負, 0: 正)
C: キャリーフラグ (1: キャリー/ボローあり, 0: なし)
V: オーバーフローフラグ (1: オーバーフローあり, 0: なし)



1. **プログラミング**
   エディタなどを使用してCソースファイル、アセンブリソースファイルを作成します。

2. **コンパイル、アセンブル、リンク**
   2-1) ワークベンチwb33を起動します。
   2-2) wb33の[Make edit]ボタンでmakeファイルを作成し、必要に応じてカスタマイズします。
   2-3) wb33の[MAKE]ボタンでmakeを実行します。
   makeはCコンパイラgcc33、プリプロセッサpp33、命令エクステンダext33、アセンブラas33、リンカlk33を順次実行し、実行形式のオブジェクトファイル(.srf)を生成します。各ツールは個別に実行することもできます。

3. **デバッグ**
   3-1) wb33の[Par gen]ボタンでパラメータファイルを作成し、必要に応じてカスタマイズします。
   3-2) wb33の[DB33]ボタンでデバッガdb33を起動します。
   3-3) 実行形式のオブジェクトファイル(.srf)をデバッガにロードし、デバッグコマンドを使用してデバッグを行います。

4. **マスクデータの作成**
   プログラムの完成後、マスク化のためのマスクデータファイルを作成します。
   4-1) バイナリ/HEXコンバータhex33を使用して、内蔵ROMデータHEXファイルを生成します。
   4-2) できあがったファイルをセイコーエプソンに提出してください。

概要

S1C33 Family C Compiler Packageの各開発ツールの起動、ツールの入力ファイルや主要な起動オプションの選択がwb33のウィンドウ上でマウス操作のみで行えます。

エラーメッセージ

Can not execute command	ツールやコマンドが実行できません。
Can not open file	ソース/make/オプションファイルがオープンできません。
Can not delete file	ファイルが削除できません。
Write error	make/オプションファイルへの書き込みが行えません。
Read error	ソース/オプションファイルが読み込めません。
R/W error	ファイルの読み込み/書き込みができません。
File size over 32KB, so cut down to 32KB	32KBを越えるソースファイルが選択されました。

実行ウィンドウ

[Open option window]ボタン

オプションウィンドウを開きます。

ツールボタン

各ツールを実行します。

ディレクトリ/ファイルリストボックス

各ツールの入力ファイルを選択します。

[Del]ボタン

選択したファイルを削除します。

[Refresh]ボタン

ファイルリストボックスを最新の内容に更新します。

[Editor]ボタン

指定のエディタで、選択したソースファイルなどを開きます。

コマンドラインエディットボックス

ツールの起動コマンドなどの表示、修正が行えます。

[Make edit]ボタン

makeファイルを作成するためのウィンドウを開きます。

[Par gen]ボタン

パラメータファイルを作成するためのウィンドウを開きます。

[MAKE clean]ボタン

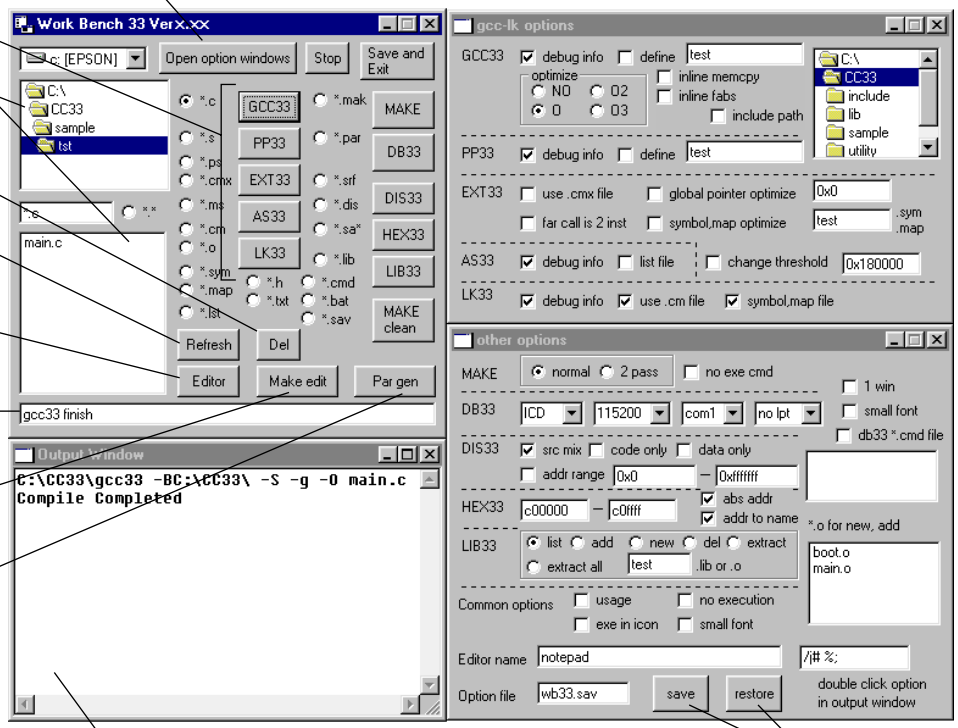
makeファイルのcleanを実行し、makeで生成したファイルを削除します。

[Stop]ボタン

ツールの実行を中止します。makeや単一ファイルの処理中には無効です。

[Save and Exit]ボタン

オプション設定を保存し、wb33を終了します。



出力ウィンドウ

ソースファイルの内容やツールの実行結果を表示します。ソースを表示させるには、実行ウィンドウをファイルリストボックス内のソースファイル名をダブルクリックしてください。

[gcc-lk options]ウィンドウ

gcc33、pp33、ext33、as33、lk33の起動オプションが選択できます。

[other options]ウィンドウ

make、db33、dis33、hex33、lib33の起動オプション、およびwb33のオプションが選択できます。

wb33オプション

- [usage]: ツールの使用方法を出力します。
- [no execution]: ツールを実行せずに、起動コマンドをコマンドラインエディットボックスに表示します。
- [exe in icon]: ツールを最小化状態で実行します。
- [small font]: 出力ウィンドウのフォントを変更します。
- [Editor name]: ソースファイルなどの表示に使用するエディタの起動コマンドを入力します。タグジャンプ用のエディタコマンドも設定できます。

[save/restore]ボタン

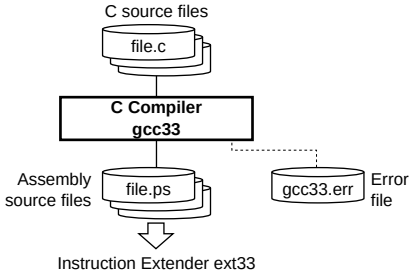
オプションウィンドウの設定状態を[Option file]で指定したファイルに保存またはファイルから復帰させます。

概要

gcc33はANSI C準拠のCコンパイラ(GNU Cコンパイラ)で、Cソースファイルをアセンブリソースファイルにコンパイルします。

＊ GNU Cコンパイラは Free Software Foundation, Incで開発されました。

フローチャート



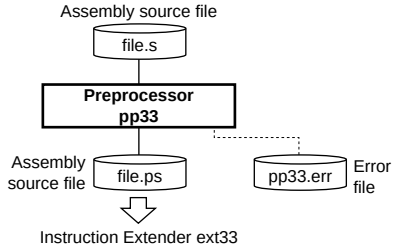
起動コマンド

```
GNU C Compiler for E0C33 Ver 2.7.2 (Rev x.xx)
Usage:
    gcc33 -S [options] filename
Options:
    -g : generate debug information
    -O : optimize output code
    -E : preprocess source files and output the results to stdout
    -B<directory>\ : specifies the directoy which cc1.exe and cpp.exe exit
    -I<directory>  : specifies the directoy which include files exist
    -D<macro=defn> : define macro "macro" as "defn"
    -D<macro>      : define macro "macro" as '1'
    -merr          : produce log file (gcc33.err)
    -mno-memcpy    : produce string variable initializer statement inline
Output:
    Extended assembler source files(.ps) for ext33
Example:
    gcc33 -S -Bc:\usr\local\bin\ -O -g test.c
```

概要

プリプロセッサpp33はアセンブリ言語によるプログラム開発時にソースファイルを最初に処理するツールです。複数のステートメント群を1つの命令のように使用することができるマクロ文や、他のファイルを挿入するインクルード文など、アセンブリソース作成のための付加機能を処理し、命令エクステンダext33に入力するアセンブリソースファイルを生成します。

フローチャート



起動コマンド

```
Pre Processor 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    pp33 [options] filename
Options:
    -e : produce log file (pp33.err)
    -g : generate debug information in output file
    -d string : define string
Output:
    Assembler source file for ext33 (.ps)
Example:
    pp33 -e -g -d TYPE1 test.s
```

エラーメッセージ

Cannot open file.	ソースファイル、インクルードファイルがオープンできません。
Cannot allocate memory.	メモリが確保できません。
Cannot open output file.	出力ファイルがオープンできません。
Cannot open working file.	作業用の一時ファイルがオープンできません。
Cannot write file.	ファイルへの書き込みができません。
Cannot close file.	ファイルをクローズできません。
Cannot read file. Line size is too long.	ステートメントが長すぎて読み込めません。 (最大255文字)
Preprocessor limit : macro label number full.	マクロ展開時に、分岐用内部ラベル数が制限を越えました。(最大9999個)
Invalid syntax.	プリプロセッサ擬似命令が間違った形式で記述されています。
Nesting level too deep.	#include擬似命令のネスティングが制限(2レベル)を越えています。
Unknown preprocessor instruction.	不明なプリプロセッサ命令です。
Too many macro parameters.	33個以上のマクロ仮パラメータが定義されています。
Invalid macro parameter.	パラメータの並びが不正です。($1~$32)
Invalid macro label.	マクロ定義内の分岐用内部ラベルが異常です。($$1~$$64)
Invalid expression.	数値演算子の記述形式が不正です。
Multi symbol.	#define、#defnumで同じ名前が多重定義されています。

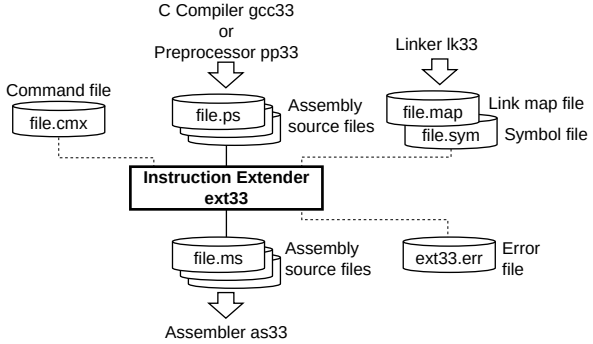
ワーニングメッセージ

Multi define symbol.	同じデファイン名、または数値デファイン名が複数定義されています。

概要

命令エクステンダext33はS1C33000命令セットが持つ即値拡張命令(ext)を最適化するツールです。アセンブリソース作成時にext命令を意識せずにプログラムを記述可能な拡張命令を、このツールが提供します。また、リンク後のシンボルマップ情報を元にコードをより最適化する2パスmakeにも対応しています。

フローチャート



起動コマンド

```
Extender 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    ext33 [options] filename
    ext33 [options] -c commandfile (.cmx)
Options:
    -e           : produce log file (ext33.err)
    -lk program  : optimize with program information (program.sym, program.map)
    -gp address  : optimize with global pointer (0x0 - 0xffffffff)
    -near        : specifies all xjmp and xcall extract 2 instruction
    -j threshold : specifies jump optimization threshold (0x100 - 0x1fffff)
Output:
    Assembler source file for as33(.ms)
Example:
    ext33 -e -lk test -gp 0x8000 test.ps
```

エラーメッセージ

Cannot allocate memory.	メモリが確保できません。
Cannot open input file "<file name>".	入力ファイルがオープンできません。
Invalid input filename "<file name>".	入力ファイル名の指定が不正です。
Cannot open output file "<file name>".	出力ファイルがオープンできません。
Cannot open error file "ext33.err".	エラーファイルがオープンできません。
Cannot open command file "<file name>".	コマンドファイルがオープンできません。
Cannot write error file.	エラーファイルへの書き込みができません。
Cannot write output file "<file name>".	出力ファイルへの書き込みができません。
Too long filename "<file name>".	ファイル名が長すぎます。(パスを含め、最大255文字)
Invalid command file description "<description>".	コマンドファイル内の<description>が不正です。
Invalid command file format.	コマンドファイル形式が不正です。
Invalid jump threshold "<parameter>".	-jオプションのパラメータが不正です。
Invalid GP address "<parameter>".	-gpオプションのパラメータが不正です。
Too many input files.	入力ファイルが多すぎます。(最大682)
No input file is specified.	入力ファイルが指定されていません。
Invalid syntax.	拡張命令の文法エラーです。

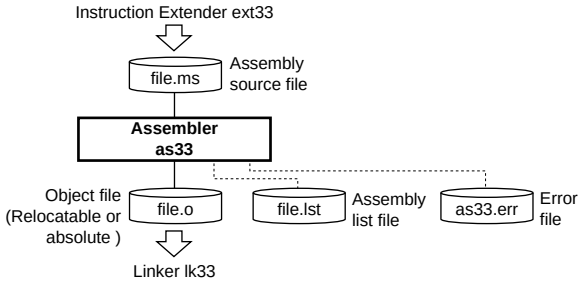
ワーニングメッセージ

Map file "<file name>" does not exist.	リンクマップファイルが見つかりません。
Symbol file "<file name>" does not exist.	シンボルファイルが見つかりません。
No map information in map file.	マップ情報がリンクマップファイル内に存在しません。
Invalid map file format.	不正なマップファイル形式です。
Invalid symbol file format.	不正なシンボルファイル形式です。
Cannot find the symbol "<symbol>" in symbol table.	<symbol>の情報がシンボルファイルに見当たりません。
Operand exceeds maximum address.	オペランドのアドレスが32ビットの有効範囲を越えています。
Invalid address operand.	不正な境界アドレスが指定されています。
Invalid operand value.	拡張命令のオペランドの指定が有効範囲外です。

概要

アセンブラas33は命令エクステンダが出力するアセンブリソースファイルをアセンブルし、ソースファイルのニーモニックをS1C33000のオブジェクト(機械語)コードに変換します。結果はリンクおよびライブラリ化が可能なオブジェクトファイルとして出力されます。

フローチャート



起動コマンド

```
Assembler 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    as33 [options] <filename>
Options:
    -e : produce log file (as33.err)
    -g : generate debug information in object file
    -l : produce list file
Output:
    object file (.o)
    list file   (.lst)
    log file    (as33.err)
Example:
    as33 -e -g -l sample.ms
```

エラーメッセージ

Out of memory.	メモリが確保できません。
File open error.	ファイルがオープンできません。
File access error.	オープンしたファイルがリード/ライトできません。
Invalid file name.	ソースファイルが出力ファイルと同じ拡張子(.o)を持っています。
Filename length limit exceeded - 255.	ファイル名の長さが255文字を越えました。
Directory path length limit exceeded - 255.	ディレクトリを含めたファイル名の長さが255文字を越えました。
Line length limit exceeded - 255.	１行の文字数が255文字を越えました。
Symbol name length limit exceeded - 32.	シンボル名が128文字を越えました。
Token length limit exceeded - 64.	数値データが64桁を越えました。
Multiple statements on the same line.	１行に複数のステートメントがあります。
Invalid statement syntax.	不正な文字が使われています。
Invalid instruction. - "<instruction>"	不正なインストラクション名が使われています。
Invalid register. - "<register>"	不正なレジスタ名が使われています。
Invalid directive. - "<directive>"	不正な擬似命令が使われています。
Invalid symbol mask. - "<mask>"	不正なシンボルマスクが使われています。
Invalid instruction syntax.	インストラクションの文法が不正です。
Invalid directive syntax.	擬似命令の文法が不正です。
Multiply declared symbol. - "<symbol>"	シンボルが二重に宣言されています。
Multiply defined symbol. - "<symbol>"	シンボルが二重に定義されています。
Incorrect section type for statement.	セクションタイプが不正です。
Memory mapping conflict.	メモリ配置でアドレスが重複しています。
Section count limit exceeded - 256.	セクション数が256を越えました。

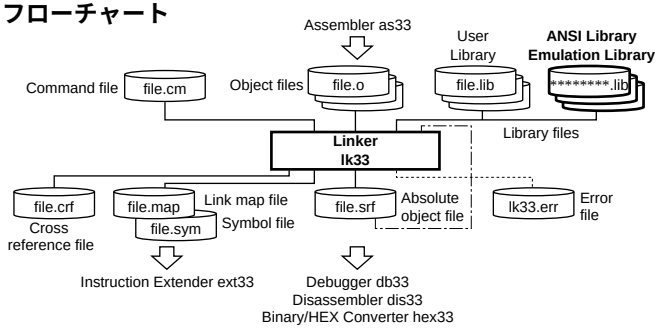
ワーニングメッセージ

Numeric range.	インストラクションオペランドの数値が不正です。
Unknown escape sequence.	.ascii擬似命令に不正なエスケープシーケンスが使われています。
Escape sequence out of range for character.	.ascii擬似命令中の文字コードが0xffを越えています。
¥x used with no following hex digits.	.ascii擬似命令の16進数値指定に16進で表記される数値がありません。

概要

アセンブラas33によって生成したオブジェクトコードのメモリロケーションを決定し、実行可能なオブジェクトコードを生成します。複数のオブジェクトファイルやライブラリも、このツールによって1つにまとめられます。

フローチャート



起動コマンド

```
Linker 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 1997-2001
Usage:
    lk33 [options] <filenames>
Options:
    -e : produce log file (lk33.err)
    -g : generate debug information in output file
    -s : generate symbol information (.sym)
    -m : generate map information (.map)
    -cref : generate cross reference information (.crf)
    -c CommandFile : execute lk33 commands from CommandFile (.cm)
Output:
    SRF33 object file (.srf)
Example:
    lk33 -e -g -s -m -c test.cm
```

リンカコマンド(Ver. 2.1)

(起動時オプションを除く)

-w	ワーニングレベルの設定(グローバルBSSシンボルの重複をワーニングとしない)
-l <library path>	ライブラリパスの指定(最大4パス)
-o <output file>	出力ファイル名の設定
-defsym <symbol> = <value>	グローバルシンボルの設定(最大255)
-d	重複グローバルBSS領域の削除
-align <alignment>	アライメント指定
-code <address/section>	リロケータブルなCODEセクション開始位置の設定
-code <address/section> { <files>}	指定ファイルのCODEセクション開始位置の設定
-data <address/section>	リロケータブルなDATAセクション開始位置の設定
-data <address/section> { <files>}	指定ファイルのDATAセクション開始位置の設定
-bss <address/section>	リロケータブルなBSSセクション開始位置の設定
-ucode <address/section>	リロケータブルな仮想CODEセクション開始位置の設定
-ucode <address/section> { <files>}	指定ファイルの共用CODEセクション開始位置の設定
-udata <address/section>	リロケータブルな仮想DATAセクション開始位置の設定
-udata <address/section> { <files>}	指定ファイルの共用DATAセクション開始位置の設定
-ubss <address/section> { <files>}	指定ファイルの共用BSSセクション開始位置の設定
-objsym	セクションシンボルの作成
-section <section>	仮想・共用セクション名の設定
-section <section> = <address>	出力セクション名と開始アドレスの指定
-ld	ローダ用srf33ファイルの出力
-inclink	.srfファイルのリンクを許可

リンカコマンド(Ver. 3.0)

(起動時オプションを除く)

-w	ワーニングレベルの設定
-l <library path>	ライブラリパスの指定(最大4パス)
-o <output file>	出力ファイル名の設定
-defsym <symbol> = <value>	グローバルシンボルの設定(最大255)
-d	重複グローバルBBS領域の削除
-objsym	セクションシンボルの作成
-ld	ローダ用srf33ファイルの出力
-inclink	.srfファイルのリンク許可
-align <alignment>	アライメント指定
-v3	lk33 Ver. 3コマンドイネーブル
-codeblock <block> { <files>}	CODEセクションのブロック化
-datablock <block> { <files>}	DATAセクションのブロック化
-bssblock <block> { <files>}	BSSセクションのブロック化
-ucodeblock <block> { <files>}	CODEセクションの並列(ユニオン)ブロック化
-udatablock <block> { <files>}	DATAセクションの並列(ユニオン)ブロック化
-ubssblock <block> { <files>}	BSSセクションの並列(ユニオン)ブロック化
-addr <address/symbol> { <blocks>}	ブロックの配置アドレス指定
-defaddr <symbol> = <address>	アドレス定義

エラー/ワーニングメッセージ

エラーメッセージ

Cannot open input file.	入力ファイル(*.o、*.lib)がオープンできません。
Cannot open output file.	出力ファイルが作成できません。
Cannot open working file.	中間ファイルが作成できません。
Cannot allocate memory.	メモリが確保できません。
Cannot read a file.	ファイルが読み込めません。
Unknown option near line = #.	リンカコマンドファイル内の#行付近に認識できないリンカコマンドが記述されています。
Invalid parameter near line = #.	リンカコマンドファイル内、#行付近のリンカコマンドのパラメータの記述に誤りがあります。
Not define section name near line = #.	リンカコマンドファイル内の#行付近で指定されているセクション名が定義されていません。
Uninitialized section name near line = #.	リンカコマンドファイル内の#行付近で指定されているセクション名はアドレスが定義されていません。
-v3 command is not in the 1st line.	-v3コマンドはコマンドファイルの先頭になければなりません。
This command is invalid near line =#.	不正なコマンドが#行付近にあります。
Too long symbol length near line =#.	シンボル名、ブロック名の文字長は最大130文字です。
Too long file path length near line =#.	オブジェクトファイル名の長さは最大256文字です。
Use invalid block name '<symbol>' near line =#.	ブロック名が不正です。
There is the file name same in an inside parenthesis near line =#.	{ }内に同じオブジェクトファイル名(フルパス)が複数使用されています。
'<symbol>' Not found symbol name near line =#.	シンボル名が見つかりません。
'<symbol>' is invalid block name near line =#.	ブロック名が不正です。
'<symbol>' is not defined near line =#.	ブロックが定義されていません。
There is the block name same in an inside parenthesis near line =#.	{ }内で指定されたブロック名が、同じ-addr内または他の-addrで重複しています。
'<symbol>' already defined near line =#.	ブロックが多重定義されています。
The object file name defined as '<block name>' and '<block name>' overlaps.	同一のオブジェクト定義を含む複数のブロックを、同じ-addr定義内、または他の-addr定義内で指定することはできません。
'@' is required for the initial character of '<symbol>'.	-uXXXblockで指定した共用ブロックのアドレスを指定する場合、ブロック名の先頭には"@"が必要です。
'<symbol>': There is no definition of an effective object file name.	{ }内に有効なファイルがありません。

エラーメッセージ

Code section map out of range.	CODEセクションの配置アドレスがリンク可能範囲外です。
Data section map out of range.	DATAセクションの配置アドレスがリンク可能範囲外です。
Bss section map out of range.	BSSセクションの配置アドレスがリンク可能範囲外です。
Too many object files.	リンクするオブジェクトファイル数が4000個の制限を越えました。
Too many output sections.	出力セクション数が256個の制限を越えました。
Too many input sections.	デフォルトセクションに配置されるセクション数が4000個の制限を越えました。
Too many library files.	ライブラリファイル数が256個の制限を越えました。
Too many global bss symbol.	BSSセクションのグローバルシンボルが1024個の制限を越えました。
Too many object symbols.	オブジェクトシンボル数が36000個の制限を越えました。
Too many U sections.	Uセクション数が256個の制限を越えました。
Too many section symbols.	セクションシンボル数が256個の制限を越えました。
Too many U section symbols.	Uセクションシンボル数が256個の制限を越えました。
No object files.	リンクするオブジェクトファイルの指定がありません。
Not SRF33 Object file or library file.	入力ファイルがsrf33のオブジェクトファイルまたはライブラリファイルではありません。
Too many defaddr commands.	アドレスの定義が256個を越えています。
Too many output blockes.	ブロック名の定義が256個を越えています。
Address out of range near line =#.	指定可能なアドレス範囲は0x0～0xffffffffです。
Address out of range.	ブロックの配置アドレスが0xffffffffを越えています。
Too many specified files near line =#.	{ }内の指定ファイル数が4000個を越えています。
Too many addr commands.	-addrコマンドの定義が256個を越えています。
Too many specified blockes.	{ }内の指定ブロック数が256個を越えています。
Chain information size is greater than file size.	srf33オブジェクトファイルに問題があります。
Chain seek address is greater than file size.	srf33オブジェクトファイルに問題があります。
Undefined symbol type.	srf33オブジェクトファイルに問題があります。
Undefined relocation type.	srf33オブジェクトファイルに問題があります。

エラー/ワーニングメッセージ

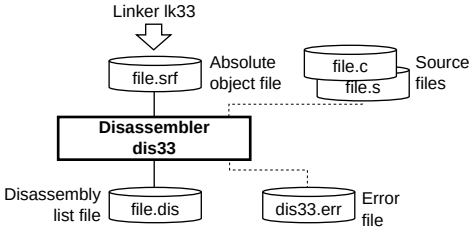
ワーニングメッセージ

'<symbol>' already defined in '<file name2>'.	<file name1>で定義している<symbol>はすでに<file name2>で定義済みです。
unresolved external symbol '<symbol>'.	未定義シンボルが参照されています。
Out of relocation range Address = <address>.	命令がリロケーション(再配置)可能範囲を越えています。
Section mapping conflict Address = <address> .	セクションが<address>から重複しています。
Cannot mapping to USection '<file name>(<section>)'	コマンドファイルに記述してある<file name>の<section>は再配置できません。
Cannot find relocatable section '<file name>(<section>)'.	<file name>のリロケータブルセクション<section>が見つかりません。
Alignment <address 1> -> <address 2>.	コマンドファイルで指定されている<address 1>を<address 2>としてアライメントを調整しました。
Cannot access section information.	セクション情報にアクセスできません。
Invalid loader information.	-dコマンド使用時に、-ucode、-udata、-ubssが指定されています。
-v3 command is already defined.	-v3コマンドが2つ以上あります。
'<symbol>' already address name defined.	同じシンボル名に異なったアドレスを定義しています。
'<symbol>' already block name defined.	同じブロック名を多重定義しています。
address already defined near line =#.	同じアドレス(またはシンボル)が異なったブロックに定義されています。
Cannot find link file '%s'.	指定されたファイル名がリンクファイルに見つかりません。
Address mapping conflict.	ブロックの配置が他の-addrのブロックアドレスと重複しています。
Cannot mapping to vertual block '<block name>'('<file name>').	仮想ブロック名(デフォルト以外)の中に"xxx.lib:xxx.o"形式のファイル名があります。

概要

リンカが出力するsrf33形式のオブジェクトファイルをディスアセンブルし、ニーモニックコードとソースが参照できるファイルを生成します。リンク後にソースファイルと絶対アドレスの対応を見たい場合などに有効です。

フローチャート



起動コマンド

```
Disassembler 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x.
Usage:
    dis33 [options] <file name>
Options:
    -e : produce log file (dis33.err)
    -m : generate disassemble code with source mix
    -c : generate disassemble code only
    -d : generate data dump only
    -a address1 address2
       : specify disassemble area
         address1 - start address, hexadecimal number
         address2 - end address, hexadecimal number
Output:
    Disassemble file (.dis)
    Log file (dis33.err)
Example:
    dis33 -e -m -a 0x80000 0x8ffff sample.srf
```

エラーメッセージ

Cannot open file, <file name>.	ファイルがオープンできません。
Cannot close file, <file name>.	ファイルがクローズできません。
Cannot write to file, <file name>.	ファイルへの書き込みができません。
Cannot close SROFF file.	オブジェクトファイルがクローズできません。
Cannot load data, memory allocation failure.	メモリを確保できないため、 セクション情報がロードできません。
Cannot load debug information, memory allocation failure.	メモリを確保できないため、 デバッグ情報がロードできません。
Cannot load data, file read failure, <file name>.	ファイルが読み込めないため、 セクション情報をロードできません。
Cannot load debug information, file read failure, <file name>.	ファイルが読み込めないため、 デバッグ情報がロードできません。
Invalid file name, <file name>.	入力ファイル名に".dis"が指定されました。
Too many sections.	srf33オブジェクトファイルに問題があります。
Cannot load data, please check SROFF file.	srf33オブジェクトファイルに問題があります。
Cannot load debug information, please check SROFF file.	srf33オブジェクトファイルに問題があります。

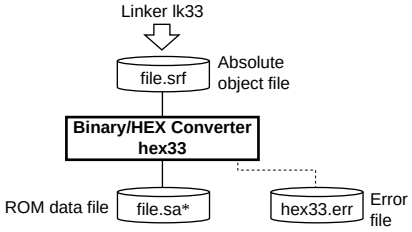
ワーニングメッセージ

No debug information.	入力ファイルにデバッグ情報がありません。
Cannot open file, <file name>.	ミックス出力時にソースファイルがオープンできません。
Line number of source file is invalid.	ソースファイルが変更され、行数が足りません。

概要

リンカが出力するsrf33形式のオブジェクトファイルを、ROM書き込み用およびマスクデータ作成用のモトローラS3フォーマットのHEXファイルに変換します。

フローチャート



起動コマンド

```
HEX Data Converter 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    hex33 [options] <start address> <end address> <file name>
Options:
    -e : produce log file (hex33.err)
    -x : add start and end addresses to the file name extension
    -z : make converted address absolute
    -r : check all data within start and end area
Output:
    hex file  (.sa, .sa_<start address>_<end address>)
    log file  (hex33.err)
Example:
    hex33 -x -z 80000 80fff sample.srf
```

エラーメッセージ

File open error.	ファイルがオープンできません。
File write error.	ファイルへの書き込みができません。
File read error.	ファイルの読み込みができません。
Memory allocation error.	メモリが確保できません。
"<file name>" file could not be opened.	ファイル<file name>をオープンできません。
Input file is not SRF33 file.	入力ファイルがsrf33形式のオブジェクトファイルではありません。
Start address error.	変換開始アドレスが不正です。
End address error.	変換終了アドレスが不正です。
Out of area in address <address>.	変換データが指定アドレス範囲を越えました。
Chain information size is greater than file size.	srf33オブジェクトファイルに問題があります。
Chain seek address is greater than file size.	srf33オブジェクトファイルに問題があります。
File control flag error.	srf33オブジェクトファイルに問題があります。
Section address error.	srf33オブジェクトファイルに問題があります。
Section ID error.	srf33オブジェクトファイルに問題があります。
Data conflicted at <address>.	srf33オブジェクトファイルに問題があります。

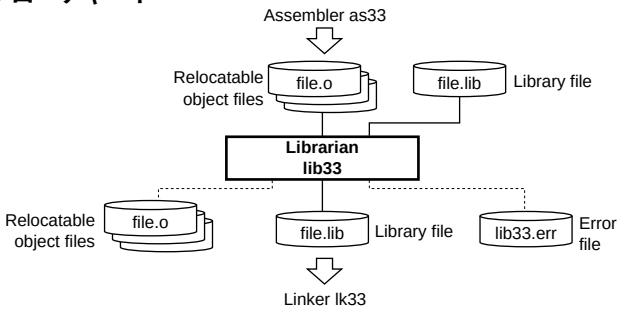
ワーニングメッセージ

Section information chain is not found.	srf33オブジェクトファイルに問題があります。

概要

ライブラリの編集ツールです。アセンブラが生成したオブジェクトファイルをライブラリに登録することができます。ライブラリからオブジェクトを削除したり、オブジェクトファイルに復元することもできます。

フローチャート



起動コマンド

```
Librarian 33 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    lib33 [option] <library-file> <files>
Options:
    -e : produce log file (lib33.err)
    -a : add new files after an existing member of the library
    -d : delete members from the library
    -l : display a table listing the contents of library
    -x : extract members from the library
Output:
    srf33 library file (.lib)
Example:
    lib33 -a libc.lib putc.o getc.o
```

エラーメッセージ

Cannot open file	ファイルがオープンできません。
Not srf33 library file	指定したライブラリファイルがsrf33形式ではありません。
Max object files	ライブラリ内のオブジェクト数が上限を越えました。(最大4000)

ワーニングメッセージ

Not srf33 object file	指定オブジェクトがsrf33形式ではありません。
Multiple object file '<file name>'	指定のオブジェクトファイルはすでに登録されています。
Cannot find '<file name>' in library	指定のオブジェクトファイルがライブラリ内に見つかりません。

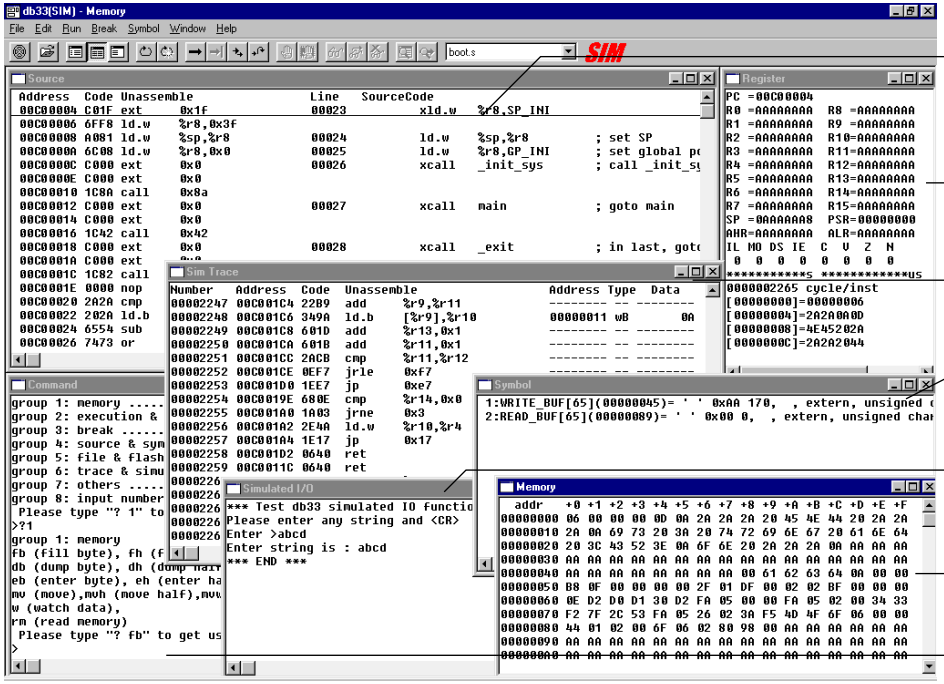
概要

S5U1C33000H、S5U1C33104H、S5U1C330M2Sを制御してデバッグを行うソフトウェアです。パソコン上でデバッグを行うシミュレーション機能も持っています。ブレークやステップ実行など、頻繁に使用するコマンドはツールバーに登録されており、キーボード操作の量を抑えています。また、Cソースレベルおよびアセンブリソースレベルデバッグに対応しており、各種のデータもマルチウィンドウとして表示できるため、デバッグ作業が効率良く行えます。

起動コマンド

```
 - usage -
db33.exe -p parameter_file [-MODE] [-c command_file] [-w] [-comX] [-b baud_rate] [-sf] [-lptX] [-tX]
    -p             ... parameter file
    -MODE          ... select debugger mode
      -sim           ... simulator mode
      -simdll        ... simulator + dll mode
      -mon           ... debug monitor mode
      -icd           ... ICD mode
      -ice           ... ICE mode
      default        ... ICE mode
    -c             ... command file
    -w             ... open only command window
    -comX (X:1-8) ... com port, default com1
    -b             ... baud rate, 4800, 9600, 19200, 38400(default), 57600, 115200 [bps]
                                  default baud rate   ICE mode 38400
                                  default baud rate   ICD mode 115200
                                  default baud rate   debug monitor mode 38400
    -sf            ... display with small font
    -lptX (X:1-2) ... parallel port
    -tX (X:1-20)  ... waiting time after port open
```

ウィンドウ



[Source]ウィンドウ
プログラムを逆アセンブルコード、ソースコードまたは両方をミックスした形で表示します。

[Register]ウィンドウ
レジスタ値、監視メモリの内容、実行カウンタの値を表示します。

[Trace]ウィンドウ
トレースデータを表示します。

[Symbol]ウィンドウ
監視シンボルの情報を表示します。

[Simulated I/O]ウィンドウ
シミュレーテッドI/O機能のデータ入力、データ出力に使用します。

[Memory]ウィンドウ
メモリデータを表示します。

[Command]ウィンドウ
デバッグコマンドの入力、コマンド実行結果の表示に使用します。

ツールバー

Key break 実行中のプログラムを強制ブレークします。

Load file srf33オブジェクトファイルをロードします。

Source [Source]ウィンドウをソース表示モードに切り換えます。

Mix [Source]ウィンドウをミックス表示モードに切り換えます。

Unassemble [Source]ウィンドウを逆アセンブル表示モードに切り換えます。

Reset cold CPUをコールドリセットします。

Reset hot CPUをホットリセットします。

Go プログラムを現在のPCアドレスから連続的に実行します。

Go to *1 プログラムを現在のPCアドレスから[Source]ウィンドウのカーソルが置かれたアドレスまで実行します。

Step プログラムを現在のPCアドレスから1ステップ実行します。

Next プログラムを現在のPCアドレスから1ステップ実行します。関数、サブルーチン、ソフトウェア割り込みルーチン中は1ステップとして実行されます。

Soft PC break *1 [Source]ウィンドウのカーソル位置のアドレスをソフトウェアPCブレークポイントに設定します。

Hard PC break *1 [Source]ウィンドウのカーソル位置のアドレスをハードウェアPCブレークポイントに設定します。

ツールバー

Symbol watch *1 [Source]ウィンドウのカーソル位置にあるシンボルの情報を[Command]ウィンドウに表示します。

Symbol add *1 [Source]ウィンドウのカーソル位置にあるシンボルを監視シンボルリストに登録し、そのシンボル情報を[Symbol]/[Command]ウィンドウに表示します。

Symbol delete *2 [Symbol]ウィンドウのカーソル位置にあるシンボルを監視シンボルリスト から削除します。

Display trace S5U1C33000Hによるトレースを中断し、トレースメモリ内のデータを[Trace]ウィンドウに表示します。

Resume trace [Display trace]で中断したトレースを再開します。

[Select source]コンボボックス
[Source]ウィンドウに表示させるソースを選択します。
リストされるファイル名は、読み込んだsrf33オブジェクトファイルのデバッグ情報に含まれているファイルです。
[Source]ウィンドウが逆アセンブルモードの場合も選択可能で、その場合は、選択したソースに対応するコードが表示されます。

ファンクションキー

[F3] (コマンド再表示)
[Command]ウィンドウに最後に入力したコマンドをプロンプトの位置に再表示します。再表示させたコマンドは[Enter]キーの入力で再実行できます。

*1: [Source]ウィンドウが開いている場合にのみ使用可能
*2: [Symbol]ウィンドウが開いている場合にのみ使用可能

メニュー

File
- Load file..　Ctrl+O
- Exit
- ✔ 1 Command
- 2 Register
- 3 Source

[File]メニュー

Load file... srf33オブジェクトファイルをロードします。
Exit db33を終了します。

リストからウィンドウ名を選択すると、そのウィンドウがアクティブになります。

Edit
- Paste　Ctrl+V
- Copy　Ctrl+C

[Edit]メニュー

Paste *1 コピーされているテキスト(コマンドやシンボルなど)を[Command]ウィンドウにペーストします。
Copy *2 [Source]ウィンドウと[Memory]ウィンドウからシンボル名などをコピーします。

Run
- Go
- Go to
- Step
- Next
- Step out
- Reset cold
- Reset hot

[Run]メニュー

Go プログラムを現在のPCアドレスから連続的に実行します。
Go to *3 プログラムを現在のPCアドレスから[Source]ウィンドウのカーソルが置かれたアドレスまで実行します。
Step プログラムを現在のPCアドレスから1ステップ実行します。
Next プログラムを現在のPCアドレスから1ステップ実行します。関数、サブルーチン、ソフトウェア割り込みルーチン中は連続実行します。
Step out 現在の関数やサブルーチンを終了させます。
Reset cold CPUをコールドリセットします。
Reset hot CPUをホットリセットします。

Break
- Soft PC..
- Hard PC...
- Data..
- Sequentil...
- Area break
- Bus break
- All clear

[Break]メニュー

Soft PC... ソフトウェアPCブレークポイントを設定します。
Hard PC... ハードウェアPCブレークポイントを設定します。
Data... データブレーク条件を設定します。
Sequential... シーケンシャルブレーク条件を設定します。
Area break エリアブレーク条件を設定します。
Bus break バスブレーク条件を設定します。
All clear ブレーク条件をすべて解除します。

メニュー

Symbol
- Watch
- Add
- Delete

[Symbol]メニュー

Watch *3 [Source]ウィンドウのカーソル位置にあるシンボルの情報を[Command]ウィンドウに表示します。
Add *3 [Source]ウィンドウのカーソル位置にあるシンボルを監視シンボルリストに登録し、そのシンボル情報を[Symbol]ウィンドウまたは[Command]ウィンドウに表示します。
Delete *4 [Symbol]ウィンドウのカーソル位置にあるシンボルを監視シンボルリスト から削除します。

Window
- Command
- Source
- Memory
- Register
- Trace
- Symbol
- StdIO

[Window]メニュー

選択したウィンドウを開きます。

Help
- About db33...

[Help]メニュー

About db33... db33のバージョンを表示します。

*1: [Command]ウィンドウにのみ有効
*2: [Source]ウィンドウと[Memory]ウィンドウで使用可能
*3: [Source]ウィンドウが開いている場合にのみ使用可能
*4: [Symbol]ウィンドウが開いている場合にのみ使用可能

デバッグコマンド

メモリ操作

fb [addr1 addr2 data]	領域のフィル (バイト単位)	ICD/ICE/SIM/MON
fh [addr1 addr2 data]	領域のフィル (ハーフワード単位)	ICD/ICE/SIM/MON
fw [addr1 addr2 data]	領域のフィル (ワード単位)	ICD/ICE/SIM/MON
db [addr1 [addr2]]	メモリダンプ (バイト単位)	ICD/ICE/SIM/MON
dh [addr1 [addr2]]	メモリダンプ (ハーフワード単位)	ICD/ICE/SIM/MON
dw [addr1 [addr2]]	メモリダンプ (ワード単位)	ICD/ICE/SIM/MON
df	ファイルへのメモリダンプ出力	ICD/ICE/SIM/MON
eb [addr1]	データ入力 (バイト単位)	ICD/ICE/SIM/MON
eh [addr1]	データ入力 (ハーフワード単位)	ICD/ICE/SIM/MON
ew [addr1]	データ入力 (ワード単位)	ICD/ICE/SIM/MON
mv [addr1 addr2 addr3]	領域のコピー (バイト単位)	ICD/ICE/SIM/MON
mvh [addr1 addr2 addr3]	領域のコピー (ハーフワード単位)	ICD/ICE/SIM/MON
mvw [addr1 addr2 addr3]	領域のコピー (ワード単位)	ICD/ICE/SIM/MON
w	監視データアドレス設定	ICD/ICE/SIM/MON
rm [addr1 addr2]	ターゲットメモリリード	ICD
mc [file]	srf33 ファイルとメモリの比較	ICD/ICE/SIM/MON

レジスタ操作

rd	レジスタ表示	ICD/ICE/SIM/MON
rs	レジスタ変更	ICD/ICE/SIM/MON

プログラム実行

g [addr]	連続実行	ICD/ICE/SIM/MON
s [step]	ステップ実行	ICD/ICE/SIM/MON
n [step]	スキップ付きステップ実行	ICD/ICE/SIM/MON
o	ステップアウト	ICD/ICE/SIM/MON
call/callh [func [arg1 [arg2 [arg3]]]]	ユーザ関数コール	ICD/ICE/SIM/MON
callmd	コールモードの設定	ICD/ICE/SIM/MON

CPUリセット

rstc	コールドリセット	ICD/ICE/SIM/MON
rsth	ホットリセット	ICD/ICE/SIM/MON

割り込み

int [type level]	割り込み発生	SIM

ブレーク

bp [no. status [addr]]	ソフトウェアPCブレークポイントの設定/解除	ICD/ICE/SIM/MON
bs [addr]	ソフトウェアPCブレークポイントの設定	ICD/ICE/SIM/MON
bc addr	ソフトウェアPCブレークポイントの解除	ICD/ICE/SIM/MON
bh [addr]	ハードウェアPCブレークポイント1の設定	ICD/ICE/SIM/MON
bhc	ハードウェアPCブレークポイント1の解除	ICD/ICE/SIM/MON
bh2 [addr]	ハードウェアPCブレークポイント2の設定	ICD/ICE/SIM/MON
bhc2	ハードウェアPCブレークポイント2の解除	ICD/ICE/SIM/MON
bph [no. status [addr]]	ハードウェアPCブレークポイントの設定/解除	ICD/ICE/SIM/MON
bd [mode addr {r | w | *}]	データブレークの設定	ICD/ICE/SIM/MON
bsq [mode hit [c1 [c2 [c3]]]]	シーケンシャルブレークの設定	ICE
ba	エリアブレークの設定	ICD+MEM
bb	バスブレークの設定	ICD+MEM
bl	全ブレーク条件の表示	ICD/ICE/SIM/MON
bac	全ブレーク条件の解除	ICD/ICE/SIM/MON

プログラム表示

u [addr]	逆アセンブル表示	ICD/ICE/SIM/MON
sc [addr]	ソース表示	ICD/ICE/SIM/MON
m [addr]	逆アセンブル&ソース表示	ICD/ICE/SIM/MON
ss string	ソース内の文字列検索	ICD/ICE/SIM/MON

シンボル情報

sy [option]	シンボル一覧	ICD/ICE/SIM/MON
sa [symbol [-switch]]	[Symbol]ウィンドウへの登録	ICD/ICE/SIM/MON
sd no.	[Symbol]ウィンドウからの削除	ICD/ICE/SIM/MON
sw symbol [-switch]	シンボル情報の表示	ICD/ICE/SIM/MON
sw scope	シンボル情報一覧の表示	ICD/ICE/SIM/MON

ファイル読み込み

lf [file]	srf33形式ファイルの読み込み	ICD/ICE/SIM/MON
lh [file offset]	モトローラS3形式ファイルの読み込み	ICD/ICE/SIM/MON
ld [file]	デバッグ情報の読み込み	ICD/ICE/SIM/MON

フラッシュメモリ操作

fls	ターゲットフラッシュメモリの設定	ICD/ICE/MON
fle	ターゲットフラッシュメモリの消去	ICD/ICE/MON
lfl	S5U1C33104Hフラッシュメモリの読み出し	ICE
sfl	S5U1C33104Hフラッシュメモリへの書き込み	ICE
efl	S5U1C33104Hフラッシュメモリ消去	ICE
maf	S5U1C33104Hフラッシュメモリマップ情報の表示	ICE

トレース

tm	トレースモードの設定	ICD/ICE/SIM
tm2	シミュレータトレースモードの設定	SIM
td [no.]	トレース情報の表示	ICD/ICE
ts	トレース情報の検索	ICD/ICE
tf	トレース情報の保存	ICD/ICE

シミュレーテッドI/O

stdin	データ入力シミュレーション	ICD/ICE/SIM/MON
stdout	データ出力シミュレーション	ICD/ICE/SIM/MON

その他

com [file]	コマンドファイル実行	ICD/ICE/SIM/MON
cmw [file]	コマンドファイルのウェイト付実行	ICD/ICE/SIM/MON
log [file]	ログ出力ON/OFF	ICD/ICE/SIM/MON
od	オプションデータダンプ	ICE
ct [[option] value]	データ形式変換表示	ICD/ICE/SIM/MON
ext [addr]	拡張命令への逆変換	ICD/ICE/SIM/MON
ma	マップ情報の表示	ICD/ICE/SIM/MON
md	モード設定	ICD/ICE/SIM/MON
alias [key [string]]	コマンドエイリアス	ICD/ICE/SIM/MON
exit	終了	ICD/ICE/SIM/MON
?	コマンドusageの表示	ICD/ICE/SIM/MON

コマンドパラメータの入力形式

数値

使用可能文字	デフォルト: [0~9] 10進数: +[0~9] 16進数: 0x[0~9, a~f, A~F]
デフォルト形式	10進数 ("+"は省略可)
有効範囲	コマンドにより異なる

データ

使用可能文字	デフォルト: [0~9, a~f, A~F] 10進数: +[0~9] 16進数: 0x[0~9, a~f, A~F]
デフォルト形式	16進数 ("0x"は省略可)
有効範囲	コマンドにより異なる

アドレス

使用可能文字	デフォルト: [0~9, a~f, A~F] 10進数: +[0~9] 16進数: 0x[0~9, a~f, A~F] シンボル: シンボル参照 行番号: 行番号参照
デフォルト形式	16進数 ("0x"は省略可)
有効範囲	0x0~0x0ffffffff

行番号

形式	#<line number> <file name>#<line number>
使用可能文字	[0~9] (符号なし10進数のみ)
有効範囲	1~ソースファイルの行数

シンボル

使用可能文字	*, _, a~z, A~Z, 0~9, ., ->, [,] 例) i *fp ar[2] st->mem st.mem
シンボル検索パス	カレントブロック→カレント関数→カレントファイル→外部シンボル→16進数
スコープ指定	[file]/[function]/symbol (".": カレントファイル、カレント関数を指定) //symbol 外部シンボル /./symbol カレント関数内のオート/スタティック変数 .//symbol カレントファイル内のスタティック変数 file//symbol 指定ファイル内のスタティック変数 /function/symbol 指定関数内のオート/スタティック変数 ./function/symbol カレントファイル内指定関数のオート/スタティック変数 file/function/symbol 指定ファイル内指定関数のオート/スタティック変数

パラメータファイル

パラメータ

```
CHIP        <name>                  チップ名(33XXX)
IROM        <size>                  内蔵ROMサイズ(0–20000)
FOPT        <size>                  オプションサイズ(0–3fff)
PRC VER     <ver1> <ver2>           S5U1C33xxxPボードバージョン範囲(0–ff)
PRC STATUS  ***************         S5U1C33xxxPボードステータス(16ビット), [0, 1, *]
{MCU | MPU}                         ブートアドレス(MCU=0x80000, MPU=0xc00000)
VER         <ver>                   ユーザー用パラメータファイルバージョン(0–ff)

; Emulation memory allocation (max 8 areas, 1MB/area, 1MB boundary)

EMRAM       #00000 #fffff           RAMエミュレーション領域(#=0–ff), W only
EMROM       #00000 #fffff           ROMエミュレーション領域(#=0–ff), R/W

; Map allocation (max 31 areas, 256-byte boundary)

RAM   <start addr> <end addr> [<wait>] [B]  内蔵RAM, ターゲットボードRAMマップ, R/W
IO    <start addr> <end addr> [<wait>] [B]  内蔵I/O, ターゲットボードI/Oマップ, R/W
ROM   <start addr> <end addr> [<wait>] [B]  ターゲットボードROMマップ, W only
ERAM  <start addr> <end addr> [<wait>] [B]  エミュレーションメモリマップ(RAM), R/W
EROM  <start addr> <end addr> [<wait>] [B]  エミュレーションメモリマップ(ROM), W only
EIO   <start addr> <end addr> [<wait>] [B]  エミュレーションメモリマップ(I/O), R/W
                                            <wait>はシミュレータモード時のウェイト指定
                                            "B"はビッグエンディアン指定

; Stack area except internal RAM area (max 8 areas, 256-byte boundary)

STACK <bottom addr> <top addr>              スタックマップ

; MEM33 configuration

;!MEM33_CE  9                               S5U1C330Mアドレス(CE No.)
;!MEM33_CEFUNC  00                          CEFUNC値
;!MEM33_DELAY  2                            S5U1C330Mディレイ
;!MEM33_CE10EX  c00000                      エリア10外部メモリ先頭アドレス
;!MEM33_P30                                 P30をCEとして使用
;!MEM33_P34                                 P34をCEとして使用
;!MEM33_CE7_DRAM                            エリア7(13)にDRAMを使用
;!MEM33_CE8_DRAM                            エリア8(14)にDRAMを使用
;!MEM33_WRH_MASK                            WRHマスク
;!MEM33_MAP_DISABLE                         S5U1C330Mマップブレークディセーブル
;!MEM33_CE_DISABLE                          S5U1C330M CEブレークディセーブル
;!MEM33_NOCE_DISABLE                        S5U1C330M NO CEブレークディセーブル

END                                         エンドマーク
```

ステータス/エラー/ワーニングメッセージ

ブレークステータスメッセージ

Break by software PC break.	ソフトウェアPCブレークポイントによりブレーク
Break by hardware PC break.	ハードウェアPCブレークポイント1によりブレーク
Break by hardware PC break2.	ハードウェアPCブレークポイント2によりブレーク
Break by temporary break.	gコマンドのテンポラリブレークポイントによりブレーク
Break by data break.	データブレーク条件によりブレーク
Break by read memory.	メモリリードによるデータブレーク
Break by write memory.	メモリライトによるデータブレーク
Break by sequential break.	シーケンシャルブレーク条件によりブレーク
Break by key break, xxxx.	[Key break]ボタンによりアドレスxxxxでブレーク
Break by accessing no map area.	マップされていない領域へのアクセスによりブレーク
Break by writing ROM area.	リードオンリー領域への書き込みによりブレーク
Break by out of SP area.	スタック領域外に対するスタック操作によりブレーク
Break by external break.	S5U1C33104H/S5U1C33000HのBRKIN端子への信号入力によりブレーク
Break by illegal instruction.	シミュレータモードで不当命令の実行によりブレーク
Break by MEM33 bus break.	S5U1C33xxxMのバスブレーク条件によりブレーク
Break by MEM33 ce break.	S5U1C33xxxMのCEブレーク条件によりブレーク
Break by MEM33 area break.	S5U1C33xxxMのエリアブレーク条件によりブレーク
Break by MEM33 map break.	S5U1C33xxxMのマップブレーク条件によりブレーク

エラーメッセージ

Address is in no map area.	アドレス(シンボル)がマップ領域外です。
Address is not 2 byte boundary.	プログラムコードのアドレスが2バイト境界ではありません。
Address mask range (0-0xFFFFFF).	アドレスマスクが有効範囲外です。
Address range (0-0xFFFFFFF).	入力アドレスは有効範囲外です。
address1 > address2.	開始アドレスが終了アドレスを越えています。
Already exist input address.	設定済みのブレークアドレスを指定しました。
Argument1 range (0-0xFFFFFFF).	引数1が有効範囲外です。
Argument2 range (0-0xFFFFFFF).	引数2が有効範囲外です。
Argument3 range (0-0xFFFFFFF).	引数3が有効範囲外です。
Break number (1-16).	ソフトウェアPCブレークポイント番号が有効範囲外です。
Break number (1-8).	ハードウェアPCブレークポイント番号が有効範囲外です。
Cannot add symbol any more.	すでに99個のシンボルが登録済みです。
Cannot allocate memory.	メモリの割り付けができません。
Cannot close file.	ファイルがクローズできません。
Cannot compare data, file open failure, xxx.srf.	xxx.srfがオープンできません。
Cannot get file status.	ファイル情報が不正です。
Cannot get input, please check the system.	入力処理でエラーが発生しました。
Cannot get memory.	メモリ確保に失敗しました。
Cannot load data, file open failure srf33.	ファイルのロードに失敗しました。ファイルがオープンできません。
Cannot load data, file read failure srf33.	ファイルのロードに失敗しました。srf33ファイルが読み込めません。
Cannot load data, memory allocation failure srf33.	ファイルのロードに失敗しました。メモリが確保できません。
Cannot load data, please check SRF33 file srf33.	ファイルのロードに失敗しました。srf33実行形式以外のファイルが指定されました。
Cannot load debug information, debug information is wrong.	デバッグ情報のロードに失敗しました。デバッグ情報が不正です。(プログラム/データの読み込みは成功)
Cannot load debug information, file open failure.	デバッグ情報のロードに失敗しました。ソースファイルがオープンできません。(プログラム/データの読み込みは成功)
Cannot load debug information, file read failure.	デバッグ情報のロードに失敗しました。ソースファイルが読み込めません。(プログラム/データの読み込みは成功)
Cannot load debug information, memory allocation failure.	デバッグ情報のロードに失敗しました。メモリが確保できません。(プログラム/データの読み込みは成功)
Cannot load debug information, please check SRF33 file.	デバッグ情報のロードに失敗しました。srf33形式が不正です。(プログラム/データの読み込みは成功)
Cannot load debug information, too many lines.	デバッグ情報のロードに失敗しました。ソースファイルの行数が多すぎます。(プログラム/データの読み込みは成功)

ステータス/エラー/ワーニングメッセージ

エラーメッセージ

Cannot open com port.	パソコンのCOMポートが開けません。またはボーレートの設定ができません。
Cannot open file any more.	これ以上ファイルをオープンできません。
Cannot open file.	ファイルをオープンできません。
Cannot open parallel port.	パソコンのパラレルポートが開けません。
Cannot open parameter file.	パラメータファイルがオープンできません。
Cannot open stdio file.	stdioのファイルがオープンできません。
Cannot read chip name.	パラメータファイルのチップ名情報が読み込めません。
Cannot read emulation memory map.	パラメータファイルのエミュレーションメモリマップ情報が読み込めません。
Cannot read FO area size.	パラメータファイルのFOPTサイズ情報が読み込めません。
Cannot read IROM size.	パラメータファイルの内蔵ROM領域サイズ情報が読み込めません。
Cannot read MCU/MPU information.	パラメータファイルのMCU/MPUモード情報が読み込めません。
Cannot read parameter file version.	パラメータファイルのバージョン情報が読み込めません。
Cannot read PRC board status.	パラメータファイルのS5U1C33xxxPボードステータス情報が読み込めません。
Cannot read PRC board version.	パラメータファイルのS5U1C33xxxPボードバージョン情報が読み込めません。
Cannot read stdin file.	stdinのファイルが読み込めません。
Cannot resist address any more.	すでに8個所のハードウェアPCブレークが登録されています。
Cannot set address any more.	すでに16個所のソフトウェアPCブレークポイントが設定されています。
Cannot set alias anymore.	すでに100件のエイリアスが登録されています。
Cannot set enable address anymore.	すでに2個所のハードウェアPCブレークが設定されています。
Cannot set hardware PC break.	ハードウェアPCブレークポイントが設定できません。
Cannot set software PC break.	ソフトウェアPCブレークポイントが設定できません。
Cannot set temporary break.	テンポラリブレークポイントが設定できません。
Cannot write flash memory.	フラッシュメモリに書き込めません。
Cannot write log file.	ログファイルに書き込めません。
Cannot write stdout file.	stdoutのファイルにデータが書き込めません。
Cannot write trace file.	トレースファイルに書き込めません。
Chip name should be 5 characters.	チップ名が5文字ではありません。
Chip name should be start with "33".	チップ名が"33"で始まっていません。
Command is not supported at present mode.	現モードで未対応のコマンドを実行しました。
Communication data size error.	通信データのサイズが不正です。
Communication error.	S5U1C33104Hとの通信中にオーバーラン、フレーミング、またはBCCエラーが発生しました。

エラーメッセージ

CPU down.	S5U1C33xxxPボードの動作不良です。
CPU is not running.	S5U1C33104HのCPUが動作していません。
CPU is running.	S5U1C33104HのCPUが実行中です。
Current mode is not source mode.	ソース表示モード以外で文字列の検索はできません。
Data alignment error.	srf33ファイルのアライメントが不正です。
Data incomplete.	ファイルの構造が不正です。
Data mask range (0-0xFFFF).	データマスクが有効範囲外です。
Data mask range (0-0xFFFFFFFF).	データマスクが有効範囲外です。
Data range (0-0xFF).	入力データは有効範囲外です。
Data range (0-0xFFFF).	入力データは有効範囲外です。
Data range (0-0xFFFFFFFF).	入力データは有効範囲外です。
Debug data failure.	デバッグデータが異常です。
Diagnostic test failure.	S5U1C33104Hの自己診断でエラーが発生しました。
Duplicate input address.	ブレークアドレスを重複して指定しました。
Duplicate input break number.	ブレークポイント番号を重複して指定しました。
Empty file.	ファイルにデータがありません。
Erase routine is not set.	フラッシュメモリ消去ルーチンが設定されていません。
File end during guidance input.	コマンドファイル内のガイダンスの記述が途中で終了しています。
File not found.	指定ファイルが見つかりません。
Flash memory error.	フラッシュメモリへの書き込みまたは消去ができません。
Flash memory is not mapped.	フラッシュメモリがマップされていません。
Flash ROM is protected.	フラッシュメモリにプロテクトがかかっています。
FO address range (0-0x3FFF).	オプションダンプアドレスが有効範囲外です。
Format error.	フォーマットが異常です。
Function not found.	関数が見つかりません。
ICE is busy.	S5U1C33104Hが処理中です。
ICE is free run mode.	S5U1C33104Hがフリーランモードです。
ICE is maintenance mode.	S5U1C33104Hが保守モードです。
ICE is not mapped.	S5U1C33104Hがマップされていません。
ICE system error.	S5U1C33104Hのシステムエラーです。
Interrupt level (0-15).	割り込みレベルが有効範囲外です。
Interrupt type (0-215).	割り込みタイプが有効範囲外です。
Invalid address or invalid CEFUNC of parameter file.	不正なアドレスまたは、CEFUNCで指定したアドレスの有効範囲外です。
Invalid break address.	未設定のブレークアドレスを指定しました。
Invalid break number.	未設定のブレークポイント番号を指定しました。
Invalid command or parameter.	コマンドまたはパラメータが不正です。
Invalid emulation memory map.	パラメータファイルのエミュレーションメモリマップ情報が不正です。
Invalid file name.	ファイル名が不正です。

ステータス/エラー/ワーニングメッセージ

エラーメッセージ

Invalid group number.	コマンドグループ番号が不正です。
Invalid map command or invalid sequence.	マップファイルに不正な文字がある、または順序が不適当です。
Invalid memory map.	パラメータファイルのメモリマップ情報が不正です。
Invalid parameter.	パラメータが不正です。
Invalid stack map.	パラメータファイルのスタックマップ情報が不正です。
Invalid value of MEM33_CE10EX (C00000-FFFFFF).	エリア10外部メモリスタートアドレスが有効範囲外です。
Invalid value of MEM33_CEFUNC (00,01,10,11).	CEFUNC値が有効範囲外です。
Invalid value of MEM33_DELAY (0-3).	ディレイ値が有効範囲外です。
Invalid value (0-65535).	有効範囲外の値です。
Invalid value.	入力数値が不正です。
IROM size is too long.	IROMサイズが大きすぎます。
MEM33 is not supported at area0, area1 and area2 (0-0x7FFFF).	S5U1C33xxxMは、エリア0, 1, 2をサポートしていません。
Memory verify failure.	メモリとの比較でエラーになりました。
No "END" in parameter file.	パラメータファイルにエンドマーク(END)がありません。
No "MEM33_CEFUNC" in parameter file.	MEM33_CEFUNCの指定がありません。
No map area.	入力アドレスはマップ領域外です。
No symbol at the number.	未登録のシンボル番号を指定しました。
Not ASCII character.	文字列の指定にASCIIキャラクタ以外の文字が含まれています。
Not defined ID.	S5U1C33104Hの応答IDが無効です。
Not found input strings.	入力文字列が見つかりません。
Number of emulation memory is wrong.	パラメータファイルのエミュレーションメモリ数が不正です。
Number of parameter.	パラメータの数が不正です。
On tracing.	S5U1C33104Hが実行データをトレース中です。
Parallel interface time out.	パラレルポートによるファイルのロード中にタイムアウトとなりました。
Parallel port is busy.	パラレルポートが使用中です。
Pointer pointed no map area.	ポインタ変数がマップ領域外を指しています。
Post line range (0-256).	トレース検索結果の後データ表示行数が有効範囲外です。
Pre line range (0-256).	トレース検索結果の前データ表示行数が有効範囲外です。
Register variable cannot be changed to address.	レジスタ変数シンボルでアドレスを指定しました。
Reset timeout.	S5U1C33104HのCPUがリセットできません。
Sequential break format error.	S5U1C33104Hに設定するシーケンシャルブレーク条件が不正です。

エラーメッセージ

Shared RAM is busy.	S5U1C33104Hの内部エラーです。
Source window not opened.	[Source]ウィンドウが閉じています。
Start block > End block.	開始ブロック番号が終了ブロック番号を越えています。
Start cycle number > End cycle number.	開始サイクル番号が終了サイクル番号を越えています。
Stdout data size.	出力バッファのデータサイズが不正です。
Step range (1-65535).	シングルステップ数が有効範囲外です。
Symbol is too long.	シンボル名が長すぎます。
Symbol not found, xxx.	シンボルxxxが見つかりません。
Symbol not found.	シンボルが見つかりません。
Symbol not in scope.	シンボルがスコープ内にありません。
Target down.	S5U1C33xxxPボードが正常に動作していません。またはリセットが解除されません。
Target instruction cannot be extended.	EXTで拡張できない命令です。
Time out.	S5U1C33104Hとの通信中にタイムアウトが発生しました。
Too many include file.	インクルードファイルの数が制限を越えました。
Too many source file.	ソースファイルが長すぎます。
Top address > Bottom address.	先頭アドレスが終了アドレスを越えています。
Top address and bottom address should be same CE.	先頭アドレスと終了アドレスは同一エリアでありません。
Trace range (0-131071).	トレースサイクル番号が無効です。(ICDモード)
Trace range (0-32767).	トレースサイクル番号が無効です。(ICEモード)
Verify error.	フラッシュメモリ書き込み時にベリファイエラーが発生しました。
Wrong data.	ファイルのデータが不正です。
Wrong header.	ファイルのヘッダが不正です。

ステータス/エラー/ワーニングメッセージ

ワーニングメッセージ

Area10 external memory start address is not 32k byte boundary (XXX).	エリア10外部メモリスタートアドレスが32Kバイトバウンダリではありません。
Cannot set CE break (XXX-XXX).	CEブレークをS5U1C33xxxMに設定できませんでした。
Cannot set map break (XXX-XXX).	MAPブレークをS5U1C33xxxMに設定できませんでした。
Debugger mode does not match with a target.	起動オプション(-icd、-ice、-mon)で指定したモードと接続されているデバッグターゲットが異なります。
Duplicate MEM33 address (CE) with memory map (XXX-XXX).	S5U1C33xxxMのアドレスとメモリマップが重複しています。
Duplicate memory map attribute (CE break as RAM XXX-XXX).	同一エリアにROM、RAM、IOが混在しています。
Emulation memory address is not 1M byte boundary.	パラメータファイルのエミュレーションメモリ情報で1Mバイト境界以外のアドレスが指定されています。
FO size (0-0x4000), map as 0x4000.	FOサイズ情報が不正なため、0x4000としてマップしました。
FO size should be an even number, map as 0xXXXXXXXX.	FOサイズが奇数のため、0xXXXXXXXXとしてマップしました。
Invalid line, move to next valid line.	指定の行番号にはアドレスがありません。直後の有効アドレスを使用します。
IROM size (0-0x80000), map as 0x80000.	IROMサイズ情報が不正なため、0x80000としてマップしました。
Line number of source file is invalid.	ソースファイルに指定の行番号はありません。
MEM33 break information was over written by user program.	S5U1C33xxxMにユーザプログラムが書き込みをし、ブレーク情報を破壊した可能性があります。
MEM33 not found.	パラメータファイルで指定されたアドレスにS5U1C33xxxMがありません。
Memory map is not 256 byte boundary.	パラメータファイルのメモリマップ情報で256バイト境界以外のアドレスが指定されています。
Memory map is not 32k byte boundary (XXX-XXX).	メモリマップが32Kバイト単位ではありません。
No debug information.	srf33ファイルにデバッグ情報がありません。
No source, display on mix mode.	ソース情報がありません。プログラムはミックス表示されます。
Not found "Return Code".	プログラム中にリターンコードが見つかりません。
Number of source line exceeded 65535.	行番号が有効範囲外です。
PRC status does not match.	S5U1C33xxxPボードのステータスがパラメータファイルの指定と異なります。
PRC version does not match.	S5U1C33xxxPボードのバージョンがパラメータファイルの指定と異なります。

ワーニングメッセージ

Round down to multiple of 16.	入力アドレスは16バイト境界にアライメントされました。
Round down to multiple of 2.	入力アドレスは2バイト境界にアライメントされました。
Round down to multiple of 4.	入力アドレスは4バイト境界にアライメントされました。
Stack map is not 256 byte boundary.	パラメータファイルのスタックマップ情報で256バイト境界以外のアドレスが指定されています。

概要

コンパイルからアセンブルまでをmakeファイル内に記述された依存リストに従って自動実行します。実行内容を記述するmakeファイルはワークベンチwb33によって生成可能です。

起動コマンド

```
Make for 33/63 Ver x.xx
Copyright (C) SEIKO EPSON CORP. 199x
Usage:
    make [options] [target]
Options:
    -f <file name> : makefile name
    -h  : output usage
    -n  : no command execution
Example:
    make -f test.mak
    make -f test.mak CLEAN
```

エラーメッセージ

Cannot open XXXXXXXX	makeファイルがオープンできません。
Cannot open tmp file	テンポラリファイルがオープンできません。
Invalid syntax near line #	不正な文法で記述されています。
Invalid suffix .XXX	サフィックスが未定義です。
Invalid macro name XXXX	マクロ名が未定義です。
Abnormal termination in XXXX	モジュール処理中にエラーが発生しました。
Not enough memory	メモリが不足しています。
Too long string	1つの文字列が1000文字を越えています。
No target found	ターゲットが見つかりません。
Don't know how to make XXXX	ターゲットファイルがありません。
Command exit with X	コマンドが異常終了しました。

ワーニングメッセージ

XXXX is up-to-date	最終ターゲットはすでに更新されています。

makeファイルの文法

依存リスト1

```
<target file>:  <dependent file 1> [<dependent file 2> ....]
[ TAB           <command 1>
  TAB           <command 2>
                      :          ]
```

コマンドは以下の場合に実行されます。
- ターゲットファイルが作成されていない場合
- 依存ファイルがターゲットファイルよりも新しい場合

コマンドを省略すると、対応するサフィックスリストのコマンドを実行します。

依存リスト2

```
<target name>:
  TAB           <command 1>
[ TAB           <command 2>
                      :          ]
```

ターゲット名はラベルとして使用されます。

サフィックス定義

```
.SUFFIXES: .xxx  .yyy  .zzz  ....
```

サフィックスリスト

```
<.suffix of dependent file 1> <.suffix of target file>:
  TAB           <command 1>
[ TAB           <command 2>
                      :          ]
```

マクロ定義

```
<macro name> = <macro body>
```

マクロ参照

```
$(<macro name>)
```

シンボル

@<command>	コマンドのエコーをOFFします。
-<command>	実行結果(エラーコード)を無視します。
$*	ターゲットファイル名(拡張子なし)に置き換わります。
$@	ターゲットファイル名(拡張子あり)に置き換わります。

不動小数点エミュレーションライブラリ fp.lib

Double型演算

__adddf3	**加算**	(%r11, %r10) ← (%r13, %r12) + (%r15, %r14)
__subdf3	**減算**	(%r11, %r10) ← (%r13, %r12) - (%r15, %r14)
__muldf3	**乗算**	(%r11, %r10) ← (%r13, %r12) * (%r15, %r14)
__divdf3	**除算**	(%r11, %r10) ← (%r13, %r12) / (%r15, %r14)
__negdf2	**符号反転**	(%r11, %r10) ← -(%r13, %r12)

Float型演算

__addsf3	**加算**	%r10 ← %r12 + %r13
__subsf3	**減算**	%r10 ← %r12 - %r13
__mulsf3	**乗算**	%r10 ← %r12 * %r13
__divsf3	**除算**	%r10 ← %r12 / %r13
__negsf2	**符号反転**	%r10 ← -%r12

型変換

__fixunsdfsi	double → unsigned int	%r10 ← (unsigned int) (%r13, %r12)
__fixdfsi	double → int	%r10 ← (int) (%r13, %r12)
__floatsidf	int → double	(%r11, %r10) ← (double) %r12
__fixunssfsi	float → unsigned int	%r10 ← (unsigned int) %r12
__fixsfsi	float → int	%r10 ← (int) %r12
__floatsisf	int → float	%r10 ← (float) %r12
__truncdfsf2	double → float	%r10 ← (float) (%r13, %r12)
__extendsfdf2	float → double	(%r11, %r10) ← (double) %r12

比較

__fcmpd	double type	Changes %psr by (%r13, %r12) - (%r15, %r14)
__fcmps	float type	Chenges %psr by %r12 - %13

整数除算ライブラリ idiv.lib

整数除算

__divsi3	**符号付き除算**	%r10 ← %r12 / %r13
__udivsi3	**符号なし除算**	%r10 ← %r12 / %r13

剰余演算

__modsi3	**符号付き演算**	%r10 ← %r12 % %r13
__umodsi3	**符号なし演算**	%r10 ← %r12 % %r13

不動小数点データフォーマット

Double型データ

63	62 … 52	51 … 0
S	**指数部**	**仮数部**

Double型有効範囲

+0:	0.0e+0 0x00000000 00000000
-0:	-0.0e+0 0x80000000 00000000
最大正規化数:	1.79769e+308 0x7fefffff ffffffff
最小正規化数:	2.22507e-308 0x00100000 00000000
最大非正規化数:	2.22507e-308 0x000fffff ffffffff
最小非正規化数:	4.94065e-324 0x00000000 00000001
無限大:	0x7ff00000 00000000
-無限大:	0xfff00000 00000000

Float型データ

31	30 … 23	22 … 0
S	**指数部**	**仮数部**

Float型有効範囲

+0:	0.0e+0f 0x00000000
-0:	-0.0e+0f 0x80000000
最大正規化数:	3.40282e+38f 0x7f7fffff
最小正規化数:	1.17549e-38f 0x00800000
最大非正規化数:	1.17549e-38f 0x007fffff
最小非正規化数:	1.40129e-45f 0x00000001
無限大:	0x7f800000
-無限大:	0xff800000

io.lib (ヘッダファイル: stdio.h)

fopen( )	FILE *fopen(char *filename, char *mode);	(dummy)∗1
freopen( )	FILE *freopen(char *filename, char *mode, FILE *stream);	(dummy)∗1
fclose( )	int fclose(FILE *stream);	(dummy)
fflush( )	int fflush(FILE *stream);	(dummy)
fseek( )	int fseek(FILE *stream, long int offset, int orign);	(dummy)∗1
ftell( )	long int ftell(FILE *stream);	(dummy)
rewind( )	void rewind(FILE *stream);	(dummy)
fgetpos( )	int fgetpos(FILE *stream, fpos_t *ptr);	(dummy)
fsetpos( )	int fsetpos(FILE *stream, fpos_t *ptr);	(dummy)∗1
fread( )	size_t fread(void *ptr, size_t size, size_t count, FILE *stream);	∗1, ∗2
fwrite( )	size_t fwrite(void *ptr, size_t size, size_t count, FILE *stream);	∗1, ∗2
fgetc( )	int fgetc(FILE *stream);	∗2
getc( )	int getc(FILE *stream);	∗1, ∗2
getchar( )	int getchar( );	∗1, ∗2
ungetc( )	int ungetc(int c, FILE *stream);	∗1
fgets( )	char *fgets(char *s, int n, FILE *stream);	∗1, ∗2
gets( )	char *gets(char *s);	∗1, ∗2
fputc( )	int fputc(int c, FILE *stream);	∗2
putc( )	int putc(int c, FILE *stream);	∗1, ∗2
putchar( )	int putchar(int c);	∗1, ∗2
fputs( )	int fputs(char *s, FILE *stream);	∗1, ∗2
puts( )	int puts(char *s);	∗1, ∗2
remove( )	int remove(char *filename);	(dummy)∗1
rename( )	int rename(char *oldname, char *newname);	(dummy)∗1
setbuf( )	void setbuf(FILE *stream, char *buf);	(dummy)
setvbuf( )	int setvbuf(FILE *stream, char *buf, int type, size_t size);	(dummy)
tmpfile( )	FILE *tmpfile( );	(dummy)∗1
tmpnam( )	char *tmpnam(char *buf);	(dummy)∗1
feof( )	int feof(FILE *stream);	(dummy)
ferror( )	int ferror(FILE *stream);	(dummy)
clearerr( )	void clearerr(FILE *stream);	(dummy)
perror( )	void perror(char *s);	∗1, ∗2
fscanf( )	int fscanf(FILE *stream, char *format, ...);	∗1, ∗2
scanf( )	int scanf(char *format, ...);	∗1, ∗2
sscanf( )	int sscanf(char *s, char *format, ...);	∗1, ∗2
fprintf( )	int fprintf(FILE *stream, char *format, ...);	∗1, ∗2
printf( )	int printf(char *format, ...);	∗1, ∗2
sprintf( )	int sprintf(char *s, char *format, ...);	∗1, ∗2
vfprintf( )	int vfprintf(FILE *stream, char *format, va_list arg);	∗1, ∗2
vprintf( )	int vprintf(char *format, va_list arg);	∗1, ∗2
vsprintf( )	int vsprintf(char *s, char *format, va_list arg);	

lib.lib (ヘッダファイル: stdlib.h, time.h)

malloc( )	void *malloc(size_t size);	∗1
calloc( )	void *calloc(size_t elt_count, size_t elt_size);	∗1
free( )	void free(void *ptr);	∗1
realloc( )	void *realloc(void *ptr, size_t size);	∗1
system( )	int system(char *command);	
exit( )	void exit(int status);	
abort( )	void abort( );	
atexit( )	int atexit(void (*func)(void));	
getenv( )	char *getenv(char *str);	
bsearch( )	void *bsearch(void *key, void *base, size_t count, size_t size, int (*compare)(void *, void *));	
qsort( )	void qsort(void *base, size_t count, size_t size, int (*compare)(void *, void *));	
abs( )	int abs(int x);	
labs( )	long int labs(long int x);	
div( )	div_t div(int n, int d);	∗1
ldiv( )	ldiv_t ldiv(int n, int d);	∗1
rand( )	int rand( );	
srand( )	void srand(unsigned int seed);	
atol( )	long int atol(char *str);	
atoi( )	int atoi(char *str);	∗1
atof( )	double atof(char *str);	∗1
strtod( )	double strtod(char *str, char **ptr);	∗1
strtol( )	long int strtol(char *str, char **ptr, int base);	∗1
strtoul( )	unsigned long int strtoul(char *str, char **ptr, int base);	∗1
setjmp( )	int setjmp(jmp_buf *ptr);	
longjmp( )	void longjmp(jmp_buf *ptr, int status);	
clock( )	clock_t clock( );	(dummy)
asctime( )	char *asctime(struct tm *ts);	(dummy)
ctime( )	char *ctime(time_t *timeptr);	(dummy)
difftime( )	double difftime(time_t ti, time_t t2);	(dummy)
gmtime( )	struct tm *gmtime(time_t *t);	
localtime( )	struct tm *localtime(time_t *t);	(dummy)
mktime( )	time_t mktime(struct tm *tmptr);	
time( )	time_t time(time_t *tptr);	∗1

∗1 **グローバル変数の宣言と初期化が必要です。**
∗2 **下位レベル関数と入出力バッファの定義が必要です。**

math.lib (ヘッダファイル: math.h, errno.h, float.h, limits.h)

fabs( )	double fabs(double x);	∗1
ceil( )	double ceil(double x);	∗1
floor( )	double floor(double x);	∗1
fmod( )	double fmod(double x, double y);	∗1
exp( )	double exp(double x);	∗1
log( )	double log(double x);	∗1
log10( )	double log10(double x);	∗1
frexp( )	double frexp(double x, int *nptr);	∗1
ldexp( )	double ldexp(double x, int n);	∗1
modf( )	double modf(double x, double *nptr);	∗1
pow( )	double pow(double x, double y);	∗1
sqrt( )	double sqrt(double x);	∗1
sin( )	double sin(double x);	∗1
cos( )	double cos(double x);	∗1
tan( )	double tan(double x);	∗1
asin( )	double asin(double x);	∗1
acos( )	double acos(double x);	∗1
atan( )	double atan(double x);	
atan2( )	double atan2(double y, double x);	∗1
sinh( )	double sinh(double x);	∗1
cosh( )	double cosh(double x);	∗1
tanh( )	double tanh(double x);	

ctype.lib (ヘッダファイル: ctype.h)

isalnum( )	int isalnum(char c);
isalpha( )	int isalpha(char c);
iscntrl( )	int iscntrl(char c);
isdigit( )	int isdigit(char c);
isgraph( )	int isgraph(char c);
islower( )	int islower(char c);
isprint( )	int isprint(char c);
ispunct( )	int ispunct(char c);
isspace( )	int isspace(char c);
isupper( )	int isupper(char c);
isxdigit( )	int isxdigit(char c);
tolower( )	int tolower(char c);
toupper( )	int toupper(char c);

stdarg.h

va_atart( )	void va_start(va_list ap, type lastarg);
va_arg( )	type va_arg(va_list ap, type);
va_end( )	void va_end(va_list ap);

∗1 **グローバル変数の宣言と初期化が必要です。**

string.lib (ヘッダファイル: string.h)

memchr( )	char *memchr(char *s, int c, int n);
memcmp( )	int memcmp(char *s1, char *s2, int n);
memcry( )	char *memcpy(char *s1, char *s2, int n);
memmove( )	char *memmove(char *s1, char *s2, int n);
memset( )	char *memset(char *s, int c, int n);
strcat( )	char *strcat(char *s1, char *s2);
strchr( )	char *strchr(char *s, int c);
strcmp( )	int strcmp(char *s1, char *s2);
strcpy( )	char *strcpy(char *s1, char *s2);
strcspn( )	size_t *strcspn(char *s1, char *s2);
strerror( )	char *strerror(int code);
strlen( )	size_t strlen(char *s);
strncat( )	size_t strncat(char *s1, char *s2, int n);
strncmp( )	int strncmp(char *s1, char *s2, int n);
strncpy( )	char *strncpy(char *s1, char *s2, int n);
strpbrk( )	char *strpbrk(char *s1, char *s2);
strrchr( )	char *strrchr(char *s, int c);
strspn( )	size_t strspn(char *s1, char *s2);
strstr( )	char *strstr(char *s1, char *s2);
strtok( )	char *strtok(char *s1, char *s2);

∗1 グローバル変数の宣言と初期化

FILE _iob[FOPEN_MAX+1];	_iob[N]._flg=_UGETN; _iob[N]._buf=0; _iob[N]._fd=N; (N=0: stdin, N=1: stdout, N=2: stderr)
FILE *stdin;	stdin=&_iob[0];
FILE *stdout;	stdout=&_iob[1];
FILE *stderr;	stderr=&_iob[2];
int errno;	errno=0;
unsigned int seed;	seed=1;
unsigned char *ucNxtAlcp;	ucNxtAlcp=(unsigned char *)<stack bottom address>
unsigned char *ucBefAlcp;	ucBefAlcp=(unsigned char *)NULL;
unsigned char *end_alloc;	end_alloc=(unsigned char *)<stack top address>
time_t gm_sec;	gm_sec=1;

∗2 下位レベル関数の定義

read( )	int read(int fd, char *buf, int nbytes); unsigned char READ_BUF[65]; (**他の名称も可**)
	unsigned char READ_EOF;
write( )	int write(int fd, char *buf, int nbytes); unsigned char WRITE_BUF[65]; (**他の名称も可**)

命令一覧中のシンボル

レジスタ/レジスタデータ

%rd, rd: ディスティネーションとなる汎用レジスタ(R0~R15)、またはレジスタの内容
%rs, rs: ソースとなる汎用レジスタ(R0~R15)、またはレジスタの内容
%rb, rb: レジスタ間接アドレッシングのベースレジスタを保持している汎用レジスタ(R0~R15)、またはベースアドレス
%sd, sd: ディスティネーションとなる特殊レジスタ(PSR, SP, ALR, AHR)、またはレジスタの内容
%ss, ss: ソースとなる特殊レジスタ(PSR, SP, ALR, AHR)、またはレジスタの内容
%sp, sp: スタックポインタまたはスタックポインタの内容

メモリ/アドレス/メモリデータ

[%rb]: レジスタ間接アドレッシング指定
[%rb]+: ポストインクリメント付きレジスタ間接アドレッシング指定
[%sp+immX], [%rb+immX], [%rb+symbol±immX]:
ディスプレースメント付きレジスタ間接アドレッシング指定
B[XXX]: XXXで指定されるアドレス、またはそのアドレスにストアされているバイトデータ
H[XXX]: XXXでベースアドレスが指定されるハーフワード領域、またはそこにストアされているハーフワードデータ
W[XXX]: XXXでベースアドレスが指定されるワード領域、またはそこにストアされているワードデータ

即値

immX: 符号なしXビット即値
signX: 符号付きXビット即値

シンボル/ラベル

Symbol: アドレスを示すシンボル
Label: 分岐先ラベル

ビットフィールド

(X): データのビットX
(X:Y): ビットXからビットYまでのビットフィールド
{X, Y}: ビット構成

機能

←: 右側の内容が左側の項目にロードされることを示します。
+: 加算
-: 減算
&: 論理積
|: 論理和
^: 排他的論理和
!: 論理否定
×: 乗算

フラグ

MO: MACオーバーフローフラグ
DS: 被除数符号フラグ
Z: ゼロフラグ
N: ネガティブフラグ
C: キャリーフラグ
V: オーバーフローフラグ
–: 変更なし
↔: セット(1)またはリセット(0)
0: リセット(0)

R8

–: 基本命令(展開なし)
○: R8レジスタ(gp)を使用して展開されることを示します。
×: R8レジスタを使用せずに展開されることを示します。

R9

–: 基本命令(展開なし)
○: ほぼすべての条件でR9レジスタを使用して展開されることを示します。
△: 条件によってはR9レジスタを使用して展開されることを示します。
×: R9レジスタを使用せずに展開されることを示します。

D

○: ディレイド命令として使用可能なことを示します。
–: ディレイド命令として使用できないことを示します。

注意

- 命令一覧にはS1C33000命令セットの基本命令の他に、拡張命令(xorを除くxで始まる命令)も含まれています。
  "Italic" の文字で記載されている命令は、上位互換の拡張命令が用意されていることを示します。
  拡張命令はext33を使用して基本命令に展開する必要があります。
- いくつかの拡張命令は展開の際にR9レジスタをスクラッチレジスタとして使用します。したがって、拡張命令を使用する前にR9レジスタにデータをロードしないでください。
- グローバルポインタを使用する場合、グローバルポインタアドレスをR8レジスタが保持します。R8レジスタにアドレスを設定後は、R8レジスタを変更しないでください。

分類	ニーモニック		機能	フラグ						R8	R9	D
	オペコード	オペランド		MO	DS	C	V	Z	N			
符号付きバイトデータ転送	ld.b	%rd, %rs	rd(7:0)←rs(7:0), rd(31:8)←rs(7)	–	–	–	–	–	–	–	–	–
		%rd, [%rb]	rd(7:0)←B[rb], rd(31:8)←B[rb](7)	–	–	–	–	–	–	–	–	–
		%rd, [%rb]+	rd(7:0)←B[rb], rd(31:8)←B[rb](7), rb←rb+1	–	–	–	–	–	–	–	–	–
		%rd, [%sp+imm6]	rd(7:0)←B[sp+imm6], rd(31:8)←B[sp+imm6](7)	–	–	–	–	–	–	–	–	–
		[%rb], %rs	B[rb]←rs(7:0)	–	–	–	–	–	–	–	–	–
		[%rb]+, %rs	B[rb]←rs(7:0), rb←rb+1	–	–	–	–	–	–	–	–	–
		[%sp+imm6], %rs	B[sp+imm6]←rs(7:0)	–	–	–	–	–	–	–	–	–
	xld.b	%rd, [imm32]	rd(7:0)←B[imm32], rd(31:8)←B[imm32](7)	–	–	–	–	–	–	×	○	–
		%rd, [symbol+imm32]	rd(7:0)←B[symbol+imm32], rd(31:8)←B[symbol+imm32](7)	–	–	–	–	–	–	○	○	–
		%rd, [symbol-imm32]	rd(7:0)←B[symbol-imm32], rd(31:8)←B[symbol-imm32](7)	–	–	–	–	–	–	×	○	–
		%rd, [%rb+imm32] *1	rd(7:0)←B[rb+imm32], rd(31:8)←B[rb+imm32](7)	–	–	↔	↔	↔	↔	×	△	–
		%rd, [%rb+symbol+imm32] *1	rd(7:0)←B[rb+symbol+imm32], rd(31:8)←B[rb+symbol+imm32](7)	–	–	↔	↔	↔	↔	×	△	–
		%rd, [%rb+symbol-imm32] *1	rd(7:0)←B[rb+symbol-imm32], rd(31:8)←B[rb+symbol-imm32](7)	–	–	↔	↔	↔	↔	×	○	–
		%rd, [%sp+imm32]	rd(7:0)←B[sp+imm32], rd(31:8)←B[sp+imm32](7)	–	–	–	–	–	–	×	×	–
		[imm32], %rs	B[imm32]←rs(7:0)	–	–	–	–	–	–	×	○	–
		[symbol+imm32], %rs	B[symbol+imm32]←rs(7:0)	–	–	–	–	–	–	○	○	–
		[symbol-imm32], %rs	B[symbol-imm32]←rs(7:0)	–	–	–	–	–	–	×	○	–
		[%rb+imm32], %rs *1	B[rb+imm32]←rs(7:0)	–	–	↔	↔	↔	↔	×	△	–
		[%rb+symbol+imm32], %rs *1	B[rb+symbol+imm32]←rs(7:0)	–	–	↔	↔	↔	↔	×	△	–
		[%rb+symbol-imm32], %rs *1	B[rb+symbol-imm32]←rs(7:0)	–	–	↔	↔	↔	↔	×	○	–
		[%sp+imm32], %rs	B[sp+imm32]←rs(7:0)	–	–	–	–	–	–	×	×	–
符号なしバイトデータ転送	ld.ub	%rd, %rs	rd(7:0)←rs(7:0), rd(31:8)←0	–	–	–	–	–	–	–	–	–
		%rd, [%rb]	rd(7:0)←B[rb], rd(31:8)←0	–	–	–	–	–	–	–	–	–
		%rd, [%rb]+	rd(7:0)←B[rb], rd(31:8)←0, rb←rb+1	–	–	–	–	–	–	–	–	–
		%rd, [%sp+imm6]	rd(7:0)←B[sp+imm6], rd(31:8)←0	–	–	–	–	–	–	–	–	–
	xld.ub	%rd, [imm32]	rd(7:0)←B[imm32], rd(31:8)←0	–	–	–	–	–	–	×	○	–
		%rd, [symbol+imm32]	rd(7:0)←B[symbol+imm32], rd(31:8)←0	–	–	–	–	–	–	○	○	–
		%rd, [symbol-imm32]	rd(7:0)←B[symbol-imm32], rd(31:8)←0	–	–	–	–	–	–	×	○	–
		%rd, [%rb+imm32] *1	rd(7:0)←B[rb+imm32], rd(31:8)←0	–	–	↔	↔	↔	↔	×	△	–
		%rd, [%rb+symbol+imm32] *1	rd(7:0)←B[rb+symbol+imm32], rd(31:8)←0	–	–	↔	↔	↔	↔	×	△	–
		%rd, [%rb+symbol-imm32] *1	rd(7:0)←B[rb+symbol-imm32], rd(31:8)←0	–	–	↔	↔	↔	↔	×	○	–
		%rd, [%sp+imm32]	rd(7:0)←B[sp+imm32], rd(31:8)←0	–	–	–	–	–	–	×	×	–

注意

*1) **これらの拡張命令は、条件によって展開に"add"命令が使用されることがあります。その結果によってはフラグが変更されることがあります。**

分類	ニーモニック		機能	フラグ						R8	R9	D
	オペコード	オペランド		MO	DS	C	V	Z	N			
符号付きハーフワードデータ転送	ld.h	%rd, %rs	rd(15:0)←rs(15:0), rd(31:16)←rs(15)	–	–	–	–	–	–	–	–	–
		%rd, [%rb]	rd(15:0)←H[rb], rd(31:16)←H[rb](15)	–	–	–	–	–	–	–	–	–
		%rd, [%rb]+	rd(15:0)←H[rb], rd(31:16)←H[rb](15), rb←rb+2	–	–	–	–	–	–	–	–	–
		%rd, [%sp+imm6]	rd(15:0)←H[sp+imm7], rd(31:16)←H[sp+imm7](15); imm7={imm6,0}	–	–	–	–	–	–	–	–	–
		[%rb], %rs	H[rb]←rs(15:0)	–	–	–	–	–	–	–	–	–
		[%rb]+, %rs	H[rb]←rs(15:0), rb←rb+2	–	–	–	–	–	–	–	–	–
		[%sp+imm6], %rs	H[sp+imm7]←rs(15:0); imm7={imm6,0}	–	–	–	–	–	–	–	–	–
	xld.h	%rd, [imm32]	rd(15:0)←H[imm32], rd(31:16)←H[imm32](15)	–	–	–	–	–	–	×	○	–
		%rd, [symbol+imm32]	rd(15:0)←H[symbol+imm32], rd(31:16)←H[symbol+imm32](15)	–	–	–	–	–	–	○	○	–
		%rd, [symbol-imm32]	rd(15:0)←H[symbol-imm32], rd(31:16)←H[symbol-imm32](15)	–	–	–	–	–	–	×	○	–
		%rd, [%rb+imm32] *1	rd(15:0)←H[rb+imm32], rd(31:16)←H[rb+imm32](15)	–	–	↔	↔	↔	↔	×	△	–
		%rd, [%rb+symbol+imm32] *1	rd(15:0)←H[rb+symbol+imm32], rd(31:16)←H[rb+symbol+imm32](15)	–	–	↔	↔	↔	↔	×	△	–
		%rd, [%rb+symbol-imm32] *1	rd(15:0)←H[rb+symbol-imm32], rd(31:16)←H[rb+symbol-imm32](15)	–	–	↔	↔	↔	↔	×	○	–
		%rd, [%sp+imm32]	rd(15:0)←H[sp+imm32], rd(31:16)←H[sp+imm32](15)	–	–	–	–	–	–	×	×	–
		[imm32], %rs	H[imm32]←rs(15:0)	–	–	–	–	–	–	×	○	–
		[symbol+imm32], %rs	H[symbol+imm32]←rs(15:0)	–	–	–	–	–	–	○	○	–
		[symbol-imm32], %rs	H[symbol-imm32]←rs(15:0)	–	–	–	–	–	–	×	○	–
		[%rb+imm32], %rs *1	H[rb+imm32]←rs(15:0)	–	–	↔	↔	↔	↔	×	△	–
		[%rb+symbol+imm32], %rs *1	H[rb+symbol+imm32]←rs(15:0)	–	–	↔	↔	↔	↔	×	△	–
		[%rb+symbol-imm32], %rs *1	H[rb+symbol-imm32]←rs(15:0)	–	–	↔	↔	↔	↔	×	○	–
		[%sp+imm32], %rs	H[sp+imm32]←rs(15:0)	–	–	–	–	–	–	×	×	–
符号なしハーフワードデータ転送	ld.uh	%rd, %rs	rd(15:0)←rs(15:0), rd(31:16)←0	–	–	–	–	–	–	–	–	–
		%rd, [%rb]	rd(15:0)←H[rb], rd(31:16)←0	–	–	–	–	–	–	–	–	–
		%rd, [%rb]+	rd(15:0)←H[rb], rd(31:16)←0, rb←rb+2	–	–	–	–	–	–	–	–	–
		%rd, [%sp+imm6]	rd(15:0)←H[sp+imm7], rd(31:16)←0; imm7={imm6,0}	–	–	–	–	–	–	–	–	–
	xld.uh	%rd, [imm32]	rd(15:0)←H[imm32], rd(31:16)←0	–	–	–	–	–	–	×	○	–
		%rd, [symbol+imm32]	rd(15:0)←H[symbol+imm32], rd(31:16)←0	–	–	–	–	–	–	○	○	–
		%rd, [symbol-imm32]	rd(15:0)←H[symbol-imm32], rd(31:16)←0	–	–	–	–	–	–	×	○	–
		%rd, [%rb+imm32] *1	rd(15:0)←H[rb+imm32], rd(31:16)←0	–	–	↔	↔	↔	↔	×	△	–
		%rd, [%rb+symbol+imm32] *1	rd(15:0)←H[rb+symbol+imm32], rd(31:16)←0	–	–	↔	↔	↔	↔	×	△	–
		%rd, [%rb+symbol-imm32] *1	rd(15:0)←H[rb+symbol-imm32], rd(31:16)←0	–	–	↔	↔	↔	↔	×	○	–
		%rd, [%sp+imm32]	rd(15:0)←H[sp+imm32], rd(31:16)←0	–	–	–	–	–	–	×	×	–

注意

*1) これらの拡張命令は、条件によって展開に"add"命令が使用されることがあります。その結果によってはフラグが変更されることがあります。

分類	ニーモニック		機能	フラグ						R8	R9	D
	オペコード	オペランド		MO	DS	C	V	Z	N			
ワードデータ転送	ld.w	%rd, %rs	rd←rs	–	–	–	–	–	–	–	–	○
		%sd, %rs	sd←rs	–	–	–	–	–	–	–	–	–
		%rd, %ss	rd←ss	–	–	–	–	–	–	–	–	–
		%rd, sign6	rd(5:0)←sign6(5:0), rd(31:6)←sign6(5)	–	–	–	–	–	–	–	–	○
		%rd, [%rb]	rd←W[rb]	–	–	–	–	–	–	–	–	–
		%rd, [%rb]+	rd←W[rb], rb←rb+4	–	–	–	–	–	–	–	–	–
		%rd, [%sp+imm6]	rd←W[sp+imm8]; imm8={imm6,00}	–	–	–	–	–	–	–	–	–
		[%rb], %rs	W[rb]←rs	–	–	–	–	–	–	–	–	–
		[%rb]+, %rs	W[rb]←rs, rb←rb+4	–	–	–	–	–	–	–	–	–
		[%sp+imm6], %rs	W[sp+imm8]←rs; imm8={imm6,00}	–	–	–	–	–	–	–	–	–
	xld.w	%rd, sign32	rd←sign32	–	–	–	–	–	–	×	×	–
		%rd, symbol+imm32	rd←symbol+imm32	–	–	–	–	–	–	×	×	–
		%rd, symbol-imm32	rd←symbol-imm32	–	–	–	–	–	–	×	×	–
		%rd, [imm32]	rd←W[imm32]	–	–	–	–	–	–	×	○	–
		%rd, [symbol+imm32]	rd←W[symbol+imm32]	–	–	–	–	–	–	○	○	–
		%rd, [symbol-imm32]	rd←W[symbol-imm32]	–	–	–	–	–	–	×	○	–
		%rd, [%rb+imm32] *1	rd←W[rb+imm32]	–	–	↔	↔	↔	↔	×	△	–
		%rd, [%rb+symbol+imm32] *1	rd←W[rb+symbol+imm32]	–	–	↔	↔	↔	↔	×	△	–
		%rd, [%rb+symbol-imm32] *1	rd←W[rb+symbol-imm32]	–	–	↔	↔	↔	↔	×	○	–
		%rd, [%sp+imm32]	rd←W[sp+imm32]	–	–	–	–	–	–	×	×	–
		[imm32], %rs	W[imm32]←rs	–	–	–	–	–	–	×	○	–
		[imm32], %sp	W[imm32]←sp	–	–	–	–	–	–	×	○	–
		[symbol+imm32], %rs	W[symbol+imm32]←rs	–	–	–	–	–	–	○	○	–
		[symbol-imm32], %rs	W[symbol-imm32]←rs	–	–	–	–	–	–	×	○	–
		[symbol+imm32], %sp	W[symbol+imm32]←sp	–	–	–	–	–	–	○	○	–
		[symbol-imm32], %sp	W[symbol-imm32]←sp	–	–	–	–	–	–	×	○	–
		[%rb+imm32], %rs *1	W[rb+imm32]←rs	–	–	↔	↔	↔	↔	×	△	–
		[%rb+imm32], %sp *1	W[rb+imm32]←sp	–	–	↔	↔	↔	↔	×	○	–
		[%rb+symbol+imm32], %rs *1	W[rb+symbol+imm32]←rs	–	–	↔	↔	↔	↔	×	△	–
		[%rb+symbol-imm32], %rs *1	W[rb+symbol-imm32]←rs	–	–	↔	↔	↔	↔	×	○	–
		[%rb+symbol+imm32], %sp *1	W[rb+symbol+imm32]←sp	–	–	↔	↔	↔	↔	×	○	–
		[%rb+symbol-imm32], %sp *1	W[rb+symbol-imm32]←sp	–	–	↔	↔	↔	↔	×	○	–
		[%sp+imm32], %rs	W[sp+imm32]←rs	–	–	–	–	–	–	×	×	–
		[%sp+imm32], %sp	W[sp+imm32]←sp	–	–	–	–	–	–	×	○	–

注意

*1) **これらの拡張命令は、条件によって展開に"add"命令が使用されることがあります。その結果によってはフラグが変更されることがあります。**

分類	ニーモニック		機能	フラグ						R8	R9	D
	オペコード	オペランド		MO	DS	C	V	Z	N			
論理演算	and	%rd, %rs	rd←rd & rs	–	–	–	–	↔	↔	–	–	○
		%rd, sign6	rd←rd & sign6(符号拡張)	–	–	–	–	↔	↔	–	–	○
	xand	%rd, %rd, sign32	rd←rd & sign32	–	–	–	–	↔	↔	×	×	–
		%rd, %rs, sign32	rd←rs & sign32	–	–	–	–	↔	↔	×	×	–
	or	%rd, %rs	rd←rd | rs	–	–	–	–	↔	↔	–	–	○
		%rd, sign6	rd←rd | sign6(符号拡張)	–	–	–	–	↔	↔	–	–	○
	xoor	%rd, %rd, sign32	rd←rd | sign32	–	–	–	–	↔	↔	×	×	–
		%rd, %rs, sign32	rd←rs | sign32	–	–	–	–	↔	↔	×	×	–
	xor	%rd, %rs	rd←rd ^ rs	–	–	–	–	↔	↔	–	–	○
		%rd, sign6	rd←rd ^ sign6(符号拡張)	–	–	–	–	↔	↔	–	–	○
	xxor	%rd, %rd, sign32	rd←rd ^ sign32	–	–	–	–	↔	↔	×	×	–
		%rd, %rs, sign32	rd←rs ^ sign32	–	–	–	–	↔	↔	×	×	–
	not	%rd, %rs	rd←!rs	–	–	–	–	↔	↔	–	–	○
		%rd, sign6	rd←!sign6(符号拡張)	–	–	–	–	↔	↔	–	–	○
	xnot	%rd, sign32	rd←!sign32	–	–	–	–	↔	↔	×	×	–
算術演算	add	%rd, %rs	rd←rd + rs	–	–	↔	↔	↔	↔	–	–	○
		%rd, imm6	rd←rd + imm6(ゼロ拡張)	–	–	↔	↔	↔	↔	–	–	○
		%sp, imm10	sp←sp + imm12(ゼロ拡張); imm12={imm10,00}	–	–	–	–	–	–	–	–	○
	xadd	%rd, %rd, imm32	rd←rd + imm32	–	–	↔	↔	↔	↔	×	×	–
		%sp, %sp, imm32	sp←sp + imm32	–	–	↔	↔	↔	↔	×	○	–
		%rd, %rs, imm32	rd←rs + imm32	–	–	↔	↔	↔	↔	×	×	–
		%rd, %sp, imm32	rd←sp + imm32	–	–	↔	↔	↔	↔	×	×	–
		%rd, %rd, %sp	rd←rd + sp	–	–	↔	↔	↔	↔	×	○	–
		%sp, %sp, %rs	sp←sp + rs	–	–	↔	↔	↔	↔	×	○	–
	adc	%rd, %rs	rd←rd + rs + C	–	–	↔	↔	↔	↔	–	–	○
	sub	%rd, %rs	rd←rd - rs	–	–	↔	↔	↔	↔	–	–	○
		%rd, imm6	rd←rd - imm6(ゼロ拡張)	–	–	↔	↔	↔	↔	–	–	○
		%sp, imm10	sp←sp - imm12(ゼロ拡張); imm12={imm10,00}	–	–	–	–	–	–	–	–	○
	xsub	%rd, %rd, imm32	rd←rd - imm32	–	–	↔	↔	↔	↔	×	×	–
		%sp, %sp, imm32	sp←sp - imm32	–	–	↔	↔	↔	↔	×	○	–
		%rd, %rs, imm32	rd←rs - imm32	–	–	↔	↔	↔	↔	×	×	–
		%rd, %sp, imm32	rd←sp - imm32	–	–	↔	↔	↔	↔	×	×	–
		%rd, %rd, %sp	rd←rd - sp	–	–	↔	↔	↔	↔	×	○	–
		%sp, %sp, %rs	sp←sp - rs	–	–	↔	↔	↔	↔	×	○	–
	sbc	%rd, %rs	rd←rd - rs - C	–	–	↔	↔	↔	↔	–	–	–

注意

分類	ニーモニック		機能	フラグ						R8	R9	D
	オペコード	オペランド		MO	DS	C	V	Z	N			
算術演算	cmp	%rd, %rs	rd - rs	–	–	↔	↔	↔	↔	–	–	○
	%rd, sign6		rd - sign6(符号拡張)	–	–	↔	↔	↔	↔	–	–	○
	xcmp	%rd, sign32	rd - sign32	–	–	↔	↔	↔	↔	×	×	–
		%rd, %sp	rd - sp	–	–	↔	↔	↔	↔	×	○	–
		%sp, %rs	sp - rs	–	–	↔	↔	↔	↔	×	○	–
	mlt.h	%rd, %rs	alr←rd(15:0) × rs(15:0); 符号付き乗算	–	–	–	–	–	–	–	–	○
	mltu.h	%rd, %rs	alr←rd(15:0) × rs(15:0); 符号なし乗算	–	–	–	–	–	–	–	–	○
	mlt.w	%rd, %rs	{ahr, alr}←rd × rs; 符号付き乗算	–	–	–	–	–	–	–	–	–
	mltu.w	%rd, %rs	{ahr, alr}←rd × rs; 符号なし乗算	–	–	–	–	–	–	–	–	–
	div0s	%rs	符号付き除算第1ステップ; alr = 被除数, rs = 除数	–	↔	–	–	–	↔	–	–	–
	div0u	%rs	符号なし除算第1ステップ; alr = 被除数, rs = 除数	–	0	–	–	–	0	–	–	–
	div1	%rs	ステップ除算 (∗1); alr←商, ahr←余り(符号なし)	–	–	–	–	–	–	–	–	–
	div2s	%rs	符号付き除算データ補正1 (∗2)	–	–	–	–	–	–	–	–	–
	div3s		符号付き除算データ補正2 (∗2); alr←商, ahr←余り	–	–	–	–	–	–	–	–	–
シフト&ローテート	*srl*	*%rd, imm4*	右方向論理シフト>>imm4ビット; imm4=0–8, MSBには0をセット	–	–	–	–	↔	↔	–	–	○
		%rd, %rs	右方向論理シフト>>rsビット; rs=0–8, MSBには0をセット	–	–	–	–	↔	↔	–	–	○
	xsrl	%rd, imm5	右方向論理シフト>>imm5ビット; imm5=0–31, MSBには0をセット	–	–	–	–	↔	↔	×	×	–
		%rd, %rs	右方向論理シフト>>rsビット; rs=0–31, MSBには0をセット	–	–	↔	↔	↔	↔	×	○	–
	sll	*%rd, imm4*	左方向論理シフト>>imm4ビット; imm4=0–8, LSBには0をセット	–	–	–	–	↔	↔	–	–	○
		%rd, %rs	左方向論理シフト>>rsビット; rs=0–8, LSBには0をセット	–	–	–	–	↔	↔	–	–	○
	xsll	%rd, imm5	左方向論理シフト>>imm5ビット; imm5=0–31, LSBには0をセット	–	–	–	–	↔	↔	×	×	–
		%rd, %rs	左方向論理シフト>>rsビット; rs=0–31, LSBには0をセット	–	–	↔	↔	↔	↔	×	○	–
	sra	*%rd, imm4*	右方向算術シフト>>imm4ビット; imm4=0–8, MSBには符号をコピー	–	–	–	–	↔	↔	–	–	○
		%rd, %rs	右方向算術シフト>>rsビット; rs=0–8, MSBには符号をコピー	–	–	–	–	↔	↔	–	–	○
	xsra	%rd, imm5	右方向算術シフト>>imm5ビット; imm5=0–31, MSBには符号をコピー	–	–	–	–	↔	↔	×	×	–
		%rd, %rs	右方向算術シフト>>rsビット; rs=0–31, MSBには符号をコピー	–	–	↔	↔	↔	↔	×	○	–
	sla	*%rd, imm4*	左方向算術シフト>>imm4ビット; imm4=0–8, LSBには0をセット	–	–	–	–	↔	↔	–	–	○
		%rd, %rs	左方向算術シフト>>rsビット; rs=0–8, LSBには0をセット	–	–	–	–	↔	↔	–	–	○
	xsla	%rd, imm5	左方向算術シフト>>imm5ビット; imm5=0–31, LSBには0をセット	–	–	–	–	↔	↔	×	×	–
		%rd, %rs	左方向算術シフト>>rsビット; rs=0–31, LSBには0をセット	–	–	↔	↔	↔	↔	×	○	–
	rr	*%rd, imm4*	右方向ローテート>>imm4ビット; imm4=0–8, LSB → MSB	–	–	–	–	↔	↔	–	–	○
		%rd, %rs	右方向ローテート>>rsビット; rs=0–8, LSB → MSB	–	–	–	–	↔	↔	–	–	○
	xrr	%rd, imm5	右方向ローテート>>imm5ビット; imm5=0–31, LSB → MSB	–	–	–	–	↔	↔	×	×	–
		%rd, %rs	右方向ローテート>>rsビット; rs=0–8, LSB → MSB	–	–	↔	↔	↔	↔	×	○	–

注意

∗1) 32ビット ÷ 32ビットの場合、div1命令を32回実行する必要があります。符号なし除算の場合はdiv1命令によって除算結果がalrおよびahrにロードされます。

∗2) div2s命令とdiv3s命令は符号なし除算では不要です。

分類	ニーモニック		機能	フラグ						R8	R9	D
	オペコード	オペランド		MO	DS	C	V	Z	N			
シフト&ローテート	*rl*	*%rd, imm4*	**左方向ローテート>>**imm4**ビット**; imm4=0–8, MSB → LSB	–	–	–	–	↔	↔	–	–	○
		%rd, %rs	**左方向ローテート>>**rs**ビット**; rs=0–8, MSB → LSB	–	–	–	–	↔	↔	–	–	○
	xrl	%rd, imm5	**左方向ローテート>>**imm5**ビット**; imm5=0–31, MSB → LSB	–	–	–	–	↔	↔	×	×	–
		%rd, %rs	**左方向ローテート>>**rs**ビット**; rs=0–31, MSB → LSB	–	–	↔	↔	↔	↔	×	○	–
ビット操作	*btst*	*[%rb], imm3*	Z flag←1 if B[rb](imm3)=0	–	–	–	–	↔	–	–	–	–
	xbtst	[imm32], imm3	Z flag←1 if B[imm32](imm3)=0	–	–	–	–	↔	–	×	○	–
		[symbol+imm32], imm3	Z flag←1 if B[symbol+imm32](imm3)=0	–	–	–	–	↔	–	○	○	–
		[symbol-imm32], imm3	Z flag←1 if B[symbol-imm32](imm3)=0	–	–	–	–	↔	–	×	○	–
		[%rb+imm32], imm3 *1	Z flag←1 if B[rb+imm32](imm3)=0	–	–	↔	↔	↔	↔	×	○	–
		[%rb+symbol+imm32], imm3 *1	Z flag←1 if B[rb+symbol+imm32](imm3)=0	–	–	↔	↔	↔	↔	×	○	–
		[%rb+symbol-imm32], imm3 *1	Z flag←1 if B[rb+symbol-imm32](imm3)=0	–	–	↔	↔	↔	↔	×	○	–
		[%sp+imm32], imm3 *1	Z flag←1 if B[sp+imm32](imm3)=0	–	–	↔	↔	↔	↔	×	○	–
	bclr	*[%rb], imm3*	B[imm32](imm3)←0	–	–	–	–	–	–	–	–	–
	xbclr	[imm32], imm3	B[imm32](imm3)←0	–	–	–	–	–	–	×	○	–
		[symbol+imm32], imm3	B[symbol+imm32](imm3)←0	–	–	–	–	–	–	○	○	–
		[symbol-imm32], imm3	B[symbol-imm32](imm3)←0	–	–	–	–	–	–	×	○	–
		[%rb+imm32], imm3 *1	B[rb+imm32](imm3)←0	–	–	↔	↔	↔	↔	×	○	–
		[%rb+symbol+imm32], imm3 *1	B[rb+symbol+imm32](imm3)←0	–	–	↔	↔	↔	↔	×	○	–
		[%rb+symbol-imm32], imm3 *1	B[rb+symbol-imm32](imm3)←0	–	–	↔	↔	↔	↔	×	○	–
		[%sp+imm32], imm3 *1	B[sp+imm32](imm3)←0	–	–	↔	↔	↔	↔	×	○	–
	bset	*[%rb], imm3*	B[rb](imm3)←1	–	–	–	–	–	–	–	–	–
	xbset	[imm32], imm3	B[imm32](imm3)←1	–	–	–	–	–	–	×	○	–
		[symbol+imm32], imm3	B[symbol+imm32](imm3)←1	–	–	–	–	–	–	○	○	–
		[symbol-imm32], imm3	B[symbol-imm32](imm3)←1	–	–	–	–	–	–	×	○	–
		[%rb+imm32], imm3 *1	B[rb+imm32](imm3)←1	–	–	↔	↔	↔	↔	×	○	–
		[%rb+symbol+imm32], imm3 *1	B[rb+symbol+imm32](imm3)←1	–	–	↔	↔	↔	↔	×	○	–
		[%rb+symbol-imm32], imm3 *1	B[rb+symbol-imm32](imm3)←1	–	–	↔	↔	↔	↔	×	○	–
		[%sp+imm32], imm3 *1	B[sp+imm32](imm3)←1	–	–	↔	↔	↔	↔	×	○	–
	bnot	*[%rb], imm3*	B[rb](imm3)←!B[rb](imm3)	–	–	–	–	–	–	–	–	–
	xbnot	[imm32], imm3	B[imm32](imm3)←!B[imm32](imm3)	–	–	–	–	–	–	×	○	–
		[symbol+imm32], imm3	B[symbol+imm32](imm3)←!B[symbol+imm32](imm3)	–	–	–	–	–	–	○	○	–
		[symbol-imm32], imm3	B[symbol-imm32](imm3)←!B[symbol-imm32](imm3)	–	–	–	–	–	–	×	○	–
		[%rb+imm32], imm3 *1	B[rb+imm32](imm3)←!B[rb+imm32](imm3)	–	–	↔	↔	↔	↔	×	○	–
		[%rb+symbol+imm32], imm3 *1	B[rb+symbol+imm32](imm3)←!B[rb+symbol+imm32](imm3)	–	–	↔	↔	↔	↔	×	○	–
		[%rb+symbol-imm32], imm3 *1	B[rb+symbol-imm32](imm3)←!B[rb+symbol-imm32](imm3)	–	–	↔	↔	↔	↔	×	○	–
		[%sp+imm32], imm3 *1	B[sp+imm32](imm3)←!B[sp+imm32](imm3)	–	–	↔	↔	↔	↔	×	○	–

注意

*1) **これらの拡張命令は、条件によって展開に"add"命令が使用されることがあります。その結果によってはフラグが変更されることがあります。**

分類	ニーモニック		機能	フラグ						R8	R9	D
	オペコード	オペランド		MO	DS	C	V	Z	N			
分岐	*jrgt* *jrgt.d*	*sign8*	pc←pc+sign9 if !Z&!(N^V) is true; sign9={sign8,0}	–	–	–	–	–	–	–	–	–
	xjrgt	label+imm32	pc←label+imm32 if !Z&!(N^V) is true	–	–	–	–	–	–	×	×	–
	xjrgt.d	sign32	pc←pc+sign32 if !Z&!(N^V) is true	–	–	–	–	–	–	×	×	–
	jrge *irge.d*	*sign8*	pc←pc+sign9 if !(N^V) is true; sign9={sign8,0}	–	–	–	–	–	–	–	–	–
	xjrge	label+imm32	pc←label+imm32 if !(N^V) is true	–	–	–	–	–	–	×	×	–
	xirge.d	sign32	pc←pc+sign32 if !(N^V) is true	–	–	–	–	–	–	×	×	–
	jrlt *jrlt.d*	*sign8*	pc←pc+sign9 if N^V is true; sign9={sign8,0}	–	–	–	–	–	–	–	–	–
	xjrlt	label+imm32	pc←label+imm32 if N^V is true	–	–	–	–	–	–	×	×	–
	xjrlt.d	sign32	pc←pc+sign32 if N^V is true	–	–	–	–	–	–	×	×	–
	jrle *jrle.d*	*sign8*	pc←pc+sign9 if Z | (N^V) is true; sign9={sign8,0}	–	–	–	–	–	–	–	–	–
	xjrle	label+imm32	pc←label+imm32 if Z | (N^V) is true	–	–	–	–	–	–	×	×	–
	xjrle.d	sign32	pc←pc+sign32 if Z | (N^V) is true	–	–	–	–	–	–	×	×	–
	jrugt *jrugt.d*	*sign8*	pc←pc+sign9 if !Z&!C is true; sign9={sign8,0}	–	–	–	–	–	–	–	–	–
	xjrugt	label+imm32	pc←label+imm32 if !Z&!C is true	–	–	–	–	–	–	×	×	–
	xjrugt.d	sign32	pc←pc+sign32 if !Z&!C is true	–	–	–	–	–	–	×	×	–
	jruge *jruge.d*	*sign8*	pc←pc+sign9 if !C is true; sign9={sign8,0}	–	–	–	–	–	–	–	–	–
	xjruge	label+imm32	pc←label+imm32 if !C is true	–	–	–	–	–	–	×	×	–
	xjruge.d	sign32	pc←pc+sign32 if !C is true	–	–	–	–	–	–	×	×	–
	jrult *jrult.d*	*sign8*	pc←pc+sign9 if C is true; sign9={sign8,0}	–	–	–	–	–	–	–	–	–
	xjrult	label+imm32	pc←label+imm32 if C is true	–	–	–	–	–	–	×	×	–
	xjrult.d	sign32	pc←pc+sign32 if C is true	–	–	–	–	–	–	×	×	–
	jrule *jrule.d*	*sign8*	pc←pc+sign9 if Z | C is true; sign9={sign8,0}	–	–	–	–	–	–	–	–	–
	xjrule	label+imm32	pc←label+imm32 if Z | C is true	–	–	–	–	–	–	×	×	–
	xjrule.d	sign32	pc←pc+sign32 if Z | C is true	–	–	–	–	–	–	×	×	–
	jreq *jreq.d*	*sign8*	pc←pc+sign9 if Z is true; sign9={sign8,0}	–	–	–	–	–	–	–	–	–
	xjreq	label+imm32	pc←label+imm32 if Z is true	–	–	–	–	–	–	×	×	–
	xjreq.d	sign32	pc←pc+sign32 if Z is true	–	–	–	–	–	–	×	×	–

注意

分類	ニーモニック		機能	フラグ						R8	R9	D
	オペコード	オペランド		MO	DS	C	V	Z	N			
分岐	*jrne* *jrne.d*	*sign8*	pc←pc+sign9 if !Z is true; sign9={sign8,0}	–	–	–	–	–	–	–	–	–
	xjrne	label+imm32	pc←label+imm32 if !Z is true	–	–	–	–	–	–	×	×	–
	xjrne.d	sign32	pc←pc+sign32 if !Z is true	–	–	–	–	–	–	×	×	–
	call	*sign8*	sp←sp-4, W[sp]←pc+2, pc←pc+sign9; sign9={sign8,0}	–	–	–	–	–	–	–	–	–
	call.d	%rb	sp←sp-4, W[sp]←pc+2, pc←rb	–	–	–	–	–	–	–	–	–
	xcall	label+imm32	sp←sp-4, W[sp]←pc+2, pc←label+imm32	–	–	–	–	–	–	×	×	–
	xcall.d	sign32	sp←sp-4, W[sp]←pc+2, pc←pc+sign32	–	–	–	–	–	–	×	×	–
	jp	*sign8*	pc←pc+sign9; sign9={sign8,0}	–	–	–	–	–	–	–	–	–
	jp.d	%rb	pc←rb	–	–	–	–	–	–	–	–	–
	xjp	abel+imm32	pc←pc+label+imm32	–	–	–	–	–	–	×	×	–
	xjp.d	sign32	pc←pc+sign32	–	–	–	–	–	–	×	×	–
	ret ret.d		pc←W[sp], sp←sp+4	–	–	–	–	–	–	–	–	–
	reti		psr←W[sp], sp←sp+4, pc←W[sp], sp←sp+4	↔	↔	↔	↔	↔	↔	–	–	–
	retd		デバッグ例外処理ルーチンからのリターン (S5U1C33xxxHソフトウェア用)	–	–	–	–	–	–	–	–	–
	int	imm2	sp←sp-4, W[sp]←pc+2, sp←sp-4, W[sp]←psr, pc←ソフトウェア例外ベクタ	–	–	–	–	–	–	–	–	–
	brk		デバッグ例外 (S5U1C33xxxHソフトウェア用)	–	–	–	–	–	–	–	–	–
即値拡張	*ext*	*imm13*	次の命令の即値またはオペランドを拡張	–	–	–	–	–	–	–	–	–
プッシュ&ポップ	pushn	%rs	"sp←sp-4, W[sp]←rn"をrn=rs~r0まで繰り返し	–	–	–	–	–	–	–	–	–
	popn	%rd	"rn←W[sp], sp←sp+4"をrn=r0~rdまで繰り返し	–	–	–	–	–	–	–	–	–
積和演算	mac	%rs	"{ahr, alr}←{ahr, alr} + H[<rs+1>]+ × H[<rs+2>]+"をrs回繰り返し (*1)	↔	–	–	–	–	–	–	–	–
システム制御	nop		ノーオペレーション; pc←pc+2	–	–	–	–	–	–	–	–	–
	halt		Haltモードに設定	–	–	–	–	–	–	–	–	–
	slp		Sleepモードに設定	–	–	–	–	–	–	–	–	–
その他	scan0	%rd, %rs	rsの最上位バイト中の"0"ビットをスキャン, rd←MSBからのオフセット	–	–	↔	0	↔	0	–	–	○
	scan1	%rd, %rs	rsの最上位バイト中の"1"ビットをスキャン, rd←MSBからのオフセット	–	–	↔	0	↔	0	–	–	○
	swap	%rd, %rs	rd(31:24)←rs(7:0), rd(23:16)←rs(15:8), rd(15:8)←rs(23:16), rd(7:0)←rs(31:24)	–	–	–	–	–	–	–	–	○
	mirror	%rd, %rs	rd(31:24)←rs(24:31), rd(23:16)←rs(16:23), rd(15:8)←rs(8:15), rd(7:0)←rs(0:7)	–	–	–	–	–	–	–	–	○

注意

*1) <rs+1>, <rs+2>: rsレジスタに続く2つのレジスタの内容 (例: rs=r0: <rs+1>=r1, <rs+2>=r2; rs=r15: <rs+1>=r0, <rs+2>=r1), 1回の演算ごとにポストインクリメント(+2)
mac命令はオプションの乗除算回路を内蔵した機種に限り実行可能です。

拡張命令の展開形式 (1)

拡張命令		展開形式				
オペコード	オペランド	条件1	条件2	条件3	条件4	条件5
xld.b **xld.ub** **xld.h** **xld.uh** **xld.w**	**%rd, [imm32]** **[imm32], %rs** *1 <gp未使用> 例) xld.w %rd,[imm32]	imm32≤0x1f ld.w %r9,imm32(5:0) ld.w %rd,[%r9]	0x1f<imm32≤0x3ffff ext imm32(18:6) ld.w %r9,imm32(5:0) ld.w %rd,[%r9]	0x3ffff<imm32 ext imm32(31:19) ext imm32(18:6) ld.w %r9,imm32(5:0) ld.w %rd,[%r9]	–	–
	%rd, [imm32] **[imm32], %rs** *1 <gp使用> 例) xld.w %rd,[imm32] (sign32=-gp+imm32)	sign32=0x0 ld.w %rd,[%r8]	0x0<sign32≤0x1fff ext sign32(12:0) ld.w %rd,[%r8]	0x1fff<sign32≤0x3ffffff ext sign32(25:13) ext sign32(12:0) ld.w %rd,[%r8]	0x3ffffff<sign32 ext imm32(31:19) ext imm32(18:6) ld.w %r9,imm32(5:0) ld.w %rd, [%r9]	imm32<gp imm32**の値に従って**gp**未使用時の展開形式となります。**
	%rd, [symbol+imm32] **[symbol+imm32], %rs** *1 <gp未使用> 例) xld.w %rd,[symbol+imm32]	symbol+imm32≤0x1f ld.w %r9,symbol+imm32@l ld.w %rd,[%r9]	0x1f<symbol+imm32≤0x3ffff ext symbol+imm32@m ld.w %r9,symbol+imm32@l ld.w %rd,[%r9]	0x3ffff<symbol+imm32 *2 ext symbol+imm32@h ext symbol+imm32@m ld.w %r9,symbol+imm32@l ld.w %rd, [%r9]	–	–
	%rd, [symbol+imm32] **[symbol+imm32], %rs** *1 <gp使用> 例) xld.w %rd,[symbol+imm32] (sign32=-gp+imm32)	symbol+sign32=0x0 ld.w %rd,[%r8]	0x0<symbol+sign32≤0x1fff ext symbol+sign32@al ld.w %rd,[%r8]	0x1fff<symbol+sign32≤0x3ffffff *2 ext symbol+sign32@ah ext symbol+sign32@al ld.w %rd,[%r8]	0x3ffffff<symbol+sign32 ext symbol+imm32@h ext symbol+imm32@m ld.w %r9,symbol+imm32@l ld.w %rd, [%r9]	symbol+imm32<gp symbol+imm32**の値に従って**gp**未使用時の展開形式となります。**
	%rd, [symbol-imm32] **[symbol-imm32], %rs** *1 例) xld.w %rd,[symbol-imm32]	**固定** ext symbol-imm32@h ext symbol-imm32@m ld.w %r9,symbol-imm32@l ld.w %rd, [%r9]	–	–	–	–
	%rd, [%rb+imm32] **[%rb+imm32], %rs** *1 例) xld.w %rd,[%rb+imm32]	imm32=0x0 ld.w %rd,[%rb]	0x0<imm32≤0x1fff ext imm32(12:0) ld.w %rd,[%rb]	0x1fff<imm32≤0x3ffffff ext imm32(25:13) ext imm32(12:0) ld.w %rd,[%rb]	0x3ffffff<imm32 ext imm32(31:19) ext imm32(18:6) ld.w %r9,imm32(5:0) add %r9,%rb ld.w %rd,[%r9]	–
	%rd, [%rb+symbol+imm32] **[%rb+symbol+imm32], %rs** *1 例) xld.w %rd,[%rb+symbol+imm32]	symbol+imm32=0x0 ld.w %rd,[%rb]	0x0<symbol+imm32≤0x1fff ext symbol+imm32al ld.w %rd,[%rb]	0x1fff<symbol+imm32≤0x3ffffff ext symbol+imm32@ah ext symbol+imm32@al ld.w %rd,[%rb]	0x3ffffff<symbol+imm32 *2 ext symbol+imm32@h ext symbol+imm32@m ld.w %r9,symbol+imm32@l add %r9,%rb ld.w %rd, [%r9]	–

注意

*1) **これらのオペランドはxld.b、 xld.h、xld.w命令でのみ使用可能です。** *2) **シンボルが不定な場合も、この形式に展開されます。**

拡張命令の展開形式 (2)

拡張命令		展開形式				
オペコード	オペランド	条件1	条件2	条件3	条件4	条件5
xld.b **xld.ub** **xld.h** **xld.uh** **xld.w**	**%rd, [%rb+symbol-imm32]** **[%rb+symbol-imm32], %rs** *1 例) xld.w %rd,[%rb+symbol-imm32]	固定 ext symbol-imm32@h ext symbol-imm32@m ld.w %r9,symbol-imm32@l add %r9,%rb ld.w %rd, [%r9]	–	–	–	–
xld.b **xld.ub**	**%rd, [%sp+imm32]** **[%sp+imm32], %rs** *1 例) xld.b %rd,[%sp+imm32]	imm32≤0x3f ld.b %rd,[%sp+imm32(5:0)]	0x3f<imm32≤0x7ffff ext imm32(18:6) ld.b %rd,[%sp+imm32(5:0)]	0x7ffff<imm32 ext imm32(31:19) ext imm32(18:6) ld.b %rd,[%sp+imm32(5:0)]	–	–
xld.h **xld.uh**	**%rd, [%sp+imm32]** **[%sp+imm32], %rs** *1 例) xld.h %rd,[%sp+imm32]	imm32≤0x7f ld.h %rd,[%sp+imm32(6:1)]	0x7f<imm32≤0x7ffff ext imm32(18:6) ld.h %rd,[%sp+imm32(5:0)]	0x7ffff<imm32 ext imm32(31:19) ext imm32(18:6) ld.h %rd,[%sp+imm32(5:0)]	–	–
xld.w	**%rd, [%sp+imm32]** **[%sp+imm32], %rs** *1 例) xld.w %rd,[%sp+imm32]	imm32≤0xff ld.w %rd,[%sp+imm32(7:2)]	0xff<imm32≤0x7ffff ext imm32(18:6) ld.w %rd,[%sp+imm32(5:0)]	0x7ffff<imm32 ext imm32(31:19) ext imm32(18:6) ld.w %rd,[%sp+imm32(5:0)]	–	–
	[imm32], %sp <gp未使用> 例) xld.w [imm32],%sp	imm32≤0x1f ld.w %r9,%sp pushn %r0 ld.w %r0,imm32(5:0) ld.w [%r0],%r9 popn %r0	0x1f<imm32≤0x3ffff ld.w %r9,%sp pushn %r0 ext imm32(18:6) ld.w %r0,imm32(5:0) ld.w [%r0],%r9 popn %r0	0x3ffff<imm32 ld.w %r9,%sp pushn %r0 ext imm32(31:19) ext imm32(18:6) ld.w %r0,imm32(5:0) ld.w [%r0],%r9 popn %r0	–	–
	[imm32], %sp <gp使用> 例) xld.w [imm32],%sp (sign32=-gp+imm32)	sign32=0x0 ld.w %r9,%sp ld.w [%r8],%r9	0x0<sign32≤0x1fff ld.w %r9,%sp ext sign32(12:0) ld.w [%r8],%r9	0x1fff<sign32≤0x3fffff ld.w %r9,%sp ext sign32(25:13) ext sign32(12:0) ld.w [%r8],%r9	0x3fffff<sign32 ld.w %r9,%sp pushn %r0 ext imm32(31:19) ext imm32(18:6) ld.w %r0,imm32(5:0) ld.w [%r0],%r9 popn %r0	imm32<gp imm32の値に従ってgp未使用時の展開形式となります。

注意

*1) **これらのオペランドはxld.b、 xld.h、xld.w命令でのみ使用可能です。**

拡張命令の展開形式 (3)

拡張命令		展開形式				
オペコード	オペランド	条件1	条件2	条件3	条件4	条件5
xld.w	**[symbol+imm32], %sp** <gp未使用> 例) xld.w [symbol+imm32],%sp	symbol+imm32≤0x1f ld.w %r9,%sp pushn %r0 ld.w %r0,symbol+imm32@l ld.w [%r0],%r9 popn %r0	0x1f<symbol+imm32≤0x3ffff ld.w %r9,%sp pushn %r0 ext symbol+imm32@m ld.w %r0,symbol+imm32@l ld.w [%r0],%r9 popn %r0	0x3ffff<symbol+imm32 *1 ld.w %r9,%sp pushn %r0 ext symbol+imm32@h ext symbol+imm32@m ld.w %r0,symbol+imm32@l ld.w [%r0],%r9 popn %r0	–	–
	[symbol+imm32], %sp <gp使用> 例) xld.w [symbol+imm32],%sp (sign32=-gp+imm32)	symbol+sign32=0x0 ld.w %r9,%sp ld.w [%r8],%r9	0x0<symbol+sign32≤0x1fff ld.w %r9,%sp ext symbol+sign32@al ld.w [%r8],%r9	0x1fff<symbol+sign32≤0x3fffff ld.w %r9,%sp ext symbol+sign32@ah ext symbol+sign32@al ld.w [%r8],%r9	0x3fffff<symbol+sign32 *1 ld.w %r9,%sp pushn %r0 ext symbol+imm32@h ext symbol+imm32@m ld.w %r0,symbol+imm32@l ld.w [%r0],%r9 popn %r0	symbol+imm32<gp **symbol+imm32の値に従ってgp未使用時の展開形式となります。**
	[symbol-imm32], %sp 例) xld.w [symbol-imm32],%sp	**固定** ld.w %r9,%sp pushn %r0 ext symbol-imm32@h ext symbol-imm32@m ld.w %r0,symbol-imm32@l ld.w [%r0],%r9 popn %r0	–	–	–	–
	[%rb+imm32], %sp 例) xld.w [%rb+imm32],%sp	imm32=0x0 ld.w %r9,%sp ld.w [%rb],%r9	0x0<imm32≤0x1fff ld.w %r9,%sp ext imm32(12:0) ld.w [%rb],%r9	0x1fff<imm32≤0x3fffff ld.w %r9,%sp ext imm32(25:13) ext imm32(12:0) ld.w [%rb],%r9	0x3fffff<imm32, %rb≠%r0 ld.w %r9,%sp pushn %r0 ext imm32(31:19) ext imm32(18:6) ld.w %r0,imm32(5:0) add %r0,%rb ld.w [%r0],%r9 popn %r0	0x3fffff<imm32, %rb=%r0 ld.w %r9,%sp pushn %r1 ext imm32(31:19) ext imm32(18:6) ld.w %r1,imm32(5:0) add %r1,%rb ld.w [%r1],%r9 popn %r1

注意

*1) **シンボルが不定な場合も、この形式に展開されます。**

拡張命令の展開形式 (4)

拡張命令		展開形式				
オペコード	オペランド	条件1	条件2	条件3	条件4	条件5
xld.w	**[%rb+symbol+imm32], %sp**	symbol+imm32=0x0	0x0<symbol+imm32≤0x1fff	0x1fff<symbol+imm32≤0x3fffff	0x3fffff<symbol+imm32,%rb≠%r0 *1	0x3fffff<symbol+imm32,%rb=%r0 *2
例) xld.w [%rb+symbol+imm32],%sp		ld.w %r9,%sp ld.w [%rb],%r9	ld.w %r9,%sp ext symbol+imm32al ld.w [%rb],%r9	ld.w %r9,%sp ext symbol+imm32@ah ext symbol+imm32@al ld.w [%rb],%r9	ld.w %r9,%sp pushn %r0 ext symbol+imm32@h ext symbol+imm32@m ld.w %r0,symbol+imm32@l add %r0,%rb ld.w [%r0],%r9 popn %r0	ld.w %r9,%sp pushn %r1 ext symbol+imm32@h ext symbol+imm32@m ld.w %r1,symbol+imm32@l add %r1,%rb ld.w [%r1],%r9 popn %r1
	[%rb+symbol-imm32], %sp	%rb≠%r0	%rb=%r0	–	–	–
例) xld.w [%rb+symbol-imm32],%sp		ld.w %r9,%sp pushn %r0 ext symbol-imm32@h ext symbol-imm32@m ld.w %r0,symbol-imm32@l add %r0,%rb ld.w [%r0],%r9 popn %r0	ld.w %r9,%sp pushn %r1 ext symbol-imm32@h ext symbol-imm32@m ld.w %r1,symbol-imm32@l add %r1,%rb ld.w [%r1],%r9 popn %r1			
	[%sp+imm32], %sp	imm32≤0xff	0xff<imm32≤0x7ffff	0x7ffff<imm32	–	–
例) xld.w [%sp+imm32],%sp		ld.w %r9,%sp ld.w [%sp+imm32(7:2)],%r9	ld.w %r9,%sp ext imm32(18:6) ld.w [%sp+imm32(5:0)],%r9	ld.w %r9,%sp ext imm32(31:19) ext imm32(18:6) ld.w [%sp+imm32(5:0)],%r9		
	%rd, sign32	-32≤sign32≤31	-262144≤sign32<-32 or 31<sign32≤262143	sign32<-262144 or 262143<sign32	–	–
例) xld.w %rd,sign32		ld.w %rd,sign32(5:0)	ext sign32(18:6) ld.w %rd,sign32(5:0)	ext sign32(31:19) ext sign32(18:6) ld.w %rd,sign32(5:0)		
	%rd, symbol+imm32	symbol+imm32≤0x1f	0x1f<symbol+imm32≤0x3ffff	0x3ffff<symbol+imm32 *3	–	–
例) xld.w %rd,sign32		ld.w %rd,symbol+imm32	ext symbol+imm32@m ld.w %rd,symbol+imm32@l	ext symbol+imm32@h ext symbol+imm32@m ld.w %rd,symbol+imm32@l		
	%rd, symbol-imm32	**固定**	–	–	–	–
例) xld.w %rd,sign32		ext symbol-imm32@h ext symbol-imm32@m ld.w %rd,symbol-imm32@l				

注意

*1) **シンボルが不定で**%rb≠%r0**の場合も、この形式に展開されます。** *2) **シンボルが不定で**%rb=%r0**の場合も、この形式に展開されます。**

*3) **シンボルが不定な場合も、この形式に展開されます。**

拡張命令の展開形式 (5)

拡張命令		展開形式				
オペコード	オペランド	条件1	条件2	条件3	条件4	条件5
xand **xoor** **xxor**	**%rd, %rd, sign32**	-32≤sign32≤31	-262144≤sign32<-32 or 31<sign32≤262143	sign32<-262144 or 262143<sign32	–	–
	例) xand %rd,%rd,sign32	and %rd,sign32(5:0)	ext sign32(18:6) and %rd,sign32(5:0)	ext sign32(31:19) ext sign32(18:6) and %rd,sign32(5:0)		
	%rd, %rs, sign32	0x0≤sign32≤0x1fff (within 13 bits)	0x1fff<sign32≤0x3ffffff (within 26 bits)	0x3ffffff<sign32<0xfffc0000 (26 bits<sign32<-262144)	0xfffc0000≤sign32<0xfffffffe0 (-262144≤sign32<-32)	0xfffffffe0≤sign32≤0xffffffff (-32≤sign32≤-1)
	例) xand %rd,%rs,sign32	ext sign32(12:0) and %rd,%rs	ext sign32(25:13) ext sign32(12:0) and %rd,%rs	ld.w %rd,%rs ext sign32(31:19) ext sign32(18:6) and %rd,sign32(5:0)	ld.w %rd,%rs ext sign32(18:6) and %rd,sign32(5:0)	ld.w %rd,%rs and %rd,sign32(5:0)
xnot	**%rd, sign32**	-32≤sign32≤31	-262144≤sign32<-32 or 31<sign32≤262143	sign32<-262144 or 262143<sign32	–	–
	例) xnot %rd,%rd,sign32	not %rd,sign32(5:0)	ext sign32(18:6) not %rd,sign32(5:0)	ext sign32(31:19) ext sign32(18:6) not %rd,sign32(5:0)		
xadd **xsub**	**%rd, %rd, imm32**	imm32≤0x3f	0x3f<imm32≤0x7ffff	0x7ffff<imm32	–	–
	例) xadd %rd,%rd,imm32	add %rd,imm32(5:0)	ext imm32(18:6) add %rd,imm32(5:0)	ext imm32(31:19) ext imm32(18:6) add %rd,imm32(5:0)		
	%rd, %rs, imm32	imm32≤0x1fff	0x1fff<imm32≤0x3ffffff	0x3ffffff<imm32	–	–
	例) xadd %rd,%rs,imm32	ext imm32(12:0) add %rd,%rs	ext imm32(25:13) ext imm32(12:0) add %rd,%rs	ld.w %rd,%rs ext imm32(31:19) ext imm32(18:6) add %rd,imm32(5:0)		
	%rd, %sp, imm32	imm32≤0x3f	0x3f<imm32≤0x7ffff	0x7ffff<imm32	–	–
	例) xadd %rd,%sp,imm32	ld.w %rd,%sp add %rd,imm32(5:0)	ld.w %rd,%sp ext imm32(18:6) add %rd,imm32(5:0)	ld.w %rd,%sp ext imm32(31:19) ext imm32(18:6) add %rd,imm32(5:0)		
	%sp, %sp, imm32	imm32≤0xfff	0xfff<imm32≤0x7ffff	0x7ffff<imm32	–	–
	例) xadd %rd,%sp,imm32	add %sp,imm32(11:2)	ld.w %r9,%sp ext imm32(18:6) add %r9,imm32(5:0) ld.w %sp,%r9	ld.w %r9,%sp ext imm32(31:19) ext imm32(18:6) add %r9,imm32(5:0) ld.w %sp,%r9		

注意

拡張命令の展開形式 (6)

拡張命令		展開形式				
オペコード	オペランド	条件1	条件2	条件3	条件4	条件5
xadd **xsub**	**%rd, %rd, %sp**	固定	–	–	–	–
	例) xadd %rd,%rd,%sp	ld.w %r9,%sp add %rd,%r9				
	%sp, %sp, %rs	固定	–	–	–	–
	例) xadd %rd,%rs,imm32	ld.w %r9,%sp add %r9,%rs ld.w %sp,%r9				
xcmp	**%rd, sign32**	-32≤sign32≤31	-262144≤sign32<-32 or 31<sign32≤262143	sign32<-262144 or 262143<sign32	–	–
	例) xcmp %rd,sign32	cmp %rd,sign32(5:0)	ext sign32(18:6) cmp %rd,sign32(5:0)	ext sign32(31:19) ext sign32(18:6) cmp %rd,sign32(5:0)		
	%rd, %sp	固定	–	–	–	–
	例) xcmp %rd,%sp	ld.w %r9,%sp cmp %rd,%r9				
	%sp, %rs	固定	–	–	–	–
	例) xcmp %rd,%sp	ld.w %r9,%sp cmp %r9,%rs				
xsrl **xsll** **xsra** **xsla** **xrr** **xrl**	**%rd, %rs**	固定	–	–	–	–

	例) xsrl %rd,%rs	ld.w %r9,%rs
		and %r9,0x1f
		cmp %r9,0x8
		jrle 4
		srl %rd,0x8
		jp.d -3
		sub %r9,0x8
		srl %rd,%r9				

	%rd, imm5	imm5≤8	8<imm5<16	imm5=16	16<imm5≤24	24<imm5
	例) xsrl %rd,imm5	srl %rd,imm5(3:0)	srl %rd,0x8 srl %rd,imm5(2:0)	srl %rd,0x8 srl %rd,0x8	srl %rd,0x8 srl %rd,0x8 srl %rd,imm5(3:0)	srl %rd,0x8 srl %rd,0x8 srl %rd,0x8 srl %rd,imm5(2:0)

注意

拡張命令の展開形式 (7)

拡張命令		展開形式				
オペコード	オペランド	条件1	条件2	条件3	条件4	条件5
xbtst **xbclr** **xbset** **xbnot**	**[imm32], imm3** <gp未使用> 例) xbtst.w [imm32],imm3	imm32≤0x1f ld.w %r9,imm32(5:0) btst [%r9],imm3	0x1f<imm32≤0x3ffff ext imm32(18:6) ld.w %r9,imm32(5:0) btst [%r9],imm3	0x3ffff<imm32 ext imm32(31:19) ext imm32(18:6) ld.w %r9,imm32(5:0) btst [%r9],imm3	–	–
	[imm32], imm3 <gp使用> 例) xbtst.w [imm32],imm3 (sign32=-gp+imm32)	sign32=0x0 btst [%r8],imm3	0x0<sign32≤0x1fff ext sign32(12:0) btst [%r8],imm3	0x1fff<sign32≤0x3ffffff ext sign32(25:13) ext sign32(12:0) btst [%r8],imm3	0x3ffffff<sign32 ext imm32(31:19) ext imm32(18:6) ld.w %r9,imm32(5:0) btst [%r9],imm3	imm32<gp imm32の値に従ってgp未使用時の展開形式となります。
	[symbol+imm32], imm3 <gp未使用> 例) xbtst [symbol+imm32],imm3	symbol+imm32≤0x1f ld.w %r9,symbol+imm32@l btst [%r9],imm3	0x1f<symbol+imm32≤0x3ffff ext symbol+imm32@m ld.w %r9,symbol+imm32@l btst [%r9],imm3	0x3ffff<symbol+imm32 *1 ext symbol+imm32@h ext symbol+imm32@m ld.w %r9,symbol+imm32@l btst [%r9],imm3	–	–
	[symbol+imm32], imm3 <gp使用> 例) xbtst [symbol+imm32],imm3 (sign32=-gp+imm32)	symbol+sign32=0x0 btst [%r8],imm3	0x0<symbol+sign32≤0x1fff ext symbol+sign32@al btst [%r8],imm3	0x1fff<symbol+sign32≤0x3ffffff *1 ext symbol+sign32@ah ext symbol+sign32@al btst [%r8],imm3	0x3ffffff<symbol+sign32 ext symbol+imm32@h ext symbol+imm32@m ld.w %r9,symbol+imm32@l btst [%r9],imm3	symbol+imm32<gp symbol+imm32の値に従ってgp未使用時の展開形式となります。
	[symbol-imm32], imm3 例) xbtst [symbol-imm32],imm3	**固定** ext symbol-imm32@h ext symbol-imm32@m ld.w %r9,symbol-imm32@l btst [%r9],imm3	–	–	–	–
	[%rb+imm32], imm3 例) xbtst [%rb+imm32],imm3	imm32=0x0 btst [%rb],imm3	0x0<imm32≤0x1fff ext imm32(12:0) btst [%rb],imm3	0x1fff<imm32≤0x3ffffff ext imm32(25:13) ext imm32(12:0) btst [%rb],imm3	0x3ffffff<imm32 ext imm32(31:19) ext imm32(18:6) ld.w %r9,imm32(5:0) add %r9,%rb btst [%r9],imm3	–
	[%rb+symbol+imm32], imm3 例) xbtst [%rb+symbol-imm32],imm3	symbol+imm32=0x0 btst [%rb],imm3	0x0<symbol+imm32≤0x1fff ext symbol+imm32@al btst [%rb],imm3	0x1fff<symbol+imm32≤0x3ffffff ext symbol+imm32@ah ext symbol+imm32@al btst [%rb],imm3	0x3ffffff<symbol+imm32 ext symbol+imm32@h ext symbol+imm32@m ld.w %r9,symbol+imm32@l add %r9,%rb btst [%r9],imm3	–

注意

*1) **シンボルが不定な場合も、この形式に展開されます。**

拡張命令の展開形式 (8)

拡張命令		展開形式				
オペコード	オペランド	条件1	条件2	条件3	条件4	条件5
xbtst xbclr xbset xbnot	[%rb+symbol-imm32], imm3 例) xbtst [%rb+symbol-imm32],imm3	固定 ext symbol-imm32@h ext symbol-imm32@m ld.w %r9,symbol-imm32@l add %r9,%rb btst [%r9],imm3	–	–	–	–
	[%sp+imm32], imm3 例) xbtst [%sp+imm32],imm3	imm32=0x0 ld.w %r9,%sp btst [%r9],imm3	imm32≤0x3f ld.w %r9,%sp add %r9,imm32(5:0) btst [%r9],imm3	0x3f<imm32≤0x7ffff ld.w %r9,%sp ext imm32(18:6) add %r9,imm32(5:0) btst [%r9],imm3	0x7ffff<imm32 ld.w %r9,%sp ext imm32(31:19) ext imm32(18:6) add %r9,imm32(5:0) btst [%r9],imm3	–
xcall xcall.d xjp xjp.d xjr*1 xjr*1.d	label+imm32 例) xcall label+imm32	1命令に展開 *2 call label+imm32	2命令に展開 *2 ext label+imm32@rm call label+imm32@rl	3命令に展開 *2 ext label+imm32@rh ext label+imm32@rm call label+imm32@rl	–	–
	sign32 例) xcall,sign32	-256≤sign32≤254 call sign32(8:1)	-2097152≤sign32<-256 or 254<sign32≤2097150 ext sign32(21:9) call sign32(8:1)	sign32<-2097152 or 2097150<sign32 ext sign32(31:22)<<0x3 ext sign32(21:9) call sign32(8:1)	–	–

注意

*1) xjreq, xjreq.d, xjrne, xjrne.d, xjrgt, xjrgt.d, xjrge, xjrge.d, xjrlt, xjrlt.d, xjrle, xjrle.d, xjrugt, xjrugt.d, xjruge, xjruge.d, xjrult, xjrult.d, xjrule, xjrule.d

*2) **展開命令数**

相対距離 (絶対値) #A	命令とラベルの位置	-nearフラグ	展開命令数
0〜126 #B	同一ファイル内	–	1
	異なるファイル	–	2
〜敷居値 #C	–	–	2
〜0x7fffffff #D	–	あり	2
	–	なし	3
相対距離不明	–	あり	2
	–	なし	3

#A: 数値は拡張分岐命令から分岐先ラベルまでの命令数を示します。
#B: プラスアドレス方向の場合は125
#C: 同一ファイル内のプラスアドレス方向の場合は敷居値-1まで
#D: 同一ファイル内のプラスアドレス方向の場合は0x7ffffffeまで
敷居値は-jオプションで指定した値の1/2です。-jオプションのデフォルト値0x180000の場合、敷居値は0xc0000となります。
低アドレスに分岐する場合、3回ほどの繰り返しにより相対距離を決めているため、ソースによってはこの値よりも敷居値が低くなる場合があります。

pp33擬似命令

#include	<file name>	ファイルの挿入
#define	<define name> [<string>]	指定のデファイン名で文字列/数値を定義
#macro	<macro name> [$1] [,$2] ... [,$32]	指定のマクロ名でステートメント列を定義
	<statements>	マクロ内の分岐ラベルは$$1〜$$64のシンボルで指定
#endm		
#ifdef	<name>	条件アセンブル
	<statements 1>	<name>が定義済み: <statements 1>をアセンブル
[#else		<name>が未定義: <statements 2> をアセンブル
	<statements 2>]	
#endif		
#ifndef	<name>	条件アセンブル
	<statements 1>	<name>が未定義: <statements 1>をアセンブル
[#else		<name>が定義済み: <statements 2>をアセンブル
	<statements 2>]	
#endif		

as33擬似命令

.abs		アブソリュートアセンブル指定(ファイルの先頭に記述)
.code		CODEセクションを宣言
.data		DATAセクションを宣言
.comm	<global symbol> <size>	BSSセクション内にグローバル領域を確保
.lcomm	<local symbol> <size>	BSSセクション内にローカル領域を確保
.org	<address>	絶対アドレスを設定(アブソリュート専用)
.set	<symbol> <address>	シンボルに絶対アドレスを定義(アブソリュート専用)
.word	<data>[<data> ... <data>]	CODE/DATAセクション内にワードデータを定義
.word	<symbol>	CODE/DATAセクション内にワードデータを定義
.half	<data>[<data> ... <data>]	CODE/DATAセクション内にハーフワードデータを定義
.byte	<data>[<data> ... <data>]	CODE/DATAセクション内にバイトデータを定義
.ascii	"<string>"	CODE/DATAセクション内にASCII文字列を定義
.space	<size>	CODE/DATAセクション内に空白領域(0x0)を定義
.align	<N>	指定境界アドレスにアライメント
.global	<symbol>	シンボルのグローバル宣言
.list		アセンブリリスト出力をON(.list)/OFF(.nolist)
.nolist		(-lオプション指定時のみ有効)
.file	"<file name>"	ソースデバッグ情報の出力
.endfile		
.loc	<line number>	
.def	<symbol>, <parameters>, endef	シンボルデバッグ情報の出力

演算子 (pp33の機能)

		(優先順位)	例
( )	カッコ	(1)	1+(1+2*5)
+	単項プラス	(2)	+0xff
-	単項マイナス	(2)	-123
~	論理否定	(2)	~0x1234
^H	ビットフィールド(31:19)の取得	(2)	0x1234^H
^M	ビットフィールド(18:6)の取得	(2)	0x1234^M
^L	ビットフィールド(5:0)の取得	(2)	0x1234^L
^AH	ビットフィールド(25:13)の取得	(2)	0x1234^AH
^AL	ビットフィールド(12:0)の取得	(2)	0x1234^AL
*	乗算	(3)	0xf*5
/	除算	(3)	0x123/0x56
%%	剰余	(3)	0x123%%0x56
+	加算	(4)	1+2
-	減算	(4)	0xff-0b111
<<	左シフト	(5)	0x113<<3
>>	右シフト	(5)	1>>2
&	論理積	(6)	0b1101&0b111
^	排他的論理和	(7)	12^35
|	論理和	(8)	0x123|0xff

式の項には数値と数値デファイン名が使用可能です。
式は符号付き32ビットとして演算が行われます。
演算子と数値の間にはスペースやタブを入れないでください。

ディスプレースメント付きシンボル (as33の機能)

as33は次のようなディスプレースメント付きのシンボル表示にも対応します。
LABEL+imm32　LABEL+sign32

シンボルマスク (as33の機能)

@rh (or @RH)	ext	LABEL@rh	LABEL@rh=<LABEL-PC>(31:22)
@rm (or @RM)	ext	LABEL@m	LABEL@m=<LABEL-PC>(21:9)
@rl (or @RL)	<op>	LABEL@rl	LABEL@rl=<LABEL-PC>(8:0)
@h (or @H)	ext	LABEL@h	LABEL@h=LABEL(31:19)
@m (or @M)	ext	LABEL@h	LABEL@m=LABEL(18:6)
@l (or @L)	ld.w	%rd, LABEL@l	LABEL@l=LABEL(5:0)
@ah (or @AH)	ext	LABEL@ah	LABEL@ah=LABEL(25:13)
@al (or @AL)	ext	LABEL@al	LABEL@al=LABEL(12:0)

<op>　call/call.d, jp/ip.d, jrgt/jrgt.d, jrge/jrge.d, jrlt/ jrlt.d, jrle/jrle.d, jrugt/jrugt.d, jruge/jruge.d, jrult/ jrult.d, jrule/jrule.d, jreq/jreq.d, jrne/jrne.d

ENERGY SAVING
EPSON

夢の「省」技術。

「省」の技術を追求するエプソン電子デバイス。
半導体、液晶表示体、水晶デバイスのデバイス群が、
夢の商品創造をサポートします。
エプソンはエナジーセービングです。

セイコーエプソン株式会社
電子デバイス営業本部

ED営業推進部　〒191-8501　東京都日野市日野421-8
IC営業技術G　TEL (042) 587-5816 (直通)　FAX (042) 587-5624

〈東日本〉
ED東京営業部　〒191-8501　東京都日野市日野421-8
東京IC営業G　TEL (042) 587-5313 (直通)　FAX (042) 587-5116

〈西日本〉
ED大阪営業部　〒541-0059　大阪市中央区博労町3-5-1　エプソン大阪ビル15F
TEL (06) 6120-6000 (代表)　FAX (06) 6120-6100

〈東海・北陸〉
ED名古屋営業部　〒461-0005　名古屋市東区東桜1-10-24　栄大野ビル4F
TEL (052) 953-8031 (代表)　FAX (052) 953-8041

〈長野〉
ED長野営業部　〒392-8502　長野県諏訪市大和3-3-5
TEL (0266) 58-8171 (直通)　FAX (0266) 58-9917

〈東北〉
ED仙台営業所　〒980-0013　宮城県仙台市青葉区花京院1-1-20　花京院スクエア19F
TEL (022) 263-7975 (代表)　FAX (022) 263-7990

インターネットによる電子デバイスのご紹介　http://www.epson.co.jp/device/

2001年9月改訂 Ⓛ
1998年6月作成 ⓂB